

oneMKL - Data Parallel C++ Developer Reference

Contents

Chapter 1: Intel® oneAPI Math Kernel Library - Data Parallel C++ Developer Reference

What's New	10
oneMKL Initialization on GPU.....	11
Introduction to the Intel® oneAPI Math Kernel Library (oneMKL) BLAS and LAPACK with DPC++	11
Overview of Intel® oneMKL BLAS Routines for Data Parallel C++	14
Overview of Intel® oneAPI Math Kernel Library (oneMKL) Sparse BLAS for DPC++	14
Overview of Intel® oneMKL LAPACK Routines for Data Parallel C++	16
Data Types	16
Matrix Storage	19
Scalar Arguments	26
Error Handling	28
Known Limitations	29
BLAS Routines	29
BLAS Level 1 Routines	29
asum	30
axpy	32
copy	35
dot	37
dotc	39
dotu	42
iamax.....	44
iamin	46
nrm2.....	49
rot	51
rotg	53
rotr	55
rotmg.....	60
scal.....	64
sdsdot	66
swap	68
BLAS Level 2 Routines	71
gbmv	72
gemv	76
ger	79
gerc	82
geru	85
hbm	88
hemv	91
her	94
her2.....	96
hpmv	99
hpr	102
hpr2.....	104
sbmv.....	107
spmv.....	110

spr.....	113
spr2.....	115
symv.....	118
syr.....	121
syr2.....	123
tbm.....	126
tbsv.....	129
tpmv.....	132
tpsv.....	135
trmv.....	137
trsv.....	140
BLAS Level 3 Routines	143
gemm	144
hemm	150
her2k.....	154
herk.....	159
symm.....	163
syr2k.....	167
syrk.....	172
trmm.....	176
trsm.....	184
BLAS-like Extensions	191
axpby.....	193
axpy_batch	195
copy_batch	200
dgmm_batch.....	205
gemm_batch.....	211
gemm_bias.....	223
gemmt.....	230
gemv_batch.....	236
syrk_batch.....	242
trsm_batch	250
omatcopy	257
imatcopy	261
omatadd.....	264
omatcopy_batch	270
imatcopy_batch.....	276
omatadd_batch	282
Compute Modes	289
Numerical Reproducibility	293
Sparse BLAS Routines	293
Sparse BLAS Matrix Handle Contract between User and Library	297
Sparse BLAS Supported Data and Integer Types.....	298
Sparse Storage Formats.....	299
oneapi::mkl::sparse::init_matrix_handle	306
oneapi::mkl::sparse::release_matrix_handle.....	307
oneapi::mkl::sparse::set_csr_data.....	309
oneapi::mkl::sparse::set_coo_data.....	311
oneapi::mkl::sparse::set_matrix_property	314
oneapi::mkl::sparse::optimize_gemv	316
oneapi::mkl::sparse::optimize_trmv	317
oneapi::mkl::sparse::optimize_trsv.....	319
oneapi::mkl::sparse::optimize_gemm	321
oneapi::mkl::sparse::optimize_trsm.....	324
oneapi::mkl::sparse::gemv.....	326

oneapi::mkl::sparse::gemvdot	329
oneapi::mkl::sparse::symv	331
oneapi::mkl::sparse::trmv	333
oneapi::mkl::sparse::trsv	336
oneapi::mkl::sparse::gemm	340
oneapi::mkl::sparse::trsm	343
oneapi::mkl::sparse::omatadd	347
oneapi::mkl::sparse::init_matmat_descr	353
oneapi::mkl::sparse::set_matmat_data	354
oneapi::mkl::sparse::get_matmat_data	355
oneapi::mkl::sparse::release_matmat_descr	357
oneapi::mkl::sparse::matmat	358
oneapi::mkl::sparse::matmatd	364
oneapi::mkl::sparse::omatcopy	367
oneapi::mkl::sparse::omatconvert	369
oneapi::mkl::sparse::sort_matrix	374
oneapi::mkl::sparse::update_diagonal_values	375
LAPACK Routines	377
gebrd	377
gebrd (USM Version)	379
gebrd_scratchpad_size	382
geinv_batch (Group Version)	383
geinv_batch_scratchpad_size (Group Version)	384
gels (USM Version)	386
gels_scratchpad_size	388
gels_batch (Buffer Strided Version)	389
gels_batch (USM Strided Version)	391
gels_batch_scratchpad_size (Strided Version)	393
gels_batch (Group Version)	394
gels_batch_scratchpad_size (Group Version)	397
geqrf	398
geqrf (USM Version)	400
geqrf_batch (Buffer Strided Version)	402
geqrf_batch (Group Version)	403
geqrf_batch (USM Strided Version)	406
geqrf_batch_scratchpad_size (Group Version)	407
geqrf_batch_scratchpad_size (Strided Version)	409
geqrf_scratchpad_size	410
gerqf	411
gerqf (USM Version)	412
gerqf_scratchpad_size	414
gesv (USM Version)	415
gesv_scratchpad_size	417
gesvd	418
gesvd (USM Version)	421
gesvd_scratchpad_size	424
gesvda_batch (Buffer Strided Version)	425
gesvda_batch (USM Strided Version)	429
gesvda_batch_scratchpad_size (Strided Version)	433
getrf	435
getrf (USM Version)	436
getrf_batch (Buffer Strided Version)	438
getrf_batch (Group Version)	440
getrf_batch (USM Strided Version)	442
getrf_batch_scratchpad_size (Group Version)	444

getrf_batch_scratchpad_size (Strided Version)	446
getrf_scratchpad_size	447
getrfnp	447
getrfnp (USM Version)	449
getrfnp_scratchpad_size	451
getrfnp_batch (Buffer Strided Version)	452
getrfnp_batch (Group Version)	454
getrfnp_batch (USM Strided Version)	456
getrfnp_batch_scratchpad_size (Group Version)	458
getrfnp_batch_scratchpad_size (Strided Version)	459
getri	460
getri (USM Version)	461
getri_batch (Buffer Strided Version)	463
getri_batch (Group Version)	465
getri_batch (USM Strided Version)	466
getri_batch_scratchpad_size (Group Version)	468
getri_batch_scratchpad_size (Strided Version)	469
getri_batch (Out-of-place, Buffer Strided Version)	470
getri_batch (Out-of-place, USM Strided Version)	472
getri_batch_scratchpad_size (Strided Version)	474
getri_scratchpad_size	475
getrs	476
getrs (USM Version)	478
getrs_batch (Buffer Strided Version)	480
getrs_batch (Group Version)	482
getrs_batch (USM Strided Version)	484
getrs_batch_scratchpad_size (Group Version)	487
getrs_batch_scratchpad_size (Strided Version)	488
getrs_scratchpad_size	489
getrsnp_batch (Buffer Strided Version)	490
getrsnp_batch (USM Strided Version)	492
getrsnp_batch_scratchpad_size (Strided Version)	495
heevd	496
heevd (USM Version)	498
heevd_scratchpad_size	500
heevx	501
heevx (USM Version)	504
heevx_scratchpad_size	507
hegvd	509
hegvd (USM Version)	511
hegvd_scratchpad_size	514
hegvx	515
hegvx (USM Version)	518
hegvx_scratchpad_size	521
hetrd	523
hetrd (USM Version)	525
hetrd_scratchpad_size	527
hetrf	528
hetrf (USM Version)	530
hetrf_scratchpad_size	532
orgbr	533
orgbr (USM Version)	535
orgbr_scratchpad_size	537
orgqr	539
orgqr (USM Version)	540

orgqr_batch (Buffer Strided Version)	542
orgqr_batch (Group Version)	544
orgqr_batch (USM Strided Version).....	546
orgqr_batch_scratchpad_size (Group Version).....	548
orgqr_batch_scratchpad_size (Strided Version)	550
orgqr_scratchpad_size	551
orgtr	552
orgtr (USM Version).....	553
orgtr_scratchpad_size.....	554
ormqr.....	555
ormqr (USM Version)	557
ormqr_scratchpad_size	559
ormrq.....	560
ormrq (USM Version)	562
ormrq_scratchpad_size	564
ormtr	565
ormtr (USM Version).....	567
ormtr_scratchpad_size.....	569
potrf.....	570
potrf (USM Version)	572
potrf_batch (Buffer Strided Version)	574
potrf_batch (Group Version)	576
potrf_batch (USM Strided Version)	578
potrf_batch_scratchpad_size (Group Version)	580
potrf_batch_scratchpad_size (Strided Version)	582
potrf_scratchpad_size.....	583
potri	584
potri (USM Version)	585
potri_scratchpad_size	587
potrs	588
potrs (USM Version)	590
potrs_batch (Buffer Strided Version)	592
potrs_batch (Group Version).....	594
potrs_batch (USM Strided Version)	596
potrs_batch_scratchpad_size (Group Version)	599
potrs_batch_scratchpad_size (Strided Version)	600
potrs_scratchpad_size	602
syevd	603
syevd (USM Version)	605
syevd_scratchpad_size	607
syevx	608
syevx (USM Version)	611
syevx_scratchpad_size.....	614
sygvd	616
sygvd (USM Version)	618
sygvd_scratchpad_size	621
sygvx	622
sygvx (USM Version)	625
sygvx_scratchpad_size	628
sytrd	630
sytrd (USM Version)	632
sytrd_scratchpad_size	634
sytrf	635
sytrf (USM Version)	637
sytrf_scratchpad_size	639

trtri (USM Version)	640
trtri_scratchpad_size	642
trtrs	643
trtrs (USM Version)	645
trtrs_scratchpad_size	647
ungbr	648
ungbr (USM Version)	651
ungbr_scratchpad_size	653
ungqr	654
ungqr (USM Version)	656
ungqr_batch (Buffer Strided Version)	658
ungqr_batch (Group Version)	660
ungqr_batch (USM Strided Version)	662
ungqr_batch_scratchpad_size (Group Version)	664
ungqr_batch_scratchpad_size (Strided Version)	665
ungqr_scratchpad_size	666
ungtr	667
ungtr (USM Version)	669
ungtr_scratchpad_size	670
unmqr	671
unmqr (USM Version)	673
unmqr_scratchpad_size	675
unmrq	676
unmrq (USM Version)	678
unmrq_scratchpad_size	680
unmtr	681
unmtr (USM Version)	683
unmtr_scratchpad_size	685
Vector Mathematical Functions	686
Special Value Notations	687
VM Mathematical Functions	687
Arithmetic Functions	693
Power and Root Functions	725
Exponential and Logarithmic Functions	762
Trigonometric Functions	788
Hyperbolic Functions	844
Special Functions	864
Rounding Functions	919
VM Service Functions	942
set_mode	942
get_mode	943
set_status	945
get_status	946
clear_status	947
create_error_handler	948
Miscellaneous VM Functions	952
copysign	952
nextafter	955
fdim	958
fmax	961
fmin	963
maxmag	966
minmag	969
Random Number Generators	971
Random Number Generators Host Routines	972

Random Number Generators Routines.....	973
oneMKL RNG Usage Model.....	973
Device Support.....	975
Generate Routine.....	976
Engines (Basic Random Number Generators).....	977
Service Routines.....	990
Distributions.....	998
Random Number Generators Device Routines.....	1041
oneMKL RNG Device Usage Model.....	1042
Device Generate Routines.....	1047
Device Engines (Basic Random Number Generators).....	1049
Host-Side Helpers.....	1055
Device Service Routines.....	1057
Device Distributions.....	1059
Engine Adaptors.....	1079
Summary Statistics.....	1080
oneMKL Summary Statistics Usage Model.....	1080
Summary Statistics Device Support.....	1082
Dataset.....	1083
Service Functions to Create Dataset.....	1084
oneapi::mkl::stats::make_dataset.....	1085
Summary Statistics Routines.....	1086
oneapi::mkl::stats::raw_sum.....	1086
oneapi::mkl::stats::central_sum.....	1088
oneapi::mkl::stats::central_sum with User-provided Mean.....	1090
oneapi::mkl::stats::raw_moment.....	1092
oneapi::mkl::stats::central_moment.....	1094
oneapi::mkl::stats::central_moment with User-provided Mean ..	1096
oneapi::mkl::stats::mean.....	1098
oneapi::mkl::stats::variation.....	1100
oneapi::mkl::stats::variation with User-provided Mean.....	1101
oneapi::mkl::stats::skewness.....	1103
oneapi::mkl::stats::skewness with User-provided Mean.....	1105
oneapi::mkl::stats::kurtosis.....	1107
oneapi::mkl::stats::kurtosis with User-provided Mean.....	1108
oneapi::mkl::stats::min.....	1110
oneapi::mkl::stats::max.....	1112
oneapi::mkl::stats::min_max.....	1113
Fourier Transform Functions.....	1115
Scoped enumerations.....	1117
The descriptor class template.....	1124
Constructors, assignment operators, and destructor.....	1125
Configuration-setting member functions.....	1126
Configuration-querying member functions.....	1132
Configuration- and queue-committing member function.....	1138
Providing an externally-allocated workspace.....	1138
Configuring the placement of the DFT results.....	1141
Allowing destruction of input data.....	1141
Limiting the number of usable threads.....	1142
Configuring data layouts.....	1142
Compute functions.....	1151
Configuring and computing a DFT in DPC++.....	1155
Distributed DFT.....	1161
Data Distribution Among Processes.....	1163
Scoped enumerations.....	1164

The distributed_descriptor class template	1165
Compute Functions	1173
Data Fitting	1179
Common Terms.....	1179
Splines	1184
Linear Spline	1187
Cubic Splines	1188
Interpolate Function	1190
Examples	1192
Bibliography	1193
Appendix A: oneMKL Functionality	1194
BLAS Functionality	1194
LAPACK Functionality	1194
DFT Functionality	1196
Sparse BLAS SYCL Functionality	1197
Sparse Solvers Functionality	1200
Random Number Generators Functionality	1200
Vector Math Functionality	1202
Data Fitting Functionality	1202
Summary Statistics Functionality	1203
Notices and Disclaimers.....	1204

Intel® oneAPI Math Kernel Library - Data Parallel C++ Developer Reference

1

The Intel® oneAPI Math Kernel Library (oneMKL) improves performance with math routines for software applications that solve large computational problems. oneMKL provides BLAS and LAPACK linear algebra routines, fast Fourier transforms, vectorized math functions, random number generation functions, and other functionality.

What's New

This publication describes the Data Parallel C++ (DPC++) interface.

DPC++ is ISO C++ plus Khronos SYCL with Intel extensions. For more information, see [Intel® oneAPI DPC++ Compiler](#).

Basic Linear Algebra Subprograms (BLAS)	The BLAS routines provide vector, matrix-vector, and matrix-matrix operations.
Sparse BLAS	The Sparse BLAS routines provide basic operations on sparse vectors and matrices.
LAPACK	The LAPACK routines solve systems of linear equations, least square problems, eigenvalue and singular value problems, and Sylvester's equations.
Random Number Generators	The Random Number Generators provides a set of routines implementing commonly used pseudorandom and quasi-random generators with continuous and discrete distributions.
Summary Statistics	Summary Statistics provides routines that compute basic statistical estimates for single and double precision multi-dimensional datasets.
Vector Mathematics Functions	The Vector Mathematical Functions compute core mathematical functions on vector arguments.
Fourier Transform Functions	The Fourier Transform Functions offer several options for computing Fast Fourier Transforms (FFTs).
Data Fitting	The Data Fitting provides spline-based interpolation capabilities that can be used for spline construction (Linear, Cubic, Quadratic etc.), to perform cell-search operations, and to approximate functions, function derivatives, or integrals.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

What's New

This developer reference documents the Intel® oneAPI Math Kernel Library (oneMKL) release for the Data Parallel C++ (DPC++) interface. The manual has been updated to reflect enhancements to the product as well as improvements and error corrections.

Product and Performance Information

Performance varies by use, configuration, and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

oneMKL Initialization on GPU

When we run an application with oneMKL functions on GPU, we spend time on some service routines as well. Here's what happens inside the library when we call oneMKL functions on GPU:

- The first step is to check which oneMKL verbose mode was chosen. oneMKL verbose mode is needed to profile oneMKL usage in the application. You can read more about oneMKL Verbose mode in the documentation here:

[Using oneMKL Verbose Mode \(Linux\)](#)[Using oneMKL Verbose Mode \(Windows\)](#)

The oneMKL Verbose feature is supported by BLAS (and BLAS-like extensions), LAPACK, FFT, and (in the DPC++ API only) RNG.

- The next item in the list is the oneMKL GPU information detector, which checks which GPU is present on the system in order to run the optimal code implementation. It's looking for architecture, stepping, tiles, backend, and other important parameters.
- The next step is to create a kernel and all that it requires (creating a program cache, adding a new kernel to the cache, searching for existing kernels, and so forth).
- Finally, some time is spent in the oneMKL memory manager, where the required memory is allocated or freed using the internal oneMKL implementation. oneMKL has a memory manager that provides a list of [support functions](#), the ability to [redefine memory functions](#), and internal fast memory allocations with memory reuse.

Let's look at details of running a oneMKL [gemm](#) example from `${ONEAPI_ROOT}/share/doc/mkl/examples/examples_sycl.tgz/sycl/blas/source/gemm.cpp` (Linux) or `%ONEAPI_ROOT%\share\doc\mkl\examples\examples_sycl.zip\sycl\blas\source\gemm.cpp` (Windows). Just execution of BLAS `gemm` with the single-precision real data type on an Intel® Data Center GPU Max Series card took 48.795 milliseconds:

- Checking for oneMKL Verbose mode settings took 0.012 milliseconds.
- The sum of all times for getting the information about the GPU device took around 1.568 milliseconds.
- Creating the `gemm` kernel took around 0.958 milliseconds.
- Kernel cache allocation took around 0.01 milliseconds.
- At the end of the application when the oneMKL library is unloading, we also clean caches. This took 0.111 milliseconds.

In addition to the time above, we need to make sure that before running the `gemm` function, we have all required memory allocated for the A, B, and C matrices. This took an additional 0.084 milliseconds. And after running `gemm` we need to clean all allocated memory, which took around 0.14.

If we run `gemm` on CPU, the classic C/Fortran implementation will be used. Refer to the C/Fortran version of the developer reference for a corresponding discussion of those initialization costs.

Introduction to the Intel® oneAPI Math Kernel Library (oneMKL) BLAS and LAPACK with DPC++

This guide provides an overview of the Intel® oneAPI Math Kernel Library (oneMKL) BLAS (Basic Linear Algebra Subprograms) and LAPACK (Linear Algebra Package) application programming interfaces for the Data Parallel C++ (DPC++) implementation of SYCL. It is aimed at users who have had some prior experience with the standard BLAS and LAPACK APIs.

In general, the DPC++ APIs for BLAS and LAPACK are similar to the standard BLAS and LAPACK APIs, sharing the same routine names and argument orders. Unlike standard routines, however, DPC++ routines are designed to run asynchronously on a compute device (CPU or GPU) and typically use device memory for inputs and outputs. To support this functionality, the data types of many arguments have changed and each routine takes an additional argument (a DPC++ queue), which specifies where the routine should be executed. There are several smaller API changes that are described below.

In oneMKL, all DPC++ routines and associated data types belong to the `oneapi::mkl` namespace. CPU-based oneMKL routines are still available via the C interface (which uses the global namespace). Additionally, each BLAS-specific routine is in the `oneapi::mkl::blas`, `oneapi::mkl::blas::column_major`, and `oneapi::mkl::blas::row_major` namespaces. Each LAPACK-specific routine is in the `oneapi::mkl::lapack` namespace. Currently, LAPACK DPC++ APIs do not support matrices stored using row major layout.

By default, column major layout is assumed for all BLAS functions in the `oneapi::mkl::blas` namespace. BLAS functions in the `oneapi::mkl::blas::column_major` namespace can also be used when matrices are stored using column major layout. To use row major layout, BLAS functions in the `oneapi::mkl::blas::row_major` namespace must be used. For example, `oneapi::mkl::blas::gemm` is the DPC++ routine for matrix multiplication using column major layout for storing matrices, while `::cblas_{s, d, c, z}gemm` is the traditional CPU-based version.

Differences between Standard BLAS/LAPACK and DPC++ oneMKL APIs

Naming

DPC++ BLAS and LAPACK APIs are overloaded on precision. For example, unlike standard BLAS API having four different routines for GEMM computation with names based on precision (`sgemm`, `dgemm`, `cgemm` and `zgemm`), the DPC++ BLAS has only one entry point for GEMM computation named `gemm` accepting `float`, `double`, `half`, `bfloat16`, `std::complex<float>`, and `std::complex<double>` data types.

References

All DPC++ objects (buffers and queues) are passed by reference, rather than by pointer. Other parameters are typically passed by value.

Queues

Every DPC++ BLAS and LAPACK routine has an extra parameter at the beginning: A DPC++ queue (type `queue&`), where computational tasks are submitted. A queue can be associated with a CPU device or a GPU device. The CPU and GPU devices are supported for all BLAS functions. Refer to the documentation for individual LAPACK functions to see which devices are supported.

Vector and Matrix Types

DPC++ has two APIs for storing data on a device and sharing data between devices and the host: the buffer API and the unified shared memory (USM) API. DPC++ BLAS and LAPACK routines support both APIs.

With the buffer API, vector and matrix inputs to DPC++ BLAS and LAPACK routines are DPC++ buffer types. Currently, all buffers must be one-dimensional, but you can use DPC++'s `buffer::reinterpret()` member function to convert a higher-dimensional buffer to a one-dimensional one.

For the USM API, vector and matrix inputs to DPC++ BLAS and LAPACK routines are pointers of the appropriate type, but the pointers must point to memory allocated by one of the DPC++ USM allocation routines (eg `malloc_host`, `malloc_shared`, or `malloc_device`). Memory that is allocated with the usual `malloc` or `new` routines cannot be used in the Intel® oneAPI Math Kernel Library (oneMKL) DPC++ interfaces.

For example, the `gemv` routine takes a matrix `A` and vectors `x`, `y`. For the real double precision case, each of these parameters has types:

- `double*` in standard BLAS;
- `buffer<double, 1>&` in DPC++ BLAS with the buffer API;

- `double*` in DPC++ BLAS with the USM API, with the restriction that the memory the pointer refers to must be allocated in a device-accessible way using a DPC++ USM allocation routine.

Scalars

Scalar inputs are passed by value for all BLAS functions.

Complex Numbers

In DPC++, complex numbers are represented with C++ `std::complex` types. For instance, `MKL_Complex8` can be replaced by `std::complex<float>`.

This is true for scalar, vector, and matrix arguments. For instance, a double-precision complex vector would have type `buffer<std::complex<double>, 1>`.

Return Values

Some BLAS and LAPACK routines (`dot`, `nrm2`, `asum`, `iamax`) return a scalar result as their return value. In DPC++, to support asynchronous computation, these routines take an additional argument that occurs at the end of the argument list. The result value is stored in this buffer (for buffer API) or pointer (for USM API) when the computation completes. These routines, like the other DPC++ routines, have a return type of `void` for buffer API or `sycl::event` for USM API.

Computation Options (Character Parameters)

Standard BLAS and LAPACK use special alphabetic characters to control operations: transposition of matrices, storage of symmetric and triangular matrices, etc. In DPC++, these special characters are replaced by scoped enum types for extra type safety.

For example, the BLAS matrix-vector multiplication `dgemv` takes a character argument `trans`, which can be one of `N` or `T`, specifying whether the input matrix `A` should be transposed before multiplication.

In DPC++, `trans` is a member of the scoped enum type `oneapi::mkl::transpose`. You can use the traditional character-based names `oneapi::mkl::transpose::N` and `oneapi::mkl::transpose::T`, or the equivalent, more descriptive names `oneapi::mkl::transpose::nontrans` and `oneapi::mkl::transpose::trans`.

See the [Data Types](#) for more information on the new types.

Matrix Layout (Row Major and Column Major)

The standard BLAS and LAPACK APIs require a Fortran layout for matrices (column major), where matrices are stored column-by-column in memory and the entries in each column are stored in consecutive memory locations. By default, oneMKL for DPC++ likewise assumes this matrix layout. The `oneapi::mkl::blas::row_major` namespace must be used for row major layout for BLAS. Row major layout is not supported directly for LAPACK, but you can transpose row major input matrices, call the desired DPC++ LAPACK routines, and then transpose the output matrices back to row major layout.

Example for BLAS

Below is a short excerpt of a program calling standard BLAS `dgemm`:

```
double *A = ..., *B = ..., *C = ...;
double alpha = 2.0, beta = 3.0;

int m = 16, n = 20, k = 24;
int lda = m, ldb = n, ldc = m;
```

```
dgemm("N", "T", &m, &n, &k, &alpha, A, &lda, B, &ldb, &beta, C, &ldc);
```

The DPC++ equivalent of this excerpt would be as follows:

```
using namespace sycl;
using namespace oneapi::mkl;

queue Q(...);
buffer A = ..., B = ..., C = ...;
int m = 16, n = 20, k = 24;
int lda = m, ldb = n, ldc = m;
blas::gemm(Q, transpose::N, transpose::T, m, n, k, 2.0, A, lda, B, ldb, 3.0, C, ldc);
```

Overview of Intel® oneMKL BLAS Routines for Data Parallel C++

The following pages describe the oneMKL BLAS routines for Data Parallel C++ (DPC++), all of which are declared in the header file `oneapi/mkl/blas.hpp`.

Several conventions are used throughout this document:

- All oneMKL for DPC++ data types and non domain specific functions are inside the `oneapi::mkl::` namespace.
- All oneMKL BLAS functions for DPC++ are inside the `oneapi::mkl::blas` namespace.
- The routines are templated on precision. Each routine has a table detailing the supported precisions.

Device Support

DPC++ supports the following types of devices:

- CPU device: Performs computations on a CPU using OpenCL™.
- GPU device: Performs computations on a GPU using OpenCL™ or Level Zero.

In the current release of oneMKL BLAS for DPC++, all standard Level1, Level2, and Level3 BLAS routines and the BLAS extensions support CPU and GPU devices.

Overview of Intel® oneAPI Math Kernel Library (oneMKL) Sparse BLAS for DPC++

The following pages describe the oneMKL Sparse BLAS computational routines for DPC++ in detail. These routines, along with other helper routines (see [Sparse BLAS Routines](#) for the full list), are declared in the header file `oneapi/mkl/spblas.hpp`.

Several conventions are used throughout this document:

- All oneMKL DPC++ data types and non-domain-specific functions are inside the `oneapi::mkl::` namespace.
- All oneMKL DPC++ Sparse BLAS functions are inside the `oneapi::mkl::sparse` namespace.
- For brevity, the `sycl` namespace is omitted from DPC++ object types such as buffers and queues. For example, a single-precision, 1D buffer A would be written `buffer<float,1> &A` instead of `sycl::buffer<float,1> &A`.

- Computational routines are overloaded on precision. Unless otherwise specified, all oneMKL Sparse BLAS computational routines support `float`, `double`, `std::complex<float>`, and `std::complex<double>` floating point types, and do not support mixed-precision computations yet.
- oneMKL sparse BLAS domain currently does not offer bitwise-reproducibility (BWR) guarantees for most of its APIs.
- For sparse matrix row and column indices, oneMKL Sparse BLAS supports `std::int32_t` and `std::int64_t` integer types for all supported matrix formats. Matrix handle creation routines are overloaded on integer types.
- Some APIs require user-provided temporary workspaces. In case of `sycl::buffer` APIs, the temporary workspaces are of type `sycl::buffer<std::uint8_t, 1> *`, whereas in the case of USM APIs, they are of type `void *`.
- For users of USM APIs, usage of oneMKL with all types of allocations (`device`, `shared`, and `host`) are supported; however, performance between them may differ. For maximum performance of Sparse BLAS APIs, we recommend using oneMKL with device memory allocations (`sycl::malloc_device()`) as much as possible except where specified otherwise, but explicit data movement associated with that is users' responsibility.

Device Support

DPC++ supports several types of devices:

- CPU device: Performs computations on a CPU using OpenCL™.
- GPU device: Performs computations on a GPU using OpenCL™ or Level Zero.

Each routine details the device types that are currently supported.

In the current release of oneMKL DPC++ Sparse BLAS, all listed routines support use on CPU and GPU devices with the Compressed Sparse Row (CSR) matrix format unless otherwise noted. Limited support with the Coordinate (COO) matrix format is also available, specified in the documentation of each API.

Routine	Description
Level 2:	
sparse::gemv	General sparse matrix-dense vector product
sparse::gemvdot	General sparse matrix-dense vector product with fused dot product
sparse::symv	Symmetric sparse matrix-dense vector product
sparse::trmv	Triangular sparse matrix-dense vector product
sparse::trsv	Triangular solve of sparse matrix against a dense vector.
Level 3:	
sparse::gemm	General sparse matrix-dense matrix product with dense matrix output
sparse::trsm	Triangular solve of sparse matrix against a dense matrix.
sparse::omatadd	General sparse matrix-sparse matrix addition with sparse matrix output.
sparse::matmat	General sparse matrix-sparse matrix product with sparse matrix output.

Routine	Description
<code>sparse::matmatd</code>	General sparse matrix-sparse matrix product with dense matrix output.

Overview of Intel® oneMKL LAPACK Routines for Data Parallel C++

The following pages describe the oneMKL LAPACK routines for Data Parallel C++ (DPC++) in detail, all of which are declared in the header file `oneapi/mkl/lapack.hpp`.

Several conventions are used throughout this document:

- All oneMKL for DPC++ data types and non domain specific functions are inside the `oneapi::mkl` namespace.
- All oneMKL LAPACK functions for DPC++ are inside the `oneapi::mkl::lapack` namespace.
- For brevity, the `sycl` namespace is omitted from DPC++ object types, such as buffers and queues. For example a single-precision, 1D buffer A would be written `buffer<float,1> &A` instead of `sycl::buffer<float,1> &A`.
- The routines are overloaded on precision. Each routine has a table detailing the supported precisions and devices.

Device Support

DPC++ supports the following types of devices:

- CPU device: Performs computations on a CPU using OpenCL™.
- GPU device: Performs computations on a GPU using OpenCL™ or Level Zero. Note that some or all of the computations may be performed on the CPU.

Each routine details the device types that are currently supported.

Data Types

BLAS and LAPACK Data Types

Intel® oneAPI Math Kernel Library (oneMKL) BLAS and LAPACK for Data Parallel C++ (DPC++) introduces several new enumeration data types, which are type-safe versions of the traditional Fortran characters in BLAS and LAPACK. They are declared in `oneapi/mkl/types.hpp`, which is included automatically when you include `oneapi/mkl/blas.hpp` or `oneapi/mkl/lapack.hpp`. Like all oneMKL DPC++ functionality, they belong to the namespace `oneapi::mkl::`.

Each enumeration value comes with two names: A single-character name (the traditional BLAS/LAPACK character) and a longer, descriptive name. The two names are exactly equivalent and may be used interchangeably.

Transpose

The `transpose` type specifies whether an input matrix should be transposed and/or conjugated. It can take the following values:

Short Name	Long Name	Description
<code>transpose::N</code>	<code>transpose::nontrans</code>	Do not transpose or conjugate the matrix.
<code>transpose::T</code>	<code>transpose::trans</code>	Transpose the matrix.
<code>transpose::C</code>	<code>transpose::conjtrans</code>	Perform Hermitian transpose (transpose and conjugate). Only applicable to complex matrices.

Uplo

The `uplo` type specifies whether the lower or upper triangle of a triangular, symmetric, or Hermitian matrix should be accessed. It can take the following values:

Short Name	Long Name	Description
<code>uplo::U</code>	<code>uplo::upper</code>	Access the upper triangle of the matrix.
<code>uplo::L</code>	<code>uplo::lower</code>	Access the lower triangle of the matrix.

In both cases, elements that are not in the selected triangle are not accessed or updated.

Diag

The `diag` type specifies the values on the diagonal of a triangular matrix. It can take the following values:

Short Name	Long Name	Description
<code>diag::N</code>	<code>diag::nonunit</code>	The matrix is not unit triangular. The diagonal entries are stored with the matrix data.
<code>diag::U</code>	<code>diag::unit</code>	The matrix is unit triangular (the diagonal entries are all 1s). The diagonal entries in the matrix data are not accessed.

Side

The `side` type specifies the order of matrix multiplication when one matrix has a special form (triangular, symmetric, or Hermitian):

Short Name	Long Name	Description
<code>side::L</code>	<code>side::left</code>	The special form matrix is on the left in the multiplication.
<code>side::R</code>	<code>side::right</code>	The special form matrix is on the right in the multiplication.

Offset

The `offset` type specifies whether the offset to apply to an output matrix is a fix offset, column offset or row offset. It can take the following values

Short Name	Long Name	Description
<code>offset::F</code>	<code>offset::fix</code>	The offset to apply to the output matrix is fix, all the inputs in the <code>C_offset</code> matrix has the same value given by the first element in the <code>co</code> array.
<code>offset::C</code>	<code>offset::column</code>	The offset to apply to the output matrix is a column offset, that is to say all the columns in the <code>C_offset</code> matrix are the same and given by the elements in the <code>co</code> array.
<code>offset::R</code>	<code>offset::row</code>	The offset to apply to the output matrix is a row offset, that is to say all the rows in the <code>C_offset</code> matrix are the same and given by the elements in the <code>co</code> array.

`value_or_pointer`

The `value_or_pointer` type is used as a wrapper to enable users to pass either a pointer or a value to BLAS routines that take scalar parameters (usually named `alpha` or `beta`). Its use is described in [Scalar Arguments](#).

Vector Math Data Types

oneMKL VM for Data Parallel C++ (DPC++) introduces a slice type, available in the `oneapi::mkl` namespace. Slices are used in the DPC++ VM Strided APIs. oneMKL slices accept positive, zero, and negative strides, for forward, static, and backward traversals of a vector, respectively.

Constructors	Description
<code>slice()</code>	Default constructor equivalent to <code>slice(0, 0, 0)</code> .
<code>slice(std::size_t start, std::size_t size, std::int64_t stride)</code>	Slice defining the start index, the number of values to select, and the stride between two elements.
<code>slice(constslice& other)</code>	Copy constructor.

For example:

- `slice(1, 5, 2)` defines a selector of elements at indices 1, 3, 5, 7, 9 in a buffer or array;
- `slice(0, 5, 0)` defines a selector of a single element at index 0, repeated five times;
- `slice(9, 4, -3)` defines a selector of elements at indices 9, 6, 3, 0.

A slice is considered invalid if it produces negative indices. A slice of size 0 selects no element. If a slice may cause out-of-bounds memory accesses, the behavior is undefined.

The behavior of VM Strided APIs used with invalid or non-equal slices is configurable with several oneMKL VM mode values. See the [set_mode](#) function for possible values and their descriptions.

In the `slice_cyclic` mode, zero-sized slices are invalid.

Matrix Storage

The oneMKL BLAS and LAPACK routines for DPC++ use several matrix and vector storage formats. These are the same formats used in traditional Fortran BLAS/LAPACK.

General Matrix

A general matrix A of m rows and n columns with leading dimension lda is represented as a one dimensional array a of size of at least $lda * n$ if column major layout is used and at least $lda * m$ if row major layout is used. Before entry in any BLAS function using a general matrix, the leading m by n part of the array a must contain the matrix A . For column (respectively, row) major layout, the elements of each column (respectively, row) are contiguous in memory while the elements of each row (respectively, column) are at distance lda from the same elements in the previous row (respectively, column).

Visually, the matrix

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} & \dots & A_{1n} \\ A_{21} & A_{22} & A_{23} & \dots & A_{2n} \\ A_{31} & A_{32} & A_{33} & \dots & A_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & A_{m3} & \dots & A_{mn} \end{bmatrix}$$

is stored in memory as an array

For column major layout

$$a = [\underbrace{A_{11}, A_{21}, A_{31}, \dots, A_{m1}, *, \dots, *}_{lda}, \underbrace{A_{12}, A_{22}, A_{32}, \dots, A_{m2}, *, \dots, *}_{lda}, \dots, \underbrace{A_{1n}, A_{2n}, A_{3n}, \dots, A_{mn}, *, \dots, *}_{lda}]$$

$lda \times n$

For row major layout

$$a = [\underbrace{A_{11}, A_{12}, A_{13}, \dots, A_{1n}, *, \dots, *}_{lda}, \underbrace{A_{21}, A_{22}, A_{23}, \dots, A_{2n}, *, \dots, *}_{lda}, \dots, \underbrace{A_{m1}, A_{m2}, A_{m3}, \dots, A_{mn}, *, \dots, *}_{lda}]$$

$m \times lda$

Triangular Matrix

A triangular matrix A of n rows and n columns with leading dimension lda is represented as a one dimensional array a , of a size of at least $lda * n$. When column (respectively, row) major layout is used, the elements of each column (respectively, row) are contiguous in memory while the elements of each row (respectively, column) are at distance lda from the same elements in the previous row (respectively, column).

Before entry in any BLAS function using a triangular matrix,

- If `upper_lower = uplo::upper`, the leading n by n upper triangular part of the array a must contain the upper triangular part of the matrix A . The strictly lower triangular part of the array a is not referenced. In other words, the matrix

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} & \dots & A_{1n} \\ * & A_{22} & A_{23} & \dots & A_{2n} \\ * & * & A_{33} & \dots & A_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ * & * & * & \dots & A_{nn} \end{bmatrix}$$

is stored in memory as the array

- For column major layout

$$a = \underbrace{[A_{11}, *, \dots, *, A_{12}, A_{22}, *, \dots, *, \dots, A_{1n}, A_{2n}, A_{3n}, \dots, A_{nn}, *, \dots, *]}_{\text{lda} \times n}$$

lda lda lda

- For row major layout

$$a = \underbrace{[A_{11}, A_{12}, A_{13}, \dots, A_{1n}, *, \dots, *, *, A_{22}, A_{23}, \dots, A_{2n}, *, \dots, *, \dots, *, \dots, *, A_{nn}, *, \dots, *]}_{\text{lda} \times n}$$

lda lda lda

- If `upper_lower = uplo::lower`, the leading `n` by `n` lower triangular part of the array `a` must contain the lower triangular part of the matrix `A`. The strictly upper triangular part of the array `a` is not referenced. That is, the matrix

$$A = \begin{bmatrix} A_{11} & * & * & \dots & * \\ A_{21} & A_{22} & * & \dots & * \\ A_{31} & A_{32} & A_{33} & \dots & * \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & A_{n3} & \dots & A_{nn} \end{bmatrix}$$

is stored in memory as the array

- For column major layout

$$a = [\underbrace{A_{11}, A_{21}, A_{31}, \dots, A_{n1}, *, \dots, *}_{lda}, \underbrace{*, A_{22}, A_{32}, \dots, A_{n2}, *, \dots, *}_{lda}, \dots, \underbrace{*, \dots, *, A_{nn}, *, \dots, *}_{lda}]$$

$lda \times n$

- For row major layout

$$a = [\underbrace{A_{11}, *, \dots, *}_{lda}, \underbrace{A_{21}, A_{22}, *, \dots, *}_{lda}, \dots, \underbrace{A_{n1}, A_{n2}, A_{n3}, \dots, A_{nn}, *, \dots, *}_{lda}]$$

$lda \times n$

Band Matrix

A general band matrix A of m rows and n columns with kl sub-diagonals, ku super-diagonals and leading dimension lda is represented as a one dimensional array a of size at least $lda * n$ (respectively, $lda * m$) if column (respectively, row) major layout is used.

Before entry in any BLAS function using a general band matrix, the leading $(kl + ku + 1)$ by n (respectively, m) part of the array a must contain the matrix A . This matrix must be supplied column-by-column (respectively, row-by-row), with the main diagonal of the matrix in row ku (respectively, column kl) of the array (0-based indexing), the first super-diagonal starting at position 1 (respectively, 0) in row $(ku - 1)$ (respectively, column $(kl + 1)$), the first sub-diagonal starting at position 0 (respectively, 1) in row $(ku + 1)$ (respectively, column $(kl - 1)$), and so on. Elements in the array a that do not correspond to elements in the band matrix (such as the top left ku -by- ku triangle) are not referenced.

Visually, the matrix A

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} & \dots & A_{1,ku+1} & * & \dots & \dots & \dots & \dots & \dots & * \\ A_{21} & A_{22} & A_{23} & A_{24} & \dots & A_{2,ku+2} & * & \dots & \dots & \dots & \dots & * \\ A_{31} & A_{32} & A_{33} & A_{34} & A_{35} & \dots & A_{3,ku+3} & * & \dots & \dots & \dots & * \\ \vdots & A_{42} & A_{43} & \ddots & \ddots & \ddots & \ddots & \ddots & * & \dots & \dots & \vdots \\ A_{kl+1,1} & \vdots & A_{53} & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & * & \dots & \vdots \\ * & A_{kl+2,2} & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & * & A_{kl+3,3} & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & * \\ \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \vdots & \vdots & * & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & A_{n-ku,n} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & A_{m-2,n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & A_{m-1,n} \\ * & * & * & \dots & \dots & \dots & * & A_{m,m-kl} & \dots & A_{m,n-2} & A_{m,n-1} & A_{m,n} \end{bmatrix}$$

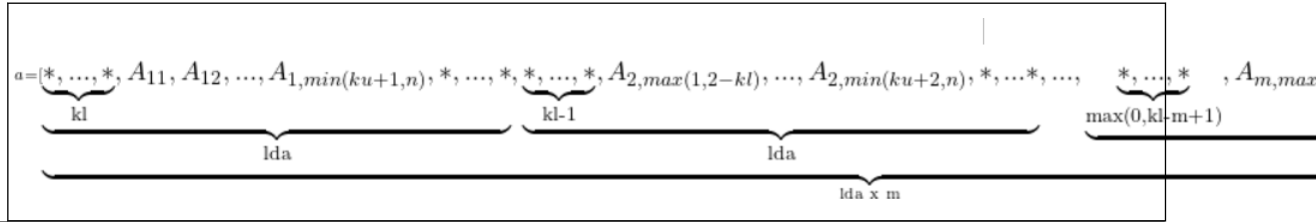
is stored in memory as an array

For column major layout

$$a = [\underbrace{*, \dots, *}_{ku}, \underbrace{A_{11}, A_{12}, \dots, A_{\min(kl+1,m),1}}_{lda}, \underbrace{*, \dots, *, *, \dots, *, A_{\max(1,2-ku),2}, \dots, A_{\min(kl+2,m),2}}_{ku-1}, \underbrace{*, \dots, *, \dots, *, \dots, *, A_{\max(1,2-ku),2}, \dots, A_{\min(kl+2,m),2}}_{lda}, \underbrace{*, \dots, *}_{\max(0,ku-n+1)}, A_{\max(1,2-ku),2}, \dots, A_{\min(kl+2,m),2}]$$

$lda \times n$

For row major layout



The following program segment transfers a band matrix from conventional full matrix storage (variable `matrix`, with leading dimension `ldm`) to band storage (variable `a`, with leading dimension `lda`):

- Using column major layout

```
for (j = 0; j < n; j++) {
    k = ku - j;
    for (i = max(0, j - ku); i < min(m, j + kl + 1); i++) {
        a[(k + i) + j * lda] = matrix[i + j * ldm];
    }
}
```

- Using row major layout

```
for (i = 0; i < m; i++) {
    k = kl - i;
    for (j = max(0, i - kl); j < min(n, i + ku + 1); j++) {
        a[(k + j) + i * lda] = matrix[j + i * ldm];
    }
}
```

Triangular Band Matrix

A triangular band matrix A of n rows and n columns with k sub/super-diagonals and leading dimension `lda` is represented as a one dimensional array `a` of size at least `lda * n`.

Before entry in any BLAS function using a triangular band matrix,

- If `upper_lower = uplo::upper`, the leading $(k + 1)$ by n part of the array `a` must contain the upper triangular band part of the matrix A . When using column major layout, this matrix must be supplied column-by-column (respectively, row-by-row) with the main diagonal of the matrix in row (k) (respectively, column 0) of the array, the first super-diagonal starting at position 1 (respectively, 0) in row ($k - 1$) (respectively, column 1), and so on. Elements in the array `a` that do not correspond to elements in the triangular band matrix (such as the bottom left k by k triangle) are not referenced.

Visually, the matrix

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} & \dots & A_{1,k+1} & * & \dots & \dots & \dots & \dots & \dots & * \\ * & A_{22} & A_{23} & A_{24} & \dots & A_{2,k+2} & * & \dots & \dots & \dots & \dots & * \\ \vdots & * & A_{33} & A_{34} & A_{35} & \dots & A_{3,k+3} & * & \dots & \dots & \dots & * \\ \vdots & \vdots & * & \ddots & \ddots & \ddots & \ddots & \ddots & * & \dots & \dots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & * & \dots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & * \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & A_{n-k,n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & A_{n-2,n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \ddots & A_{n-1,n} \\ * & * & * & \dots & \dots & \dots & \dots & \dots & \dots & \dots & * & A_{n,n} \end{bmatrix}$$

is stored as an array

- For column major layout

$$a = [\underbrace{*, \dots, *, A_{11}, *, \dots, *, *, \dots, *, A_{\max(1,2-k),2}, \dots, A_{2,2}, *, \dots, *, \dots}_{\text{lda} \times n}, \underbrace{*, \dots, *, A_{\max(1,n-k),n}, \dots, A_{n,n}, *, \dots, *}_{\text{lda}}]$$

$\underbrace{\hspace{10em}}_{\text{lda} \times n}$

- For row major layout

$$a = [\underbrace{A_{11}, A_{21}, \dots, A_{\min(k+1,n),1}, *, \dots, *, A_{2,2}, \dots, A_{\min(k+2,n),2}, *, \dots, *, \dots}_{\text{lda} \times n}, \underbrace{A_{n,n}, *, \dots, *}_{\text{lda}}]$$

$\underbrace{\hspace{10em}}_{\text{lda} \times n}$

The following program segment transfers a band matrix from conventional full matrix storage (variable `matrix`, with leading dimension `ldm`) to band storage (variable `a`, with leading dimension `lda`):

- Using column major layout

```
for (j = 0; j < n; j++) {
    m = k - j;
    for (i = max(0, j - k); i <= j; i++) {
        a[(m + i) + j * lda] = matrix[i + j * ldm];
    }
}
```

- Using row major layout

```
for (i = 0; i < n; i++) {
    m = -i;
    for (j = i; j < min(n, i + k + 1); j++) {
        a[(m + j) + i * lda] = matrix[j + i * ldm];
    }
}
```

- If `upper_lower = uplo::lower`, the leading $(k + 1)$ by n part of the array `a` must contain the upper triangular band part of the matrix `A`. This matrix must be supplied column-by-column with the main diagonal of the matrix in row 0 of the array, the first sub-diagonal starting at position 0 in row 1, and so on. Elements in the array `a` that do not correspond to elements in the triangular band matrix (such as the bottom right k by k triangle) are not referenced.

That is, the matrix

$$A = \begin{bmatrix} A_{11} & * & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & * \\ A_{21} & A_{22} & * & \dots & \dots & \dots & \dots & \dots & \dots & \dots & * \\ A_{31} & A_{32} & A_{33} & * & \dots & \dots & \dots & \dots & \dots & \dots & * \\ \vdots & A_{42} & A_{43} & \ddots & \ddots & \dots & \dots & \dots & \dots & \dots & \vdots \\ A_{k+1,1} & \vdots & A_{53} & \ddots & \ddots & \ddots & \dots & \dots & \dots & \dots & \vdots \\ * & A_{k+2,2} & \vdots & \ddots & \ddots & \ddots & \dots & \dots & \dots & \dots & \vdots \\ \vdots & * & A_{k+3,3} & \ddots & \ddots & \ddots & \ddots & \dots & \dots & \dots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \dots & \dots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \dots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ * & * & * & \dots & \dots & \dots & * & A_{n,n-k} & \dots & A_{n,n-2} & A_{n,n-1} & A_{n,n} \end{bmatrix}$$

is stored as the array

- For column major layout

$$a = [\underbrace{A_{11}, A_{21}, \dots, A_{\min(k+1,n),1}, *, \dots, *}_{lda}, \underbrace{A_{2,2}, \dots, A_{\min(k+2,n),2}, *, \dots, *}_{lda}, \dots, \underbrace{A_{n,n}, *, \dots *}_{lda}]$$

$lda \times n$

- For row major layout

$$a = [\underbrace{*, \dots, *}_k, \underbrace{*, \dots, *}_{k-1}, \underbrace{A_{\max(1,2-k),2}, \dots, A_{2,2}, *, \dots, *}_{lda}, \underbrace{*, \dots, *}_{\max(0,k-n+1)}, \underbrace{A_{\max(1,n-k),n}, \dots, A_{n,n}, *, \dots *}_{lda}]$$

$lda \times n$

The following program segment transfers a band matrix from conventional full matrix storage (variable `matrix`, with leading dimension `ldm`) to band storage (variable `a`, with leading dimension `lda`):

- Using column major layout

```
for (j = 0; j < n; j++) {
    m = -j;
    for (i = j; i < min(n, j + k + 1); i++) {
        a[(m + i) + j * lda] = matrix[i + j * ldm];
    }
}
```

- Using row major layout

```
for (i = 0; i < n; i++) {
    m = k - i;
    for (j = max(0, i - k); j <= i; j++) {
```



```

    a[(m + j) + i * lda] = matrix[j + i * ldm];
}
}

```

Packed Triangular Matrix

A triangular matrix A of n rows and n columns is represented in packed format as a one dimensional array a of size at least $(n*(n + 1))/2$. All elements in the upper or lower part of the matrix A are stored contiguously in the array a .

Before entry in any BLAS function using a triangular packed matrix,

- If `upper_lower = uplo::upper`, the first $(n*(n + 1))/2$ elements in the array a must contain the upper triangular part of the matrix A packed sequentially, column by column so that $a[0]$ contains A_{11} , $a[1]$ and $a[2]$ contain A_{12} and A_{22} respectively, and so on. Hence, the matrix

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} & \dots & A_{1n} \\ * & A_{22} & A_{23} & \dots & A_{2n} \\ * & * & A_{33} & \dots & A_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ * & * & * & \dots & A_{nn} \end{bmatrix}$$

is stored as the array

- For column major layout

$$a = [A_{11}, A_{12}, A_{22}, A_{13}, A_{23}, A_{33}, \dots, A_{(n-1),n}, A_{nn}]$$

- For row major layout

$$a = [A_{11}, A_{12}, A_{13}, \dots, A_{1n}, A_{22}, A_{23}, \dots, A_{2n}, \dots, A_{(n-1),(n-1)}, A_{(n-1),n}, A_{nn}]$$

- If `upper_lower = uplo::lower`, if column (respectively, row) major layout is used, the first $(n*(n + 1))/2$ elements in the array a must contain the lower triangular part of the matrix A packed sequentially, column by column (respectively, row by row) so that $a[0]$ contains A_{11} , $a[1]$ and $a[2]$ contain A_{21} and A_{31} respectively, and so on. The matrix

$$A = \begin{bmatrix} A_{11} & * & * & \dots & * \\ A_{21} & A_{22} & * & \dots & * \\ A_{31} & A_{32} & A_{33} & \dots & * \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & A_{n3} & \dots & A_{nn} \end{bmatrix}$$

is stored as the array

- For column major layout

$$a = [A_{11}, A_{21}, A_{31}, \dots, A_{n1}, A_{22}, A_{32}, \dots, A_{n2}, \dots, A_{(n-1),(n-1)}, A_{n,(n-1)}, A_{nn}]$$

- For row major layout

$$a = [A_{11}, A_{21}, A_{22}, A_{31}, A_{32}, A_{33}, \dots, A_{n,(n-1)}, A_{nn}]$$

Vector

A vector x of n elements with increment `incx` is represented as a one dimensional array x of size at least $(1 + (n - 1) * \text{abs}(\text{incx}))$.

Visually, the vector

$$X = (X_1, X_2, X_3, \dots, X_n)$$

is stored in memory as an array

$$\begin{aligned} x &= [\underbrace{X_1, *, \dots, *}_{\text{incx}}, \underbrace{X_2, *, \dots, *}_{\text{incx}}, \dots, \underbrace{X_{n-1}, *, \dots, *}_{\text{incx}}, X_n] \quad \text{if } \text{incx} > 0 \\ &\quad \underbrace{\hspace{10em}}_{1 + (n-1) \times \text{incx}} \\ x &= [\underbrace{X_n, *, \dots, *}_{-\text{incx}}, \underbrace{X_{n-1}, *, \dots, *}_{-\text{incx}}, \dots, \underbrace{X_2, *, \dots, *}_{-\text{incx}}, X_1] \quad \text{if } \text{incx} < 0 \\ &\quad \underbrace{\hspace{10em}}_{1 + (1-n) \times \text{incx}} \end{aligned}$$

Scalar Arguments

Basic Use

The USM version of oneMKL BLAS routines for DPC++ will accept either a scalar (for example `float`) or pointer (`float*`) for parameters that represent a single fixed value (not a vector or matrix). These parameters are often named `alpha` or `beta` in BLAS.

For example, users can call `gemv` with pointers:

```
float *alpha_ptr = sycl::malloc_shared<float>(1, queue);
float *beta_ptr = sycl::malloc_shared<float>(1, queue);
// fill alpha_ptr and beta_ptr with desired values
oneapi::mkl::blas::column_major::gemv(queue, trans, m, n, alpha_ptr, lda, x, incx, beta_ptr,
                                       y, incy).wait();
```

or with literal values:

```
oneapi::mkl::blas::column_major::gemv(queue, trans, m, n, 2, lda, x, incx, 2.7,
                                       y, incy).wait();
```

Users can even mix scalar and pointer parameters in a single call:

```
float *alpha_ptr = sycl::malloc_shared<float>(1, queue);
oneapi::mkl::blas::column_major::gemv(queue, trans, m, n, alpha_ptr, lda, x, incx, 2.7,
                                       y, incy).wait();
```

Pointers provided for scalar parameters may be SYCL-managed pointers to either device or host memory (for example pointers created with `sycl::malloc_device`, `sycl::malloc_shared`, or `sycl::malloc_host`), or they may be raw pointers created with `malloc` or `new`.

For most users, this is all they need to know. A few details about how this is implemented are provided below.

Wrapper type for scalar parameters

The USM version of oneMKL BLAS routines use a templated `value_or_pointer<T>` wrapper to enable either pointers or values to be passed to routines that take a scalar parameter. This type is in the `oneapi::mkl::` namespace and defined in the header file `oneapi/mkl/types.hpp`.

In general, users should not explicitly use this type in their code. There is no need to construct an object of type `value_or_pointer` in order to use the oneMKL functions that include it in their function signatures. Instead, values and pointers in user code will be implicitly converted to this type when a user calls a oneMKL function.

The `value_or_pointer<T>` wrapper has two constructors, one that converts a value of type `T` (or anything convertible to `T`) to `value_or_pointer<T>`, and another that converts a pointer to `T` to `value_or_pointer<T>`. Internally, the oneMKL functions can behave slightly differently depending on whether the underlying data is a value or a pointer, and if it points to host-side memory or device-side memory, but these uses should be transparent to users.

Dependencies and pointer dereferencing

For scalar parameters passed to oneMKL BLAS routines as pointers, the timing of pointer dereferencing depends on whether it is a USM-managed pointer or a raw pointer.

For a USM-managed pointer, it is dereferenced at kernel launch after the dependencies passed to the function have been resolved, so the value may be assigned asynchronously in another event passed as a dependency to the routine.

A raw pointer (such as those allocated with `malloc` or `new`) is dereferenced at the function call, so it must be valid when the function is called. In this case the data must be valid when the function is called and it may not be assigned asynchronously.

Error Handling

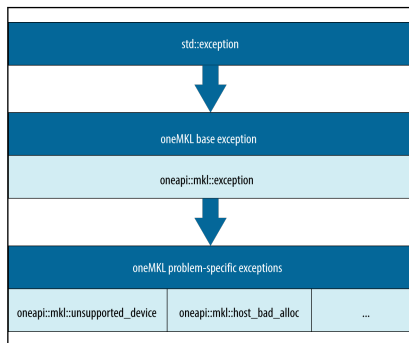
oneMKL error handling relies on the mechanism of C++ exceptions. Should errors occur, they are propagated at the point of a function call where they are caught using standard C++ error handling mechanisms.

Exception Classification

Exception classification in oneMKL is aligned with C++ Standard Library classification. oneMKL introduces a class that defines the base class in the hierarchy of oneMKL exception classes. All oneMKL routines throw exceptions inherited from this base class.

In the hierarchy of oneMKL exceptions, `oneapi::mkl::exception` is the base class inherited from the `std::exception` class. All other oneMKL exception classes are derived from this base class.

Hierarchy of oneMKL exceptions



All oneMKL problem-specific exceptions are listed in the following table.

Exception Class	Description
<code>oneapi::mkl::unsupported_device</code>	Reports a problem when the routine is not supported on a specific device
<code>oneapi::mkl::host_bad_alloc</code>	Reports a problem that occurred during memory allocation on the host
<code>oneapi::mkl::device_bad_alloc</code>	Reports a problem that occurred during memory allocation on a specific device
<code>oneapi::mkl::unimplemented</code>	Reports a problem when a specific routine has not been implemented for the specified parameters
<code>oneapi::mkl::invalid_argument</code>	Reports problem when arguments to the routine were rejected
<code>oneapi::mkl::uninitialized</code>	Reports problem when a handle (descriptor) has not been initialized
<code>oneapi::mkl::computation_error</code>	Reports any computation error that occurred inside the oneMKL routine
<code>oneapi::mkl::batch_error</code>	Reports errors that occurred inside batch oneMKL routines

Known Limitations

oneMKL DPC++ routines currently may not work with the following SYCL extensions.

Discard Events

Using oneMKL DPC++ routines with queues created with the `discard_events` property is not supported and may lead to unexpected exceptions being thrown.

SYCL Graph

Recording oneMKL DPC++ routines with the SYCL Graph extension is not supported. Unexpected behavior may occur.

BLAS Routines

The Intel® oneAPI Math Kernel Library provides a Data Parallel C++ interface to some of the BLAS routines. The routine descriptions are arranged in several sections:

- [BLAS Level 1 Routines](#) (vector-vector operations)
- [BLAS Level 2 Routines](#) (matrix-vector operations)
- [BLAS Level 3 Routines](#) (matrix-matrix operations)
- [BLAS-like Extensions](#)

BLAS Level 1 Routines

This section describes BLAS Level 1 routines, which perform vector-vector operations. The following table lists the BLAS Level 1 routine groups and the data types associated with them.

Routine Group	Data Types	Description
asum	float, double, mixed float and <code>std::complex<float></code> , mixed double and <code>std::complex<double></code>	Sum of vector magnitudes
axpy	<code>sycl::half</code> , <code>oneapi::mkl::bfloat16</code> , float, double, <code>std::complex<float></code> , <code>std::complex<double></code>	Scalar-vector product
copy	float, double, <code>std::complex<float></code> , <code>std::complex<double></code>	Copy vector
dot	<code>sycl::half</code> , <code>oneapi::mkl::bfloat16</code> , float, double, mixed float and double	Dot product
sdspot	mixed float and double	Dot product with double precision

Routine Group	Data Types	Description
dotc	<code>std::complex<float></code> , <code>std::complex<double></code>	Dot product conjugated
dotu	<code>std::complex<float></code> , <code>std::complex<double></code>	Dot product unconjugated
nrm2	<code>sycl::half</code> , <code>oneapi::mkl::bfloat16</code> , <code>float</code> , <code>double</code> , mixed <code>float</code> and <code>std::complex<float></code> , mixed <code>double</code> and <code>std::complex<double></code>	Vector 2-norm (Euclidean norm)
rot	<code>sycl::half</code> , <code>oneapi::mkl::bfloat16</code> , <code>float</code> , <code>double</code> , mixed <code>float</code> and <code>std::complex<float></code> , mixed <code>double</code> and <code>std::complex<double></code>	Plane rotation of points
rotg	<code>float</code> , <code>double</code> , <code>std::complex<float></code> , <code>std::complex<double></code>	Generate Givens rotation of points
rotm	<code>float</code> , <code>double</code>	Modified Givens plane rotation of points
rotmg	<code>float</code> , <code>double</code>	Generate modified Givens plane rotation of points
scal	<code>sycl::half</code> , <code>oneapi::mkl::bfloat16</code> , <code>float</code> , <code>double</code> , <code>std::complex<float></code> , <code>std::complex<double></code> , mixed <code>float</code> and <code>std::complex<float></code> , mixed <code>double</code> and <code>std::complex<double></code>	Vector-scalar product
swap	<code>float</code> , <code>double</code> , <code>std::complex<float></code> , <code>std::complex<double></code>	Vector-vector swap
iamax	<code>float</code> , <code>double</code> , <code>std::complex<float></code> , <code>std::complex<double></code>	Index of the maximum absolute value element of a vector
iamin	<code>float</code> , <code>double</code> , <code>std::complex<float></code> , <code>std::complex<double></code>	Index of the minimum absolute value element of a vector

[asum](#)

Computes the sum of magnitudes of the vector elements.

Description

The `asum` routine computes the sum of the magnitudes of elements of a real vector, or the sum of magnitudes of the real and imaginary parts of elements of a complex vector. The operation is defined as:

$$\text{result} \leftarrow \sum_{i=1}^n (|\text{Re}(X_i)| + |\text{Im}(X_i)|)$$

where:

- `x` is a vector with `n` elements

`asum` supports the following precisions:

T	Tres
<code>float</code>	<code>float</code>
<code>double</code>	<code>double</code>
<code>std::complex<float></code>	<code>float</code>
<code>std::complex<double></code>	<code>double</code>

asum (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void asum(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<Tres,1> &result)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void asum(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<Tres,1> &result)
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>n</code>	Number of elements in vector <code>x</code> .
<code>x</code>	Buffer holding input vector <code>x</code> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
<code>incx</code>	Stride of vector <code>x</code> .

Output Parameters

result Buffer where the scalar result is stored.

asum (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event asum(sycl::queue &queue,
                    std::int64_t n,
                    const T *x,
                    std::int64_t incx,
                    Tres *result,
                    const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event asum(sycl::queue &queue,
                    std::int64_t n,
                    const T *x,
                    std::int64_t incx,
                    Tres *result,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vector x .
x	Pointer to input vector x . Size of the array holding vector x must be least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x .
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

result	Pointer to where the scalar result is stored.
---------------	---

Return Values

Output event to wait on to ensure computation is complete.

axpy

Computes a vector-scalar product and adds the result to a vector.

Description

The `axpy` routines compute a scalar-vector product and add the result to a vector. The operation is defined as:

$$y \leftarrow \alpha * x + y$$

where:

- x and y are vectors of n elements
- α is a scalar

`axpy` supports the following precisions:

T
<code>sycl::half</code>
<code>oneapi::mkl::bfloat16</code>
<code>float</code>
<code>double</code>
<code>std::complex<float></code>
<code>std::complex<double></code>

axpy (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void axpy(sycl::queue &queue,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void axpy(sycl::queue &queue,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>n</code>	Number of elements in vector x .
<code>alpha</code>	Specifies scalar α .

<code>x</code>	Buffer holding input vector <code>x</code> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
<code>incx</code>	Stride of vector <code>x</code> .
<code>y</code>	Buffer holding input vector <code>y</code> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
<code>incy</code>	Stride of vector <code>y</code> .
Output Parameters	
<code>y</code>	Buffer holding updated vector <code>y</code> .

Examples

An example of how to use `axpy` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/blas/source/axpy.cpp
```

axpy (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event axpy(sycl::queue &queue,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event axpy(sycl::queue &queue,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>n</code>	Number of elements in vector <code>x</code> .
<code>alpha</code>	Specifies the scalar <code>alpha</code> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
<code>x</code>	Pointer to input vector <code>x</code> . Size of the array holding vector <code>x</code> must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.

<code>incx</code>	Stride of vector x .
<code>y</code>	Pointer to input vector y . Size of the array holding vector y must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
<code>incy</code>	Stride of vector y .
<code>dependencies</code>	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

<code>y</code>	Pointer to updated vector y .
----------------	---------------------------------

Return Values

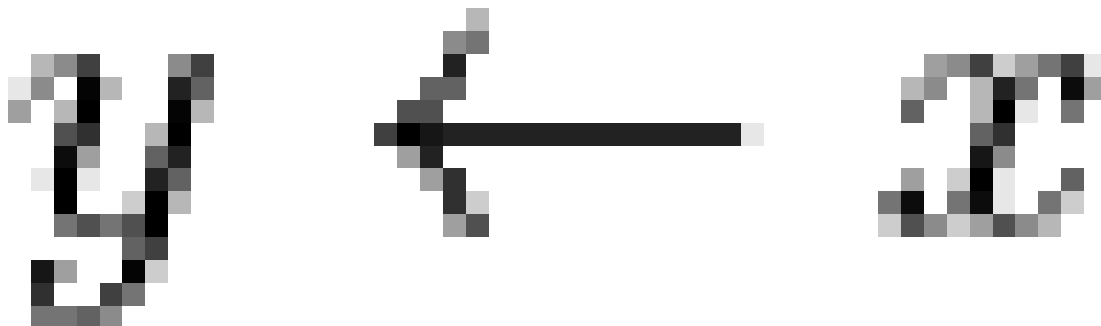
Output event to wait on to ensure computation is complete.

copy

Copies a vector to another vector.

Description

The `copy` routines copy one vector to another. The operation is defined as:



where:

- x and y are vectors of n elements

`copy` supports the following precisions:

T
<code>float</code>
<code>double</code>
<code>std::complex<float></code>
<code>std::complex<double></code>

copy (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void copy(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void copy(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vector <i>x</i> .
x	Buffer holding input vector <i>x</i> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> .
incy	Stride of vector <i>y</i> .

Output Parameters

y	Buffer holding updated vector <i>y</i> .
---	--

copy (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event copy(sycl::queue &queue,
                    std::int64_t n,
                    const T *x,
                    std::int64_t incx,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event copy(sycl::queue &queue,
                    std::int64_t n,
                    const T *x,
                    std::int64_t incx,
                    T *y,
```

```

std::int64_t incy,
const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vector x .
x	Pointer to the input vector x . Size of the array holding vector x must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x .
incy	Stride of vector y .
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y	Pointer to updated vector y .
---	---------------------------------

Return Values

Output event to wait on to ensure computation is complete.

dot

Computes the `dot` product of two real vectors.

Description

The `dot` routines perform a dot product between two vectors. The operation is defined as:

$$\text{result} = \sum_{i=1}^n X_i Y_i$$

`dot` supports the following precisions:

T	Tres
<code>sycl::half</code>	<code>sycl::half</code>
<code>oneapi::mkl::bfloat16</code>	<code>oneapi::mkl::bfloat16</code>

T	Tres
float	float
double	double
float	double

NOTE For mixed precision version (inputs are float while result is double), `dot` product is computed with double precision.

dot (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void dot(sycl::queue &queue,
             std::int64_t n,
             sycl::buffer<T,1> &x,
             std::int64_t incx,
             sycl::buffer<T,1> &y,
             std::int64_t incy,
             sycl::buffer<Tres,1> &result)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void dot(sycl::queue &queue,
             std::int64_t n,
             sycl::buffer<T,1> &x,
             std::int64_t incx,
             sycl::buffer<T,1> &y,
             std::int64_t incy,
             sycl::buffer<Tres,1> &result)
}
```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vectors <code>x</code> and <code>y</code> .
x	Buffer holding input vector <code>x</code> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <code>x</code> .
y	Buffer holding input vector <code>y</code> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <code>y</code> .

Output Parameters

result	Buffer where the result (a scalar) will be stored.
--------	--

dot (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event dot(sycl::queue &queue,
                   std::int64_t n,
                   const T *x,
                   std::int64_t incx,
                   const T *y,
                   std::int64_t incy,
                   Tres *result,
                   const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event dot(sycl::queue &queue,
                   std::int64_t n,
                   const T *x,
                   std::int64_t incx,
                   const T *y,
                   std::int64_t incy,
                   Tres *result,
                   const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vectors <i>x</i> and <i>y</i> .
x	Pointer to input vector <i>x</i> . Size of the array holding vector <i>x</i> must be least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> .
y	Pointer to input vector <i>y</i> . Size of the array holding vector <i>y</i> must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> .
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

result	Pointer to where the result (a scalar) will be stored.
--------	--

Return Values

Output event to wait on to ensure computation is complete.

dotc

Computes the dot product of two complex vectors, conjugating the first vector.

Description

The `dotc` routines perform a dot product between two complex vectors, conjugating the first one. The operation is defined as:

result

=

$$\sum_{i=1}^n \overline{X_i} Y_i$$

`dotc` supports the following precisions:

T
<code>std::complex<float></code>
<code>std::complex<double></code>

dotc (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void dotc(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &result)
}

namespace oneapi::mkl::blas::row_major {
    void dotc(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &result)
}
```

Input Parameters

- queue
- The queue where the routine should be executed.
- n
- The number of elements in vectors `x` and `y`.
- x
- Buffer holding input vector `x`. Size of the buffer must be at least `(1 + (n - 1)*abs(incx))`. See [Matrix Storage](#) for more details.

incx	The stride of vector x .
y	Buffer holding input vector y . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	The stride of vector y .
Output Parameters	
result	The buffer where the result (a scalar) is stored.

dotc (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event dotc(sycl::queue &queue,
                    std::int64_t n,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *result,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event dotc(sycl::queue &queue,
                    std::int64_t n,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *result,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
n	The number of elements in vectors x and y .
x	Pointer to input vector x . Size of the array holding input vector x must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	The stride of vector x .
y	Pointer to input vector y . Size of the array holding input vector y must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	The stride of vector y .
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

result The pointer to where the result (a scalar) is stored.

Return Values

Output event to wait on to ensure computation is complete.

dotu

Computes the dot product of two complex vectors.

Description

The `dotu` routines perform a dot product between two complex vectors. The operation is defined as:

$$\text{result} = \sum_{i=1}^n X_i Y_i$$

`dotu` supports the following precisions:

T
<code>std::complex<float></code>
<code>std::complex<double></code>

dotu (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void dotu(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &result)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void dotu(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
```

```

        std::int64_t incy,
        sycl::buffer<T,1> &result)
    }

```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vectors x and y .
x	Buffer holding input vector x . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x .
y	Buffer holding input vector y . The buffer must have size at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y .

Output Parameters

result	Buffer where the result (a scalar) is stored.
--------	---

dotu (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event dotu(sycl::queue &queue,
                    std::int64_t n,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *result,
                    const std::vector<sycl::event> &dependencies = {})
}

```

```

namespace oneapi::mkl::blas::row_major {
    sycl::event dotu(sycl::queue &queue,
                     std::int64_t n,
                     const T *x,
                     std::int64_t incx,
                     const T *y,
                     std::int64_t incy,
                     T *result,
                     const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vectors x and y .
x	Pointer to the input vector x . Size of the array holding input vector x must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.

<code>incx</code>	Stride of vector <code>x</code> .
<code>y</code>	Pointer to input vector <code>y</code> . Size of the array holding input vector <code>y</code> must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
<code>incy</code>	Stride of vector <code>y</code> .
<code>dependencies</code>	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

<code>result</code>	Pointer to where the result (a scalar) is stored.
---------------------	---

Return Values

Output event to wait on to ensure computation is complete.

iamax

Finds the index of the element with the largest absolute value in a vector.

Description

The `iamax` routines return an index `i` such that `x[i]` has the maximum absolute value of all elements in vector `x` (real variants), or such that $(|\text{Re}(x[i])| + |\text{Im}(x[i])|)$ is maximal (complex variants).

`iamax` supports a result type `Tresult` of `std::int32_t` or `std::int64_t` and support the following precisions for the data type `T`:

T
<code>float</code>
<code>double</code>
<code>std::complex<float></code>
<code>std::complex<double></code>

NOTE The index is zero-based if `base` is set to `index_base::zero` (default) or one-based if it is set to `index_base::one`.

If either `n` or `incx` is not positive, the routine returns 0, regardless of the base of the index selected.

If more than one vector element is found with the same largest absolute value, the index of the first one encountered is returned.

If the vector contains NaN values, then the routine returns the index of the first NaN.

iamax (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void iamax(sycl::queue &queue,
               std::int64_t n,
```

```

        sycl::buffer<T, 1> &x,
        std::int64_t incx,
        sycl::buffer<Tresult, 1> &result,
        index_base base = index_base::zero)
    }

namespace oneapi::mkl::blas::row_major {
    void iamax(sycl::queue &queue,
               std::int64_t n,
               sycl::buffer<T, 1> &x,
               std::int64_t incx,
               sycl::buffer<Tresult, 1> &result,
               index_base base = index_base::zero)
    }
}

```

Input Parameters

queue	The queue where the routine will be executed.
n	The number of elements in vector <i>x</i> .
x	The buffer that holds the input vector <i>x</i> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	The stride of vector <i>x</i> .
base	Indicates how the output value is indexed. If omitted, defaults to zero-based indexing.

oneapi::mkl::index_base::zero	Zero-based (C-style) indexing: indices start at 0.
oneapi::mkl::index_base::one	One-based (Fortran-style) indexing: indices start at 1.

Output Parameters

result	The buffer where the index <i>i</i> of the maximal element is stored.
--------	---

Examples

An example of how to use `iamax` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/blas/source/iamax.cpp
```

iamax (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event iamax(sycl::queue &queue,
                     std::int64_t n,
                     const T *x,
                     std::int64_t incx,
                     Tresult *result,

```

```

        index_base base = index_base::zero,
        const std::vector<sycl::event> &dependencies = {})
    }

namespace oneapi::mkl::blas::row_major {
    sycl::event iamax(sycl::queue &queue,
        std::int64_t n,
        const T *x,
        std::int64_t incx,
        Tresult *result,
        index_base base = index_base::zero,
        const std::vector<sycl::event> &dependencies = {})
    }
}

```

Input Parameters

queue	The queue where the routine will be executed.
n	The number of elements in vector x .
x	The pointer to input vector x . Size of the array holding input vector x must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	The stride of vector x .
base	Indicates how the output value is indexed. If omitted, defaults to zero-based indexing.

oneapi::mkl::index_base::zero	Zero-based (C-style) indexing: indices start at 0.
oneapi::mkl::index_base::one	One-based (Fortran-style) indexing: indices start at 1.

dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.
--------------	--

Output Parameters

result	The pointer to where the index i of the maximal element is stored.
--------	--

Return Values

Output event to wait on to ensure computation is complete.

iamin

Finds the index of the element with the smallest absolute value in a vector

Description

The `iamin` routines return an index i such that $x[i]$ has the minimum absolute value of all elements in vector x (real variants), or such that $(|\text{Re}(x[i])| + |\text{Im}(x[i])|)$ is minimal (complex variants).

`iamin` supports a result type `Tresult` of `std::int32_t` or `std::int64_t` and support the following precisions for the data type `T`:

T

```
float
double
std::complex<float>
std::complex<double>
```

NOTE The index is zero-based if `base` is set to `index_base::zero` (default) or one-based if it is set to `index_base::one`.

If either `n` or `incx` is not positive, the routine returns 0, regardless of the base of the index selected.

If more than one vector element is found with the same smallest absolute value, the index of the first one encountered is returned.

If the vector contains NaN values, then the routine returns the index of the first NaN.

iamin (Buffer Version)**Syntax**

```
namespace oneapi::mkl::blas::column_major {
    void iamin(sycl::queue &queue,
               std::int64_t n,
               sycl::buffer<T,1> &x,
               std::int64_t incx,
               sycl::buffer<Tresult,1> &result,
               index_base base = index_base::zero)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void iamin(sycl::queue &queue,
               std::int64_t n,
               sycl::buffer<T,1> &x,
               std::int64_t incx,
               sycl::buffer<Tresult,1> &result,
               index_base base = index_base::zero)
}
```

Input Parameters

queue	The queue where the routine will be executed.
n	Number of elements in vector <code>x</code> .
x	Buffer holding input vector <code>x</code> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <code>x</code> .
base	Indicates how the output value is indexed. If omitted, defaults to zero-based indexing.

<code>oneapi::mkl::index_base::zero</code>	Zero-based (C-style) indexing: indices start at 0.
--	--

oneapi::mkl::index_base::one

One-based (Fortran-style) indexing: indices start at 1.

Output Parameters

result Buffer where the index *i* of the minimum element is stored.

iamin (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event iamin(sycl::queue &queue,
                     std::int64_t n,
                     const T *x,
                     std::int64_t incx,
                     Tresult *result,
                     index_base base = index_base::zero,
                     const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event iamin(sycl::queue &queue,
                     std::int64_t n,
                     const T *x,
                     std::int64_t incx,
                     Tresult *result,
                     index_base base = index_base::zero,
                     const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue The queue where the routine will be executed.

n Number of elements in vector *x*.

x The pointer to input vector *x*. Size of the array holding input vector *x* must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See [Matrix Storage](#) for more details.

incx Stride of vector *x*.

base Indicates how the output value is indexed. If omitted, defaults to zero-based indexing.

oneapi::mkl::index_base::zero

Zero-based (C-style) indexing: indices start at 0.

oneapi::mkl::index_base::one

One-based (Fortran-style) indexing: indices start at 1.

dependencies List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

result Pointer to where the index *i* of the minimum element is stored.

Return Values

Output event to wait on to ensure computation is complete.

nrm2

Computes the Euclidean norm of a vector.

Description

The `nrm2` routines compute Euclidean norm of a vector. The operation is defined as:

$$\text{result} = ||x||$$

where:

- `x` is a vector of `n` elements

`nrm2` supports the following precisions:

T	Tres
<code>sycl::half</code>	<code>sycl::half</code>
<code>oneapi::mkl::bfloat16</code>	<code>oneapi::mkl::bfloat16</code>
<code>float</code>	<code>float</code>
<code>double</code>	<code>double</code>
<code>std::complex<float></code>	<code>float</code>
<code>std::complex<double></code>	<code>double</code>

nrm2 (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void nrm2(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<Tres,1> &result)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void nrm2(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<Tres,1> &result)
}
```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vector x .
x	Buffer holding input vector x . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x .

Output Parameters

result	Buffer where the Euclidean norm of the vector x is stored.
--------	--

nrm2 (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event nrm2(sycl::queue &queue,
                    std::int64_t n,
                    const T *x,
                    std::int64_t incx,
                    Tres *result,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event nrm2(sycl::queue &queue,
                    std::int64_t n,
                    const T *x,
                    std::int64_t incx,
                    Tres *result,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vector x .
x	Pointer to input vector x . Size of the array holding input vector x must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x .
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

result	Pointer to where the Euclidean norm of the vector x is stored.
--------	--

Return Values

Output event to wait on to ensure computation is complete.

rot

Performs rotation of points in the plane.

Description

Given two vectors x and y of n elements, the `rot` routines compute four scalar-vector products and update the input vectors with the sum of two of these scalar-vector products. The operation is defined as:

$$\begin{bmatrix} x \\ y \end{bmatrix} \leftarrow \begin{bmatrix} c * x + s * y \\ -s * x + c * y \end{bmatrix}$$

If s is a complex type, the operation is defined as:

$$\begin{bmatrix} x \\ y \end{bmatrix} \leftarrow \begin{bmatrix} c * x + s * y \\ -\text{conj}(s) * x + c * y \end{bmatrix}$$

`rot` supports the following precisions:

T	Tc	Ts
<code>sycl::half</code>	<code>sycl::half</code>	<code>sycl::half</code>
<code>oneapi::mkl::bfloat16</code>	<code>oneapi::mkl::bfloat16</code>	<code>oneapi::mkl::bfloat16</code>
<code>float</code>	<code>float</code>	<code>float</code>
<code>double</code>	<code>double</code>	<code>double</code>
<code>std::complex<float></code>	<code>float</code>	<code>std::complex<float></code>
<code>std::complex<double></code>	<code>double</code>	<code>std::complex<double></code>
<code>std::complex<float></code>	<code>float</code>	<code>float</code>
<code>std::complex<double></code>	<code>double</code>	<code>double</code>

rot (Buffer Version)**Syntax**

```
namespace oneapi::mkl::blas::column_major {
    void rot(sycl::queue &queue,
            std::int64_t n,
            sycl::buffer<T,1> &x,
            std::int64_t incx,
            sycl::buffer<T,1> &y,
            std::int64_t incy,
```

```

        Tc c,
        Ts s)
}

namespace oneapi::mkl::blas::row_major {
    void rot(sycl::queue &queue,
            std::int64_t n,
            sycl::buffer<T,1> &x,
            std::int64_t incx,
            sycl::buffer<T,1> &y,
            std::int64_t incy,
            Tc c,
            Ts s)
}

```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vector <i>x</i> .
x	Buffer holding input vector <i>x</i> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> .
y	Buffer holding input vector <i>y</i> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> .
c	Scaling factor.
s	Scaling factor.

Output Parameters

x	Buffer holding updated buffer <i>x</i> .
y	Buffer holding updated buffer <i>y</i> .

rot (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event rot(sycl::queue &queue,
                  std::int64_t n,
                  T *x,
                  std::int64_t incx,
                  T *y,
                  std::int64_t incy,
                  oneapi::mkl::value_or_pointer<Tc> c,

```

```

        oneapi::mkl::value_or_pointer<Ts> s,
        const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event rot(sycl::queue &queue,
        std::int64_t n,
        T *x,
        std::int64_t incx,
        T *y,
        std::int64_t incy,
        oneapi::mkl::value_or_pointer<Tc> c,
        oneapi::mkl::value_or_pointer<Ts> s,
        const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vector <i>x</i> .
x	Pointer to input vector <i>x</i> . Size of the array holding input vector <i>x</i> must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> .
y	Pointer to input vector <i>y</i> . Size of the array holding input vector <i>y</i> must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> .
c	Scaling factor. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
s	Scaling factor. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

x	Pointer to updated vector <i>x</i> .
y	Pointer to updated vector <i>y</i> .

Return Values

Output event to wait on to ensure computation is complete.

rotg

Computes the parameters for a Givens rotation.

Description

Given the Cartesian coordinates (a, b) of a point, the `rotg` routines return the parameters c , s , r , and z associated with the Givens rotation. The parameters c and s define a unitary matrix such that:

$$\begin{bmatrix} c & s \\ -s & c \end{bmatrix} \cdot \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix}$$

The parameter z is defined such that if $|a| > |b|$, z is s ; otherwise if c is not 0 z is $1/c$; otherwise z is 1.

`rotg` supports the following precisions:

T	Tc
float	float
double	double
std::complex<float>	float
std::complex<double>	double

rotg (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void rotg(sycl::queue &queue,
              sycl::buffer<T,1> &a,
              sycl::buffer<T,1> &b,
              sycl::buffer<Tc,1> &c,
              sycl::buffer<T,1> &s)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void rotg(sycl::queue &queue,
              sycl::buffer<T,1> &a,
              sycl::buffer<T,1> &b,
              sycl::buffer<Tc,1> &c,
              sycl::buffer<T,1> &s)
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed
<code>a</code>	Buffer holding x -coordinate of the point.
<code>b</code>	Buffer holding y -coordinate of the point.

Output Parameters

<code>a</code>	Buffer holding parameter r associated with the Givens rotation.
----------------	---

b	Buffer holding parameter z associated with the Givens rotation.
c	Buffer holding parameter c associated with the Givens rotation.
s	Buffer holding parameter s associated with the Givens rotation.

rotg (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event rotg(sycl::queue &queue,
                    T *a,
                    T *b,
                    Tc *c,
                    T *s,
                    const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event rotg(sycl::queue &queue,
                    T *a,
                    T *b,
                    Tc *c,
                    T *s,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed
a	Pointer to x -coordinate of the point.
b	Pointer to y -coordinate of the point.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

a	Pointer to parameter x associated with the Givens rotation.
b	Pointer to parameter z associated with the Givens rotation.
c	Pointer to parameter c associated with the Givens rotation.
s	Pointer to parameter s associated with the Givens rotation.

Return Values

Output event to wait on to ensure computation is complete.

rotm

Performs modified Givens rotation of points in the plane.

Description

Given two vectors x and y , each vector element of these vectors is replaced as follows:

$$\begin{bmatrix} x_i \\ y_i \end{bmatrix} = H \begin{bmatrix} x_i \\ y_i \end{bmatrix}$$

for i from 1 to n , where H is a modified Givens transformation matrix.

`rotm` supports the following precisions:

T
float
double

rotm (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void rotm(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &param)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void rotm(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &param)
}
```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vector x .
x	Buffer holding input vector x . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.

incx	Stride of vector x .
y	Buffer holding input vector y . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y .
param	Buffer holding an array of size 5. The elements of the <code>param</code> array are: <code>param[0]</code> contains a switch, <code>flag</code> . The other array elements <code>param[1-4]</code> contain the components of the modified Givens transformation matrix H : h_{11} , h_{21} , h_{12} , and h_{22} , respectively.

Depending on the values of `flag`, the components of H are set as follows:

`flag = -1.0:`

$$H = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}$$

`flag = 0.0:`

$$H = \begin{bmatrix} 1.0 & h_{12} \\ h_{21} & 1.0 \end{bmatrix}$$

`flag = 1.0:`

$$H = \begin{bmatrix} h_{11} & 1.0 \\ -1.0 & h_{22} \end{bmatrix}$$

`flag = -2.0:`

$$H = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix}$$

In the last three cases, the matrix entries of 1.0, -1.0, 0.0 are assumed based on the value of `flag` and are not required to be set in the `param` vector.

Output Parameters

<code>x</code>	Buffer holding updated buffer <code>x</code> .
<code>y</code>	Buffer holding updated buffer <code>y</code> .

rotm (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event rotm(sycl::queue &queue,
                    std::int64_t n,
                    T *x,
                    std::int64_t incx,
                    T *y,
                    std::int64_t incy,
                    const T *param,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event rotm(sycl::queue &queue,
                    std::int64_t n,
                    T *x,
                    std::int64_t incx,
                    T *y,
                    std::int64_t incy,
                    const T *param,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>n</code>	Number of elements in vector <code>x</code> .
<code>x</code>	Pointer to the input vector <code>x</code> . Size of the array holding vector <code>x</code> must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
<code>incx</code>	Stride of vector <code>x</code> .

yparam	Pointer to the input vector y . Size of the array holding vector y must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y .
param	Pointer to an array of size 5. The elements of the <code>param</code> array are: <code>param[0]</code> contains a switch, <code>flag</code> . The other array elements <code>param[1-4]</code> contain the components of the modified Givens transformation matrix H : h_{11} , h_{21} , h_{12} , and h_{22} , respectively.

Depending on the values of `flag`, the components of H are set as follows:

`flag = -1.0:`

$$H = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}$$

`flag = 0.0:`

$$H = \begin{bmatrix} 1.0 & h_{12} \\ h_{21} & 1.0 \end{bmatrix}$$

`flag = 1.0:`

$$H = \begin{bmatrix} h_{11} & 1.0 \\ -1.0 & h_{22} \end{bmatrix}$$

`flag = -2.0:`

$$H = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix}$$

In the last three cases, the matrix entries of 1.0, -1.0, 0.0 are assumed based on the value of `flag` and are not required to be set in the `param` vector.

dependencies List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

x Pointer to updated array `x`.
y Pointer to updated array `y`.

Return Values

Output event to wait on to ensure computation is complete.

rotmg

Computes the parameters for a modified Givens rotation.

Description

Given Cartesian coordinates (`x1`, `y1`) of an input vector, the `rotmg` routines compute the components of a modified Givens transformation matrix `H` that zeros the `y`-component of the resulting vector:

$$\begin{bmatrix} x1 \\ 0 \end{bmatrix} = H \begin{bmatrix} x1 & \sqrt{d1} \\ y1 & \sqrt{d2} \end{bmatrix}$$

`rotmg` supports the following precisions:

T
<code>float</code>
<code>double</code>

rotmg (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void rotmg(sycl::queue &queue,
               sycl::buffer<T,1> &d1,
               sycl::buffer<T,1> &d2,
               sycl::buffer<T,1> &x1,
               sycl::buffer<T,1> y1,
               sycl::buffer<T,1> &param)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void rotmg(sycl::queue &queue,
               sycl::buffer<T,1> &d1,
               sycl::buffer<T,1> &d2,
               sycl::buffer<T,1> &x1,
               sycl::buffer<T,1> y1,
               sycl::buffer<T,1> &param)
}
```

Input Parameters

queue	The queue where the routine should be executed.
d1	Buffer holding the scaling factor for x -coordinate of the input vector.
d2	Buffer holding the scaling factor for y -coordinate of the input vector.
x1	Buffer holding x -coordinate of the input vector.
y1	Scalar specifying y -coordinate of the input vector.

Output Parameters

d1	Buffer holding the first diagonal element of the updated matrix.
d2	Buffer holding the second diagonal element of the updated matrix.
x1	Buffer holding x -coordinate of the rotated vector before scaling
param	Buffer holding an array of size 5. The elements of the <code>param</code> array are: <code>param[0]</code> contains a switch, <code>flag</code>. The other array elements <code>param[1-4]</code> contain the components of the array H: h_{11}, h_{21}, h_{12}, and h_{22}, respectively. Depending on the values of <code>flag</code>, the components of H are set as follows: <code>flag = -1.0:</code>

$$H = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}$$

flag = 0.0:

$$H = \begin{bmatrix} 1.0 & h_{12} \\ h_{21} & 1.0 \end{bmatrix}$$

flag = 1.0:

$$H = \begin{bmatrix} h_{11} & 1.0 \\ -1.0 & h_{22} \end{bmatrix}$$

flag = -2.0:

$$H = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix}$$

In the last three cases, the matrix entries of 1.0, -1.0, and 0.0 are assumed based on the value of `flag` and are not required to be set in the `param` vector.

rotmg (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event rotmg(sycl::queue &queue,
        T *d1,
        T *d2,
        T *x1,
        oneapi::mkl::value_or_pointer<T> y1,
        T *param,
        const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event rotmg(sycl::queue &queue,
        T *d1,
        T *d2,
```

```

    T *x1,
    oneapi::mkl::value_or_pointer<T> y1,
    T *param,
    const std::vector<sycl::event> &dependencies = {}
}

```

Input Parameters

queue	The queue where the routine should be executed.
d1	Pointer to the scaling factor for x-coordinate of the input vector.
d2	Pointer to the scaling factor for y-coordinate of the input vector.
x1	Pointer to x-coordinate of the input vector.
y1	Scalar specifying y-coordinate of the input vector. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

d1	Pointer to the first diagonal element of the updated matrix.
d2	Pointer to the second diagonal element of the updated matrix.
x1	Pointer to x-coordinate of the rotated vector before scaling
param	Pointer to an array of size 5. The elements of the <code>param</code> array are: <code>param[0]</code> contains a switch, <code>flag</code>. The other array elements <code>param[1-4]</code> contain the components of the array <code>H</code>: h_{11}, h_{21}, h_{12}, and h_{22}, respectively. Depending on the values of <code>flag</code>, the components of <code>H</code> are set as follows:

`flag = -1.0:`

$$H = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}$$

`flag = 0.0:`

$$H = \begin{bmatrix} 1.0 & h_{12} \\ h_{21} & 1.0 \end{bmatrix}$$

`flag = 1.0:`

$$H = \begin{bmatrix} h_{11} & 1.0 \\ -1.0 & h_{22} \end{bmatrix}$$

`flag = -2.0:`

$$H = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix}$$

In the last three cases, the matrix entries of 1.0, -1.0, and 0.0 are assumed based on the value of `flag` and are not required to be set in the `param` vector.

Return Values

Output event to wait on to ensure computation is complete.

scal

Computes the product of a vector by a scalar.

Description

The `scal` routines computes a scalar-vector product. The operation is defined as:

$$x \leftarrow \alpha * x$$

where:

- x is a vector of n elements
- α is a scalar

`scal` supports the following precisions:

T	Ts
<code>sycl::half</code>	<code>sycl::half</code>
<code>oneapi::mkl::bfloat16</code>	<code>oneapi::mkl::bfloat16</code>
<code>float</code>	<code>float</code>
<code>double</code>	<code>double</code>
<code>std::complex<float></code>	<code>std::complex<float></code>
<code>std::complex<double></code>	<code>std::complex<double></code>
<code>std::complex<float></code>	<code>float</code>
<code>std::complex<double></code>	<code>double</code>

scal (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void scal(sycl::queue &queue,
              std::int64_t n,
              Ts alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void scal(sycl::queue &queue,
              std::int64_t n,
              Ts alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>n</code>	Number of elements in vector x .
<code>alpha</code>	Specifies the scalar α .
<code>x</code>	Buffer holding input vector x . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
<code>incx</code>	Stride of vector x .

Output Parameters

<code>x</code>	Buffer holding updated buffer x .
----------------	-------------------------------------

scal (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event scal(sycl::queue &queue,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<Ts> alpha,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event scal(sycl::queue &queue,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<Ts> alpha,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vector <i>x</i> .
alpha	Specifies the scalar <i>alpha</i> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
x	Pointer to the input vector <i>x</i> . Size of the array must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> .

Output Parameters

x	Pointer to updated array <i>x</i> .
---	-------------------------------------

Return Values

Output event to wait on to ensure computation is complete.

sdsdot

Computes a vector-vector dot product with double precision.

Description

The `sdsdot` routines perform a dot product between two vectors with double precision. The operation is defined as:

$$\text{result} = sb + \sum_{i=1}^n X_i Y_i$$

sdsdot (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void sdsdot(sycl::queue &queue,
               std::int64_t n,
               float sb,
               sycl::buffer<float,1> &x,
               std::int64_t incx,
               sycl::buffer<float,1> &y,
               std::int64_t incy,
               sycl::buffer<float,1> &result)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void sdsdot(sycl::queue &queue,
               std::int64_t n,
               float sb,
               sycl::buffer<float,1> &x,
               std::int64_t incx,
               sycl::buffer<float,1> &y,
               std::int64_t incy,
               sycl::buffer<float,1> &result)
}
```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vectors <i>x</i> and <i>y</i> .
sb	Single precision scalar to be added to the dot product.
x	Buffer holding input vector <i>x</i> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> .
y	Buffer holding input vector <i>y</i> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> .

Output Parameters

result	Buffer where the result (a scalar) will be stored. If $n < 0$ the result is <i>sb</i> .
--------	---

sdsdot (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event sdsdot(sycl::queue &queue,
                      std::int64_t n,
                      float sb,
                      const float *x,
                      std::int64_t incx,
                      const float *y,
                      std::int64_t incy,
                      float *result,
                      const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event sdsdot(sycl::queue &queue,
                      std::int64_t n,
                      float sb,
                      const float *x,
                      std::int64_t incx,
                      const float *y,
                      std::int64_t incy,
                      float *result,
                      const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vectors <i>x</i> and <i>y</i> .
sb	Single precision scalar to be added to the dot product.
x	Pointer to input vector <i>x</i> . Size of the array must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> .
y	Pointer to the input vector <i>y</i> . Size of the array must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> .
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

result	Pointer to where the result (a scalar) will be stored. If <i>n</i> < 0 the result is <i>sb</i> .
--------	--

Return Values

Output event to wait on to ensure computation is complete.

swap

Swaps a vector with another vector.

Description

Given two vectors of n elements, x and y , the `swap` routines return vectors y and x swapped, each replacing the other. The operation is defined as:

$$\begin{bmatrix} y \\ x \end{bmatrix} \leftarrow \begin{bmatrix} x \\ y \end{bmatrix}$$

`swap` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

swap (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void swap(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void swap(sycl::queue &queue,
              std::int64_t n,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

Input Parameters

queue The queue where the routine should be executed.

<code>n</code>	Number of elements in vectors <code>x</code> and <code>y</code> .
<code>x</code>	Buffer holding input vector <code>x</code> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
<code>incx</code>	Stride of vector <code>x</code> .
<code>y</code>	Buffer holding input vector <code>y</code> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
<code>incy</code>	Stride of vector <code>y</code> .

Output Parameters

<code>x</code>	Buffer holding updated buffer <code>x</code> , that is, the input vector <code>y</code> .
<code>y</code>	Buffer holding updated buffer <code>y</code> , that is, the input vector <code>x</code> .

swap (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event swap(sycl::queue &queue,
                    std::int64_t n,
                    T *x,
                    std::int64_t incx,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event swap(sycl::queue &queue,
                    std::int64_t n,
                    T *x,
                    std::int64_t incx,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>n</code>	Number of elements in vectors <code>x</code> and <code>y</code> .
<code>x</code>	Pointer to input vector <code>x</code> . Size of the array must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
<code>incx</code>	Stride of vector <code>x</code> .
<code>y</code>	Pointer to input vector <code>y</code> . Size of the array must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
<code>incy</code>	Stride of vector <code>y</code> .

dependencies List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

x Pointer to updated array x , that is, the input vector y .

y Pointer to updated array y , that is, the input vector x .

Return Values

Output event to wait on to ensure computation is complete.

BLAS Level 2 Routines

This section describes BLAS Level 2 routines, which perform matrix-vector operations. The following table lists the BLAS Level 2 routine groups and the data types associated with them.

Routine Groups	Data Types	Description
gbmv	float, double, std::complex<float>, std::complex<double>	Matrix-vector product using a general band matrix
gemv	float, double, std::complex<float>, std::complex<double>	Matrix-vector product using a general matrix
ger	float, double	Rank-1 update of a general matrix
gerc	std::complex<float>, std::complex<double>	Rank-1 update of a conjugated general matrix
geru	std::complex<float>, std::complex<double>	Rank-1 update of a general matrix, unconjugated
hbmV	std::complex<float>, std::complex<double>	Matrix-vector product using a Hermitian band matrix
hemv	std::complex<float>, std::complex<double>	Matrix-vector product using a Hermitian matrix
her	std::complex<float>, std::complex<double>	Rank-1 update of a Hermitian matrix
her2	std::complex<float>, std::complex<double>	Rank-2 update of a Hermitian matrix
hpmv	std::complex<float>, std::complex<double>	Matrix-vector product using a Hermitian packed matrix
hpr	std::complex<float>, std::complex<double>	Rank-1 update of a Hermitian packed matrix
hpr2	std::complex<float>, std::complex<double>	Rank-2 update of a Hermitian packed matrix

Routine Groups	Data Types	Description
sbmv	float, double	Matrix-vector product using symmetric band matrix
spmv	float, double	Matrix-vector product using a symmetric packed matrix
spr	float, double	Rank-1 update of a symmetric packed matrix
spr2	float, double	Rank-2 update of a symmetric packed matrix
symv	float, double, std::complex<float>, std::complex<double>	Matrix-vector product using a symmetric matrix
syr	float, double, std::complex<float>, std::complex<double>	Rank-1 update of a symmetric matrix
syr2	float, double, std::complex<float>, std::complex<double>	Rank-2 update of a symmetric matrix
tbmv	float, double, std::complex<float>, std::complex<double>	Matrix-vector product using a triangular band matrix
tbsv	float, double, std::complex<float>, std::complex<double>	Solution of a linear system of equations with a triangular band matrix
tpmv	float, double, std::complex<float>, std::complex<double>	Matrix-vector product using a triangular packed matrix
tpsv	float, double, std::complex<float>, std::complex<double>	Solution of a linear system of equations with a triangular packed matrix
trmv	float, double, std::complex<float>, std::complex<double>	Matrix-vector product using a triangular matrix
trsv	float, double, std::complex<float>, std::complex<double>	Solution of a linear system of equations with a triangular matrix

[gbmv](#)

Computes a matrix-vector product with a general band matrix.

Description

The [gbmv](#) routines compute a scalar-matrix-vector product and add the result to a scalar-vector product, with a general band matrix. The operation is defined as:

$$y \leftarrow \alpha * op(A) * x + \beta * y$$

where:

- $op(A)$ is one of $op(A) = A$, or $op(A) = A^T$, or $op(A) = A^H$
- α and β are scalars
- A is $m \times n$ matrix with kl sub-diagonals and ku super-diagonals
- x and y are vectors

gbmv supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

gbmv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void gbmvm(queue &queue,
               oneapi::mkl::transpose trans,
               std::int64_t m,
               std::int64_t n,
               std::int64_t kl,
               std::int64_t ku,
               T alpha,
               sycl::buffer<T,1> &a,
               std::int64_t lda,
               sycl::buffer<T,1> &x,
               std::int64_t incx,
               T beta,
               sycl::buffer<T,1> &y,
               std::int64_t incy)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void gbmvm(queue &queue,
               oneapi::mkl::transpose trans,
               std::int64_t m,
               std::int64_t n,
               std::int64_t kl,
               std::int64_t ku,
               T alpha,
               sycl::buffer<T,1> &a,
               std::int64_t lda,
               sycl::buffer<T,1> &x,
               std::int64_t incx,
               T beta,
```

```

    sycl::buffer<T,1> &y,
    std::int64_t incy)
}

```

Input Parameters

queue	The queue where the routine should be executed.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A . See Data Types for more details.
m	Number of rows of matrix A . Must be at least zero.
n	Number of columns of matrix A . Must be at least zero.
kl	Number of sub-diagonals of matrix A . Must be at least zero.
ku	Number of super-diagonals of matrix A . Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
a	Buffer holding input matrix A . Size of the buffer must be at least $\text{lda} * n$ if column major layout is used, or at least $\text{lda} * m$ if row major layout is used. See Matrix Storage for more details.
lda	Leading dimension of matrix A . Must be at least $(kl + ku + 1)$ and positive.
x	Buffer holding input vector x . The length(len) of vector x is n if A is not transposed, and m if A is transposed. Size of the buffer must be at least $(1 + (\text{len} - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.
beta	Scaling factor for vector y . If $\text{beta} = 0$, vector y does not need to be initialized before calling <code>gbmv</code> .
y	Buffer holding input/output vector y . The length(len) of vector y is m , if A is not transposed, and n if A is transposed. Size of the buffer must be at least $(1 + (\text{len} - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y .

Output Parameters

y	Buffer holding updated vector y .
---	-------------------------------------

gbmv (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event gbmV(queue &queue,
                    oneapi::mkl::transpose trans,
                    std::int64_t m,
                    std::int64_t n,
                    std::int64_t kl,
                    std::int64_t ku,
                    oneapi::mkl::value_or_pointer<T> alpha,

```

```

        const T *a,
        std::int64_t lda,
        const T *x,
        std::int64_t incx,
        oneapi::mkl::value_or_pointer<T> beta,
        T *y,
        std::int64_t incy,
        const std::vector<sycl::event> &dependencies = {}
    }

namespace oneapi::mkl::blas::row_major {
    sycl::event gbmv(queue &queue,
        oneapi::mkl::transpose trans,
        std::int64_t m,
        std::int64_t n,
        std::int64_t kl,
        std::int64_t ku,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        const T *x,
        std::int64_t incx,
        oneapi::mkl::value_or_pointer<T> beta,
        T *y,
        std::int64_t incy,
        const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A . See Data Types for more details.
m	Number of rows of matrix A . Must be at least zero.
n	Number of columns of matrix A . Must be at least zero.
kl	Number of sub-diagonals of matrix A . Must be at least zero.
ku	Number of super-diagonals of matrix A . Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A . Size of the array must be at least $\text{lda} * n$ if column major layout is used, or at least $\text{lda} * m$ if row major layout is used. See Matrix Storage for more details.
lda	Leading dimension of matrix A . Must be at least $(kl + ku + 1)$ and positive.
x	Pointer to input vector x . The length <code>len</code> of vector x is n if A is not transposed, and m if A is transposed. Size of the array holding input vector x must be at least $(1 + (\text{len} - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.

beta	Scaling factor for vector y . If $\text{beta} = 0$, vector y does not need to be initialized before calling <code>gbmv</code> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
y	Pointer to input/output vector y . The length <code>len</code> of vector y is m , if A is not transposed, and n if A is transposed. Size of the array holding input/output vector y must be at least $(1 + (\text{len} - 1) * \text{abs}(\text{incy}))$ where <code>len</code> is this length. See Matrix Storage for more details.
incy	Stride of vector y .
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y	Pointer to updated vector y .
-----	---------------------------------

Return Values

Output event to wait on to ensure computation is complete.

gemv

Computes a matrix-vector product using a general matrix.

Description

The `gemv` routines compute a scalar-matrix-vector product and add the result to a scalar-vector product, with a general matrix. The operation is defined as:

$$y \leftarrow \alpha * op(A) * x + \text{beta} * y$$

where:

- $op(A)$ is one of $op(A) = A$, or $op(A) = A^T$, or $op(A) = A^H$
- α and beta are scalars
- A is $m \times n$ matrix
- x and y are vectors

`gemv` supports the following precisions:

T
<code>float</code>
<code>double</code>
<code>std::complex<float></code>
<code>std::complex<double></code>

gemv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void gemv(sycl::queue &queue,
              oneapi::mkl::transpose trans,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              T beta,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void gemv(sycl::queue &queue,
              oneapi::mkl::transpose trans,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              T beta,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

Input Parameters

queue	The queue where the routine should be executed.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A . See Data Types for more details.
m	Number of rows of matrix A . Must be at least zero.
n	Number of columns of matrix A . Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
a	Buffer holding input matrix A . Size of the buffer must be at least $\text{lda} * n$ if column major layout is used, or at least $\text{lda} * m$ if row major layout is used. See Matrix Storage for more details.
lda	Leading dimension of matrix A . Must be positive and at least m if column major layout is used or at least n if row major layout is used.
x	Buffer holding input vector x . The length len of vector x is n if A is not transposed, and m if A is transposed. Size of the buffer must be at least $(1 + (\text{len} - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.

beta	Scaling factor for vector <i>y</i> . If beta = 0, vector <i>y</i> does not need to be initialized before calling <code>gemv</code> .
<i>y</i>	Buffer holding input/output vector <i>y</i> . The length <code>len</code> of vector <i>y</i> is <i>m</i> , if <i>A</i> is not transposed, and <i>n</i> if <i>A</i> is transposed. Size of the buffer must be at least $(1 + (\text{len} - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> . Must not be zero.
Output Parameters	
<i>y</i>	Buffer holding updated vector <i>y</i> .

Examples

An example of how to use `gemv` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/blas/source/gemv.cpp
```

gemv (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event gemv(sycl::queue &queue,
                    oneapi::mkl::transpose trans,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *x,
                    std::int64_t incx,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event gemv(sycl::queue &queue,
                    oneapi::mkl::transpose trans,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *x,
                    std::int64_t incx,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
-------	---

trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A . See Data Types for more details.
m	Number of rows of matrix A . Must be at least zero.
n	Number of columns of matrix A . Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A . Size of the array must be at least $\text{lda} * n$ if column major layout is used, or at least $\text{lda} * m$ if row major layout is used.
lda	Leading dimension of matrix A . Must be positive and at least m if column major layout is used or at least n if row major layout is used.
x	Pointer to the input vector x . The length <code>len</code> of vector x is n if A is not transposed, and m if A is transposed. Size of the array holding vector x must be at least $(1 + (\text{len} - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.
beta	Scaling factor for vector y . If $\text{beta} = 0$, vector y does not need to be initialized before calling <code>gemv</code> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
y	Pointer to input/output vector y . The length <code>len</code> of vector y is m , if A is not transposed, and n if A is transposed. Size of the array holding input/output vector y must be at least $(1 + (\text{len} - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y . Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y	Pointer to updated vector y .
---	---------------------------------

Return Values

Output event to wait on to ensure computation is complete.

ger

Computes a rank-1 update of a general matrix.

Description

The `ger` routines compute a scalar-vector-vector product and add the result to a general matrix. The operation is defined as:

$$A \leftarrow \text{alpha} * x * y^T + A$$

where:

- `alpha` is scalar
- `A` is $m \times n$ matrix
- `x` is a vector length m
- `y` is a vector length n

`ger` supports the following precisions:

T
float
double

ger (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void ger(sycl::queue &queue,
             std::int64_t m,
             std::int64_t n,
             T alpha,
             sycl::buffer<T,1> &x,
             std::int64_t incx,
             sycl::buffer<T,1> &y,
             std::int64_t incy,
             sycl::buffer<T,1> &a,
             std::int64_t lda)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void ger(sycl::queue &queue,
             std::int64_t m,
             std::int64_t n,
             T alpha,
             sycl::buffer<T,1> &x,
             std::int64_t incx,
             sycl::buffer<T,1> &y,
             std::int64_t incy,
             sycl::buffer<T,1> &a,
             std::int64_t lda)
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>m</code>	Number of rows of matrix <code>A</code> . Must be at least zero.
<code>n</code>	Number of columns of matrix <code>A</code> . Must be at least zero.
<code>alpha</code>	Scaling factor for the matrix-vector product.
<code>x</code>	Buffer holding input vector <code>x</code> . Size of the buffer must be at least $(1 + (m - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
<code>incx</code>	Stride of vector <code>x</code> . Must not be zero.

y	Buffer holding input/output vector <i>y</i> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> . Must not be zero.
a	Buffer holding input matrix <i>A</i> . Size of the buffer must be at least $\text{lda} * n$ if column major layout is used, or at least $\text{lda} * m$ if row major layout is used.
lda	Leading dimension of matrix <i>A</i> . Must be positive and at least <i>m</i> if column major layout is used or at least <i>n</i> if row major layout is used.

Output Parameters

a	Buffer holding updated matrix <i>A</i> .
---	--

ger (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event ger(sycl::queue &queue,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *x,
        std::int64_t incx,
        const T *y,
        std::int64_t incy,
        T *a,
        std::int64_t lda,
        const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event ger(sycl::queue &queue,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *x,
        std::int64_t incx,
        const T *y,
        std::int64_t incy,
        T *a,
        std::int64_t lda,
        const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
m	Number of rows of matrix <i>A</i> . Must be at least zero.
n	Number of columns of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.

<code>x</code>	Pointer to input vector <code>x</code> . Size of the array holding input vector <code>x</code> must be at least $(1 + (m - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
<code>incx</code>	Stride of vector <code>x</code> . Must not be zero.
<code>y</code>	Pointer to input/output vector <code>y</code> . Size of the array holding input/output vector <code>y</code> must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
<code>incy</code>	Stride of vector <code>y</code> . Must not be zero.
<code>a</code>	Pointer to input matrix <code>A</code> . Size of the array must be at least <code>lda * n</code> if column major layout is used, or at least <code>lda * m</code> if row major layout is used.
<code>lda</code>	Leading dimension of matrix <code>A</code> . Must be positive and at least <code>m</code> if column major layout is used or at least <code>n</code> if row major layout is used.
<code>dependencies</code>	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

<code>a</code>	Pointer to updated matrix <code>A</code> .
----------------	--

Return Values

Output event to wait on to ensure computation is complete.

gerc

Computes a rank-1 update (conjugated) of a general complex matrix.

Description

The `gerc` routines compute a scalar-vector-vector product and add the result to a general matrix. The operation is defined as:

$$A \leftarrow \alpha * x * y^H + A$$

where:

- `alpha` is a scalar
- `A` is `m x n` matrix
- `x` is a vector of length `m`
- `y` is vector of length `n`

`gerc` supports the following precisions:

T
<code>std::complex<float></code>
<code>std::complex<double></code>

gerc (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void gerc(sycl::queue &queue,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &a,
              std::int64_t lda)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void gerc(sycl::queue &queue,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &a,
              std::int64_t lda)
}
```

Input Parameters

queue	The queue where the routine should be executed.
m	Number of rows of matrix A. Must be at least zero.
n	Number of columns of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
x	Buffer holding input vector x. Size of the buffer must be at least $(1 + (m - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.
y	Buffer holding input/output vector y. Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y. Must not be zero.
a	Buffer holding input matrix A. Size of the buffer must be at least $\text{lda} * n$ if column major layout is used, or at least $\text{lda} * m$ if row major layout is used. See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be positive and at least m if column major layout is used or at least n if row major layout is used.

Output Parameters

a	Buffer holding updated matrix A.
---	----------------------------------

gerc (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event gerc(sycl::queue &queue,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *a,
                    std::int64_t lda,
                    const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event gerc(sycl::queue &queue,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *a,
                    std::int64_t lda,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
m	Number of rows of matrix A. Must be at least zero.
n	Number of columns of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
x	Pointer to input vector x. Size of the array holding input vector x must be at least $(1 + (m - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.
y	Pointer to input/output vector y. Size of the array holding input/output vector y must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y. Must not be zero.
a	Pointer to input matrix A. Size of the array must be at least $\text{lda} * n$ if column major layout is used, or at least $\text{lda} * m$ if row major layout is used.
lda	Leading dimension of matrix A. Must be positive and at least m if column major layout is used or at least n if row major layout is used.

dependencies List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

a Pointer to updated matrix A .

Return Values

Output event to wait on to ensure computation is complete.

geru

Computes a rank-1 update (unconjugated) of a general complex matrix.

Description

The `geru` routines compute a scalar-vector-vector product and add the result to a general matrix. The operation is defined as:

$$A \leftarrow \alpha * x * y^T + A$$

where:

- `alpha` is a scalar
- `A` is $m \times n$ matrix
- `x` is a vector of length m
- `y` is a vector of length n

`geru` supports the following precisions:

T
<code>std::complex<float></code>
<code>std::complex<double></code>

geru (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void geru(sycl::queue &queue,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
```

```

        sycl::buffer<T,1> &a,
        std::int64_t lda)
}

namespace oneapi::mkl::blas::row_major {
    void geru(sycl::queue &queue,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &a,
              std::int64_t lda)
}

```

Input Parameters

queue	The queue where the routine should be executed.
m	Number of rows of matrix A. Must be at least zero.
n	Number of columns of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
x	Buffer holding input vector x . Size of the buffer must be at least $(1 + (m - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.
y	Buffer holding input/output vector y . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y . Must not be zero.
a	Buffer holding input matrix A. Size of the buffer must be at least $\text{lda} * n$ if column major layout is used, or at least $\text{lda} * m$ if row major layout is used.
lda	Leading dimension of matrix A. Must be positive and at least m if column major layout is used or at least n if row major layout is used.

Output Parameters

a	Buffer holding updated matrix A.
---	----------------------------------

geru (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event geru(sycl::queue &queue,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,

```

```

        const T *y,
        std::int64_t incy,
        T *a,
        std::int64_t lda,
        const std::vector<sycl::event> &dependencies = {})
    }

namespace oneapi::mkl::blas::row_major {
    sycl::event geru(sycl::queue &queue,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *x,
        std::int64_t incx,
        const T *y,
        std::int64_t incy,
        T *a,
        std::int64_t lda,
        const std::vector<sycl::event> &dependencies = {})
    }

```

Input Parameters

queue	The queue where the routine should be executed.
m	Number of rows of matrix A. Must be at least zero.
n	Number of columns of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
x	Pointer to input vector x. Size of the array holding input vector x must be at least $(1 + (m - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.
y	Pointer to input/output vector y. Size of the array holding input/output vector y must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y. Must not be zero.
a	Pointer to input matrix A. Size of the array must be at least $\text{lda} * n$ if column major layout is used, or at least $\text{lda} * m$ if row major layout is used.
lda	Leading dimension of matrix A. Must be positive and at least m if column major layout is used or at least n if row major layout is used.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

a	Pointer to updated matrix A.
---	------------------------------

Return Values

Output event to wait on to ensure computation is complete.

hbmrv

Computes a matrix-vector product using a hermitian band matrix.

Description

The `hbmrv` routines compute a scalar-matrix-vector product and add the result to a scalar-vector product, with a hermitian band matrix. The operation is defined as:

$$y \leftarrow \alpha * A * x + \beta * y$$

where:

- `alpha` and `beta` are scalars
- `A` is $n \times n$ hermitian band matrix, with `k` super-diagonals
- `x` and `y` are vectors of length `n`

`hbmrv` supports the following precisions:

T
<code>std::complex<float></code>
<code>std::complex<double></code>

hbmrv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void hbmrv(sycl::queue &queue,
               oneapi::mkl::uplo upper_lower,
               std::int64_t n,
               std::int64_t k,
               T alpha,
               sycl::buffer<T,1> &a,
               std::int64_t lda,
               sycl::buffer<T,1> &x,
               std::int64_t incx,
               T beta,
               sycl::buffer<T,1> &y,
               std::int64_t incy)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void hbmrv(sycl::queue &queue,
               oneapi::mkl::uplo upper_lower,
               std::int64_t n,
               std::int64_t k,
               T alpha,
               sycl::buffer<T,1> &a,
               std::int64_t lda,
               sycl::buffer<T,1> &x,
               std::int64_t incx,
               T beta,
```



```

    sycl::buffer<T,1> &y,
    std::int64_t incy)
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
k	Number of super-diagonals of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
a	Buffer holding input matrix <i>A</i> . Size of the buffer must be at least <i>lda</i> * <i>n</i> . See Matrix Storage for more details.
lda	Leading dimension of matrix <i>A</i> . Must be at least (<i>k</i> + 1) and positive.
x	Buffer holding input vector <i>x</i> . Size of the buffer must be at least (1 + (<i>n</i> - 1)*abs(<i>incx</i>)). See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
beta	Scaling factor for vector <i>y</i> . If <i>beta</i> = 0, vector <i>y</i> does not need to be initialized before calling <i>hbm</i> v.
y	Buffer holding input/output vector <i>y</i> . Size of the buffer must be at least (1 + (<i>n</i> - 1)*abs(<i>incy</i>)). See Matrix Storage for more details.
incy	Stride of vector <i>y</i> . Must not be zero.

Output Parameters

y	Buffer holding updated vector <i>y</i> .
---	--

hbm (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event hbm(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        std::int64_t n,
        std::int64_t k,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        const T *x,
        std::int64_t incx,
        oneapi::mkl::value_or_pointer<T> beta,
        T *y,

```

```

        std::int64_t incy,
        const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event hbmV(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        std::int64_t n,
        std::int64_t k,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        const T *x,
        std::int64_t incx,
        oneapi::mkl::value_or_pointer<T> beta,
        T *y,
        std::int64_t incy,
        const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix A. Must be at least zero.
k	Number of super-diagonals of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. Size of the array holding input matrix A must be at least $lda * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least $(k + 1)$ and positive.
x	Pointer to input vector x. Size of the array holding input vector x must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.
beta	Scaling factor for vector y. If <code>beta = 0</code> , vector y does not need to be initialized before calling <code>hbmV</code> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
y	Pointer to input/output vector y. Size of the array holding input/output vector y must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y. Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y Pointer to updated vector y .

Return Values

Output event to wait on to ensure computation is complete.

hemv

Computes a matrix-vector product using a hermitian matrix.

Description

The `hemv` routines compute a scalar-matrix-vector product and add the result to a scalar-vector product, with a hermitian matrix. The operation is defined as:

$$y \leftarrow \alpha * A * x + \beta * y$$

where:

- `alpha` and `beta` are scalars
- `A` is $n \times n$ hermitian matrix
- `x` and `y` are vectors of length `n`

`hemv` supports the following precisions:

T
<code>std::complex<float></code>
<code>std::complex<double></code>

hemv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void hemv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              T beta,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void hemv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
```

```

        std::int64_t incx,
        T beta,
        sycl::buffer<T,1> &y,
        std::int64_t incy)
    }

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
a	Buffer holding input matrix <i>A</i> . Size of the buffer must be at least <i>lda</i> * <i>n</i> . See Matrix Storage for more details.
lda	Leading dimension of matrix <i>A</i> . Must be at least <i>n</i> and positive.
x	Buffer holding input vector <i>x</i> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
beta	Scaling factor for vector <i>y</i> . If <i>beta</i> = 0, vector <i>y</i> does not need to be initialized before calling <i>hemv</i> .
y	Buffer holding input/output vector <i>y</i> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> . Must not be zero.

Output Parameters

y	Buffer holding updated vector <i>y</i> .
---	--

hemv (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event hemv(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        const T *x,
        std::int64_t incx,
        oneapi::mkl::value_or_pointer<T> beta,
        T *y,

```

```

        std::int64_t incy,
        const std::vector<sycl::event> &dependencies = {})
    }

namespace oneapi::mkl::blas::row_major {
    sycl::event hemv(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        const T *x,
        std::int64_t incx,
        oneapi::mkl::value_or_pointer<T> beta,
        T *y,
        std::int64_t incy,
        const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix <i>A</i> . Size of the array holding input matrix <i>A</i> must be at least <code>lda * n</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix <i>A</i> . Must be at least <i>n</i> and positive.
x	Pointer to input vector <i>x</i> . Size of the array holding input vector <i>x</i> must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
beta	Scaling factor for vector <i>y</i> . If <code>beta = 0</code> , vector <i>y</i> does not need to be initialized before calling <code>hemv</code> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
y	Pointer to input/output vector <i>y</i> . Size of the array holding input/output vector <i>y</i> must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> . Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y	Pointer to updated vector <i>y</i> .
---	--------------------------------------

Return Values

Output event to wait on to ensure computation is complete.

her

Computes a rank-1 update of a hermitian matrix.

Description

The `her` routines compute a scalar-vector-vector product and add the result to a hermitian matrix. The operation is defined as:

$$A \leftarrow \alpha * x * x^H + A$$

where:

- `alpha` is scalar
- `A` is $n \times n$ hermitian matrix
- `x` is a vector of length n

`her` supports the following precisions:

T	Treal
<code>std::complex<float></code>	<code>float</code>
<code>std::complex<double></code>	<code>double</code>

her (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void her(sycl::queue &queue,
             oneapi::mkl::uplo upper_lower,
             std::int64_t n,
             Treal alpha,
             sycl::buffer<T,1> &x,
             std::int64_t incx,
             sycl::buffer<T,1> &a,
             std::int64_t lda)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void her(sycl::queue &queue,
             oneapi::mkl::uplo upper_lower,
             std::int64_t n,
             Treal alpha,
             sycl::buffer<T,1> &x,
             std::int64_t incx,
             sycl::buffer<T,1> &a,
             std::int64_t lda)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
x	Buffer holding input vector <i>x</i> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
a	Buffer holding input matrix <i>A</i> . Size of the buffer must be at least $\text{lda} * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix <i>A</i> . Must be at least <i>n</i> and positive.

Output Parameters

a	Buffer holding updated upper triangular part of the hermitian matrix <i>A</i> if <code>upper_lower=upper</code> or updated lower triangular part of the hermitian matrix <i>A</i> if <code>upper_lower=lower</code>. If <code>alpha</code> is zero, <i>A</i> matrix is unchanged, otherwise imaginary parts of the diagonal elements are set to zero.
---	---

her (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event her(sycl::queue &queue,
                   oneapi::mkl::uplo upper_lower,
                   std::int64_t n,
                   oneapi::mkl::value_or_pointer<Treal> alpha,
                   const T *x,
                   std::int64_t incx,
                   T *a,
                   std::int64_t lda,
                   const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event her(sycl::queue &queue,
                   oneapi::mkl::uplo upper_lower,
                   std::int64_t n,
                   oneapi::mkl::value_or_pointer<Treal> alpha,
                   const T *x,
                   std::int64_t incx,
                   T *a,
                   std::int64_t lda,
                   const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
x	Pointer to input vector <i>x</i> . Size of the array holding input vector <i>x</i> must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
a	Pointer to input matrix <i>A</i> . Size of the array holding input matrix <i>A</i> must be at least <code>lda * n</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix <i>A</i> . Must be at least <i>n</i> and positive.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

a	Pointer to updated upper triangular part of the hermitian matrix <i>A</i> if <code>upper_lower=upper</code> or updated lower triangular part of the hermitian matrix <i>A</i> if <code>upper_lower=lower</code>. If <code>alpha</code> is zero, <i>A</i> matrix is unchanged, otherwise imaginary parts of the diagonal elements are set to zero.
---	---

Return Values

Output event to wait on to ensure computation is complete.

her2

Computes a rank-2 update of a hermitian matrix.

Description

The `her2` routines compute two scalar-vector-vector products and add them to a hermitian matrix. The operation is defined as:

$$A \leftarrow \alpha * x * y^H + \text{conjg}(\alpha) * y * x^H + A$$

where:

- `alpha` is a scalar
- *A* is *n* x *n* hermitian matrix
- *x* and *y* are vectors or length *n*

`her2` supports the following precisions:

T

std::complex<float>

std::complex<double>

her2 (Buffer Version)**Syntax**

```
namespace oneapi::mkl::blas::column_major {
    void her2(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &a,
              std::int64_t lda)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void her2(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &a,
              std::int64_t lda)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of columns of matrix A . Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
x	Buffer holding input vector x . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.
y	Buffer holding input/output vector y . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y . Must not be zero.
a	Buffer holding input matrix A . Size of the buffer must be at least $\text{lda} * n$. See Matrix Storage for more details.

lda Leading dimension of matrix A. Must be at least `n` and positive.

Output Parameters

a Buffer holding updated upper triangular part of the hermitian matrix A if `upper_lower=upper`, or updated lower triangular part of the hermitian matrix A if `upper_lower=lower`.

If `alpha` is zero, A matrix is unchanged, otherwise imaginary parts of the diagonal elements are set to zero.

her2 (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event her2(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *a,
                    std::int64_t lda,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event her2(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *a,
                    std::int64_t lda,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue The queue where the routine should be executed.

upper_lower Specifies whether matrix A is upper or lower triangular. See [Data Types](#) for more details.

n Number of columns of matrix A. Must be at least zero.

alpha Scaling factor for the matrix-vector product. See [Scalar Arguments](#) for more information on the `value_or_pointer` data type.

x Pointer to input vector `x`. Size of the array holding input vector `x` must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See [Matrix Storage](#) for more details.

incx	Stride of vector x . Must not be zero.
y	Pointer to input/output vector y . Size of the array holding input/output vector y must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y . Must not be zero.
a	Pointer to input matrix A . Size of the array holding input matrix A must be at least $\text{lda} * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix A . Must be at least n and positive.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

a	Pointer to updated upper triangular part of the hermitian matrix A if <code>upper_lower=upper</code>, or updated lower triangular part of the hermitian matrix A if <code>upper_lower=lower</code>. If <code>alpha</code> is zero, A matrix is unchanged, otherwise imaginary parts of the diagonal elements are set to zero.
---	--

Return Values

Output event to wait on to ensure computation is complete.

hpmv

Computes a matrix-vector product using a hermitian packed matrix.

Description

The `hpmv` routines compute a scalar-matrix-vector product and add the result to a scalar-vector product, with a hermitian packed matrix. The operation is defined as:

$$y \leftarrow \alpha * A * x + \beta * y$$

where:

- `alpha` and `beta` are scalars
- A is $n \times n$ hermitian matrix supplied in packed form
- x and y are vectors of length n

`hpmv` supports the following precisions:

T
<code>std::complex<float></code>
<code>std::complex<double></code>

hpmv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void hpmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              T beta,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void hpmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              T beta,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
a	Buffer holding input matrix <i>A</i> . Size of the buffer must be at least $(n*(n+1))/2$. See Matrix Storage for more details. The imaginary parts of the diagonal elements need not be set and are assumed to be zero.
x	Buffer holding input vector <i>x</i> . Size of the buffer must be at least $(1 + (n - 1)*abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
beta	Scaling factor for vector <i>y</i> . If <i>beta</i> = 0, vector <i>y</i> does not need to be initialized before calling <i>hpmv</i> .
y	Buffer holding input/output vector <i>y</i> . Size of the buffer must be at least $(1 + (n - 1)*abs(incy))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> . Must not be zero.

Output Parameters

y Buffer holding updated vector y.

hpmv (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event hpmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    const T *x,
                    std::int64_t incx,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event hpmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    const T *x,
                    std::int64_t incx,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. Size of the array holding input matrix A must be at least $(n*(n+1))/2$. See Matrix Storage for more details. The imaginary parts of the diagonal elements need not be set and are assumed to be zero.
x	Pointer to input vector x. Size of the array holding input vector x must be at least $(1 + (n - 1)*abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.

beta	Scaling factor for vector y . If $\text{beta} = 0$, vector y does not need to be initialized before calling <code>hpmv</code> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
y	Pointer to input/output vector y . Size of the array holding input/output vector y must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y . Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y	Pointer to updated vector y .
-----	---------------------------------

Return Values

Output event to wait on to ensure computation is complete.

hpr

Computes a rank-1 update of a hermitian packed matrix.

Description

The `hpr` routines compute a scalar-vector-vector product and add the result to a hermitian packed matrix. The operation is defined as:

$$A \leftarrow \alpha * x * x^H + A$$

where:

- α is scalar
- A is $n \times n$ hermitian matrix, supplied in packed form
- x is a vector of length n

`hpr` supports the following precisions:

T	Treal
<code>std::complex<float></code>	float
<code>std::complex<double></code>	double

hpr (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void hpr(sycl::queue &queue,
             oneapi::mkl::uplo upper_lower,
             std::int64_t n,
             Treal alpha,
             sycl::buffer<T,1> &x,
```

```

        std::int64_t incx,
        sycl::buffer<T,1> &a)
    }

namespace oneapi::mkl::blas::row_major {
    void hpr(sycl::queue &queue,
             oneapi::mkl::uplo upper_lower,
             std::int64_t n,
             Treal alpha,
             sycl::buffer<T,1> &x,
             std::int64_t incx,
             sycl::buffer<T,1> &a)
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix A . Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
x	Buffer holding input vector x . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.
a	Buffer holding input matrix A . Size of the buffer must be at least $(n * (n - 1)) / 2$. See Matrix Storage for more details. The imaginary parts of the diagonal elements do not need to be set and are assumed to be zero.

Output Parameters

a	Buffer holding updated upper triangular part of the hermitian matrix A if <code>upper_lower=upper</code> , or updated lower triangular part of the hermitian matrix A if <code>upper_lower=lower</code> . If <code>alpha</code> is zero, A matrix is unchanged, otherwise imaginary parts of the diagonal elements are set to zero.
---	---

hpr (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event hpr(sycl::queue &queue,
                   oneapi::mkl::uplo upper_lower,
                   std::int64_t n,
                   oneapi::mkl::value_or_pointer<Treal> alpha,
                   const T *x,
                   std::int64_t incx,

```

```

        T *a,
        const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event hpr(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<Treal> alpha,
        const T *x,
        std::int64_t incx,
        T *a,
        const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
x	Pointer to input vector x. Size of the array holding input vector x must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.
a	Pointer to input matrix A. Size of the array holding input matrix A must be at least $(n * (n - 1)) / 2$. See Matrix Storage for more details. The imaginary parts of the diagonal elements do not need to be set and are assumed to be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

a	Pointer to updated upper triangular part of the hermitian matrix A if <code>upper_lower=upper</code>, or updated lower triangular part of the hermitian matrix A if <code>upper_lower=lower</code>. If alpha is zero, A matrix is unchanged, otherwise imaginary parts of the diagonal elements are set to zero.
---	--

Return Values

Output event to wait on to ensure computation is complete.

hpr2

Performs a rank-2 update of a hermitian packed matrix.

Description

The `hpr2` routines compute two scalar-vector-vector products and add them to a hermitian packed matrix. The operation is defined as:

$$A \leftarrow \alpha * x * y^H + \text{conjg}(\alpha) * y * x^H + A$$

where:

- `alpha` is a scalar
- `A` is $n \times n$ hermitian matrix, supplied in packed form
- `x` and `y` are vectors of length `n`

`hpr2` supports the following precisions:

T
<code>std::complex<float></code>
<code>std::complex<double></code>

hpr2 (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void hpr2(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &a)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void hpr2(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &a)
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>upper_lower</code>	Specifies whether matrix <code>A</code> is upper or lower triangular. See Data Types for more details.
<code>n</code>	Number of rows and columns of matrix <code>A</code> . Must be at least zero.
<code>alpha</code>	Scaling factor for the matrix-vector product.

x	Buffer holding input vector x . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.
y	Buffer holding input/output vector y . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y . Must not be zero.
a	Buffer holding input matrix A . Size of the buffer must be at least $(n * (n - 1)) / 2$. See Matrix Storage for more details. The imaginary parts of the diagonal elements need not be set and are assumed to be zero.

Output Parameters

a	Buffer holding updated upper triangular part of the hermitian matrix A if <code>upper_lower=upper</code> , or updated lower triangular part of the hermitian matrix A if <code>upper_lower=lower</code> . If α is zero, A matrix is unchanged, otherwise imaginary parts of the diagonal elements are set to zero.
---	--

hpr2 (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event hpr2(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *a,
                    const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event hpr2(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *a,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
-------	---

upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
x	Pointer to input vector <i>x</i> . Size of the array holding input vector <i>x</i> must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
y	Pointer to input/output vector <i>y</i> . Size of the array holding input/output vector <i>y</i> must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> . Must not be zero.
a	Pointer to input matrix <i>A</i>. Size of the array holding input matrix <i>A</i> must be at least $(n * (n - 1)) / 2$. See Matrix Storage for more details. The imaginary parts of the diagonal elements need not be set and are assumed to be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

a	Pointer to updated upper triangular part of the hermitian matrix <i>A</i> if <code>upper_lower=upper</code>, or updated lower triangular part of the hermitian matrix <i>A</i> if <code>upper_lower=lower</code>. If <code>alpha</code> is zero, <i>A</i> matrix is unchanged, otherwise imaginary parts of the diagonal elements are set to zero.
---	--

Return Values

Output event to wait on to ensure computation is complete.

sbmv

Computes a matrix-vector product with a symmetric band matrix.

Description

The `sbmv` routines compute a scalar-matrix-vector product and add the result to a scalar-vector product, with a symmetric band matrix. The operation is defined as:

$$y \leftarrow \alpha * A * x + \beta * y$$

where:

- `alpha` and `beta` are scalars
- *A* is *n* × *n* symmetric matrix with *k* super-diagonals
- *x* and *y* are vectors of length *n*

sbmv supports the following precisions:

T
float
double

sbmv (Buffer Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    void sbmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              std::int64_t k,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              T beta,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}

namespace oneapi::mkl::blas::row_major {
    void sbmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              std::int64_t k,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              T beta,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix A. Must be at least zero.
k	Number of super-diagonals of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
a	Buffer holding input matrix A. Size of the buffer must be at least lda * n. See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least (k + 1) and positive.

x	Buffer holding input vector x . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.
beta	Scaling factor for vector y . If $\text{beta} = 0$, vector y does not need to be initialized before calling <code>sbmv</code> .
y	Buffer holding input/output vector y . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y . Must not be zero.

Output Parameters

y	Buffer holding updated vector y .
---	-------------------------------------

sbmv (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event sbmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    std::int64_t k,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *x,
                    std::int64_t incx,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event sbmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    std::int64_t k,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *x,
                    std::int64_t incx,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
-------	---

upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
k	Number of super-diagonals of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix <i>A</i> . Size of the array holding input matrix <i>A</i> must be at least <code>lda * n</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix <i>A</i> . Must be at least <code>(k + 1)</code> and positive.
x	Pointer to input vector <i>x</i> . Size of the array holding input vector <i>x</i> must be at least <code>(1 + (n - 1)*abs(incx))</code> . See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
beta	Scaling factor for vector <i>y</i> . If <code>beta = 0</code> , vector <i>y</i> does not need to be initialized before calling <code>sbmv</code> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
y	Pointer to input/output vector <i>y</i> . Size of the array holding input/output vector <i>y</i> must be at least <code>(1 + (n - 1)*abs(incy))</code> . See Matrix Storage for more details.
incy	Stride of vector <i>y</i> . Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y	Pointer to updated vector <i>y</i> .
---	--------------------------------------

Return Values

Output event to wait on to ensure computation is complete.

spmv

Computes a matrix-vector product with a symmetric packed matrix.

Description

The `spmv` routines compute a scalar-matrix-vector product and add the result to a scalar-vector product, with a symmetric packed matrix. The operation is defined as:

$$y \leftarrow \alpha * A * x + \beta * y$$

where:

- `alpha` and `beta` are scalars
- *A* is *n* x *n* symmetric matrix, supplied in packed form
- *x* and *y* are vectors of length *n*

`spmv` supports the following precisions:

T
float
double

spmv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void spmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              T beta,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void spmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              T beta,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
a	Buffer holding input matrix <i>A</i> . Size of the buffer must be at least $(n*(n+1))/2$. See Matrix Storage for more details.
x	Buffer holding input vector <i>x</i> . Size of the buffer must be at least $(1 + (n - 1)*abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
beta	Scaling factor for vector <i>y</i> . If <i>beta</i> = 0, vector <i>y</i> does not need to be initialized before calling <code>spmv</code> .

y	Buffer holding input/output vector <i>y</i> . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> . Must not be zero.

Output Parameters

y	Buffer holding updated vector <i>y</i> .
---	--

spmv (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event spmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    const T *x,
                    std::int64_t incx,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event spmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    const T *x,
                    std::int64_t incx,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix <i>A</i> . Size of the array holding input matrix <i>A</i> must be at least $(n * (n + 1)) / 2$. See Matrix Storage for more details.
x	Pointer to input vector <i>x</i> . Size of the array holding input vector <i>x</i> must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.

incx	Stride of vector x . Must not be zero.
beta	Scaling factor for vector y . If <code>beta = 0</code> , vector y does not need to be initialized before calling <code>spmv</code> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
y	Pointer to input/output vector y . Size of the array holding input/output vector y must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y . Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y	Pointer to updated vector y .
---	---------------------------------

Return Values

Output event to wait on to ensure computation is complete.

spr

Performs a rank-1 update of a symmetric packed matrix.

Description

The `spr` routines compute a scalar-vector-vector product and add the result to a symmetric packed matrix. The operation is defined as;

$$A \leftarrow \alpha * x * x^T + A$$

where:

- `alpha` is scalar
- `A` is $n \times n$ symmetric matrix, supplied in packed form
- `x` is a vector of length `n`

`spr` supports the following precisions:

T
float
double

spr (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void spr(sycl::queue &queue,
             oneapi::mkl::uplo upper_lower,
             std::int64_t n,
```

```

        T alpha,
        sycl::buffer<T,1> &x,
        std::int64_t incx,
        sycl::buffer<T,1> &a)
}

namespace oneapi::mkl::blas::row_major {
    void spr(sycl::queue &queue,
             oneapi::mkl::uplo upper_lower,
             std::int64_t n,
             T alpha,
             sycl::buffer<T,1> &x,
             std::int64_t incx,
             sycl::buffer<T,1> &a)
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
x	Buffer holding input vector x. Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.
a	Buffer holding input matrix A. Size of the buffer must be at least $(n * (n - 1)) / 2$. See Matrix Storage for more details.

Output Parameters

a	Buffer holding updated upper triangular part of the symmetric matrix A if upper_lower=upper, or updated lower triangular part of the symmetric matrix A if upper_lower=lower.
---	---

spr (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event spr(sycl::queue &queue,
                   oneapi::mkl::uplo upper_lower,
                   std::int64_t n,
                   oneapi::mkl::value_or_pointer<T> alpha,
                   const T *x,
                   std::int64_t incx,

```

```

        T *a,
        const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event spr(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *x,
        std::int64_t incx,
        T *a,
        const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
x	Pointer to input vector x. Size of the array holding input vector x must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.
a	Pointer to input matrix A. Size of the array holding input matrix A must be at least $(n * (n - 1)) / 2$. See Matrix Storage for more details.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

a	Pointer to updated upper triangular part of the symmetric matrix A if <code>upper_lower=upper</code> , or updated lower triangular part of the symmetric matrix A if <code>upper_lower=lower</code> .
---	---

Return Values

Output event to wait on to ensure computation is complete.

spr2

Computes a rank-2 update of a symmetric packed matrix.

Description

The `spr2` routines compute two scalar-vector-vector products and add them to a symmetric packed matrix. The operation is defined as:

$$A \leftarrow \alpha * x * y^T + \alpha * y * x^T + A$$

where:

- `alpha` is scalar
- `A` is `n` x `n` symmetric matrix, supplied in packed form
- `x` and `y` are vectors of length `n`

`spr2` supports the following precisions:

T
float
double

spr2 (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void spr2(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &a)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void spr2(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &a)
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>upper_lower</code>	Specifies whether matrix <code>A</code> is upper or lower triangular. See Data Types for more details.
<code>n</code>	Number of rows and columns of matrix <code>A</code> . Must be at least zero.
<code>alpha</code>	Scaling factor for the matrix-vector product.
<code>x</code>	Buffer holding input vector <code>x</code> . Size of the buffer must be at least <code>(1 + (n - 1)*abs(incx))</code> . See Matrix Storage for more details.

incx	Stride of vector x . Must not be zero.
y	Buffer holding input/output vector y . Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y . Must not be zero.
a	Buffer holding input matrix A . Size of the buffer must be at least $(n * (n - 1)) / 2$. See Matrix Storage for more details.

Output Parameters

a	Buffer holding updated upper triangular part of the symmetric matrix A if <code>upper_lower=upper</code> or updated lower triangular part of the symmetric matrix A if <code>upper_lower=lower</code> .
---	---

spr2 (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event spr2(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *a,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event spr2(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *a,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix A . Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.

<code>x</code>	Pointer to input vector <code>x</code> . Size of the array holding input vector <code>x</code> must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
<code>incx</code>	Stride of vector <code>x</code> . Must not be zero.
<code>y</code>	Pointer to input/output vector <code>y</code> . Size of the array holding input/output vector <code>y</code> must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
<code>incy</code>	Stride of vector <code>y</code> . Must not be zero.
<code>a</code>	Pointer to input matrix <code>A</code> . Size of the array holding input matrix <code>A</code> must be at least $(n * (n - 1)) / 2$. See Matrix Storage for more details.
<code>dependencies</code>	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

<code>a</code>	Pointer to updated upper triangular part of the symmetric matrix <code>A</code> if <code>upper_lower=upper</code> or updated lower triangular part of the symmetric matrix <code>A</code> if <code>upper_lower=lower</code> .
----------------	---

Return Values

Output event to wait on to ensure computation is complete.

symv

Computes a matrix-vector product for a symmetric matrix.

Description

The `symv` routines compute a scalar-matrix-vector product and add the result to a scalar-vector product, with a symmetric matrix. The operation is defined as:

$$y \leftarrow \alpha * A * x + \beta * y$$

where:

- `alpha` and `beta` are scalars
- `A` is $n \times n$ symmetric matrix
- `x` and `y` are vectors of length `n`

`symv` supports the following precisions:

T
<code>float</code>
<code>double</code>
<code>std::complex<float></code>
<code>std::complex<double></code>

symv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void symv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              T beta,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void symv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              T beta,
              sycl::buffer<T,1> &y,
              std::int64_t incy)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
a	Buffer holding input matrix <i>A</i> . Size of the buffer must be at least $lda * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix <i>A</i> . Must be at least <i>n</i> and positive.
x	Buffer holding input vector <i>x</i> . Size of the buffer must be at least $(1 + (n - 1) * abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
beta	Scaling factor for vector <i>y</i> . If <i>beta</i> = 0, vector <i>y</i> does not need to be initialized before calling <i>symv</i> .
y	Buffer holding input/output vector <i>y</i> . Size of the buffer must be at least $(1 + (n - 1) * abs(incy))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> . Must not be zero.

Output Parameters

y Buffer holding updated vector *y*.

symv (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event symv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *x,
                    std::int64_t incx,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event symv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *x,
                    std::int64_t incx,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *y,
                    std::int64_t incy,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue The queue where the routine should be executed.

upper_lower Specifies whether matrix *A* is upper or lower triangular. See [Data Types](#) for more details.

n Number of rows and columns of matrix *A*. Must be at least zero.

alpha Scaling factor for the matrix-vector product. See [Scalar Arguments](#) for more information on the `value_or_pointer` data type.

a Pointer to input matrix *A*. Size of the array holding input matrix *A* must be at least `lda * n`. See [Matrix Storage](#) for more details.

lda Leading dimension of matrix *A*. Must be at least *n* and positive.

x Pointer to input vector *x*. Size of the array holding input vector *x* must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See [Matrix Storage](#) for more details.

<code>incx</code>	Stride of vector <code>x</code> . Must not be zero.
<code>beta</code>	Scaling factor for vector <code>y</code> . If <code>beta = 0</code> , vector <code>y</code> does not need to be initialized before calling <code>symv</code> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
<code>y</code>	Pointer to input/output vector <code>y</code> . Size of the array holding input/output vector <code>y</code> must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
<code>incy</code>	Stride of vector <code>y</code> . Must not be zero.
<code>dependencies</code>	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

<code>y</code>	Pointer to updated vector <code>y</code> .
----------------	--

Return Values

Output event to wait on to ensure computation is complete.

`syr`

Computes a rank-1 update of a symmetric matrix.

Description

The `syr` routines compute a scalar-vector-vector product and add the result to a matrix, with a symmetric matrix. The operation is defined as:

$$A \leftarrow \text{alpha} * x * x^T + A$$

where:

- `alpha` is scalar
- `A` is $n \times n$ symmetric matrix
- `x` is a vector of length `n`

`syr` supports the following precisions:

T
<code>float</code>
<code>double</code>
<code>std::complex<float></code>
<code>std::complex<double></code>

`syr` (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void syr(sycl::queue &queue,
             oneapi::mkl::uplo upper_lower,
             std::int64_t n,
             T alpha,
             sycl::buffer<T,1> &x,
             std::int64_t incx,
             sycl::buffer<T,1> &a,
             std::int64_t lda)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void syr(sycl::queue &queue,
             oneapi::mkl::uplo upper_lower,
             std::int64_t n,
             T alpha,
             sycl::buffer<T,1> &x,
             std::int64_t incx,
             sycl::buffer<T,1> &a,
             std::int64_t lda)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of columns of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
x	Buffer holding input vector x. Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.
a	Buffer holding input matrix A. Size of the buffer must be at least $\text{lda} * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least n and positive.

Output Parameters

a	Buffer holding updated upper triangular part of the symmetric matrix A if <code>upper_lower=upper</code> or updated lower triangular part of the symmetric matrix A if <code>upper_lower=lower</code> .
---	---

syr (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event syr(sycl::queue &queue,
                   oneapi::mkl::uplo upper_lower,
                   std::int64_t n,
```

```

        oneapi::mkl::value_or_pointer<T> alpha,
        const T *x,
        std::int64_t incx,
        T *a,
        std::int64_t lda,
        const std::vector<sycl::event> &dependencies = {})
    }

namespace oneapi::mkl::blas::row_major {
    sycl::event syr(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *x,
        std::int64_t incx,
        T *a,
        std::int64_t lda,
        const std::vector<sycl::event> &dependencies = {})
    }

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of columns of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
x	Pointer to input vector x. Size of the array holding input vector x must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.
a	Pointer to input matrix A. Size of the array holding input matrix A must be at least <code>lda * n</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least n and positive.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

a	Pointer to updated upper triangular part of the symmetric matrix A if <code>upper_lower=upper</code> or updated lower triangular part of the symmetric matrix A if <code>upper_lower=lower</code> .
---	---

Return Values

Output event to wait on to ensure computation is complete.

syr2

Computes a rank-2 update of a symmetric matrix.

Description

The `syr2` routines compute two scalar-vector-vector products, add them and add the result to a matrix, with a symmetric matrix. The operation is defined as:

$$A \leftarrow \alpha * x * y^T + \alpha * y * x^T + A$$

where:

- `alpha` is a scalar
- `A` is $n \times n$ symmetric matrix
- `x` and `y` are vectors of length `n`

`syr2` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

syr2 (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void syr2(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &a,
              std::int64_t lda)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void syr2(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &x,
              std::int64_t incx,
              sycl::buffer<T,1> &y,
              std::int64_t incy,
              sycl::buffer<T,1> &a,
              std::int64_t lda)
}
```

Input Parameters

`queue` The queue where the routine should be executed.

upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
n	Number of columns of matrix A. Must be at least zero.
alpha	Scaling factor for the matrix-vector product.
x	Buffer holding input vector x. Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.
y	Buffer holding input/output vector y. Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector y. Must not be zero.
a	Buffer holding input matrix A. Size of the buffer must be at least $\text{lda} * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least n and positive.
Output Parameters	
a	Buffer holding updated upper triangular part of the symmetric matrix A if upper_lower=upper, or updated lower triangular part of the symmetric matrix A if upper_lower=lower.

syr2 (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event syr2(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *a,
                    std::int64_t lda,
                    const std::vector<syck::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event syr2(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *x,
                    std::int64_t incx,
                    const T *y,
                    std::int64_t incy,
                    T *a,
```

```

        std::int64_t lda,
        const std::vector<sycl::event> &dependencies = {})
    }

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
n	Number of columns of matrix <i>A</i> . Must be at least zero.
alpha	Scaling factor for the matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
x	Pointer to input vector <i>x</i> . Size of the array holding input vector <i>x</i> must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
y	Pointer to input/output vector <i>y</i> . Size of the array holding input/output vector <i>y</i> must be at least $(1 + (n - 1) * \text{abs}(\text{incy}))$. See Matrix Storage for more details.
incy	Stride of vector <i>y</i> . Must not be zero.
a	Pointer to input matrix <i>A</i> . Size of the array holding input matrix <i>A</i> must be at least <code>lda * n</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix <i>A</i> . Must be at least <i>n</i> and positive.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

a	Pointer to updated upper triangular part of the symmetric matrix <i>A</i> if <code>upper_lower=upper</code> , or updated lower triangular part of the symmetric matrix <i>A</i> if <code>upper_lower=lower</code> .
---	---

Return Values

Output event to wait on to ensure computation is complete.

tbmv

Computes a matrix-vector product using a triangular band matrix.

Description

The `tbmv` routines compute a matrix-vector product with a triangular band matrix. The operation is defined as:

$$x \leftarrow \text{op}(A) * x$$

where:

- $\text{op}(A)$ is one of $\text{op}(A) = A$, or $\text{op}(A) = A^T$, or $\text{op}(A) = A^H$
- A is $n \times n$ unit or non-unit, upper or lower triangular band matrix, with $(k + 1)$ diagonals
- x is a vector of length n

`tbmv` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

tbmv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void tbmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              std::int64_t k,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void tbmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              std::int64_t k,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

Input Parameters

queue The queue where the routine should be executed.

upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details..
trans	Specifies $op(A)$, the transposition operation applied to matrix A . See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
n	Numbers of rows and columns of matrix A . Must be at least zero.
k	Number of sub/super-diagonals of matrix A . Must be at least zero.
a	Buffer holding input matrix A . Size of the buffer must be at least $lda * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix A . Must be at least $(k + 1)$ and positive.
x	Buffer holding input vector x . Size of the buffer must be at least $(1 + (n - 1) * abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.

Output Parameters

x	Buffer holding updated vector x .
---	-------------------------------------

tbmv (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event tbmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    std::int64_t k,
                    const T *a,
                    std::int64_t lda,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event tbmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    std::int64_t k,
                    const T *a,
                    std::int64_t lda,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}
```


Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies $op(A)$, the transposition operation applied to matrix A . See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
n	Numbers of rows and columns of matrix A . Must be at least zero.
k	Number of sub/super-diagonals of matrix A . Must be at least zero.
a	Pointer to input matrix A . Size of the array holding input matrix A must be at least $lda * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix A . Must be at least $(k + 1)$ and positive.
x	Pointer to input vector x . Size of the array holding input vector x must be at least $(1 + (n - 1) * abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

x	Pointer to updated vector x .
---	---------------------------------

Return Values

Output event to wait on to ensure computation is complete.

tbsv

Solves a system of linear equations whose coefficients are in a triangular band matrix.

Description

The `tbsv` routines solve a system of linear equations whose coefficients are in a triangular band matrix. The operation is defined as:

$$op(A) * x = b$$

where:

- $op(A)$ is one of $op(A) = A$, or $op(A) = A^T$, or $op(A) = A^H$
- A is $n \times n$ unit or non-unit, upper or lower triangular band matrix, with $(k + 1)$ diagonals

- `b` and `x` are vectors of length `n`

`tbsv` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

tbsv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void tbsv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              std::int64_t k,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void tbsv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              std::int64_t k,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <code>A</code> is upper or lower triangular. See Data Types for more details.
trans	Specifies <code>op(A)</code> , the transposition operation applied to matrix <code>A</code> . See Data Types for more details.
unit_diag	Specifies whether matrix <code>A</code> is unit triangular or not. See Data Types for more details.
n	Number of rows and columns of matrix <code>A</code> . Must be at least zero.

k	Number of sub/super-diagonals of matrix A. Must be at least zero.
a	Buffer holding input matrix A. Size of the buffer must be at least $\text{lda} * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least $(k + 1)$ and positive.
x	Buffer holding input vector x. Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.

Output Parameters

x	Buffer holding solution vector x.
---	-----------------------------------

tbsv (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event tbsv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    std::int64_t k,
                    const T *a,
                    std::int64_t lda,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event tbsv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    std::int64_t k,
                    const T *a,
                    std::int64_t lda,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A. See Data Types for more details.

unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
n	Number of rows and columns of matrix A . Must be at least zero.
k	Number of sub/super-diagonals of matrix A . Must be at least zero.
a	Pointer to input matrix A . Size of the array holding input matrix A must be at least $lda * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix A . Must be at least $(k + 1)$ and positive.
x	Pointer to input vector x . Size of the array holding input vector x must be at least $(1 + (n - 1) * abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

x	Pointer to solution vector x .
---	----------------------------------

Return Values

Output event to wait on to ensure computation is complete.

tpmv

Computes a matrix-vector product using a triangular packed matrix.

Description

The `tpmv` routines compute a matrix-vector product with a triangular packed matrix. The operation is defined as:

$$x \leftarrow op(A) * x$$

where:

- $op(A)$ is one of $op(A) = A$, or $op(A) = A^T$, or $op(A) = A^H$
- A is $n \times n$ unit or non-unit, upper or lower triangular band matrix, supplied in packed form
- x is a vector of length n

`tpmv` supports the following precisions:

T
float
double

T

std::complex<float>

std::complex<double>

tpmv (Buffer Version)**Syntax**

```
namespace oneapi::mkl::blas::column_major {
    void tpmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              sycl::buffer<T,1> &a,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void tpmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              sycl::buffer<T,1> &a,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies op(A), the transposition operation applied to matrix A. See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
n	Number of rows and columns of matrix A. Must be at least zero.
a	Buffer holding input matrix A. Size of the buffer must be at least $(n*(n+1))/2$. See Matrix Storage for more details.
x	Buffer holding input vector x. Size of the buffer must be at least $(1 + (n - 1)*abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.

Output Parameters

x	Buffer holding updated vector x.
---	----------------------------------

tpmv (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event tpmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    const T *a,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event tpmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    const T *a,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
trans	Specifies $op(A)$, the transposition operation applied to matrix <i>A</i> . See Data Types for more details.
unit_diag	Specifies whether matrix <i>A</i> is unit triangular or not. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
a	Pointer to input matrix <i>A</i> . Size of the array holding input matrix <i>A</i> must be at least $(n*(n+1))/2$. See Matrix Storage for more details.
x	Pointer to input vector <i>x</i> . Size of the array holding input vector <i>x</i> must be at least $(1 + (n - 1)*abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

x	Pointer to updated vector <i>x</i> .
---	--------------------------------------

Return Values

Output event to wait on to ensure computation is complete.

tpsv

Solves a system of linear equations whose coefficients are in a triangular packed matrix.

Description

The `tpsv` routines solve a system of linear equations whose coefficients are in a triangular packed matrix. The operation is defined as:

$$\text{op}(A) * x = b$$

where:

- $\text{op}(A)$ is one of $\text{op}(A) = A$, or $\text{op}(A) = A^T$, or $\text{op}(A) = A^H$
- A is $n \times n$ unit or non-unit, upper or lower triangular band matrix, supplied in packed form
- b and x are vectors of length n

`tpsv` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

tpsv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void tpsv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              std::int64_t k,
              sycl::buffer<T,1> &a,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void tpsv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              std::int64_t k,
```

```

    sycl::buffer<T,1> &a,
    sycl::buffer<T,1> &x,
    std::int64_t incx)
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A. See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
n	Number of rows and columns of matrix A. Must be at least zero.
a	Buffer holding input matrix A. Size of the buffer must be at least $(n*(n+1))/2$. See Matrix Storage for more details.
x	Buffer holding the n-element right-hand side vector b. Size of the buffer must be at least $(1 + (n - 1)*\text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.

Output Parameters

x	Buffer holding solution vector x.
---	-----------------------------------

tpsv (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event tpsv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    std::int64_t k,
                    const T *a,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}

```

```

namespace oneapi::mkl::blas::row_major {
    sycl::event tpsv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    std::int64_t k,
                    const T *a,
                    T *x,

```



```
std::int64_t incx,
const std::vector<sycl::event> &dependencies = {}))
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies $op(A)$, the transposition operation applied to matrix A . See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
n	Number of rows and columns of matrix A . Must be at least zero.
a	Pointer to input matrix A . Size of the array holding input matrix A must be at least $(n*(n+1))/2$. See Matrix Storage for more details.
x	Pointer to the n -element right-hand side vector b . Size of the array holding the n -element right-hand side vector b must be at least $(1 + (n - 1)*abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

x	Pointer to solution vector x .
---	----------------------------------

Return Values

Output event to wait on to ensure computation is complete.

trmv

Computes a matrix-vector product using a triangular matrix.

Description

The `trmv` routines compute a matrix-vector product with a triangular matrix. The operation is defined as:

$$x \leftarrow op(A) * x$$

where:

- $op(A)$ is one of $op(A) = A$, or $op(A) = A^T$, or $op(A) = A^H$
- A is $n \times n$ unit or non-unit, upper or lower triangular band matrix

- x is a vector of length n

`trmv` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

trmv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void trmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void trmv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies $op(A)$, the transposition operation applied to matrix A . See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
n	Number of rows and columns of matrix A . Must be at least zero.
a	Buffer holding input matrix A . Size of the buffer must be at least $lda * n$. See Matrix Storage for more details.

lda	Leading dimension of matrix A. Must be at least n and positive.
x	Buffer holding input vector x. Size of the buffer must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.

Output Parameters

x	Buffer holding updated vector x.
---	----------------------------------

trmv (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event trmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    const T *a,
                    std::int64_t lda,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event trmv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    const T *a,
                    std::int64_t lda,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies op(A), the transposition operation applied to matrix A. See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
n	Number of rows and columns of matrix A. Must be at least zero.
a	Pointer to input matrix A. Size of the array holding input matrix A must be at least $\text{lda} * n$. See Matrix Storage for more details.

lda	Leading dimension of matrix A . Must be at least n and positive.
x	Pointer to input vector x . Size of the array holding input vector x must be at least $(1 + (n - 1) * \text{abs}(\text{incx}))$. See Matrix Storage for more details.
incx	Stride of vector x . Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

x	Pointer to updated vector x .
---	---------------------------------

Return Values

Output event to wait on to ensure computation is complete.

trsv

Solves a system of linear equations whose coefficients are in a triangular matrix.

Description

The `trsv` routines solve a system of linear equations whose coefficients are in a triangular matrix. The operation is defined as:

$$\text{op}(A) * x = b$$

where:

- $\text{op}(A)$ is one of $\text{op}(A) = A$, or $\text{op}(A) = A^T$, or $\text{op}(A) = A^H$
- A is $n \times n$ unit or non-unit, upper or lower triangular matrix
- b and x are vectors of length n

`trsv` supports the following precisions:

T
float
double
<code>std::complex<float></code>
<code>std::complex<double></code>

trsv (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void trsv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void trsv(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t n,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &x,
              std::int64_t incx)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>A</i> is upper or lower triangular. See Data Types for more details.
trans	Specifies $op(A)$, the transposition operation applied to matrix <i>A</i> . See Data Types for more details.
unit_diag	Specifies whether matrix <i>A</i> is unit triangular or not. See Data Types for more details.
n	Number of rows and columns of matrix <i>A</i> . Must be at least zero.
a	Buffer holding input matrix <i>A</i> . Size of the buffer must be at least $lda * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix <i>A</i> . Must be at least <i>n</i> and positive.
x	Buffer holding the <i>n</i> -element right-hand side vector <i>b</i> . Size of the buffer must be at least $(1 + (n - 1) * abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector <i>x</i> . Must not be zero.

Output Parameters

x	Buffer holding solution vector <i>x</i> .
---	---

trsv (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event trsv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    const T *a,
                    std::int64_t lda,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event trsv(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t n,
                    const T *a,
                    std::int64_t lda,
                    T *x,
                    std::int64_t incx,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies $op(A)$, the transposition operation applied to matrix A. See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
n	Number of rows and columns of matrix A. Must be at least zero.
a	Pointer to input matrix A. Size of the array holding input matrix A must be at least $lda * n$. See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least n and positive.
x	Pointer to the n-element right-hand side vector b. Size of the array holding the n-element right-hand side vector b must be at least $(1 + (n - 1) * abs(incx))$. See Matrix Storage for more details.
incx	Stride of vector x. Must not be zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

x Pointer to solution vector x.

Return Values

Output event to wait on to ensure computation is complete.

BLAS Level 3 Routines

BLAS Level 3 routines perform matrix-matrix operations. The following table lists the BLAS Level 3 routine groups and the data types associated with them.

Routine Group	Data Types	Description
gemm	std::int8_t, oneapi::mkl::bfloat16, sycl::half, float, double, std::complex<float>, std::complex<double>, mixed	Computes a matrix-matrix product with general matrices.
hemm	std::complex<float>, std::complex<double>	Computes a matrix-matrix product where one input matrix is Hermitian and one is general.
herk	std::complex<float>, std::complex<double>	Performs a Hermitian rank-k update.
her2k	std::complex<float>, std::complex<double>	Performs a Hermitian rank-2k update.
symm	float, double, std::complex<float>, std::complex<double>	Computes a matrix-matrix product where one input matrix is symmetric and one matrix is general.
syrk	float, double, std::complex<float>, std::complex<double>	Performs a symmetric rank-k update.
syr2k	float, double, std::complex<float>, std::complex<double>	Performs a symmetric rank-2k update.
trmm	float, double, std::complex<float>, std::complex<double>	Computes a matrix-matrix product where one input matrix is triangular and one input matrix is general.
trsm	float, double, std::complex<float>, std::complex<double>	Solves a triangular matrix equation (forward or backward solve).

The BLAS functions are blocked where possible to restructure the code in a way that increases the localization of data reference, enhances cache memory use, and reduces the dependency on the memory bus.

The code is distributed across the processors to maximize parallelism.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

gemm

Computes a matrix-matrix product with general matrices.

Description

The `gemm` routines compute a scalar-matrix-matrix product and add the result to a scalar-matrix product, with general matrices. The operation is defined as:

$$C \leftarrow \alpha * op(A) * op(B) + \beta * C$$

where:

- `op(x)` is one of `op(x) = x`, or `op(x) = xT`, or `op(x) = xH`
- `alpha` and `beta` are scalars
- `A`, `B` and `C` are matrices
- `op(A)` is `m x k` matrix
- `op(B)` is `k x n` matrix
- `C` is `m x n` matrix

`gemm` supports the following precisions:

Ta (A matrix)	Tb (B matrix)	Tc (C matrix)	Ts (alpha/beta)
<code>sycl::half</code>	<code>sycl::half</code>	<code>sycl::half</code>	<code>sycl::half</code>
<code>sycl::half</code>	<code>sycl::half</code>	<code>float</code>	<code>float</code>
<code>oneapi::mkl::bfloat16</code>	<code>oneapi::mkl::bfloat16</code>	<code>oneapi::mkl::bfloat16</code>	<code>float</code>
<code>oneapi::mkl::bfloat16</code>	<code>oneapi::mkl::bfloat16</code>	<code>float</code>	<code>float</code>
<code>std::int8_t</code>	<code>std::int8_t</code>	<code>std::int32_t</code>	<code>float</code>
<code>std::int8_t</code>	<code>std::int8_t</code>	<code>float</code>	<code>float</code>
<code>float</code>	<code>float</code>	<code>float</code>	<code>float</code>
<code>double</code>	<code>double</code>	<code>double</code>	<code>double</code>
<code>std::complex<float></code>	<code>std::complex<float></code>	<code>std::complex<float></code>	<code>std::complex<float></code>
<code>std::complex<double></code>	<code>std::complex<double></code>	<code>std::complex<double></code>	<code>std::complex<double></code>

gemm (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void gemm(sycl::queue &queue,
              oneapi::mkl::transpose transa,
              oneapi::mkl::transpose transb,
              std::int64_t m,
              std::int64_t n,
              std::int64_t k,
              Ts alpha,
              sycl::buffer<Ta,1> &a,
              std::int64_t lda,
              sycl::buffer<Tb,1> &b,
              std::int64_t ldb,
              Ts beta,
              sycl::buffer<Tc,1> &c,
              std::int64_t ldc,
              compute_mode mode = compute_mode::unset)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void gemm(sycl::queue &queue,
              oneapi::mkl::transpose transa,
              oneapi::mkl::transpose transb,
              std::int64_t m,
              std::int64_t n,
              std::int64_t k,
              Ts alpha,
              sycl::buffer<Ta,1> &a,
              std::int64_t lda,
              sycl::buffer<Tb,1> &b,
              std::int64_t ldb,
              Ts beta,
              sycl::buffer<Tc,1> &c,
              std::int64_t ldc,
              compute_mode mode = compute_mode::unset)
}
```

Input Parameters

queue	The queue where the routine should be executed.
transa	Specifies $\text{op}(A)$, the transposition operation applied to matrix A . See Data Types for more details.
transb	Specifies $\text{op}(B)$, the transposition operation applied to matrix B . See Data Types for more details.
m	Number of rows of matrix $\text{op}(A)$ and matrix C . Must be at least zero.
n	Number of columns of matrix $\text{op}(B)$ and matrix C . Must be at least zero.
k	Number of columns of matrix $\text{op}(A)$ and rows of matrix $\text{op}(B)$. Must be at least zero.
alpha	Scaling factor for matrix-matrix product.
a	Buffer holding input matrix A . See Matrix Storage for more details.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	A is $m \times k$ matrix. Size of array a must be at least $lda * k$	A is $k \times m$ matrix. Size of array a must be at least $lda * m$
Row major	A is $m \times k$ matrix. Size of array a must be at least $lda * m$	A is $k \times m$ matrix. Size of array a must be at least $lda * k$

lda

Leading dimension of matrix A. Must be positive.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least m	Must be at least k
Row major	Must be at least k	Must be at least m

b

Buffer holding input matrix B. See [Matrix Storage](#) for more details.

	transb = transpose::nontrans	transb = transpose::trans Or trans = transpose::conjtrans
Column major	B is $k \times n$ matrix. Size of array b must be at least $ldb * n$	B is $n \times k$ matrix. Size of array b must be at least $ldb * k$
Row major	B is $k \times n$ matrix. Size of array b must be at least $ldb * k$	B is $n \times k$ matrix. Size of array b must be at least $ldb * n$

ldb

Leading dimension of matrix B. Must be positive.

	transb = transpose::nontrans	transb = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least k	Must be at least n
Row major	Must be at least n	Must be at least k

beta Scaling factor for matrix **C**.

c Buffer holding input/output matrix **C**. See [Matrix Storage](#) for more details.

Column major	C is $m \times n$ matrix. Size of array c must be at least $ldc * n$
Row major	C is $m \times n$ matrix. Size of array c must be at least $ldc * m$

ldc Leading dimension of matrix **C**. Must be positive.

Column major	Must be at least m
Row major	Must be at least n

mode *Optional.* Compute mode settings. See [Compute Modes](#) for more details.

Output Parameters

c Output buffer overwritten by $\alpha * \text{op}(A) * \text{op}(B) + \beta * C$.

NOTE If $\beta = 0$, matrix **C** does not need to be initialized before calling `gemm`.

Examples

An example of how to use buffer version of `gemm` can be found in oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/blas/source/gemm.cpp
```

gemm (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event gemm(sycl::queue &queue,
        oneapi::mkl::transpose transa,
        oneapi::mkl::transpose transb,
        std::int64_t m,
        std::int64_t n,
        std::int64_t k,
        oneapi::mkl::value_or_pointer<Ts> alpha,
        const Ta *a,
        std::int64_t lda,
        const Tb *b,
        std::int64_t ldb,
        oneapi::mkl::value_or_pointer<Ts> beta,
        Tc *c,
        std::int64_t ldc,
```

```

        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event gemm(sycl::queue &queue,
        oneapi::mkl::transpose transa,
        oneapi::mkl::transpose transb,
        std::int64_t m,
        std::int64_t n,
        std::int64_t k,
        oneapi::mkl::value_or_pointer<Ts> alpha,
        const Ta *a,
        std::int64_t lda,
        const Tb *b,
        std::int64_t ldb,
        oneapi::mkl::value_or_pointer<Ts> beta,
        Tc *c,
        std::int64_t ldc,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
transa	Specifies $\text{op}(A)$, the transposition operation applied to matrix A. See Data Types for more details.
transb	Specifies $\text{op}(B)$, the transposition operation applied to matrix B. See Data Types for more details.
m	Number of rows of matrix $\text{op}(A)$ and matrix C. Must be at least zero.
n	Number of columns of matrix $\text{op}(B)$ and matrix C. Must be at least zero.
k	Number of columns of matrix $\text{op}(A)$ and rows of matrix $\text{op}(B)$. Must be at least zero.
alpha	Scaling factor for matrix-matrix product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. See Matrix Storage for more details.

	A not transposed	A transposed
Column major	A is $m \times k$ matrix. Size of array a must be at least $\text{lda} * k$	A is $k \times m$ matrix. Size of array a must be at least $\text{lda} * m$
Row major	A is $m \times k$ matrix. Size of array a must be at least $\text{lda} * m$	A is $k \times m$ matrix. Size of array a must be at least $\text{lda} * k$

lda	Leading dimension of matrix A. Must be positive.
-----	--

	A not transposed	A transposed
Column major	Must be at least m	Must be at least k
Row major	Must be at least k	Must be at least m

b Pointer to input matrix B. See [Matrix Storage](#) for more details.

	B not transposed	B transposed
Column major	B is $k \times n$ matrix. Size of array b must be at least $ldb * n$	B is $n \times k$ matrix. Size of array b must be at least $ldb * k$
Row major	B is $k \times n$ matrix. Size of array b must be at least $ldb * k$	B is $n \times k$ matrix. Size of array b must be at least $ldb * n$

ldb Leading dimension of matrix B. Must be positive.

	B not transposed	B transposed
Column major	Must be at least k	Must be at least n
Row major	Must be at least n	Must be at least k

beta Scaling factor for matrix C. See [Scalar Arguments](#) for more information on the `value_or_pointer` data type.

c Pointer to input/output matrix C. See [Matrix Storage](#) for more details.

Column major	C is $m \times n$ matrix. Size of array c must be at least $ldc * n$
Row major	C is $m \times n$ matrix. Size of array c must be at least $ldc * m$

ldc Leading dimension of matrix C. Must be positive.

Column major	Must be at least m
Row major	Must be at least n

mode *Optional.* Compute mode settings. See [Compute Modes](#) for more details.

dependencies *Optional.* List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

`mode` and `dependencies` may be omitted independently; it is not necessary to specify `mode` in order to provide dependencies.

Output Parameters

c Pointer to output matrix overwritten by $\alpha * \text{op}(A) * \text{op}(B) + \text{beta} * C$.

NOTE If `beta = 0`, matrix `C` does not need to be initialized before calling `gemm`.

Return Values

Output event to wait on to ensure computation is complete.

Examples

An example of how to use USM version of `gemm` can be found in oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/blas/source/gemm_usm.cpp
```

hemm

Computes a matrix-matrix product where one input matrix is hermitian and one is general.

Description

The `hemm` routines compute a scalar-matrix-matrix product and add the result to a scalar-matrix product, where one of the matrices in the multiplication is hermitian. The argument `left_right` determines if the hermitian matrix, `A`, is on the left of the multiplication (`left_right = side::left`) or on the right (`left_right = side::right`). The operation is defined as:

If (`left_right = side::left`),

$$C \leftarrow \alpha * A * B + \beta * C$$

If (`left_right = side::right`),

$$C \leftarrow \alpha * B * A + \beta * C$$

where:

- `alpha` and `beta` are scalars
- `A` is either `m x m` or `n x n` hermitian matrix
- `B` and `C` are `m x n` matrices

`hemm` supports the following precisions:

T
<code>std::complex<float></code>
<code>std::complex<double></code>

hemm (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void hemm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              std::int64_t m,
              std::int64_t n,
```

```

    T alpha,
    sycl::buffer<T,1> &a,
    std::int64_t lda,
    sycl::buffer<T,1> &b,
    std::int64_t ldb,
    T beta,
    sycl::buffer<T,1> &c,
    std::int64_t ldc,
    compute_mode mode = compute_mode::unset)
}

namespace oneapi::mkl::blas::row_major {
    void hemm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &b,
              std::int64_t ldb,
              T beta,
              sycl::buffer<T,1> &c,
              std::int64_t ldc,
              compute_mode mode = compute_mode::unset)
}

```

Input Parameters

queue	The queue where the routine should be executed.
left_right	Specifies whether matrix A is on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
m	Number of rows of matrix B and matrix C. Must be at least zero.
n	Number of columns of matrix B and matrix C. Must be at least zero.
alpha	Scaling factor for matrix-matrix product.
a	Buffer holding input matrix A. Size of the buffer must be at least $lda * m$ if <code>left_right = side::left</code> or $lda * n$ if <code>left_right = side::right</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least m if <code>left_right = side::left</code> or at least n if <code>left_right = side::right</code> . Must be positive.
b	Buffer holding input matrix B. Size of the buffer must be at least $ldb * n$ if column major layout or at least $ldb * m$ if row major layout is used. See Matrix Storage for more details.
ldb	Leading dimension of matrix B. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
beta	Scaling factor for matrix C.

<code>c</code>	Buffer holding input/output matrix <code>C</code> . Size of the buffer must be at least <code>ldc * n</code> if column major layout or at least <code>ldc * m</code> if row major layout is used. See Matrix Storage for more details.
<code>ldc</code>	Leading dimension of matrix <code>C</code> . Must be at least <code>m</code> if column major layout or at least <code>n</code> if row major layout is used. Must be positive.
<code>mode</code>	<i>Optional</i> . Compute mode settings. See Compute Modes for more details.

Output Parameters

<code>c</code>	Output buffer overwritten by $\alpha * A * B + \beta * C$ if <code>left_right = side::left</code> or $\alpha * B * A + \beta * C$ if <code>left_right = side::right</code> .
----------------	--

NOTE If `beta = 0`, matrix `C` does not need to be initialized before calling `hemm`.

hemm (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event hemm(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *b,
                    std::int64_t ldb,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *c,
                    std::int64_t ldc,
                    compute_mode mode = compute_mode::unset,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event hemm(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *b,
                    std::int64_t ldb,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *c,
                    std::int64_t ldc,
                    compute_mode mode = compute_mode::unset,
                    const std::vector<sycl::event> &dependencies = {})
}
```


Input Parameters

queue	The queue where the routine should be executed.
left_right	Specifies whether matrix A is on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
m	Number of rows of matrix B and matrix C. Must be at least zero.
n	Number of columns of matrix B and matrix C. Must be at least zero.
alpha	Scaling factor for matrix-matrix product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. Size of the array must be at least $\text{lda} * m$ if <code>left_right = side::left</code> or $\text{lda} * n$ if <code>left_right = side::right</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least m if <code>left_right = side::left</code> or at least n if <code>left_right = side::right</code> . Must be positive.
b	Pointer to input matrix B. Size of the array must be at least $\text{ldb} * n$ if column major layout or at least $\text{ldb} * m$ if row major layout is used. See Matrix Storage for more details.
ldb	Leading dimension of matrix B. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
beta	Scaling factor for matrix C. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
c	Pointer to input/output matrix C. Size of the array must be at least $\text{ldc} * n$ if column major layout or at least $\text{ldc} * m$ if row major layout is used. See Matrix Storage for more details.
ldc	Leading dimension of matrix C. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
dependencies	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.

Output Parameters

c	Pointer to output matrix overwritten by $\alpha * A * B + \beta * C$ if <code>left_right = side::left</code> or $\alpha * B * A + \beta * C$ if <code>left_right = side::right</code> .
---	---

NOTE If `beta = 0`, matrix C does not need to be initialized before calling `hemm`.

Return Values

Output event to wait on to ensure computation is complete.

her2k

Performs a hermitian rank-2k update.

Description

The `her2k` routines perform a rank-2k update of an $n \times n$ hermitian matrix C by general matrices A and B . The operation is defined as:

If `trans = transpose::nontrans`,

$$C \leftarrow \alpha * A * B^H + \text{conjg}(\alpha) B * A^H + \text{beta} * C$$

where:

- A is $n \times k$ and B is $k \times n$.

If `trans = transpose::conjtrans`,

$$C \leftarrow \alpha * A^H * B + \text{conjg}(\alpha) * B^H * A + \text{beta} * C$$

where:

- A is $k \times n$ and B is $n \times k$.

In both cases:

- α is a complex scalar and beta is a real scalar
- C is hermitian matrix, A and B are general matrices
- The inner dimension of both matrix multiplications is k

`her2k` supports the following precisions:

T	Treal
<code>std::complex<float></code>	<code>float</code>
<code>std::complex<double></code>	<code>double</code>

her2k (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void her2k(sycl::queue &queue,
               oneapi::mkl::uplo upper_lower,
               oneapi::mkl::transpose trans,
               std::int64_t n,
               std::int64_t k,
               T alpha,
               sycl::buffer<T,1> &a,
               std::int64_t lda,
               sycl::buffer<T,1> &b,
               std::int64_t ldb,
               Treval beta,
               sycl::buffer<T,1> &c,
```

```

        std::int64_t ldc,
        compute_mode mode = compute_mode::unset)
}

namespace oneapi::mkl::blas::row_major {
    void her2k(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose trans,
        std::int64_t n,
        std::int64_t k,
        T alpha,
        sycl::buffer<T,1> &a,
        std::int64_t lda,
        sycl::buffer<T,1> &b,
        std::int64_t ldb,
        Treal beta,
        sycl::buffer<T,1> &c,
        std::int64_t ldc,
        compute_mode mode = compute_mode::unset)
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix C is upper or lower triangular. See Data Types for more details.
trans	Specifies the transposition operation applied as described above. Supported operations are <code>transpose::nontrans</code> and <code>transpose::conjtrans</code> .
n	Number of rows and columns of matrix C. Must be at least zero.
k	Inner dimension of matrix multiplications. Must be at least zero.
alpha	Complex scaling factor for the rank-2k update.
a	Buffer holding input matrix A. See Matrix Storage for more details.

	trans = transpose::nontrans	trans = transpose::conjtrans
Column major	A is $n \times k$ matrix. Size of array a must be at least $lda * k$	A is $k \times n$ matrix. Size of array a must be at least $lda * n$
Row major	A is $n \times k$ matrix. Size of array a must be at least $lda * n$	A is $k \times n$ matrix. Size of array a must be at least $lda * k$

lda	Leading dimension of matrix A. Must be positive.
-----	--

	trans = transpose::nontrans	trans = transpose::conjtrans
Column major	Must be at least n	Must be at least k
Row major	Must be at least k	Must be at least n

b

Buffer holding input matrix B . See [Matrix Storage](#) for more details.

	trans = transpose::nontrans	trans = transpose::conjtrans
Column major	B is $k \times n$ matrix. Size of array b must be at least $ldb * n$	B is $n \times k$ matrix. Size of array b must be at least $ldb * k$
Row major	B is $k \times n$ matrix. Size of array b must be at least $ldb * k$	B is $n \times k$ matrix. Size of array b must be at least $ldb * n$

ldb

Leading dimension of matrix B . Must be positive.

	trans = transpose::nontrans	trans = transpose::conjtrans
Column major	Must be at least k	Must be at least n
Row major	Must be at least n	Must be at least k

beta

Real scaling factor for matrix C .

c

Buffer holding input/output matrix C . Size of the buffer must be at least $ldc * n$. See [Matrix Storage](#) for more details.

ldc

Leading dimension of matrix C . Must be positive and at least n .

mode

Optional. Compute mode settings. See [Compute Modes](#) for more details.

Output Parameters

c

Output buffer overwritten by updated C matrix.

her2k (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event her2k(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose trans,
        std::int64_t n,
        std::int64_t k,
```

```

oneapi::mkl::value_or_pointer<T> alpha,
const T *a,
std::int64_t lda,
const T *b,
std::int64_t ldb,
oneapi::mkl::value_or_pointer<Treal> beta,
T *c,
std::int64_t ldc,
compute_mode mode = compute_mode::unset,
const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event her2k(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose trans,
        std::int64_t n,
        std::int64_t k,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        const T *b,
        std::int64_t ldb,
        oneapi::mkl::value_or_pointer<Treal> beta,
        T *c,
        std::int64_t ldc,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix C is upper or lower triangular. See Data Types for more details.
trans	Specifies the transposition operation applied as described above. Supported operations are <code>transpose::nontrans</code> and <code>transpose::conjtrans</code> .
n	Number of rows and columns of matrix C. Must be at least zero.
k	Inner dimension of matrix multiplications. Must be at least zero.
alpha	Complex scaling factor for the rank-2k update. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. See Matrix Storage for more details.

	trans = transpose::nontrans	trans = transpose::conjtrans
Column major	A is $n \times k$ matrix. Size of array a must be at least $lda * k$	A is $k \times n$ matrix. Size of array a must be at least $lda * n$

	trans = transpose::nontrans	trans = transpose::conjtrans
Row major	A is $n \times k$ matrix. Size of array <i>a</i> must be at least $lda * n$	A is $k \times n$ matrix. Size of array <i>a</i> must be at least $lda * k$

lda

Leading dimension of matrix A. Must be positive.

	trans = transpose::nontrans	trans = transpose::conjtrans
Column major	Must be at least n	Must be at least k
Row major	Must be at least k	Must be at least n

b

Pointer to input matrix B. See [Matrix Storage](#) for more details.

	trans = transpose::nontrans	trans = transpose::conjtrans
Column major	B is $k \times n$ matrix. Size of array <i>b</i> must be at least $ldb * n$	B is $n \times k$ matrix. Size of array <i>b</i> must be at least $ldb * k$
Row major	B is $k \times n$ matrix. Size of array <i>b</i> must be at least $ldb * k$	B is $n \times k$ matrix. Size of array <i>b</i> must be at least $ldb * n$

ldb

Leading dimension of matrix B. Must be positive.

	trans = transpose::nontrans	trans = transpose::conjtrans
Column major	Must be at least k	Must be at least n
Row major	Must be at least n	Must be at least k

beta

Real scaling factor for matrix C. See [Scalar Arguments](#) for more information on the `value_or_pointer` data type.

c

Pointer to input/output matrix C. Size of the array must be at least $ldc * n$. See [Matrix Storage](#) for more details.

ldc

Leading dimension of matrix C. Must be positive and at least n .

mode

Optional. Compute mode settings. See [Compute Modes](#) for more details.

dependencies

Optional. List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

`mode` and `dependencies` may be omitted independently; it is not necessary to specify `mode` in order to provide dependencies.

Output Parameters

`c` Pointer to output matrix overwritten by updated `C` matrix.

Return Values

Output event to wait on to ensure computation is complete.

herk

Performs a hermitian rank-k update.

Description

The `herk` routines compute a rank-k update of a hermitian matrix `C` by a general matrix `A`. The operation is defined as:

$$C \leftarrow \alpha * op(A) * op(A)^H + \beta * C$$

where:

- `op(X)` is one of `op(X) = X` or `op(X) = XH`
- `alpha` and `beta` are real scalars
- `C` is `n x n` hermitian matrix
- `op(A)` is `n x k` general matrix

`herk` supports the following precisions:

T	Treal
<code>std::complex<float></code>	<code>float</code>
<code>std::complex<double></code>	<code>double</code>

herk (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void herk(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              std::int64_t n,
              std::int64_t k,
              Treal alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              Treal beta,
              sycl::buffer<T,1> &c,
```

```

        std::int64_t ldc,
        compute_mode mode = compute_mode::unset)
}

namespace oneapi::mkl::blas::row_major {
    void herk(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              std::int64_t n,
              std::int64_t k,
              Treal alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              Treal beta,
              sycl::buffer<T,1> &c,
              std::int64_t ldc,
              compute_mode mode = compute_mode::unset)
}

```

Input Parameters

- queue

The queue where the routine should be executed.
- upper_lower

Specifies whether matrix C is upper or lower triangular. See [Data Types](#) for more details.
- trans

Specifies $op(A)$, the transposition operation applied to matrix A. Supported operations are `transpose::nontrans` and `transpose::conjtrans`. See [Data Types](#) for more details.
- n

Number of rows and columns of matrix C. Must be at least zero.
- k

Number of columns of matrix $op(A)$. Must be at least zero.
- alpha

Real scaling factor for the rank-k update.
- a

Buffer holding input matrix A. See [Matrix Storage](#) for more details.

	<code>trans = transpose::nontrans</code>	<code>trans = transpose::conjtrans</code>
Column major	A is $n \times k$ matrix. Size of array a must be at least $lda * k$	A is $k \times n$ matrix. Size of array a must be at least $lda * n$
Row major	A is $n \times k$ matrix. Size of array a must be at least $lda * n$	A is $k \times n$ matrix. Size of array a must be at least $lda * k$

- lda

Leading dimension of matrix A. Must be positive.

	<code>trans = transpose::nontrans</code>	<code>trans = transpose::conjtrans</code>
Column major	Must be at least n	Must be at least k

	trans = transpose::nontrans	trans = transpose::conjtrans
Row major	Must be at least k	Must be at least n

beta	Real scaling factor for matrix C.
c	Buffer holding input/output matrix C. Size of the buffer must be at least ldc * n. See Matrix Storage for more details.
ldc	Leading dimension of matrix C. Must be positive and at least n.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.

Output Parameters

c	Output buffer overwritten by $\alpha * \text{op}(A) * \text{op}(A)^H + \beta * C$. The imaginary parts of the diagonal elements are set to zero.
---	---

herk (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event herk(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    std::int64_t n,
                    std::int64_t k,
                    oneapi::mkl::value_or_pointer<Treal> alpha,
                    const T *a,
                    std::int64_t lda,
                    oneapi::mkl::value_or_pointer<Treal> beta,
                    T *c,
                    std::int64_t ldc,
                    compute_mode mode = compute_mode::unset,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event herk(sycl::queue &queue,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    std::int64_t n,
                    std::int64_t k,
                    oneapi::mkl::value_or_pointer<Treal> alpha,
                    const T *a,
                    std::int64_t lda,
                    oneapi::mkl::value_or_pointer<Treal> beta,
                    T *c,
                    std::int64_t ldc,
                    compute_mode mode = compute_mode::unset,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix <i>C</i> is upper or lower triangular. See Data Types for more details.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix <i>A</i> . Supported operations are <code>transpose::nontrans</code> and <code>transpose::conjtrans</code> . See Data Types for more details.
n	Number of rows and columns of matrix <i>C</i> . Must be at least zero.
k	Number of columns of matrix $\text{op}(A)$. Must be at least zero.
alpha	Real scaling factor for the rank- <i>k</i> update. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix <i>A</i> . See Matrix Storage for more details.

	trans = transpose::nontrans	trans = transpose::conjtrans
Column major	<i>A</i> is $n \times k$ matrix. Size of array <i>a</i> must be at least $\text{lda} * k$	<i>A</i> is $k \times n$ matrix. Size of array <i>a</i> must be at least $\text{lda} * n$
Row major	<i>A</i> is $n \times k$ matrix. Size of array <i>a</i> must be at least $\text{lda} * n$	<i>A</i> is $k \times n$ matrix. Size of array <i>a</i> must be at least $\text{lda} * k$

lda Leading dimension of matrix *A*. Must be positive.

	trans = transpose::nontrans	trans = transpose::conjtrans
Column major	Must be at least <i>n</i>	Must be at least <i>k</i>
Row major	Must be at least <i>k</i>	Must be at least <i>n</i>

beta	Real scaling factor for matrix <i>C</i> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
c	Pointer to input/output matrix <i>C</i> . Size of the array must be at least $\text{ldc} * n$. See Matrix Storage for more details.
ldc	Leading dimension of matrix <i>C</i> . Must be positive and at least <i>n</i> .
mode	<i>Optional</i> . Compute mode settings. See Compute Modes for more details.
dependencies	<i>Optional</i> . List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <i>mode</i> and <i>dependencies</i> may be omitted independently; it is not necessary to specify <i>mode</i> in order to provide dependencies.

Output Parameters

c Pointer to output matrix overwritten by $\alpha * \text{op}(A) * \text{op}(A)^H + \beta * C$. The imaginary parts of the diagonal elements are set to zero.

Return Values

Output event to wait on to ensure computation is complete.

symm

Computes a matrix-matrix product where one input matrix is symmetric and one matrix is general.

Description

The `symm` routines compute a scalar-matrix-matrix product and add the result to a scalar-matrix product, where one of the matrices in the multiplication is symmetric. The argument `left_right` determines if the symmetric matrix `A` is on the left of the multiplication (`left_right = side::left`) or on the right (`left_right = side::right`). The operation is defined as:

If (`left_right = side::left`),

$$C \leftarrow \alpha * A * B + \beta * C$$

If (`left_right = side::right`),

$$C \leftarrow \alpha * B * A + \beta * C$$

where:

- `alpha` and `beta` are scalars
- `A` is either `m x m` or `n x n` symmetric matrix
- `B` and `C` are `m x n` matrices

`symm` supports the following precisions:

T
<code>float</code>
<code>double</code>
<code>std::complex<float></code>
<code>std::complex<double></code>

symm (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void symm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
```

```

        std::int64_t lda,
        sycl::buffer<T,1> &b,
        std::int64_t ldb,
        T beta,
        sycl::buffer<T,1> &c,
        std::int64_t ldc,
        compute_mode mode = compute_mode::unset)
}

namespace oneapi::mkl::blas::row_major {
    void symm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &b,
              std::int64_t ldb,
              T beta,
              sycl::buffer<T,1> &c,
              std::int64_t ldc,
              compute_mode mode = compute_mode::unset)
}

```

Input Parameters

queue	The queue where the routine should be executed.
left_right	Specifies whether matrix A is on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
m	Number of rows of matrix B and matrix C. Must be at least zero.
n	Number of columns of matrix B and matrix C. Must be at least zero.
alpha	Scaling factor for matrix-matrix product.
a	Buffer holding input matrix A. Size of the buffer must be at least $lda * m$ if <code>left_right = side::left</code> or $lda * n$ if <code>left_right = side::right</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least m if <code>left_right = side::left</code> or at least n if <code>left_right = side::right</code> . Must be positive.
b	Buffer holding input matrix B. Size of the buffer must be at least $ldb * n$ if column major layout or at least $ldb * m$ if row major layout is used. See Matrix Storage for more details.
ldb	Leading dimension of matrix B. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
beta	Scaling factor for matrix C.

<code>c</code>	Buffer holding input/output matrix <code>C</code> . Size of the buffer must be at least <code>ldc * n</code> if column major layout or at least <code>ldc * m</code> if row major layout is used. See Matrix Storage for more details.
<code>ldc</code>	Leading dimension of matrix <code>C</code> . Must be at least <code>m</code> if column major layout or at least <code>n</code> if row major layout is used. Must be positive.
<code>mode</code>	<i>Optional</i> . Compute mode settings. See Compute Modes for more details.

Output Parameters

<code>c</code>	Output buffer overwritten by $\alpha * A * B + \beta * C$ if <code>left_right = side::left</code> or $\alpha * B * A + \beta * C$ if <code>left_right = side::right</code> .
----------------	--

NOTE If `beta = 0`, matrix `C` does not need to be initialized before calling `symm`.

`symm` (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event symm(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *b,
                    std::int64_t ldb,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *c,
                    std::int64_t ldc,
                    compute_mode mode = compute_mode::unset,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event symm(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    oneapi::mkl::uplo upper_lower,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *b,
                    std::int64_t ldb,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *c,
                    std::int64_t ldc,
                    compute_mode mode = compute_mode::unset,
                    const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
left_right	Specifies whether matrix A is on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
m	Number of rows of matrix B and matrix C. Must be at least zero.
n	Number of columns of matrix B and matrix C. Must be at least zero.
alpha	Scaling factor for matrix-matrix product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. Size of the array must be at least $\text{lda} * m$ if <code>left_right = side::left</code> or $\text{lda} * n$ if <code>left_right = side::right</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least m if <code>left_right = side::left</code> or at least n if <code>left_right = side::right</code> . Must be positive.
b	Pointer to input matrix B. Size of the array must be at least $\text{ldb} * n$ if column major layout or at least $\text{ldb} * m$ if row major layout is used. See Matrix Storage for more details.
ldb	Leading dimension of matrix B. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
beta	Scaling factor for matrix C. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
c	Pointer to input/output matrix C. Size of the array must be at least $\text{ldc} * n$ if column major layout or at least $\text{ldc} * m$ if row major layout is used. See Matrix Storage for more details.
ldc	Leading dimension of matrix C. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
dependencies	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.

Output Parameters

c	Pointer to output matrix overwritten by $\alpha * A * B + \beta * C$ if <code>left_right = side::left</code> or $\alpha * B * A + \beta * C$ if <code>left_right = side::right</code> .
---	---

NOTE If `beta = 0`, matrix C does not need to be initialized before calling `symm`.

Return Values

Output event to wait on to ensure computation is complete.

syr2k

Performs a symmetric rank-2k update.

Description

The `syr2k` routines perform a rank-2k update of an $n \times n$ symmetric matrix C by general matrices A and B . The operation is defined as:

If `trans = transpose::nontrans`,

$$C \leftarrow \alpha * (A * B^T + B * A^T) + \beta * C$$

where A and B are $n \times k$ matrices.

If `trans = transpose::trans`,

$$C \leftarrow \alpha * (A^T * B + B^T * A) + \beta * C$$

where A and B are $k \times n$ matrices.

In both cases:

- `alpha` and `beta` are scalars
- C is symmetric matrix, A and B are general matrices
- The inner dimension of both matrix multiplications is k

`syr2k` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

syr2k (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void syr2k(sycl::queue &queue,
               oneapi::mkl::uplo upper_lower,
               oneapi::mkl::transpose trans,
               std::int64_t n,
               std::int64_t k,
               T alpha,
               sycl::buffer<T,1> &a,
               std::int64_t lda,
               sycl::buffer<T,1> &b,
               std::int64_t ldb,
               T beta,
```

```

        sycl::buffer<T,1> &c,
        std::int64_t ldc,
        compute_mode mode = compute_mode::unset)
}

namespace oneapi::mkl::blas::row_major {
    void syr2k(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose trans,
        std::int64_t n,
        std::int64_t k,
        T alpha,
        sycl::buffer<T,1> &a,
        std::int64_t lda,
        sycl::buffer<T,1> &b,
        std::int64_t ldb,
        T beta,
        sycl::buffer<T,1> &c,
        std::int64_t ldc,
        compute_mode mode = compute_mode::unset)
}

```

Input Parameters

- queue

The queue where the routine should be executed.
- upper_lower

Specifies whether matrix `c` is upper or lower triangular. See [Data Types](#) for more details.
- trans

Specifies the transposition operation applied as described above. Conjugation is never performed even if `trans = transpose::conjtrans`.
- n

Number of rows and columns of matrix `C`. Must be at least zero.
- k

Inner dimension of matrix multiplications. Must be at least zero.
- alpha

Complex scaling factor for the rank-2k update.
- a

Buffer holding input matrix `A`. See [Matrix Storage](#) for more details.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	A is $n \times k$ matrix. Size of array <code>a</code> must be at least <code>lda * k</code>	A is $k \times n$ matrix. Size of array <code>a</code> must be at least <code>lda * n</code>
Row major	A is $n \times k$ matrix. Size of array <code>a</code> must be at least <code>lda * n</code>	A is $k \times n$ matrix. Size of array <code>a</code> must be at least <code>lda * k</code>

- lda

Leading dimension of matrix `A`. Must be positive.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least n	Must be at least k
Row major	Must be at least k	Must be at least n

b Buffer holding input matrix **B**. See [Matrix Storage](#) for more details.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	B is $n \times k$ matrix. Size of array b must be at least $ldb * k$	B is $k \times n$ matrix. Size of array b must be at least $ldb * n$
Row major	B is $n \times k$ matrix. Size of array b must be at least $ldb * n$	B is $k \times n$ matrix. Size of array b must be at least $ldb * k$

ldb Leading dimension of matrix **B**. Must be positive.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least n	Must be at least k
Row major	Must be at least k	Must be at least n

beta Real scaling factor for matrix **C**.

c Buffer holding input/output matrix **C**. Size of the buffer must be at least $ldc * n$. See [Matrix Storage](#) for more details.

ldc Leading dimension of matrix **C**. Must be positive and at least n .

mode *Optional*. Compute mode settings. See [Compute Modes](#) for more details.

Output Parameters

c Output buffer overwritten by updated **C** matrix.

syr2k (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event syr2k(sycl::queue &queue,
                     oneapi::mkl::uplo upper_lower,
                     oneapi::mkl::transpose trans,
                     std::int64_t n,
                     std::int64_t k,
                     oneapi::mkl::value_or_pointer<T> alpha,
                     const T *a,
                     std::int64_t lda,
                     const T *b,
                     std::int64_t ldb,
                     oneapi::mkl::value_or_pointer<T> beta,
                     T *c,
                     std::int64_t ldc,
                     compute_mode mode = compute_mode::unset,
                     const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event syr2k(sycl::queue &queue,
                     oneapi::mkl::uplo upper_lower,
                     oneapi::mkl::transpose trans,
                     std::int64_t n,
                     std::int64_t k,
                     oneapi::mkl::value_or_pointer<T> alpha,
                     const T *a,
                     std::int64_t lda,
                     const T *b,
                     std::int64_t ldb,
                     oneapi::mkl::value_or_pointer<T> beta,
                     T *c,
                     std::int64_t ldc,
                     compute_mode mode = compute_mode::unset,
                     const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix C is upper or lower triangular. See Data Types for more details.
trans	Specifies the transposition operation applied as described above. Conjugation is never performed even if <code>trans = transpose::conjtrans</code> .
n	Number of rows and columns of matrix C. Must be at least zero.
k	Inner dimension of matrix multiplications. Must be at least zero.
alpha	Complex scaling factor for the rank-2k update. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. See Matrix Storage for more details.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	A is $n \times k$ matrix. Size of array <i>a</i> must be at least $lda * k$	A is $k \times n$ matrix. Size of array <i>a</i> must be at least $lda * n$
Row major	A is $n \times k$ matrix. Size of array <i>a</i> must be at least $lda * n$	A is $k \times n$ matrix. Size of array <i>a</i> must be at least $lda * k$

lda

Leading dimension of matrix A. Must be positive.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least n	Must be at least k
Row major	Must be at least k	Must be at least n

b

Pointer to input matrix B. See [Matrix Storage](#) for more details.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	B is $n \times k$ matrix. Size of array <i>b</i> must be at least $ldb * k$	B is $k \times n$ matrix. Size of array <i>b</i> must be at least $ldb * n$
Row major	B is $n \times k$ matrix. Size of array <i>b</i> must be at least $ldb * n$	B is $k \times n$ matrix. Size of array <i>b</i> must be at least $ldb * k$

ldb

Leading dimension of matrix B. Must be positive.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least n	Must be at least k
Row major	Must be at least k	Must be at least n

beta	Real scaling factor for matrix <code>C</code> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
c	Pointer to input/output matrix <code>C</code> . Size of the array must be at least <code>ldc * n</code> . See Matrix Storage for more details.
ldc	Leading dimension of matrix <code>C</code> . Must be positive and at least <code>n</code> .
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
dependencies	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.

Output Parameters

c	Pointer to output matrix overwritten by updated <code>C</code> matrix.
---	--

Return Values

Output event to wait on to ensure computation is complete.

syrk

Performs a symmetric rank-k update.

Description

The `syrk` routines perform a rank-k update of a symmetric matrix `C` by a general matrix `A`. The operation is defined as:

$$C \leftarrow \alpha * op(A) * op(A)^T + \beta * C$$

where:

- `op(x)` is one of `op(x) = x` or `op(x) = xT`
- `alpha` and `beta` are scalars
- `C` is `n x n` symmetric matrix,
- `op(A)` is `n x k` general matrix

`syrk` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

syrk (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void syrk(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              std::int64_t n,
              std::int64_t k,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              T beta,
              sycl::buffer<T,1> &c,
              std::int64_t ldc,
              compute_mode mode = compute_mode::unset)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void syrk(sycl::queue &queue,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              std::int64_t n,
              std::int64_t k,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              T beta,
              sycl::buffer<T,1> &c,
              std::int64_t ldc,
              compute_mode mode = compute_mode::unset)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix C is upper or lower triangular. See Data Types for more details.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A. Conjugation is never performed even if <code>trans = transpose::conjtrans</code> . See Data Types for more details.
n	Number of rows and columns of matrix C. Must be at least zero.
k	Number of columns of matrix $\text{op}(A)$. Must be at least zero.
alpha	Scaling factor for the rank-k update.
a	Buffer holding input matrix A. See Matrix Storage for more details.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	A is $n \times k$ matrix. Size of array <code>a</code> must be at least <code>lda * k</code>	A is $k \times n$ matrix. Size of array <code>a</code> must be at least <code>lda * n</code>
Row major	A is $n \times k$ matrix. Size of array <code>a</code> must be at least <code>lda * n</code>	A is $k \times n$ matrix. Size of array <code>a</code> must be at least <code>lda * k</code>

lda Leading dimension of matrix A. Must be positive.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least <code>n</code>	Must be at least <code>k</code>
Row major	Must be at least <code>k</code>	Must be at least <code>n</code>

beta Scaling factor for matrix C.

c Buffer holding input/output matrix C. Size of the buffer must be at least `ldc * n`. See [Matrix Storage](#) for more details.

ldc Leading dimension of matrix C. Must be positive and at least `n`.

mode *Optional*. Compute mode settings. See [Compute Modes](#) for more details.

Output Parameters

c Output buffer overwritten by $\alpha * \text{op}(A) * \text{op}(A)^T + \text{beta} * C$.

syrk (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event syrk(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose trans,
        std::int64_t n,
        std::int64_t k,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        oneapi::mkl::value_or_pointer<T> beta,
        T *c,
        std::int64_t ldc,
```

```

        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {}))
}

namespace oneapi::mkl::blas::row_major {
    sycl::event syrk(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose trans,
        std::int64_t n,
        std::int64_t k,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        oneapi::mkl::value_or_pointer<T> beta,
        T *c,
        std::int64_t ldc,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {}))
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix C is upper or lower triangular. See Data Types for more details.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A. Conjugation is never performed even if <code>trans = transpose::conjtrans</code> . See Data Types for more details.
n	Number of rows and columns of matrix C. Must be at least zero.
k	Number of columns of matrix $\text{op}(A)$. Must be at least zero.
alpha	Scaling factor for the rank-k update. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. See Matrix Storage for more details.

	trans = transpose::nontrans	trans = transpose::trans or trans = transpose::conjtrans
Column major	A is $n \times k$ matrix. Size of array a must be at least $\text{lda} * k$	A is $k \times n$ matrix. Size of array a must be at least $\text{lda} * n$
Row major	A is $n \times k$ matrix. Size of array a must be at least $\text{lda} * n$	A is $k \times n$ matrix. Size of array a must be at least $\text{lda} * k$

lda	Leading dimension of matrix A. Must be positive.
-----	--

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least n	Must be at least k
Row major	Must be at least k	Must be at least n

beta	Scaling factor for matrix C. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
c	Pointer to input/output matrix C. Size of the array must be at least <code>ldc * n</code> . See Matrix Storage for more details.
ldc	Leading dimension of matrix C. Must be positive and at least n.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
dependencies	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.

Output Parameters

c	Pointer to output matrix, overwritten by $\alpha * \text{op}(A) * \text{op}(A)^T + \text{beta} * C$.
---	---

Return Values

Output event to wait on to ensure computation is complete.

trmm

Computes a matrix-matrix product where one input matrix is triangular and other matrix is general.

Description

The `trmm` routines compute a scalar-matrix-matrix product where one of the matrices in the multiplication is triangular. The argument `left_right` determines if the triangular matrix, A, is on the left of the multiplication (`left_right = side::left`) or on the right (`left_right = side::right`).

There are two operations available, an in-place operation and an out-of-place operation. The in-place operation is defined as:

If (`left_right = side::left`),

$$B \leftarrow \alpha * \text{op}(A) * B$$

If (`left_right = side::right`),

$$B \leftarrow \alpha * B * op(A)$$

The out-of-place operation is defined as:

If (left_right = side::left),

$$C \leftarrow \alpha * op(A) * B + \beta * C$$

If (left_right = side::right),

$$C \leftarrow \alpha * B * op(A) + \beta * C$$

where:

- $op(A)$ is one of $op(A) = A$, or $op(A) = A^T$, or $op(A) = A^H$
- α and β are scalars
- A is either $m \times m$ or $n \times n$ triangular matrix
- B is $m \times n$ general matrix
- C is $m \times n$ general matrix

trmm supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

trmm (Buffer Version)

In-place API

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void trmm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &b,
```

```

        std::int64_t ldb,
        compute_mode mode = compute_mode::unset)
}

namespace oneapi::mkl::blas::row_major {
    void trmm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &b,
              std::int64_t ldb,
              compute_mode mode = compute_mode::unset)
}

```

Input Parameters

queue	The queue where the routine will be executed.
left_right	Specifies whether matrix A is on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies $op(A)$, the transposition operation applied to matrix A. See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
m	Number of rows of matrix B. Must be at least zero.
n	Number of columns of matrix B. Must be at least zero.
alpha	Scaling factor for matrix-matrix product.
a	Buffer holding input matrix A. Size of the buffer must be at least $lda * m$ if <code>left_right = side::left</code> or $lda * n$ if <code>left_right = side::right</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least m if <code>left_right = side::left</code> or at least n if <code>left_right = side::right</code> . Must be positive.
b	Buffer holding input matrix B. Size of the buffer must be at least $ldb * n$ if column major layout or at least $ldb * m$ if row major layout is used. See Matrix Storage for more details.
ldb	Leading dimension of matrix B. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.

Output Parameters

- b Output buffer overwritten by $\alpha * \text{op}(A) * B$ if `left_right = side::left` or $\alpha * B * \text{op}(A)$ if `left_right = side::right`.

NOTE If `alpha = 0`, matrix `B` is set to zero, and `A` and `B` do not need to be initialized at entry.

Out-of-place API

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void trmm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &b,
              std::int64_t ldb,
              T beta,
              sycl::buffer<T,1> &c,
              std::int64_t ldc,
              compute_mode mode = compute_mode::unset)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void trmm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &b,
              std::int64_t ldb,
              T beta,
              sycl::buffer<T,1> &c,
              std::int64_t ldc,
              compute_mode mode = compute_mode::unset)
}
```

Input Parameters

- | | |
|-------------------------|---|
| <code>queue</code> | The queue where the routine will be executed. |
| <code>left_right</code> | Specifies whether matrix <code>A</code> is on the left side or right side of the multiplication. See Data Types for more details. |

upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A. See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
m	Number of rows of matrix B. Must be at least zero.
n	Number of columns of matrix B. Must be at least zero.
alpha	Scaling factor for matrix-matrix product.
a	Buffer holding input matrix A. Size of the buffer must be at least $\text{lda} * m$ if $\text{left_right} = \text{side}::\text{left}$ or $\text{lda} * n$ if $\text{left_right} = \text{side}::\text{right}$. See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least m if $\text{left_right} = \text{side}::\text{left}$ or at least n if $\text{left_right} = \text{side}::\text{right}$. Must be positive.
b	Buffer holding input matrix B. Size of the buffer must be at least $\text{ldb} * n$ if column major layout or at least $\text{ldb} * m$ if row major layout is used. See Matrix Storage for more details.
ldb	Leading dimension of matrix B. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
beta	Scaling factor for matrix C.
c	Buffer holding input/output matrix C. Size of the buffer must be at least $\text{ldc} * n$ if column major layout or at least $\text{ldc} * m$ if row major layout is used. See Matrix Storage for more details.
ldc	Leading dimension of matrix C. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
Output Parameters	
c	Output buffer overwritten by $\alpha * \text{op}(A) * B + \beta * C$ if $\text{left_right} = \text{side}::\text{left}$ or $\alpha * B * \text{op}(A) + \beta * C$ if $\text{left_right} = \text{side}::\text{right}$.

trmm (USM Version)

In-place API

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event trmm(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t m,
```

```

        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        T *b,
        std::int64_t ldb,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {}
    }

namespace oneapi::mkl::blas::row_major {
    sycl::event trmm(sycl::queue &queue,
        oneapi::mkl::side left_right,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose trans,
        oneapi::mkl::diag unit_diag,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        T *b,
        std::int64_t ldb,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {}
    )
}

```

Input Parameters

queue	The queue where the routine will be executed.
left_right	Specifies whether matrix A is on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies op(A), the transposition operation applied to matrix A. See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
m	Number of rows of matrix B. Must be at least zero.
n	Number of columns of matrix B. Must be at least zero.
alpha	Scaling factor for matrix-matrix product. See Scalar Arguments for more information on the value_or_pointer data type.
a	Pointer to input matrix A. Size of the array must be at least $lda * m$ if <code>left_right = side::left</code> or $lda * n$ if <code>left_right = side::right</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least m if <code>left_right = side::left</code> or at least n if <code>left_right = side::right</code> . Must be positive.

<code>b</code>	Pointer to input matrix <code>B</code> . Size of the array must be at least <code>ldb * n</code> if column major layout or at least <code>ldb * m</code> if row major layout is used. See Matrix Storage for more details.
<code>ldb</code>	Leading dimension of matrix <code>B</code> . Must be at least <code>m</code> if column major layout or at least <code>n</code> if row major layout is used. Must be positive.
<code>mode</code>	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
<code>dependencies</code>	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.

Output Parameters

<code>b</code>	Pointer to output matrix overwritten by <code>alpha * op(A) * B</code> if <code>left_right = side::left</code> or <code>alpha * B * op(A)</code> if <code>left_right = side::right</code> .
----------------	---

NOTE If `alpha = 0`, matrix `B` is set to zero, and `A` and `B` do not need to be initialized at entry.

Return Values

Output event to wait on to ensure computation is complete.

Out-of-place API

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event trmm(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *b,
                    std::int64_t ldb,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *c,
                    std::int64_t ldc,
                    compute_mode mode = compute_mode::unset,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event trmm(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
```

```

std::int64_t m,
std::int64_t n,
oneapi::mkl::value_or_pointer<T> alpha,
const T *a,
std::int64_t lda,
const T *b,
std::int64_t ldb,
oneapi::mkl::value_or_pointer<T> beta,
T *c,
std::int64_t ldc,
compute_mode mode = compute_mode::unset,
const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine will be executed.
left_right	Specifies whether matrix A is on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies op(A), the transposition operation applied to matrix A. See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
m	Number of rows of matrix B. Must be at least zero.
n	Number of columns of matrix B. Must be at least zero.
alpha	Scaling factor for matrix-matrix product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. Size of the array must be at least $lda * m$ if <code>left_right = side::left</code> or $lda * n$ if <code>left_right = side::right</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least m if <code>left_right = side::left</code> or at least n if <code>left_right = side::right</code> . Must be positive.
b	Pointer to input matrix B. Size of the array must be at least $ldb * n$ if column major layout or at least $ldb * m$ if row major layout is used. See Matrix Storage for more details.
ldb	Leading dimension of matrix B. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
beta	Scaling factor for matrix C. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
c	Pointer to input/output matrix C. Size of the array must be at least $ldc * n$ if column major layout or at least $ldc * m$ if row major layout is used. See Matrix Storage for more details.

ldc	Leading dimension of matrix <code>C</code> . Must be at least <code>m</code> if column major layout or at least <code>n</code> if row major layout is used. Must be positive.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
dependencies	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.

Output Parameters

<code>c</code>	Pointer to output matrix overwritten by $\alpha * \text{op}(A) * B + \beta * C$ if <code>left_right = side::left</code> or $\alpha * B * \text{op}(A) + \beta * C$ if <code>left_right = side::right</code> .
----------------	---

Return Values

Output event to wait on to ensure computation is complete.

trsm

Solves a triangular matrix equation (forward or backward solve).

Description

The `trsm` routines solve triangular matrix equations where one of the matrices in the multiplication is triangular. The argument `left_right` determines if the triangular matrix, `A`, is on the left of the multiplication (`left_right = side::left`) or on the right (`left_right = side::right`). The operation is defined as:

There are two operations available, an in-place operation and an out-of-place operation. The in-place operation is defined as:

If (`left_right = side::left`),

$$\text{op}(A) * X = \alpha * B$$

If (`left_right = side::right`),

$$X * \text{op}(A) = \alpha * B$$

The out-of-place operation is defined as:

If (`left_right = side::left`),

$$\text{op}(A) * X = \alpha * B, C \leftarrow X + \beta * C$$

If (`left_right = side::right`),

$$X * \text{op}(A) = \alpha * B, C \leftarrow X + \beta * C$$

where:

- $\text{op}(A)$ is one of $\text{op}(A) = A$, or $\text{op}(A) = A^T$, or $\text{op}(A) = A^H$
- α and β are scalars
- A is either $m \times m$ or $n \times n$ triangular matrix
- B , X , and C are $m \times n$ general matrices

For the in-place operation, the matrix B is overwritten by solution matrix X , while for the out-of-place operation, B remains untouched and the solution is added to a scaled C matrix.

`trsm` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

trsm (Buffer Version)

In-place API

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void trsm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose transa,
              oneapi::mkl::diag unit_diag,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &b,
              std::int64_t ldb,
              compute_mode mode = compute_mode::unset)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void trsm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &b,
              std::int64_t ldb,
              compute_mode mode = compute_mode::unset)
}
```

Input Parameters

queue	The queue where the routine will be executed.
left_right	Specifies whether matrix A is on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A . See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
m	Number of rows of matrix B . Must be at least zero.
n	Number of columns of matrix B . Must be at least zero.
alpha	Scaling factor for the solution.
a	Buffer holding input matrix A . Size of the buffer must be at least $\text{lda} * m$ if $\text{left_right} = \text{side}::\text{left}$ or $\text{lda} * n$ if $\text{left_right} = \text{side}::\text{right}$. See Matrix Storage for more details.
lda	Leading dimension of matrix A . Must be at least m if $\text{left_right} = \text{side}::\text{left}$ or at least n if $\text{left_right} = \text{side}::\text{right}$. Must be positive.
b	Buffer holding input matrix B . Size of the buffer must be at least $\text{ldb} * n$ if column major layout or at least $\text{ldb} * m$ if row major layout is used. See Matrix Storage for more details.
ldb	Leading dimension of matrix B . Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.

Output Parameters

b	Output buffer overwritten by solution matrix X .
---	--

NOTE If $\alpha = 0$, matrix B is set to zero, and A and B do not need to be initialized before calling `trsm`.

Out-of-place API

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void trsm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose transa,
              oneapi::mkl::diag unit_diag,
              std::int64_t m,
              std::int64_t n,
```

```

    T alpha,
    sycl::buffer<T,1> &a,
    std::int64_t lda,
    sycl::buffer<T,1> &b,
    std::int64_t ldb,
    T beta,
    sycl::buffer<T,1> &c,
    std::int64_t ldc,
    compute_mode mode = compute_mode::unset)
}

namespace oneapi::mkl::blas::row_major {
    void trsm(sycl::queue &queue,
              oneapi::mkl::side left_right,
              oneapi::mkl::uplo upper_lower,
              oneapi::mkl::transpose trans,
              oneapi::mkl::diag unit_diag,
              std::int64_t m,
              std::int64_t n,
              T alpha,
              sycl::buffer<T,1> &a,
              std::int64_t lda,
              sycl::buffer<T,1> &b,
              std::int64_t ldb,
              T beta,
              sycl::buffer<T,1> &c,
              std::int64_t ldc,
              compute_mode mode = compute_mode::unset)
}

```

Input Parameters

queue	The queue where the routine will be executed.
left_right	Specifies whether matrix A is on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies op(A), the transposition operation applied to matrix A. See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
m	Number of rows of matrix B. Must be at least zero.
n	Number of columns of matrix B. Must be at least zero.
alpha	Scaling factor for the solution.
a	Buffer holding input matrix A. Size of the buffer must be at least $lda * m$ if <code>left_right = side::left</code> or $lda * n$ if <code>left_right = side::right</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least <code>m</code> if <code>left_right = side::left</code> or at least <code>n</code> if <code>left_right = side::right</code> . Must be positive.

<code>b</code>	Buffer holding input matrix <code>B</code> . Size of the buffer must be at least <code>ldb * n</code> if column major layout or at least <code>ldb * m</code> if row major layout is used. See Matrix Storage for more details.
<code>ldb</code>	Leading dimension of matrix <code>B</code> . Must be at least <code>m</code> if column major layout or at least <code>n</code> if row major layout is used. Must be positive.
<code>beta</code>	Scaling factor for matrix <code>C</code> .
<code>c</code>	Buffer holding input/output matrix <code>C</code> . Size of the buffer must be at least <code>ldc * n</code> if column major layout or at least <code>ldc * m</code> if row major layout is used. See Matrix Storage for more details.
<code>ldc</code>	Leading dimension of matrix <code>C</code> . Must be at least <code>m</code> if column major layout or at least <code>n</code> if row major layout is used. Must be positive.
<code>mode</code>	<i>Optional</i> . Compute mode settings. See Compute Modes for more details.
Output Parameters	
<code>c</code>	Output buffer overwritten by solution matrix <code>X + beta * C</code> .

trsm (USM Version)

In-place API

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event trsm(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    T *b,
                    std::int64_t ldb,
                    compute_mode mode = compute_mode::unset,
                    const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event trsm(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    T *b,
                    std::int64_t ldb,
```

```

compute_mode mode = compute_mode::unset,
const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine will be executed.
left_right	Specifies whether matrix A is on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies op(A), the transposition operation applied to matrix A. See Data Types for more details.
unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
m	Number of rows of matrix B. Must be at least zero.
n	Number of columns of matrix B. Must be at least zero.
alpha	Scaling factor for the solution. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. Size of the array must be at least $\text{lda} * m$ if <code>left_right = side::left</code> or $\text{lda} * n$ if <code>left_right = side::right</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least m if <code>left_right = side::left</code> or at least n if <code>left_right = side::right</code> . Must be positive.
b	Pointer to input matrix B. Size of the array must be at least $\text{ldb} * n$ if column major layout or at least $\text{ldb} * m$ if row major layout is used. See Matrix Storage for more details.
ldb	Leading dimension of matrix B. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
dependencies	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.

Output Parameters

b	Pointer to output matrix overwritten by solution matrix X.
---	--

NOTE If `alpha = 0`, matrix B is set to zero, and A and B do not need to be initialized before calling `trsm`.

Return Values

Output event to wait on to ensure computation is complete.

Out-of-place API

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event trsm(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *b,
                    std::int64_t ldb,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *c,
                    std::int64_t ldc,
                    compute_mode mode = compute_mode::unset,
                    const std::vector<sycl::event> &dependencies = {})
}

```

```

namespace oneapi::mkl::blas::row_major {
    sycl::event trsm(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    oneapi::mkl::uplo upper_lower,
                    oneapi::mkl::transpose trans,
                    oneapi::mkl::diag unit_diag,
                    std::int64_t m,
                    std::int64_t n,
                    oneapi::mkl::value_or_pointer<T> alpha,
                    const T *a,
                    std::int64_t lda,
                    const T *b,
                    std::int64_t ldb,
                    oneapi::mkl::value_or_pointer<T> beta,
                    T *c,
                    std::int64_t ldc,
                    compute_mode mode = compute_mode::unset,
                    const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine will be executed.
left_right	Specifies whether matrix A is on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrix A is upper or lower triangular. See Data Types for more details.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A. See Data Types for more details.

unit_diag	Specifies whether matrix A is unit triangular or not. See Data Types for more details.
m	Number of rows of matrix B. Must be at least zero.
n	Number of columns of matrix B. Must be at least zero.
alpha	Scaling factor for the solution. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. Size of the array must be at least $\text{lda} * m$ if <code>left_right = side::left</code> or $\text{lda} * n$ if <code>left_right = side::right</code> . See Matrix Storage for more details.
lda	Leading dimension of matrix A. Must be at least m if <code>left_right = side::left</code> or at least n if <code>left_right = side::right</code> . Must be positive.
b	Pointer to input matrix B. Size of the array must be at least $\text{ldb} * n$ if column major layout or at least $\text{ldb} * m$ if row major layout is used. See Matrix Storage for more details.
ldb	Leading dimension of matrix B. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
beta	Scaling factor for matrix C. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
c	Pointer to input/output matrix C. Size of the array must be at least $\text{ldc} * n$ if column major layout or at least $\text{ldc} * m$ if row major layout is used. See Matrix Storage for more details.
ldc	Leading dimension of matrix C. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
dependencies	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.

Output Parameters

c	Pointer to output matrix overwritten by solution matrix $X + \text{beta} * C$.
---	---

Return Values

Output event to wait on to ensure computation is complete.

BLAS-like Extensions

Intel® oneAPI Math Kernel Library (oneMKL) DPC++ provides additional routines to extend the functionality of the BLAS routines. These include routines to compute many independent vector-vector, vector-matrix, matrix-matrix operations.

The following table lists these routines.

Routine	Data Types	Description
axpy	float, double, std::complex<float>, std::complex<double>	Computes a vector-scalar product added to a scaled-vector.
axpy_batch	float, double, std::complex<float>, std::complex<double>	Computes groups of vector-scalar product added to a vector.
copy_batch	float, double, std::complex<float>, std::complex<double>	Computes groups of vector copies.
dgmm_batch	float, double, std::complex<float>, std::complex<double>	Computes a group of diagonal matrix-matrix product.
gemm_batch	std::int8_t, oneapi::mkl::bfloat16, sycl::half, float, double, std::complex<float>, std::complex<double>, mixed	Computes groups of matrix-matrix product with general matrices.
gemm_bias	mixed std::int8_t, std::uint8_t, and std::int32_t	Computes a matrix-matrix product with general matrices and mixed precision.
gemmt	float, double, std::complex<float>, std::complex<double>	Computes a matrix-matrix product with general matrices, but updates only the upper/lower triangular part of the output matrix.
gemv_batch	float, double, std::complex<float>, std::complex<double>	Computes a group of matrix-vector product using general matrices.
syrk_batch	float, double, std::complex<float>, std::complex<double>	Computes rank-k updates on a group of symmetric matrices by a group of general matrices.
trsm_batch	float, double, std::complex<float>, std::complex<double>	Solves a triangular matrix equation for a group of matrices.
omatcopy	float, double, std::complex<float>, std::complex<double>	Computes an out-of-place matrix copy or transposition.
imatcopy	float, double, std::complex<float>, std::complex<double>	Computes an in-place matrix copy or transposition.
omatadd	float, double, std::complex<float>, std::complex<double>	Computes a sum of two general matrices, with optional transposes.

Routine	Data Types	Description
omatcopy_batch	float, double, std::complex<float>, std::complex<double>	Computes groups of out-of-place matrix copies or transpositions.
imatcopy_batch	float, double, std::complex<float>, std::complex<double>	Computes groups of in-place matrix copies or transpositions.
omatadd_batch	float, double, std::complex<float>, std::complex<double>	Computes groups of matrix additions.

axpby

Computes a vector-scalar product added to a scaled-vector.

Description

The `axpby` routines compute two scalar-vector products and add them:

$$y \leftarrow beta * y + alpha * x$$

where:

- `x` and `y` are vectors of `n` elements
- `alpha` and `beta` are scalars

`axpby` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

axpby (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void axpby(sycl::queue &queue,
               std::int64_t n,
               T alpha,
               sycl::buffer<T,1> &x,
               std::int64_t incx,
               T beta,
```

```

        sycl::buffer<T,1> &y,
        std::int64_t incy);
    }

```

```

namespace oneapi::mkl::blas::row_major {
    void axpby(sycl::queue &queue,
        std::int64_t n,
        T alpha,
        sycl::buffer<T,1> &x,
        std::int64_t incx,
        T beta,
        sycl::buffer<T,1> &y,
        std::int64_t incy);
}

```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vectors <i>x</i> and <i>y</i> .
alpha	Specifies the scalar <i>alpha</i> .
x	Buffer holding input vector <i>x</i> . Size of the buffer must be at least $1 + (n - 1) * \text{abs}(\text{incx})$. See Matrix Storage for more details.
incx	Stride between two consecutive elements of vector <i>x</i> .
beta	Specifies the scalar <i>beta</i> .
y	Buffer holding input vector <i>y</i> . Size of the buffer must be at least $1 + (n - 1) * \text{abs}(\text{incy})$. See Matrix Storage for more details.
incy	Stride between two consecutive elements of vector <i>y</i> .

Output Parameters

y	Buffer holding updated vector <i>y</i> .
---	--

axpby (USM Version)

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event axpby(sycl::queue &queue,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *x,
        std::int64_t incx,
        oneapi::mkl::value_or_pointer<T> beta,
        T *y,
        std::int64_t incy,
        const std::vector<event> &dependencies = {});
}

```

```

namespace oneapi::mkl::blas::row_major {
    sycl::event axpby(sycl::queue &queue,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,

```

```

const T *x,
std::int64_t incx,
oneapi::mkl::value_or_pointer<T> beta,
T *y,
std::int64_t incy,
const std::vector<event> &dependencies = {});
}

```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vectors <i>x</i> and <i>y</i> .
alpha	Specifies the scalar <i>alpha</i> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
x	Pointer to the input vector <i>x</i> . Size of the array must be at least $1 + (n - 1) * \text{abs}(\text{incx})$. See Matrix Storage for more details.
incx	Stride between two consecutive elements of vector <i>x</i> .
beta	Specifies the scalar <i>beta</i> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
y	Pointer to the input vector <i>y</i> . Size of the array must be at least $1 + (n - 1) * \text{abs}(\text{incy})$. See Matrix Storage for more details.
incy	Stride between two consecutive elements of vector <i>y</i> .
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y	Array holding updated vector <i>y</i> .
---	---

Return Values

Output event to wait on to ensure computation is complete.

axpy_batch

Computes a group of `axpy` operations.

Description

The `axpy_batch` routines are batched versions of [axpy](#), performing multiple `axpy` operations in a single call. Each `axpy` operation adds a scalar-vector product to a vector.

`axpy_batch` supports the following precisions:

T
float
double

T

```
std::complex<float>
```

```
std::complex<double>
```

axpy_batch (Buffer Version)

Buffer version of `axpy_batch` supports only strided API.

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
    X and Y are vectors at offset i * stridex and i * stridey in x and y
    Y = alpha * X + Y
end for
```

where:

- alpha is scalar
- X and Y are vectors

For strided API, all vectors `x` and `y` have same parameters (size, increments) and are stored at constant stride given by `stridex` and `stridey` from each other. The `x` and `y` arrays contain all the input vectors. Total number of vectors in `x` and `y` are given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void axpy_batch(sycl::queue &queue,
                   std::int64_t n,
                   T alpha,
                   sycl::buffer<T, 1> &x,
                   std::int64_t incx,
                   std::int64_t stridex,
                   sycl::buffer<T, 1> &y,
                   std::int64_t incy,
                   std::int64_t stridey,
                   std::int64_t batch_size)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void axpy_batch(sycl::queue &queue,
                   std::int64_t n,
                   T alpha,
                   sycl::buffer<T, 1> &x,
                   std::int64_t incx,
                   std::int64_t stridex,
                   sycl::buffer<T, 1> &y,
                   std::int64_t incy,
                   std::int64_t stridey,
                   std::int64_t batch_size)
}
```

Input Parameters

`queue` The queue where the routine should be executed.

<code>n</code>	Number of elements in vectors <code>X</code> and <code>Y</code> .
<code>alpha</code>	Specifies the scalar <code>alpha</code> .
<code>x</code>	Buffer holding input vectors <code>X</code> . Size of the buffer must be at least <code>batch_size * stridex</code> .
<code>incx</code>	Stride between two consecutive elements of <code>X</code> vectors. Must not be zero.
<code>stridex</code>	Stride between two consecutive <code>X</code> vectors. Must be at least zero.
<code>y</code>	Buffer holding input/output vectors <code>Y</code> . Size of the buffer must be at least <code>batch_size * stridey</code> .
<code>incy</code>	Stride between two consecutive elements of <code>Y</code> vectors. Must not be zero.
<code>stridey</code>	Stride between two consecutive <code>Y</code> vectors. Must be at least $(1 + (n-1)*abs(incy))$. See Matrix Storage for more details.
<code>batch_size</code>	Number of <code>axpy</code> computations to perform. Must be at least zero.
Output Parameters	
<code>y</code>	Output buffer overwritten by <code>batch_sizeaxpy</code> operations of the form $alpha * X + Y$.

axpy_batch (USM Version)

USM version of `axpy_batch` supports group API and strided API.

Group API

The type `Ti` of integer pointers in the group API may be either `std::int64_t` or `std::int32_t`.

Group API operation is defined as:

```

idx = 0
for i = 0 ... group_count - 1
    for j = 0 ... group_size - 1
        X and Y are vectors at x[idx] and y[idx]
        Y = alpha[i] * X + Y
        idx = idx + 1
    end for
end for

```

where:

- `alpha` is scalar
- `X` and `Y` are vectors

For group API, each group contains vectors with the same parameters (size and increment). The `x` and `y` arrays contain the pointers for all the input vectors. Total number of vectors in `x` and `y` are given by:

$$total_batch_count = \sum_{i=0}^{group_count-1} group_size[i]$$

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event axpy_batch(sycl::queue &queue,
                          const Ti *n,
                          const T *alpha,
                          const T **x,
                          const Ti *incx,
                          T **y,
                          const Ti *incy,
                          std::int64_t group_count,
                          const Ti *group_size,
                          const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event axpy_batch(sycl::queue &queue,
                          const Ti *n,
                          const T *alpha,
                          const T **x,
                          const Ti *incx,
                          T **y,
                          const Ti *incy,
                          std::int64_t group_count,
                          const Ti *group_size,
                          const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
n	Array of <code>group_count</code> integers. <code>n[i]</code> specifies number of elements in vectors <code>x</code> and <code>y</code> for every vector in group <code>i</code> .
alpha	Array of <code>group_count</code> scalar elements. <code>alpha[i]</code> specifies scaling factor for vector <code>x</code> in group <code>i</code> .
x	Array of pointers to input vectors <code>x</code> with size <code>total_batch_count</code> . Size of the array allocated for the <code>x</code> vector of the group <code>i</code> must be at least $(1 + (n[i] - 1) * \text{abs}(incx[i]))$. See Matrix Storage for more details.
incx	Array of <code>group_count</code> integers. <code>incx[i]</code> specifies stride of vector <code>x</code> in group <code>i</code> . Must not be zero.
y	Array of pointers to input/output vectors <code>y</code> with size <code>total_batch_count</code> . Size of the array allocated for the <code>y</code> vector of the group <code>i</code> must be at least $(1 + (n[i] - 1) * \text{abs}(incy[i]))$. See Matrix Storage for more details.
incy	Array of <code>group_count</code> integers. <code>incy[i]</code> specifies the stride of vector <code>y</code> in group <code>i</code> . Must not be zero.
group_count	Number of groups. Must be at least zero.
group_size	Array of <code>group_count</code> integers. <code>group_size[i]</code> specifies the number of <code>axpy</code> operations in group <code>i</code> . Each element in <code>group_size</code> must be at least zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y Array of pointers holding Y vectors, overwritten by `total_batch_count`axpy operations of the form $\alpha * X + Y$.

Return Values

Output event to wait on to ensure computation is complete.

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
    X and Y are vectors at offset i * stridex and i * stridey in x and y
    Y = alpha * X + Y
end for
```

where:

- α is scalar
- X and Y are vectors

For strided API, all vectors X and Y have same parameters (size, increments) and are stored at constant stride given by `stridex` and `stridey` from each other. The x and y arrays contain all the input vectors. Total number of vectors in x and y are given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event axpy_batch(sycl::queue &queue,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *x,
        std::int64_t incx,
        std::int64_t stridex,
        T *y,
        std::int64_t incy,
        std::int64_t stridey,
        std::int64_t batch_size,
        const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event axpy_batch(sycl::queue &queue,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *x,
        std::int64_t incx,
        std::int64_t stridex,
        T *y,
        std::int64_t incy,
        std::int64_t stridey,
        std::int64_t batch_size,
        const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue The queue where the routine should be executed.

<code>n</code>	Number of elements in vectors <code>X</code> and <code>Y</code> .
<code>alpha</code>	Specifies the scalar <code>alpha</code> . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
<code>x</code>	Pointer to input vectors <code>X</code> . Size of the array must be at least <code>batch_size * stridex</code> .
<code>incx</code>	Stride between two consecutive elements of <code>X</code> vectors. Must not be zero.
<code>stridex</code>	Stride between two consecutive <code>X</code> vectors. Must be at least zero.
<code>y</code>	Pointer to input/output vectors <code>Y</code> . Size of the array must be at least <code>batch_size * stridey</code> .
<code>incy</code>	Stride between two consecutive elements of <code>Y</code> vectors. Must not be zero.
<code>stridey</code>	Stride between two consecutive <code>Y</code> vectors. Must be at least $(1 + (n-1)*abs(incy))$. See Matrix Storage for more details.
<code>batch_size</code>	Number of <code>axpy</code> computations to perform. Must be at least zero.
<code>dependencies</code>	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

<code>y</code>	Pointer to output vectors <code>Y</code> overwritten by <code>batch_sizeaxpy</code> operations of the form $alpha * X + Y$.
----------------	--

Return Values

Output event to wait on to ensure computation is complete.

`copy_batch`

Computes a group of `copy` operations.

Description

The `copy_batch` routines are batched versions of [copy](#), performing multiple `copy` operations in a single call. Each `copy` operation copies one vector to another.

`copy_batch` supports the following precisions:

T
<code>float</code>
<code>double</code>
<code>std::complex<float></code>
<code>std::complex<double></code>

`copy_batch (Buffer Version)`

Buffer version of `copy_batch` supports only strided API.

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
    X and Y are vectors at offset i * stridex and i * stridey in x and y
    Y = X
end for
```

where:

- X and Y are vectors

For strided API, all vectors x (y) have same parameters (size, increments) and are stored at constant stridex (stridey) from each other. The x and y arrays contain all the input vectors. Total number of vectors in x and y are given by batch_size parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void copy_batch(sycl::queue &queue,
        std::int64_t n,
        sycl::buffer<T, 1> &x,
        std::int64_t incx,
        std::int64_t stridex,
        sycl::buffer<T, 1> &y,
        std::int64_t incy,
        std::int64_t stridey,
        std::int64_t batch_size)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void copy_batch(sycl::queue &queue,
        std::int64_t n,
        sycl::buffer<T, 1> &x,
        std::int64_t incx,
        std::int64_t stridex,
        sycl::buffer<T, 1> &y,
        std::int64_t incy,
        std::int64_t stridey,
        std::int64_t batch_size)
}
```

Input Parameters

queue	The queue where the routine should be executed.
n	Number of elements in vectors X and Y.
x	Buffer holding input vectors X. Size of the buffer must be at least <code>batch_size * stridex</code> .
incx	Stride between two consecutive elements of X vectors. Must not be zero.
stridex	Stride between two consecutive X vectors. Must be at least zero.
y	Buffer holding input/output vectors Y. Size of the buffer must be at least <code>batch_size * stridey</code> .
incy	Stride between two consecutive elements of Y vectors. Must not be zero.

stridey Stride between two consecutive **Y** vectors. Must be at least $(1 + (n-1)*abs(incy))$. See [Matrix Storage](#) for more details.

batch_size Number of `copy` computations to perform. Must be at least zero.

Output Parameters

y Output buffer overwritten by `batch_size` `copy` operations.

copy_batch (USM Version)

USM version of `copy_batch` supports group API and strided API.

Group API

The type `Ti` of integer pointers in the group API may be either `std::int64_t` or `std::int32_t`.

Group API operation is defined as:

```
idx = 0
for i = 0 ... group_count - 1
    for j = 0 ... group_size - 1
        X and Y are vectors at x[idx] and y[idx]
        Y = X
        idx = idx + 1
    end for
end for
```

where:

- `X` and `Y` are vectors

For group API, each group contains vectors with the same parameters (size and increment). The `x` and `y` arrays contain the pointers for all the input vectors. Total number of vectors in `x` and `y` are given by:

$$total_batch_count = \sum_{i=0}^{group_count-1} group_size[i]$$

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event copy_batch(sycl::queue &queue,
                          const Ti *n,
                          const T **x,
                          const Ti *incx,
                          T **y,
                          const Ti *incy,
                          std::int64_t group_count,
                          const Ti *group_size,
                          const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event copy_batch(sycl::queue &queue,
                          const Ti *n,
                          const T **x,
                          const Ti *incx,
```

```

    T **y,
    const Ti *incy,
    std::int64_t group_count,
    const Ti *group_size,
    const std::vector<sycl::event> &dependencies = {}
}

```

Input Parameters

queue	The queue where the routine should be executed.
n	Array of <code>group_count</code> integers. <code>n[i]</code> specifies number of elements in vectors <code>x</code> and <code>y</code> for every vector in group <code>i</code> .
x	Array of pointers to input vectors <code>x</code> with size <code>total_batch_count</code> . Size of the array allocated for the <code>x</code> vector of the group <code>i</code> must be at least $(1 + (n[i] - 1) * \text{abs}(\text{incx}[i]))$. See Matrix Storage for more details.
incx	Array of <code>group_count</code> integers. <code>incx[i]</code> specifies stride of vector <code>x</code> in group <code>i</code> . Must not be zero.
y	Array of pointers to input/output vectors <code>y</code> with size <code>total_batch_count</code> . Size of the array allocated for the <code>y</code> vector of the group <code>i</code> must be at least $(1 + (n[i] - 1) * \text{abs}(\text{incy}[i]))$. See Matrix Storage for more details.
incy	Array of <code>group_count</code> integers. <code>incy[i]</code> specifies the stride of vector <code>y</code> in group <code>i</code> . Must not be zero.
group_count	Number of groups. Must be at least zero.
group_size	Array of <code>group_count</code> integers. <code>group_size[i]</code> specifies the number of copy operations in group <code>i</code> . Each element in <code>group_size</code> must be at least zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y	Array of pointers holding <code>y</code> vectors, overwritten by <code>total_batch_count</code> copy operations.
---	--

Return Values

Output event to wait on to ensure computation is complete.

Examples

An example of how to use USM version of `copy_batch` can be found in oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/blas/source/copy_batch_usm.cpp
```

Strided API

Strided API operation is defined as:

```

for i = 0 ... batch_size - 1
    X and Y are vectors at offset i * stridex and i * stridey in x and y
    Y = X
end for

```

where:

- X and Y are vectors

For strided API, all vectors x (y) have same parameters (size, increments) and are stored at constant `stridx` (`stridey`) from each other. The x and y arrays contain all the input vectors. Total number of vectors in x and y are given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event copy_batch(sycl::queue &queue,
                          std::int64_t n,
                          const T *x,
                          std::int64_t incx,
                          std::int64_t stridx,
                          T *y,
                          std::int64_t incy,
                          std::int64_t stridey,
                          std::int64_t batch_size,
                          const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event copy_batch(sycl::queue &queue,
                          std::int64_t n,
                          const T *x,
                          std::int64_t incx,
                          std::int64_t stridx,
                          T *y,
                          std::int64_t incy,
                          std::int64_t stridey,
                          std::int64_t batch_size,
                          const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>n</code>	Number of elements in vectors X and Y .
<code>x</code>	Pointer to input vectors X . Size of the array must be at least <code>batch_size * stridx</code> .
<code>incx</code>	Stride between two consecutive elements of X vectors. Must not be zero.
<code>stridx</code>	Stride between two consecutive X vectors. Must be at least zero.
<code>y</code>	Pointer to input/output vectors Y . Size of the array must be at least <code>batch_size * stridey</code> .
<code>incy</code>	Stride between two consecutive elements of Y vectors. Must not be zero.
<code>stridey</code>	Stride between two consecutive Y vectors. Must be at least $(1 + (n-1)*abs(incy))$. See Matrix Storage for more details.
<code>batch_size</code>	Number of <code>copy</code> computations to perform. Must be at least zero.

dependencies List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y Output vectors overwritten by `batch_sizecopy` operations.

Return Values

Output event to wait on to ensure computation is complete.

dgmm_batch

Computes a group of (diagonal matrix-matrix product (`dgmm`) operations.

Description

The `dgmm_batch` routines perform multiple diagonal matrix-matrix product (`dgmm`) operations in a single call. The diagonal matrices are stored as dense vectors and the operations are performed with groups of matrices and vectors.

`dgmm_batch` supports the following precisions:

T
<code>float</code>
<code>double</code>
<code>std::complex<float></code>
<code>std::complex<double></code>

dgmm_batch (Buffer Version)

Buffer version of `dgmm_batch` supports only strided API.

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
    A and C are matrices at offset i * stridea in a, i * stridec in c.
    X is a vector at offset i * stridex in x
    if (left_right == side::left)
        C = diag(X) * A
    else
        C = A * diag(X)
end for
```

where:

- A is a matrix
- X is a diagonal matrix stored as a vector

For strided API, all matrices A and C and vector X have the same parameters (size, increments) and are stored at a constant stride given by `stridea`, `stridec` and `stridex` from each other.

The `a` and `x` buffers contain all the input matrices. Total number of matrices in `a` and `x` are given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void dgemm_batch(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    std::int64_t m,
                    std::int64_t n,
                    sycl::buffer<T,1> &a,
                    std::int64_t lda,
                    std::int64_t stridea,
                    sycl::buffer<T,1> &x,
                    std::int64_t incx,
                    std::int64_t stridex,
                    sycl::buffer<T,1> &c,
                    std::int64_t ldc,
                    std::int64_t stridec,
                    std::int64_t batch_size);
}
```

```
namespace oneapi::mkl::blas::row_major {
    void dgemm_batch(sycl::queue &queue,
                    oneapi::mkl::side left_right,
                    std::int64_t m,
                    std::int64_t n,
                    sycl::buffer<T,1> &a,
                    std::int64_t lda,
                    std::int64_t stridea,
                    sycl::buffer<T,1> &x,
                    std::int64_t incx,
                    std::int64_t stridex,
                    sycl::buffer<T,1> &c,
                    std::int64_t ldc,
                    std::int64_t stridec,
                    std::int64_t batch_size);
}
```

Input Parameters

queue	The queue where the routine should be executed.
left_right	Specifies the position of the diagonal matrix in the product. See Data Types for more details.
m	Number of rows of matrix A and matrix C. Must be at least zero.
n	Number of columns of matrix A and matrix C. Must be at least zero.
a	Buffer holding input matrices A. Size of the buffer must be at least $lda * k + stridea * (batch_size - 1)$ where k is n if column major layout or m if row major layout is used.
lda	Leading dimension of matrices A. Must be at least m if column major layout or n if row major layout is used. Must be positive.
stridea	Stride between two consecutive A matrices. Must be at least zero. See Matrix Storage for more details.

<code>x</code>	Buffer holding input matrices <code>X</code> . Size of the buffer must be at least $(1 + (\text{len} - 1) * \text{abs}(\text{incx})) + \text{stridex} * (\text{batch_size} - 1)$ where <code>len</code> is <code>n</code> if the diagonal matrix is on the right of the product or <code>m</code> otherwise.
<code>incx</code>	Stride between two consecutive elements of the <code>X</code> vectors.
<code>stridex</code>	Stride between two consecutive <code>X</code> vectors. Must be at least zero. See Matrix Storage for more details.
<code>c</code>	Buffer holding input/output matrices <code>C</code> . Size of the buffer must be at least $\text{batch_size} * \text{stridec}$.
<code>ldc</code>	Leading dimension of matrices <code>C</code> . Must be at least <code>m</code> if column major layout or <code>n</code> if row major layout is used. Must be positive.
<code>stridec</code>	Stride between two consecutive <code>C</code> matrices. Must be at least $\text{ldc} * n$ if column major layout or $\text{ldc} * m$ if row major layout is used. See Matrix Storage for more details.
<code>batch_size</code>	Number of <code>dgmm</code> computations to perform. Must be at least zero.
Output Parameters	
<code>c</code>	Buffer holding output matrices <code>C</code> overwritten by <code>batch_sizedgmm</code> operations.

dgmm_batch (USM Version)

USM version of `dgmm_batch` supports group API and strided API.

Group API

The type `Ti` of integer pointers in the group API may be either `std::int64_t` or `std::int32_t`.

Group API operation is defined as:

```

idx = 0
for i = 0 ... group_count - 1
    for j = 0 ... group_size - 1
        A and C are matrices at a[idx] and c[idx]
        X is a vector at x[idx]
        if (left_right[idx] == side::left)
            C = diag(X) * A
        else
            C = A * diag(X)
        idx = idx + 1
    end for
end for

```

where:

- `A` is a matrix
- `X` is a diagonal matrix stored as a vector

For group API, each group contain matrices and vectors with the same parameters (size, increment). The `a` and `x` arrays contain the pointers for all the input matrices. Total number of matrices in `a` and `x` are given by:

$$total_batch_count = \sum_{i=0}^{group_count-1} group_size[i]$$

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event dgmm_batch(sycl::queue &queue,
                          const oneapi::mkl::side *left_right,
                          const Ti *m,
                          const Ti *n,
                          const T **a,
                          const Ti *lda,
                          const T **x,
                          const Ti *incx,
                          T **c,
                          const Ti *ldc,
                          std::int64_t group_count,
                          const Ti *group_size,
                          const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event dgmm_batch(sycl::queue &queue,
                          const oneapi::mkl::side *left_right,
                          const Ti *m,
                          const Ti *n,
                          const T **a,
                          const Ti *lda,
                          const T **x,
                          const Ti *incx,
                          T **c,
                          const Ti *ldc,
                          std::int64_t group_count,
                          const Ti *group_size,
                          const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>left_right</code>	Array of <code>group_count</code> parameters. <code>left_right[i]</code> specifies the position of the diagonal matrix in group <code>i</code> . See Data Types for more details.
<code>m</code>	Array of <code>group_count</code> integers. <code>m[i]</code> specifies number of rows of <code>A</code> for every matrix in group <code>i</code> . All entries must be at least zero.
<code>n</code>	Array of <code>group_count</code> integers. <code>n[i]</code> specifies number of columns of <code>A</code> for every matrix in group <code>i</code> . All entries must be at least zero.
<code>a</code>	Array of pointers to input matrices <code>A</code> with size <code>total_batch_count</code> . Size of the array must be at least <code>lda[i] * n[i]</code> if column major layout or at least <code>lda[i] * m[i]</code> if row major layout is used. See Matrix Storage for more details.

<code>lda</code>	Array of <code>group_count</code> integers. <code>lda[i]</code> specifies the leading dimension of <code>A</code> for every matrix in group <code>i</code> . All entries must be positive and at least <code>m[i]</code> if column major layout or at least <code>n[i]</code> if row major layout is used.
<code>x</code>	Array of pointers to input vectors <code>x</code> with size <code>total_batch_count</code> . Size of the array must be at least $(1 + \text{len}[i] - 1) * \text{abs}(\text{incx}[i])$ where <code>len[i]</code> is <code>n[i]</code> if diagonal matrix is on the right of the product or <code>m[i]</code> otherwise. See Matrix Storage for more details.
<code>incx</code>	Array of <code>group_count</code> integers. <code>incx[i]</code> specifies the stride of <code>x</code> for every vector in group <code>i</code> . All entries must be positive.
<code>c</code>	Array of pointers to input/output matrices <code>C</code> with size <code>total_batch_count</code> . Size of the array must be at least <code>ldc[i] * n[i]</code> if column major layout or at least <code>ldc[i] * m[i]</code> if row major layout is used. See Matrix Storage for more details.
<code>ldc</code>	Array of <code>group_count</code> integers. <code>ldc[i]</code> specifies the leading dimension of <code>C</code> for every matrix in group <code>i</code> . All entries must be positive and at least <code>m[i]</code> if column major layout or at least <code>n[i]</code> if row major layout is used.
<code>group_count</code>	Specifies number of groups. Must be at least zero.
<code>group_size</code>	Array of <code>group_count</code> integers. <code>group_size[i]</code> specifies the number of diagonal matrix-matrix product operations in group <code>i</code> . All entries must be at least zero.
<code>dependencies</code>	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

<code>c</code>	Array of pointers to output matrices <code>C</code> overwritten by <code>total_batch_count</code> <code>tdgmm</code> operations.
----------------	--

Return Values

Output event to wait on to ensure computation is complete.

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
  A and C are matrices at offset i * stridea in a, i * stridec in c.
  X is a vector at offset i * stridex in x
  if (left_right == side::left)
    C = diag(X) * A
  else
    C = A * diag(X)
end for
```

where:

- `A` is a matrix
- `x` is a diagonal matrix stored as a vector

For strided API, all matrices `A` and `C` and vector `x` have the same parameters (size, increments) and are stored at a constant stride given by `stridea`, `stridec` and `stridex` from each other.

The `a` and `x` buffers contain all the input matrices. Total number of matrices in `a` and `x` are given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event dgmm_batch(sycl::queue &queue,
                          oneapi::mkl::side left_right,
                          std::int64_t m,
                          std::int64_t n,
                          const T *a,
                          std::int64_t lda,
                          std::int64_t stridea,
                          const T *x,
                          std::int64_t incx,
                          std::int64_t stridex,
                          T *c,
                          std::int64_t ldc,
                          std::int64_t stridec,
                          std::int64_t batch_size,
                          const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event dgmm_batch(sycl::queue &queue,
                          oneapi::mkl::side left_right,
                          std::int64_t m,
                          std::int64_t n,
                          const T *a,
                          std::int64_t lda,
                          std::int64_t stridea,
                          const T *x,
                          std::int64_t incx,
                          std::int64_t stridex,
                          T *c,
                          std::int64_t ldc,
                          std::int64_t stridec,
                          std::int64_t batch_size,
                          const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>left_right</code>	Specifies the position of the diagonal matrix in the product. See Data Types for more details.
<code>m</code>	Number of rows of matrix <code>A</code> and matrix <code>C</code> . Must be at least zero.
<code>n</code>	Number of columns of matrix <code>A</code> and matrix <code>C</code> . Must be at least zero.
<code>a</code>	Pointer to input matrices <code>A</code> . Size of the array must be at least <code>lda * k + stridea * (batch_size - 1)</code> where <code>k</code> is <code>n</code> if column major layout or <code>m</code> if row major layout is used.
<code>lda</code>	Leading dimension of matrices <code>A</code> . Must be at least <code>m</code> if column major layout or <code>n</code> if row major layout is used. Must be positive.

stridea	Stride between two consecutive A matrices. Must be at least zero. See Matrix Storage for more details.
x	Pointer to input matrices X. Size of the array must be at least $(1 + (\text{len} - 1) * \text{abs}(\text{incx})) + \text{stridex} * (\text{batch_size} - 1)$ where len is n if the diagonal matrix is on the right of the product or m otherwise.
incx	Stride between two consecutive elements of the X vectors.
stridex	Stride between two consecutive X vectors. Must be at least zero. See Matrix Storage for more details.
c	Pointer to input/output matrices C. Size of the array must be at least $\text{batch_size} * \text{stridec}$.
ldc	Leading dimension of matrices C. Must be at least m if column major layout or n if row major layout is used. Must be positive.
stridec	Stride between two consecutive C matrices. Must be at least $\text{ldc} * n$ if column major layout or $\text{ldc} * m$ if row major layout is used. See Matrix Storage for more details.
batch_size	Number of <code>dgemm</code> computations to perform. Must be at least zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

c	Pointer to output matrices C overwritten by <code>batch_sizedgemm</code> operations.
---	--

Return Values

Output event to wait on to ensure computation is complete.

gemm_batch

Computes groups of matrix-matrix product with general matrices.

Description

The `gemm_batch` routines are batched versions of [gemm](#), performing multiple `gemm` operations in a single call. Each `gemm` operation performs a matrix-matrix product with general matrices.

`gemm_batch` supports the following precisions:

Ta (A matrices)	Tb (B matrices)	Tc (C matrices)	Ts (alpha/beta)
sycl::half	sycl::half	sycl::half	sycl::half
sycl::half	sycl::half	float	float
oneapi::mkl::bfloat 16	oneapi::mkl::bfloat 16	oneapi::mkl::bfloat 16	float

Ta (A matrices)	Tb (B matrices)	Tc (C matrices)	Ts (alpha/beta)
oneapi::mkl::bfloat 16	oneapi::mkl::bfloat 16	float	float
std::int8_t	std::int8_t	std::int32_t	float
std::int8_t	std::int8_t	float	float
float	float	float	float
double	double	double	double
std::complex<float>	std::complex<float>	std::complex<float>	std::complex<float>
std::complex<double >	std::complex<double >	std::complex<double >	std::complex<double >

gemm_batch (Buffer Version)

Buffer version of `gemm_batch` supports only strided API.

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
    A, B and C are matrices at offset i * stridea, i * strideb, i * stridec in a, b and c.
    C = alpha * op(A) * op(B) + beta * C
end for
```

where:

- $op(x)$ is one of $op(x) = x$, or $op(x) = x^T$, or $op(x) = x^H$
- α and β are scalars
- A, B, and C are matrices
- $op(A)$ is $m \times k$, $op(B)$ is $k \times n$, and C is $m \times n$

For strided API, `a`, `b` and `c` buffers contains all the input matrices. The stride between matrices is given by the stride parameters. Total number of matrices in `a`, `b` and `c` buffers is given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void gemm_batch(sycl::queue &queue,
        oneapi::mkl::transpose transa,
        oneapi::mkl::transpose transb,
        std::int64_t m,
        std::int64_t n,
        std::int64_t k,
        Ts alpha,
        sycl::buffer<Ta,1> &a,
        std::int64_t lda,
        std::int64_t stridea,
        sycl::buffer<Tb,1> &b,
        std::int64_t ldb,
        std::int64_t strideb,
        Ts beta,
```

```

        sycl::buffer<Tc,1> &c,
        std::int64_t ldc,
        std::int64_t stridec,
        std::int64_t batch_size,
        compute_mode mode = compute_mode::unset)
}

namespace oneapi::mkl::blas::row_major {
    void gemm_batch(sycl::queue &queue,
        oneapi::mkl::transpose transa,
        oneapi::mkl::transpose transb,
        std::int64_t m,
        std::int64_t n,
        std::int64_t k,
        Ts alpha,
        sycl::buffer<Ta,1> &a,
        std::int64_t lda,
        std::int64_t stridea,
        sycl::buffer<Tb,1> &b,
        std::int64_t ldb,
        std::int64_t strideb,
        Ts beta,
        sycl::buffer<Tc,1> &c,
        std::int64_t ldc,
        std::int64_t stridec,
        std::int64_t batch_size,
        compute_mode mode = compute_mode::unset)
}

```

Input Parameters

queue	The queue where the routine should be executed.
transa	Specifies $\text{op}(A)$, transposition operation applied to matrices A . See Data Types for more details.
transb	Specifies $\text{op}(B)$, transposition operation applied to matrices B . See Data Types for more details.
m	Number of rows of matrices $\text{op}(A)$ and matrices C . Must be at least zero.
n	Number of columns of matrices $\text{op}(B)$ and matrices C . Must be at least zero.
k	Number of columns of matrices $\text{op}(A)$ and rows of matrices $\text{op}(B)$. Must be at least zero.
alpha	Scaling factor for matrix-matrix products.
a	Buffer holding input matrices A . Size of the buffer must be at least <code>stridea * batch_size</code> .
lda	Leading dimension of matrices A . Must be positive.

	<code>transa = transpose::nontrans</code>	<code>transa = transpose::trans</code> or <code>trans = transpose::conjtrans</code>
Column major	Must be at least m	Must be at least k
Row major	Must be at least k	Must be at least m

stridea

Stride between two consecutive **A** matrices.

	<code>transa = transpose::nontrans</code>	<code>transa = transpose::trans</code> or <code>trans = transpose::conjtrans</code>
Column major	Must be at least $lda * k$	Must be at least $lda * m$
Row major	Must be at least $lda * m$	Must be at least $lda * k$

b

Buffer holding input matrices **B**. Size of the buffer must be at least `strideb * batch_size`.

ldb

Leading dimension of matrices **B**. Must be positive.

	<code>transa = transpose::nontrans</code>	<code>transa = transpose::trans</code> or <code>trans = transpose::conjtrans</code>
Column major	Must be at least k	Must be at least n
Row major	Must be at least n	Must be at least k

strideb

Stride between two consecutive **B** matrices.

	<code>transa = transpose::nontrans</code>	<code>transa = transpose::trans</code> or <code>trans = transpose::conjtrans</code>
Column major	Must be at least $ldb * n$	Must be at least $ldb * k$
Row major	Must be at least $ldb * k$	Must be at least $ldb * n$

beta

Scaling factor for matrices **C**.

<code>c</code>	Buffer holding input/output matrices <code>C</code> . Size of the buffer must be at least <code>stridedc * batch_size</code> .				
<code>ldc</code>	Leading dimension of matrices <code>C</code> . Must be positive. <table border="1"> <tr> <td>Column major</td><td>Must be at least <code>m</code></td></tr> <tr> <td>Row major</td><td>Must be at least <code>n</code></td></tr> </table>	Column major	Must be at least <code>m</code>	Row major	Must be at least <code>n</code>
Column major	Must be at least <code>m</code>				
Row major	Must be at least <code>n</code>				
<code>stridedc</code>	Stride between two consecutive <code>C</code> matrices. <table border="1"> <tr> <td>Column major</td><td>Must be at least <code>ldc * n</code></td></tr> <tr> <td>Row major</td><td>Must be at least <code>ldc * m</code></td></tr> </table>	Column major	Must be at least <code>ldc * n</code>	Row major	Must be at least <code>ldc * m</code>
Column major	Must be at least <code>ldc * n</code>				
Row major	Must be at least <code>ldc * m</code>				
<code>batch_size</code>	Specifies the number of matrix multiply operations to perform. Must be at least zero.				
<code>mode</code>	<i>Optional</i> . Compute mode settings. See Compute Modes for more details.				
Output Parameters					
<code>c</code>	Output buffer overwritten by <code>batch_size</code> <code>gemm</code> operations of the form $\alpha * \text{op}(A) * \text{op}(B) + \beta * C$.				

NOTE If `beta = 0`, matrices `C` do not need to be initialized before calling `gemm_batch`.

gemm_batch (USM Version)

USM version of `gemm_batch` supports group API and strided API.

Group API

The type `Ti` of integer pointers in the group API may be either `std::int64_t` or `std::int32_t`.

Group API operation is defined as:

```

idx = 0
for i = 0 ... group_count - 1
  for j = 0 ... group_size - 1
    A, B, and C are matrices in a[idx], b[idx] and c[idx]
    C = alpha[i] * op(A) * op(B) + beta[i] * C
    idx = idx + 1
  end for
end for

```

where:

- `op(x)` is one of `op(x) = x`, or `op(x) = xT`, or `op(x) = xH`
- `alpha` and `beta` are scalars
- `A`, `B`, and `C` are matrices
- `op(A)` is `m × k`, `op(B)` is `k × n`, and `C` is `m × n`

For group API, `a`, `b` and `c` arrays contain the pointers for all the input matrices. The total number of matrices in `a`, `b` and `c` are given by:

$$total_batch_count = \sum_{i=0}^{group_count-1} group_size[i]$$

Group API supports both pointer and span inputs.

The advantage of using span instead of pointer is that the sizes of the array can vary and the size of the span can be queried at runtime. For each `gemm` parameter, except the output matrices, span can be of size 1, the number of groups or the total batch size. For output matrices, to ensure all computation are independent, size of the span must be the total batch size.

Depending on the size of spans, each parameter for the `gemm` computation is used as follows:

span size = 1	Parameter is reused for all <code>gemm</code> computations
span size = <code>group_count</code>	Parameter is reused for all <code>gemm</code> within a group. Each group will have a different value for this parameter. This is same as <code>gemm_batch</code> group API with pointers.
span size = <code>total_batch_count</code>	Each <code>gemm</code> computation use a different value for this parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event gemm_batch(sycl::queue &queue,
        const oneapi::mkl::transpose *transa,
        const oneapi::mkl::transpose *transb,
        const Ti *m,
        const Ti *n,
        const Ti *k,
        const Ts *alpha,
        const Ta **a,
        const Ti *lda,
        const Tb **b,
        const Ti *ldb,
        const Ts *beta,
        Tc **c,
        const Ti *ldc,
        std::int64_t group_count,
        const Ti *group_size,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {})

    sycl::event gemm_batch(sycl::queue &queue,
        const sycl::span<oneapi::mkl::transpose> &transa,
        const sycl::span<oneapi::mkl::transpose> &transb,
        const sycl::span<std::int64_t> &m,
        const sycl::span<std::int64_t> &n,
        const sycl::span<std::int64_t> &k,
        const sycl::span<Ts> &alpha,
        const sycl::span<const Ta*> &a,
        const sycl::span<std::int64_t> &lda,
        const sycl::span<const Tb*> &b,
        const sycl::span<std::int64_t> &ldb,
```



```

        const sycl::span<Ts> &beta,
        sycl::span<Tc*> &c,
        const sycl::span<std::int64_t> &lDC,
        size_t group_count,
        const sycl::span<size_t> &group_sizes,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {}
    }

namespace oneapi::mkl::blas::row_major {
    sycl::event gemm_batch(sycl::queue &queue,
        const oneapi::mkl::transpose *transa,
        const oneapi::mkl::transpose *transb,
        const Ti *m,
        const Ti *n,
        const Ti *k,
        const Ts *alpha,
        const Ta **a,
        const Ti *lda,
        const Tb **b,
        const Ti *ldb,
        const Ts *beta,
        Tc **c,
        const Ti *lDC,
        std::int64_t group_count,
        const Ti *group_size,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {}

    sycl::event gemm_batch(sycl::queue &queue,
        const sycl::span<oneapi::mkl::transpose> &transa,
        const sycl::span<oneapi::mkl::transpose> &transb,
        const sycl::span<std::int64_t> &m,
        const sycl::span<std::int64_t> &n,
        const sycl::span<std::int64_t> &k,
        const sycl::span<Ts> &alpha,
        const sycl::span<const Ta*> &a,
        const sycl::span<std::int64_t> &lda,
        const sycl::span<const Tb*> &b,
        const sycl::span<std::int64_t> &ldb,
        const sycl::span<Ts> &beta,
        sycl::span<Tc*> &c,
        const sycl::span<std::int64_t> &lDC,
        size_t group_count,
        const sycl::span<size_t> &group_sizes,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {}
    }
}

```

Input Parameters

queue	The queue where the routine should be executed.
transa	Array or span of <code>group_count</code> <code>oneapi::mkl::transpose</code> values. <code>transa[i]</code> specifies <code>op(A)</code> , the transposition operation applied to matrices <code>A</code> in group <code>i</code> . See Data Types for more details.

- transb** Array or span of `group_count` `oneapi::mkl::transpose` values. `transb[i]` specifies `op(B)`, the transposition operation applied to matrices `B` in group `i`. See [Data Types](#) for more details.
- m** Array or span of `group_count` integers. `m[i]` specifies number of rows of matrices `op(A)` and matrices `C` in group `i`. All entries must be at least zero.
- n** Array or span of `group_count` integers. `n[i]` specifies number of columns of matrices `op(B)` and matrices `C` in group `i`. All entries must be at least zero.
- k** Array or span of `group_count` integers. `k[i]` specifies number of columns of matrices `op(A)` and rows of matrices `op(B)` in group `i`. All entries must be at least zero.
- alpha** Array or span of `group_count` scalar elements. `alpha[i]` specifies scaling factor for matrix-matrix products in group `i`.
- a** Array of `total_batch_count` pointers for input matrices `A` or span of `total_batch_count` input matrices `A`. See [Matrix Storage](#) for more details.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	Size of array <code>A[i]</code> must be at least <code>lda[i] * k[i]</code>	Size of array <code>A[i]</code> must be at least <code>lda[i] * m[i]</code>
Row major	Size of array <code>A[i]</code> must be at least <code>lda[i] * m[i]</code>	Size of array <code>A[i]</code> must be at least <code>lda[i] * k[i]</code>

- lda** Array or span of `group_count` integers. `lda[i]` specifies leading dimension of matrices `A` in group `i`. Must be positive.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least <code>m[i]</code>	Must be at least <code>k[i]</code>
Row major	Must be at least <code>k[i]</code>	Must be at least <code>m[i]</code>

- b** Array of `total_batch_count` pointers for input matrices `B` or span of `total_batch_count` input matrices `B`. See [Matrix Storage](#) for more details.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	Size of array B[i] must be at least $\text{ldb}[i] * n[i]$	Size of array B[i] must be at least $\text{ldb}[i] * k[i]$
Row major	Size of array B[i] must be at least $\text{ldb}[i] * k[i]$	Size of array B[i] must be at least $\text{ldb}[i] * n[i]$

ldb Array or span of `group_count` integers. `ldb[i]` specifies leading dimension of matrices B in group i. Must be positive.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least $k[i]$	Must be at least $n[i]$
Row major	Must be at least $n[i]$	Must be at least $k[i]$

beta Array or span of `group_count` scalar elements. `beta[i]` specifies scaling factor for matrices C in group i.

c Array of `total_batch_count` pointers for input/output matrices C or span of `total_batch_count` input/output matrices C. See [Matrix Storage](#) for more details.

Column major	Size of array C[i] must be at least $\text{ldc}[i] * n[i]$
Row major	Size of array C[i] must be at least $\text{ldc}[i] * m[i]$

ldc Array or span of `group_count` integers. `ldc[i]` specifies leading dimension of matrices C in group i. Must be positive.

Column major	Must be at least $m[i]$
Row major	Must be at least $n[i]$

group_count Number of groups. Must be at least zero.

group_size Array or span of `group_count` integers. `group_size[i]` specifies the number of `gemm` operations in group i. Each element in `group_size` must be at least zero.

mode *Optional*. Compute mode settings. See [Compute Modes](#) for more details.

dependencies *Optional.* List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

`mode` and `dependencies` may be omitted independently; it is not necessary to specify `mode` in order to provide dependencies.

Output Parameters

c Array of pointers to output matrices `C` overwritten by `total_batch_count` gemm operations of the form $\alpha * \text{op}(A) * \text{op}(B) + \beta * C$.

NOTE If $\beta = 0$, matrices `C` do not need to be initialized before calling `gemm_batch`.

Return Values

Output event to wait on to ensure computation is complete.

Examples

An example of how to use USM version of `gemm_batch` can be found in oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/blas/source/gemm_batch.cpp
```

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
    A, B and C are matrices at offset i * stridea, i * strideb, i * stridec in a, b and c.
    C = alpha * op(A) * op(B) + beta * C
end for
```

where:

- `op(x)` is one of $\text{op}(x) = x$, or $\text{op}(x) = x^T$, or $\text{op}(x) = x^H$
- `alpha` and `beta` are scalars
- `A`, `B`, and `C` are matrices
- `op(A)` is $m \times k$, `op(B)` is $k \times n$, and `C` is $m \times n$

For strided API, `a`, `b` and `c` arrays contains all the input matrices. The stride between matrices is either given by the stride parameters. Total number of matrices in `a`, `b` and `c` arrays is given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event gemm_batch(sycl::queue &queue,
                          oneapi::mkl::transpose transa,
                          oneapi::mkl::transpose transb,
                          std::int64_t m,
                          std::int64_t n,
                          std::int64_t k,
                          oneapi::mkl::value_or_pointer<Ts> alpha,
                          const Ta *a,
                          std::int64_t lda,
                          std::int64_t stridea,
                          const Tb *b,
                          std::int64_t ldb,
                          std::int64_t strideb,
```

```

        oneapi::mkl::value_or_pointer<Ts> beta,
        Tc *c,
        std::int64_t ldc,
        std::int64_t stridec,
        std::int64_t batch_size,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {})
    }

namespace oneapi::mkl::blas::row_major {
    sycl::event gemm_batch(sycl::queue &queue,
        oneapi::mkl::transpose transa,
        oneapi::mkl::transpose transb,
        std::int64_t m,
        std::int64_t n,
        std::int64_t k,
        oneapi::mkl::value_or_pointer<Ts> alpha,
        const Ta *a,
        std::int64_t lda,
        std::int64_t stridea,
        const Tb *b,
        std::int64_t ldb,
        std::int64_t strideb,
        oneapi::mkl::value_or_pointer<Ts> beta,
        Tc *c,
        std::int64_t ldc,
        std::int64_t stridec,
        std::int64_t batch_size,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {})
    }
}

```

Input Parameters

queue	The queue where the routine should be executed.
transa	Specifies op(A), transposition operation applied to matrices A. See Data Types for more details.
transb	Specifies op(B), transposition operation applied to matrices B. See Data Types for more details.
m	Number of rows of matrices op(A) and matrices C. Must be at least zero.
n	Number of columns of matrices op(B) and matrices C. Must be at least zero.
k	Number of columns of matrices op(A) and rows of matrices op(B). Must be at least zero.
alpha	Scaling factor for matrix-matrix products. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrices A. Size of the array must be at least <code>stridea * batch_size</code> .
lda	Leading dimension of matrices A. Must be positive.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtran s
Column major	Must be at least m	Must be at least k
Row major	Must be at least k	Must be at least m

stridea

Stride between two consecutive A matrices.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtran s
Column major	Must be at least $lda * k$	Must be at least $lda * m$
Row major	Must be at least $lda * m$	Must be at least $lda * k$

b

Pointer to input matrices B. Size of the array must be at least `strideb * batch_size`.

ldb

Leading dimension of matrices B. Must be positive.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtran s
Column major	Must be at least k	Must be at least n
Row major	Must be at least n	Must be at least k

strideb

Stride between two consecutive B matrices.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtran s
Column major	Must be at least $ldb * n$	Must be at least $ldb * k$
Row major	Must be at least $ldb * k$	Must be at least $ldb * n$

beta	Scaling factor for matrices C. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.				
c	Pointer to input/output matrices C. Size of the array must be at least <code>stridedc * batch_size</code> .				
ldc	Leading dimension of matrices C. Must be positive. <table border="1" data-bbox="483 428 1469 543"> <tr> <td>Column major</td><td>Must be at least <code>m</code></td></tr> <tr> <td>Row major</td><td>Must be at least <code>n</code></td></tr> </table>	Column major	Must be at least <code>m</code>	Row major	Must be at least <code>n</code>
Column major	Must be at least <code>m</code>				
Row major	Must be at least <code>n</code>				
stridedc	Stride between two consecutive C matrices. <table border="1" data-bbox="483 611 1469 726"> <tr> <td>Column major</td><td>Must be at least <code>ldc * n</code></td></tr> <tr> <td>Row major</td><td>Must be at least <code>ldc * m</code></td></tr> </table>	Column major	Must be at least <code>ldc * n</code>	Row major	Must be at least <code>ldc * m</code>
Column major	Must be at least <code>ldc * n</code>				
Row major	Must be at least <code>ldc * m</code>				
batch_size	Specifies the number of matrix multiply operations to perform. Must be at least zero.				
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.				
dependencies	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.				

Output Parameters

c	Pointer to output matrices C overwritten by <code>batch_size</code> gemm operations of the form $\alpha * \text{op}(A) * \text{op}(B) + \text{beta} * C$.
---	--

NOTE If `beta = 0`, matrices C do not need to be initialized before calling `gemm_batch`.

Return Values

Output event to wait on to ensure computation is complete.

gemm_bias

Computes a matrix-matrix product using general integer matrices with bias.

Description

The `gemm_bias` routines compute a scalar-matrix-matrix product and add the result to a scalar-matrix product, using general integer matrices with biases/offsets. The operation is defined as:

$$C \leftarrow \alpha * (\text{op}(A) - A_offset) * (\text{op}(B) - B_offset) + \text{beta} * C + C_offset$$

where:

- `op(x)` is one of `op(x) = x`, or `op(x) = xT`, or `op(x) = xH`
- `alpha` and `beta` are scalars

- `A_offset` is $m \times k$ matrix with every element equal to the value `ao`
- `B_offset` is $k \times n$ matrix with every element equal to the value `bo`
- `C_offset` is $m \times n$ matrix defined by the `co` buffer. See [Data Types](#) for more details.
- `A`, `B`, and `C` are matrices
- `op(A)` is $m \times k$, `op(B)` is $k \times n$, and `C` is $m \times n$

`gemm_bias` supports the following precisions:

Ta	Tb
<code>std::uint8_t</code>	<code>std::uint8_t</code>
<code>std::int8_t</code>	<code>std::uint8_t</code>
<code>std::uint8_t</code>	<code>std::int8_t</code>
<code>std::int8_t</code>	<code>std::int8_t</code>

gemm_bias (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void gemm_bias(sycl::queue &queue,
                  oneapi::mkl::transpose transa,
                  oneapi::mkl::transpose transb,
                  oneapi::mkl::offset offsetc,
                  std::int64_t m,
                  std::int64_t n,
                  std::int64_t k,
                  float alpha,
                  sycl::buffer<Ta,1> &a,
                  std::int64_t lda,
                  Ta ao,
                  sycl::buffer<Tb,1> &b,
                  std::int64_t ldb,
                  Tb bo,
                  float beta,
                  sycl::buffer<std::int32_t,1> &c,
                  std::int64_t ldc,
                  sycl::buffer<std::int32_t,1> &co,
                  compute_mode mode = compute_mode::unset)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void gemm_bias(sycl::queue &queue,
                  oneapi::mkl::transpose transa,
                  oneapi::mkl::transpose transb,
                  oneapi::mkl::offset offsetc,
                  std::int64_t m,
                  std::int64_t n,
                  std::int64_t k,
                  float alpha,
                  sycl::buffer<Ta,1> &a,
                  std::int64_t lda,
                  Ta ao,
                  sycl::buffer<Tb,1> &b,
                  std::int64_t ldb,
                  Tb bo,
```



```

float beta,
sycl::buffer<std::int32_t,1> &c,
std::int64_t ldc,
sycl::buffer<std::int32_t,1> &co,
compute_mode mode = compute_mode::unset)
}

```

Input Parameters

queue	The queue where the routine should be executed.
transa	Specifies $\text{op}(A)$, the transposition operation applied to matrix A. See Data Types for more details.
transb	Specifies $\text{op}(B)$, the transposition operation applied to matrix B. See Data Types for more details.
offsetc	Specifies the form of C_offset used in the matrix multiplication. See Data Types for more details.
m	Number of rows of matrix $\text{op}(A)$ and matrix C. Must be at least zero.
n	Number of columns of matrix $\text{op}(B)$ and matrix C. Must be at least zero.
k	Number of columns of matrix $\text{op}(A)$ and rows of matrix $\text{op}(B)$. Must be at least zero.
alpha	Scaling factor for matrix-matrix product.
a	Buffer holding input matrix A. See Matrix Storage for more details.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtran s
Column major	A is $m \times k$ matrix. Size of array a must be at least $lda * k$	A is $k \times m$ matrix. Size of array a must be at least $lda * m$
Row major	A is $m \times k$ matrix. Size of array a must be at least $lda * m$	A is $k \times m$ matrix. Size of array a must be at least $lda * k$

lda Leading dimension of matrix A. Must be positive.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtran s
Column major	Must be at least m	Must be at least k
Row major	Must be at least k	Must be at least m

ao Specifies the scalar offset value for matrix A.

b Buffer holding input matrix B. See [Matrix Storage](#) for more details.

	transb = transpose::nontrans	transb = transpose::trans Or trans = transpose::conjtrans
Column major	B is $k \times n$ matrix. Size of array b must be at least $ldb * n$	B is $n \times k$ matrix. Size of array b must be at least $ldb * k$
Row major	B is $k \times n$ matrix. Size of array b must be at least $ldb * k$	B is $n \times k$ matrix. Size of array b must be at least $ldb * n$

ldb Leading dimension of matrix B. Must be positive.

	transb = transpose::nontrans	transb = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least k	Must be at least n
Row major	Must be at least n	Must be at least k

bo Specifies the scalar offset value for matrix B.

beta Scaling factor for matrix C.

c Buffer holding input/output matrix C. See [Matrix Storage](#) for more details.

Column major	C is $m \times n$ matrix. Size of array c must be at least $ldc * n$
Row major	C is $m \times n$ matrix. Size of array c must be at least $ldc * m$

ldc Leading dimension of matrix C. Must be positive.

Column major	Must be at least m
Row major	Must be at least n

co Buffer holding the offset values for matrix C.

If `offset_type = offset::fix`, size of co array must be at least 1.

If `offset_type = offset::col`, size of co array must be at least $\max(1, m)$.

If `offset_type = offset::row`, size of co array must be at least $\max(1, n)$.

See [Data Types](#) for more details.

mode *Optional.* Compute mode settings. See [Compute Modes](#) for more details.

Output Parameters

c Output buffer overwritten by $\alpha * (\text{op}(A) - A_offset) * (\text{op}(B) - B_offset) + \beta * C + C_offset$.

NOTE If $\beta = 0$, matrix C does not need to be initialized before calling `gemm_bias`.

gemm_bias (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event gemm_bias(sycl::queue &queue,
        oneapi::mkl::transpose transa,
        oneapi::mkl::transpose transb,
        oneapi::mkl::offset offsetc,
        std::int64_t m,
        std::int64_t n,
        std::int64_t k,
        oneapi::mkl::value_or_pointer<float> alpha,
        const Ta *a,
        std::int64_t lda,
        Ta ao,
        const Tb *b,
        std::int64_t ldb,
        Tb bo,
        oneapi::mkl::value_or_pointer<float> beta,
        std::int32_t *c,
        std::int64_t ldc,
        const std::int32_t *co,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event gemm_bias(sycl::queue &queue,
        oneapi::mkl::transpose transa,
        oneapi::mkl::transpose transb,
        oneapi::mkl::offset offsetc,
        std::int64_t m,
        std::int64_t n,
        std::int64_t k,
        oneapi::mkl::value_or_pointer<float> alpha,
        const Ta *a,
        std::int64_t lda,
        Ta ao,
        const Tb *b,
        std::int64_t ldb,
        Tb bo,
        oneapi::mkl::value_or_pointer<float> beta,
        std::int32_t *c,
        std::int64_t ldc,
        const std::int32_t *co,
```

```

compute_mode mode = compute_mode::unset,
const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
transa	Specifies op(A), the transposition operation applied to matrix A. See Data Types for more details.
transb	Specifies op(B), the transposition operation applied to matrix B. See Data Types for more details.
offsetc	Specifies the form of C_offset used in the matrix multiplication. See Data Types for more details.
m	Number of rows of matrix op(A) and matrix C. Must be at least zero.
n	Number of columns of matrix op(B) and matrix C. Must be at least zero.
k	Number of columns of matrix op(A) and rows of matrix op(B). Must be at least zero.
alpha	Scaling factor for matrix-matrix product. See Scalar Arguments for more information on the value_or_pointer data type.
a	Pointer to input matrix A. See Matrix Storage for more details.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	A is $m \times k$ matrix. Size of array a must be at least $lda * k$	A is $k \times m$ matrix. Size of array a must be at least $lda * m$
Row major	A is $m \times k$ matrix. Size of array a must be at least $lda * m$	A is $k \times m$ matrix. Size of array a must be at least $lda * k$

lda Leading dimension of matrix A. Must be positive.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least m	Must be at least k
Row major	Must be at least k	Must be at least m

ao Specifies the scalar offset value for matrix A.

b Pointer to input matrix B. See [Matrix Storage](#) for more details.

	transb = transpose::nontrans	transb = transpose::trans or trans = transpose::conjtrans
Column major	B is $k \times n$ matrix. Size of array b must be at least $ldb * n$	B is $n \times k$ matrix. Size of array b must be at least $ldb * k$
Row major	B is $k \times n$ matrix. Size of array b must be at least $ldb * k$	B is $n \times k$ matrix. Size of array b must be at least $ldb * n$

ldb Leading dimension of matrix B. Must be positive.

	transb = transpose::nontrans	transb = transpose::trans or trans = transpose::conjtrans
Column major	Must be at least k	Must be at least n
Row major	Must be at least n	Must be at least k

bo Specifies the scalar offset value for matrix B.

beta Scaling factor for matrix C. See [Scalar Arguments](#) for more information on the `value_or_pointer` data type.

c Pointer to input/output matrix C. See [Matrix Storage](#) for more details.

Column major	C is $m \times n$ matrix. Size of array c must be at least $ldc * n$
Row major	C is $m \times n$ matrix. Size of array c must be at least $ldc * m$

ldc Leading dimension of matrix C. Must be positive.

Column major	Must be at least m
Row major	Must be at least n

co Pointer to array holding offset values for matrix C.

If `offset_type = offset::fix`, size of **co** array must be at least 1.

If `offset_type = offset::col`, size of **co** array must be at least $\max(1, m)$.

If `offset_type = offset::row`, size of **co** array must be at least $\max(1, n)$.

See [Data Types](#) for more details.

mode *Optional.* Compute mode settings. See [Compute Modes](#) for more details.

dependencies *Optional.* List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

`mode` and `dependencies` may be omitted independently; it is not necessary to specify `mode` in order to provide dependencies.

Output Parameters

c Pointer to output matrix `C` overwritten by $\alpha * (\text{op}(A) - A_offset) * (\text{op}(B) - B_offset) + \beta * C + C_offset$.

NOTE If `beta = 0`, matrix `C` does not need to be initialized before calling `gemm_bias`.

Return Values

Output event to wait on to ensure computation is complete.

gemmt

Computes a matrix-matrix product with general matrices, but updates only the upper or lower triangular part of the result matrix.

Description

The `gemmt` routines compute a scalar-matrix-matrix product and add the result to the upper or lower part of a scalar-matrix product, with general matrices. The operation is defined as:

$$C \leftarrow \alpha * \text{op}(A) * \text{op}(B) + \beta * C$$

where:

- `op(x)` is one of `op(x) = x`, or `op(x) = xT`, or `op(x) = xH`
- `alpha` and `beta` are scalars
- `A`, `B`, and `C` are matrices
- `op(A)` is `n × k`, `op(B)` is `k × n`, and `C` is `n × n`

`gemmt` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

gemmt (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void gemmt(sycl::queue &queue,
               oneapi::mkl::uplo upper_lower,
               oneapi::mkl::transpose transa,
               oneapi::mkl::transpose transb,
               std::int64_t n,
               std::int64_t k,
               T alpha,
               sycl::buffer<T,1> &a,
               std::int64_t lda,
               sycl::buffer<T,1> &b,
               std::int64_t ldb,
               T beta,
               sycl::buffer<T,1> &c,
               std::int64_t ldc,
               compute_mode mode = compute_mode::unset)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void gemmt(sycl::queue &queue,
               oneapi::mkl::uplo upper_lower,
               oneapi::mkl::transpose transa,
               oneapi::mkl::transpose transb,
               std::int64_t n,
               std::int64_t k,
               T alpha,
               sycl::buffer<T,1> &a,
               std::int64_t lda,
               sycl::buffer<T,1> &b,
               std::int64_t ldb,
               T beta,
               sycl::buffer<T,1> &c,
               std::int64_t ldc,
               compute_mode mode = compute_mode::unset)
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix C is upper or lower triangular. See Data Types for more details.
transa	Specifies $\text{op}(A)$, the transposition operation applied to matrix A. See Data Types for more details.
transb	Specifies $\text{op}(B)$, the transposition operation applied to matrix B. See Data Types for more details.
n	Number of rows of matrix $\text{op}(A)$ and matrix C. Must be at least zero.
k	Number of columns of matrix $\text{op}(A)$ and rows of matrix $\text{op}(B)$. Must be at least zero.
alpha	Scaling factor for matrix-matrix product.
a	Buffer holding input matrix A. See Matrix Storage for more details.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	A is $n \times k$ matrix. Size of array <i>a</i> must be at least $lda * k$	A is $k \times n$ matrix. Size of array <i>a</i> must be at least $lda * n$
Row major	A is $n \times k$ matrix. Size of array <i>a</i> must be at least $lda * n$	A is $k \times n$ matrix. Size of array <i>a</i> must be at least $lda * k$

lda

Leading dimension of matrix A. Must be positive.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least n	Must be at least k
Row major	Must be at least k	Must be at least n

b

Buffer holding input matrix B. See [Matrix Storage](#) for more details.

	transb = transpose::nontrans	transb = transpose::trans Or trans = transpose::conjtrans
Column major	B is $k \times n$ matrix. Size of array <i>b</i> must be at least $ldb * n$	B is $n \times k$ matrix. Size of array <i>b</i> must be at least $ldb * k$
Row major	B is $k \times n$ matrix. Size of array <i>b</i> must be at least $ldb * k$	B is $n \times k$ matrix. Size of array <i>b</i> must be at least $ldb * n$

ldb

Leading dimension of matrix B. Must be positive.

	transb = transpose::nontrans	transb = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least k	Must be at least n
Row major	Must be at least n	Must be at least k

beta Scaling factor for matrix **C**.

c Buffer holding input/output matrix **C**. See [Matrix Storage](#) for more details.

Column major	C is $m \times n$ matrix. Size of array c must be at least $ldc * n$
Row major	C is $m \times n$ matrix. Size of array c must be at least $ldc * m$

ldc Leading dimension of matrix **C**. Must be positive.

Column major	Must be at least m
Row major	Must be at least n

mode *Optional.* Compute mode settings. See [Compute Modes](#) for more details.

Output Parameters

c Output buffer overwritten by upper or lower triangular part of $\alpha * \text{op(A)} * \text{op(B)} + \text{beta} * \text{C}$.

NOTE If **beta** = 0, matrix **C** does not need to be initialized before calling `gemmt`.

gemmt (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event gemmt(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose transa,
        oneapi::mkl::transpose transb,
        std::int64_t n,
        std::int64_t k,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T* a,
        std::int64_t lda,
        const T* b,
        std::int64_t ldb,
        oneapi::mkl::value_or_pointer<T> beta,
        T* c,
        std::int64_t ldc,
        compute_mode mode = compute_mode::unset,
        const std::vector<syce::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event gemmt(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose transa,
        oneapi::mkl::transpose transb,
        std::int64_t n,
        std::int64_t k,
```

```

oneapi::mkl::value_or_pointer<T> alpha,
const T* a,
std::int64_t lda,
const T* b,
std::int64_t ldb,
oneapi::mkl::value_or_pointer<T> beta,
T* c,
std::int64_t ldc,
compute_mode mode = compute_mode::unset,
const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrix C is upper or lower triangular. See Data Types for more details.
transa	Specifies op(A), the transposition operation applied to matrix A. See Data Types for more details.
transb	Specifies op(B), the transposition operation applied to matrix B. See Data Types for more details.
n	Number of rows of matrix op(A) and matrix C. Must be at least zero.
k	Number of columns of matrix op(A) and rows of matrix op(B). Must be at least zero.
alpha	Scaling factor for matrix-matrix product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. See Matrix Storage for more details.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	A is $n \times k$ matrix. Size of array a must be at least $lda * k$	A is $k \times n$ matrix. Size of array a must be at least $lda * n$
Row major	A is $n \times k$ matrix. Size of array a must be at least $lda * n$	A is $k \times n$ matrix. Size of array a must be at least $lda * k$

lda	Leading dimension of matrix A. Must be positive.
-----	--

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtran s
Column major	Must be at least n	Must be at least k
Row major	Must be at least k	Must be at least n

b Pointer to input matrix B. See [Matrix Storage](#) for more details.

	transb = transpose::nontrans	transb = transpose::trans Or trans = transpose::conjtran s
Column major	B is $k \times n$ matrix. Size of array b must be at least $ldb * n$	B is $n \times k$ matrix. Size of array b must be at least $ldb * k$
Row major	B is $k \times n$ matrix. Size of array b must be at least $ldb * k$	B is $n \times k$ matrix. Size of array b must be at least $ldb * n$

ldb Leading dimension of matrix B. Must be positive.

	transb = transpose::nontrans	transb = transpose::trans Or trans = transpose::conjtran s
Column major	Must be at least k	Must be at least n
Row major	Must be at least n	Must be at least k

beta Scaling factor for matrix C. See [Scalar Arguments](#) for more information on the `value_or_pointer` data type.

c Pointer to input/output matrix C. See [Matrix Storage](#) for more details.

Column major	C is $m \times n$ matrix. Size of array c must be at least $ldc * n$
Row major	C is $m \times n$ matrix. Size of array c must be at least $ldc * m$

ldc Leading dimension of matrix C. Must be positive.

Column major	Must be at least m
Row major	Must be at least n

mode *Optional.* Compute mode settings. See [Compute Modes](#) for more details.

dependencies *Optional.* List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

`mode` and `dependencies` may be omitted independently; it is not necessary to specify `mode` in order to provide dependencies.

Output Parameters

c Pointer to output matrix `C` overwritten by upper or lower triangular part of `alpha * op(A)*op(B) + beta * C`.

NOTE If `beta = 0`, matrix `C` does not need to be initialized before calling `gemmt`.

Return Values

Output event to wait on to ensure computation is complete.

gemv_batch

Computes a group of `gemv` operations.

Description

The `gemv_batch` routines are batched versions of [gemv](#), performing multiple `gemv` operations in a single call. Each `gemv` operations perform a scalar-matrix-vector product and add the result to a scalar-vector product.

`gemv_batch` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

gemv_batch (Buffer Version)

Buffer version of `gemv_batch` supports only strided API.

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
  A is a matrix at offset i * stridea in a.
  X and Y are vectors at offset i * stridex, i * stridey, in x and y.
  Y = alpha * op(A) * X + beta * Y
end for
```

where:

- `op(A)` is one of `op(A) = A`, or `op(A) = AT`, or `op(A) = AH`

- `alpha` and `beta` are scalars
- `A` is matrix and `x` and `y` are vectors

For strided API, `x` and `y` buffers contain all the input vectors. The stride between vectors is either given by the stride parameters. Total number of vectors in `x` and `y` buffers is given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void gemv_batch(sycl::queue &queue,
                   oneapi::mkl::transpose trans,
                   std::int64_t m,
                   std::int64_t n,
                   T alpha,
                   sycl::buffer<T,1> &a,
                   std::int64_t lda,
                   std::int64_t stridea,
                   sycl::buffer<T,1> &x,
                   std::int64_t incx,
                   std::int64_t stridex,
                   T beta,
                   sycl::buffer<T,1> &y,
                   std::int64_t incy,
                   std::int64_t stridey,
                   std::int64_t batch_size)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void gemv_batch(sycl::queue &queue,
                   oneapi::mkl::transpose trans,
                   std::int64_t m,
                   std::int64_t n,
                   T alpha,
                   sycl::buffer<T,1> &a,
                   std::int64_t lda,
                   std::int64_t stridea,
                   sycl::buffer<T,1> &x,
                   std::int64_t incx,
                   std::int64_t stridex,
                   T beta,
                   sycl::buffer<T,1> &y,
                   std::int64_t incy,
                   std::int64_t stridey,
                   std::int64_t batch_size)
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>trans</code>	Specifies <code>op(A)</code> , the transposition operation applied to matrices <code>A</code> . See Data Types for more details.
<code>m</code>	Number of rows of matrices <code>op(A)</code> . Must be at least zero.
<code>n</code>	Number of columns of matrices <code>op(A)</code> . Must be at least zero.
<code>alpha</code>	Scaling factor for matrix-vector product.

<code>a</code>	The buffer holding input matrices <code>A</code> . Size of the buffer must be at least <code>stridea * batch_size</code> .
<code>lda</code>	Leading dimension of matrices <code>A</code> . Must be positive and at least <code>m</code> if column major layout or at least <code>n</code> if row major layout is used.
<code>stridea</code>	Stride between two consecutive <code>A</code> matrices. Must be at least zero.
<code>x</code>	Buffer holding input vectors <code>x</code> . Size of the buffer must be at least <code>stridex * batch_size</code> .
<code>incx</code>	Stride between two consecutive elements of <code>x</code> vectors. Must not be zero.
<code>stridex</code>	Stride between two consecutive <code>x</code> vectors. Must be at least zero.
<code>beta</code>	Scaling factor for vectors <code>y</code> .
<code>y</code>	Buffer holding input/output vectors <code>y</code> . Size of the buffer must be at least <code>stridey * batch_size</code> .
<code>incy</code>	Stride between two consecutive elements of <code>y</code> vectors. Must not be zero.
<code>stridey</code>	Stride between two consecutive <code>y</code> vectors. Must be at least $(1 + (m - 1) * \text{abs}(\text{incy}))$ if layout is column major or $(1 + (n - 1) * \text{abs}(\text{incy}))$ if row major layout is used.
<code>batch_size</code>	Number of <code>gemv</code> computations to perform. Must be at least zero.

Output Parameters

<code>y</code>	Output buffer overwritten by <code>batch_size</code> <code>gemv</code> operations of the form $\text{alpha} * \text{op}(A) * X + \text{beta} * Y$.
----------------	---

`gemv_batch` (USM Version)

USM version of `gemv_batch` supports the group API and the strided API.

Group API

The type `Ti` of integer pointers in the group API may be either `std::int64_t` or `std::int32_t`.

Group API operation is defined as:

```

idx = 0
for i = 0 ... group_count - 1
  for j = 0 ... group_size - 1
    A is an m x n matrix in a[idx]
    X and Y are vectors in x[idx] and y[idx]
    Y = alpha[i] * op(A) * X + beta[i] * Y
    idx = idx + 1
  end for
end for

```

where:

- `op(A)` is one of `op(A) = A`, or `op(A) = AT`, or `op(A) = AH`
- `alpha` and `beta` are scalars
- `A` is matrix and `x` and `y` are vectors

For group API, x and y arrays contain the pointers for all the input vectors. a array contains the pointers to all input matrices. The total number of vectors in x and y and matrices in a is given by:

$$total_batch_count = \sum_{i=0}^{group_count-1} group_size[i]$$

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event gemv_batch(sycl::queue &queue,
                          const oneapi::mkl::transpose *trans,
                          const Ti *m,
                          const Ti *n,
                          const T *alpha,
                          const T **a,
                          const Ti *lda,
                          const T **x,
                          const Ti *incx,
                          const T *beta,
                          T **y,
                          const Ti *incy,
                          std::int64_t group_count,
                          const Ti *group_size,
                          const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event gemv_batch(sycl::queue &queue,
                          const oneapi::mkl::transpose *trans,
                          const Ti *m,
                          const Ti *n,
                          const T *alpha,
                          const T **a,
                          const Ti *lda,
                          const T **x,
                          const Ti *incx,
                          const T *beta,
                          T **y,
                          const Ti *incy,
                          std::int64_t group_count,
                          const Ti *group_size,
                          const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
trans	Array of <code>group_count</code> <code>oneapi::mkl::transpose</code> values. <code>transa[i]</code> specifies <code>op(A)</code> , the transposition operation applied to matrices A in group i . See Data Types for more details.
m	Array of <code>group_count</code> integers. <code>m[i]</code> specifies number of rows of matrices <code>op(A)</code> in group i . All entries must be at least zero.

<code>n</code>	Array of <code>group_count</code> integers. <code>n[i]</code> specifies number of columns of matrices <code>op(A)</code> in group <code>i</code> . All entries must be at least zero.
<code>alpha</code>	Array of <code>group_count</code> scalar elements. <code>alpha[i]</code> specifies scaling factor for matrix-vector products in group <code>i</code> .
<code>a</code>	Array of <code>total_batch_count</code> pointers for input matrices <code>A</code> . See Matrix Storage for more details.
<code>lda</code>	Array of <code>group_count</code> integers. <code>lda[i]</code> specifies leading dimension of matrices <code>A</code> in group <code>i</code> . Must be positive and at least <code>m[i]</code> if column major layout or at least <code>n[i]</code> if row major layout is used.
<code>x</code>	Array of <code>total_batch_count</code> pointers for input vectors <code>x</code> . See Matrix Storage for more details.
<code>incx</code>	Array of <code>group_count</code> integers. <code>incx[i]</code> specifies stride of vectors <code>x</code> in group <code>i</code> . Must not be zero.
<code>beta</code>	Array of <code>group_count</code> scalar elements. <code>beta[i]</code> specifies scaling factor for vectors <code>y</code> in group <code>i</code> .
<code>y</code>	Array of <code>total_batch_count</code> pointers for input/output vectors <code>y</code> . See Matrix Storage for more details.
<code>incy</code>	Array of <code>group_count</code> integers. <code>incy[i]</code> specifies stride of vectors <code>y</code> in group <code>i</code> . Must not be zero.
<code>group_count</code>	Number of groups. Must be at least zero.
<code>group_size</code>	Array of <code>group_count</code> integers. <code>group_size[i]</code> specifies the number of <code>gemv</code> operations in group <code>i</code> . Each element in <code>group_size</code> must be at least zero.
<code>dependencies</code>	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

<code>y</code>	Array of pointers to output vectors <code>y</code> overwritten by <code>total_batch_count</code> <code>gemv</code> operations of the form $\alpha * op(A) * x + \beta * y$.
----------------	--

Return Values

Output event to wait on to ensure computation is complete.

Examples

An example of how to use USM version of `gemv_batch` can be found in oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/blas/source/gemv_batch_usm.cpp
```

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
    A is a matrix at offset i * stridea in a.
    X and Y are vectors at offset i * stridex, i * stridey, in x and y.
    Y = alpha * op(A) * X + beta * Y
end for
```

where:

- $\text{op}(A)$ is one of $\text{op}(A) = A$, or $\text{op}(A) = A^T$, or $\text{op}(A) = A^H$
- α and β are scalars
- A is matrix and X and Y are vectors

For strided API, x and y arrays contain all the input vectors. The stride between vectors is given by the stride parameters. Total number of vectors in x and y arrays is given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event gemv_batch(sycl::queue &queue,
        oneapi::mkl::transpose trans,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        std::int64_t stridea,
        const T *x,
        std::int64_t incx,
        std::int64_t stridex,
        oneapi::mkl::value_or_pointer<T> beta,
        T *y,
        std::int64_t incy,
        std::int64_t stridey,
        std::int64_t batch_size)
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event gemv_batch(sycl::queue &queue,
        oneapi::mkl::transpose trans,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        std::int64_t stridea,
        const T *x,
        std::int64_t incx,
        std::int64_t stridex,
        oneapi::mkl::value_or_pointer<T> beta,
        T *y,
        std::int64_t incy,
        std::int64_t stridey,
        std::int64_t batch_size)
}
```

Input Parameters

queue The queue where the routine should be executed.

trans	Specifies $\text{op}(A)$, the transposition operation applied to matrices A . See Data Types for more details.
m	Number of rows of matrices $\text{op}(A)$. Must be at least zero.
n	Number of columns of matrices $\text{op}(A)$. Must be at least zero.
alpha	Scaling factor for matrix-vector product. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrices A . Size of the array must be at least <code>stridea * batch_size</code> .
lda	Leading dimension of matrices A . Must be positive and at least m if column major layout or at least n if row major layout is used.
stridea	Stride between two consecutive A matrices. Must be at least zero.
x	Pointer to input vectors X . Size of the array must be at least <code>stridex * batch_size</code> .
incx	Stride between two consecutive elements of X vectors. Must not be zero.
stridex	Stride between two consecutive X vectors. Must be at least zero.
beta	Scaling factor for vectors Y . See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
y	Pointer to input/output vectors Y . Size of the array must be at least <code>stridey * batch_size</code> .
incy	Stride between two consecutive elements of Y vectors. Must not be zero.
stridey	Stride between two consecutive Y vectors. Must be at least $(1 + (m - 1) * \text{abs}(\text{incy}))$ if layout is column major or $(1 + (n - 1) * \text{abs}(\text{incy}))$ if row major layout is used.
batch_size	Number of <code>gemv</code> computations to perform. Must be at least zero.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

y	Pointer to output vectors Y overwritten by <code>batch_size</code> <code>gemv</code> operations of the form $\text{alpha} * \text{op}(A) * X + \text{beta} * Y$.
---	---

Return Values

Output event to wait on to ensure computation is complete.

[syrk_batch](#)

Computes a group of `syrk` operations.

Description

The `syrk_batch` routines are batched versions of [syrk](#), performing multiple `syrk` operations in a single call. Each `syrk` operation performs a rank-k update with general matrices.

`syrk_batch` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

`syrk_batch` (Buffer Version)

Buffer version of `syrk_batch` supports only strided API.

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
    A and C are matrices at offset i * stridea and i * stridec in a and c.
    C = alpha * op(A) * op(A)^T + beta * C
end for
```

where:

- `op(x)` is one of `op(x) = x`, or `op(x) = xT`, or `op(x) = xH`
- `alpha` and `beta` are scalars
- `A` is general matrix and `C` is symmetric matrix
- `op(A)` is $n \times k$ and `C` is $n \times n$

For strided API, `a` and `c` buffers contain all the input matrices. The stride between matrices is given by the stride parameters. Total number of matrices in `a` and `c` buffers is given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void syrk_batch(sycl::queue &queue,
                   oneapi::mkl::uplo upper_lower,
                   oneapi::mkl::transpose trans,
                   std::int64_t n,
                   std::int64_t k,
                   T alpha,
                   sycl::buffer<T,1> &a,
                   std::int64_t lda,
                   std::int64_t stridea,
                   T beta,
                   sycl::buffer<T,1> &c,
                   std::int64_t ldc,
                   std::int64_t stridec,
```

```

        std::int64_t batch_size,
        compute_mode mode = compute_mode::unset)
}

namespace oneapi::mkl::blas::row_major {
    void syrkh_batch(sycl::queue &queue,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose trans,
        std::int64_t n,
        std::int64_t k,
        T alpha,
        sycl::buffer<T,1> &a,
        std::int64_t lda,
        std::int64_t stridea,
        T beta,
        sycl::buffer<T,1> &c,
        std::int64_t ldc,
        std::int64_t stridec,
        std::int64_t batch_size,
        compute_mode mode = compute_mode::unset)
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrices C are upper or lower triangular. See Data Types for more details.
trans	Specifies $op(A)$, transposition operation applied to matrices A . Conjugation is never performed even if <code>trans = transpose::conjtrans</code> . See Data Types for more details.
n	Number of rows and columns of matrices C . Must be at least zero.
k	Number of columns of matrices $op(A)$. Must be at least zero.
alpha	Scaling factor for rank-k update.
a	Buffer holding input matrices A . Size of the buffer must be at least <code>stridea * batch_size</code> .
lda	Leading dimension of matrices A . Must be positive.

	<code>transa = transpose::nontrans</code>	<code>transa = transpose::trans</code> or <code>trans = transpose::conjtrans</code>
Column major	Must be at least <i>n</i>	Must be at least <i>k</i>
Row major	Must be at least <i>k</i>	Must be at least <i>n</i>

stridea	Stride between two consecutive A matrices.
---------	---

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least $lda * k$	Must be at least $lda * n$
Row major	Must be at least $lda * n$	Must be at least $lda * k$

beta	Scaling factor for matrices C .
c	Buffer holding input/output matrices C . Size of the buffer must be at least $stridec * batch_size$.
ldc	Leading dimension of matrices C . Must be positive and at least n .
stridec	Stride between two consecutive C matrices. Must be least $ldc * n$.
batch_size	Specifies the number of matrix multiply operations to perform.
mode	<i>Optional</i> . Compute mode settings. See Compute Modes for more details.

Output Parameters

c	Output buffer overwritten by <code>batch_sizesyrk</code> operations of the form $\alpha * op(A) * op(A)^T + \beta * C$.
----------	--

syrk_batch (USM Version)

USM version of `syrk_batch` supports group API and strided API.

Group API

The type **Ti** of integer pointers in the group API may be either `std::int64_t` or `std::int32_t`.

Group API operation is defined as:

```

idx = 0
for i = 0 ... group_count - 1
    for j = 0 ... group_size - 1
        A, and C are matrices in a[idx] and c[idx]
        C = alpha[i] * op(A) * op(A)^T + beta[i] * C
        idx := idx + 1
    end for
end for

```

where:

- $op(x)$ is one of $op(x) = x$, or $op(x) = x^T$, or $op(x) = x^H$
- α and β are scalars
- **A** is general matrix and **C** is symmetric matrix
- $op(A)$ is $n \times k$ and **C** is $n \times n$

For group API, **a** and **c** arrays contain the pointers for all the input matrices. The total number of matrices in **a** and **c** are given by:

$$total_batch_count = \sum_{i=0}^{group_count-1} group_size[i]$$

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event syrk_batch(sycl::queue &queue,
                          const oneapi::mkl::uplo *upper_lower,
                          const oneapi::mkl::transpose *trans,
                          const Ti *n,
                          const Ti *k,
                          const T *alpha,
                          const T **a,
                          const Ti *lda,
                          const T *beta,
                          T **c,
                          const Ti *ldc,
                          std::int64_t group_count,
                          const Ti *group_size,
                          compute_mode mode = compute_mode::unset,
                          const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event syrk_batch(sycl::queue &queue,
                          const oneapi::mkl::uplo *upper_lower,
                          const oneapi::mkl::transpose *trans,
                          const Ti *n,
                          const Ti *k,
                          const T *alpha,
                          const T **a,
                          const Ti *lda,
                          const T *beta,
                          T **c,
                          const Ti *ldc,
                          std::int64_t group_count,
                          const Ti *group_size,
                          compute_mode mode = compute_mode::unset,
                          const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Array of <code>group_count</code> <code>oneapi::mkl::uplo</code> values. <code>upper_lower[i]</code> specifies whether matrices <code>C</code> are upper or lower triangular in group <code>i</code> . See Data Types for more details.
trans	Array of <code>group_count</code> <code>oneapi::mkl::transpose</code> values. <code>trans[i]</code> specifies <code>op(A)</code> , transposition operation applied to matrices <code>A</code> in group <code>i</code> . See Data Types for more details.
n	Array of <code>group_count</code> integers. <code>n[i]</code> specifies number of rows and columns of matrices <code>C</code> in group <code>i</code> . All entries must be at least zero.

k Array of `group_count` integers. `k[i]` specifies number of columns of matrices `op(A)` in group `i`. All entries must be at least zero.

alpha Array of `group_count` scalar elements. `alpha[i]` specifies scaling factor for every rank-`k` update in group `i`.

a Array of `total_batch_count` pointers for input matrices `A`. See [Matrix Storage](#) for more details.

	<code>trans = transpose::nontrans</code>	<code>trans = transpose::trans</code> or <code>trans = transpose::conjtrans</code>
Column major	Size of array <code>A[i]</code> must be at least <code>lda[i] * k[i]</code>	Size of array <code>A[i]</code> must be at least <code>lda[i] * n[i]</code>
Row major	Size of array <code>A[i]</code> must be at least <code>lda[i] * n[i]</code>	Size of array <code>A[i]</code> must be at least <code>lda[i] * k[i]</code>

lda Array of `group_count` integers. `lda[i]` specifies leading dimension of matrices `A` in group `i`. Must be positive.

	<code>trans = transpose::nontrans</code>	<code>trans = transpose::trans</code> or <code>trans = transpose::conjtrans</code>
Column major	Must be at least <code>n[i]</code> .	Must be at least <code>k[i]</code> .
Row major	Must be at least <code>k[i]</code> .	Must be at least <code>n[i]</code> .

beta Array of `group_count` scalar elements. `beta[i]` specifies scaling factor for matrices `C` in group `i`.

c Array of `total_batch_count` pointers for input/output matrices `C`. Size of array `C[i]` must be at least `ldc[i] * n[i]`. See [Matrix Storage](#) for more details.

ldc Array of `group_count` integers. `ldc[i]` specifies leading dimension of matrices `C` in group `i`. Must be positive.

group_count Number of groups. Must be at least zero.

group_size Array of `group_count` integers. `group_size[i]` specifies the number of `syrk` operations in group `i`. Each element in `group_size` must be at least zero.

mode *Optional*. Compute mode settings. See [Compute Modes](#) for more details.

dependencies *Optional*. List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

mode and dependencies may be omitted independently; it is not necessary to specify mode in order to provide dependencies.

Output Parameters

c Array of pointers to output matrices **C** overwritten by total_batch_countsyрк operations of the form $\alpha * \text{op}(A) * \text{op}(A)^T + \beta * C$.

Return Values

Output event to wait on to ensure computation is complete.

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
    A and C are matrices at offset i * stridea and i * stridec in a and c.
    C = alpha * op(A) * op(A)^T + beta * C
end for
```

where:

- $\text{op}(X)$ is one of $\text{op}(X) = X$, or $\text{op}(X) = X^T$, or $\text{op}(X) = X^H$
- α and β are scalars
- **A** is general matrix and **C** is symmetric matrix
- $\text{op}(A)$ is $n \times k$ and **C** is $n \times n$

For strided API, **a** and **c** arrays contain all the input matrices. The stride between matrices is given by the stride parameters. Total number of matrices in **a** and **c** arrays is given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event syrk_batch(sycl::queue &queue,
                          oneapi::mkl::uplo upper_lower,
                          oneapi::mkl::transpose trans,
                          std::int64_t n,
                          std::int64_t k,
                          oneapi::mkl::value_or_pointer<T> alpha,
                          const T *a,
                          std::int64_t lda,
                          std::int64_t stridea,
                          oneapi::mkl::value_or_pointer<T> beta,
                          T *c,
                          std::int64_t ldc,
                          std::int64_t stridec,
                          std::int64_t batch_size,
                          compute_mode mode = compute_mode::unset,
                          const std::vector<sycl::event> &dependencies = {})
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event syrk_batch(sycl::queue &queue,
                          oneapi::mkl::uplo upper_lower,
                          oneapi::mkl::transpose trans,
                          std::int64_t n,
                          std::int64_t k,
                          oneapi::mkl::value_or_pointer<T> alpha,
```



```

const T *a,
std::int64_t lda,
std::int64_t stridea,
oneapi::mkl::value_or_pointer<T> beta,
T *c,
std::int64_t ldc,
std::int64_t stridec,
std::int64_t batch_size,
compute_mode mode = compute_mode::unset,
const std::vector<sycl::event> &dependencies = {})
}

```

Input Parameters

queue	The queue where the routine should be executed.
upper_lower	Specifies whether matrices C are upper or lower triangular. See Data Types for more details.
trans	Specifies $op(A)$, transposition operation applied to matrices A . Conjugation is never performed even if <code>trans = transpose::conjtrans</code> . See Data Types for more details.
n	Number of rows and columns of matrices C . Must be at least zero.
k	Number of columns of matrices $op(A)$. Must be at least zero.
alpha	Scaling factor for rank-k update. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrices A . Size of the array must be at least <code>stridea * batch_size</code> .
lda	Leading dimension of matrices A . Must be positive.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least <i>n</i>	Must be at least <i>k</i>
Row major	Must be at least <i>k</i>	Must be at least <i>n</i>

stridea Stride between two consecutive **A** matrices.

	transa = transpose::nontrans	transa = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least <code>lda * k</code>	Must be at least <code>lda * n</code>

	<code>transa = transpose::nontrans</code>	<code>transa = transpose::trans</code> Or <code>trans = transpose::conjtrans</code>
Row major	Must be at least <code>lda * n</code>	Must be at least <code>lda * k</code>

beta	Scaling factor for matrices C. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
c	Pointer to input/output matrices C. Size of the array must be at least <code>stridec * batch_size</code> .
ldc	Leading dimension of matrices C. Must be positive and at least <code>n</code> .
stridec	Stride between two consecutive C matrices. Must be least <code>ldc * n</code> .
batch_size	Specifies the number of matrix multiply operations to perform.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
dependencies	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.

Output Parameters

c	Pointer to output matrices C overwritten by <code>batch_sizesyrk</code> operations of the form $\alpha * \text{op}(A) * \text{op}(A)^T + \text{beta} * C$.
---	---

Return Values

Output event to wait on to ensure computation is complete.

trsm_batch

Computes a group of `trsm` operations.

Description

The `trsm_batch` routines are batched versions of [trsm](#), performing multiple `trsm` operations in a single call. Each `trsm` solves an equation of the form $\text{op}(A) * X = \alpha * B$ or $X * \text{op}(A) = \alpha * B$.

`trsm_batch` supports the following precisions:

T
float
double
<code>std::complex<float></code>

T

```
std::complex<double>
```

trsm_batch (Buffer Version)

Buffer version of trsm_batch supports only strided API.

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
    A and B are matrices at offset i * stridea and i * strideb in a and b.
    if (left_right == side::left) then
        compute X such that op(A) * X = alpha * B
    else
        compute X such that X * op(A) = alpha * B
    B = X
end for
```

where:

- op(A) is one of $op(A) = A$, or $op(A) = A^T$, or $op(A) = A^H$
- alpha is a scalar
- A is either $m \times m$ or $n \times n$ triangular matrix
- B and X are $m \times n$ general matrices

On return, matrix B is overwritten by solution matrix X.

For strided API, a and b buffers contains all the input matrices. The stride between matrices is given by the stride parameters. Total number of matrices in a and b buffers is given by batch_size parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void trsm_batch(sycl::queue &queue,
                   oneapi::mkl::side left_right,
                   oneapi::mkl::uplo upper_lower,
                   oneapi::mkl::transpose trans,
                   oneapi::mkl::diag unit_diag,
                   std::int64_t m,
                   std::int64_t n,
                   T alpha,
                   sycl::buffer<T,1> &a,
                   std::int64_t lda,
                   std::int64_t stridea,
                   sycl::buffer<T,1> &b,
                   std::int64_t ldb,
                   std::int64_t strideb,
                   std::int64_t batch_size,
                   compute_mode mode = compute_mode::unset)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void trsm_batch(sycl::queue &queue,
                   oneapi::mkl::side left_right,
                   oneapi::mkl::uplo upper_lower,
                   oneapi::mkl::transpose trans,
                   oneapi::mkl::diag unit_diag,
```

```

std::int64_t m,
std::int64_t n,
T alpha,
sycl::buffer<T,1> &a,
std::int64_t lda,
std::int64_t stridea,
sycl::buffer<T,1> &b,
std::int64_t ldb,
std::int64_t strideb,
std::int64_t batch_size,
compute_mode mode = compute_mode::unset)
}

```

Input Parameters

queue	The queue where the routine should be executed.
left_right	Specifies whether matrices A are on the left side or right side of the multiplication. See Data Types for more details.
upper_lower	Specifies whether matrices A are upper or lower triangular. See Data Types for more details.
trans	Specifies $\text{op}(\mathbf{A})$, transposition operation applied to matrices A . See Data Types for more details.
unit_diag	Specifies whether matrices A are unit triangular or not. See Data Types for more details.
m	Number of rows of matrices B . Must be at least zero.
n	Number of columns of matrices B . Must be at least zero.
alpha	Scaling factor for the solution.
a	Buffer holding input matrices A . Size of the buffer must be at least <code>stridea * batch_size</code> .
lda	Leading dimension of matrices A . Must be at least <code>m</code> if <code>left_right = side::left</code> or at least <code>n</code> if <code>left_right = side::right</code> . Must be positive.
stridea	Stride between two consecutive A matrices.
b	Buffer holding input/output matrices B . Size of the buffer must be at least <code>strideb * batch_size</code> .
ldb	Leading dimension of matrices B . Must be at least <code>m</code> if column major layout or at least <code>n</code> if row major layout is used. Must be positive.
strideb	Stride between two consecutive B matrices.
batch_size	Specifies number of triangular linear systems to solve.
mode	<i>Optional</i> . Compute mode settings. See Compute Modes for more details.

Output Parameters

b	Output buffer overwritten by <code>batch_size</code> solution matrices X .
---	---

NOTE If $\alpha = 0$, matrices B are set to zero, and A and B do not need to be initialized before calling `trsm_batch..`

`trsm_batch` (USM Version)

USM version of `trsm_batch` supports group API and strided API.

Group API

The type T_i of integer pointers in the group API may be either `std::int64_t` or `std::int32_t`.

Group API operation is defined as:

```

idx = 0
for i = 0 ... group_count - 1
  for j = 0 ... group_size - 1
    A and B are matrices in a[idx] and b[idx]
    if (left_right == side::left) then
      compute X such that op(A) * X = alpha[i] * B
    else
      compute X such that X * op(A) = alpha[i] * B
    end if
    B = X
    idx = idx + 1
  end for
end for

```

where:

- $\text{op}(A)$ is one of $\text{op}(A) = A$, or $\text{op}(A) = A^T$, or $\text{op}(A) = A^H$
- α is a scalar
- A is either $m \times m$ or $n \times n$ triangular matrix
- B and X are $m \times n$ general matrices

On return, matrix B is overwritten by solution matrix X .

For group API, a and b arrays contain the pointers for all the input matrices. The total number of matrices in a and b are given by:

$$total_batch_count = \sum_{i=0}^{group_count-1} group_size[i]$$

Syntax

```

namespace oneapi::mkl::blas::column_major {
  sycl::event trsm_batch(sycl::queue &queue,
                        const oneapi::mkl::side *left_right,
                        const oneapi::mkl::uplo *upper_lower,
                        const oneapi::mkl::transpose *trans,
                        const oneapi::mkl::diag *unit_diag,
                        const Ti *m,
                        const Ti *n,
                        const T *alpha,
                        const T **a,
                        const Ti *lda,
                        T **b,

```

```

        const Ti *ldb,
        std::int64_t group_count,
        const Ti *group_size,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {})
    }

namespace oneapi::mkl::blas::row_major {
    sycl::event trsm_batch(sycl::queue &queue,
        const oneapi::mkl::side *left_right,
        const oneapi::mkl::uplo *upper_lower,
        const oneapi::mkl::transpose *trans,
        const oneapi::mkl::diag *unit_diag,
        const Ti *m,
        const Ti *n,
        const T *alpha,
        const T **a,
        const Ti *lda,
        T **b,
        const Ti *ldb,
        std::int64_t group_count,
        const Ti *group_size,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {})
    }
}

```

Input Parameters

queue	The queue where the routine should be executed.
left_right	Array of <code>group_count</code> <code>oneapi::mkl::side</code> values. <code>left_right[i]</code> specifies whether matrices <code>A</code> are on the left side or right side of the multiplication in group <code>i</code> . See Data Types for more details.
upper_lower	Array of <code>group_count</code> <code>oneapi::mkl::uplo</code> values. <code>upper_lower[i]</code> specifies whether matrices <code>A</code> are upper or lower triangular in group <code>i</code> . See Data Types for more details.
trans	Array of <code>group_count</code> <code>oneapi::mkl::transpose</code> values. <code>trans[i]</code> specifies <code>op(A)</code> , transposition operation applied to matrices <code>A</code> in each group <code>i</code> . See Data Types for more details.
unit_diag	Array of <code>group_count</code> <code>oneapi::mkl::diag</code> values. <code>unit_diag[i]</code> specifies whether matrices <code>A</code> are unit triangular or not. See Data Types for more details.
m	Array of <code>group_count</code> integers. <code>m[i]</code> specifies number of rows of matrices <code>B</code> in group <code>i</code> . All entries must be at least zero.
n	Array of <code>group_count</code> integers. <code>n[i]</code> specifies number of columns of matrices <code>B</code> in group <code>i</code> . All entries must be at least zero.
alpha	Array of <code>group_count</code> scalar elements. <code>alpha[i]</code> specifies scaling factors for the solutions in group <code>i</code> .
a	Array of <code>total_batch_count</code> pointers for input matrices <code>A</code> . See Matrix Storage for more details.

<code>lda</code>	Array of <code>group_count</code> integers. <code>lda[i]</code> specifies leading dimension of matrices <code>A</code> in group <code>i</code> . Must be at least <code>m[i]</code> if <code>left_right[i] = side::left</code> or at least <code>n[i]</code> if <code>left_right[i] = side::right</code> . All entries must be positive.
<code>b</code>	Array of <code>total_batch_count</code> pointers for input/output matrices <code>B</code> . See Matrix Storage for more details.
<code>ldb</code>	Array of <code>group_count</code> integers. <code>ldb[i]</code> specifies leading dimension of matrices <code>B</code> in group <code>i</code> . Must be at least <code>m[i]</code> if column major layout or at least <code>n[i]</code> if row major layout is used. All entries must be positive.
<code>group_count</code>	Number of groups. Must be at least zero.
<code>group_size</code>	Array of <code>group_count</code> integers. <code>group_size[i]</code> specifies the number of <code>trsm</code> operations in group <code>i</code> . Each element in <code>group_size</code> must be at least zero.
<code>mode</code>	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
<code>dependencies</code>	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.

Output Parameters

<code>b</code>	Array of pointers to output matrices <code>B</code> overwritten by <code>total_batch_count</code> solution matrices <code>X</code> .
----------------	--

NOTE If `alpha = 0`, matrices `B` are set to zero, and `A` and `B` do not need to be initialized before calling `trsm_batch..`

Return Values

Output event to wait on to ensure computation is complete.

Strided API

Strided API operation is defined as:

```
for i = 0 ... batch_size - 1
    A and B are matrices at offset i * stridea and i * strideb in a and b.
    if (left_right == side::left) then
        compute X such that op(A) * X = alpha * B
    else
        compute X such that X * op(A) = alpha * B
    B = X
end for
```

where:

- `op(A)` is one of `op(A) = A`, or `op(A) = AT`, or `op(A) = AH`
- `alpha` is a scalar
- `A` is either `m × m` or `n × n` triangular matrix
- `B` and `X` are `m × n` general matrices

On return, matrix `B` is overwritten by solution matrix `X`.

For strided API, `a` and `b` arrays contain all the input matrices. The stride between matrices is given by the stride parameters. Total number of matrices in `a` and `b` arrays is given by `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event trsm_batch(sycl::queue &queue,
        oneapi::mkl::side left_right,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose trans,
        oneapi::mkl::diag unit_diag,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        std::int64_t stridea,
        T *b,
        std::int64_t ldb,
        std::int64_t strideb,
        std::int64_t batch_size,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {})
}

namespace oneapi::mkl::blas::row_major {
    sycl::event trsm_batch(sycl::queue &queue,
        oneapi::mkl::side left_right,
        oneapi::mkl::uplo upper_lower,
        oneapi::mkl::transpose trans,
        oneapi::mkl::diag unit_diag,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        std::int64_t stridea,
        T *b,
        std::int64_t ldb,
        std::int64_t strideb,
        std::int64_t batch_size,
        compute_mode mode = compute_mode::unset,
        const std::vector<sycl::event> &dependencies = {})
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>left_right</code>	Specifies whether matrices <code>A</code> are on the left side or right side of the multiplication. See Data Types for more details.
<code>upper_lower</code>	Specifies whether matrices <code>A</code> are upper or lower triangular. See Data Types for more details.
<code>trans</code>	Specifies <code>op(A)</code> , transposition operation applied to matrices <code>A</code> . See Data Types for more details.

unit_diag	Specifies whether matrices A are unit triangular or not. See Data Types for more details.
m	Number of rows of matrices B. Must be at least zero.
n	Number of columns of matrices B. Must be at least zero.
alpha	Scaling factor for the solution. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrices A. Size of the array must be at least <code>stridea * batch_size</code> .
lda	Leading dimension of matrices A. Must be at least m if <code>left_right = side::left</code> or at least n if <code>left_right = side::right</code> . Must be positive.
stridea	Stride between two consecutive A matrices.
b	Pointer to input/output matrices B. Size of the array must be at least <code>strideb * batch_size</code> .
ldb	Leading dimension of matrices B. Must be at least m if column major layout or at least n if row major layout is used. Must be positive.
strideb	Stride between two consecutive B matrices.
batch_size	Specifies number of triangular linear systems to solve.
mode	<i>Optional.</i> Compute mode settings. See Compute Modes for more details.
dependencies	<i>Optional.</i> List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies. <code>mode</code> and <code>dependencies</code> may be omitted independently; it is not necessary to specify <code>mode</code> in order to provide dependencies.

Output Parameters

b	Pointer to output matrix B overwritten by <code>batch_size</code> solution matrices X.
---	--

NOTE If `alpha = 0`, matrices B are set to zero, and A and B do not need to be initialized before calling `trsm_batch..`

Return Values

Output event to wait on to ensure computation is complete.

omatcopy

Computes an out-of-place scaled matrix transpose or copy operation using a general matrix.

Description

The `omatcopy` routine performs an out-of-place scaled matrix copy or transposition. The operation is defined as:

$$B \leftarrow \alpha * op(A)$$

where:

- $op(x)$ is one of $op(x) = x$, or $op(x) = x^T$, or $op(x) = x^H$
- α is a scalar
- A and B are matrices
- A is $m \times n$ matrix
- B is $m \times n$ matrix if op is non-transpose and an $n \times m$ matrix otherwise.

`omatcopy` supports the following precisions:

T
float
double
<code>std::complex<float></code>
<code>std::complex<double></code>

omatcopy (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void omatcopy(sycl::queue &queue,
                  oneapi::mkl::transpose trans,
                  std::int64_t m,
                  std::int64_t n,
                  T alpha,
                  sycl::buffer<T, 1> &a,
                  std::int64_t lda,
                  sycl::buffer<T, 1> &b,
                  std::int64_t ldb);
}
```

```
namespace oneapi::mkl::blas::row_major {
    void omatcopy(sycl::queue &queue,
                  oneapi::mkl::transpose trans,
                  std::int64_t m,
                  std::int64_t n,
                  T alpha,
                  sycl::buffer<T, 1> &a,
                  std::int64_t lda,
                  sycl::buffer<T, 1> &b,
                  std::int64_t ldb);
}
```

Input Parameters

`queue` The queue where the routine will be executed.

trans	Specifies $\text{op}(A)$, the transposition operation applied to the matrix A . See Data Types for more details.
m	Number of rows for the matrix A . Must be at least zero.
n	Number of columns for the matrix A . Must be at least zero.
alpha	Scaling factor for the matrix transposition or copy.
a	Buffer holding the input matrix A . Must have size at least $\text{lda} * n$ for column-major and at least $\text{lda} * m$ for row-major.
lda	Leading dimension of the matrix A . If matrices are stored using column major layout, lda must be at least m . If matrices are stored using row major layout, lda must be at least n . Must be positive.
ldb	Leading dimension of the matrix B . Must be positive.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least m	Must be at least n
Row major	Must be at least n	Must be at least m

Output Parameters

b	Output buffer, overwritten by $\text{alpha} * \text{op}(A)$.
---	---

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	B is $m \times n$ matrix. Size of array b must be at least $\text{ldb} * n$	B is $n \times m$ matrix. Size of array a must be at least $\text{ldb} * m$
Row major	B is $m \times n$ matrix. Size of array b must be at least $\text{ldb} * m$	B is $n \times m$ matrix. Size of array b must be at least $\text{ldb} * n$

omatcopy (USM Version)

Syntax

USM arrays:

```
namespace oneapi::mkl::blas::column_major {
    sycl::event omatcopy(sycl::queue &queue,
        oneapi::mkl::transpose trans,
        std::int64_t m,
```

```

        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        T *b,
        std::int64_t ldb,
        const std::vector<sycl::event> &dependencies = {});
}

namespace oneapi::mkl::blas::row_major {
    sycl::event omatcopy(sycl::queue &queue,
        oneapi::mkl::transpose trans,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        T *b,
        std::int64_t ldb,
        const std::vector<sycl::event> &dependencies = {});
}

```

Input Parameters

queue	The queue where the routine will be executed.
trans	Specifies $\text{op}(A)$, the transposition operation applied to matrix A . See Data Types for more details.
m	Number of rows for the matrix A . Must be at least zero.
n	Number of columns for the matrix A . Must be at least zero.
alpha	Scaling factor for the matrix transposition or copy. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A . Must have size at least $\text{lda} * n$ for column-major and at least $\text{lda} * m$ for row-major.
lda	Leading dimension of the matrix A . If matrices are stored using column major layout, lda must be at least m . If matrices are stored using row major layout, lda must be at least n . Must be positive.
ldb	Leading dimension of the matrix B . Must be positive.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least m	Must be at least n
Row major	Must be at least n	Must be at least m

dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.
--------------	--

Output Parameters

b Pointer to output matrix B overwritten by $\alpha * \text{op}(A)$.

	trans = transpose::nontrans	trans = transpose::trans or trans = transpose::conjtrans
Column major	B is $m \times n$ matrix. Size of array b must be at least $\text{ldb} * n$	B is $n \times m$ matrix. Size of array a must be at least $\text{ldb} * m$
Row major	B is $m \times n$ matrix. Size of array b must be at least $\text{ldb} * m$	B is $n \times m$ matrix. Size of array b must be at least $\text{ldb} * n$

Return Values

Output event to wait for to ensure computation is complete.

imatcopy

Computes an in-place scaled matrix transpose or copy operation using a general matrix.

Description

The `imatcopy` routine performs an in-place scaled matrix copy or transposition. The operation is defined as:

$$AB \leftarrow \alpha * \text{op}(AB)$$

where:

- $\text{op}(x)$ is one of $\text{op}(x) = x$, or $\text{op}(x) = x^T$, or $\text{op}(x) = x^H$
- α is a scalar
- AB is a matrix
- AB is $m \times n$ on input.

`imatcopy` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

imatcopy (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void imatcopy(sycl::queue &queue,
                 oneapi::mkl::transpose trans,
                 std::int64_t m,
                 std::int64_t n,
                 T alpha,
                 sycl::buffer<T, 1> &ab,
                 std::int64_t lda,
                 std::int64_t ldb);
}
```

```
namespace oneapi::mkl::blas::row_major {
    void imatcopy(sycl::queue &queue,
                 oneapi::mkl::transpose trans,
                 std::int64_t m,
                 std::int64_t n,
                 T alpha,
                 sycl::buffer<T, 1> &ab,
                 std::int64_t lda,
                 std::int64_t ldb);
}
```

Input Parameters

- queue
- The queue where the routine will be executed.
- trans
- Specifies op(*AB*), the transposition operation applied to the matrix *AB*. See [Data Types](#) for more details.
- m
- Number of rows for the matrix *AB* on input. Must be at least zero.
- n
- Number of columns for the matrix *AB* on input. Must be at least zero.
- alpha
- Scaling factor for the matrix transposition or copy.
- ab
- Buffer holding the input/output matrix *AB*. Must have size as follows:

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Size of array <i>ab</i> must be at least max(<i>lda</i> , <i>ldb</i>) * <i>n</i>	Size of array <i>ab</i> must be at least max(<i>lda</i> , <i>ldb</i>) * max(<i>m</i> , <i>n</i>)
Row major	Size of array <i>ab</i> must be at least max(<i>lda</i> , <i>ldb</i>) * <i>m</i>	Size of array <i>ab</i> must be at least max(<i>lda</i> , <i>ldb</i>) * max(<i>m</i> , <i>n</i>)

- lda
- Leading dimension of the matrix *AB* on input. If matrices are stored using column major layout, *lda* must be at least *m*. If matrices are stored using row major layout, *lda* must be at least *n*. Must be positive.
- ldb
- Leading dimension of the matrix *AB* on output. Must be positive.

	trans = transpose::nontrans	trans = transpose::trans or trans = transpose::conjtrans
Column major	Must be at least m	Must be at least n
Row major	Must be at least n	Must be at least m

Output Parameters

ab Output buffer, overwritten by $\alpha * \text{op}(\text{AB})$.

imatcopy (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event imatcopy(sycl::queue &queue,
        oneapi::mkl::transpose trans,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        T *ab,
        std::int64_t lda,
        std::int64_t ldb,
        const std::vector<sy::event> &dependencies = {});
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event imatcopy(sycl::queue &queue,
        oneapi::mkl::transpose trans,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        T *ab,
        std::int64_t lda,
        std::int64_t ldb,
        const std::vector<sy::event> &dependencies = {});
}
```

Input Parameters

queue The queue where the routine will be executed.

trans Specifies $\text{op}(\text{AB})$, the transposition operation applied to the matrix **AB**. See [Data Types](#) for more details.

m Number of rows for the matrix **AB** on input. Must be at least zero.

n Number of columns for the matrix **AB** on input. Must be at least zero.

alpha Scaling factor for the matrix transposition or copy. See [Scalar Arguments](#) for more information on the `value_or_pointer` data type.

ab Pointer to input/output matrix **AB**. Must have size as follows:

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Size of array <i>ab</i> must be at least $\max(\text{lda}, \text{ldb}) * n$	Size of array <i>ab</i> must be at least $\max(\text{lda}, \text{ldb}) * \max(m, n)$
Row major	Size of array <i>ab</i> must be at least $\max(\text{lda}, \text{ldb}) * m$	Size of array <i>ab</i> must be at least $\max(\text{lda}, \text{ldb}) * \max(m, n)$

lda Leading dimension of the matrix *AB* on input. If matrices are stored using column major layout, *lda* must be at least *m*. If matrices are stored using row major layout, *lda* must be at least *n*. Must be positive.

ldb Leading dimension of the matrix *AB* on output. Must be positive.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least <i>m</i>	Must be at least <i>n</i>
Row major	Must be at least <i>n</i>	Must be at least <i>m</i>

dependencies List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

ab Pointer to output matrix *AB* overwritten by $\alpha * \text{op}(\text{AB})$.

Return Values

Output event to wait for to ensure computation is complete.

omatadd

Computes a sum of two general matrices, with optional transposes.

Description

The `omatadd` routine performs an out-of-place scaled matrix addition with optional transposes in the arguments. The operation is defined as:

$$C \leftarrow \alpha * \text{op}(A) + \beta * \text{op}(B)$$

where:

- $\text{op}(x)$ is one of $\text{op}(x) = x$, or $\text{op}(x) = x^T$, or $\text{op}(x) = x^H$
- α and β are scalars
- A , B and C are matrices
- $\text{op}(A)$ is $m \times n$ matrix
- $\text{op}(B)$ is $m \times n$ matrix
- C is $m \times n$ matrix

In general, A , B , and C must not overlap in memory, with the exception of the following in-place operations:

- A and C may point to the same memory if $\text{op}(A)$ is non-transpose and $\text{lda} = \text{ldc}$;
- B and C may point to the same memory if $\text{op}(B)$ is non-transpose and $\text{ldb} = \text{ldc}$.

`omatadd` supports the following precisions:

T
float
double
<code>std::complex<float></code>
<code>std::complex<double></code>

omatadd (Buffer Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void omatadd(sycl::queue &queue,
                 oneapi::mkl::transpose transa,
                 oneapi::mkl::transpose transb,
                 std::int64_t m,
                 std::int64_t n,
                 T alpha,
                 sycl::buffer<T, 1> &a,
                 std::int64_t lda,
                 T beta,
                 sycl::buffer<T, 1> &b,
                 std::int64_t ldb,
                 sycl::buffer<T, 1> &c,
                 std::int64_t ldc)
}
```

```
namespace oneapi::mkl::blas::row_major {
    void omatadd(sycl::queue &queue,
                 oneapi::mkl::transpose transa,
                 oneapi::mkl::transpose transb,
                 std::int64_t m,
                 std::int64_t n,
                 T alpha,
                 sycl::buffer<T, 1> &a,
                 std::int64_t lda,
                 T beta,
                 sycl::buffer<T, 1> &b,
                 std::int64_t ldb,
                 sycl::buffer<T, 1> &c,
                 std::int64_t ldc)
}
```

Input Parameters

queue	The queue where the routine will be executed.
transa	Specifies $\text{op}(\mathbf{A})$, the transposition operation applied to the matrix $\mathbf{A}$.
transb	Specifies $\text{op}(\mathbf{B})$, the transposition operation applied to the matrix $\mathbf{B}$.
m	Number of rows for the result matrix $\mathbf{C}$. Must be at least zero.
n	Number of columns for the result matrix $\mathbf{C}$. Must be at least zero.
alpha	Scaling factor for the matrix $\mathbf{A}$.
a	Buffer holding the input matrix $\mathbf{A}$. If <code>alpha</code> is zero, this buffer is never accessed. Otherwise, its size must satisfy the following:

	<code>transa = transpose::nontrans</code>	<code>transa = transpose::trans</code> or <code>transa = transpose::conjtrans</code>
Column major	$\mathbf{A}$ is $m \times n$ matrix. Size of array <code>a</code> must be at least $\text{lda} * n$	$\mathbf{A}$ is $n \times m$ matrix. Size of array <code>a</code> must be at least $\text{lda} * m$
Row major	$\mathbf{A}$ is $m \times n$ matrix. Size of array <code>a</code> must be at least $\text{lda} * m$	$\mathbf{A}$ is $n \times m$ matrix. Size of array <code>a</code> must be at least $\text{lda} * n$

`lda` Leading dimension of matrix $\mathbf{A}$. Must be positive

	<code>transa = transpose::nontrans</code>	<code>transa = transpose::trans</code> or <code>transa = transpose::conjtrans</code>
Column major	Must be at least m	Must be at least n
Row major	Must be at least n	Must be at least m

`beta` Scaling factor for the matrix $\mathbf{B}$.

`b` Buffer holding the input matrix $\mathbf{B}$. If `beta` is zero, this buffer is never accessed. Otherwise, its size must satisfy the following:

	transb = transpose::nontrans	transb = transpose::trans Or transb = transpose::conjtrans
Column major	B is $m \times n$ matrix. Size of array b must be at least $ldb * n$	B is $n \times m$ matrix. Size of array b must be at least $ldb * m$
Row major	B is $m \times n$ matrix. Size of array b must be at least $ldb * m$	B is $n \times m$ matrix. Size of array b must be at least $ldb * n$

ldb

Leading dimension of matrix B. Must be positive.

	transb = transpose::nontrans	transb = transpose::trans Or transb = transpose::conjtrans
Column major	Must be at least m	Must be at least n
Row major	Must be at least n	Must be at least m

ldc

Leading dimension of the matrix C. If matrices are stored using column major layout, **ldc** must be at least m . If matrices are stored using row major layout, **ldc** must be at least n . Must be positive.

Output Parameters

c

Output buffer overwritten by $\alpha * \text{op}(A) + \beta * \text{op}(B)$.

Column major	C is $m \times n$ matrix. Size of array c must be at least $ldc * n$
Row major	C is $m \times n$ matrix. Size of array c must be at least $ldc * m$

omatadd (USM Version)

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event omatadd(sycl::queue &queue,
        oneapi::mkl::transpose transa,
        oneapi::mkl::transpose transb,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        oneapi::mkl::value_or_pointer<T> beta,
```

```

        const T *b,
        std::int64_t ldb,
        T *c,
        std::int64_t ldc,
        const std::vector<sycl::event> &dependencies = {});
    }

namespace oneapi::mkl::blas::row_major {
    sycl::event omatadd(sycl::queue &queue,
        oneapi::mkl::transpose transa,
        oneapi::mkl::transpose transb,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        oneapi::mkl::value_or_pointer<T> beta,
        const T *b,
        std::int64_t ldb,
        T *c,
        std::int64_t ldc,
        const std::vector<sycl::event> &dependencies = {});
}

```

Input Parameters

queue	The queue where the routine will be executed.
transa	Specifies $op(A)$, the transposition operation applied to matrix A. See Data Types for more details.
transb	Specifies $op(B)$, the transposition operation applied to matrix B. See Data Types for more details.
m	Number of rows for the result matrix C. Must be at least zero.
n	Number of columns for the result matrix C. Must be at least zero.
alpha	Scaling factor for the matrix A. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Pointer to input matrix A. See Matrix Storage for more details. If <code>alpha</code> is zero, <code>a</code> is never accessed and may be a null pointer. Otherwise, it must satisfy the following:

	transa = transpose::nontrans	transa = transpose::trans Or transa = transpose::conjtrans
Column major	A is $m \times n$ matrix. Size of array <code>a</code> must be at least <code>lda * n</code>	A is $n \times m$ matrix. Size of array <code>a</code> must be at least <code>lda * m</code>

	transa = transpose::nontrans	transa = transpose::trans or transa = transpose::conjtrans
Row major	A is $m \times n$ matrix. Size of array <i>a</i> must be at least $lda * m$	A is $n \times m$ matrix. Size of array <i>a</i> must be at least $lda * n$

lda

Leading dimension of matrix A. Must be positive

	transa = transpose::nontrans	transa = transpose::trans or transa = transpose::conjtrans
Column major	Must be at least m	Must be at least n
Row major	Must be at least n	Must be at least m

beta

Scaling factor for the matrix B. See [Scalar Arguments](#) for more information on the `value_or_pointer` data type.

b

Pointer to input matrix B. See [Matrix Storage](#) for more details. If *beta* is zero, *b* is never accessed and may be a null pointer. Otherwise, it must satisfy the following:

	transb = transpose::nontrans	transb = transpose::trans or transb = transpose::conjtrans
Column major	B is $m \times n$ matrix. Size of array <i>b</i> must be at least $ldb * n$	B is $n \times m$ matrix. Size of array <i>b</i> must be at least $ldb * m$
Row major	B is $m \times n$ matrix. Size of array <i>b</i> must be at least $ldb * m$	B is $n \times m$ matrix. Size of array <i>b</i> must be at least $ldb * n$

ldb

Leading dimension of matrix B. Must be positive.

	transb = transpose::nontrans	transb = transpose::trans or transb = transpose::conjtrans
Column major	Must be at least m	Must be at least n

	transb = transpose::nontrans	transb = transpose::trans Or transb = transpose::conjtrans
Row major	Must be at least n	Must be at least m

ldc Leading dimension of the matrix **C**. If matrices are stored using column major layout, **ldc** must be at least m . If matrices are stored using row major layout, **ldc** must be at least n . Must be positive.

dependencies List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

c Pointer to output matrix overwritten by $\alpha * \text{op}(A) + \beta * \text{op}(B)$.

Column major	C is $m \times n$ matrix. Size of array c must be at least $\text{ldc} * n$
Row major	C is $m \times n$ matrix. Size of array c must be at least $\text{ldc} * m$

Return Values

Output event to wait for to ensure computation is complete.

omatcopy_batch

Computes a group of out-of-place scaled matrix transpose or copy operations using general matrices.

Description

The `omatcopy_batch` routines perform a series of out-of-place scaled matrix copies or transpositions. They are similar to the `omatcopy` routines, but the `omatcopy_batch` routines perform matrix operations with a group of matrices.

`omatcopy_batch` supports the following precisions:

T
<code>float</code>
<code>double</code>
<code>std::complex<float></code>
<code>std::complex<double></code>

omatcopy_batch (Buffer Version)

Buffer version of `omatcopy_batch` supports only strided API.

Strided API

The operation for the strided API is defined as:

```
for i = 0 ... batch_size - 1
  A and B are matrices at offset i * stride_a in a and i * stride_b in b
  B = alpha * op(A)
end for
```

where:

- $\text{op}(X)$ is one of $\text{op}(X) = X$, $\text{op}(X) = X'$, or $\text{op}(X) = \text{conjg}(X')$
- α is a scalar
- A and B are matrices

For the strided API, the single input buffer contains all the input matrices, and the single output buffer contains all the output matrices. The locations of the individual matrices within the buffer are given by stride lengths, while the number of matrices is given by the `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
  void omatcopy_batch(sycl::queue &queue,
                     transpose trans,
                     std::int64_t m,
                     std::int64_t n,
                     T alpha,
                     sycl::buffer<T, 1> &a,
                     std::int64_t lda,
                     std::int64_t stride_a,
                     sycl::buffer<T, 1> &b,
                     std::int64_t ldb,
                     std::int64_t stride_b,
                     std::int64_t batch_size);
}
```

```
namespace oneapi::mkl::blas::row_major {
  void omatcopy_batch(sycl::queue &queue,
                     transpose trans,
                     std::int64_t m,
                     std::int64_t n,
                     T alpha,
                     sycl::buffer<T, 1> &a,
                     std::int64_t lda,
                     std::int64_t stride_a,
                     sycl::buffer<T, 1> &b,
                     std::int64_t ldb,
                     std::int64_t stride_b,
                     std::int64_t batch_size);
}
```

Input Parameters

queue	The queue where the routine should be executed.
trans	Specifies $\text{op}(A)$, the transposition operation applied to the matrices A .
m	Number of rows for each matrix A . Must be at least zero.
n	Number of columns for each matrix A . Must be at least zero.
alpha	Scaling factor for the matrix transposition or copy.

- a** Buffer holding the input matrices A. Must have size at least `stride_a*batch_size`.
- lda** Leading dimension of the A matrices. If matrices are stored using column major layout, `lda` must be at least `m`. If matrices are stored using row major layout, `lda` must be at least `n`. Must be positive.
- stride_a** Stride between the different A matrices. If matrices are stored using column major layout, `stride_a` must be at least `lda*n`. If matrices are stored using row major layout, `stride_a` must be at least `lda*m`.
- ldb** Leading dimension of the matrices B. Must be positive and satisfy:

	<code>trans = transpose::nontrans</code>	<code>trans = transpose::trans</code> Or <code>trans = transpose::conjtrans</code>
Column major	Must be at least <code>m</code>	Must be at least <code>n</code>
Row major	Must be at least <code>n</code>	Must be at least <code>m</code>

- stride_b** Stride between the different B matrices in the buffer `b`. Must be positive and satisfy:

	<code>trans = transpose::nontrans</code>	<code>trans = transpose::trans</code> Or <code>trans = transpose::conjtrans</code>
Column major	Must be at least <code>ldb*n</code>	Must be at least <code>ldb*m</code>
Row major	Must be at least <code>ldb*m</code>	Must be at least <code>ldb*n</code>

- batch_size** Specifies the number of matrices to transpose or copy. Must be at least zero.

Output Parameters

- b** Output buffer, overwritten by `batch_size` matrix transpose or copy operations of the form `alpha*op(A)`. Must have size at least `stride_b*batch_size`.

omatcopy_batch (USM Version)

USM version of `omatcopy_batch` supports group API and strided API.

Group API

The type `Ti` of integer pointers in the group API may be either `std::int64_t` or `std::int32_t`.

The operation for the group API is defined as:

```
idx = 0
for i = 0 ... group_count - 1
    m, n, alpha, lda, ldb and group_size at position i in their respective arrays
    for j = 0 ... group_size - 1
```



```

        A and B are matrices at position idx in their respective arrays
        B = alpha * op(A)
        idx := idx + 1
    end for
end for

```

where:

- $op(X)$ is one of $op(X) = X$, $op(X) = X'$, or $op(X) = \text{conjg}(X')$
- α is a scalar
- A and B are matrices

For the group API, the matrices are given by arrays of pointers. A and B represent matrices stored at addresses pointed to by a and b respectively. The total number of entries in a and b are given by:

$$total_batch_count = \sum_{i=0}^{group_count-1} group_size[i]$$

Syntax

```

namespace oneapi::mkl::blas::column_major {
    sycl::event omatcopy_batch(sycl::queue &queue,
                              const transpose *trans,
                              const Ti *m,
                              const Ti *n,
                              const T *alpha,
                              const T **a,
                              const Ti *lda,
                              T **b,
                              const Ti *ldb,
                              std::int64_t group_count,
                              const Ti *groupsize,
                              const std::vector<sycl::event> &dependencies = {});
}

```

```

namespace oneapi::mkl::blas::row_major {
    sycl::event omatcopy_batch(sycl::queue &queue,
                              const transpose *trans,
                              const Ti *m,
                              const Ti *n,
                              const T *alpha,
                              const T **a,
                              const Ti *lda,
                              T **b,
                              const Ti *ldb,
                              std::int64_t group_count,
                              const Ti *groupsize,
                              const std::vector<sycl::event> &dependencies = {});
}

```

Input Parameters

queue	The queue where the routine should be executed.
trans	Array of size <code>group_count</code> . Each element <code>i</code> in the array specifies $op(A)$ the transposition operation applied to the matrices A.

<code>m</code>	Array of <code>group_count</code> integers. <code>m[i]</code> specifies the number of rows of <code>A[i]</code> . Each entry must be at least zero.
<code>n</code>	Array of <code>group_count</code> integers. <code>n[i]</code> specifies the number of columns of <code>A[i]</code> . Each entry must be at least zero.
<code>alpha</code>	Array of size <code>group_count</code> containing scaling factors for the operation.
<code>a</code>	Array of size <code>total_batch_count</code> of pointers to <code>A</code> matrices. If matrices are stored in column major layout, the array allocated for each <code>A</code> matrix of the group <code>i</code> must be of size at least <code>lda[i] * n[i]</code> . If matrices are stored in row major layout, the array allocated for each <code>A</code> matrix of the group <code>i</code> must be of size at least <code>lda[i]*m[i]</code> .
<code>lda</code>	Array of <code>group_count</code> integers. <code>lda[i]</code> specifies the leading dimension of the <code>A[i]</code> matrix. If matrices are stored using column major layout, <code>lda[i]</code> must be at least <code>m[i]</code> . If matrices are stored using row major layout, <code>lda[i]</code> must be at least <code>n[i]</code> . Each must be positive.
<code>ldb</code>	Array of <code>group_count</code> integers. <code>ldb[i]</code> specifies the leading dimension of the <code>B[i]</code> matrix. Each <code>ldb[i]</code> must be positive and satisfy:

	<code>trans[i] = transpose::nontrans</code>	<code>trans[i] = transpose::trans</code> OR <code>trans[i] = transpose::conjtrans</code>
Column major	Must be at least <code>m[i]</code>	Must be at least <code>n[i]</code>
Row major	Must be at least <code>n[i]</code>	Must be at least <code>m[i]</code>

<code>group_count</code>	Number of groups. Must be at least 0.
<code>group_size</code>	Array of size <code>group_count</code> . The element <code>group_size[i]</code> is the number of matrices in the group <code>i</code> . Each element in <code>group_size</code> must be at least 0.
<code>dependencies</code>	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

<code>b</code>	Output array of pointers to <code>B</code> matrices, overwritten by <code>total_batch_count</code> matrix transpose or copy operations of the form <code>alpha*op(A)</code> . If matrices are stored using column major layout, the array allocated for each <code>B</code> matrix of the group <code>i</code> must be of size at least <code>ldb[i] * n[i]</code> if <code>B</code> is not transposed or <code>ldb[i]*m[i]</code> if <code>B</code> is transposed. If matrices are stored using row major layout, the array allocated for each <code>B</code> matrix of the group <code>i</code> must be of size at least <code>ldb[i] * m[i]</code> if <code>B</code> is not transposed or <code>ldb[i]*n[i]</code> if <code>B</code> is transposed.
----------------	--

Return Values

Output event to wait on to ensure computation is complete.

Strided API

The operation for the strided API is defined as:

```
for i = 0 ... batch_size - 1
  A and B are matrices at offset i * stride_a in a and i * stride_b in b
  B = alpha * op(A)
end for
```

where:

- $\text{op}(X)$ is one of $\text{op}(X) = X$, $\text{op}(X) = X'$, or $\text{op}(X) = \text{conjg}(X')$
- α is a scalar
- A and B are matrices

For the strided API, the single input array contains all the input matrices, and the single output array contains all the output matrices. The locations of the individual matrices within the array are given by stride lengths, while the number of matrices is given by the `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
  sycl::event omatcopy_batch(sycl::queue &queue,
                           transpose trans,
                           std::int64_t m,
                           std::int64_t n,
                           oneapi::mkl::value_or_pointer<T> alpha,
                           const T *a,
                           std::int64_t lda,
                           std::int64_t stride_a,
                           T *b,
                           std::int64_t ldb,
                           std::int64_t stride_b,
                           std::int64_t batch_size,
                           const std::vector<sycl::event> &dependencies = {});
}
```

```
namespace oneapi::mkl::blas::row_major {
  sycl::event omatcopy_batch(sycl::queue &queue,
                           transpose trans,
                           std::int64_t m,
                           std::int64_t n,
                           oneapi::mkl::value_or_pointer<T> alpha,
                           const T *a,
                           std::int64_t lda,
                           std::int64_t stride_a,
                           T *b,
                           std::int64_t ldb,
                           std::int64_t stride_b,
                           std::int64_t batch_size,
                           const std::vector<sycl::event> &dependencies = {});
}
```

Input Parameters

<code>trans</code>	Specifies $\text{op}(A)$, the transposition operation applied to the matrices A.
<code>m</code>	Number of rows for each matrix A. Must be at least zero.
<code>n</code>	Number of columns for each matrix A. Must be at least zero.

alpha

Scaling factor for the matrix transposition or copy. See [Scalar Arguments](#) for more information on the `value_or_pointer` data type.

a

Array holding the input matrices A. Must have size at least `stride_a*batch_size`.

lda

Leading dimension of the A matrices. If matrices are stored using column major layout, `lda` must be at least `m`. If matrices are stored using row major layout, `lda` must be at least `n`. Must be positive.

stride_a

Stride between the different A matrices. If matrices are stored using column major layout, `stride_a` must be at least `lda*n`. If matrices are stored using row major layout, `stride_a` must be at least `lda*m`.

ldb

Leading dimension of the matrices B. Must be positive and satisfy:

	<code>trans = transpose::nontrans</code>	<code>trans = transpose::trans</code> Or <code>trans = transpose::conjtrans</code>
Column major	Must be at least <code>m</code>	Must be at least <code>n</code>
Row major	Must be at least <code>n</code>	Must be at least <code>m</code>

stride_b

Stride between the different B matrices in the array b. Must be positive and satisfy:

	<code>trans = transpose::nontrans</code>	<code>trans = transpose::trans</code> Or <code>trans = transpose::conjtrans</code>
Column major	Must be at least <code>ldb*n</code>	Must be at least <code>ldb*m</code>
Row major	Must be at least <code>ldb*m</code>	Must be at least <code>ldb*n</code>

batch_size

Specifies the number of matrices to transpose or copy. Must be at least zero.

dependencies

List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

b

Output array, overwritten by `batch_size` matrix transpose or copy operations of the form `alpha*op(A)`. Must have size at least `stride_b*batch_size`.

Return Values

Output event to wait on to ensure computation is complete.

imatcopy_batch

Computes a group of in-place scaled matrix transpose or copy operations using general matrices.

Description

The `imatcopy_batch` routines perform a series of in-place scaled matrix copies or transpositions. They are similar to the `imatcopy` routines, but the `imatcopy_batch` routines perform their operations with groups of matrices. The groups contain matrices with the same parameters.

`imatcopy_batch` supports the following precisions:

T
float
double
std::complex<float>
std::complex<double>

`imatcopy_batch` (Buffer Version)

Buffer version of `imatcopy_batch` supports only strided API.

Strided API

The operation for the strided API is defined as:

```
for i = 0 ... batch_size - 1
    AB is a matrix at offset i * stride in ab
    AB = alpha * op(AB)
end for
```

where:

- `op(X)` is one of `op(X) = X`, `op(X) = X'`, or `op(X) = conjg(X')`
- `alpha` is a scalar
- `AB` is a matrix to be transformed in place

For the strided API, the single buffer `AB` contains all the matrices to be transformed in place. The locations of the individual matrices within the buffer are given by stride lengths, while the number of matrices is given by the `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void imatcopy_batch(sycl::queue &queue,
                       transpose trans,
                       std::int64_t m,
                       std::int64_t n,
                       T alpha,
                       sycl::buffer<T, 1> &ab,
                       std::int64_t lda,
                       std::int64_t ldb,
                       std::int64_t stride,
                       std::int64_t batch_size);
}
```

```
namespace oneapi::mkl::blas::row_major {
    void imatcopy_batch(sycl::queue &queue,
                       transpose trans,
```

```

std::int64_t m,
std::int64_t n,
T alpha,
sycl::buffer<T, 1> &ab,
std::int64_t lda,
std::int64_t ldb,
std::int64_t stride,
std::int64_t batch_size);
}

```

Input Parameters

queue	The queue where the routine should be executed.
trans	Specifies $\text{op}(AB)$, the transposition operation applied to the matrices AB .
m	Number of rows for each matrix AB on input. Must be at least 0.
n	Number of columns for each matrix AB on input. Must be at least 0.
alpha	Scaling factor for the matrix transpose or copy operation.
ab	Buffer holding the matrices AB . Must have size at least $\text{stride} \times \text{batch_size}$.
lda	Leading dimension of the AB matrices on input. If matrices are stored using column major layout, lda must be at least m . If matrices are stored using row major layout, lda must be at least n . Must be positive.
ldb	Leading dimension of the matrices AB on output. Must be positive.

	trans = transpose::nontrans	trans = transpose::trans Or trans = transpose::conjtrans
Column major	Must be at least m	Must be at least n
Row major	Must be at least n	Must be at least m

stride	Stride between the different AB matrices. It must be at least $\max(\text{ldb}, \text{lda}) * \max(k_a, k_b)$, where: k_a is m if column major layout is used or n if row major layout is used k_b is n if column major layout is used and AB is not transposed, or m otherwise
batch_size	Specifies the number of matrices to transpose or copy. Must be at least zero.

Output Parameters

ab	Output buffer, overwritten by batch_size matrix multiply operations of the form $\alpha * \text{op}(AB)$.
----	--

imatcopy_batch (USM Version)

USM version of `imatcopy_batch` supports group API and strided API.

Group API

The type `Ti` of integer pointers in the group API may be either `std::int64_t` or `std::int32_t`.

The operation for the group API is defined as:

```
idx = 0
for i = 0 ... group_count - 1
    m,n, alpha, lda, ldb and group_size at position i in their respective arrays
    for j = 0 ... group_size - 1
        AB is a matrix at position idx in AB
        AB = alpha * op(AB)
        idx := idx + 1
    end for
end for
```

where:

- `op(X)` is one of `op(X) = X`, `op(X) = X'`, or `op(X) = conjg(X')`
- `alpha` is a scalar
- `AB` is a matrix to be transformed in place

For the group API, the matrices are given by arrays of pointers. `AB` represents a matrix stored at the address pointed to by `ab`. The total number of entries in `ab` is given by:

$$total_batch_count = \sum_{i=0}^{group_count-1} group_size[i]$$

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event imatcopy_batch(sycl::queue &queue,
                             const transpose *trans,
                             const Ti *m,
                             const Ti *n,
                             const T *alpha, T **ab,
                             const Ti *lda,
                             const Ti *ldb,
                             std::int64_t group_count,
                             const Ti *groupsize,
                             const std::vector<sycl::event> &dependencies = {});
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event imatcopy_batch(sycl::queue &queue,
                              const transpose *trans,
                              const Ti *m,
                              const Ti *n,
                              const T *alpha, T **ab,
                              const Ti *lda,
                              const Ti *ldb,
                              std::int64_t group_count,
                              const Ti *groupsize,
                              const std::vector<sycl::event> &dependencies = {});
}
```

Input Parameters

queue	The queue where the routine should be executed.
trans	Array of size <code>group_count</code> . Each element <code>i</code> in the array specifies <code>op(AB)</code> the transposition operation applied to the matrices <code>AB</code> .
m	Array of <code>group_count</code> integers. <code>m[i]</code> specifies the number of rows in <code>AB[i]</code> on input. Each entry must be at least zero.
n	Array of <code>group_count</code> integers. <code>n[i]</code> specifies the number of columns in <code>AB[i]</code> on input. Each entry must be at least zero.
alpha	Array of size <code>group_count</code> containing scaling factors for the matrix transpositions or copies.
ab	Array of size <code>total_batch_count</code> , holding pointers to arrays used to store <code>AB</code> matrices.
lda	Array of <code>group_count</code> integers. <code>lda[i]</code> specifies the leading dimension of the matrix input <code>AB</code> . If matrices are stored using column major layout, <code>lda[i]</code> must be at least <code>m[i]</code> . If matrices are stored using row major layout, <code>lda[i]</code> must be at least <code>n[i]</code> . Must be positive.
ldb	Array of <code>group_count</code> integers. <code>ldb[i]</code> specifies the leading dimension of the matrix <code>AB</code> on output. Each <code>ldb[i]</code> must be positive and satisfy:

	<code>trans[i] = transpose::nontrans</code>	<code>trans[i] = transpose::trans</code> OR <code>trans[i] = transpose::conjtrans</code>
Column major	Must be at least <code>m[i]</code>	Must be at least <code>n[i]</code>
Row major	Must be at least <code>n[i]</code>	Must be at least <code>m[i]</code>

group_count	Number of groups. Must be at least 0.
group_size	Array of size <code>group_count</code> . The element <code>group_size[i]</code> is the number of matrices in the group <code>i</code> . Each element in <code>group_size</code> must be at least 0.
dependencies	List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

ab	Output array of pointers to <code>AB</code> matrices, overwritten by <code>total_batch_count</code> matrix transpose or copy operations of the form <code>alpha*op(AB)</code> .
----	---

Return Values

Output event to wait on to ensure computation is complete.

Strided API

The operation for the strided API is defined as:

```
for i = 0 ... batch_size - 1
  AB is a matrix at offset i * stride in ab
  AB = alpha * op(AB)
end for
```

where:

- $\text{op}(X)$ is one of $\text{op}(X) = X$, $\text{op}(X) = X'$, or $\text{op}(X) = \text{conjg}(X')$
- α is a scalar
- AB is a matrix to be transformed in place

For the strided API, the single array `ab` contains all the matrices AB to be transformed in place. The locations of the individual matrices within the array are given by stride lengths, while the number of matrices is given by the `batch_size` parameter.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event imatcopy_batch(sycl::queue &queue,
                             transpose trans,
                             std::int64_t m,
                             std::int64_t n,
                             oneapi::mkl::value_or_pointer<T> alpha,
                             T *ab,
                             std::int64_t lda,
                             std::int64_t ldb,
                             std::int64_t stride,
                             std::int64_t batch_size,
                             const std::vector<sycl::event> &dependencies = {});
}
```

```
namespace oneapi::mkl::blas::column_major {
    sycl::event imatcopy_batch(sycl::queue &queue,
                             transpose trans,
                             std::int64_t m,
                             std::int64_t n,
                             oneapi::mkl::value_or_pointer<T> alpha,
                             T *ab,
                             std::int64_t lda,
                             std::int64_t ldb,
                             std::int64_t stride,
                             std::int64_t batch_size,
                             const std::vector<sycl::event> &dependencies = {});
}
```

Input Parameters

<code>queue</code>	The queue where the routine should be executed.
<code>trans</code>	Specifies $\text{op}(AB)$, the transposition operation applied to the matrices AB .
<code>m</code>	Number of rows for each matrix AB on input. Must be at least 0.
<code>n</code>	Number of columns for each matrix AB on input. Must be at least 0.
<code>alpha</code>	Scaling factor for the matrix transpose or copy operation. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.

ab Array holding the matrices **AB**. Must have size at least `stride*batch_size`.

lda Leading dimension of the **AB** matrices on input. If matrices are stored using column major layout, `lda` must be at least `m`. If matrices are stored using row major layout, `lda` must be at least `n`. Must be positive.

ldb Leading dimension of the matrices **AB** on output. Must be positive.

	trans = transpose::nontrans	trans = transpose::trans or trans = transpose::conjtrans
Column major	Must be at least <code>m</code>	Must be at least <code>n</code>
Row major	Must be at least <code>n</code>	Must be at least <code>m</code>

stride Stride between the different **AB** matrices. It must be at least `max(ldb, lda)*max(ka, kb)`, where:

- `ka` is `m` if column major layout is used or `n` if row major layout is used
- `kb` is `n` if column major layout is used and **AB** is not transposed, or `m` otherwise

batch_size Specifies the number of matrices to transpose or copy. Must be at least zero.

dependencies List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

ab Output array, overwritten by `batch_size` matrix multiply operations of the form `alpha*op(AB)`.

Return Values

Output event to wait on to ensure computation is complete.

omatadd_batch

Computes a group of out-of-place scaled matrix additions using general matrices.

Description

The `omatadd_batch` routines perform a series of out-of-place scaled matrix additions. They are similar to the `omatadd` routines, but the `omatadd_batch` routines perform matrix operations with a group of matrices.

`omatadd_batch` supports the following precisions:

T
<code>float</code>
<code>double</code>

T

```
std::complex<float>
```

```
std::complex<double>
```

omatadd_batch (Buffer Version)

Strided API

The matrices are always in a strided format for `omatadd_batch`. The operation is defined as:

```
for i = 0 ... batch_size - 1
    A is a matrix at offset i * stride_a in a
    B is a matrix at offset i * stride_b in b
    C is a matrix at offset i * stride_c in c
    C = alpha * op(A) + beta * op(B)
end for
```

where:

- `op(X)` is one of `op(X) = X`, `op(X) = X'`, or `op(X) = conjg(X')`
- `alpha` and `beta` are scalars
- `A`, `B` and `C` are matrices

The input buffers `a` and `b` contain all the input matrices, and the single output buffer `c` contains all the output matrices. The locations of the individual matrices within the buffer or array are given by stride lengths, while the number of matrices is given by the `batch_size` parameter.

In general, the `a`, `b`, and `c` buffers must not overlap in memory, with the exception of the following in-place operations:

- `a` and `c` may point to the same memory if `op(A)` is non-transpose and all the `A` matrices within `a` have the same parameters as all the respective `C` matrices within `c`;
- `b` and `c` may point to the same memory if `op(B)` is non-transpose and all the `B` matrices within `b` have the same parameters as all the respective `C` matrices within `c`.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    void omatadd_batch(sycl::queue &queue,
                      transpose transa,
                      transpose transb,
                      std::int64_t m,
                      std::int64_t n,
                      T alpha,
                      sycl::buffer<T, 1> &a,
                      std::int64_t lda,
                      std::int64_t stride_a,
                      T beta,
                      sycl::buffer<T, 1> &b,
                      std::int64_t ldb,
                      std::int64_t stride_b,
                      sycl::buffer<T, 1> &c,
                      std::int64_t ldc,
```

```

        std::int64_t stride_c,
        std::int64_t batch_size);
}

namespace oneapi::mkl::blas::row_major {
    void omatadd_batch(sycl::queue &queue,
        transpose transa,
        transpose transb,
        std::int64_t m,
        std::int64_t n,
        T alpha,
        sycl::buffer<T, 1> &a,
        std::int64_t lda,
        std::int64_t stride_a,
        T beta,
        sycl::buffer<T, 1> &b,
        std::int64_t ldb,
        std::int64_t stride_b,
        sycl::buffer<T, 1> &c,
        std::int64_t ldc,
        std::int64_t stride_c,
        std::int64_t batch_size);
}

```

Input Parameters

queue	The queue where the routine should be executed.
transa	Specifies $\text{op}(A)$, the transposition operation applied to the matrices A.
transb	Specifies $\text{op}(B)$, the transposition operation applied to the matrices B.
m	Number of rows for the result matrix C. Must be at least zero.
n	Number of columns for the result matrix C. Must be at least zero.
alpha	Scaling factor for the matrices A.
a	Buffer holding the input matrices A. If alpha is zero, a is never accessed. Otherwise it must have size at least <code>stride_a*batch_size</code> .
lda	Leading dimension of the A matrices. Must be positive and satisfy:

	transa = transpose::nontrans	transa = transpose::trans or transa = transpose::conjtrans
Column major	Must be at least m	Must be at least n
Row major	Must be at least n	Must be at least m

stride_a	Stride between the different A matrices in the buffer a. Must be positive and satisfy:
----------	--

	transa = transpose::nontrans	transa = transpose::trans Or transa = transpose::conjtrans
Column major	Must be at least $lda*n$	Must be at least $lda*m$
Row major	Must be at least $lda*m$	Must be at least $lda*n$

beta Scaling factor for the matrices B.

b Buffer holding the input matrices B. If **beta** is zero, **b** is never accessed. Otherwise it must have size at least $stride_b*batch_size$.

ldb Leading dimension of the B matrices. Must be positive and satisfy:

	transb = transpose::nontrans	transb = transpose::trans Or transb = transpose::conjtrans
Column major	Must be at least m	Must be at least n
Row major	Must be at least n	Must be at least m

stride_b Stride between the different B matrices in the buffer **b**. Must be positive and satisfy:

	transb = transpose::nontrans	transb = transpose::trans Or transb = transpose::conjtrans
Column major	Must be at least $ldb*n$	Must be at least $ldb*m$
Row major	Must be at least $ldb*m$	Must be at least $ldb*n$

ldc Leading dimension of the C matrices. If matrices are stored using column major layout, **ldc** must be at least m . If matrices are stored using row major layout, **ldc** must be at least n . Must be positive.

stride_c Stride between the different C matrices. If matrices are stored using column major layout, **stride_c** must be at least $ldc*n$. If matrices are stored using row major layout, **stride_c** must be at least $ldc*m$.

batch_size Specifies the number of input and output matrices to add. Must be at least zero.

dependencies List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

`c` Output buffer, overwritten by `batch_size` matrix addition operations of the form $\alpha * \text{op}(A) + \beta * \text{op}(B)$. Must have size at least `stride_c * batch_size`.

`omatadd_batch` (USM Version)

Strided API

The matrices are always in a strided format for `omatadd`. The operation is defined as:

```
for i = 0 ... batch_size - 1
    A is a matrix at offset i * stride_a in a
    B is a matrix at offset i * stride_b in b
    C is a matrix at offset i * stride_c in c
    C = alpha * op(A) + beta * op(B)
end for
```

where:

- `op(X)` is one of `op(X) = X`, `op(X) = X'`, or `op(X) = conjg(X')`
- `alpha` and `beta` are scalars
- `A`, `B` and `C` are matrices

The input buffers `a` and `b` contain all the input matrices, and the single output buffer `c` contains all the output matrices. The locations of the individual matrices within the buffer or array are given by stride lengths, while the number of matrices is given by the `batch_size` parameter.

In general, the `a`, `b`, and `c` buffers must not overlap in memory, with the exception of the following in-place operations:

- `a` and `c` may point to the same memory if `op(A)` is non-transpose and all the `A` matrices within `a` have the same parameters as all the respective `C` matrices within `c`;
- `b` and `c` may point to the same memory if `op(B)` is non-transpose and all the `B` matrices within `b` have the same parameters as all the respective `C` matrices within `c`.

Syntax

```
namespace oneapi::mkl::blas::column_major {
    sycl::event omatadd_batch(sycl::queue &queue,
                            transpose transa,
                            transpose transb,
                            std::int64_t m,
                            std::int64_t n,
                            oneapi::mkl::value_or_pointer<T> alpha,
                            const T *a,
                            std::int64_t lda,
                            std::int64_t stride_a,
                            oneapi::mkl::value_or_pointer<T> beta,
                            const T *b,
                            std::int64_t ldb,
                            std::int64_t stride_b,
                            T *c,
                            std::int64_t ldc,
                            std::int64_t stride_c,
                            std::int64_t batch_size,
                            const std::vector<sycl::event> &dependencies = {});
}
```

```
namespace oneapi::mkl::blas::row_major {
    sycl::event omatadd_batch(sycl::queue &queue,
                            transpose transa,
```

```

        transpose transb,
        std::int64_t m,
        std::int64_t n,
        oneapi::mkl::value_or_pointer<T> alpha,
        const T *a,
        std::int64_t lda,
        std::int64_t stride_a,
        oneapi::mkl::value_or_pointer<T> beta,
        const T *b,
        std::int64_t ldb,
        std::int64_t stride_b,
        T *c,
        std::int64_t ldc,
        std::int64_t stride_c,
        std::int64_t batch_size,
        const std::vector<sycl::event> &dependencies = {});
}

```

Input Parameters

queue	The queue where the routine should be executed.
transa	Specifies $\text{op}(A)$, the transposition operation applied to the matrices A.
transb	Specifies $\text{op}(B)$, the transposition operation applied to the matrices B.
m	Number of rows for the result matrix C. Must be at least zero.
n	Number of columns for the result matrix C. Must be at least zero.
alpha	Scaling factor for the matrices A. See Scalar Arguments for more information on the <code>value_or_pointer</code> data type.
a	Array holding the input matrices A. If alpha is zero, a is never accessed and may be a null pointer. Otherwise it must have size at least <code>stride_a*batch_size</code> .
lda	Leading dimension of the A matrices. Must be positive and satisfy:

	transa = transpose::nontrans	transa = transpose::trans Or transa = transpose::conjtrans
Column major	Must be at least m	Must be at least n
Row major	Must be at least n	Must be at least m

stride_a	Stride between the different A matrices in the array a. Must be positive and satisfy:
----------	---

	transa = transpose::nontrans	transa = transpose::trans Or transa = transpose::conjtrans
Column major	Must be at least $lda*n$	Must be at least $lda*m$
Row major	Must be at least $lda*m$	Must be at least $lda*n$

beta Scaling factor for the matrices B. See [Scalar Arguments](#) for more information on the `value_or_pointer` data type.

b Array holding the input matrices B. If `beta` is zero, `b` is never accessed and may be a null pointer. Otherwise it must have size at least `stride_b*batch_size`.

ldb Leading dimension of the B matrices. Must be positive and satisfy:

	transb = transpose::nontrans	transb = transpose::trans Or transb = transpose::conjtrans
Column major	Must be at least m	Must be at least n
Row major	Must be at least n	Must be at least m

stride_b Stride between the different B matrices in the array `b`. Must be positive and satisfy:

	transb = transpose::nontrans	transb = transpose::trans Or transb = transpose::conjtrans
Column major	Must be at least $ldb*n$	Must be at least $ldb*m$
Row major	Must be at least $ldb*m$	Must be at least $ldb*n$

ldc Leading dimension of the C matrices. If matrices are stored using column major layout, `ldc` must be at least m . If matrices are stored using row major layout, `ldc` must be at least n . Must be positive.

stride_c Stride between the different C matrices. If matrices are stored using column major layout, `stride_c` must be at least $ldc*n$. If matrices are stored using row major layout, `stride_c` must be at least $ldc*m$.

batch_size Specifies the number of input and output matrices to add. Must be at least zero.

dependencies List of events to wait for before starting computation, if any. If omitted, defaults to no dependencies.

Output Parameters

`c` Output array, overwritten by `batch_size` matrix addition operations of the form $\alpha * \text{op}(A) + \beta * \text{op}(B)$. Must have size at least `stride_c * batch_size`.

Return Values

Output event to wait on to ensure computation is complete.

Compute Modes

BLAS level-3 routines and extensions support *alternate compute modes*, which can provide increased performance in exchange for different numerical properties or reduced accuracy.

A list of one or more allowed modes can be specified either at compile time, on a per-call or per-source-file basis, or at runtime, using the `MKL_BLAS_COMPUTE_MODE` environment variable. oneMKL will automatically select an appropriate implementation from this list, taking into account routine parameters and hardware characteristics. In case none of the allowed alternate modes are supported by the given routine on the selected device, or if none of the allowed alternate modes are expected to improve performance, oneMKL will automatically fall back to a standard implementation.

NOTE Whether a particular mode provides additional performance will depend on a number of factors, including routine type, matrix sizes, transpose parameters, and hardware configuration. In the current oneMKL release, alternate implementations are available for `gemm`, `gemmt`, `syrk`, and `syr2k` on selected hardware for real single precision, and `gemm` only for complex single precision.

By default, oneMKL does not enable any alternate compute modes. The `MKL_BLAS_COMPUTE_MODE` environment variable is intended for quickly evaluating whether alternate compute modes provide performance benefits and acceptable accuracy for an application. After initial testing, alternate mode settings can be permanently applied within the application using the per-call or per-source-file APIs.

When running on GPU, oneMKL's verbose output indicates which mode is used for each call, whether the standard mode or one of the alternate modes. See **Checking Which Mode Is Used** for more details.

Mode Settings

Available alternate modes are described in the table below. Multiple modes can be combined, allowing oneMKL to choose any of the allowed modes that is expected to provide best performance.

For per-call or per-source-file mode, alternate compute modes are selected by OR'ing together one or more `oneapi::mkl::blas::compute_mode` values from the following table, e.g.:

```
using oneapi::mkl::blas;
auto mode_settings = compute_mode::float_to_bf16x2 | compute_mode::float_to_tf32; /* allow
either of these two modes */
```

When making global changes with the `MKL_BLAS_COMPUTE_MODE` environment variable, multiple modes are combined with commas:

```
set MKL_BLAS_COMPUTE_MODE=FLOAT_TO_BF16X2,FLOAT_TO_TF32
```

compute_mode enum value	Environment variable setting	Description
<code>compute_mode::float_to_bf16</code> 6	<code>FLOAT_TO_BF16</code>	Convert single-precision inputs to bfloat16 format internally; output is accumulated in single precision.

compute_mode enum value	Environment variable setting	Description
compute_mode::float_to_bf16x2	FP16_TO_BF16X2	Accuracy will be reduced, with possibly much higher performance. Convert each single-precision input value to a sum of two bfloat16 values internally; output is accumulated in single precision. Expected accuracy is between that of standard single precision arithmetic and bfloat16 arithmetic. <i>Note:</i> infinite inputs may produce unexpected NaNs in the output.
compute_mode::float_to_bf16x3	FP16_TO_BF16X3	Convert each single-precision input value to a sum of three bfloat16 values internally; output is accumulated in single precision. Expected accuracy is comparable to standard single precision arithmetic in most cases. <i>Note:</i> infinite inputs may produce unexpected NaNs in the output.
compute_mode::float_to_tf32	FP16_TO_TF32	Convert each single-precision input value to tf32 format internally; output is accumulated in single precision. Expected accuracy is between float_to_bf16 and float_to_bf16x2.
compute_mode::complex_3m	COMPLEX_3M	Reduce the four real multiplications in a standard complex multiplication to three real multiplications. Expected accuracy is comparable to standard arithmetic in most cases.
compute_mode::any	ANY	Allow any alternate compute mode.

compute_mode enum value	Environment variable setting	Description
compute_mode::standard	STANDARD	Do not allow any alternate compute modes.
compute_mode::prefer_alter nate	PREFER_ALTERNATE	Used in conjunction with one or more alternate modes. Prefer an alternate compute mode over the standard implementation whenever available, even if performance may be reduced.
compute_mode::force_alter nate	FORCE_ALTERNATE	Used in conjunction with one or more alternate modes. Never use the standard implementation; if no allowed alternate implementation is available, an exception will be thrown.

Per-Call Mode Settings

BLAS level-3 routines and extensions support an optional `oneapi::mkl::blas::compute_mode` argument to specify a desired mode setting, at the end of the parameter list. For USM APIs, the `compute_mode` argument goes before the list of input dependencies, if any; either argument may be omitted. For example:

```
using oneapi::mkl::blas;

sycl::buffer<float, 1> a_buffer, c_buffer;
float *a_ptr, *c_ptr;
/* ... */

// Buffer API
syrk(my_queue, n, k, uplo, trans, alpha, a_buffer, lda, beta, c_buffer, ldc,
compute_mode::float_to_bf16);

// Buffer API, forcing float_to_bf16 mode
syrk(my_queue, n, k, uplo, trans, alpha, a_buffer, lda, beta, c_buffer, ldc,
compute_mode::float_to_bf16 | compute_mode::force_alternate);

// USM API, without dependencies
syrk(my_queue, n, k, uplo, trans, alpha, a_ptr, lda, beta, c_ptr, ldc,
compute_mode::float_to_bf16);

// USM API, with dependencies
syrk(my_queue, n, k, uplo, trans, alpha, a_ptr, lda, beta, c_ptr, ldc,
compute_mode::float_to_bf16, {event1, event2});

// USM API, dependencies but no special compute_mode settings
syrk(my_queue, n, k, uplo, trans, alpha, a_ptr, lda, beta, c_ptr, ldc, {event1, event2});
```

Per-Source-File Mode Settings

You can provide default mode settings for all calls within a source file by defining the `MKL_BLAS_COMPUTE_MODE` macro before including any oneMKL header files. This macro must be set to an expression of type `oneapi::mkl::blas::compute_mode`.

```
#define MKL_BLAS_COMPUTE_MODE oneapi::mkl::blas::compute_mode::complex_3m
#include <oneapi/mkl.hpp>

void my_function() {
    /* ... */

    // 3M mode will be allowed by default:
    gemm(my_queue, m, n, k, trans_a, trans_b, alpha, a, lda, b, ldb, beta, c, ldc);
}
```

`compute_mode` parameters passed to a oneMKL routine take precedence over the default setting:

```
#define MKL_BLAS_COMPUTE_MODE oneapi::mkl::blas::compute_mode::complex_3m
#include <oneapi/mkl.hpp>

void my_function() {
    /* ... */

    // Provided compute_mode overrides the default 3M mode.
    gemm(my_queue, m, n, k, trans_a, trans_b, alpha, a, lda, b, ldb, beta, c, ldc,
    compute_mode::standard);
}
```

Runtime Mode Settings

The `MKL_BLAS_COMPUTE_MODE` environment variable allows you to set default application-wide alternate compute mode settings, as a quick method for evaluating alternate compute modes at runtime. For example, with `bash` or a similar shell:

```
// my_application.cpp
#include <oneapi/mkl.hpp>

int main() {
    /* ... */

    // Call to gemm without a compute_mode argument:
    oneapi::mkl::blas::gemm(my_queue, /* ... */);
}

export MKL_BLAS_COMPUTE_MODE=FLOAT_TO_BF16X3
my_application      # gemm may use bf16x3 arithmetic.
```

Any per-call or per-source-file mode settings take precedence over the `MKL_BLAS_COMPUTE_MODE` environment variable. One result of this is that compiling an application with `-DMKL_BLAS_COMPUTE_MODE=oneapi::mkl::blas::compute_mode::standard` effectively disables the environment variable.

Checking Which Mode Is Used

On GPU, when oneMKL's verbose mode is enabled, information on the compute mode(s) enabled and used for each call is provided in the verbose log. For example, in the following call, `float_to_bf16` and `float_to_tf32` were both enabled, and `float_to_bf16` was selected (some output omitted for clarity):

```
MKL_VERBOSE oneapi::mkl::blas::column_major::gemm[float](0x7ffd39046350,...,float_to_bf16|
float_to_tf32) mode:float_to_bf16 host:nan device:nan GPU0
```

Verbose mode can be enabled by setting the `MKL_VERBOSE` environment variable to 1, or via the `mkl_verbose` API. For more information, see "Using oneMKL Verbose Mode" in the Intel(R) oneAPI Math Kernel Library Developer Guide, available in the [Intel Software Documentation Library](#).

Numerical Reproducibility

Executing a BLAS routine with the same inputs may result in slightly different results from run to run. This is due, fundamentally, to the limited precision of floating-point arithmetic. In particular, the results of a summation

$$a_0 + a_1 + \dots + a_n$$

will depend on the order of operations, because floating-point addition is not associative. Because of this nonassociative behavior, some algorithms used by oneMKL BLAS routines may produce slightly different results depending on the ordering of internal operations.

In some cases, it may be desirable to ensure deterministic (run-to-run bitwise reproducible) results. oneMKL supports *conditional numerical reproducibility (CNR)* modes that provide bitwise reproducibility guarantees under certain circumstances. CNR mode is available on CPU for all BLAS DPC++ APIs, and on GPU for level-3 routines and level-3 extensions.

More information on CNR mode may be found in the oneMKL Developer Guide ([Windows](#), [Linux](#)).

Sparse BLAS Routines

The Intel® oneAPI Math Kernel Library (oneMKL) provides a C++ with SYCL interface to some of the Sparse BLAS routines. This section describes such Sparse BLAS routines included in oneMKL.

The Sparse BLAS domain provides basic operations on sparse matrices and vectors. The fundamental object that encompasses the sparse matrix and that is used in the Sparse BLAS domain is `sparse::matrix_handle_t`. See [Sparse BLAS Matrix Handle Contract between User and Library](#) for a discussion of the way that library APIs will interact with the handle as well as the usage contract with regards to the matrix handle for both users and the library. The currently supported sparse matrix formats in the `sparse::matrix_handle_t` object can be found in [Sparse Storage Formats](#). In addition to this main sparse matrix handle, there are other data structures and `enum` objects that are part of the sparse BLAS domain, listed in the table below.

Structure/enum	Description
<code>sparse::matrix_handle_t</code>	Pointer to an opaque sparse matrix object
sparse::property	User-provided matrix data guarantees (such as sorted input data)

Structure/enum	Description
<code>sparse::omatadd_alg</code>	Algorithm <code>enum</code> for use with the <code>sparse::omatadd</code> API
<code>sparse::omatadd_descr_t</code>	Pointer to an opaque descriptor object for use with the <code>sparse::omatadd</code> API
<code>sparse::matrix_view_descr</code>	<code>enum</code> for use with the <code>sparse::set_matmat_data</code> API
<code>sparse::matmat_request</code>	<code>enum</code> for use with the <code>sparse::matmat</code> API
<code>sparse::matmat_descr_t</code>	Pointer to an opaque descriptor object for use with the <code>sparse::matmat</code> API
<code>sparse::omatconvert_alg</code>	Algorithm <code>enum</code> for use with the <code>sparse::omatconvert</code> API
<code>sparse::omatconvert_descr_t</code>	Pointer to an opaque descriptor object for use with the <code>sparse::omatconvert</code> API

The routines that enable these Sparse BLAS operations can be separated into 4 groups:

1. State management routines
2. Analysis routines (also called inspector stage or optimize stage routines)
3. Execution routines
4. Helper routines.

The state management routines include initialization, destruction and APIs for setting data, formats and properties in the different sparse objects like the `sparse::matrix_handle_t` or `sparse::matmat_descr_t`.

In an analysis routine, the library inspects the matrix properties including size, sparsity pattern and available parallelism and can create new data structures or copies of the user data which applies matrix format or structure changes to enable a more optimized algorithm for the desired operation. The user data is not changed by any such analysis or optimizations. The optimizations created in the analysis routines may be reused by multiple execution routines to improve performance. For a given matrix, an analysis routine would typically be called a single time for each operation whereas the corresponding execution routines may be called multiple times.

The execution routines are where the actual matrix-matrix, matrix-vector operations take place and may use the data, optimizations and properties stored in the handle to perform the desired operation.

The helper routines operate on the data in the matrix handle and may include things like data copy/transpose into other handles, sorting of data within a handle, or converting the sparse matrix format from one handle to another.

State Management Routines	Data Types	Description
<code>sparse::init_matrix_handle</code>	N/A	Initialize a <code>sparse::matrix_handle_t</code> object
<code>sparse::release_matrix_handle</code>	N/A	Release a <code>sparse::matrix_handle_t</code> object
<code>sparse::set_csr_data</code>	float, double, <code>std::complex<float></code> , <code>std::complex<double></code>	Set internal representation of <code>sparse::matrix_handle_t</code> to compressed sparse row (CSR) format with user-provided CSR data arrays.

State Management Routines	Data Types	Description
sparse::set_coo_data	float, double, std::complex<float>, std::complex<double>	Set internal representation of sparse::matrix_handle_t to coordinate (COO) format with user-provided COO data arrays.
sparse::set_matrix_property	N/A	Set special properties of user-provided matrix data in sparse::matrix_handle_t that can provide hints for optimizations.
sparse::init_omatadd_descr	N/A	Initialize a sparse::omatadd_descr_t object. For use with sparse::omatadd
sparse::release_omatadd_descr	N/A	Release a sparse::omatadd_descr_t object. For use with sparse::omatadd
sparse::init_matmat_descr	N/A	Initialize a sparse::matmat_descr_t object. For use with sparse::matmat
sparse::set_matmat_data	N/A	Populate a sparse::matmat_descr_t object with desired operation description. For use with sparse::matmat
sparse::get_matmat_data	N/A	Query the operation description housed in a sparse::matmat_descr_t object. For use with sparse::matmat
sparse::release_matmat_descr	N/A	Release a sparse::matmat_descr_t object. For use with sparse::matmat
sparse::init_omatconvert_descr	N/A	Initialize a sparse::omatconvert_descr_t object. For use with sparse::omatconvert
sparse::release_omatconvert_descr	N/A	Release a sparse::omatconvert_descr_t object. For use with sparse::omatconvert
Analysis Routines	Data Types	Description
sparse::optimize_gemv	N/A	Perform internal optimizations for the sparse::gemv operation.

Analysis Routines	Data Types	Description
<code>sparse::optimize_trmv</code>	N/A	Perform internal optimizations for the <code>sparse::trmv</code> operation.
<code>sparse::optimize_trsv</code>	N/A	Perform internal optimizations for the <code>sparse::trsv</code> operation.
<code>sparse::optimize_gemm</code>	N/A	Perform internal optimizations for the <code>sparse::gemm</code> operation.
<code>sparse::optimize_trsm</code>	N/A	Perform internal optimizations for the <code>sparse::trsm</code> operation.

Execution Routines	Data Types	Description
<code>sparse::gemv</code>	float, double, std::complex<float>, std::complex<double>	General sparse matrix-dense vector product
<code>sparse::gemvdot</code>	float, double, std::complex<float>, std::complex<double>	General sparse matrix-dense vector product with fused dot product
<code>sparse::symv</code>	float, double, std::complex<float>, std::complex<double>	Symmetric sparse matrix-dense vector product
<code>sparse::trmv</code>	float, double, std::complex<float>, std::complex<double>	Triangular sparse matrix-dense vector product
<code>sparse::trsv</code>	float, double, std::complex<float>, std::complex<double>	Triangular solve of sparse matrix against a dense vector.
<code>sparse::gemm</code>	float, double, std::complex<float>, std::complex<double>	General sparse matrix-dense matrix product
<code>sparse::trsm</code>	float, double, std::complex<float>, std::complex<double>	Triangular solve of sparse matrix against a dense matrix.
<code>sparse::omatadd</code>	float, double, std::complex<float>, std::complex<double>	General sparse matrix-sparse matrix addition
<code>sparse::matmat</code>	float, double, std::complex<float>, std::complex<double>	General sparse matrix-sparse matrix product
<code>sparse::matmatd</code>	float, double, std::complex<float>, std::complex<double>	Product of two sparse matrices with a dense result

Helper Routines	Data Types	Description
<code>sparse::omatcopy</code>	float, double, std::complex<float>, std::complex<double>	General sparse matrix out-of-place copy/transposition into a new matrix handle

Helper Routines	Data Types	Description
<code>sparse::omatconvert</code>	float, double, std::complex<float>, std::complex<double>	General sparse matrix out-of-place conversion into a new matrix handle
<code>sparse::sort_matrix</code>	float, double, std::complex<float>, std::complex<double>	General sparse matrix sort of matrix format in matrix handle
<code>sparse::update_diagonal_values</code>	float, double, std::complex<float>, std::complex<double>	Change values of the main diagonal entries in matrix handle

Sparse BLAS Matrix Handle Contract between User and Library

The sparse matrix handle (`sparse::matrix_handle_t`) takes in data from the **User** in a call of the form `sparse::set_<format>_data(q, handle, /*user data*/)`, for example `sparse::set_csr_data(q, handle, /*user data*/)`. Unlike most other oneMKL domains, the `sparse::matrix_handle_t` along with user data persists outside of individual calls to the oneMKL library. When the **User** subsequently makes a call to a oneMKL Sparse BLAS API with that handle, the **Library** uses that data stored in the handle internally for the various operations. Because both users and library have access to the data at the same time while the matrix handle exists, there must be some agreements about what can and cannot be done by each party. We therefore introduce an implicit contract between the **User** and the **Library** that describes each party's roles and responsibilities with respect to the Sparse BLAS SYCL APIs and the use of the `sparse::matrix_handle_t`.

Description of `sparse::matrix_handle_t` object

First, we note that the `sparse::matrix_handle_t` represents a generic sparse matrix object with some internal matrix format or representation. Providing device USM pointers for that matrix data in the `sparse::matrix_handle_t` object binds that handle to the `sycl::context` and `sycl::device` associated with the device USM pointer and can be used on any device within the context that is compatible with that `sycl::device` (can read from the particular device peer to peer). Using shared and host USM pointers or `sycl::buffer`'s for the matrix data binds that `sparse::matrix_handle_t` object to the associated `sycl::context` and could be used on any device in the context. It is the **User's** responsibility to make sure they are using the matrix handle with appropriate queues and devices within the given context.

Second, the `sparse::matrix_handle_t` can best be described as a "view of **User's** matrix data arrays with an opaque state attached to it". That is, it is a lightweight view to begin with, but through the use of `sparse::optimize_*` APIs and in some cases, the execute APIs (like `sparse::matmat()`), the hidden state could grow with different internal optimizations/structures that can persist through the lifetime of the handle, with the goal of enabling superior performance for the desired operations.

User and Library agreements with respect to `sparse::matrix_handle_t` object

The following agreements describe **User** and **Library** roles with respect to the handle data ownership and use.

User agreements:

1. The **User** owns any data that is provided to the `sparse::matrix_handle_t` and is responsible to create and dispose of them correctly. That is, the data is created by the **User** then passed to the matrix handle using a **Library** API such as `sparse::set_csr_data()`. The data can be disposed of only after all uses of the matrix handle and the release of the matrix handle have finished.

2. The **User** agrees not to modify the data arrays directly while they are in a `sparse::matrix_handle_t`. Any changes must happen before attaching to a matrix handle or, when available, indirectly through use of a Sparse BLAS library API that states it will modify the matrix handle data in specified ways.

Library agreements:

1. The **Library** may create and store data in the `sparse::matrix_handle_t` during a Sparse BLAS API library call. Such data is owned by the Library, and the **Library** is responsible for its disposal which occurs at the respective [matrix handle release](#). The **User** cannot access this data.
2. The **Library** agrees to not modify the user-provided data arrays in the `sparse::matrix_handle_t` unless through a library API that specifically states it may change the data (such as `sparse::sort_matrix()` or `sparse::omatcopy()`).

With the understanding of these agreements, along with understanding of the [more advanced use recommendations](#), the **User** can safely use the oneMKL SYCL Sparse BLAS APIs to accelerate their workloads and applications, and the **Library** can rely on a consistent and known state of the user data through the lifetime of the handle.

Advanced use recommendations

1. The `sparse::matrix_handle_t` object is **not** currently considered thread-safe, so it must be used serially on the host.
2. Most of the Sparse BLAS operations consume the user-provided data in the `sparse::matrix_handle` in a read-only fashion. Exceptions are APIs that are filling a `sparse::matrix_handle_t` such as `sparse::matmat()` for the output C matrix (A and B are still read-only) or `sparse::omatcopy()` and other APIs that explicitly state they are modifying the user data, such as `sparse::sort_matrix()`.
 - a. It is recommended for the **User** to **not** use the same data arrays in multiple `sparse::matrix_handle_t`'s on the same context and device, but with care it may be possible to do so safely. Doing so may affect the performance.
 - b. It is also recommended for the **User** to **not** use the raw data arrays that are being used in a `sparse::matrix_handle_t` in a read-only fashion in other kernels or library calls independent of the `sparse::matrix_handle_t` on the same context and device, but with care it may be possible to do so safely. Doing so may affect the performance.

Example of `sparse::matrix_handle_t` usage workflow

An example workflow to demonstrate the usage model is the following:

```
// (A) Allocate user memory for sparse matrix such as with SYCL USM device alloc or sycl::buffer
// (B) Do anything with matrix arrays, such as memcpy some data from host
// (C) Create a sparse matrix handle and pass in matrix arrays
// (D) Make some library calls of your choice (for instance with matrix handle)
// (E) Destroy the matrix handle via call to sparse::release_matrix_handle()
// (F) Do anything with user memory - modify, copy, etc
// (G) Create another sparse matrix handle using the same arrays
// (H) Make other library calls of your choice
// (I) Destroy the matrix handle via call to sparse::release_matrix_handle()
// (J) Do whatever you want with user memory
```

Sparse BLAS Supported Data and Integer Types

Data Types <DATA_TYPE>	Integer Types <INT_TYPE>
float	std::int32_t

Data Types <DATA_TYPE>	Integer Types <INT_TYPE>
double	std::int64_t
std::complex<float>	
std::complex<double>	

Sparse Storage Formats

There are a variety of matrix storage formats available for representing a sparse matrix. The Intel® oneAPI Math Kernel Library (oneMKL) Sparse BLAS library provides support for the following sparse matrix formats:

Sparse Matrix Formats Supported in oneMKL Sparse BLAS
Compressed Sparse Row (CSR)
Coordinate (COO)

NOTE On repeated indices/non-zeros in sparse matrices: By the nature of sparse matrix formats, it is possible to create sparse matrices with so-called 'repeated' indices where the actual value of the non-zero is considered as a sum of its values where the non-zero row and column indices are repeated. Using such matrices in this form with oneMKL is **currently undefined behavior** with no guarantees of correctness. It is user-responsibility to ensure that non-zero values are not repeated and are appropriately 'compressed' into a single non-zero value. Unless otherwise explicitly specified examples, statements, and explanations in the oneMKL documentation assume sparse matrix arrays without repeated indices representing a given non-zero value.

NOTE On explicit zeros in sparse matrices: It is possible to create sparse matrices with structural non-zeros that have explicitly stored zero values. All references of 'non-zeros' in the oneMKL Sparse BLAS domain mean structural non-zeros that are stored in the matrix (which includes explicitly stored zero values).

Compressed Sparse Row (CSR)

One of the most common sparse formats is the CSR (sometimes called 3-array CSR or CSR3) format that is represented by scalar sizes (`nrows`, `ncols`), as well as three data arrays: `row_ptr`, `col_ind` and `values`, and the `index_base` parameter. Some versions of this format also explicitly store the number of non-zero elements (`nnz`) but this can be extracted from the `row_ptr` array as described below in the description of the `row_ptr` so we will not include it here.

CSR Matrix Format Elements	Description
<code>nrows</code>	Number of rows in the sparse matrix.
<code>ncols</code>	Number of columns in the sparse matrix.
<code>index</code>	Parameter that is used to specify whether the matrix has zero or one-based indexing.
<code>values</code>	An array that contains the non-zero elements of the sparse matrix stored row by row.

CSR Matrix Format Elements	Description
<code>col_ind</code>	An integer array of column indices for non-zero elements stored in the <code>values</code> array, such that <code>col_ind[i]</code> is the column number (using zero- or one-based indexing) of the element of the sparse matrix stored in <code>values[i]</code> .
<code>row_ptr</code>	An integer array of size equal to <code>nrows + 1</code> . Element <code>j</code> of this integer array gives the position of the element in the <code>values</code> array that is first non-zero element in a row <code>j</code> of <code>A</code> . Note that this position is equal to <code>row_ptr[j] - index</code> . The last element of the <code>row_ptr</code> array (<code>row_ptr[nrows]</code>) stores the sum of the number of non-zero elements (<code>nnz</code>) and <code>index</code> . That is, <code>nnz = row_ptr[nrows] - index</code> .

The 3-array CSR format that oneMKL supports has sorted rows by definition. However, the column indices within each row may or may not be sorted (in ascending order) leading to a possibility of unsorted data. The CSR format (with no repeated column indices that oneMKL currently requires) has a unique sorted form. However, in order to provide complete functional support for CSR matrices in either situation, oneMKL assumes CSR matrix handles to be **unsorted by default**. If the CSR format arrays are known to be in sorted form where all column indices within each row are sorted in ascending order, then users are strongly encouraged to provide that information to oneMKL through the [`sparse::set\_matrix\_property\(\)`](#) API to possibly allow better algorithm choices and optimizations to deliver better performance. This is particularly important for APIs like triangular solve or sparse matrix addition.

A summary of sorted states for CSR format is given in the table below.

CSR Sortedness State	Description	Appropriate oneMKL <code>sparse::property</code>
Unsorted	Column indices within each row may not be ordered	None (default)
Sorted	Sorted by rows and sorted by columns within each row	<code>sparse::property::sorted</code>

Examples of CSR format

The following 3 examples show how the sparse CSR format can be used:

- [CSR Case 1: Sorted square matrix with zero-based indexing](#)
- [CSR Case 2: Sorted rectangular matrix with one-based indexing and an empty row](#)
- [CSR Case 3: Unsorted rectangular matrix with zero-based indexing](#)

CSR Case 1: Sorted square matrix with zero-based indexing

Assuming zero-based indexing and a real square matrix.

$$A = \begin{pmatrix} 1.0 & 0.0 & 2.0 \\ 0.0 & -1.0 & 4.0 \\ 3.0 & 0.0 & 0.0 \end{pmatrix}$$

nrows	3				
ncols	3				
index	0				
row_ptr	0	2	4	5	
col_ind	0	2	1	2	0
values	1.0	2.0	-1.0	4.0	3.0

CSR Case 2: Sorted rectangular matrix with one-based indexing and an empty row

Assuming one-based indexing and real rectangular matrix with an empty row.

$$A = \begin{pmatrix} 1.0 & 0.0 & 2.0 & 0.0 & 0.0 \\ 0.0 & -1.0 & 4.0 & 0.0 & 1.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 3.0 & 0.0 & 0.0 & 1.0 & 0.0 \end{pmatrix}$$

nrows	4						
ncols	5						
index	1						
row_ptr	1	3	6	6	8		
col_ind	1	3	2	3	5	1	4
values	1.0	2.0	-1.0	4.0	1.0	3.0	1.0

CSR Case 3: Unsorted rectangular matrix with zero-based indexing

Unsorted CSR example: Assuming zero-based indexing and a real rectangular matrix, we note that the CSR format does not require column indices to be sorted within a given row, but `values` and `col_ind` arrays must be consistent with each other. Having the sorted property is not necessary, but can lead to better

performance in actual runs due to better algorithms and data locality being enabled. See [sparse::set_matrix_property\(\)](#) and [sparse::sort_matrix](#) for more details on how one could set the sorted property when it is applicable or reorder the matrix to achieve it when desired.

$$A = \begin{pmatrix} 1.0 & 0.0 & 2.0 & 0.0 & 0.0 \\ 0.0 & -1.0 & 4.0 & 0.0 & 1.0 \\ 1.0 & 2.0 & 3.0 & 4.0 & 0.0 \\ 3.0 & 0.0 & 0.0 & 0.0 & 0.0 \end{pmatrix}$$

nrows	4									
ncols	5									
index	0									
row_ptr	0	2	5	9	10					
col_ind	0	2	4	1	2	1	2	0	3	0
values	1.0	2.0	1.0	-1.0	4.0	2.0	3.0	1.0	4.0	3.0

Coordinate (COO) format

The simplest sparse matrix format is the COO format, represented by scalar sizes (`nrows`, `ncols`, `nnz`), as well as three data arrays: `row_ind`, `col_ind`, and `values`, along with an `index` parameter. Each tuple, (`row_ind[i]`, `col_ind[i]`, `values[i]`), represents a non-zero value in the sparse matrix.

NOTE In the case of COO format, the transpose of a matrix represented with the array triplet (`row_ind`, `col_ind`, `values`) is (`col_ind`, `row_ind`, `values`); that is, interchanging the row and column index arrays of a COO matrix gives its transposed form.

Unlike CSR format which has a dichotomy of unsorted versus sorted forms, COO format arrays have five broad sortedness states, two of which are unique:

- Arbitrary, non-unique unsorted order,
- Two partially sorted non-unique forms corresponding to
 - Unsorted CSR-style state with sorted row indices but column indices within rows being unsorted,
 - Unsorted compressed sparse column (CSC)-style state with sorted column indices but row indices within columns being unsorted,
- Two unique, fully sorted forms (assuming no repeated indices contributing to the same structural non-zero location) corresponding to
 - Sorted CSR-style state with sorted row indices then column indices sorted within rows, and
 - Sorted CSC-style state with sorted column indices and row indices sorted within columns.

Knowing which of the above sortedness states the COO arrays are in allows for different optimization and algorithm choices as well as trade-offs in math APIs using COO format. However, in order to provide complete functional support for COO matrices out-of-the-box, oneMKL assumes them to be **fully unsorted by default**. In the context of COO matrices, the `sparse::property::sortedenum` (see [sparse::property](#))

corresponds to the sorted CSR-style state that users can specify on the handle, if their arrays are sorted in that form, to allow oneMKL to possibly deliver better performance compared to default unsorted case. oneMKL currently has no plans to add support for either sorted or unsorted CSC-style states. A future release of oneMKL may add a `sparse::property::partially_sorted` or a `sparse::property::sorted_by_rows` property to allow users to indicate partially sorted (i.e., unsorted CSR-style) COO arrays to the library. The current release of oneMKL has no difference in treatment between partially sorted COO arrays, which are simply treated as unsorted by default, but does have and will continue to add optimizations for `sparse::property::sorted` case. Therefore, whenever applicable, users are strongly encouraged to specify the `sparse::property::sorted` property using the `sparse::set_matrix_property()` API on the matrix handle if the COO arrays are known to be in sorted-CSR form.

Note that `sparse::property::sorted` must not be specified on a COO matrix handle created with arrays that are CSC-style sorted as that property is meant for CSR-style sorted state of COO arrays. However, because the transpose of a COO matrix is immediately available by interchanging its row and column index arrays, the transpose of a COO matrix with CSC-style-sorted arrays is automatically in CSR-style-sorted state. To possibly still take advantage of optimizations of CSC-style sorted arrays, users can create a matrix handle for the transposed matrix with the original arrays interchanged and specify the `sparse::property::sorted` on that handle with the `sparse::set_matrix_property()` API. Accordingly, subsequent users should also adjust subsequent usage of other APIs with this transposed handle appropriately using the `mkl::transpose` enum with the APIs. For example, non-transpose `sparse::gemv()` with sorted-CSC-style COO matrix is not allowed to use the `sparse::property::sorted` property on the matrix, but the same operation is equivalent to performing `transpose::gemv` with the transposed COO matrix handle which is easily available by interchanging the `row_ind` and `col_ind` arrays. The transposed handle then naturally has sorted-CSR-style arrays which allows users to specify the `sparse::property::sorted` on it. In other words, `sparse::gemv(q, mkl::transpose::nontrans, cooAMatrix, ...)` cannot make use of `sparse::property::sorted` on sorted-CSC-style arrays in `cooAMatrix`, but `sparse::gemv(q, mkl::transpose::trans, transCooAMatrix, ...)` can make use of `sparse::property::sorted` on `transCooAMatrix` which then has sorted-CSR-style arrays. Likewise, instead of `sparse::gemv(q, mkl::transpose::trans, cooAMatrix, ...)`, users can use `sparse::gemv(q, mkl::transpose::nontrans, transCooAMatrix, ...)` to take advantage of the `sparse::property::sorted` property.

A summary of COO format sorted states is given in the table below.

COO Sortedness State	Description	Appropriate oneMKL <code>sparse::property</code>
Unsorted	Unsorted	None (default)
Unsorted CSR style/ partially sorted	Sorted by rows, but columns within each row need not be ordered	None (support may be added in the future)
Unsorted CSC style/ partially sorted	Sorted by columns, but rows within each column need not be ordered	None (no plans to add support)
Sorted CSR style	Sorted by rows and sorted by columns within each row	<code>sparse::property::sorted</code>
Sorted CSC style	Sorted by columns and sorted by rows within each column	None (no plans to add support)

COO Matrix Format Elements	Description
<code>nrows</code>	Number of rows in the sparse matrix.

COO Matrix Format Elements	Description
<code>ncols</code>	Number of columns in the sparse matrix.
<code>nnz</code>	Number of non-zeros in the sparse matrix.
<code>index</code>	Parameter that is used to specify whether the matrix has zero or one-based indexing.
<code>row_ind</code>	An integer array of row indices of non-zero elements in the <code>values</code> array, such that <code>row_ind[i]</code> is the row number (using zero- or one-based indexing) of the element of the non-zero value in <code>values[i]</code> .
<code>col_ind</code>	An integer array of column indices of non-zero elements in the <code>values</code> array, such that <code>col_ind[i]</code> is the column number (using zero- or one-based indexing) of the element of the non-zero value in <code>values[i]</code> .
<code>values</code>	An array that contains the non-zero elements of the sparse matrix, preferably but not necessarily stored in a sorted order.

Examples of COO format

The following examples show how the sparse COO format can be used:

- [COO Case 1: Unsorted rectangular matrix with zero-based indexing](#)
- [COO Case 2: Sorted rectangular matrix with one-based indexing](#)
- [COO Case 3: Partially sorted rectangular matrix with one-based indexing](#)

COO Case 1: Unsorted rectangular matrix with zero-based indexing

An example of an unsorted COO matrix in zero-based indexing and a real rectangular matrix are given below.

$$A = \begin{pmatrix} 1.0 & 0.0 & 2.0 & 0.0 & 0.0 \\ 0.0 & -1.0 & 4.0 & 0.0 & 1.0 \\ 1.0 & 2.0 & 3.0 & 4.0 & 0.0 \\ 3.0 & 0.0 & 0.0 & 0.0 & 0.0 \end{pmatrix}$$

<code>nrows</code>	4									
<code>ncols</code>	5									
<code>nnz</code>	10									
<code>index</code>	0									
<code>row_ind</code>	2	1	0	1	2	3	0	1	2	2

col_ind	1	4	2	1	0	0	0	2	3	2
values	2.0	1.0	2.0	-1.0	1.0	3.0	1.0	4.0	4.0	3.0

COO Case 2: Sorted rectangular matrix with one-based indexing

Consider the following dense matrix containing many zero values.

$$A = \begin{pmatrix} 1.0 & 0.0 & 2.0 & 0.0 & 0.0 \\ 0.0 & -1.0 & 4.0 & 0.0 & 1.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 3.0 & 0.0 & 0.0 & 1.0 & 0.0 \end{pmatrix}$$

Its two fully sorted forms, viz., CSR-style sorted and CSC-style sorted representations, are given below.

CSR-style sorted form:

nrows	4						
ncols	5						
nnz	7						
index	1						
row_ind	1	1	2	2	2	4	4
col_ind	1	3	2	3	5	1	4
values	1.0	2.0	-1.0	4.0	1.0	3.0	1.0

CSC-style sorted form:

nrows	4						
ncols	5						
nnz	7						
index	1						
row_ind	1	4	2	1	2	4	2
col_ind	1	1	2	3	3	4	5
values	1.0	3.0	-1.0	2.0	4.0	1.0	1.0

COO Case 3: Partially sorted rectangular matrix with one-based indexing

Consider the following dense matrix containing many zero values.

$$A = \begin{pmatrix} 1.0 & 0.0 & 2.0 & 0.0 & 0.0 \\ 0.0 & -1.0 & 4.0 & 0.0 & 1.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 3.0 & 0.0 & 0.0 & 1.0 & 0.0 \end{pmatrix}$$

Two of its non-unique partially sorted forms, viz., unsorted-CSR-style and unsorted-CSC-style representations, are given below.

CSR-style unsorted/partially sorted form:

nrows	4						
ncols	5						
nnz	7						
index	1						
row_ind	1	1	2	2	2	4	4
col_ind	1	3	5	2	3	4	1
values	1.0	2.0	1.0	-1.0	4.0	1.0	3.0

CSC-style unsorted/partially sorted form:

nrows	4						
ncols	5						
nnz	7						
index	1						
row_ind	4	1	2	1	2	4	2
col_ind	1	1	2	3	3	4	5
values	3.0	1.0	-1.0	2.0	4.0	1.0	1.0

oneapi::mkl::sparse::init_matrix_handle

Allocates memory on the heap for a `oneapi::mkl::sparse::matrix_handle_t` object, initializes the internals of that object to default values, and returns with the address to that object.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::init_matrix_handle` function initializes the `oneapi::mkl::sparse::matrix_handle_t` object with default values, otherwise it throws an exception.

NOTE Refer to [Error Handling](#) for a detailed description of the exceptions thrown.

The following is a recommended usage model:

```
using namespace oneapi::mkl;

// set newly created handle to nullptr before sending to
// init_matrix_handle for initialization

sparse::matrix_handle_t spMat = nullptr;
sparse::init_matrix_handle(&spMat);

// handle is now initialized and can be filled and used

sycl::event ev_set = sparse::set_csr_data(queue, spMat, /*matrix sizes and arrays */);

// when finished, clean up the handle
sycl::event ev_release = sparse::release_matrix_handle(queue, &spMat, dependencies);
ev_release.wait(); // make it blocking
```

API

Syntax

```
namespace oneapi::mkl::sparse {
    void init_matrix_handle (
        oneapi::mkl::sparse::matrix_handle_t *p_spMat)
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

<code>p_spMat</code>	The address of the <code>sparse::matrix_handle_t</code> handle to be initialized. We recommend that the <code>sparse::matrix_handle_t</code> handle object be set to <code>nullptr</code> before entry to this function to avoid confusion if it is previously initialized or not. This function allocates and initializes a <code>matrix_handle_t</code> object and sets the <code>p_spMat</code> to this new object. This initialization routine must only be called on an uninitialized <code>matrix_handle</code> object.
----------------------	---

Output Parameters

<code>p_spMat</code>	On return, the address is updated to point to a newly allocated and initialized <code>matrix_handle_t</code> object that can be filled and used to perform sparse BLAS operations.
----------------------	--

`oneapi::mkl::sparse::release_matrix_handle`

Releases internal data in the provided `oneapi::mkl::sparse::matrix_handle_t` object and then frees the handle itself.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::release_matrix_handle` routine releases (also waits for the dependencies to be finished when provided) any internal data that the `oneapi::mkl::sparse::matrix_handle_t` object holds. It then frees the object itself.

NOTE For oneMKL 2023.2 and beyond: Although the address of the `matrix_handle_t` object is passed in (`matrix_handle_t *p_spMat`), it is dereferenced (`matrix_handle_t handle = *p_spMat;`) and updated to `null` synchronously and then the `handle` is passed into the asynchronously scheduled destruction routines ensuring that asynchronous destruction is safe. In previous releases, the dereferencing occurred asynchronously, which can cause segmentation faults if `p_spMat` is a stack variable (such as `&handle`) and goes out of scope before the asynchronous execution (including the dereferencing) begins.

NOTE Refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

A recommended usage model is the following

```
using namespace oneapi::mkl;

// set newly created handle to nullptr before sending to
// init_matrix_handle for initialization

sparse::matrix_handle_t spMat = nullptr;
sparse::init_matrix_handle(&spMat);

// handle is now initialized and can be filled and used

sparse::set_csr_data(queue, spMat, /*matrix sizes and arrays */);

// when finished, clean up the handle
sycl::event ev_release = sparse::release_matrix_handle(queue, &spMat, dependencies);
ev_release.wait(); // make it blocking or pass event along to others
```

API

Syntax

```
namespace oneapi::mkl::sparse {

    sycl::event release_matrix_handle (
        sycl::queue & queue,
        oneapi::mkl::sparse::matrix_handle_t *p_spMat,
        const std::vector<sycl::event> &dependencies = {});

}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

queue	Specifies the SYCL command queue which will be used for SYCL kernels execution.
p_spMat	The address of the <code>sparse::matrix_handle_t</code> handle to object to be released, containing the sparse matrix and other internal data, initialized with <code>oneapi::mkl::sparse::init_matrix_handle</code> routine, and filled with user data using one of the <code>oneapi::mkl::sparse::set_<sparse_matrix_type>_data</code> routines. The <code>oneapi::mkl::sparse::optimize_xyz</code> routines may have also created additional internally allocated data which would be released in this call.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` and `coo` on both CPU and GPU devices.

dependencies	A vector of type <code>const std::vector<sycl::event></code> containing the list of any events that the <code>handle</code> depends on before executing the release of the matrix handle.
--------------	---

Output Parameters

p_spMat	The address of the <code>sparse::matrix_handle_t</code> handle that will be scheduled to be updated to point to a null object and the passed in handle will be scheduled for deallocation and cleanup.
---------	--

Return Values

<code>sycl::event</code>	SYCL event which can be waited upon or added as a dependency for the completion of the deallocation cleanup routines.
--------------------------	---

oneapi::mkl::sparse::set_csr_data

Takes a matrix handle and the user-provided Compressed Sparse Row (CSR) matrix arrays and fills the internal CSR data structure of the matrix handle.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::set_csr_data` routine takes a `sparse::matrix_handle_t` for a sparse matrix of dimensions `nrows` -by- `ncols` represented in the [CSR format](#), and fills the internal state of the matrix handle with the user provided arrays in CSR format. Refer to [the user/library contract](#) for information on the use of the `sparse::matrix_handle_t` object.

NOTE Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

API

Syntax

Using SYCL buffers:

```
namespace oneapi::mkl:sparse {
    void set_csr_data (
        sycl::queue &queue,
        oneapi::mkl:sparse::matrix_handle_t spMat,
        const INT_TYPE nrows,
        const INT_TYPE ncols,
        oneapi::mkl:index_base index,
        sycl::buffer<INT_TYPE, 1> &row_ptr,
        sycl::buffer<INT_TYPE, 1> &col_ind,
        sycl::buffer<DATA_TYPE, 1> &values);
}
```

Using USM pointers:

```
namespace oneapi::mkl:sparse {

    sycl::event set_csr_data (
        sycl::queue &queue,
        oneapi::mkl:sparse::matrix_handle_t spMat,
        const INT_TYPE nrows,
        const INT_TYPE ncols,
        oneapi::mkl:index_base index,
        INT_TYPE *row_ptr,
        INT_TYPE *col_ind,
        DATA_TYPE *values,
        const std::vector<sycl::event> &dependencies = {} );

}
```

Include Files

- oneapi/mkl/spblas.hpp

Input Parameters

queue	Specifies the SYCL command queue which will be used for SYCL kernels execution.
spMat	Handle to object containing sparse matrix and other internal data for subsequent Sparse BLAS operations.
nrows	Number of rows of the input matrix.
ncols	Number of columns of the input matrix.
index	Indicates how input arrays are indexed.

oneapi::mkl:index_base::zero	Zero-based (C-style) indexing: indices start at 0.
oneapi::mkl:index_base::one	One-based (Fortran-style) indexing: indices start at 1.

row_ptr	SYCL memory object containing an array of length <code>nrows+1</code> , containing row-wise starting and ending locations of non-zero column indices and values in <code>index</code> -based numbering. Could be a SYCL buffer or a device-accessible USM pointer. Refer to Sparse Storage Formats for a detailed description of <code>row_ptr</code> .
---------	---

col_ind	SYCL memory object which stores an array containing the column indices in index-based numbering. Could be a SYCL buffer or a device-accessible USM pointer. Refer to Sparse Storage Formats for a detailed description of col_ind.
values	SYCL memory object which stores an array containing the non-zero elements of the input matrix. Could be a SYCL buffer or a device-accessible USM pointer. Refer to Sparse Storage Formats for a detailed description of values.
dependencies	USM API only. A vector of type <code>const std::vector<sycl::event> &</code> containing the list of events that the <code>oneapi::mkl::sparse::set_csr_data</code> routine depends on.

Summary of Input CSR Arrays

A summary of the CSR input arrays, sizes, and types (see [Sparse BLAS Supported Data and Integer Types](#)) is contained in the following table. We use the term `number of non-zeros` or for short, `nnz`, to refer to the count of elements of the matrix and sparsity pattern represented by the CSR matrix format as described in [Sparse Storage Formats](#).

For `sycl::buffer` inputs, the arrays are of type `sycl::buffer<T, 1> &`.

For USM inputs, the arrays must be device-accessible and of type `T *`. The recommended USM memory type is described in the following table. In general, using USM device memory will provide a better performance than USM shared, which in turn will give better performance than USM host, but all are supported as they are all device-accessible.

array	length (elements)	T	USM Memory Type
row_ptr	nrows + 1	INT_TYPE	device accessible (USM device or USM shared or USM host)
col_ind	nnz	INT_TYPE	device accessible (USM device or USM shared or USM host)
values	nnz	DATA_TYPE	device accessible (USM device or USM shared or USM host)

Output Parameters

spMat	Handle to object containing sparse matrix and other internal data for subsequent DPC++ Sparse BLAS operations.
sycl::event	USM API only. A <code>sycl::event</code> that can be used to track the completion of asynchronous events that were enqueued during the API call that continue the chain of events from the input dependencies.

oneapi::mkl::sparse::set_coo_data

Takes a matrix handle and the user-provided Coordinate (COO) matrix format arrays and fills the internal COO data structure of the matrix handle.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::set_coo_data` routine takes a `sparse::matrix_handle_t` for a sparse matrix of dimensions `nrows` -by- `ncols` represented in the [COO format](#), and fills the internal state of the matrix handle with the user provided arrays in COO format. Refer to [the user/library contract](#) for information on the use of the `sparse::matrix_handle_t` object.

NOTE Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

API

Syntax

Using SYCL buffers:

```
namespace oneapi::mkl::sparse {
    void set_coo_data (
        sycl::queue &queue,
        oneapi::mkl::sparse::matrix_handle_t spMat,
        const INT_TYPE nrows,
        const INT_TYPE ncols,
        const INT_TYPE nnz,
        oneapi::mkl::index_base index,
        sycl::buffer<INT_TYPE, 1> &row_ind,
        sycl::buffer<INT_TYPE, 1> &col_ind,
        sycl::buffer<DATA_TYPE, 1> &values);
}
```

Using USM pointers:

```
namespace oneapi::mkl::sparse {

    sycl::event set_coo_data (
        sycl::queue &queue,
        oneapi::mkl::sparse::matrix_handle_t spMat,
        const INT_TYPE nrows,
        const INT_TYPE ncols,
        const INT_TYPE nnz,
        oneapi::mkl::index_base index,
        INT_TYPE *row_ind,
        INT_TYPE *col_ind,
        DATA_TYPE *values,
        const std::vector<sycl::event> &dependencies = {} );

}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

<code>queue</code>	Specifies the SYCL command queue which will be used for SYCL kernels execution.
<code>spMat</code>	Handle to object containing sparse matrix and other internal data for subsequent Sparse BLAS operations.

nrows	Number of rows of the input matrix.
ncols	Number of columns of the input matrix.
nnz	Number of non-zero elements in the input matrix.
index	Indicates how input arrays are indexed. oneapi::mkl::index_base::zero Zero-based (C-style) indexing: indices start at 0. oneapi::mkl::index_base::one One-based (Fortran-style) indexing: indices start at 1.
row_ind	SYCL memory object of length <code>nnz</code> which stores an array containing the row indices in <code>index</code> -based numbering. Could be a SYCL buffer or a device-accessible USM pointer. Refer to Sparse Storage Formats for a detailed description of <code>row_ind</code> .
col_ind	SYCL memory object of length <code>nnz</code> which stores an array containing the column indices in <code>index</code> -based numbering. Could be a SYCL buffer or a device-accessible USM pointer. Refer to Sparse Storage Formats for a detailed description of <code>col_ind</code> .
values	SYCL memory object which stores an array containing the non-zero elements of the input matrix. Could be a SYCL buffer or a device-accessible USM pointer. Refer to Sparse Storage Formats for a detailed description of <code>values</code> .
dependencies	USM API only. A vector of type <code>const std::vector<sycl::event> &</code> containing the list of events that the <code>oneapi::mkl::sparse::set_coo_data</code> routine depends on.

Summary of Input COO Arrays

A summary of the COO input arrays, sizes, and types (see [Sparse BLAS Supported Data and Integer Types](#)) is contained in the following table. We use the term `number of non-zeros` or for short, `nnz`, to refer to the count of elements of the matrix and sparsity pattern represented by the COO matrix format as described in [Sparse Storage Formats](#).

For `sycl::buffer` inputs, the arrays are of type `sycl::buffer<T, 1> &`.

For USM inputs, the arrays must be device-accessible and of type `T *`. The recommended USM memory type is described in the following table. In general, using USM device memory will provide a better performance than USM shared, which in turn will give better performance than USM host, but all are supported as they are all device-accessible.

array	length (elements)	T	USM Memory Type
row_ind	nnz	INT_TYPE	device accessible (USM device or USM shared or USM host)
col_ind	nnz	INT_TYPE	device accessible (USM device or USM shared or USM host)

array	length (elements)	T	USM Memory Type
values	nnz	DATA_TYPE	device accessible (USM device or USM shared or USM host)

Output Parameters

spMat	Handle to object containing sparse matrix and other internal data for subsequent DPC++ Sparse BLAS operations.
sycl::event	USM API only. A <code>sycl::event</code> that can be used to track the completion of asynchronous events that were enqueued during the API call that continue the chain of events from the input dependencies.

oneapi::mkl::sparse::set_matrix_property

Sets matrix properties present in the user provided data provided to the `sparse::matrix_handle_t` that can serve as optimization hints for library algorithms. Properties are not verified by the library but accepted as truth from the user who specified them.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `oneapi::mkl::sparse::set_matrix_property` routine enables the user to set some properties of the user-provided matrix data in the `sparse::matrix_handle_t` object that can act as hints for the internal algorithms in subsequent library calls.

The `oneapi::mkl::sparse::property` enum class is defined in the `oneapi/mkl/spblas.hpp` header file

```
namespace oneapi::mkl::sparse {
    enum class property : char {
        symmetric,
        sorted
    };
}
```

where `symmetric` refers to the matrix being symmetric and the full pattern is present in the data arrays. The `sorted` property indicates that the matrix data is sorted in whatever manner is natural to the particular matrix format. See [sparse::sort_matrix](#) for more details on sorting. The library will not verify that these properties are true, but will take them as truth from the user. Setting them may affect performance as certain internal optimizations may not need to be done if they are present. The properties may also be set internally by the library when it is applicable (for instance after a call to [sparse::sort_matrix](#)).

A common usage model for setting matrix properites is the following:

```
using namespace oneapi::mkl;
sparse::matrix_handle_t spMat = nullptr;

sparse::init_matrix_handle(&spMat);

sparse::set_csr_data(main_queue, spMat, nrows, ncolumns,
                    index_base::zero, ia_buffer, ja_buffer, a_buffer);

// The csr ia/ja/a arrays are of a full square symmetric matrix that is sorted
```

```
// with increasing columns in each row, so we can set both symmetric and
// sorted property
sparse::set_matrix_property(spMat, sparse::property::symmetric);
sparse::set_matrix_property(spMat, sparse::property::sorted);

sparse::optimize_trsv(main_queue, uplo::lower, transpose::nontrans,
                      diag::nonunit, spMat);
sparse::optimize_trsv(main_queue, uplo::upper, transpose::nontrans,
                      diag::nonunit, spMat);
sparse::optimize_gemv(main_queue, transpose::nontrans, spMat);

// implement some algorithm using trsv/gemv operations (like a preconditioned
// conjugate gradient algorithm with symmetric Gauss-Seidel preconditioner)

sparse::release_matrix_handle(&spMat);
```

API

Syntax

```
namespace oneapi::mkl::sparse {
    void set_matrix_property(oneapi::mkl::sparse::matrix_handle_t spMat,
                           oneapi::mkl::sparse::property property_value);
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

spMat Handle to object containing sparse matrix and other internal data where property will be set. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` and `coo` on both CPU and GPU devices. However, for matrices in COO format, `property` is currently ignored but may enable enhanced performance for that format in the future.

property_value Sparse matrix property being set as in the sparse matrix handle object.

<code>sparse::property::symmetric</code>	data in matrix handle represents a symmetric matrix and has the full pattern and values provided
<code>sparse::property::sorted</code>	data in matrix handle is sorted according to natural state for the given format

Examples

An example of how to use `oneapi::mkl::sparse::set_matrix_property` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_conjugate_gradient.cpp
```

oneapi::mkl::sparse::optimize_gemv

Performs internal optimizations for oneapi::mkl::sparse::gemv by analyzing the matrix structure.

- [Description](#)
- [API](#)

Description

NOTE Refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

The oneapi::mkl::sparse::optimize_gemv routine analyzes matrix structure and performs optimizations. Optimized data is then stored in the matrix handle.

API

Syntax

Using USM and SYCL buffers:

```
namespace oneapi::mkl::sparse {
    sycl::event optimize_gemv (
        sycl::queue &queue,
        oneapi::mkl::transpose opA,
        oneapi::mkl::sparse::matrix_handle_t A,
        const std::vector<sycl::event> &dependencies = {});
}
```

Include Files

- oneapi/mkl/spblas.hpp

Input Parameters

queue	Specifies the SYCL command queue which will be used for SYCL kernels execution.
opA	Specifies operation <code>op()</code> on input matrix.

oneapi::mkl::transpose::nontrans	Non-transpose, $op(A) = A$.
oneapi::mkl::transpose::trans	Transpose, $op(A) = A^T$.
oneapi::mkl::transpose::conjtrans	Conjugate transpose, $op(A) = A^H$.

A Handle to object containing sparse matrix and other internal data. Created using one of the oneapi::mkl::sparse::set_sparse_matrix_type_data routines.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` and `coo` on both CPU and GPU devices.

dependencies A vector of type `std::vector<sycl::event>` & containing the list of events that the `oneapi::mkl::sparse::optimize_gemv` routine depends on.

Return Values

`sycl::event` SYCL event which can be waited upon or added as a dependency for the completion of the `optimize_gemv` routine.

```
sycl::event ev_opt = sparse::optimize_gemv(queue, opA,
A); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_gemv(queue, opA, A,
{}); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_gemv(queue, opA, A,
dependencies); // Allowed use in case of USM
sparse::optimize_gemv(queue, opA,
A); // OK -- Recommended use in case
of sycl::buffer
static_cast<void>(sparse::optimize_gemv(queue, opA,
A)); // OK -- Recommended use in case of sycl::buffer -
explicitly say we aren't using the event
sparse::optimize_gemv(queue, opA, A,
{}); // Not recommended in case of
sycl::buffer, but supported;
sparse::optimize_gemv(queue, opA, A,
dependencies); // Not recommended in case of
sycl::buffer, but supported;
sycl::event ev_opt = sparse::optimize_gemv(queue, opA,
A); // Allowed use in case of sycl::buffer, but most
sycl::buffer oneMKL APIs do not have ability to pass in dependencies
```

oneapi::mkl::sparse::optimize_trmv

Performs internal optimizations for `oneapi::mkl::sparse::trmv` by analyzing the matrix structure.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::optimize_trmv` routine analyzes matrix structure and performs optimizations. Optimized data is then stored in the matrix handle.

NOTE Refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

API

Syntax

Using USM and SYCL buffers:

```
namespace oneapi::mkl::sparse {
    sycl::event optimize_trmv (
        sycl::queue &queue,
        oneapi::mkl::uplo uplo_val,
        oneapi::mkl::transpose opA,
```

```

oneapi::mkl::diag diag_val,
oneapi::mkl::sparse::matrix_handle_t A,
const std::vector<sycl::event> &dependencies = {});
}

```

Include Files

- oneapi/mkl/spblas.hpp

Input Parameters

queue Specifies the SYCL command queue which will be used for SYCL kernels execution.

uplo_val Specifies which part of the matrix is to be processed.

<code>oneapi::mkl::uplo::lower</code>	The lower triangular matrix part is processed.
<code>oneapi::mkl::uplo::upper</code>	The upper triangular matrix part is processed.

opA Specifies operation `op()` on input matrix.

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $op(A) = A$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $op(A) = A^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $op(A) = A^H$.

NOTE Currently, the only supported case for operation is `oneapi::mkl::transpose::nontrans`.

diag_val Specifies if the diagonal used for computations is unit or not.

<code>oneapi::mkl::diag::nonunit</code>	Diagonal elements might not be equal to one.
<code>oneapi::mkl::diag::unit</code>	Diagonal elements are equal to one.

NOTE Currently, the only supported case for `diag_val` is `oneapi::mkl::diag::nonunit`.

A Handle to object containing sparse matrix and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` on CPU and GPU devices, and `coo` only on CPU device.

dependencies A vector of type `std::vector<sycl::event>` & containing the list of events that the `oneapi::mkl::sparse::optimize_trmv` routine depends on.

Return Values

sycl::event SYCL event which can be waited upon or added as a dependency for the completion of the `optimize_trmv` routine.

```
sycl::event ev_opt = sparse::optimize_trmv(queue, uplo_val, opA,
diag_val, A);           // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_trmv(queue, uplo_val, opA,
diag_val, A, {});       // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_trmv(queue, uplo_val, opA,
diag_val, A, dependencies); // Allowed use in case of USM
sparse::optimize_trmv(queue, uplo_val, opA, diag_val,
A);                     // OK Recommended use in case of
sycl::buffer
static_cast<void>(sparse::optimize_trmv(queue, uplo_val, opA, diag_val,
A));                     // OK Recommended use in case of sycl::buffer
(exPLICITLY say we aren't using the event)(will be used in examples)
sparse::optimize_trmv(queue, uplo_val, opA, diag_val, A,
{});                     // Not recommended in case of
sycl::buffer, but supported;
sparse::optimize_trmv(queue, uplo_val, opA, diag_val, A,
dependencies);           // Not recommended in case of
sycl::buffer, but supported;
sycl::event ev_opt = sparse::optimize_trmv(queue, uplo_val, opA,
diag_val, A);           // Allowed use in case of sycl::buffer, but
most sycl::buffer oneMKL APIs do not have ability to pass in
dependencies
```

oneapi::mkl::sparse::optimize_trsv

Performs internal optimizations for `oneapi::mkl::sparse::trsv` by analyzing the provided matrix structure and operation parameters.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::optimize_trsv` routine analyzes matrix structure and performs optimizations. Optimized data is then stored in the matrix handle.

NOTE Refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

API

Syntax

Using USM and SYCL buffers:

```
namespace oneapi::mkl::sparse {
    sycl::event optimize_trsv (
        sycl::queue &queue,
```

```

oneapi::mkl::uplo uplo_val,
oneapi::mkl::transpose opA,
oneapi::mkl::diag diag_val,
oneapi::mkl::sparse::matrix_handle_t A,
const std::vector<sycl::event> &dependencies = {});
}

```

Include Files

- oneapi/mkl/spblas.hpp

Input Parameters

queue Specifies the SYCL command queue which will be used for SYCL kernels execution.

uplo_val Specifies which part of the matrix is to be processed.

<code>oneapi::mkl::uplo::lower</code>	The lower triangular matrix part is processed.
<code>oneapi::mkl::uplo::upper</code>	The upper triangular matrix part is processed.

opA Specifies operation `op()` on input matrix.

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $op(A) = A$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $op(A) = A^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $op(A) = A^H$.

NOTE Currently, the only supported case for operation is `oneapi::mkl::transpose::nontrans`.

diag_val Specifies if the diagonal used for computations is unit or not.

<code>oneapi::mkl::diag::nonunit</code>	Diagonal elements might not be equal to one.
<code>oneapi::mkl::diag::unit</code>	Diagonal elements are equal to one.

A Handle to object containing sparse matrix and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` on CPU and GPU devices, and `coo` only on CPU device.

dependencies A vector of type `std::vector<sycl::event>` containing the list of events that the `oneapi::mkl::sparse::optimize_trsv` routine depends on.

Return Values (USM Only)

`sycl::event` SYCL event which can be waited upon or added as a dependency for the completion of the `optimize_trsv` routine.

```
sycl::event ev_opt = sparse::optimize_trsv(queue, uplo_val, opA,
diag_val, A); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_trsv(queue, uplo_val, opA,
diag_val, A, {}); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_trsv(queue, uplo_val, opA,
diag_val, A, dependencies); // Allowed use in case of USM
sparse::optimize_trsv(queue, uplo_val, opA, diag_val,
A); // OK Recommended use in case of
sycl::buffer
static_cast<void>(sparse::optimize_trsv(queue, uplo_val, opA, diag_val,
A)); // OK Recommended use in case of sycl::buffer
(explicitly say we aren't using the event)(will be used in examples)
sparse::optimize_trsv(queue, uplo_val, opA, diag_val, A,
{}); // Not recommended in case of
sycl::buffer, but supported;
sparse::optimize_trsv(queue, uplo_val, opA, diag_val, A,
dependencies); // Not recommended in case of
sycl::buffer, but supported;
sycl::event ev_opt = sparse::optimize_trsv(queue, uplo_val, opA,
diag_val, A); // Allowed use in case of sycl::buffer, but
most sycl::buffer oneMKL APIs do not have ability to pass in
dependencies
```

oneapi::mkl::sparse::optimize_gemm

Performs internal optimizations for `oneapi::mkl::sparse::gemm` by analyzing the provided matrix structure and operation parameters.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::optimize_gemm` routine analyzes matrix structure and performs optimizations. Optimized data is then stored in the matrix handle. We have two versions of `oneapi::mkl::sparse::optimize_gemm`; one analyzes the sparse matrix pattern only, and the other version could take information about the `layout` and dimensions of the dense matrices and may allow further optimizations based on the given input.

NOTE Refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

API

Syntax

Using USM and SYCL buffers:

```
namespace oneapi::mkl::sparse {
    sycl::event optimize_gemm (
        sycl::queue &queue,
        oneapi::mkl::transpose opA,
```

```

    oneapi::mkl::sparse::matrix_handle_t A,
    const std::vector<sycl::event> &dependencies = {});

sycl::event optimize_gemm (
    sycl::queue &queue,
    oneapi::mkl::layout layout_val,
    oneapi::mkl::transpose opA,
    oneapi::mkl::transpose opX,
    oneapi::mkl::sparse::matrix_handle_t A,
    const std::int64_t columns,
    const std::vector<sycl::event> &dependencies = {});
}

```

Include Files

- oneapi/mkl/spblas.hpp

Input Parameters

queue	Specifies the SYCL command queue that will be used for SYCL kernels execution.
layout_val	Specifies the storage scheme in memory for the dense matrices. Note that this layout applies to both X and Y dense matrices.
opA	Specifies operation $op()$ on the input sparse matrix.

oneapi::mkl::transpose::nontrans	Non-transpose, $op(A) = A$.
oneapi::mkl::transpose::trans	Transpose, $op(A) = A^T$.
oneapi::mkl::transpose::conjtrans	Conjugate transpose, $op(A) = A^H$.

opX	Specifies operation $op()$ on the input dense matrix.
-----	---

oneapi::mkl::transpose::nontrans	Non-transpose, $op(X) = X$.
oneapi::mkl::transpose::trans	Transpose, $op(X) = X^T$.
oneapi::mkl::transpose::conjtrans	Conjugate transpose, $op(X) = X^H$.

NOTE Currently, the only supported case for this operation is
oneapi::mkl::transpose::nontrans.

A	Handle to object containing the sparse matrix and other internal data. Created using one of the oneapi::mkl::sparse::set_<sparse_matrix_type>_data routines.
---	--

NOTE The supported cases for `<sparse_matrix_type>` are `csr` and `coo` on CPU and GPU devices for the `sparse::gemm()` routine. Currently, the `sparse::optimize_gemm()` routine would be mainly beneficial to the `coo` sparse matrix on GPU device.

columns	Number of columns of matrix <code>op(X)</code> and <code>Y</code> of the following <code>sparse::gemm()</code> routine.
dependencies	A vector of type <code>std::vector<sycl::event></code> containing the list of events that the <code>oneapi::mkl::sparse::optimize_gemm</code> routine depends on.
Return Values	
<code>sycl::event</code>	SYCL event that can be waited upon or added as a dependency for the completion of the <code>optimize_gemm</code> routine.

```
sycl::event ev_opt = sparse::optimize_gemm(queue, opA,
A); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_gemm(queue, opA, A,
{}); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_gemm(queue, opA, A,
dependencies); // Allowed use in case of USM
sparse::optimize_gemm(queue, opA,
A); // OK Recommended use in case of
sycl::buffer
static_cast<void>(sparse::optimize_gemm(queue, opA,
A)); // OK Recommended use in case of sycl::buffer
(explicitly say we aren't using the event)(will be used in sycl::buffer
examples)
sparse::optimize_gemm(queue, opA, A,
{}); // Not recommended in case of
sycl::buffer, but supported;
sparse::optimize_gemm(queue, opA, A,
dependencies); // Not recommended in case of
sycl::buffer, but supported;
sycl::event ev_opt = sparse::optimize_gemm(queue, opA,
A); // Allowed use in case of sycl::buffer, but most
sycl::buffer oneMKL APIs do not have ability to pass in dependencies

sycl::event ev_opt = sparse::optimize_gemm(queue, layout_val, opA, opX,
A, columns); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_gemm(queue, layout_val, opA, opX,
A, columns, {}); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_gemm(queue, layout_val, opA, opX,
A, columns, dependencies); // Allowed use in case of USM
sparse::optimize_gemm(queue, layout_val, opA, opX, A,
columns); // OK Recommended use in
case of sycl::buffer
static_cast<void>(sparse::optimize_gemm(queue, layout_val, opA, opX, A,
columns)); // OK Recommended use in case of
sycl::buffer (explicitly say we aren't using the event)(will be used in
sycl::buffer examples)
sparse::optimize_gemm(queue, layout_val, opA, opX, A, columns,
{}); // Not recommended in case of
sycl::buffer, but supported;
```

```
sparse::optimize_gemm(queue, layout_val, opA, opX, A, columns,
dependencies); // Not recommended in case of
sycl::buffer, but supported;
sycl::event ev_opt = sparse::optimize_gemm(queue, layout_val, opA, opX,
A, columns); // Allowed use in case of sycl::buffer, but
most sycl::buffer oneMKL APIs do not have ability to pass in
dependencies
```

oneapi::mkl::sparse::optimize_trsm

Performs internal optimizations for `oneapi::mkl::sparse::trsm` by analyzing the provided matrix structure and operation parameters.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::optimize_trsm` routine analyzes matrix structure and performs optimizations. Optimized data is then stored in the matrix handle. We have two versions of `oneapi::mkl::sparse::optimize_trsm`; one analyzes the sparse matrix pattern only, and the other version could take information about the `layout` and dimensions of the dense matrices and may allow further optimizations based on the given input.

NOTE Refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

API

Syntax

Using USM and SYCL buffers:

```
namespace oneapi::mkl::sparse {
    sycl::event optimize_trsm (
        sycl::queue &queue,
        oneapi::mkl::uplo uplo_val,
        oneapi::mkl::transpose opA,
        oneapi::mkl::diag diag_val,
        oneapi::mkl::sparse::matrix_handle_t A,
        const std::vector<sycl::event> &dependencies = {});

    sycl::event optimize_trsm (
        sycl::queue &queue,
        oneapi::mkl::layout layout_val,
        oneapi::mkl::uplo uplo_val,
        oneapi::mkl::transpose opA,
        oneapi::mkl::diag diag_val,
        oneapi::mkl::sparse::matrix_handle_t A,
        const std::int64_t columns,
        const std::vector<sycl::event> &dependencies = {});
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

<code>queue</code>	Specifies the SYCL command queue that will be used for SYCL kernels execution.
<code>layout_val</code>	Specifies the storage scheme in memory for the dense matrices. Note that this layout applies to both <code>X</code> and <code>Y</code> dense matrices.
<code>uplo_val</code>	Specifies which part of the matrix is to be processed.

<code>oneapi::mkl::uplo::lower</code>	The lower triangular matrix part is processed.
<code>oneapi::mkl::uplo::upper</code>	The upper triangular matrix part is processed.

`opA` Specifies operation `op()` on input matrix.

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $op(A) = A$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $op(A) = A^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $op(A) = A^H$.

NOTE Currently, the only supported case for operation is `oneapi::mkl::transpose::nontrans`.

`diag_val` Specifies whether the diagonal used for computations is unit or not.

<code>oneapi::mkl::diag::nonunit</code>	Diagonal elements might not be equal to one.
<code>oneapi::mkl::diag::unit</code>	Diagonal elements are equal to one.

`A` Handle to object containing sparse matrix and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` on CPU and GPU devices, and `coo` only on CPU device.

`columns` Number of columns of matrix `op(X)` and `Y` of following `sparse::trsm()` routine.

`dependencies` A vector of type `std::vector<sy::event>` containing the list of events that the `oneapi::mkl::sparse::optimize_trsm` routine depends on.

Return Values (USM Only)

sycl::event

SYCL event that can be waited upon or added as a dependency for the completion of the `optimize_trsm` routine.

```
sycl::event ev_opt = sparse::optimize_trsm(queue, uplo_val, opA,
diag_val, A); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_trsm(queue, uplo_val, opA,
diag_val, A, {}); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_trsm(queue, uplo_val, opA,
diag_val, A, dependencies); // Allowed use in case of USM
sparse::optimize_trsm(queue, uplo_val, opA, diag_val,
A); // OK Recommended use in case of
sycl::buffer
static_cast<void>(sparse::optimize_trsm(queue, uplo_val, opA, diag_val,
A)); // OK Recommended use in case of sycl::buffer
(explicitly say we aren't using the event)(will be used in sycl::buffer
examples)
sparse::optimize_trsm(queue, uplo_val, opA, diag_val, A,
{}); // Not recommended in case of
sycl::buffer, but supported;
sparse::optimize_trsm(queue, uplo_val, opA, diag_val, A,
dependencies); // Not recommended in case of
sycl::buffer, but supported;
sycl::event ev_opt = sparse::optimize_trsm(queue, uplo_val, opA,
diag_val, A); // Allowed use in case of sycl::buffer, but
most sycl::buffer oneMKL APIs do not have ability to pass in
dependencies

sycl::event ev_opt = sparse::optimize_trsm(queue, layout_val, uplo_val,
opA, diag_val, A, columns); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_trsm(queue, layout_val, uplo_val,
opA, diag_val, A, columns, {}); // Allowed use in case of USM
sycl::event ev_opt = sparse::optimize_trsm(queue, layout_val, uplo_val,
opA, diag_val, A, columns, dependencies); // Allowed use in case of USM
sparse::optimize_trsm(queue, layout_val, uplo_val, opA, diag_val, A,
columns); // OK Recommended use in
case of sycl::buffer
static_cast<void>(sparse::optimize_trsm(queue, layout_val, uplo_val,
opA, diag_val, A, columns)); // OK Recommended use in
case of sycl::buffer (explicitly say we aren't using the event)(will be
used in sycl::buffer examples)
sparse::optimize_trsm(queue, layout_val, uplo_val, opA, diag_val, A,
columns, {}); // Not recommended in case
of sycl::buffer, but supported;
sparse::optimize_trsm(queue, layout_val, uplo_val, opA, diag_val, A,
columns, dependencies); // Not recommended in case
of sycl::buffer, but supported;
sycl::event ev_opt = sparse::optimize_trsm(queue, layout_val, uplo_val,
opA, diag_val, A, columns); // Allowed use in case of
sycl::buffer, but most sycl::buffer oneMKL APIs do not have ability to
pass in dependencies
```

oneapi::mkl::sparse::gemv

Computes a sparse matrix-dense vector product.

- [Description](#)

- [API](#)
- [Examples](#)

Description

NOTE Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

The `oneapi::mkl::sparse::gemv` routine computes a sparse matrix-dense vector product defined as

$$y \leftarrow \alpha \cdot \text{op}(A) \cdot x + \beta \cdot y$$

where α and β are scalars, A is a general sparse matrix of dimensions `num_rows` rows and `num_cols` columns and $\text{op}()$ is a matrix modifier:

$$\text{op}(A) = \begin{cases} A, & \text{oneapi::mkl::transpose::nontrans} \\ A^T, & \text{oneapi::mkl::transpose::trans} \\ A^H, & \text{oneapi::mkl::transpose::conjtrans} \end{cases}$$

The dense vectors x and y are appropriately sized based on matrix product dimensions of $\text{op}(A)$.

API

Syntax

Using SYCL buffers:

```
namespace oneapi::mkl::sparse {
    void gemv (
        sycl::queue &queue,
        oneapi::mkl::transpose opA,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        sycl::buffer<DATA_TYPE, 1> &x,
        const DATA_TYPE beta,
        sycl::buffer<DATA_TYPE, 1> &y)
}
```

Using USM pointers:

```
namespace oneapi::mkl::sparse {
    sycl::event gemv (
        sycl::queue &queue,
        oneapi::mkl::transpose opA,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        const DATA_TYPE *x,
        DATA_TYPE beta,
```

```
DATA_TYPE *y,
const std::vector<sycl::event> &dependencies= {}))
}
```

Include Files

- oneapi/mkl/spblas.hpp

Input Parameters

queue Specifies the SYCL command queue which will be used for SYCL kernels execution.

opA Specifies operation `op()` on input matrix.

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $\text{op}(A) = A$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $\text{op}(A) = A^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $\text{op}(A) = A^H$.

alpha Specifies the scalar, α .

A Handle to object containing sparse matrix and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` and `coo` on both CPU and GPU devices.

x SYCL buffer or device-accessible USM pointer of size at least equal to the number of columns of input matrix if `opA = oneapi::mkl::transpose::nontrans` and at least the number of rows of input matrix otherwise.

beta Specifies the scalar, β .

y SYCL buffer or device-accessible USM pointer of size at least equal to the number of rows of the input matrix if `opA = oneapi::mkl::transpose::nontrans` and at least the number of columns of the input matrix otherwise.

dependencies A vector of type `std::vector<sycl::event>` containing the list of events that the `oneapi::mkl::sparse::gemv` routine depends on.

Output Parameters

y Overwritten by the updated vector y .

Return Values (USM Only)

sycl::event SYCL event which can be waited upon or added as a dependency for the completion of the `gemv` routine.

Examples

An example of how to use `oneapi::mkl::sparse::gemv` with SYCL buffers or USM pointers can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/coo_gemv.cpp
```

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_gemv_usm.cpp
```

oneapi::mkl::sparse::gemvdot

Computes a sparse matrix-dense vector product with dot product.

- [Description](#)
- [API](#)
- [Examples](#)

Description

NOTE Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

The `oneapi::mkl::sparse::gemvdot` routine computes a sparse matrix-dense vector product and dot product defined as

$$y \leftarrow \alpha \cdot \text{op}(A) \cdot x + \beta \cdot y$$

$$d \leftarrow x \cdot y$$

where A is a general sparse matrix, α , β , and d are scalars, x and y are dense vectors and $\text{op}()$ is a matrix modifier:

$$\text{op}(A) = \begin{cases} A, & \text{oneapi::mkl::transpose::nontrans} \\ A^T, & \text{oneapi::mkl::transpose::trans} \\ A^H, & \text{oneapi::mkl::transpose::conjtrans} \end{cases}$$

API

Syntax

Using SYCL buffers:

```
namespace oneapi::mkl::sparse {
    void gemvdot (
        sycl::queue &queue,
        oneapi::mkl::transpose opA,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        sycl::buffer<DATA_TYPE, 1> &x,
        const DATA_TYPE beta,
        sycl::buffer<DATA_TYPE, 1> &y,
        sycl::buffer<DATA_TYPE, 1> &d)
}
```

Using USM pointers:

```
namespace oneapi::mkl::sparse {
    sycl::event gemvdot (
        sycl::queue &queue,
        oneapi::mkl::transpose opA,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        const DATA_TYPE *x,
        const DATA_TYPE beta,
        DATA_TYPE *y,
        DATA_TYPE *d,
        const std::vector<sycl::event> &dependencies={})
}
```

Include Files

- oneapi/mkl/spblas.hpp

Input Parameters

queue	Specifies the SYCL command queue which will be used for SYCL kernels execution.
opA	Specifies operation <code>op()</code> on input matrix.

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $\text{op}(A) = A$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $\text{op}(A) = A^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $\text{op}(A) = A^H$.

NOTE Currently, the only supported case for operation is `oneapi::mkl::transpose::nontrans`.

alpha	Specifies the scalar, α .
A	Handle to object containing sparse matrix and other internal data. Created using one of the <code>oneapi::mkl::sparse::set_<sparse_matrix_type>_data</code> routines.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` on CPU and GPU devices, and `coo` only on CPU device.

<code>x</code>	SYCL buffer or device-accessible USM pointer of size at least equal to the number of columns of input matrix if <code>opA == oneapi::mkl::transpose::nontrans</code> and at least the number of rows of input matrix otherwise.
<code>beta</code>	Specifies the scalar, β .
<code>y</code>	SYCL buffer or device-accessible USM pointer of size at least equal to the number of rows of the input matrix if <code>opA == oneapi::mkl::transpose::nontrans</code> and at least the number of columns of the input matrix otherwise.
<code>dependencies</code>	A vector of type <code>std::vector<sy::event></code> containing the list of events that the <code>oneapi::mkl::sparse::gemvdot</code> routine depends on.
Output Parameters	
<code>y</code>	Overwritten by the updated vector $\mathbf{y}$.
<code>d</code>	Overwritten by the dot product of $\mathbf{x}$ and $\mathbf{y}$.
Return Values	
<code>sy::event</code>	SYCL event which can be waited upon or added as a dependency for the completion of the <code>gemvdot</code> routine.

Examples

An example of how to use `oneapi::mkl::sparse::gemvdot` with SYCL buffers can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_gemvdot.cpp
```

oneapi::mkl::sparse::symv

Computes a sparse matrix-dense vector product for a symmetric matrix built from the lower or upper triangular of the input matrix.

- [Description](#)
- [API](#)
- [Examples](#)

Description

NOTE Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

The `oneapi::mkl::sparse::symv` routine computes a sparse symmetric matrix-dense vector product for a real symmetric matrix built from the lower or upper triangular portion of the input matrix A defined as

$$y \leftarrow \alpha \cdot \text{sym}(A) \cdot x + \beta \cdot y$$

where α and β are scalars, A is a real-valued square sparse matrix of dimension m rows and columns, $\text{sym}()$ is a matrix modifier which symmetrizes the matrix according to the `oneapi::mkl::uplo` value and x and y are dense vectors.

For a given matrix decomposition into lower, diagonal and upper parts $A = L + D + U$, the `symv` routine with `oneapi::mkl::uplo::lower` selected will perform the matrix product with

$\text{sym}(A) = L + D + L^T$ and for `oneapi::mkl::uplo::upper` will perform the matrix product using $\text{sym}(A) = U^T + D + U$.

API

Syntax

Using SYCL buffers:

```
namespace oneapi::mkl::sparse {
    void symv (
        sycl::queue &queue,
        oneapi::mkl::uplo uplo_val,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        sycl::buffer<DATA_TYPE, 1> &x,
        const DATA_TYPE beta,
        sycl::buffer<DATA_TYPE, 1> &y)
}
```

Using USM pointers:

```
namespace oneapi::mkl::sparse {
    sycl::event symv (
        sycl::queue &queue,
        oneapi::mkl::uplo uplo_val,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        const DATA_TYPE *x,
        const DATA_TYPE beta,
        DATA_TYPE *y,
        const std::vector<sycl::event> &dependencies = {})
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

<code>queue</code>	Specifies the SYCL command queue which will be used for SYCL kernels execution.
<code>uplo_val</code>	Specifies which part is to be processed.

<code>oneapi::mkl::uplo::lower</code>	The lower part is used for symmetric product.
<code>oneapi::mkl::uplo::upper</code>	The upper part is used for symmetric product.

alpha

Specifies the scalar, α .

A

Handle to object containing sparse matrix and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE Currently, the only supported case for `<sparse_matrix_type>` is `csr`.

x

SYCL buffer or device-accessible USM pointer of size at least equal to the number of columns, m , of input matrix.

beta

Specifies the scalar, β .

y

SYCL buffer or device-accessible USM pointer of size at least equal to the number of rows, m , of the input matrix.

dependencies

A vector of type `std::vector<sycl::event>` containing the list of events that the `oneapi::mkl::sparse::symv` routine depends on.

Output Parameters

y

Overwritten by the updated vector y .

Return Values (USM Only)

`sycl::event`

SYCL event which can be waited upon or added as a dependency for the completion of the `symv` routine.

Examples

An example of how to use `oneapi::mkl::sparse::symv` with SYCL buffers can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_symv.cpp
```

oneapi::mkl::sparse::trmv

Computes a sparse matrix-dense vector product over upper or lower triangular parts of the matrix.

- [Description](#)
- [API](#)
- [Examples](#)

Description

NOTE Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

The `oneapi::mkl::sparse::trmv` routine computes a sparse matrix-dense vector product over a triangular part defined as

$$y \leftarrow \alpha \cdot \text{op}(A) \cdot x + \beta \cdot y$$

where α and β are scalars, A is a sparse triangular matrix of size m rows by m columns and x and y are dense vectors of size m . The operation $\text{op}()$ is a matrix modifier:

$$\text{op}(A) = \begin{cases} A, & \text{oneapi::mkl::transpose::nontrans} \\ A^T, & \text{oneapi::mkl::transpose::trans} \\ A^H, & \text{oneapi::mkl::transpose::conjtrans} \end{cases}$$

For a given matrix decomposition into lower, diagonal and upper parts $A = L + D + U$, the triangular matrix vector product with one of `oneapi::mkl::uplo::lower` or `oneapi::mkl::uplo::upper` selected will perform the appropriate matrix product using respectively $\text{op}(L + D)$ or $\text{op}(D + U)$ for `oneapi::mkl::diag::nonunit` or if using `oneapi::mkl::diag::unit`, will perform the appropriate matrix product for $\text{op}(L + I)$ or $\text{op}(I + U)$ where I is the identity matrix.

API

Syntax

Using SYCL buffers:

```
namespace oneapi::mkl::sparse {
    void trmv (
        sycl::queue &queue,
        oneapi::mkl::uplo uplo_val,
        oneapi::mkl::transpose opA,
        oneapi::mkl::diag diag_val,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        sycl::buffer<DATA_TYPE, 1> &x,
        const DATA_TYPE beta,
        sycl::buffer<DATA_TYPE, 1> &y)
}
```

Using USM pointers:

```
namespace oneapi::mkl::sparse {
    sycl::event trmv(
        sycl::queue &queue,
        oneapi::mkl::uplo uplo_val,
```

```

oneapi::mkl::transpose opA,
oneapi::mkl::diag
diag_val,
const DATA_TYPE alpha,
oneapi::mkl::sparse::matrix_handle_t A,
const DATA_TYPE *x,
const DATA_TYPE beta,
DATA_TYPE *y,
const std::vector<sycl::event> &dependencies = {})
}

```

Include Files

- oneapi/mkl/spblas.hpp

Input Parameters

queue Specifies the SYCL command queue which will be used for SYCL kernels execution.

uplo_val Specifies which part of the matrix is to be processed.

<code>oneapi::mkl::uplo::lower</code>	The lower triangular matrix part is processed.
<code>oneapi::mkl::uplo::upper</code>	The upper triangular matrix part is processed.

opA Specifies operation `op()` on input matrix.

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $\text{op}(A) = A$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $\text{op}(A) = A^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $\text{op}(A) = A^H$.

NOTE Currently, the only supported case for operation is `oneapi::mkl::transpose::nontrans`.

diag_val Specifies if the diagonal used for computations is unit or not.

<code>oneapi::mkl::diag::nonunit</code>	Diagonal elements might not be equal to one.
<code>oneapi::mkl::diag::unit</code>	Diagonal elements are equal to one.

alpha Specifies the scalar, α .

A Handle to object containing sparse matrix and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` on CPU and GPU devices, and `coo` only on CPU device.

<code>x</code>	SYCL buffer or device-accessible USM pointer of size at least equal to the number of columns of input matrix if <code>opA = oneapi::mkl::transpose::nontrans</code> and at least the number of rows of input matrix otherwise.
<code>beta</code>	Specifies the scalar, β .
<code>y</code>	SYCL buffer or device-accessible USM pointer of size at least equal to the number of rows of the input matrix if <code>opA = oneapi::mkl::transpose::nontrans</code> and at least the number of columns of the input matrix otherwise.
<code>dependencies</code>	A vector of type <code>std::vector<sycl::event></code> containing the list of events that the <code>oneapi::mkl::sparse::trmv</code> routine depends on.
Output Parameters	
<code>y</code>	Overwritten by the updated vector, y .
Return Values (USM Only)	
<code>sycl::event</code>	SYCL event which can be waited upon or added as a dependency for the completion of the <code>trmv</code> routine.

Examples

An example of how to use `oneapi::mkl::sparse::trmv` with SYCL buffers can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_trmv.cpp
```

oneapi::mkl::sparse::trsv

Solves a system of linear equations for a triangular sparse matrix.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `oneapi::mkl::sparse::trsv` routine solves the sparse triangular system

$$\text{op}(A) \cdot y = \alpha \cdot x$$

where A is a sparse triangular matrix of size m rows by m columns and $\text{op}()$ is a matrix modifier:

$$\text{op}(A) = \begin{cases} A, & \text{oneapi::mkl::transpose::nontrans} \\ A^T, & \text{oneapi::mkl::transpose::trans} \\ A^H, & \text{oneapi::mkl::transpose::conjtrans} \end{cases}$$

The dense vectors x and y must be at least of length m . The vector x is input right hand side data and y is the resulting output vector.

For a given matrix decomposition into lower, diagonal, and upper parts $A = L + D + U$, the triangular solve with one of `oneapi::mkl::uplo::lower` or `oneapi::mkl::uplo::upper` selected will perform the appropriate forward or backward substitution using respectively $\text{op}(L + D)$ or $\text{op}(D + U)$ for `oneapi::mkl::diag::nonunit` or, for `oneapi::mkl::diag::unit`, will perform the appropriate forward or backward substitution for $\text{op}(L + I)$ or $\text{op}(I + U)$ where I is the identity matrix.

NOTE

- Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types, and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.
- While it can make sense to pass in the same array for both RHS (x) and solution (y) in order to update the RHS in-place, we currently do not support in-place `sparse::trsv()` operation.

API

Syntax

Using SYCL buffers:

```
namespace oneapi::mkl::sparse {
    void trsv (
        sycl::queue &queue,
        oneapi::mkl::uplo uplo_val,
        oneapi::mkl::transpose opA,
        oneapi::mkl::diag diag_val,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        sycl::buffer<DATA_TYPE, 1> &x,
        sycl::buffer<DATA_TYPE, 1> &y);

    [[deprecated("Use oneapi::mkl::sparse::trsv(queue, uplo_val, opA, diag_val, /* alpha */
1.0, ...) instead.")]]
    void trsv (
        sycl::queue &queue,
        oneapi::mkl::uplo uplo_val,
        oneapi::mkl::transpose opA,
        oneapi::mkl::diag diag_val,
        oneapi::mkl::sparse::matrix_handle_t A,
        sycl::buffer<DATA_TYPE, 1> &x,
        sycl::buffer<DATA_TYPE, 1> &y);
}
```

Using USM pointers:

```

namespace oneapi::mkl::sparse {
    sycl::event trsv(
        sycl::queue &queue,
        oneapi::mkl::uplo uplo_val,
        oneapi::mkl::transpose opA,
        oneapi::mkl::diag diag_val,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        const DATA_TYPE *x,
        DATA_TYPE *y,
        const std::vector<sycl::event> &dependencies = {});

    [[deprecated("Use oneapi::mkl::sparse::trsv(queue, uplo_val, opA, diag_val, /* alpha */
1.0, ...) instead.")]]
    sycl::event trsv(
        sycl::queue &queue,
        oneapi::mkl::uplo uplo_val,
        oneapi::mkl::transpose opA,
        oneapi::mkl::diag diag_val,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        const DATA_TYPE *x,
        DATA_TYPE *y,
        const std::vector<sycl::event> &dependencies = {});
}

```

Include Files

- oneapi/mkl/spblas.hpp

Input Parameters

queue Specifies the SYCL command queue that will be used for SYCL kernels execution.

uplo_val Specifies which part of the matrix is to be processed.

oneapi::mkl::uplo::lower	The lower triangular matrix part is processed.
oneapi::mkl::uplo::upper	The upper triangular matrix part is processed.

opA Specifies operation `op()` on input matrix.

oneapi::mkl::transpose::nontrans	Non-transpose, $\text{op}(A) = A$.
oneapi::mkl::transpose::trans	Transpose, $\text{op}(A) = A^T$.
oneapi::mkl::transpose::conjtrans	Conjugate transpose, $\text{op}(A) = A^H$.

NOTE Currently, the only supported case for operation is `oneapi::mkl::transpose::nontrans`.

`diag_val`

Specifies whether the diagonal used for computations is unit or based on provided matrix data.

<code>oneapi::mkl::diag::nonunit</code>	Diagonal elements are used as provided in the sparse matrix.
<code>oneapi::mkl::diag::unit</code>	The value of one is substituted for the diagonal elements in the triangular solve algorithm.

NOTE If `oneapi::mkl::diag::nonunit` is selected, all diagonal values must be present in the sparse matrix sparsity profile and must not be zero valued. This is not necessary for the `oneapi::mkl::diag::unit` case. An exception of type `onemkl::invalid_value()` will be thrown in case this is violated.

`alpha`

Specifies the scalar, α .

`A`

Handle to object containing sparse matrix (A) and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` on CPU and GPU devices, and `coo` only on CPU device.

`x`

SYCL buffer or device-accessible USM pointer of size at least equal to the number of columns of input matrix if `opA = oneapi::mkl::transpose::nontrans` and at least the number of rows of input matrix otherwise. It is the input vector x

`dependencies`

A vector of type `std::vector<sycl::event>` containing the list of events that the `oneapi::mkl::sparse::trsv` routine depends on.

Output Parameters

`y`

SYCL buffer or device-accessible USM pointer of size at least equal to the number of rows of the input matrix if `opA = oneapi::mkl::transpose::nontrans` and at least the number of columns of the input matrix otherwise. The solution of the triangular solve is filled into this array.

Return Values (USM Only)

`sycl::event`

SYCL event that can be waited upon or added as a dependency for the completion of the `trsv` routine.

Examples

An example of how to use `oneapi::mkl::sparse::trsv` with SYCL buffers or USM can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_trsv.cpp
```

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_trsv_usm.cpp
```

oneapi::mkl::sparse::gemm

Computes a sparse matrix-dense matrix product.

- [Description](#)
- [API](#)
- [Examples](#)

Description

NOTE Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

The `oneapi::mkl::sparse::gemm` routine computes a sparse matrix-dense matrix product defined as

$$Y \leftarrow \alpha \cdot \text{op}(A) \cdot \text{op}(X) + \beta \cdot Y$$

where: α and β are scalars, A is a sparse matrix of size `num_rows` rows by `num_cols` columns, and `op()` is a matrix modifier for A and X using the following description:

$$\text{op}(A) = \begin{cases} A, & \text{oneapi::mkl::transpose::nontrans} \\ A^T, & \text{oneapi::mkl::transpose::trans} \\ A^H, & \text{oneapi::mkl::transpose::conjtrans.} \end{cases}$$

The dense matrix objects X and Y are stored with row-major or column-major layout and have an appropriate number of rows for the matrix product and `columns` number of columns.

API

Syntax

Using SYCL buffers:

```
namespace oneapi::mkl::sparse {
    void gemm(sycl::queue &queue,
              oneapi::mkl::layout layout_val,
              oneapi::mkl::transpose opA,
              oneapi::mkl::transpose opX,
              const DATA_TYPE alpha,
              matrix_handle_t A,
              sycl::buffer<DATA_TYPE, 1> &X,
              const std::int64_t columns,
              const std::int64_t ldx,
              const DATA_TYPE beta,
              sycl::buffer<DATA_TYPE, 1> &Y,
              const std::int64_t ldy);
}
```

Using USM pointers:

```

namespace oneapi::mkl::sparse {
    sycl::event gemm(
        sycl::queue &queue,
        oneapi::mkl::layout layout_val,
        oneapi::mkl::transpose opA,
        oneapi::mkl::transpose opX,
        const DATA_TYPE alpha,
        matrix_handle_t A,
        const DATA_TYPE *X,
        const std::int64_t columns,
        const std::int64_t ldx,
        const DATA_TYPE beta,
        DATA_TYPE *Y,
        const std::int64_t ldy,
        const std::vector<sycl::event> &dependencies = {});
}

```

Include Files

- oneapi/mkl/spblas.hpp

Input Parameters

queue	Specifies the SYCL command queue which will be used for SYCL kernels execution.						
layout_val	Specifies the storage scheme in memory for the dense matrices. Note that this layout applies to both X and Y dense matrices.						
opA	Specifies operation $op()$ on input matrix A .						
	<table border="1"> <tr> <td>oneapi::mkl::transpose::nontrans</td><td>Non-transpose, $op(A) = A$.</td></tr> <tr> <td>oneapi::mkl::transpose::trans</td><td>Transpose, $op(A) = A^T$.</td></tr> <tr> <td>oneapi::mkl::transpose::conjtrans</td><td>Conjugate transpose, $op(A) = A^H$.</td></tr> </table>	oneapi::mkl::transpose::nontrans	Non-transpose, $op(A) = A$.	oneapi::mkl::transpose::trans	Transpose, $op(A) = A^T$.	oneapi::mkl::transpose::conjtrans	Conjugate transpose, $op(A) = A^H$.
oneapi::mkl::transpose::nontrans	Non-transpose, $op(A) = A$.						
oneapi::mkl::transpose::trans	Transpose, $op(A) = A^T$.						
oneapi::mkl::transpose::conjtrans	Conjugate transpose, $op(A) = A^H$.						
opX	Specifies operation $op()$ on input matrix X .						

oneapi::mkl::transpose::nontrans	Non-transpose, $op(X) = X$.
oneapi::mkl::transpose::trans	Transpose, $op(X) = X^T$.
oneapi::mkl::transpose::conjtrans	Conjugate transpose, $op(X) = X^H$.

NOTE Currently, the only supported case for operation is `oneapi::mkl::transpose::nontrans`.

alpha	Specifies the scalar, α .
A	Handle to object containing sparse matrix and other internal data. Created using one of the <code>oneapi::mkl::sparse::set_<sparse_matrix_type>_data</code> routines.
X	SYCL buffer or device-accessible USM pointer of size at least <code>rows*cols</code> , where (with the assumption of <code>opX == oneapi::mkl::transpose::nontrans</code>).

	<code>layout=oneapi::mkl:::layout::col-major</code>	<code>layout=oneapi::mkl:::layout::row-major</code>
rows (number of rows in X)	ldx	if $\text{op}(A) = A$, number of columns in A if $\text{op}(A) = A^T$, number of rows in A
cols (number of columns in X)	columns	ldx

columns	Number of columns of matrix Y.
ldx	Specifies the leading dimension of matrix X. Must be positive, and at least <code>columns</code> if <code>layout_val=oneapi::mkl::layout::row-major</code> or at least number of columns in A if <code>layout_val=oneapi::mkl::layout::col-major</code> .
beta	Specifies the scalar, β .
Y	SYCL buffer or device-accessible USM pointer of size at least <code>rows*cols</code> , where:

	<code>layout=oneapi::mkl:::layout::col-major</code>	<code>layout=oneapi::mkl:::layout::row-major</code>
rows (number of rows in Y)	ldy	if $\text{op}(A) = A$, number of rows in A if $\text{op}(A) = A^T$, number of columns in A
cols (number of columns in Y)	columns	ldy

ldy	Specifies the leading dimension of matrix Y. Must be positive, and at least <code>columns</code> if <code>layout_val=oneapi::mkl::layout::row-major</code> or at least number of rows in A if <code>layout_val=oneapi::mkl::layout::col-major</code> .
dependencies	A vector of type <code>std::vector<syicl::event></code> containing the list of events that the <code>oneapi::mkl::sparse::gemm</code> routine depends on.
Output Parameters	
Y	Overwritten by the updated matrix Y.

Return Values (USM Only)

`sycl::event` SYCL event which can be waited upon or added as a dependency for the completion of the `gemm` routine.

Examples

An example of how to use `oneapi::mkl::sparse::gemm` with SYCL buffers or USM can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_gemm_row_major.cpp
```

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_gemm_row_major_usm.cpp
```

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_gemm_col_major.cpp
```

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_gemm_col_major_usm.cpp
```

oneapi::mkl::sparse::trsm

Solves a system of linear equations with multiple right-hand sides (RHS) for a triangular sparse matrix.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `oneapi::mkl::sparse::trsm` routine solves the sparse triangular system

$$\text{op}(A) \cdot Y = \alpha \cdot \text{op}(X)$$

where A is a sparse triangular matrix of size m rows by m columns and $\text{op}()$ is a matrix modifier:

$$\text{op}(A) = \begin{cases} A, & \text{oneapi::mkl::transpose::nontrans} \\ A^T, & \text{oneapi::mkl::transpose::trans} \\ A^H, & \text{oneapi::mkl::transpose::conjtrans} \end{cases}$$

The dense matrices X and Y are stored with `row-major` or `column-major` layout and have an appropriate number of rows and `columns` number of columns for computing `sparse::trsm()` operation, that is, $\text{op}(X)$ and Y are of dimensions m rows by `columns` columns. The matrix $\text{op}(X)$ is input right hand side data and Y is the resulting output matrix.

For a given matrix decomposition into lower, diagonal, and upper parts $A = L + D + U$, the triangular solve with one of `oneapi::mkl::uplo::lower` or `oneapi::mkl::uplo::upper` selected will perform the appropriate forward or backward substitution using respectively $\text{op}(L + D)$ or $\text{op}(D + U)$ for `oneapi::mkl::diag::nonunit` or, for `oneapi::mkl::diag::unit`, will perform the appropriate forward or backward substitution for $\text{op}(L + I)$ or $\text{op}(I + U)$ where I is the identity matrix.

NOTE

- Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types, and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.
 - While it can make sense to pass in the same array for both RHS (X) and solution (Y) in order to update the RHS in-place, we currently do not support in-place `sparse::trsm()` operation.
-

API**Syntax****Using SYCL buffers:**

```
namespace oneapi::mkl::sparse {
    void trsm (
        sycl::queue &queue,
        oneapi::mkl::layout layout_val,
        oneapi::mkl::transpose opA,
        oneapi::mkl::transpose opX,
        oneapi::mkl::uplo uplo_val,
        oneapi::mkl::diag diag_val,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        sycl::buffer<DATA_TYPE, 1> &X,
        const std::int64_t columns,
        const std::int64_t ldx,
        sycl::buffer<DATA_TYPE, 1> &Y,
        const std::int64_t ldy);
}
```

Using USM pointers:

```
namespace oneapi::mkl::sparse {
    sycl::event trsm(
        sycl::queue &queue,
        oneapi::mkl::layout layout_val,
        oneapi::mkl::transpose opA,
        oneapi::mkl::transpose opX,
        oneapi::mkl::uplo uplo_val,
        oneapi::mkl::diag diag_val,
        const DATA_TYPE alpha,
        oneapi::mkl::sparse::matrix_handle_t A,
        const DATA_TYPE *X,
        const std::int64_t columns,
        const std::int64_t ldx,
        DATA_TYPE *Y,
        const std::int64_t ldy,
        const std::vector<sycl::event> &dependencies = {});
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

`queue` Specifies the SYCL command queue that will be used for SYCL kernels execution.

layout_val Specifies the storage scheme in memory for the dense matrices. Note that this layout applies to both X and Y dense matrices.

opA Specifies operation $\text{op}()$ on the sparse input matrix, A .

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $\text{op}(A) = A$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $\text{op}(A) = A^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $\text{op}(A) = A^H$.

NOTE Currently, the only supported case for operation is `oneapi::mkl::transpose::nontrans`.

opX Specifies operation $\text{op}()$ on the dense input matrix, X .

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $\text{op}(X) = X$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $\text{op}(X) = X^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $\text{op}(X) = X^H$.

NOTE Currently, the only supported case for operation is `oneapi::mkl::transpose::nontrans`.

uplo_val Specifies which part of the matrix is to be processed.

<code>oneapi::mkl::uplo::lower</code>	The lower triangular matrix part is processed.
<code>oneapi::mkl::uplo::upper</code>	The upper triangular matrix part is processed.

diag_val Specifies whether the diagonal used for computations is unit or based on provided matrix data.

<code>oneapi::mkl::diag::nonunit</code>	Diagonal elements are used as provided in the sparse matrix.
<code>oneapi::mkl::diag::unit</code>	The value of one is substituted for the diagonal elements in the triangular solve algorithm.

NOTE If `oneapi::mkl::diag::nonunit` is selected, all diagonal values must be present in the sparse matrix sparsity profile and must not be zero valued. This is not necessary for the `oneapi::mkl::diag::unit` case. An exception of type `onemkl::invalid_value()` will be thrown in case this is violated.

alpha

Specifies the scalar, α .

A

Handle to object containing sparse matrix (A) and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` on CPU and GPU devices, and `coo` only on CPU device.

X

SYCL buffer or device-accessible USM pointer for the right hand side dense matrix. $\text{op}(X)$ is of dimension m by `columns` with a leading dimension stride of `ldx` based on the provided `layout` of `X`.

For example, if $\text{op}(X) = X$, i.e `opX == oneapi::mkl::transpose::nontrans`, we would have

	<code>layout=oneapi::mkl::layout::col-major</code>	<code>layout=oneapi::mkl::layout::row-major</code>
nrows of $\text{op}(X)$	<code>ldx</code>	<code>m</code>
ncols of $\text{op}(X)$	<code>columns</code>	<code>ldx</code>

columns

Number of columns of $\text{op}(X)$ and Y matrices.

ldx

Specifies the leading dimension of matrix `X`. Must be positive, and larger than or equal to the actual usable row (for `layout_val=oneapi::mkl::layout::row_major`) or column (for `layout_val=oneapi::mkl::layout::col_major`) dimension of `X`.

ldy

Specifies the leading dimension of matrix `Y`. Must be positive, and at least `columns` if `layout_val=oneapi::mkl::layout::row-major` or at least number of rows in $\text{op}(A)$ if `layout_val=oneapi::mkl::layout::col-major`.

dependencies

A vector of type `std::vector<sycl::event>` containing the list of events that the `oneapi::mkl::sparse::trsm` routine depends on.

Output Parameters

Y

SYCL buffer or device-accessible USM pointer for the dense matrix solution of the triangular solve. Y is of dimension m by `columns` with a leading dimension stride of `ldy` based on the provided `layout` of `Y`.

	<code>layout=oneapi::mkl: :layout::col-major</code>	<code>layout=oneapi::mkl: :layout::row-major</code>
nrows of Y	<code>ldy</code>	<code>m</code> (number of rows in <code>op(A)</code> and A)
ncols of Y	<code>columns</code>	<code>ldy</code>

Return Values (USM Only)

`sycl::event` SYCL event that can be waited upon or added as a dependency for the completion of the `trsm` routine.

Examples

An example of how to use `oneapi::mkl::sparse::trsm` with SYCL buffers or USM can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_trsm.cpp
```

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_trsm_usm.cpp
```

oneapi::mkl::sparse::omatadd

Computes general sparse matrix-sparse matrix addition with sparse matrix output.

- [Description](#)
- [Stages](#)
- [API](#)
- [Examples](#)

Description

NOTE Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

The `oneapi::mkl::sparse::omatadd` set of routines perform general sparse matrix-sparse matrix addition, defined as

$$C = \alpha \cdot \text{op}(A) + \beta \cdot \text{op}(B)$$

where A , B , and C are sparse matrices with mathematically consistent sizes and `op()` is a matrix modifier:

$$\text{op}(A) = \begin{cases} A, & \text{oneapi::mkl::transpose::nontrans} \\ A^T, & \text{oneapi::mkl::transpose::trans} \\ A^H, & \text{oneapi::mkl::transpose::conjtrans.} \end{cases}$$

The sparse matrices are stored in `sparse::matrix_handle_t` objects.

The output matrix C is not guaranteed to be sorted on exit from `sparse::omatadd()`, but a helper function `sparse::sort_matrix()` is provided if that is a necessary property for subsequent usage.

The input matrices, A and B , need not be sorted for use with these APIs. However, if users guarantee that both the input matrices are sorted either by calling `sparse::sort_matrix()` on the matrices beforehand, or by using the `sparse::set_matrix_property()` API to set the `sparse::property::sorted` property on the matrices, then that may significantly improve the performance of the `omatadd` APIs.

As the sparsity pattern of C and the size of its data arrays is generally not known beforehand, the `omatadd` routine is broken into several stages with different API names to enable querying the size of the temporary workspace and size of the resulting C data arrays, allocating them, and passing them back into the routine to be filled. This enables users to control and own all the C matrix data allocations. Unlike the `sparse::matmat()` API, however, the `omatadd` routines currently do not support addition involving only the sparsity patterns without any floating point values. The `sparse::omatadd()` set of APIs is broken into four APIs involving two lightweight and two computationally expensive stages given below:

Stage	Description
<code>omatadd_buffer_size</code>	Return size of temporary workspace.
<code>omatadd_analyze</code>	Count the number of non-zero values (<code>nnzC</code>) in the output sparse matrix.
<code>omatadd_get_nnz</code>	Return the calculated <code>nnzC</code> count of the output sparse matrix.
<code>omatadd</code>	Perform union of sparsity pattern and floating point accumulations into user-provided arrays of the output sparse matrix.

Stages

- Before `omatadd` stages
 - Use the `sparse::set_<xyz>_data` API with dummy arguments for row, column, and data arrays to set the sparse matrix format and the output 0-/1-based indexing. The number of rows and columns of the matrix may either be set to zero, or be mathematically consistent with the input matrix sizes at this stage.
 - Create the `omatadd_descr_t` object using the `init_omatadd_descr` API and decide on an algorithm to use through the enum, `omatadd_alg`. Do not change this enum between calls to the different `omatadd` APIs with a given set of input arguments and descriptor.
- `omatadd_buffer_size` stage
 - This is a non-blocking host-side API that does not access the input matrix arrays.
 - Use the `omatadd_buffer_size` API to get the temporary workspace size.
 - Allocate the temporary workspace to be used in subsequent stages.
- `omatadd_analyze` stage
 - This is a non-blocking asynchronous API that accesses and analyzes the sparsity patterns of the input matrices.
 - Use the `omatadd_analyze` API with the temporary workspace allocated in the previous stage.

- The temporary workspace array is internally stored in the `omatadd_descr_t` object. Do not modify or free the workspace for the duration of its use for sparse matrix addition or for the lifetime of the `omatadd_descr_t` object.
4. `omatadd_get_nnz` stage
 - Use this blocking API to get the number of non-zeros in the C matrix.
 - Allocate the row, column, and data arrays of the C matrix.
 - Call the `sparse::set_<xyz>_data` API again, this time with the valid, newly allocated arrays of C . At this point, the output 0-/1-based indexing must not be changed, and the number of rows and columns of the matrix must be mathematically consistent with the input matrix sizes for the operation.
 5. `omatadd` stage
 - Call the non-blocking, asynchronous `omatadd` API to perform the union of the sparsity pattern and floating point accumulations to fill in the user-provided output C matrix arrays.
 6. After `omatadd` stages
 - Release the `omatadd_descr_t` object using the `release_omatadd_descr` API. Reusing the descriptor for another addition operation is currently undefined behavior, but may be enabled in a future oneMKL release.
 - Release the temporary workspace array, or if the `omatadd_descr_t` object has been released, then reuse the workspace for any other purpose.
 - Release or use C matrix handle for subsequent operations.
 - If sorted output is needed for subsequent calls to other oneMKL APIs, then call the `sparse::sort_matrix()` API for sorting the output matrix arrays.

An example of this workflow for sparse matrix addition is demonstrated in the oneMKL SYCL examples listed later below.

API

Syntax

```
enum omatadd_alg
```

The `omatadd_algenum` provides users a choice of using specific algorithms implemented in oneMKL. Currently, only one algorithm is available to users. This `enum` is defined as:

```
namespace oneapi::mkl::sparse {
    enum class omatadd_alg : std::int32_t {
        default_alg = 0
    };
}
```

`omatadd_descr_t` object

`omatadd_descr_t` is an operation-specific opaque descriptor object used to store the internal state between calls to the `omatadd` set of APIs. Once a given descriptor object is used in any of the APIs, it must not be changed or `free`'d until all calls to `omatadd` APIs are completed. A pointer to the user-provided temporary

workspace is stored in this descriptor object through one of the `omatadd` set of APIs, viz., `omatadd_analyze`, described below. There are initialization and release functions associated with this descriptor object.

```
namespace oneapi::mkl::sparse {
    struct omatadd_descr;          /* Forward declaration of opaque omatadd operation descriptor */
    typedef omatadd_descr *omatadd_descr_t; /* User-facing type for use in omatadd APIs */

    /* Host-side/non-blocking */
    void init_omatadd_descr(sycl::queue &queue,
                           omatadd_descr_t *p_descr);

    /* Asynchronous/non-blocking */
    sycl::event release_omatadd_descr(sycl::queue &queue,
                                      omatadd_descr_t descr,
                                      const std::vector<sycl::event> &dependencies = {});
}
```

omatadd APIs

```
namespace oneapi::mkl::sparse {
    /* Combined USM/sycl::buffer API, host-side/non-blocking */
    void omatadd_buffer_size(sycl::queue &queue,
                             transpose opA,
                             transpose opB,
                             matrix_handle_t A,          /* oneMKL Input sparse matrix handle */
                             matrix_handle_t B,          /* oneMKL Input sparse matrix handle */
                             matrix_handle_t C,          /* oneMKL Output sparse matrix handle */
                             omatadd_alg alg,
                             omatadd_descr_t descr,      /* omatadd operation descriptor */
                             std::int64_t &sizeTempWorkspace); /* Size of temporary workspace */

    /* sycl::buffer API, asynchronous/non-blocking */
    void omatadd_analyze(sycl::queue &queue,
                         transpose opA,
                         transpose opB,
                         matrix_handle_t A,
                         matrix_handle_t B,
                         matrix_handle_t C,
                         omatadd_alg alg,
                         omatadd_descr_t descr,
                         sycl::buffer<std::uint8_t, 1> *tempWorkspace); /* Temporary workspace */

    /* USM API, asynchronous/non-blocking */
    sycl::event omatadd_analyze(sycl::queue &queue,
                                transpose opA,
                                transpose opB,
                                matrix_handle_t A,
                                matrix_handle_t B,
                                matrix_handle_t C,
                                omatadd_alg alg,
                                omatadd_descr_t descr,
                                void *tempWorkspace,      /* Temporary workspace */
                                const std::vector<sycl::event> &dependencies = {});

    /* Combined USM/sycl::buffer API, synchronous/blocking */
    void omatadd_get_nnz(sycl::queue &queue,
                         transpose opA,
                         transpose opB,
```

```

        matrix_handle_t A,
        matrix_handle_t B,
        matrix_handle_t C,
        omatadd_alg alg,
        omatadd_descr_t descr,
        std::int64_t &nnzC,          /* Returned non-zero count of C matrix */
        const std::vector<sycl::event> &dependencies = {});

/* Combined USM/sycl::buffer API, asynchronous/non-blocking */
sycl::event omatadd(sycl::queue &queue,
                   transpose opA,
                   transpose opB,
                   const DATA_TYPE alpha,                      /* A-scaling factor */
                   matrix_handle_t A,
                   const DATA_TYPE beta,                        /* B-scaling factor */
                   matrix_handle_t B,
                   matrix_handle_t C,                            /* User arrays filled */
                   omatadd_alg alg,
                   omatadd_descr_t descr,
                   const std::vector<sycl::event> &dependencies = {});
}

```

Include Files

- `oneapi/mkl/spblas.hpp`

API Parameters

Input Parameters

<code>queue</code>	Specifies the SYCL command queue to be used for execution of SYCL kernels.
<code>opA, opB</code>	Specifies operation <code>op()</code> on input matrices, A and B , as one of the <code>oneapi::mkl::transpose</code> enums. All combinations of <code>opA</code> and <code>opB</code> are supported.
<code>alpha, beta</code>	Specifies the scalars, α and β , to scale $\text{op}(A)$ and $\text{op}(B)$ matrices, respectively.
<code>A, B</code>	The matrix handles of the input sparse matrices being added. A and B need not be in a sorted state as input to <code>omatadd</code> APIs, but performance may significantly benefit from it if the <code>sparse::sort_matrix()</code> API has been called on both the matrices, or if the <code>sparse::set_matrix_property()</code> API has been called on both the matrices to set the <code>sorted</code> property to guarantee sorted user input. The order of A and B must not be changed across API calls.

NOTE Only the CSR matrix format is currently supported for A and B .

<code>alg</code>	The <code>omatadd_alg</code> enum specifying choice of algorithm to use for the operation. For a given set of inputs and descriptor, <code>alg</code> must not be changed across API calls.
------------------	---

<code>descr</code>	The <code>omatadd_descr_t</code> descriptor object storing input data, operation-specific information, and user-provided temporary workspace. It is created and destroyed using the sparse::init_omatadd_descr , sparse::release_omatadd_descr routines.
<code>p_descr</code>	Pointer to the <code>omatadd_descr_t</code> descriptor object, <code>descr</code> , used for allocating it.
<code>tempWorkspace</code>	A SYCL-aware container (<code>sycl::buffer</code> or device-accessible USM pointer) of size <code>sizeTempWorkspace</code> bytes used as a temporary workspace for the matrix addition operation. The workspace must remain valid through the full <code>omatadd</code> multi-stage calls and must not be modified between matrix addition API calls or for the lifetime of <code>descr</code>. For <code>sycl::buffer</code> inputs, <code>tempWorkspace</code> is of type <code>sycl::buffer<std::uint8_t> *</code>. For USM inputs, <code>tempWorkspace</code> is a <code>void *</code> pointer that must be device-accessible. The recommended USM memory type for this is USM device for best performance, but USM shared and USM host allocations are also supported as they are device accessible.
<code>dependencies</code>	A vector of type <code>const std::vector<sycl::event> &</code> containing the list of events that the routine being called depends on to complete first, if any.

Input/Output Parameters

<code>C</code>	The input/output matrix handle for the <code>omatadd</code> operation. The 0- or 1-based indexing parameter set in the C matrix handle is an input to the <code>omatadd</code> operation. The sparse matrix arrays are user-allocated and user-owned, and are stored in the matrix handle using one of the <code>sparse::set_<xyz>_data</code> routines. The library fills the data as part of the <code>omatadd</code> operation. The output matrix arrays are not guaranteed to be sorted.
----------------	--

NOTE Only the CSR matrix format is currently supported for C .

NOTE If sorted output data is needed, then separately call the [sparse::sort_matrix\(\)](#) API after the final `sparse::omatadd` API call.

NOTE Aliasing the C matrix handle or arrays with either of the input A and B handles or their arrays (therefore attempting an “in-place” addition operation) is undefined behavior.

Output Parameters

<code>sizeTempWorkspace</code>	An integer of type <code>std::int64_t</code> containing the size in bytes of the temporary workspace, <code>tempWorkspace</code> , that the user must allocate for <code>omatadd</code> calls. This parameter is obtained from the <code>omatadd_buffer_size</code> API.
--------------------------------	--

nnzC An integer of type `std::int64_t` containing the format specific number of non-zeros in the output C matrix, to be used by users to allocate and own the output matrix arrays. This parameter is obtained from the `omatadd_get_nnz` API.

Return types where applicable

sycl::event SYCL event that can be waited upon, and in case of USM APIs, must be carried over and added as a dependency for the completion of subsequent stages of the `omatadd` routines.

Examples

Some examples of how to use `oneapi::mkl::sparse::omatadd` with SYCL buffers or USM can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_omatadd.cpp
```

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_omatadd_usm.cpp
```

oneapi::mkl::sparse::init_matmat_descr

Allocates and initializes a `oneapi::mkl::sparse::matmat_descr_t` object to default values.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::init_matmat_descr` routine allocates and initializes the `oneapi::mkl::sparse::matmat_descr_t` object with default values, otherwise it throws an exception.

NOTE Refer to [Error Handling](#) for a detailed description of the exceptions thrown.

A common usage model for the `matmat` descriptor is the following

```
using namespace oneapi::mkl;
sparse::matmat_descr_t descr = nullptr;
sparse::init_matmat_descr(&descr);

// example descriptor for general
// C = A * B^T
sparse::matrix_view_descr viewA = sparse::matrix_view_descr::general;
sparse::matrix_view_descr viewB = sparse::matrix_view_descr::general;
sparse::matrix_view_descr viewC = sparse::matrix_view_descr::general;
transpose opA = transpose::nontrans;
transpose opB = transpose::trans;

sparse::set_matmat_data(descr, viewA, opA, viewB, opB, viewC);

// use descr in sparse::matmat() api

sparse::release_matmat_descr(&descr);
```

API

Syntax

```
namespace oneapi::mkl::sparse {
    void init_matmat_descr ( oneapi::mkl::sparse::matmat_descr_t *p_descr );
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

`p_descr` The address of an uninitialized (nullptr) `matmat_descr_t` object to be initialized in this routine.

Output Parameters

`p_descr` On return, the address will be updated to point to a newly allocated and initialized `sparse::matmat_descr_t` object ready for use in `matmat` routines.

oneapi::mkl::sparse::set_matmat_data

Sets the appropriate `oneapi::mkl::sparse::matrix_view_descr` and `oneapi::mkl::transpose` values in the `oneapi::mkl::sparse::matmat_descr_t` object reflecting the `sparse::matmat` operation to be performed: $C = \text{op}(A) \cdot \text{op}(B)$.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::set_matmat_data` routine enables the user to set the desired sparse matrix - sparse matrix operation in the `oneapi::mkl::sparse::matmat_descr_t` object which will be used in the `sparse::matmat` routine.

The `oneapi::mkl::sparse::matrix_view_descr` enum class is defined in the `oneapi/mkl/spblas.hpp` header file

```
namespace oneapi::mkl::sparse {
    enum class matrix_view_descr : std::int32_t {
        general
    };
}
```

where `general` view assumes all data is populated in the `sparse::matrix_handle_t` object for both lower, diagonal and upper portions of the matrix. A common usage model for the `matmat` descriptor is the following:

```
using namespace oneapi::mkl;
sparse::matmat_descr_t descr = nullptr;
sparse::init_matmat_descr(&descr);

// example descriptor for general
// C = A * B
sparse::matrix_view_descr viewA = sparse::matrix_view_descr::general;
sparse::matrix_view_descr viewB = sparse::matrix_view_descr::general;
sparse::matrix_view_descr viewC = sparse::matrix_view_descr::general;
transpose opA = transpose::nontrans;
```

```
transpose opB = transpose::nontrans;

sparse::set_matmat_data(descr, viewA, opA, viewB, opB, viewC);

// use descr in sparse::matmat() api

sparse::release_matmat_descr(&descr);
```

API

Syntax

```
namespace oneapi::mkl::sparse {
    void set_matmat_descr (
        sparse::matmat_descr_t descr,
        sparse::matrix_view_descr viewA,
        transpose opA,
        sparse::matrix_view_descr viewB,
        transpose opB,
        sparse::matrix_view_descr viewC);
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

viewA, viewB, viewC

`sparse::matrix_view_descr` enum values describing how the A , B and C matrix representations are to be viewed. Currently, only the **general** type is supported.

opA, opB

Specifies operation `op()` on input matrix.

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $\text{op}(A) = A$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $\text{op}(A) = A^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $\text{op}(A) = A^H$.

NOTE Currently, the only supported case for operation is `oneapi::mkl::transpose::nontrans`.

Output Parameters

descr

`sparse::matmat_descr_t` object used to define the sparse matrix - sparse matrix operation to be performed by the `sparse::matmat` routine.

`oneapi::mkl::sparse::get_matmat_data`

Queries the `oneapi::mkl::sparse::matrix_view_descr` and `oneapi::mkl::transpose` values in the `oneapi::mkl::sparse::matmat_descr_t` object reflecting the `sparse::matmat` operation to be performed: $C = \text{op}(A) \cdot \text{op}(B)$.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::get_matmat_data` routine enables user to query the enum values set for the sparse matrix - sparse matrix operation in the `oneapi::mkl::sparse::matmat_descr_t` object which will be used in the [sparse::matmat](#) routine.

The `oneapi::mkl::sparse::matrix_view_descr` enum class is defined in the `oneapi/mkl/spblas.hpp` header file

```
namespace oneapi::mkl::sparse {
    enum class matrix_view_descr : std::int32_t {
        general
    };
}
```

where `general` view assumes all data is populated in the `sparse::matrix_handle_t` object for both lower, diagonal and upper portions of the matrix. A common usage model for the `matmat` descriptor is the following:

```
using namespace oneapi::mkl;
sparse::matmat_descr_t descr = NULL;
sparse::init_matmat_descr(descr);
// someone sets matmat descr

// query what was in descr
sparse::matrix_view_descr viewA, viewB, viewC;
transpose opA, opB;
sparse::get_matmat_data(descr, viewA, opA, viewV, opB, viewC);

sparse::release_matmat_descr(descr);
```

API

Syntax

```
namespace oneapi::mkl::sparse {
    void get_matmat_descr (
        sparse::matmat_descr_t descr,
        sparse::matrix_view_descr &viewA,
        transpose &opA,
        sparse::matrix_view_descr &viewB,
        transpose &opB,
        sparse::matrix_view_descr &viewC);
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

`descr` `sparse::matmat_descr_t` object used to define the sparse matrix - sparse matrix operation to be performed by the `sparse::matmat` routine.

Output Parameters

`viewA`, `viewB`, `viewC` `sparse::matrix_view_descr` enum values describing how the A , B and C matrix representations are to be viewed. Currently, only the **general** type is supported.

`opA`, `opB` Specifies operation `op()` on input matrix.

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $\text{op}(A) = A$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $\text{op}(A) = A^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $\text{op}(A) = A^H$.

NOTE Currently, the only supported case for operation is `oneapi::mkl::transpose::nontrans`.

`oneapi::mkl::sparse::release_matmat_descr`

Releases `oneapi::mkl::sparse::matmat_descr_t` object and sets it to NULL.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::release_matmat_descr` routine releases the `oneapi::mkl::sparse::matmat_descr_t` object and sets it to NULL.

A common usage model for the `matmat` descriptor is the following

```
using namespace oneapi::mkl;
sparse::matmat_descr_t descr = nullptr;
sparse::init_matmat_descr(&descr);

// example descriptor for general
// C = A * B
sparse::matrix_view_descr viewA = sparse::matrix_view_descr::general;
sparse::matrix_view_descr viewB = sparse::matrix_view_descr::general;
sparse::matrix_view_descr viewC = sparse::matrix_view_descr::general;
transpose opA = transpose::nontrans;
transpose opB = transpose::nontrans;

sparse::set_matmat_data(descr, viewA, opA, viewB, opB, viewC);

// use descr in sparse::matmat() api

sparse::release_matmat_descr(&descr);
```

API

Syntax

```
namespace oneapi::mkl::sparse {
    void release_matmat_descr ( oneapi::mkl::sparse::matmat_descr_t *p_descr );
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

`p_descr` The address of an initialized matmat descriptor object (previously allocated in `sparse::init_matmat_descr()`) to be deallocated in this routine.

Output Parameters

`p_descr` On return, the address will be updated to point to a null `sparse::matmat_descr_t`; the matmat descriptor object that was previously pointed to, will have been deallocated.

oneapi::mkl::sparse::matmat

Computes a sparse matrix-sparse matrix product.

- [Description](#)
- [API](#)
- [Examples](#)

Description

NOTE Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

The `oneapi::mkl::sparse::matmat` routine computes a sparse matrix-sparse matrix product defined as

$$C = \text{op}(A) \cdot \text{op}(B)$$

where A , B , and C are appropriately-sized sparse matrices and $\text{op}()$ is a matrix modifier:

$$\text{op}(A) = \begin{cases} A, & \text{oneapi::mkl::transpose::nontrans} \\ A^T, & \text{oneapi::mkl::transpose::trans} \\ A^H, & \text{oneapi::mkl::transpose::conjtrans.} \end{cases}$$

The sparse matrices are stored in the `matrix_handle_t` and currently only support the compressed sparse row (CSR) matrix format.

The output matrix C is not guaranteed to be sorted on exit from `sparse::matmat()`, but a helper function `sparse::sort_matrix()` is provided if that is a necessary property for subsequent usage.

The B matrix needs to be in sorted state prior to the call to `sparse::matmat()`.

The matrix A does not require a sorted state, but performance can benefit from it.

As the size of C and its data is generally not known beforehand, the `matmat` routine is broken into several stages which allow you to query the size of the data arrays, allocate them and then pass them back into the routine to be filled. This enables you to control all the C matrix data allocations themselves. Additionally, there are cases where only the sparsity pattern of C is desired, and this routine enables you to compute C without the values array. Generally the `sparse::matmat()` algorithm is broken into three computational stages:

Stage	Description
work_estimation	do initial estimation of work and load balancing (make upper bound estimate on size of C matrix data).
compute/compute_structure	do internal products for computing the C matrix including the calculation of size of C matrix data and filling the row pointer array for C.
finalize/finalize_structure	do any remaining internal products and accumulation and transfer into final C matrix arrays.

Some additional helper stages are provided to allow you to query sizes of temporary workspace arrays or the size of the C matrix data (`nnz(C)`) to be allocated. They are set and passed to the `sparse::matmat` routine as `matmat_request` enum values:

```
namespace oneapi::mkl::sparse {
    enum class matmat_request : std::int32_t {
        get_work_estimation_buf_size,
        work_estimation,

        get_compute_structure_buf_size,
        compute_structure,
        finalize_structure,

        get_compute_buf_size,
        compute,
        get_nnz,
        finalize
    };
}
```

A common workflow involves calling `sparse::matmat()` several times with different `matmat_request`'s:

- | | |
|---------------------------------|---|
| 1. Before matmat stages | 1. Allocate C matrix row pointer array and input into C matrix handle with dummy arguments for column and data arrays (as their sizes are not known yet). |
| 2. work_estimation stage | 1. Call matmat with <code>matmat_request::get_work_estimation_buf_size</code> .
2. Allocate the work estimation temporary workspace array.
3. Call matmat with <code>matmat_request::work_estimation</code> . |
| 3. Compute stage | 1. Call matmat with <code>matmat_request::get_compute_buf_size</code> .
2. Allocate the compute temporary workspace array.
3. Call matmat with <code>matmat_request::compute</code> . |
| 4. Finalize stage | 1. Call matmat with <code>matmat_request::get_nnz</code> .
2. Allocate the C matrix column and data arrays and input into C matrix handle.
3. Call matmat with <code>matmat_request::finalize</code> . |
| 5. After matmat stages | 1. Release or reuse the matmat descriptor for another appropriate sparse matrix product.
2. Release any particular sparse matrix product temporary workspace arrays allocated through the stages for this
3. Release or use C matrix handle for subsequent operations. |

Note that the `compute_structure` and `finalize_structure` and their helpers can be used if the final result desired is purely the sparsity pattern of C .

If you do not wish to handle the allocation and memory management of the temporary workspace arrays, there is a simplifying option to skip the `get_xxx_buf_size` queries for the `work_estimation` and `compute/compute_structure` stages and pass in null pointers for the `sizeTempBuffer` and `tempBuffer` arguments in the API for those stages. In this case, the library handles the allocation and memory management of the temporary arrays themselves. The internally allocated temporary arrays will live until the C matrix handle is destroyed. However, you are always expected to query the size of C matrix data and allocate the C matrix arrays themselves.

This simplified workflow is reflected here:

- | | |
|--------------------------------|--|
| 1. Before matmat stages | 1. Allocate C matrix row_pointer array and input into C matrix handle with dummy arguments for column and data arrays (as their sizes are not known yet). |
|--------------------------------|--|

2. work_estimation stage
 1. Call `matmat` with `sizeTempBuffer` and `tempBuffer` arguments. the `matmat_request::work_estimation` and `nullptr` for
3. Compute stage
 1. Call `matmat` with `tempBuffer` arguments. `matmat_request::compute` and `nullptr` for the `sizeTempBuffer`
4. Finalize stage
 1. Call `matmat` with `matmat_request::get_nnz`.
 2. Allocate the `C` matrix handle. matrix column and data arrays and input into `C`
 3. Call `matmat` with `matmat_request::finalize`.
5. After `matmat` stages
 1. Release or reuse the `matmat` descriptor for another appropriate sparse matrix product.
 2. Release or use the `C` matrix handle for subsequent operations.

These two workflows, and additionally, an example of computing only the sparsity pattern for `C` are demonstrated in the oneMKL DPC++ examples listed below.

API

Syntax

Using SYCL buffers:

```
namespace oneapi::mkl::sparse {
    void matmat(sycl::queue &queue,
               sparse::matrix_handle_t A,
               sparse::matrix_handle_t B,
               sparse::matrix_handle_t C,
               sparse::matmat_request req,
               sparse::matmat_descr_t descr,
               sycl::buffer<std::int64_t, 1> *sizeTempBuffer,
               sycl::buffer<std::uint8_t, 1> *tempBuffer);
}
```

Using USM pointers:

```
namespace oneapi::mkl::sparse {
    sycl::event matmat(sycl::queue &queue,
                      sparse::matrix_handle_t A,
                      sparse::matrix_handle_t B,
                      sparse::matrix_handle_t C,
                      sparse::matmat_request req,
                      sparse::matmat_descr_t descr,
```

```

std::int64_t *sizeTempBuffer,
void *tempBuffer,
const std::vector<sycl::event> &dependencies);
}

```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

queue Specifies the SYCL command queue which will be used for SYCL kernels execution.

A The matrix handle for the first matrix in the sparse matrix - sparse matrix product. Matrix *A* Does not need to be in a sorted state as input to `sparse::matmt()` but performance may benefit from it.

B The matrix handle for the second matrix in the sparse matrix - sparse matrix product. Matrix *B* currently must be in a sorted state as input to `sparse::matmat()`. To ensure the sorted property, use `sparse::sort_matrix()`.

NOTE We plan to remove this sorted restriction in a future release.

C The output matrix handle from the `matmat` operation. The sparse matrix format arrays will be allocated by the user and put into the matrix handle using a `sparse::set_<xyz>_data` routine. The data will be filled by the library as part of the `matmat` operation. Note that the output matrix may not be sorted, so for user convenience, we provide the api `sparse::sort_matrix()`.

request The **matmat_request** stage in the multi-stage algorithm. See descriptions of common workflows above.

descr The **matmat_descr_t** object describing the sparse matrix-sparse matrix operation to be executed. It is manipulated using the `sparse::init_matmat_descr`, `sparse::set_matmat_data` and `sparse::release_matmat_descr` routines.

sizeTempBuffer A SYCL aware container (`sycl::buffer` or host-accessible USM pointer) of the length of one **std::int64_t** to represent the size in bytes of the **tempBuffer**. For the **matmat_request** stages with the **get_xyz** naming convention the value is set by the library to inform the user how much memory to allocate in the temporary buffer. In the other **work_estimation** and **compute/comute_structure** stages, it is passed in along with the temporary buffer, **tempBuffer**, informing the library how much space was provided in bytes.

For `sycl::buffer` inputs, **sizeTempBuffer** is of type `sycl::buffer<std::int64_t>`.

For USM inputs, **sizeTempBuffer** must be host-accessible and of `std::int64_t *` type. The recommended USM memory type is described in the following table. In general, using USM host memory will provide better performance than USM shared, but both are supported as they are both host accessible.

sizeTempBuffer	filled in stage	size (in bytes) of which array(s)	USM Memory Type
size_temp_buffer 1	get_work_estimation_buf_size	temp_buffer1 in work_estimation	host accessible (USM host or USM shared)
size_temp_buffer 2	get_compute_buf_size or get_compute_structure_buf_size	temp_buffer2 in compute or compute_structure	host accessible (USM host or USM shared)
nnz_buffer	get_nnz	C colind/values arrays for finalize finalize_structure	host accessible (USM host or USM shared)

tempBuffer

A SYCL-aware container (`sycl::buffer` or device-accessible USM pointer) of **sizeTempBuffer** bytes used as a temporary workspace in the algorithm. There are two stages where separate workspaces must be passed into the `matmat` api (**work_estimation** and **compute/compute_structure**). They must remain valid through the full `matmat` multi-stage algorithm as both may be used until the last **finalize/finalize_structure** request is completed.

For `sycl::buffer` inputs, **tempBuffers** is of type `sycl::buffer<std::uint8_t>`.

For USM inputs, **tempBuffers** must be device-accessible and is passed in as a `void *` type. The recommended USM memory type is described in the following table. In general, using USM device memory will provide a better performance than USM shared which will give better performance than USM host in `sparse::matmat()`, but all are supported as they are all device accessible.

tempBuffer	array provided in stage	size of array set in stage	USM Memory Type
temp_buffer1	work_estimation	get_work_estimation_buf_size	device accessible (USM device or USM shared or USM host)
temp_buffer2	compute or compute_structure	get_compute_buf_size or get_compute_structure_buf_size	device accessible (USM device or USM shared or USM host)

dependencies (for USM APIs only)

A vector of type `std::vector<sycl::event>` containing the list of events that the current stage of `oneapi::mkl::sparse::matmat` routine depends on.

Output Parameters

C

Data arrays for C will be allocated by the user and filled by the library as part of the `matmat` algorithm.

The output sparse matrix data arrays for C are not guaranteed to be sorted, and [sparse::sort_matrix\(\)](#) is provided in case the sorted property is desired for subsequent operations with the output sparse matrix.

Return Values (USM Only)

`sycl::event` SYCL event which can be waited upon or added as a dependency for the completion of the stages of the `matmat` routine.

Examples

Some examples of how to use `oneapi::mkl::sparse::matmat` with SYCL buffers or USM can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_matmat.cpp
share/doc/mkl/examples/sycl/sparse_blas/source/csr_matmat_simplified.cpp
share/doc/mkl/examples/sycl/sparse_blas/source/csr_matmat_structure_only.cpp

share/doc/mkl/examples/sycl/sparse_blas/source/csr_matmat_usm.cpp
share/doc/mkl/examples/sycl/sparse_blas/source/csr_matmat_simplified_usm.cpp
share/doc/mkl/examples/sycl/sparse_blas/source/csr_matmat_structure_only_usm.cpp
```

oneapi::mkl::sparse::matmatd

Computes a sparse matrix-sparse matrix product with a dense result.

- [Description](#)
- [API](#)
- [Examples](#)

Description

NOTE Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types, and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

The `oneapi::mkl::sparse::matmatd` routine computes a sparse matrix-sparse matrix product with a dense matrix result defined as

$$C \leftarrow \alpha \cdot \text{op}(A) \cdot \text{op}(B) + \beta \cdot C$$

where: α and β are scalars, A and B are sparse matrices, and C is a dense matrix of size `c_nrows` rows by `c_ncols` columns, and `op()` is a matrix modifier for A and B using the following description:

$$\text{op}(A) = \begin{cases} A, & \text{oneapi::mkl::transpose::nontrans} \\ A^T, & \text{oneapi::mkl::transpose::trans} \\ A^H, & \text{oneapi::mkl::transpose::conjtrans.} \end{cases}$$

The dense matrix object **C** is stored with row-major or column-major layout. The sparse matrix objects **A** and **B** have an appropriate number of rows and columns for the matrix product.

API

Syntax

Using SYCL buffers:

```
namespace oneapi::mkl::sparse {
    void matmatd(sycl::queue &queue,
        oneapi::mkl::layout c_layout,
        oneapi::mkl::transpose opA,
        oneapi::mkl::transpose opB,
        const DATA_TYPE alpha,
        matrix_handle_t A,
        matrix_handle_t B,
        const DATA_TYPE beta,
        sycl::buffer<DATA_TYPE, 1> &C,
        const std::int64_t c_nrows,
        const std::int64_t c_ncols,
        const std::int64_t ldc);
}
```

Using USM pointers:

```
namespace oneapi::mkl::sparse {
    sycl::event matmatd(
        sycl::queue &queue,
        oneapi::mkl::layout c_layout,
        oneapi::mkl::transpose opA,
        oneapi::mkl::transpose opB,
        const DATA_TYPE alpha,
        matrix_handle_t A,
        matrix_handle_t B,
        const DATA_TYPE beta,
        DATA_TYPE *C,
        const std::int64_t c_nrows,
        const std::int64_t c_ncols,
        const std::int64_t ldc,
        const std::vector<sycl::event> &dependencies = {});
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

<code>queue</code>	Specifies the SYCL command queue that will be used for SYCL kernels execution.
<code>c_layout</code>	Specifies the storage scheme in memory for the dense matrix C .
<code>opA</code>	Specifies operation <code>op()</code> on input matrix A .

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $\text{op}(A) = A$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $\text{op}(A) = A^T$.

<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $\text{op}(A) = A^H$.
--	--

opB

Specifies operation `op()` on input matrix B.

<code>oneapi::mkl::transpose::nontrans</code>	Non-transpose, $\text{op}(B) = B$.
<code>oneapi::mkl::transpose::trans</code>	Transpose, $\text{op}(B) = B^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	Conjugate transpose, $\text{op}(B) = B^H$.

alpha

Specifies the scalar, α .

A

Handle to object containing sparse matrix and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE The only currently supported case for `<sparse_matrix_type>` is `csr`.

B

Handle to object containing sparse matrix and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE The only currently supported case for `<sparse_matrix_type>` is `csr`.

beta

Specifies the scalar, β .

C

SYCL buffer or device-accessible USM pointer of size at least `rows*cols`, where

	<code>layout=oneapi::mkl::layout::col_major</code>	<code>layout=oneapi::mkl::layout::row_major</code>
rows (number of rows in C)	<code>ldc</code>	<code>c_ncols</code>
cols (number of columns in C)	<code>c_nrows</code>	<code>ldc</code>

c_nrows

Number of rows of matrix C.

c_ncols

Number of columns of matrix C.

ldc

Specifies the leading dimension of matrix C. Must be positive, and at least `c_ncols` if `c_layout=oneapi::mkl::layout::row_major` or at least `c_nrows` if `c_layout=oneapi::mkl::layout::col_major`.

dependencies

A vector of type `std::vector<sycl::event>` containing the list of events that the `oneapi::mkl::sparse::matmatd` routine depends on.

Output Parameters

C Overwritten by the updated matrix C.

Return Values (USM Only)

sycl::event SYCL event that can be waited upon or added as a dependency for the completion of the `matmatd` routine.

Examples

An example of how to use `oneapi::mkl::sparse::matmatd` with SYCL buffers or USM can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_matmatd.cpp
```

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr_matmatd_usm.cpp
```

oneapi::mkl::sparse::omatcopy

Performs an out-of-place copy/transpose of user data in a matrix handle into user data in a new handle without changing the sparse matrix format.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::omatcopy` API copies user-provided sparse matrix arrays stored in a given sparse matrix handle into those provided in another sparse matrix handle. The routine enables users to perform a transpose operation and/or a change in array indexing in the copy. Only *out-of-place* copy/transpose is supported through this API.

NOTE Refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

NOTE This routine modifies arrays provided to oneMKL through the matrix handle creation routines, `oneapi::mkl::sparse::set_<sparse_matrix_type>_data`. Arrays of only the output matrix handle are modified, and those of the input handle are not.

API

Syntax

Using USM and SYCL buffers:

```
namespace oneapi::mkl::sparse {
    sycl::event omatcopy (
        sycl::queue &queue,
        oneapi::mkl::transpose transpose_val,
        oneapi::mkl::sparse::matrix_handle_t spMat_in,
        oneapi::mkl::sparse::matrix_handle_t spMat_out,
        const std::vector<sycl::event> &dependencies = {});
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

<code>queue</code>	Specifies the SYCL command queue which will be used for SYCL kernels execution.
<code>transpose_val</code>	Specifies operation <code>op()</code> on input matrix.

<code>oneapi::mkl::transpose::nontrans</code>	• Non-transpose, $op(A) = A$.
<code>oneapi::mkl::transpose::trans</code>	• Transpose, $op(A) = A^T$.
<code>oneapi::mkl::transpose::conjtrans</code>	• Conjugate transpose, $op(A) = A^H$.

<code>spMat_in</code>	Handle to object containing input sparse matrix and other internal data. Created using one of the <code>oneapi::mkl::sparse::set_<sparse_matrix_type>_data</code> routines.
-----------------------	---

NOTE The supported cases for `<sparse_matrix_type>` are `csr` and `coo` on both CPU and GPU devices. `spMat_out` must be created with the same `<sparse_matrix_type>`.

<code>spMat_out</code>	Handle to object containing output sparse matrix and other internal data. Created using one of the <code>oneapi::mkl::sparse::set_<sparse_matrix_type>_data</code> routines. Note that the sparse matrix dimensions/array lengths of this output handle must be same as that of the input handle in case of <code>oneapi::mkl::transpose::nontrans</code> operation. The number of rows and columns of this output matrix handle for <code>oneapi::mkl::transpose::trans</code> and <code>oneapi::mkl::transpose::conjtrans</code> operations must be the number of columns and rows of the input matrix handle, respectively. Also note that in case of a transpose operation, the number of non-zeros in the sparse matrix remains unchanged.
------------------------	--

NOTE The supported cases for `<sparse_matrix_type>` are `csr` and `coo` on both CPU and GPU devices. `spMat_in` must be created with the same `<sparse_matrix_type>`.

<code>dependencies</code>	A vector of type <code>std::vector<sycl::event></code> & containing the list of events that the <code>oneapi::mkl::sparse::omatcopy</code> routine depends on.
---------------------------	--

Return Values

`sycl::event`

SYCL event which can be waited upon or added as a dependency for the completion of the `omatcopy` routine.

```
// Examples of API arguments and usage
sycl::event ev_opt = sparse::omatcopy(queue, trans_val, spMat_in,
spMat_out); // Allowed use in case of USM
sycl::event ev_opt = sparse::omatcopy(queue, trans_val, spMat_in,
spMat_out, {}); // Allowed use in case of USM
sycl::event ev_opt = sparse::omatcopy(queue, trans_val, spMat_in,
spMat_out, dependencies); // Allowed use in case of USM
sparse::omatcopy(queue, trans_val, spMat_in,
spMat_out); // Recommended use in
case of sycl::buffer
sparse::omatcopy(queue, trans_val, spMat_in, spMat_out,
{}); // Not recommended in case of
sycl::buffer, but supported;
sparse::omatcopy(queue, trans_val, spMat_in, spMat_out,
dependencies); // Not recommended in case of
sycl::buffer, but supported;
sycl::event ev_opt = sparse::omatcopy(queue, trans_val, spMat_in,
spMat_out); // Not recommended in case of sycl::buffer,
but supported. Note that most sycl::buffer oneMKL APIs do not have
ability to pass in dependencies
```

oneapi::mkl::sparse::omatconvert

Performs an out-of-place conversion of user data in a matrix handle into user data in a new handle in a different sparse matrix format.

- [Description](#)
- [APIs](#)
- [Examples](#)

Description

NOTE Refer to [Sparse BLAS Supported Data and Integer Types](#) for a list of supported `<DATA_TYPE>` and `<INT_TYPE>` data and integer types and refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

The `oneapi::mkl::sparse::omatconvert` set of routines can be used to convert a sparse matrix in a given sparse format, referred to as the input matrix, to another sparse matrix in another format, referred to as the output matrix. Only *out-of-place* conversion is supported through these APIs. The only sparse matrix formats supported by the APIs are compressed sparse row (CSR) and coordinate list format (COO). The sparse matrix formats for the input and output matrices must be different; please refer to the [sparse::omatcopy](#) API to perform a copy of the matrix, including potentially changing the `index_base` in said copy or copying the transpose of the matrix.

The whole conversion process is decomposed in a series of stages. Each stage consists of a call to one of the `omatconvert` routines, and may also include some memory allocation by the user. The API is designed such that minimal data is allocated internally and users are responsible for most of the required memory allocation. This applies to any temporary workspace that might be needed during the conversion process and to the memory used for the data of the output matrix.

The sizes required to allocate the data for the output matrix may depend on parameters that are known a priori, such as the number of rows or columns, and on parameters that might not, such as the number of non-zero elements in the output matrix representation. To address the second situation, APIs are provided to compute the number of non-zero elements in the output matrix before going into the actual conversion.

The stages for the `omatconvert` algorithm are given below (for the definitions, see [omatconvert APIs and stages](#)):

Stage	Description
<code>omatconvert_buffer_size</code>	Return the size of the temporary workspace.
<code>omatconvert_analyze</code>	Count the number of non-zero values (<code>nnzOut</code>) in the output sparse matrix.
<code>omatconvert_get_nnz</code>	Return the calculated <code>nnzOut</code> count of the output sparse matrix.
<code>omatconvert</code>	Perform the conversion from the input to the output matrix.

A detailed step-by-step description of the conversion process is as follows:

1. Before `omatconvert` stages:

- Create a handle for the output matrix using `oneapi::mkl::sparse::set_<sparse_matrix_type>_data`, where `<sparse_matrix_type>` represents the desired format for the output matrix.
- The indexing for the output matrix can be different than that of the input matrix.
- At this stage, the number of non-zero elements can be set to 0 and dummy arguments can be used for the matrix representation array (e.g. for CSR: `rowptr`, `colind` and `values`).
- Allocate and initialize an `omatconvert_descr_t` object using the `init_omatconvert_descr` API and decide on an algorithm to use through the `omatconvert_algenum`. The algorithm must not be changed within the series of `omatconvert` APIs calls.

2. `omatconvert_buffer_size` stage:

- This is a non-blocking API that may access the input matrix arrays.
- Call `omatconvert_buffer_size` API to get the size of the temporary workspace.
- Allocate the temporary workspace to be used in subsequent stages.

3. `omatconvert_analyze` stage:

- This is a non-blocking asynchronous API that accesses and analyzes the sparsity pattern of the input matrix, and the already known parameters of the output matrix.
- Call the `omatconvert_analyze`, passing in the temporary workspace allocated in the previous stage.
- The temporary workspace array is internally stored in the `omatconvert_descr_t` object. Do not modify or free the workspace for the duration of its use for the sparse matrix conversion, which is governed by the lifetime of the `omatconvert_descr_t` object.

4. `omatconvert_get_nnz` stage:

- This is a blocking API that returns the non-zero count obtained from the `omatconvert_analyze` stage back to the user on the host.
- Call the `omatconvert_get_nnz` API to get the non-zero count associated with the output sparse matrix.
- Allocate the appropriately sized output sparse matrix arrays.

- Call the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` API again, this time with the valid, newly allocated row, column, and data arrays of the output sparse matrix. The output 0-/1-based indexing must not be changed at this point, and the number of rows and columns of the matrix must be mathematically consistent with the input matrix sizes at this point.

5. `omatconvert` stage:

- This is a non-blocking asynchronous API that performs the sparse matrix conversion to fill the user-provided output matrix arrays in the requested sparse format.
- Call the `omatconvert` API.

6. After `omatconvert` stage:

- Release or reuse the `omatconvert_descr_t` object for another appropriate conversion.
- Release or reuse the temporary workspace arrays allocated for use with this API.

APIs

Syntax

`enum omatconvert_alg` and conversion algorithms

The `omatconvert_alg` enum provides users a choice of using specific algorithms implemented in oneMKL. This enum is defined as:

```
namespace oneapi::mkl::sparse {
    enum class omatconvert_alg : std::int32_t {
        default_alg = 0 /* More may be added in the future */
    };
}
```

The list of currently supported algorithms for all conversion configurations is given below:

Conversion type	Supported algorithms	Algorithm description
CSR -> COO	<code>omatconvert_alg::default</code>	Default algorithm for the conversion from CSR to COO
COO -> CSR	<code>omatconvert_alg::default</code>	Default algorithm for the conversion from COO to CSR. Duplicate elements are condensed.

`omatconvert_descr_t` object

`omatconvert_descr_t` is an operation-specific opaque descriptor object used to store internal state between calls to the `omatconvert` set of APIs. Once a given descriptor object is used in any of the APIs, it should not be changed or free'd until all calls to `omatconvert` APIs are completed. A pointer to the user-provided temporary workspace is stored in this descriptor object through one of the `omatconvert` set of APIs, viz., `omatconvert_analyze`, described below. There are initialization and release functions associated with this descriptor object.

```
namespace oneapi::mkl::sparse {
    struct omatconvert_descr; /* Forward declaration of opaque omatconvert operation
    descriptor */
    typedef omatconvert_descr *omatconvert_descr_t; /* User-facing type for use in omatconvert
    APIs */

    /* Host-side/non-blocking */
    void init_omatconvert_descr(sycl::queue &queue,
```

```

        omatconvert_descr_t *p_descr);

/* Asynchronous/non-blocking */
sycl::event release_omatconvert_descr(sycl::queue &queue,
                                     omatconvert_descr_t descr,
                                     const std::vector<sycl::event> &dependencies = {});
}

```

omatconvert APIs and stages

```

namespace oneapi::mkl::sparse {

/* Combined USM/sycl::buffer API, host-side/non-blocking */
void omatconvert_buffer_size(
    sycl::queue &queue,
    matrix_handle_t spMat_in,
    matrix_handle_t spMat_out,
    omatconvert_alg alg,
    omatconvert_descr_t descr,
    std::int64_t &sizeTempWorkspace);

/* sycl::buffer API, asynchronous/non-blocking */
sycl::event omatconvert_analyze(
    sycl::queue &queue,
    matrix_handle_t spMat_in,
    matrix_handle_t spMat_out,
    omatconvert_alg alg,
    omatconvert_descr_t descr,
    void *tempWorkspace,
    const std::vector<sycl::event> &dependencies = {});

/* USM API, asynchronous/non-blocking */
void omatconvert_analyze(
    sycl::queue &queue,
    matrix_handle_t spMat_in,
    matrix_handle_t spMat_out,
    omatconvert_alg alg,
    omatconvert_descr_t descr,
    sycl::buffer<std::uint8_t, 1> *tempWorkspace);

/* Combined USM/sycl::buffer API, synchronous/blocking */
void omatconvert_get_nnz(
    sycl::queue &queue,
    matrix_handle_t spMat_in,
    matrix_handle_t spMat_out,
    omatconvert_alg alg,
    omatconvert_descr_t descr,
    std::int64_t &nnzOut,
    const std::vector<sycl::event> &dependencies = {});

/* Combined USM/sycl::buffer API, asynchronous/non-blocking */
sycl::event omatconvert(
    sycl::queue &queue,
    matrix_handle_t spMat_in,
    matrix_handle_t spMat_out,
    omatconvert_alg alg,

```

```
omatconvert_descr_t descr,
const std::vector<sycl::event> &dependencies = {});
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

queue Specifies the SYCL command queue that will be used for SYCL kernels execution.

spMat_in A handle to an object containing the input sparse matrix and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` and `coo` on both CPU and GPU devices.

spMat_out A handle to an object containing the output sparse matrix and other internal data. Created using one of the `oneapi::mkl::sparse::set_<sparse_matrix_type>_data` routines. The dimensions of the sparse matrices in the input and output handles must be the same. At this stage, it is required to have allocated the data for the arrays of this handle.

NOTE The supported cases for `<sparse_matrix_type>` are `csr` and `coo` on both CPU and GPU devices. The `<sparse_matrix_type>` of `spMat_out` must be different than that of `spMat_in`.

alg The `omatconvert_algenum` specifying the algorithm to use for the conversion. See available options above.

descr The `omatconvert_descr_t` descriptor object storing input data and operation-specific information and the user-provided temporary workspace. It is created and destroyed using the `sparse::init_omatconvert_descr` and `sparse::release_omatconvert_descr` APIs.

tempWorkspace A SYCL-aware container (`sycl::buffer` or device-accessible USM pointer) of size `sizeTempWorkspace` bytes used as a temporary workspace for the matrix conversion operation. The workspace must remain valid through the full `omatconvert` multi-stage calls and should not be modified in between calls as long as such calls are being made.

For `sycl::buffer` inputs, `tempWorkspace` is of type `sycl::buffer<std::uint8_t> *`.

For USM inputs, `tempWorkspace` is a `void *` pointer that must be device-accessible. The recommended USM memory type for this is **USM device** for best performance, but USM shared and USM host allocations are also supported as they are device accessible.

dependencies A vector of type `std::vector<sycl::event>` & containing the list of events that the `oneapi::mkl::sparse::omatconvert` routine depends on.

Output Parameter

<code>sizeTempWorkspace</code>	An integer of type <code>std::int64_t</code> containing the size, in bytes, of the temporary workspace, <code>tempWorkspace</code> , that must be allocated for calls to <code>omatconvert_analyze</code> and <code>omatconvert</code> . This parameter is obtained from the <code>omatconvert_buffer_size</code> API.
<code>nnzOut</code>	An integer of type <code>std::int64_t</code> containing the format specific number of non-zeros in the output matrix, to be used to allocate the output matrix arrays. This parameter is obtained from the <code>omatconvert_get_nnz</code> API.
<code>spMat_out</code>	On return, the data for the sparse matrix will be filled according to the requested <code><sparse_matrix_type></code> .

Return Values

<code>sycl::event</code>	SYCL event that can be waited upon or added as a dependency for the completion of the <code>omatconvert</code> routine.
--------------------------	---

Examples

Examples on how to use `oneapi::mkl::sparse::omatconvert` and other related APIs, with SYCL buffers or USM, can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/sparse_blas/source/csr2coo_omatconvert.cpp
```

```
share/doc/mkl/examples/sycl/sparse_blas/source/coo2csr_omatconvert_usm.cpp
```

oneapi::mkl::sparse::sort_matrix

Performs in-place sorting of user-provided matrix arrays in a matrix handle.

- [Description](#)
- [API](#)

Description

NOTE Refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

The `oneapi::mkl::sparse::sort_matrix` API performs in-place sorting of user-provided sparse matrix arrays stored in a given sparse matrix handle.

NOTE This routine modifies user-arrays provided to oneMKL through the matrix handle creation routines, `oneapi::mkl::sparse::set_<sparse_matrix_type>_data`.

API

Syntax

Using USM and SYCL buffers:

```
namespace oneapi::mkl::sparse {
    sycl::event sort_matrix (
        sycl::queue &queue,
```

```
oneapi::mkl::sparse::matrix_handle_t spMat,
const std::vector<sycl::event> &dependencies = {});
}
```

Include Files

- oneapi/mkl/spblas.hpp

Input Parameters

queue	Specifies the SYCL command queue which will be used for SYCL kernels execution.
spMat	Handle to object containing sparse matrix and other internal data. Created using one of the <code>oneapi::mkl::sparse::set_<sparse_matrix_type>_data</code> routines.

NOTE Currently, the only supported case for `<sparse_matrix_type>` is `csr`.

dependencies	A vector of type <code>std::vector<sycl::event></code> & containing the list of events that the <code>oneapi::mkl::sparse::sort_matrix</code> routine depends on.
--------------	---

Return Values

sycl::event	SYCL event which can be waited upon or added as a dependency for the completion of the <code>sort_matrix</code> routine.
-------------	--

```
// Examples of API arguments and usage
sycl::event ev_opt = sparse::sort_matrix(queue,
spMat); // Allowed use in case of USM
sycl::event ev_opt = sparse::sort_matrix(queue, spMat,
{}); // Allowed use in case of USM
sycl::event ev_opt = sparse::sort_matrix(queue, spMat,
dependencies); // Allowed use in case of USM
sparse::sort_matrix(queue,
spMat); // Recommended use in case
of sycl::buffer
sparse::sort_matrix(queue, spMat,
{}); // Not recommended in case of
sycl::buffer, but supported;
sparse::sort_matrix(queue, spMat,
dependencies); // Not recommended in case of
sycl::buffer, but supported;
sycl::event ev_opt = sparse::sort_matrix(queue,
spMat); // Not recommended in case of sycl::buffer, but
supported. Note that most sycl::buffer oneMKL APIs do not have ability
to pass in dependencies
```

oneapi::mkl::sparse::update_diagonal_values

Changes the values of the entries on the main diagonal of a sparse matrix. This routine also updates any data structures related to optimizations that have already been done (such as with `oneapi::mkl::sparse::optimize_trsv` or `oneapi::mkl::sparse::optimize_gemv`).

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::sparse::update_diagonal_values` routine changes the values of the entries on the main diagonal of a sparse matrix. The existing sparse structure must already have all diagonal entries included in the sparsity pattern.

NOTE Refer to [Error Handling](#) for a detailed description of the possible exceptions thrown.

API

Syntax

Using SYCL buffers:

```
namespace oneapi::mkl::sparse {
    void update_diagonal_values(
        sycl::queue &queue,
        oneapi::mkl::sparse::matrix_handle_t spMat,
        sycl::buffer<DATA_TYPE, 1> &new_diag_values);
}
```

Using USM pointers:

```
namespace oneapi::mkl::sparse {
    sycl::event update_diagonal_values(
        sycl::queue &queue,
        oneapi::mkl::sparse::matrix_handle_t spMat,
        std::int64_t length,
        const DATA_TYPE* new_diag_values,
        const std::vector<sycl::event> &dependencies = {});
}
```

Include Files

- `oneapi/mkl/spblas.hpp`

Input Parameters

<code>queue</code>	Specifies the SYCL command queue which will be used for SYCL kernels execution.
<code>spMat</code>	Handle to object containing sparse matrix and other internal data. Created using one of the <code>oneapi::mkl::sparse::set_<sparse_matrix_type>_data</code> routines.

NOTE Currently, the only supported case for `<sparse_matrix_type>` is `csr`. Only GPU device is currently supported.

<code>new_diag_values</code>	SYCL buffer or device-accessible USM pointer of size at least equal to the smaller of the number of rows and number of columns of the input matrix.
<code>length</code>	Length of the input vector <code>new_diag_values</code> . Must be at least equal to the smaller of the number of rows and number of columns of the input matrix.

dependencies A vector of type `std::vector<sycl::event>` containing the list of events that the `oneapi::mkl::sparse::update_diagonal_values` routine depends on.

Return Values (USM Only)

sycl::event SYCL event which can be waited upon or added as a dependency for the completion of the `update_diagonal_values` routine.

LAPACK Routines

Intel® oneAPI Math Kernel Library (oneMKL) implements routines from the LAPACK package that are used for solving systems of linear equations, linear least squares problems, eigenvalue and singular value problems, and performing a number of related computational tasks. The library includes LAPACK routines for both real and complex data. Routines are supported for systems of equations with the following types of matrices:

- General
- Banded
- Symmetric or Hermitian positive-definite (full, packed, and rectangular full packed (RFP) storage)
- Symmetric or Hermitian positive-definite banded
- Symmetric or Hermitian indefinite (both full and packed storage)
- Symmetric or Hermitian indefinite banded
- Triangular (full, packed, and RFP storage)
- Triangular banded
- Tridiagonal
- Diagonally dominant tridiagonal.

NOTE Different arrays used as parameters to oneMKL LAPACK routines must not overlap.

Warning LAPACK routines assume that input matrices do not contain IEEE 754 special values such as INF or NaN values. Using these special values may cause LAPACK to return unexpected results or become unstable.

gebrd

Reduces a general matrix to bidiagonal form. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine reduces a general m -by- n matrix A to a bidiagonal matrix B by an orthogonal (unitary) transformation.

If $m \geq n$, the reduction is given by

$$A = QBP^H = Q \begin{pmatrix} B_1 \\ 0 \end{pmatrix} P^H = Q_1 B_1 P^H$$

where B_1 is an n -by- n upper diagonal matrix, Q and P are orthogonal or, for a complex A , unitary matrices; Q_1 consists of the first n columns of Q .

If $m < n$, the reduction is given by

$$A = QBPH = Q(B_1 \ 0)P^H = QB_1P_1^H,$$

where B_1 is an m -by- m lower diagonal matrix, Q and P are orthogonal or, for a complex A , unitary matrices; P_1 consists of the first m columns of P .

The routine does not form the matrices Q and P explicitly, but represents them as products of elementary reflectors. Routines are provided to work with the matrices Q and P in this representation:

If the matrix A is real,

- to compute Q and P explicitly, call [orgbr](#).

If the matrix A is complex,

- to compute Q and P explicitly, call [ungbr](#).

API

Syntax

```
namespace oneapi::mkl::lapack {
    void gebrd(sycl::queue &queue,
               int64_t m, int64_t n,
               sycl::buffer<T> &a, int64_t lda,
               sycl::buffer<realT> &d, sycl::buffer<realT> &e, sycl::buffer<T> &tauq, sycl::buffer<T> &taup,
               sycl::buffer<T> &scratchpad, int64_t scratchpad_size)
}
```

`gebrd` supports the following precision and devices.

T	Devices Supported
float	CPU, GPU*
double	CPU, GPU*
std::complex<float>	CPU, GPU*
std::complex<double>	CPU, GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
a	The buffer holding matrix A . The buffer must be of size at least $lda*n$.
lda	The leading dimension of a .
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.

`scratchpad_size` Size of scratchpad memory as a number of floating point elements of type `T`. Size should not be less than the value returned by the [gebrd_scratchpad_size](#) function.

Output Parameters

`a` If $m \geq n$, the diagonal and first super-diagonal of `a` are overwritten by the upper bidiagonal matrix `B`. The elements below the diagonal, with the buffer `tauq`, represent the orthogonal matrix `Q` as a product of elementary reflectors, and the elements above the first superdiagonal, with the buffer `taup`, represent the orthogonal matrix `P` as a product of elementary reflectors.

If $m < n$, the diagonal and first sub-diagonal of `a` are overwritten by the lower bidiagonal matrix `B`. The elements below the first subdiagonal, with the buffer `tauq`, represent the orthogonal matrix `Q` as a product of elementary reflectors, and the elements above the diagonal, with the buffer `taup`, represent the orthogonal matrix `P` as a product of elementary reflectors.

`d` Buffer holding array of size at least $\max(1, \min(m, n))$. Contains the diagonal elements of `B`.

`e` Buffer holding array of size at least $\max(1, \min(m, n) - 1)$. Contains the off-diagonal elements of `B`.

`tauq` Buffer holding array of size at least $\max(1, \min(m, n))$. The scalar factors of the elementary reflectors which represent the orthogonal or unitary matrix `Q`.

`taup` Buffer holding array of size at least $\max(1, \min(m, n))$. The scalar factors of the elementary reflectors which represent the orthogonal or unitary matrix `P`.

Exceptions

Exception	Description
<code>mk1::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

gebrd (USM Version)

Reduces a general matrix to bidiagonal form. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine reduces a general m -by- n matrix A to a bidiagonal matrix B by an orthogonal (unitary) transformation.

If $m \geq n$, the reduction is given by

$$A = QB P^H = Q \begin{pmatrix} B_1 \\ 0 \end{pmatrix}^T P^H = Q_1 B_1 P^H$$

where B_1 is an n -by- n upper diagonal matrix, Q and P are orthogonal or, for a complex A , unitary matrices; Q_1 consists of the first n columns of Q .

If $m < n$, the reduction is given by

$$A = QB P^H = Q(B_1 \ 0) P^H = Q B_1 P_1^H,$$

where B_1 is an m -by- m lower diagonal matrix, Q and P are orthogonal or, for a complex A , unitary matrices; P_1 consists of the first m columns of P .

The routine does not form the matrices Q and P explicitly, but represents them as products of elementary reflectors. Routines are provided to work with the matrices Q and P in this representation:

If the matrix A is real,

- to compute Q and P explicitly, call [orgbr \(USM Version\)](#).

If the matrix A is complex,

- to compute Q and P explicitly, call [ungbr \(USM Version\)](#).

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event gebrd(sycl::queue &queue,
        int64_t m,
        int64_t n,
        T *a,
        int64_t lda,
        RealT *d,
        RealT *e,
        T *tauq,
        T *taup,
        T *scratchpad,
        int64_t scratchpad_size, const std::vector<sycl::event> &events = {})
}
```

`gebrd` (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU, GPU*
double	CPU, GPU*
std::complex<float>	CPU, GPU*

T	Devices supported
<code>std::complex<double></code>	CPU, GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>m</code>	The number of rows in the matrix A ($0 \leq m$).
<code>n</code>	The number of columns in the matrix A ($0 \leq n$).
<code>a</code>	Pointer to matrix A . The size of <code>a</code> must be at least <code>lda*n</code> .
<code>lda</code>	The leading dimension of <code>a</code> .
<code>scratchpad</code>	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the gebrd_scratchpad_size function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

<code>a</code>	If $m \geq n$, the diagonal and first super-diagonal of <code>a</code> are overwritten by the upper bidiagonal matrix B. The elements below the diagonal, with the <code>tauq</code>, represent the orthogonal matrix Q as a product of elementary reflectors, and the elements above the first superdiagonal, with the <code>taup</code>, represent the orthogonal matrix P as a product of elementary reflectors. If $m < n$, the diagonal and first sub-diagonal of <code>a</code> are overwritten by the lower bidiagonal matrix B. The elements below the first subdiagonal, with the <code>tauq</code>, represent the orthogonal matrix Q as a product of elementary reflectors, and the elements above the diagonal, with the <code>taup</code>, represent the orthogonal matrix P as a product of elementary reflectors.
<code>d</code>	Pointer to memory of size at least <code>max(1, min(m,n))</code> . Contains the diagonal elements of B .
<code>e</code>	Pointer to memory of size at least <code>max(1, min(m,n) - 1)</code> . Contains the off-diagonal elements of B .
<code>tauq</code>	Pointer to memory of size at least <code>max(1, min(m, n))</code> . The scalar factors of the elementary reflectors which represent the orthogonal or unitary matrix Q .
<code>taup</code>	Pointer to memory of size at least <code>max(1, min(m, n))</code> . The scalar factors of the elementary reflectors which represent the orthogonal or unitary matrix P .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

gebrd_scratchpad_size

Computes size of scratchpad memory required for `gebrd` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type *T* the scratchpad memory to be passed to the `gebrd` (buffer or USM version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t gebrd_scratchpad_size(sycl::queue &queue,
        int64_t m,
        int64_t n, int64_t lda)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>gebrd</code> (buffer or USM version) function will be performed.
<code>m</code>	The number of rows in the matrix <i>A</i> ($0 \leq m$).
<code>n</code>	The number of columns in the matrix <i>A</i> ($0 \leq n$).
<code>lda</code>	The leading dimension of <i>a</i> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `gebrd` (buffer or USM version) function should be able to hold.

geinv_batch (Group Version)

Computes the batch of inverses of general matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the inverses A_i^{-1} of general matrices A_i , $i \in \{1 \dots \text{batch_size}\}$. The total number of problems to solve, `batch_size`, is the sum of the sizes of all the groups of parameters as provided by `group_sizesarray`.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event geinv_batch(sycl::queue &queue,
        int64_t *n,
        T **a,
        int64_t *lda,
        int64_t group_count,
        int64_t *group_sizes,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU
<code>std::complex<float></code>	CPU and GPU
<code>std::complex<double></code>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
n	Array of <code>group_count</code> parameters n_g specifying the order of the matrices A_i ($0 \leq n_g$) belonging to group g .
a	Array of <code>batch_size</code> pointers to input matrices A_i .
lda	Array of <code>group_count</code> parameters <code>lda_g</code> specifying the leading dimension of A_i ($n_g \leq \text{lda}_g$) belonging to group g .
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is the sum of all parameter group sizes.
scratchpad	Scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less then the value returned by geinv_batch_scratchpad_size (Group Version) .
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	Overwritten by the n_g -by- n_g matrices A_{i-1} .
---	--

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size must be not less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

[geinv_batch_scratchpad_size \(Group Version\)](#)

Computes the size of scratchpad memory required for the `geinv_batch` (Group Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type $\mathbb{T}$ the scratchpad memory to be passed to the [geinv_batch \(Group Version\)](#) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t geinv_batch_scratchpad_size(sycl::queue &queue,
        int64_t *n,
        int64_t *lda,
        int64_t group_count,
        int64_t *group_sizes)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
n	Array of <code>group_count</code> parameters n_g specifying the order of the matrices belonging to group g .
lda	Array of <code>group_count</code> parameters lda_g specifying the leading dimension of the matrices belonging to group g .
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is the sum of all parameter group sizes.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type $\mathbb{T}$ the scratchpad memory to be passed to the [geinv_batch \(Group Version\)](#) function must be able to hold.

gels (USM Version)

Finds the least squares solution of an overdetermined linear system or the minimum norm solution of an underdetermined system. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Uses the QR factorization to solve a linear system with full rank matrices. Each linear system is solved according to the following:

If $m \geq n$ and `trans = transpose::nontrans`, the least squares solution to the overdetermined system is computed: $\min ||A \cdot X - B||$

If $m \geq n$ and `trans = (transpose::trans or tranpose::conjtrans)`, the minimal norm solution to the underdetermined system is computed: $\min ||X||$ s.t. $A^H \cdot X = B$

$m < n$ is not supported.

On exit, the contents of `B` is overwritten with the solution vectors `X`.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event gels(sycl::queue &queue,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        int64_t nrhs,
        T *a,
        int64_t lda,
        T *b,
        int64_t ldb,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`gels` (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

`queue` Device queue where calculations will be performed.

trans	Operation applied to matrix A. Real precisions: <code>mkl::tranpose::nontrans</code> or <code>mkl::transpose::trans</code> Complex precisions: <code>mkl::tranpose::nontrans</code> or <code>mkl::transpose::conjtrans</code>
m	The number of rows of the matrix A ($m \geq n \geq 0$).
n	The number of columns of the matrix A ($m \geq n \geq 0$).
nrhs	The number of right-hand sides: the number of columns in B ($nrhs \geq 0$).
a	Pointer to the memory holding input matrix A.
lda	The leading dimension of a ($lda \geq \max(1, m)$).
b	Pointer to the memory holding input matrix B of right hand side vectors. B must be allocated enough space to store the solution vectors X, i.e. $\max(m, n)$ -by- $nrhs$ If <code>trans = transpose::nontrans</code> then B is m -by- $nrhs$, otherwise B is n -by- $nrhs$.
ldb	The leading dimension of b ($ldb \geq \max(1, \max(m, n))$).
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size must not be less than the value returned by the gels_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
a	Overwritten by the factorization data as follows: contains triangular matrix R obtained on the basis of A used in least squares computation.
b	Overwritten by least squares solutions of the problem.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size must not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

gels_scratchpad_size

Computes size of the scratchpad memory required for the `gels` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `gels` (USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t gels_scratchpad_size(sycl::queue &queue,
                                mkl::transpose trans,
                                int64_t m,
                                int64_t n,
                                int64_t nrhs,
                                int64_t lda,
                                int64_t ldb)
}
```

Input Parameters

queue	Device queue where calculations by the <code>gels</code> (USM version) function will be performed.
trans	If <code>trans=mkl::transpose::trans</code>, the routine finds the minimum norm solution the underdetermined system $A^T * X = B$. If <code>trans=mkl::transpose::conjtrans</code>, the routine finds the minimum norm solution the underdetermined system $A^H * X = B$. If <code>trans=mkl::transpose::nontrans</code>, the routine solves the least squares problem minimize $\ B - A * X \$.
m	The number of rows in the matrix <code>A</code> ($m \geq 0$).
n	The number of columns in the matrix <code>A</code> ($n \geq 0$).
nrhs	The number of right hand sides, i.e. the columns of <code>b</code> ($nrhs \geq 0$)
lda	The leading dimension of <code>a</code> ($lda \geq \max(1, m)$).
ldb	The leading dimension of <code>b</code> ($ldb \geq \max(1, \max(m, n))$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `gels` (USM version) function must be able to hold.

gels_batch (Buffer Strided Version)

Finds the least squares solutions for a batch of overdetermined linear systems. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Uses the QR factorization to solve a batch of linear systems with full rank matrices. Each linear system is solved according to the following:

If $m \geq n$ and `trans = transpose::nontrans` the least squares solution to the overdetermined system is computed: $\min ||A \cdot X - B||$

If $m \geq n$ and `trans = (transpose::trans or transpose::conjtrans)` the minimal norm solution to the underdetermined system is computed: $\min ||X||$ s.t. $A^H \cdot X = B$

On exit, the contents of `B` is overwritten with the solution vectors `X`.

Currently only the $m \geq n$ and `trans = transpose::nontrans` case is supported.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void gels_batch(sycl::queue &queue,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        int64_t nrhs,
        sycl::buffer<T> &a,
        int64_t lda,
        int64_t stride_a,
        sycl::buffer<T> &b,
        int64_t ldb,
        int64_t stride_b,
        int64_t batch_size,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	GPU
double	GPU
std::complex<float>	GPU
std::complex<double>	GPU

Input Parameters

queue	Device queue where calculations will be performed.
trans	Operation applied to matrices A_i . Real precisions: <code>mkl::transpose::nontrans</code> or <code>mkl::transpose::trans</code> Complex precisions: <code>mkl::transpose::nontrans</code> or <code>mkl::transpose::conjtrans</code>
m	The number of rows of the matrices A_i ($m \geq n \geq 0$).
n	The number of columns of the matrices A_i ($m \geq n \geq 0$).
nrhs	The number of right-hand sides: the number of columns in B_i ($nrhs \geq 0$).
a	Contains batch_size m-by-n matrices A_i
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).
b	Contains the matrices B_i of right hand side vectors. Each matrix B_i must be allocated enough space to store the solution vectors X_i , i.e. $\max(m, n)$ -by- $nrhs$ If <code>trans = transpose::nontrans</code> , then B_i is m-by- $nrhs$; otherwise, B_i is n-by- $nrhs$.
ldb	The leading dimensions of B_i ($ldb \geq \max(1, \max(m, n))$).
stride_b	The stride between the beginnings of matrices B_i inside the batch array b ($stride_b \geq \max(1, ldb * nrhs)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by stride version of gels_batch_scratchpad_size (Strided Version) .

Output Parameters

a	Overwritten by the factorization data as follows: Contains triangular matrix R obtained on the basis of A_i used in least squares computation. The <code>tau</code> vectors are not recorded.
b	Overwritten by least squares solutions of the batch of problems.

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then A_i does not have full rank, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these matrices using the <code>infos()</code> method of the exception object.

gels_batch (USM Strided Version)

Finds the least squares solutions for a batch of overdetermined linear systems. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Uses the QR factorization to solve a batch of linear systems with full rank matrices. Each linear system is solved according to the following:

If $m \geq n$ and `trans = transpose::nontrans` the least squares solution to the overdetermined system is computed: $\min ||A \cdot X - B||$

If $m \geq n$ and `trans = (transpose::trans or transpose::conjtrans)` the minimal norm solution to the underdetermined system is computed: $\min ||X||$ s.t. $A^H \cdot X = B$

On exit, the contents of `B` is overwritten with the solution vectors `X`.

Currently only $m \geq n$ and `trans = transpose::nontrans` case is supported.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event gels_batch(sycl::queue &queue,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        int64_t nrhs,
        T *a,
```

```
int64_t lda,
int64_t stride_a,
T *b,
int64_t ldb,
int64_t stride_b,
int64_t batch_size,
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {}))
}
```

This function supports the following precisions and devices:

T	Devices supported
float	GPU
double	GPU
std::complex<float>	GPU
std::complex<double>	GPU

Input Parameters

queue	Device queue where calculations will be performed.
trans	Operation applied to matrices A_i . Real precisions: <code>mkl::transpose::nontrans</code> or <code>mkl::transpose::trans</code> Complex precisions: <code>mkl::transpose::nontrans</code> or <code>mkl::transpose::conjtrans</code>
m	The number of rows of the matrices A_i ($m \geq n \geq 0$).
n	The number of columns of the matrices A_i ($m \geq n \geq 0$).
nrhs	The number of right-hand sides: the number of columns in B_i ($nrhs \geq 0$).
a	Contains <code>batch_size</code> m-by-n matrices A_i
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
b	Contains the matrices B_i of right hand side vectors. Each matrix B_i must be allocated enough space to store the solution vectors X_i , i.e. $\max(m, n)$ -by- <code>nrhs</code> If <code>trans = transpose::nontrans</code> then B_i is m-by- <code>nrhs</code> , otherwise B_i is n-by- <code>nrhs</code> .
ldb	The leading dimensions of B_i ($ldb \geq \max(1, \max(m, n))$).
stride_b	The stride between the beginnings of matrices B_i inside the batch array <code>b</code> ($stride_b \geq \max(1, ldb * nrhs)$).
batch_size	The number of problems in a batch (<code>batch_size</code> ≥ 0).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.

`scratchpad_size` Size of scratchpad memory as a number of floating point elements of type `T`. Size should not be less than the value returned by stride version of [gels_batch_scratchpad_size \(Strided Version\)](#).

`events` List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

`a` Overwritten by the factorization data as follows: Contains triangular matrix R obtained on the basis of A_i used in least squares computation. The `tau` vectors are not recorded.

`b` Overwritten by least squares solutions of the batch of problems.

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then A_i does not have full rank, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

[gels_batch_scratchpad_size \(Strided Version\)](#)

Computes size of scratchpad memory required for `gels_batch` (Strided Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `gels_batch` (Strided Version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t gels_batch_scratchpad_size(sycl::queue &queue,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        int64_t nrhs,
        int64_t lda,
        int64_t stride_a,
        int64_t ldb,
        int64_t stride_b,
        int64_t batch_size)
}
```

Input Parameters

queue	Device queue where calculations will be performed.
trans	Operation assumed to be done on input matrices A_i . Only <code>trans = mkl::transpose::nontrans</code> case is currently supported.
m	The number of rows of the matrices A_i and B_i ($m \geq 0$).
n	The number of columns of the matrices A_i ($n \geq 0$).
nrhs	The number of right-hand sides: the number of columns in B_i ($nrhs \geq 0$).
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
ldb	The leading dimensions of B_i ($ldb \geq \max(1, \max(m, n))$).
stride_b	The stride between the beginnings of matrices B_i inside the batch array <code>b</code> ($stride_b \geq \max(1, ldb * nrhs)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `gels_batch` (Strided Version) function must be able to hold.

`gels_batch` (Group Version)

Finds the least squares solutions for a batch of overdetermined linear systems. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Uses the QR factorization to solve a grouped batch of linear systems with full rank matrices. Each linear system is solved according to the following:

If $m \geq n$ and `trans = transpose::nontrans` the least squares solution to the overdetermined system is computed: $\min ||A \cdot X - B||$

If $m \geq n$ and `trans = (transpose::trans or transpose::conjtrans)` the minimal norm solution to the underdetermined system is computed: $\min ||X||$ s.t. $A^H \cdot X = B$

On exit, the contents of `B` is overwritten with the solution vectors `X`.

Currently only $m \geq n$ case is supported.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event gels_batch(sycl::queue &queue,
        mkl::transpose *trans,
        int64_t *m,
        int64_t *n,
        int64_t *nrhs,
        T **a,
        int64_t *lda,
        T **b,
        int64_t *ldb,
        int64_t group_count,
        int64_t *group_sizes,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

`queue` Device queue where calculations will be performed.

trans	Array of group_count parameters trans _g . Operation applied to matrices A _i in group g . Real precisions: <code>mkl::transpose::nontrans</code> or <code>mkl::transpose::trans</code> Complex precisions: <code>mkl::transpose::nontrans</code> or <code>mkl::transpose::conjtrans</code>
m	Array of group_count parameters m _g The number of rows of the matrices A _i in group g ($m \geq n \geq 0$).
n	Array of group_count parameters n _g The number of columns of the matrices A _i in group g ($m \geq n \geq 0$).
nrhs	Array of group_count parameters nrhs _g The number of right-hand sides: the number of columns in B _i in group g (nrhs ≥ 0).
a	Array of batch_size pointers. Contains batch_size m-by-n matrices A _i
lda	Array of group_count parameters lda _g The leading dimension of A _i in group g ($lda \geq \max(1, m)$).
b	Array of batch_size pointers Contains the matrices B _i of right hand side vectors in group g . Each matrix B _i must be allocated enough space to store the solution vectors X _i , i.e. $\max(m, n)$ -by-nrhs If trans = <code>transpose::nontrans</code> then B _i is m-by-nrhs, otherwise B _i is n-by-nrhs.
ldb	Array of group_count parameters ldb _g The leading dimensions of B _i in group g ($ldb \geq \max(1, \max(m, n))$).
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of group_count integers group_sizes _g Array element with index _g specifies the number of problems to solve for each group of parameters _g . So the total number of problems to solve, batch_size, is a sum of all parameter group sizes.
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size must not be less than the value returned by gels_batch_scratchpad_size (Group Version) .
events	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
a	Overwritten by the factorization data as follows: Contains triangular matrix R obtained on the basis of A _i used in least squares computation. The tau vectors are not recorded.
b	Overwritten by least squares solutions of the batch of problems.
Exceptions	

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then A_i does not have full rank, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

gels_batch_scratchpad_size (Group Version)

Computes size of scratchpad memory required for `gels_batch` (Group Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `gels_batch` (Group Version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t gels_batch_scratchpad_size(sycl::queue &queue,
        mkl::transpose *trans,
        int64_t *m,
        int64_t *n,
        int64_t *nrhs,
        int64_t *lda,
        int64_t *ldb,
        int64_t group_count,
        int64_t *group_sizes)
}
```

Input Parameters

queue	Device queue where calculations will be performed.
trans	Array of <code>group_count</code> parameters <code>trans_g</code> indicating the form of equations for the group g . Operation assumed to be done on input matrices A_i . Only the <code>trans = mkl::transpose::nontrans</code> case is currently supported.
m	Array of <code>group_count</code> parameters <code>m_g</code> specifying the number of rows of the matrices A_i and B_i belonging to group g ($m \geq 0$).
n	Array of <code>group_count</code> parameters <code>n_g</code> specifying the number of columns of the matrices A_i belonging in group g ($n \geq 0$).
nrhs	Array of <code>group_count</code> parameters <code>nrhs_g</code> specifying the number of right-hand sides: the number of columns in B_i belonging in group g (<code>nrhs</code> ≥ 0).
lda	Array of <code>group_count</code> parameters specifying the leading dimensions of A_i belonging to group g (<code>lda</code> $\geq \max(1, m)$).
ldb	Array of <code>group_count</code> parameters specifying the leading dimensions of B_i belonging to group g (<code>ldb</code> $\geq \max(1, \max(m, n))$).
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each group of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by using the <code>info()</code> method of the exception object.

Return Values

The number of elements of type **T** the scratchpad memory to be passed to the `gels_batch` (Group Version) function must be able to hold.

geqrf

Computes the QR factorization of a general m -by- n matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the QR factorization of a general m -by- n matrix A . No pivoting is performed.

The routine does not form the matrix Q explicitly. Instead, Q is represented as a product of $\min(m, n)$ elementary reflectors. Routines are provided to work with Q in this representation.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void geqrf(sycl::queue &queue,
               int64_t m,
               int64_t n,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<T> &tau,
               sycl::buffer<T> &scratchpad,
               int64_t scratchpad_size)
}
```

`geqrf` supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*
std::complex<float>	CPU and GPU*
std::complex<double>	CPU and GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in A ($0 \leq n$).
a	Buffer holding input matrix A . The size of a must be at least $lda * n$.
lda	The leading dimension of a ; at least $\max(1, m)$.
scratchpad	Buffer holding scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by the geqrf_scratchpad_size function.

Output Parameters

a	Overwritten by the factorization data as follows: The elements on and above the diagonal of the array contain the $\min(m, n)$ -by- n upper trapezoidal matrix R (R is upper triangular if $m \geq n$); the elements below the diagonal, with the array τ , present the orthogonal matrix Q as a product of $\min(m, n)$ elementary reflectors.
tau	Array, size at least $\max(1, \min(m, n))$. Contains scalars that define elementary reflectors for the matrix Q in its decomposition in a product of elementary reflectors.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

geqrf (USM Version)

Computes the QR factorization of a general *m*-by-*n* matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the QR factorization of a general *m*-by-*n* matrix *A*. No pivoting is performed.

The routine does not form the matrix *Q* explicitly. Instead, *Q* is represented as a product of $\min(m, n)$ elementary reflectors. Routines are provided to work with *Q* in this representation.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event geqrf(sycl::queue &queue,
        int64_t m,
        int64_t n,
        T *a,
        int64_t lda,
        T *tau,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`geqrf` (USM version) supports the following precisions and devices:

T	Devices supported
<code>float</code>	CPU and GPU*
<code>double</code>	CPU and GPU*
<code>std::complex<float></code>	CPU and GPU*

T	Devices supported
<code>std::complex<double></code>	CPU and GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>m</code>	The number of rows in the matrix A ($0 \leq m$).
<code>n</code>	The number of columns in the matrix A ($0 \leq n$).
<code>a</code>	Pointer to the memory holding input matrix A . The array must have size at least $lda \cdot n$.
<code>lda</code>	The leading dimension of a , at least $\max(1, m)$.
<code>scratchpad</code>	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the geqrf_scratchpad_size function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

<code>a</code>	Overwritten by the factorization data as follows: The elements on and above the diagonal of the array contain the $\min(m, n)$ -by- n upper trapezoidal matrix R (R is upper triangular if $m \geq n$); the elements below the diagonal, with the array <code>tau</code> , present the orthogonal matrix Q as a product of $\min(m, n)$ elementary reflectors.
<code>tau</code>	Array, size at least $\min(m, n)$. Contains scalars that define elementary reflectors for the matrix Q in its decomposition in a product of elementary reflectors.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

geqrf_batch (Buffer Strided Version)

Computes the batch of QR factorizations of a batch of general m -by- n matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the $Q_i R_i$ factorizations of a general m -by- n matrices A_i . No pivoting is performed.

The routine does not form the matrix Q_i explicitly. Instead, Q_i is represented as a product of $\min(m, n)$ elementary reflectors. Routines are provided to work with Q_i in this representation.

API

Syntax

```

namespace oneapi::mkl::lapack {
    void geqrf_batch(sycl::queue &queue,
        int64_t m,
        int64_t n,
        sycl::buffer<T> &a,
        int64_t lda,
        int64_t stride_a,
        sycl::buffer<T> &tau,
        int64_t stride_tau,
        int64_t batch_size,
        sycl::buffer<T> &scratchpad, int64_t scratchpad_size)
}

```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*
std::complex<float>	CPU and GPU*
std::complex<double>	CPU and GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i ($m \geq 0$).
n	The number of columns in the matrices A_i ($n \geq 0$).

<code>a</code>	Array holding input matrices A_i .
<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, m)$).
<code>stride_a</code>	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
<code>stride_tau</code>	The stride between the beginnings of arrays τ_{τ_i} inside the array <code>tau</code> ($stride_tau \geq \max(1, \min(m, n))$).
<code>batch_size</code>	The number of problems in a batch ($batch_size \geq 0$).
<code>scratchpad</code>	Scratchpad memory to be used by routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by geqrf_batch_scratchpad_size (Strided Version) .

Output Parameters

<code>a</code>	Overwritten by the factorization data as follows: The elements on and above the diagonal of the array contain the $\min(m, n)$ -by- n upper trapezoidal matrices R_i (R_i is upper triangular if $m \geq n$); the elements below the diagonal, with the array τ_{τ_i} , present the orthogonal matrix Q_i as a product of $\min(m, n)$ elementary reflectors.
<code>tau</code>	Array to store batch of τ_{τ_i} , each of size $\min(m, n)$, containing scalars that define elementary reflectors for the matrices Q_i in its decomposition in a product of elementary reflectors.

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less than the value returned by the <code>detail()</code> method of the exception object.

geqrf_batch (Group Version)

Computes the batch of QR factorizations of a batch of general m -by- n matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the $Q_i R_i$ factorizations of a general m -by- n matrix A_i , $i \in \{1 \dots \text{batch_size}\}$, where batch_size is a sum of all parameter group sizes as provided with the `group_sizes` array. No pivoting is performed during factorization.

The routine does not form the matrix Q_i explicitly. Instead, Q_i is represented as a product of $\min(m, n)$ elementary reflectors. Routines are provided to work with Q_i in this representation.

Total number of problems to solve, batch_size , is a sum of sizes of all of the groups of parameters as provided by `group_sizes` array.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event geqrf_batch(sycl::queue &queue,
        int64_t *m,
        int64_t *n,
        T **a,
        int64_t *lda,
        T **tau,
        int64_t group_count,
        int64_t *group_sizes,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*
std::complex<float>	CPU and GPU*
std::complex<double>	CPU and GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	Array of <code>group_count</code> parameters m_g parameters. Each of m_g specifies the number of rows in the matrices A_i from array a , belonging to group g ($m_g \geq 0$).
n	Array of <code>group_count</code> parameters n_g parameters. Each of n_g specifies the number of columns in the matrices A_i from array a , belonging to group g ($n_g \geq 0$).
a	Array of <code>batch_size</code> pointers to input matrices A_i , each being of size <code>lda_g*n_g</code> (g is an index of group to which A_i belongs).

<code>lda</code>	Array of <code>group_count</code> <code>lda_g</code> parameters, each representing the leading dimensions of input matrices A_i , from array <code>a</code> , belonging to group <code>g</code> ($lda_g \geq \max(1, m_g)$).
<code>group_count</code>	Specifies the number of groups of parameters. Must be at least 0.
<code>group_sizes</code>	Array of <code>group_count</code> integers. Array element with index <code>g</code> specifies the number of problems to solve for each of the groups of parameters <code>g</code> . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.
<code>scratchpad</code>	Scratchpad memory to be used by routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less then the value returned by geqrf_batch_scratchpad_size (Group Version) .
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

<code>a</code>	Matrices pointed to by array <code>a</code> are overwritten by the factorization data as follows: The elements on and above the diagonal of A_i contain the $\min(m_g, n_g)$-by-n_g upper trapezoidal matrices R_i (R_i is upper triangular if $m_g \geq n_g$); the elements below the diagonal, with the array <code>tau_i</code>, present the orthogonal matrix Q_i as a product of $\min(m_g, n_g)$ elementary reflectors. Here, <code>g</code> is an index of parameters group corresponding to <code>i</code>-th decomposition.
<code>tau</code>	Array of pointers to store <code>tau_i</code>, each of size $\min(m_g, n_g)$, containing scalars that define elementary reflectors for the matrices Q_i in its decomposition in a product of elementary reflectors. Here, <code>g</code> is an index of parameters group corresponding to <code>i</code>-th decomposition.

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

geqrf_batch (USM Strided Version)

Computes the batch of QR factorizations of a batch of general m -by- n matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the $Q_i R_i$ factorizations of a general m -by- n matrix A_i . No pivoting is performed.

The routine does not form the matrix Q_i explicitly. Instead, Q_i is represented as a product of $\min(m, n)$ elementary reflectors. Routines are provided to work with Q_i in this representation.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event geqrf_batch(sycl::queue &queue,
        int64_t m,
        int64_t n,
        T *a,
        int64_t lda,
        int64_t stride_a,
        T *tau,
        int64_t stride_tau,
        int64_t batch_size,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*
std::complex<float>	CPU and GPU*
std::complex<double>	CPU and GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i ($m \geq 0$).
n	The number of columns in the matrices A_i ($n \geq 0$).
a	Array holding input matrices A_i .

<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, m)$).
<code>stride_a</code>	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
<code>stride_tau</code>	The stride between the beginnings of arrays τ_{τ_i} inside the array <code>tau</code> ($stride_tau \geq \max(1, \min(m, n))$).
<code>batch_size</code>	The number of problems in a batch ($batch_size \geq 0$).
<code>scratchpad</code>	Scratchpad memory to be used by routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by geqrf_batch_scratchpad_size (Strided Version) .
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
<code>a</code>	Overwritten by the factorization data as follows: The elements on and above the diagonal of the array contain the $\min(m, n)$ -by- n upper trapezoidal matrices R_i (R_i is upper triangular if $m \geq n$); the elements below the diagonal, with the array τ_{τ_i} , present the orthogonal matrix Q_i as a product of $\min(m, n)$ elementary reflectors.
<code>tau</code>	Array to store batch of τ_{τ_i} , each of size $\min(m, n)$, containing scalars that define elementary reflectors for the matrices Q_i in its decomposition in a product of elementary reflectors.

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

geqrf_batch_scratchpad_size (Group Version)

Computes size of scratchpad memory required for `geqrf_batch` (Group Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type T the scratchpad memory to be passed to the [geqrf_batch](#) (Group Version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t geqrf_batch_scratchpad_size(sycl::queue &queue,
        int64_t *m,
        int64_t *n,
        int64_t *lda,
        int64_t group_count,
        int64_t *group_sizes)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
m	Array of <code>group_count</code> parameters m_g. Each m_g specifies the number of rows in the matrices A_i belonging to group g ($m_g \geq 0$).
n	Array of <code>group_count</code> parameters n_g. Each n_g specifies the number of columns in the matrices A_i belonging to group g ($n_g \geq 0$).
lda	Array of <code>group_count</code> parameters lda_g , each representing the leading dimension of input matrices belonging to group g ($lda_g \geq \max(1, m_g)$).
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the [geqrf_batch \(Group Version\)](#) function should be able to hold.

geqrf_batch_scratchpad_size (Strided Version)

Computes size of scratchpad memory required for `geqrf_batch (Strided Version)` function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `geqrf_batch (Strided Version)` function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    geqrf_batch_scratchpad_size(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t lda,
        int64_t stride_a,
        int64_t stride_tau,
        int64_t batch_size)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>m</code>	The number of rows in the matrices A_i ($m \geq 0$).
<code>n</code>	The number of columns in the matrices A_i ($n \geq 0$).
<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, m)$).
<code>stride_a</code>	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
<code>stride_tau</code>	The stride between the beginnings of arrays τ_i inside the array <code>tau</code> ($stride_tau \geq \max(1, \min(m, n))$).
<code>batch_size</code>	The number of problems in a batch ($batch_size \geq 0$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `geqrf_batch` (Strided Version) function should be able to hold.

geqrf_scratchpad_size

Computes size of scratchpad memory required for `geqrf` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `geqrf` (buffer or USM version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t geqrf_scratchpad_size(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t lda)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>geqrf</code> (buffer or USM version) function will be performed.
<code>m</code>	The number of rows in the matrix <code>A</code> ($0 \leq m$).
<code>n</code>	The number of columns in the matrix <code>A</code> ($0 \leq n$).
<code>lda</code>	The leading dimension of <code>a</code> .

Exceptions

Exception	Description
<code>mk1::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `geqrf` (buffer or USM version) function should be able to hold.

gerqf

Computes the RQ factorization of a general m-by-n matrix. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the RQ factorization of a general m-by-n matrix A. No pivoting is performed.

The routine does not form the matrix Q explicitly. Instead, Q is represented as a product of $\min(m, n)$ elementary reflectors. Routines are provided to work with Q in this representation.

NOTE This routine supports the Progress Routine feature.

API

Syntax

```
namespace oneapi::mk1::lapack {
    void gerqf(sycl::queue &queue,
               int64_t m,
               int64_t n,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<T> &tau,
               sycl::buffer<T> &scratchpad,
               int64_t scratchpad_size)
}
```

`gerqf` supports the following precisions and devices:

T	Devices supported
<code>float</code>	CPU
<code>double</code>	CPU
<code>std::complex<float></code>	CPU
<code>std::complex<double></code>	CPU

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
a	Buffer holding input matrix A . The buffer must have size at least $lda \cdot n$.
lda	The leading dimension of a , at least $\max(1, m)$.
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by the gerqf_scratchpad_size function.

Output Parameters

a	Overwritten by the factorization data as follows: if $m \leq n$, the upper triangle of the subarray $a(1:m, n-m+1:n)$ contains the m-by-m upper triangular matrix R; if $m \geq n$, the elements on and above the $(m-n)$-th subdiagonal contain the m-by-n upper trapezoidal matrix R In both cases, the remaining elements, with the array τ, represent the orthogonal/unitary matrix Q as a product of $\min(m, n)$ elementary reflectors.
tau	Array, size at least $\min(m, n)$. Contains scalars that define elementary reflectors for the matrix Q in its decomposition in a product of elementary reflectors.

Exceptions

Exception	Description
<code>mk1::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

gerqf (USM Version)

Computes the RQ factorization of a general m -by- n matrix A . This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the RQ factorization of a general m -by- n matrix A . No pivoting is performed.

The routine does not form the matrix Q explicitly. Instead, Q is represented as a product of $\min(m, n)$ elementary reflectors. Routines are provided to work with Q in this representation.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event gerqf(sycl::queue &queue,
        int64_t m,
        int64_t n,
        T *a,
        int64_t lda,
        T *tau,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`gerqf` (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU
double	CPU
std::complex<float>	CPU
std::complex<double>	CPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>m</code>	The number of rows in the matrix A ($0 \leq m$).
<code>n</code>	The number of columns in the matrix A ($0 \leq n$).
<code>a</code>	Pointer to the memory holding input matrix A . The memory must have size at least $lda * n$.
<code>lda</code>	The leading dimension of a , at least $\max(1, m)$.
<code>scratchpad</code>	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by the gerqf_scratchpad_size function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	Overwritten by the factorization data as follows: if $m \leq n$, the upper triangle of the subarray <code>a(1:m, n-m+1:n)</code> contains the m-by-m upper triangular matrix R; if $m \geq n$, the elements on and above the $(m-n)$-th subdiagonal contain the m-by-n upper trapezoidal matrix R In both cases, the remaining elements, with the array <code>tau</code>, represent the orthogonal/unitary matrix Q as a product of $\min(m, n)$ elementary reflectors.
tau	Array, size at least $\min(m, n)$. Contains scalars that define elementary reflectors for the matrix Q in its decomposition in a product of elementary reflectors.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>get_info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>get_detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>get_detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

gerqf_scratchpad_size

Computes size of scratchpad memory required for `gerqf` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `gerqf` (buffer or USM version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template <typename T>
    int64_t gerqf_scratchpad_size(sycl::queue &queue,
```

```
int64_t m,
int64_t n,
int64_t lda)
}
```

Input Parameters

queue	Device queue where calculations by the <code>gerqf</code> (buffer or USM version) function will be performed.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
lda	The leading dimension of a ; at least $\max(1, m)$.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>get_info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `gerqf` (buffer or USM version) function should be able to hold.

gesv (USM Version)

Computes the solution to the system of linear equations with a square coefficient matrix and multiple right-hand sides. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X the system of linear equations $A \cdot X = B$, where A is an n -by- n matrix, the columns of matrix B are individual right-hand sides, and the columns of X are the corresponding solutions.

The LU decomposition with partial pivoting and row interchanges is used to factor A as $A = P \cdot L \cdot U$, where P is a permutation matrix, L is unit lower triangular, and U is upper triangular. The factored form of A is then used to solve the system of equations $A \cdot X = B$.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event gesv(sycl::queue &queue,
        int64_t n,
        int64_t nrhs,
        T *a,
        int64_t lda,
```

```
int64_t *ipiv,
T *b,
int64_t ldb,
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {}))
}
```

gesv (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU, GPU
double	CPU, GPU
std::complex<float>	CPU, GPU
std::complex<double>	CPU, GPU

Input Parameters

queue	Device queue where calculations will be performed.
n	The order of the matrix A; ($0 \leq n$).
nrhs	The number of right-hand sides ($0 \leq \text{nrhs}$).
a	Pointer to the n-by-n coefficient matrix A.
lda	The leading dimension of a.
b	Pointer to the n-by-nrhs matrix of right hand side matrix B
ldb	The leading dimension of b.
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by the gesv_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	Overwritten by the factors L and U from the factorization of $A = P * L * U$; the unit diagonal elements of L are not stored.
b	The buffer b is overwritten by the solution matrix X.
ipiv	Array, size at least $\max(1, n)$. The ipiv array, as returned by getrf (USM Version) .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, the <i>i</i>-th diagonal element of U is zero, and the solve could not be completed. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

gesv_scratchpad_size

Computes size of scratchpad memory required for `gesv` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `gesv` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t gesv_scratchpad_size(sycl::queue &queue,
        mkl::transpose trans,
        int64_t n,
        int64_t nrhs,
        int64_t lda,
        int64_t ldb)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>gesv</code> (buffer or USM version) function will be performed.
<code>n</code>	The order of the matrix A and the number of rows in matrix B ($0 \leq n$).

<code>nrhs</code>	The number of right-hand sides ($0 \leq \text{nrhs}$).
<code>lda</code>	The leading dimension of <code>a</code> .
<code>ldb</code>	The leading dimension of <code>b</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `gesv` (buffer or USM version) function must be able to hold.

gesvd

Computes the singular value decomposition of a general rectangular matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the singular value decomposition (SVD) of a real/complex m -by- n matrix A , optionally computing the left and/or right singular vectors. The SVD is written as:

- $A = U \Sigma V^T$ for real routines
- $A = U \Sigma V^H$ for complex routines

where Σ is an m -by- n diagonal matrix, U is an m -by- m orthogonal/unitary matrix, and V is an n -by- n orthogonal/unitary matrix. The diagonal elements of Σ are the singular values of A ; they are real and non-negative, and are returned in descending order. The first $\min(m, n)$ columns of U and V are the left and right singular vectors of A .

API

Syntax

```
namespace oneapi::mkl::lapack {
    void gesvd(sycl::queue &queue,
               mkl::jobsvd jobu,
               mkl::jobsvd jobvt,
               int64_t m,
               int64_t n,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<RealT> &s,
               sycl::buffer<T> &u,
               int64_t ldu,
               sycl::buffer<T> &vt,
```

```
int64_t ldvt,
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}
```

gesvd supports the following precision and devices.

T	Devices Supported
float	CPU
double	CPU
std::complex<float>	CPU
std::complex<double>	CPU

Input Parameters

queue	Device queue where calculations will be performed.
jobu	Must be <code>jobsvd::vectors</code>, <code>job::somevec</code>, <code>jobsvd::vectorsina</code>, or <code>job::novec</code>. Specifies options for computing all or part of the matrix U. If <code>jobu = jobsvd::vectors</code>, all m columns of U are returned in the buffer <code>u</code>; if <code>jobu = job::somevec</code>, the first $\min(m, n)$ columns of U (the left singular vectors) are returned in the buffer <code>u</code>; if <code>jobu = jobsvd::vectorsina</code>, the first $\min(m, n)$ columns of U (the left singular vectors) are overwritten on the buffer <code>a</code>; if <code>jobu = job::novec</code>, no columns of U (no left singular vectors) are computed.
jobvt	Must be <code>jobsvd::vectors</code>, <code>job::somevec</code>, <code>jobsvd::vectorsina</code>, or <code>job::novec</code>. Specifies options for computing all or part of the matrix VT/VH. If <code>jobvt = jobsvd::vectors</code>, all n columns of VT/VH are returned in the buffer <code>vt</code>; if <code>jobvt = job::somevec</code>, the first $\min(m, n)$ columns of VT/VH (the left singular vectors) are returned in the buffer <code>vt</code>; if <code>jobvt = jobsvd::vectorsina</code>, the first $\min(m, n)$ columns of VT/VH (the left singular vectors) are overwritten on the buffer <code>a</code>; if <code>jobvt = job::novec</code>, no columns of VT/VH (no left singular vectors) are computed. <code>jobvt</code> and <code>jobu</code> cannot both be <code>jobsvd::vectorsina</code>.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
a	Buffer holding input matrix A . The buffer must have size at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> .
ldu	The leading dimension of <code>u</code> .
ldvt	The leading dimension of <code>vt</code> .

scratchpad Pointer to scratchpad memory to be used by the routine for storing intermediate results.

scratchpad_size Size of scratchpad memory as a number of floating point elements of type `T`. Size should not be less than the value returned by the [gesvd_scratchpad_size](#) function.

Output Parameters

a On exit,

If `jobu = jobsvd::vectorsina`, `a` is overwritten with the first $\min(m, n)$ columns of U (the left singular vectors stored columnwise);

If `jobvt = jobsvd::vectorsina`, `a` is overwritten with the first $\min(m, n)$ rows of V^T/V^H (the right singular vectors stored rowwise);

If `jobu  $\neq$  jobsvd::vectorsina` and `jobvt  $\neq$  jobsvd::vectorsina`, the contents of `a` are destroyed.

s Array containing the singular values, size at least $\max(1, \min(m, n))$. Contains the singular values of A sorted so that $s(i) \geq s(i+1)$.

u Buffer containing output matrix U . The size of the buffer must be at least `ldu*m` if `jobu = jobsvd::vectors`, and at least `ldu*min(m, n)` if `jobu = job::somevec`.

If `jobu = jobsvd::vectors`, `u` contains the m -by- m orthogonal/unitary matrix U .

If `jobu = job::somevec`, `u` contains the first $\min(m, n)$ columns of U (the left singular vectors stored column-wise).

If `jobu = job::novec` or `jobsvd::vectorsina`, `u` is not referenced.

vt Array containing V^T ; the size of `vt` must be at least `ldvt*n`.

If `jobvt = jobsvd::vectors`, `vt` contains the n -by- n orthogonal/unitary matrix V^T/V^H .

If `jobvt = job::somevec`, `vt` contains the first $\min(m, n)$ rows of V^T/V^H (the right singular vectors stored row-wise).

If `jobvt = job::novec` or `jobsvd::vectorsina`, `vt` is not referenced.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info = i</code>, then i specifies how many superdiagonals of the intermediate bidiagonal form B did not converge to zero, and

Exception	Description
	<code>scratchpad(2:min(m,n))</code> contains the unconverged superdiagonal elements of an upper bidiagonal matrix B whose diagonal is in s (not necessarily sorted). B satisfies $A = U*B*V^T$, so it has the same singular values as A, and singular vectors related by U and V^T. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

gesvd (USM Version)

Computes the singular value decomposition of a general rectangular matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the singular value decomposition (SVD) of a real/complex m -by- n matrix A , optionally computing the left and/or right singular vectors. The SVD is written as:

- $A = U*\Sigma*V^T$ for real routines
- $A = U*\Sigma*V^H$ for complex routines

where Σ is an m -by- n diagonal matrix, U is an m -by- m orthogonal/unitary matrix, and V is an n -by- n orthogonal/unitary matrix. The diagonal elements of Σ are the singular values of A ; they are real and non-negative, and are returned in descending order. The first $\min(m, n)$ columns of U and V are the left and right singular vectors of A .

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event gesvd(sycl::queue &queue,
        mkl::jobsvd jobu,
        mkl::jobsvd jobvt,
        int64_t m,
        int64_t n,
        T *a,
        int64_t lda,
        RealT *s,
        T *u,
        int64_t ldu,
        T *vt,
        int64_t ldvt,
        T *scratchpad,
```

```
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
}
```

gesvd (USM version) supports the following precision and devices.

T	Devices Supported
float	CPU
double	CPU
std::complex<float>	CPU
std::complex<double>	CPU

Input Parameters

queue	Device queue where calculations will be performed.
jobu	Must be <code>jobsvd::vectors</code>, <code>job::somevec</code>, <code>jobsvd::vectorsina</code>, or <code>job::novec</code>. Specifies options for computing all or part of the matrix U. If <code>jobu = jobsvd::vectors</code>, all m columns of U are returned in the array <code>u</code>; if <code>jobu = job::somevec</code>, the first $\min(m, n)$ columns of U (the left singular vectors) are returned in the array <code>u</code>; if <code>jobu = jobsvd::vectorsina</code>, the first $\min(m, n)$ columns of U (the left singular vectors) are overwritten on the array <code>a</code>; if <code>jobu = job::novec</code>, no columns of U (no left singular vectors) are computed.
jobvt	Must be <code>jobsvd::vectors</code>, <code>job::somevec</code>, <code>jobsvd::vectorsina</code>, or <code>job::novec</code>. Specifies options for computing all or part of the matrix VT/VH. If <code>jobvt = jobsvd::vectors</code>, all n columns of VT/VH are returned in the array <code>vt</code>; if <code>jobvt = job::somevec</code>, the first $\min(m, n)$ columns of VT/VH (the left singular vectors) are returned in the array <code>vt</code>; if <code>jobvt = jobsvd::vectorsina</code>, the first $\min(m, n)$ columns of VT/VH (the left singular vectors) are overwritten on the array <code>a</code>; if <code>jobvt = job::novec</code>, no columns of VT/VH (no left singular vectors) are computed. <code>jobvt</code> and <code>jobu</code> cannot both be <code>jobsvd::vectorsina</code>.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
a	Pointer to the memory holding the input matrix A . The memory must have size at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> .
ldu	The leading dimension of <code>u</code> .
ldvt	The leading dimension of <code>vt</code> .

scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the gesvd_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
a	On exit, If <code>jobu = jobsvd::vectorsina</code>, <code>a</code> is overwritten with the first $\min(m, n)$ columns of U (the left singular vectors stored columnwise); If <code>jobvt = jobsvd::vectorsina</code>, <code>a</code> is overwritten with the first $\min(m, n)$ rows of V^T/V^H (the right singular vectors stored rowwise); If <code>jobu $\neq$ jobsvd::vectorsina</code> and <code>jobvt $\neq$ jobsvd::vectorsina</code>, the contents of <code>a</code> are destroyed.
s	Array containing the singular values, size at least $\max(1, \min(m, n))$. Contains the singular values of A sorted so that $s(i) \geq s(i+1)$.
u	Array containing U; the size of the array must be at least <code>ldu*m</code> if <code>jobu = jobsvd::vectors</code>, and at least <code>ldu*min(m, n)</code> if <code>jobu = job::somevec</code>. If <code>jobu = jobsvd::vectors</code>, <code>u</code> contains the m-by-m orthogonal/unitary matrix U. If <code>jobu = job::somevec</code>, <code>u</code> contains the first $\min(m, n)$ columns of U (the left singular vectors stored column-wise). If <code>jobu = job::novec</code> or <code>jobsvd::vectorsina</code>, <code>u</code> is not referenced.
vt	Array containing V^T; the size of <code>vt</code> must be at least <code>ldvt*n</code>. If <code>jobvt = jobsvd::vectors</code>, <code>vt</code> contains the n-by-n orthogonal/unitary matrix V^T/V^H. If <code>jobvt = job::somevec</code>, <code>vt</code> contains the first $\min(m, n)$ rows of V^T/V^H (the right singular vectors stored row-wise). If <code>jobvt = job::novec</code> or <code>jobsvd::vectorsina</code>, <code>vt</code> is not referenced.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>get_info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value.

Exception	Description
	If <code>info = i</code>, then <code>i</code> specifies how many superdiagonals of the intermediate bidiagonal form <code>B</code> did not converge to zero, and <code>scratchpad(2:min(m,n))</code> contains the unconverged superdiagonal elements of an upper bidiagonal matrix <code>B</code> whose diagonal is in <code>s</code> (not necessarily sorted). <code>B</code> satisfies $A = U * B * V^T$, so it has the same singular values as <code>A</code>, and singular vectors related by <code>U</code> and <code>V^T</code>. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>get_detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>get_detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

gesvd_scratchpad_size

Computes size of scratchpad memory required for `gesvd` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `gesvd` (buffer or USM version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t gesvd_scratchpad_size(sycl::queue &queue,
        mkl::jobsvd jobu,
        mkl::jobsvd jobvt,
        int64_t m,
        int64_t n,
        int64_t lda,
        int64_t ldu,
        int64_t ldvt)
}
```

Input Parameters

queue	Device queue where calculations by the gesvd (buffer or USM version) function will be performed.
jobu	Must be <code>jobsvd::vectors</code>, <code>jobsvd::somevec</code>, <code>jobsvd::vectorsina</code>, or <code>jobsvd::novec</code>. Specifies options for computing all or part of the matrix U. If <code>jobu = jobsvd::vectors</code>, all m columns of U are returned in the array <code>u</code>; if <code>jobu = jobsvd::somevec</code>, the first $\min(m, n)$ columns of U (the left singular vectors) are returned in the array <code>u</code>; if <code>jobu = jobsvd::vectorsina</code>, the first $\min(m, n)$ columns of U (the left singular vectors) are overwritten on the array <code>a</code>; if <code>jobu = jobsvd::novec</code>, no columns of U (no left singular vectors) are computed.
jobvt	Must be <code>jobsvd::vectors</code>, <code>jobsvd::somevec</code>, <code>jobsvd::vectorsina</code>, or <code>jobsvd::novec</code>. Specifies options for computing all or part of the matrix VT/VH. If <code>jobvt = jobsvd::vectors</code>, all n columns of VT/VH are returned in the array <code>vt</code>; if <code>jobvt = jobsvd::somevec</code>, the first $\min(m, n)$ columns of VT/VH (the left singular vectors) are returned in the array <code>vt</code>; if <code>jobvt = jobsvd::vectorsina</code>, the first $\min(m, n)$ columns of VT/VH (the left singular vectors) are overwritten on the array <code>a</code>; if <code>jobvt = jobsvd::novec</code>, no columns of VT/VH (no left singular vectors) are computed. <code>jobvt</code> and <code>jobu</code> cannot both be <code>job::overwritevec</code>.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
lda	The leading dimension of <code>a</code> .
ldu	The leading dimension of <code>u</code> .
ldvt	The leading dimension of <code>vt</code> .

Exceptions

Exception	Description
<code>mk1::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>get_info()</code> method of the exception object.

Return Values

The number of elements of type $\mathbb{T}$ the scratchpad memory to be passed to the `gesvd` (buffer or USM version) function should be able to hold.

gesvda_batch (Buffer Strided Version)

Computes the truncated SVD factorizations for a batch of general matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the truncated SVD decomposition (or a reduced rank approximation) for a batch of general matrices A_i , as:

$$A_i = U_i * S_i * V_i^T$$

where U_i and V_i are orthogonal matrices, S_i is a diagonal matrix with singular values on the diagonal. Singular values are nonnegative and listed in decreasing order. A truncated SVD of a given matrix will produce matrices with the specified number of columns, where the number of columns is defined by the user or determined at runtime with the help of the user-defined tolerance threshold.

An approximation of each matrix can be also obtained as a product of two low-rank matrices (e.g. low rank product):

$$A_i = P_i * Q_i$$

where

$$P_i = U_i * S_i, Q_i = V_i^T$$

if the number of rows is greater or equal to the number of columns,

$$P_i = U_i, Q_i = S_i * V_i^T$$

otherwise.

The routine provides 3 possible ways for computing truncated SVD:

1. Compute truncated SVD with the help of the input array `rank` where `rank[i]` specifies the number of singular values and vectors to be computed in parameter U_i , V_i and S_i for each matrix A_i .
2. Compute truncated SVD using a tolerance threshold. While computing SVD, singular values that are less than the user-defined tolerance are treated as zero, and they are not computed but set to zero.
3. Compute truncated SVD using the effective rank. The effective rank is determined by treating as zero those singular values that are less than the user-defined tolerance threshold times the largest singular value.

The routine can be also used for computing singular values only.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void gesvda_batch(sycl::queue &queue,
        sycl::buffer<int64_t> &iparm,
        sycl::buffer<int64_t> &irank,
        int64_t m,
        int64_t n,
        sycl::buffer<T> &a,
        int64_t lda,
        int64_t stride_a,
        sycl::buffer<RealT> &s,
        int64_t stride_s,
        sycl::buffer<T> &u,
        int64_t ldu,
        int64_t stride_u,
```

```

sycl::buffer<T> &vt,
int64_t ldvt,
int64_t stride_vt,
RealT tolerance,
sycl::buffer<T> &residual,
int64_t batch_size,
sycl::buffer<T> &scratchpad, int64_t scratchpad_size)
}

```

gesvda_batch supports the following precisions and devices:

T	Devices supported
float	CPU, GPU*
double	CPU, GPU*
std::complex<float>	CPU, GPU*
std::complex<double>	CPU, GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
iparm	Array of dimension 16 specifying options to compute truncated SVD. It also specifies the type of returned SVD decomposition form. The table below describes all individual components of the iparm parameter. Default values are denoted with an asterisk (*). NOTE: iparm[4]-iparm[15] are reserved for future use.

Component description	Values
iparm[0] specifies a criterion for treating singular values as zeros	If iparm[0]=-1, default iparm values (iparm[0-2]=0, iparm[3]=1) are used If iparm[0]=0*, the truncated SVD is computed with the help of the input array irank If iparm[0]=1, the truncated SVD is computed using the parameter tolerance If iparm[0]=2, the truncated SVD is computed using the effective rank. The effective rank is determined by treating as zero those singular values that are less than the user-defined tolerance times the largest singular value
iparm[1] specifies option for computing singular vectors	If iparm[1]=0*, both singular values and singular vectors are computed If iparm[1]=1, only singular values are computed
iparm[2] specifies type of returned SVD decomposition	If iparm[2]=0*, the truncated SVD as a product of three matrices $A_i = U_i * S_i * V_i^T$

Component description	Values
	is computed. If <code>iparm[2]=1</code> , the truncated as as a low rank product $A_i = P_i * Q_i$ is computed.
<code>iparm[3]</code> specifies option for computing the residual vector	If <code>iparm[2]=0</code> , the residual vector is not computed If <code>iparm[2]=1*</code> , the residual vector is computed

<code>irank</code>	If <code>iparm[0]=0</code> or <code>iparm[0]=-1</code> , element <code>irank[i]</code> specifies the number of singular values and/or singular vectors to be computed in U_i , V_i^T and S_i for each matrix A_i .
<code>m</code>	The number of rows in the matrices A_i .
<code>n</code>	The number of columns in the matrices A_i .
<code>a</code>	Array holding input matrices A_i .
<code>lda</code>	The leading dimension of <code>a</code> . Must be at least <code>max(1, m)</code> .
<code>stride_a</code>	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> . Must be at least <code>max(1, lda * n)</code> .
<code>stride_s</code>	The stride between the beginnings of arrays S_i inside the array <code>s</code> . Must be at least <code>min(m, n)</code> .
<code>ldu</code>	The leading dimension of U_i . Must be at least <code>max(1, m)</code> .
<code>stride_u</code>	The stride between the beginnings of matrices U_i inside the batch array <code>u</code> . Must be at least <code>max(1, ldu * m)</code> .
<code>ldvt</code>	The leading dimension of V_i^T . Must be at least <code>max(1, n)</code> .
<code>stride_vt</code>	The stride between the beginnings of matrices V_i^T inside the batch array <code>vt</code> . Must be at least <code>max(1, ldvt * n)</code> .
<code>tolerance</code>	Specifies the tolerance threshold. It is only used for computing truncated SVD in the cases of <code>iparm[0]=1</code> and <code>iparm[0]=2</code> . Not used otherwise.
<code>batch_size</code>	The number of problems in a batch.
<code>scratchpad</code>	Scratchpad memory to be used by routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by gesvda_batch_scratchpad_size (Strided Version) function.

Output Parameters

<code>irank</code>	If <code>iparm[0]=-1</code> or <code>iparm[0]=0</code> , element <code>irank[i]</code> is the number of computed singular values and/or singular vectors for matrix A_i .
--------------------	---

a	Unchanged on exit, if the residual vector is not required. Otherwise contains the residual matrix $A_i - U_i * S_i * V_i^T \text{ if } iparm[2]=0,$ $A_i - P_i * Q_i \text{ if } iparm[2]=1.$
s	Array of size $\min(m, n) * batch_size$ to store batch of singular values S_i .
u	u is array of size at least $stride_u * batch_size$ to store batch of U_i if $iparm[2]=0$, or to store batch of P_i if $iparm[2]=1$.
vt	vt is array of size at least $stride_vt * batch_size$ to store batch of V_i^T if $iparm[2]=0$, or to store batch of Q_i if $iparm[2]=1$.
residual	Array of dimension $batch_size$. If $iparm[3]=1$, $residual[i]$ is the Frobenius norm of the matrix: $ A_i - U_i * S_i * V_i^T $ if $iparm[2]=0$, and $ A_i - P_i * Q_i $ if $iparm[2]=1$.

Exceptions

Exception	Description
<code>mk1::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info = 1</code>, an internal memory allocation failed. If <code>info = 2</code>, an input parameter contains an invalid value. If <code>info = 3</code>, an error in algorithm while computing singular values . If <code>info = 4</code>, the routine encountered an empty structure or matrix array.

gesvda_batch (USM Strided Version)

Computes the truncated SVD factorizations for a batch of general matrices. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the truncated SVD decomposition (or a reduced rank approximation) for a batch of general matrices A_i , as:

$$A_i = U_i * S_i * V_i^T$$

where U_i and V_i are orthogonal matrices, S_i is a diagonal matrix with singular values on the diagonal. Singular values are nonnegative and listed in decreasing order. A truncated SVD of a given matrix will produce matrices with the specified number of columns, where the number of columns is defined by the user or determined at runtime with the help of the user-defined tolerance threshold.

An approximation of each matrix can be also obtained as a product of two low-rank matrices (e.g. low rank product):

$$A_i = P_i * Q_i$$

where

$$P_i = U_i * S_i, Q_i = V_i^T$$

if the number of rows is greater or equal to the number of columns,

$$P_i = U_i, Q_i = S_i * V_i^T$$

otherwise.

The routine provides 3 possible ways for computing truncated SVD:

1. Compute truncated SVD with the help of the input array `rank` where `rank[i]` specifies the number of singular values and vectors to be computed in parameter U_i , V_i and S_i for each matrix A_i .
2. Compute truncated SVD using a tolerance threshold. While computing SVD, singular values that are less than the user-defined tolerance are treated as zero, and they are not computed but set to zero.
3. Compute truncated SVD using the effective rank. The effective rank is determined by treating as zero those singular values that are less than the user-defined tolerance threshold times the largest singular value.

The routine can be also used for computing singular values only.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void gesvda_batch(sycl::queue &queue,
        int64_t *iparm,
        int64_t *irank,
        int64_t m,
        int64_t n,
        T *a,
        int64_t lda,
        int64_t stride_a,
        RealT *s,
        int64_t stride_s,
        T *u,
        int64_t ldu,
        int64_t stride_u,
        T *vt,
        int64_t ldvt,
        int64_t stride_vt,
        RealT tolerance,
        RealT *residual,
        int64_t batch_size,
```

```
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
}
```

`gesvda_batch` supports the following precisions and devices:

T	Devices supported
float	CPU, GPU*
double	CPU, GPU*
std::complex<float>	CPU, GPU*
std::complex<double>	CPU, GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
iparm	Array of dimension 16 specifying options to compute truncated SVD. It also specifies the type of returned SVD decomposition form. The table below describes all individual components of the <code>iparm</code> parameter. Default values are denoted with an asterisk (*). NOTE: <code>iparm[4]-iparm[15]</code> are reserved for future use.

Component description	Values
<code>iparm[0]</code> specifies a criterion for treating singular values as zeros	If <code>iparm[0]=-1</code>, default <code>iparm</code> values (<code>iparm[0-2]=0</code>, <code>iparm[3]=1</code>) are used If <code>iparm[0]=0*</code>, the truncated SVD is computed with the help of the input array <code>irank</code> If <code>iparm[0]=1</code>, the truncated SVD is computed using the parameter <code>tolerance</code> If <code>iparm[0]=2</code>, the truncated SVD is computed using the effective rank. The effective rank is determined by treating as zero those singular values that are less than the user-defined tolerance times the largest singular value
<code>iparm[1]</code> specifies option for computing singular vectors	If <code>iparm[1]=0*</code>, both singular values and singular vectors are computed If <code>iparm[1]=1</code>, only singular values are computed
<code>iparm[2]</code> specifies type of returned SVD decomposition	If <code>iparm[2]=0*</code>, the truncated SVD as a product of three matrices $A_i = U_i * S_i * V_i^T$ is computed. If <code>iparm[2]=1</code>, the truncated as as a low rank product

Component description	Values
iparm[3] specifies option for computing the residual vector	$A_i = P_i * Q_i$
	is computed.
	If iparm[2]=0, the residual vector is not computed If iparm[2]=1*, the residual vector is computed
irank	If iparm[0]=0 or iparm[0]=-1, element irank[i] specifies the number of singular values and/or singular vectors to be computed in U_i , V_i^T and S_i for each matrix A_i .
m	The number of rows in the matrices A_i .
n	The number of columns in the matrices A_i .
a	Array holding input matrices A_i .
lda	The leading dimension of a. Must be at least $\max(1, m)$.
stride_a	The stride between the beginnings of matrices A_i inside the batch array a. Must be at least $\max(1, lda * n)$.
stride_s	The stride between the beginnings of arrays S_i inside the array s. Must be at least $\min(m, n)$.
ldu	The leading dimension of U_i . Must be at least $\max(1, m)$.
stride_u	The stride between the beginnings of matrices U_i inside the batch array u. Must be at least $\max(1, ldu * m)$.
ldvt	The leading dimension of V_i^T . Must be at least $\max(1, n)$.
stride_vt	The stride between the beginnings of matrices V_i^T inside the batch array vt. Must be at least $\max(1, ldvt * n)$.
tolerance	Specifies the tolerance threshold. It is only used for computing truncated SVD in the cases of iparm[0]=1 and iparm[0]=2. Not used otherwise.
batch_size	The number of problems in a batch.
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by gesvda_batch_scratchpad_size (Strided Version) function.

Output Parameters

irank	If iparm[0]=-1 or iparm[0]=0, element irank[i] is the number of computed singular values and/or singular vectors for matrix A_i .
a	Unchanged on exit, if the residual vector is not required. Otherwise contains the residual matrix $A_i - U_i * S_i * V_i^T$ if iparm[2]=0, $A_i - P_i * Q_i$ if iparm[2]=1.

s	Array of size $\min(m, n) * \text{batch_size}$ to store batch of singular values S_i .
u	u is array of size at least $\text{stride_u} * \text{batch_size}$ to store batch of U_i if $\text{iparm}[2]=0$, or to store batch of P_i if $\text{iparm}[2]=1$.
vt	vt is array of size at least $\text{stride_vt} * \text{batch_size}$ to store batch of V_i^T if $\text{iparm}[2]=0$, or to store batch of Q_i if $\text{iparm}[2]=1$.
residual	Array of dimension batch_size. If $\text{iparm}[3]=1$, $\text{residual}[i]$ is the Frobenius norm of the matrix: $ A_i - U_i * S_i * V_i^T $ if $\text{iparm}[2]=0$, and $ A_i - P_i * Q_i $ if $\text{iparm}[2]=1$.

Exceptions

Exception	Description
<code>mk1::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info = 1</code>, an internal memory allocation failed. If <code>info = 2</code>, an input parameter contains an invalid value. If <code>info = 3</code>, an error in algorithm while computing singular values . If <code>info = 4</code>, the routine encountered an empty structure or matrix array.

Return Values

Output event to wait on to ensure computation is complete.

gesvda_batch_scratchpad_size (Strided Version)

Computes size of scratchpad memory required for `gesvda_batch` (Strided Version) function. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `gesvda_batch` (Strided Version) function must be able to hold.

API

Syntax

```

namespace oneapi::mkl::lapack {
    int64_t gesvda_batch_scratchpad_size(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t lda,
        int64_t stride_a,
        int64_t ldu,
        int64_t stride_u,
        int64_t ldvt,
        int64_t stride_vt,
        int64_t batch_size)
}

```

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i .
n	The number of columns in the matrices A_i .
lda	The leading dimension of a. Must be at least $\max(1, m)$.
stride_a	The stride between the beginnings of matrices A_i inside the batch array a. Must be at least $\max(1, lda * n)$.
stride_s	The stride between the beginnings of arrays S_i inside the array s. Must be at least $\min(m, n)$.
ldu	The leading dimension of U_i . Must be at least $\max(1, m)$.
stride_u	The stride between the beginnings of matrices U_i inside the batch array u. Must be at least $\max(1, ldu * m)$.
ldvt	The leading dimension of V_i^T . Must be at least $\max(1, n)$.
stride_vt	The stride between the beginnings of matrices V_i^T inside the batch array vt. Must be at least $\max(1, ldvt * n)$.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `gesvda_batch` (Strided Version) function must be able to hold.

getrf

Computes the LU factorization of a general m-by-n matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the LU factorization of a general m-by-n matrix A as

$$A = P * L * U,$$

where P is a permutation matrix, L is lower triangular with unit diagonal elements (lower trapezoidal if $m > n$) and U is upper triangular (upper trapezoidal if $m < n$). The routine uses partial pivoting, with row interchanges.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void getrf(sycl::queue &queue,
               int64_t m,
               int64_t n,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<T> &ipiv,
               sycl::buffer<T> &scratchpad,
               int64_t scratchpad_size)
}
```

`getrf` supports the following precisions and devices.

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*
std::complex<float>	CPU and GPU*
std::complex<double>	CPU and GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in A ($0 \leq n$).
a	Buffer holding input matrix A . The array must be of size at least <code>lda * max(1, n)</code> .

lda	The leading dimension of a ($lda \geq \max(1, m)$).
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size must not be less than the value returned by the getrf_scratchpad_size function.

Output Parameters

a	Overwritten by L and U . The unit diagonal elements of L are not stored.
ipiv	Buffer holding array of size at least $\max(1, \min(m, n))$. Contains the pivot indices; for $1 \leq i \leq \min(m, n)$, row i was interchanged with row $ipiv(i)$.

Exceptions

Exception	Description
<code>mk1::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info = i</code>, <code>uii</code> is 0. The factorization has been completed, but U is exactly singular. Division by 0 will occur if you use the factor U for solving a system of linear equations. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size must not be less than the value returned by the <code>detail()</code> method of the exception object.

getrf (USM Version)

Computes the LU factorization of a general m -by- n matrix. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the LU factorization of a general m -by- n matrix A as

$$A = P * L * U,$$

where P is a permutation matrix, L is lower triangular with unit diagonal elements (lower trapezoidal if $m > n$) and U is upper triangular (upper trapezoidal if $m < n$). The routine uses partial pivoting, with row interchanges.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getrf(sycl::queue &queue,
        int64_t m,
        int64_t n,
        T *a,
        int64_t lda,
        int64_t *ipiv,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`getrf` (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU, GPU*
double	CPU, GPU*
std::complex<float>	CPU, GPU*
std::complex<double>	CPU, GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
a	Pointer to the array holding input matrix A . The array must be of size at least $lda * \max(1, n)$.
lda	The leading dimension of a ($lda \geq \max(1, m)$).
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size must not be less than the value returned by the getrf_scratchpad_size function.

events List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a Overwritten by L and U . The unit diagonal elements of L are not stored.

ipiv Array, size at least $\max(1, \min(m, n))$.
Contains the pivot indices; for $1 \leq i \leq \min(m, n)$, row i was interchanged with row $\text{ipiv}(i)$.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info = i</code>, <code>uii</code> is 0. The factorization has been completed, but U is exactly singular. Division by 0 will occur if you use the factor U for solving a system of linear equations. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size must not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

getrf_batch (Buffer Strided Version)

Computes the batch of LU factorizations of a batch of general m -by- n matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the LU factorizations of a batch of general m -by- n matrices A_i , as:

$$A_i = P_i \times L_i \times U_i$$

Where P_i is a permutation matrix, L_i is lower triangular with unit diagonal elements (lower trapezoidal if $m > n$) and U_i is upper triangular (upper trapezoidal if $m < n$). The routine uses partial pivoting with row interchanges.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void getrf_batch(
        sycl::queue &queue,
        int64_t m,
        int64_t n,
        sycl::buffer<T> &a,
        int64_t lda,
        int64_t stride_a,
        sycl::buffer<int64_t> &ipiv,
        int64_t stride_ipiv,
        int64_t batch_size,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i ($m \geq 0$).
n	The number of columns in the matrices A_i ($n \geq 0$).
a	Array holding input matrices A_i .
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).
stride_ipiv	The stride between the beginnings of arrays $ipiv_i$ inside the array ipiv ($stride_ipiv \geq \max(1, \min(m, n))$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by getrf_batch_scratchpad_size (Strided Version) .

Output Parameters

a	Overwritten by L_i and U_i . The unit diagonal elements of L_i are not stored.
---	--

ipiv

Array containing the batch of the pivot indices $ipiv_i$ each of size at least $\max(1, \min(m, n))$; for $1 \leq k \leq \min(m, n)$, row k of A_i was interchanged with row $ipiv_i(k)$.

Exceptions

Exception	Description
<code>mk1::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

getrf_batch (Group Version)

Computes the batch of LU factorizations of a batch of general m -by- n matrices. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the LU factorizations of a batch of general m -by- n matrices A_i , ($i \in \{1 \dots \text{batch_size}\}$) as

$$A_i = P_i \times L_i \times U_i$$

Where P_i is a permutation matrix, L_i is lower triangular with unit diagonal elements (lower trapezoidal if $m > n$) and U_i is upper triangular (upper trapezoidal if $m < n$). The routine uses partial pivoting with row interchanges.

Total number of problems to solve, `batch_size`, is a sum of sizes of all of the groups of parameters as provided by `group_sizes` array.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getrf_batch(sycl::queue &queue,
        int64_t *m,
        int64_t *n,
        T **a,
        int64_t *lda,
        int64_t **ipiv,
        int64_t group_count,
        int64_t *group_sizes,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
m	Array of <code>group_count</code> parameters m_g specifying the number of rows in the matrices A_i belonging to group g ($m_g \geq 0$).
n	Array of <code>group_count</code> parameters n_g specifying the number of columns in the matrices A_i belonging to group g ($n_g \geq 0$).
a	Array of <code>batch_size</code> pointers to input matrices A_i . The pointers must be device-accessible.
lda	Array of <code>group_count</code> parameters lda_g specifying the leading dimension of A_i belonging to group g ($lda_g \geq \max(1, m_g)$).
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less then the value returned by getrf_batch_scratchpad_size (Group Version) .
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	Matrices pointed to by array <code>a</code> are overwritten by L_i and U_i . The unit diagonal elements of L_i are not stored.
ipiv	Arrays of <code>batch_size</code> pointers to arrays containing pivot indices <code>ipiv_i</code> each of size at least $\max(1, \min(m_g, n_g))$; for $1 \leq k \leq \min(m_g, n_g)$, row k of A_i was interchanged with row <code>ipiv_i(k)</code> . The pointers must be device-accessible.

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non-zero, then the passed <code>scratchpad</code> is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

getrf_batch (USM Strided Version)

Computes the batch of LU factorizations of a batch of general m -by- n matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- Description
- API

Description

The routine computes the LU factorizations of a batch of general m -by- n matrices A_i , as

$$A_i = P_i \times L_i \times U_i$$

Where P_i is a permutation matrix, L_i is lower triangular with unit diagonal elements (lower trapezoidal if $m > n$) and U_i is upper triangular (upper trapezoidal if $m < n$). The routine uses partial pivoting with row interchanges.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getrf_batch(sycl::queue &queue,
        int64_t m,
        int64_t n,
        T *a,
        int64_t lda,
        int64_t stride_a,
        int64_t *ipiv,
        int64_t stride_ipiv,
        int64_t batch_size,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i ($m \geq 0$).
n	The number of columns in the matrices A_i ($n \geq 0$).
a	Array holding input matrices A_i .
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).
stride_ipiv	The stride between the beginnings of arrays $ipiv_i$ inside the array ipiv ($stride_ipiv \geq \max(1, \min(m, n))$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.

scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by getrf_batch_scratchpad_size (Strided Version) .
events	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
<code>a</code>	Overwritten by L_i and U_i . The unit diagonal elements of L_i are not stored.
<code>ipiv</code>	Array containing the batch of the pivot indices $ipiv_i$ each of size at least $\max(1, \min(m, n))$; for $1 \leq k \leq \min(m, n)$, row k of A_i was interchanged with row $ipiv_i(k)$.

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

[getrf_batch_scratchpad_size \(Group Version\)](#)

Computes size of scratchpad memory required for `getrf_batch` (Group Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the [getrf_batch \(Group Version\)](#) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t getrf_batch_scratchpad_size(sycl::queue &queue,
        int64_t *m,
        int64_t *n,
        int64_t *lda,
        int64_t group_count,
        int64_t *group_sizes)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
m	Array of <code>group_count</code> parameters m_g specifying the number of rows in the matrices belonging to group g ($m_g \geq 0$).
n	Array of <code>group_count</code> parameters n_g specifying the number of columns in the matrices belonging to group g ($n_g \geq 0$).
lda	Array of <code>group_count</code> parameters lda_g specifying the leading dimension of the matrices belonging to group g ($lda_g \geq \max(1, m_g)$).
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the [getrf_batch \(Group Version\)](#) function should be able to hold.

getrf_batch_scratchpad_size (Strided Version)

Computes size of scratchpad memory required for `getrf_batch` (Strided Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type T the scratchpad memory to be passed to the `getrf_batch` (Strided Version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t getrf_batch_scratchpad_size(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t lda,
        int64_t stride_a,
        int64_t stride_ipiv,
        int64_t batch_size)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>m</code>	The number of rows in the matrices A_i ($m \geq 0$).
<code>n</code>	The number of columns in the matrices A_i ($n \geq 0$).
<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, m)$).
<code>stride_a</code>	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
<code>stride_ipiv</code>	The stride between the beginnings of arrays $ipiv_i$ inside the array <code>ipiv</code> ($stride_ipiv \geq \max(1, \min(m, n))$).
<code>batch_size</code>	The number of problems in a batch ($batch_size \geq 0$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `getrf_batch` (Strided Version) function should be able to hold.

getrf_scratchpad_size

Computes size of scratchpad memory required for `getrf` function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type T the scratchpad memory to be passed to the `getrf` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t getrf_scratchpad_size(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t lda)
}
```

Input Parameters

queue	Device queue where calculations by the <code>getrf</code> (buffer or USM version) function will be performed.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
lda	The leading dimension of a ($lda \geq \max(1, m)$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `getrf` (buffer or USM version) function must be able to hold.

getrfnp

Computes the LU factorization of a general m -by- n matrix without pivoting. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the LU factorization of a general m -by- n matrix A as

$$A = L*U,$$

where L is lower triangular with unit diagonal elements (lower trapezoidal if $m > n$) and U is upper triangular (upper trapezoidal if $m < n$). The routine does not use pivoting.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void getrfnp(sycl::queue &queue,
        int64_t m,
        int64_t n,
        sycl::buffer<T> &a,
        int64_t lda,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

`getrfnp` supports the following precisions and devices.

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*
std::complex<float>	CPU and GPU*
std::complex<double>	CPU and GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrix A ($m \geq 0$).
n	The number of columns in A ($n \geq 0$).
a	Buffer holding input matrix A . The array must be of size at least $lda * \max(1, n)$.
lda	The leading dimension of a ($lda \geq \max(1, m)$).
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.

scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size must not be less than the value returned by the getrfnp_scratchpad_size function.
-----------------	---

Output Parameters

a	Overwritten by L and U . The unit diagonal elements of L are not stored.
---	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info = i</code>, <code>u_ii</code> is 0. The factorization has been completed, but U is exactly singular. Division by 0 will occur if you use the factor U for solving a system of linear equations. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size must not be less than the value returned by the <code>detail()</code> method of the exception object.

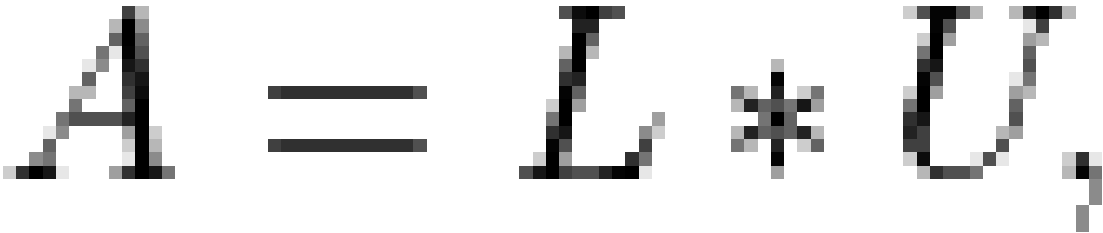
getrfnp (USM Version)

Computes the LU factorization of a general m -by- n matrix without pivoting. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the LU factorization of a general m -by- n matrix A as


$$A = L * U$$

where L is lower triangular with unit diagonal elements (lower trapezoidal if $m > n$) and U is upper triangular (upper trapezoidal if $m < n$). The routine does not use pivoting.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getrfnp(sycl::queue &queue,
        int64_t m,
        int64_t n,
        T *a,
        int64_t lda,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

getrfnp (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU, GPU*
double	CPU, GPU*
std::complex<float>	CPU, GPU*
std::complex<double>	CPU, GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrix A ($m \geq 0$).
n	The number of columns in the matrix A ($n \geq 0$).
a	Pointer to the array holding input matrix A. The array must be of size at least $lda * \max(1, n)$.
lda	The leading dimension of a ($lda \geq \max(1, m)$).
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size must not be less than the value returned by the getrfnp_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	Overwritten by L and U. The unit diagonal elements of L are not stored.
---	---

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, <code>u[i]</code> is 0. The factorization has been completed, but <i>U</i> is exactly singular. Division by 0 will occur if you use the factor <i>U</i> for solving a system of linear equations. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non-zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size must not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

getrfnp_scratchpad_size

Computes size of scratchpad memory required for `getrfnp` function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type *T* the scratchpad memory to be passed to the `getrfnp` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t getrfnp_scratchpad_size(sycl::queue &queue,
    int64_t m,
    int64_t n,
    int64_t lda)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>getrfnp</code> (buffer or USM version) function will be performed.
<code>m</code>	The number of rows in the matrix <i>A</i> ($m \geq 0$).

<code>n</code>	The number of columns in the matrix A ($n \geq 0$).
<code>lda</code>	The leading dimension of a ($lda \geq \max(1, m)$).

Exceptions

Exception	Description
<code>mk1::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `getrfnp` (buffer or USM version) function must be able to hold.

getrfnp_batch (Buffer Strided Version)

Computes the batch of LU factorizations of a batch of general m -by- n matrices. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the LU factorizations of a batch of general m -by- n matrices A_i , as:

$$A_i = L_i \times U_i$$

Where L_i is lower triangular with unit diagonal elements (lower trapezoidal if $m > n$) and U_i is upper triangular (upper trapezoidal if $m < n$). The routine does not perform any pivoting.

API

Syntax

```
namespace oneapi::mk1::lapack {
    void getrfnp_batch(
        sycl::queue &queue,
        int64_t m,
        int64_t n,
        sycl::buffer<T> &a,
        int64_t lda,
        int64_t stride_a,
        int64_t batch_size,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i ($m \geq 0$).
n	The number of columns in the matrices A_i ($n \geq 0$).
a	Array holding input matrices A_i .
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by getrfnp_batch_scratchpad_size (Strided Version) .

Output Parameters

a	Overwritten by L_i and U_i . The unit diagonal elements of L_i are not stored.
---	--

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be

Exception	Description
	obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

getrfnp_batch (Group Version)

Computes the batch of LU factorizations of a batch of general m-by-n matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the LU factorizations of a batch of general m-by-n matrices A_i , ($i \in \{1 \dots \text{batch_size}\}$) as

$$A_i = L_i \times U_i$$

Where L_i is lower triangular with unit diagonal elements (lower trapezoidal if $m > n$) and U_i is upper triangular (upper trapezoidal if $m < n$). The routine does not perform any pivoting.

Total number of problems to solve, `batch_size`, is a sum of sizes of all of the groups of parameters as provided by `group_sizes` array.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getrfnp_batch(sycl::queue &queue,
        int64_t *m,
        int64_t *n,
        T **a,
        int64_t *lda,
        int64_t group_count,
        int64_t *group_sizes,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU

T	Devices supported
<code>std::complex<float></code>	CPU and GPU
<code>std::complex<double></code>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
m	Array of <code>group_count</code> parameters m_g specifying the number of rows in the matrices A_i belonging to group g ($m_g \geq 0$).
n	Array of <code>group_count</code> parameters n_g specifying the number of columns in the matrices A_i belonging to group g ($n_g \geq 0$).
a	Array of <code>batch_size</code> pointers to input matrices A_i . The pointers must be device-accessible.
lda	Array of <code>group_count</code> parameters lda_g specifying the leading dimension of A_i belonging to group g ($lda_g \geq \max(1, m_g)$).
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less then the value returned by getrfnp_batch_scratchpad_size (Group Version) .
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	Matrices pointed to by array <code>a</code> are overwritten by L_i and U_i . The unit diagonal elements of L_i are not stored.
---	--

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

Exception	Description
	If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

getrfnp_batch (USM Strided Version)

Computes the batch of LU factorizations of a batch of general m -by- n matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the LU factorizations of a batch of general m -by- n matrices A_i , as

$$A_i = L_i \times U_i$$

Where L_i is lower triangular with unit diagonal elements (lower trapezoidal if $m > n$) and U_i is upper triangular (upper trapezoidal if $m < n$). The routine does not perform any pivoting.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getrfnp_batch(sycl::queue &queue,
        int64_t m,
        int64_t n,
        T *a,
        int64_t lda,
        int64_t stride_a,
        int64_t batch_size,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i ($m \geq 0$).
n	The number of columns in the matrices A_i ($n \geq 0$).
a	Array holding input matrices A_i .
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by getrfnp_batch_scratchpad_size (Strided Version) .
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	Overwritten by L_i and U_i . The unit diagonal elements of L_i are not stored.
---	--

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

Exception	Description
	If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

getrfnp_batch_scratchpad_size (Group Version)

Computes size of scratchpad memory required for `getrfnp_batch` (Group Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the [getrfnp_batch \(Group Version\)](#) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t getrfnp_batch_scratchpad_size(sycl::queue &queue,
        int64_t *m,
        int64_t *n,
        int64_t *lda,
        int64_t group_count,
        int64_t *group_sizes)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

`queue` Device queue where calculations will be performed.

<code>m</code>	Array of <code>group_count</code> parameters m_g specifying the number of rows in the matrices belonging to group g ($m_g \geq 0$).
<code>n</code>	Array of <code>group_count</code> parameters n_g specifying the number of columns in the matrices belonging to group g ($n_g \geq 0$).
<code>lda</code>	Array of <code>group_count</code> parameters lda_g specifying the leading dimension of the matrices belonging to group g ($lda_g \geq \max(1, m_g)$).
<code>group_count</code>	Specifies the number of groups of parameters. Must be at least 0.
<code>group_sizes</code>	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the [getrfnp_batch \(Group Version\)](#) function should be able to hold.

getrfnp_batch_scratchpad_size (Strided Version)

Computes size of scratchpad memory required for `getrfnp_batch` (Strided Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `getrfnp_batch` (Strided Version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t getrfnp_batch_scratchpad_size(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t lda,
        int64_t stride_a,
        int64_t batch_size)
}
```

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i ($m \geq 0$).
n	The number of columns in the matrices A_i ($n \geq 0$).
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).

Exceptions

Exception	Description
<code>mk1::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `getrfnp_batch` (Strided Version) function should be able to hold.

getri

Computes the inverse of an LU-factored general matrix determined by [getrf](#). This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the inverse $\text{inv}(A)$ of a general matrix A . Before calling this routine, call [getrf](#) to factorize A .

API

Syntax

```
namespace oneapi::mk1::lapack {
    void getri(sycl::queue &queue,
               int64_t n,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<T> &ipiv,
               sycl::buffer<T> &scratchpad, int64_t scratchpad_size)
}
```

`getri` supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
n	The order of the matrix A ($0 \leq n$).
a	The buffer returned by getrf . Must be of size at least $lda * \max(1, n)$.
lda	The leading dimension of a ($n \leq lda$).
ipiv	Buffer holding the array as returned by getrf . The dimension of the array must be at least $\max(1, n)$.
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by the getri_scratchpad_size function.

Output Parameters

a	Overwritten by the n-by-n matrix A .
---	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

getri (USM Version)

Computes the inverse of an LU-factored general matrix determined by `getrf` (USM Version). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the inverse $\text{inv}(A)$ of a general matrix A . Before calling this routine, call [getrf \(USM Version\)](#) to factorize A .

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getri(sycl::queue &queue,
        int64_t n,
        T *a,
        int64_t lda,
        const int64_t *ipiv,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

[getri \(USM version\)](#) supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
n	The order of the matrix A ($0 \leq n$).
a	The pointer returned by getrf (USM Version) . Must be of size at least $\text{lda} * \max(1, n)$.
lda	The leading dimension of a ($n \leq \text{lda}$).
ipiv	The array as returned by getrf (USM Version) . The dimension of ipiv must be at least $\max(1, n)$.
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by the getri_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a Overwritten by the n-by-n matrix `inv(A)`.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

getri_batch (Buffer Strided Version)

Computes the batch of inverses of an LU-factored general matrices determined by `getrf_batch` (Buffer stride version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the inverses A_i^{-1} of a general matrices A_i . Before calling this routine, call [getrf_batch \(Buffer Strided Version\)](#) function to factorize A_i .

API

Syntax

```
namespace oneapi::mkl::lapack {
    void getri_batch(sycl::queue &queue,
        int64_t n,
        sycl::buffer<T> &a,
        int64_t lda,
        int64_t stride_a,
        sycl::buffer<int64_t> &ipiv,
        int64_t stride_ipiv,
        int64_t batch_size,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
n	The order of the matrices A_i ($n \geq 0$).
a	Result of the getrf_batch (Buffer Strided Version) function
lda	The leading dimension of A_i ($lda \geq \max(1, n)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
ipiv	The array as returned by getrf_batch (Buffer Strided Version) .
stride_ipiv	The stride between the beginnings of arrays $ipiv_i$ inside the array <code>ipiv</code> ($stride_ipiv \geq \max(1, n)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by stride version of getri_batch_scratchpad_size (Strided Version) function.

Output Parameters

a	Overwritten by the n-by-n matrices A_i^{-1} .
---	---

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

getri_batch (Group Version)

Computes the batch of inverses of an LU-factored general matrices determined by the `getrf_batch` (Group Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the inverses A_i^{-1} of a general matrices A_i , $i \in \{1 \dots \text{batch_size}\}$. Before calling this routine, call [getrf_batch \(Group Version\)](#) function to factorize A_i .

The total number of problems to solve, `batch_size`, is a sum of sizes of all of the groups of parameters as provided by `group_sizes` array.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getri_batch(sycl::queue &queue,
        int64_t *n,
        T **a,
        int64_t *lda,
        const int64_t * const *ipiv,
        int64_t group_count,
        int64_t *group_sizes,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
n	Array of <code>group_count</code> parameters n_g specifying the order of the matrices A_i belonging to group g ($n_g \geq 0$).
a	Result of the getrf_batch (Group Version) function. The pointers must be device-accessible.
lda	Array of <code>group_count</code> parameters lda_g specifying the leading dimension of A_i belonging to group g ($lda_g \geq \max(1, n_g)$).

ipiv	The array as returned by getrf_batch (Group Version) . The pointers must be device-accessible.
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of group_count integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less then the value returned by getri_batch_scratchpad_size (Group Version) .
events	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
a	Overwritten by the n_g -by- n_g matrices A_i^{-1} .

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

[getri_batch \(USM Strided Version\)](#)

Computes the batch of inverses of an LU-factored general matrices determined by [getrf_batch](#) (USM stride version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the inverses A_i^{-1} of a general matrices A_i . Before calling this routine, call [getrf_batch \(USM Strided Version\)](#) function to factorize A_i .

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getri_batch(sycl::queue &queue,
        int64_t n,
        T *a,
        int64_t lda,
        int64_t stride_a,
        const int64_t *ipiv,
        int64_t stride_ipiv,
        int64_t batch_size,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
n	The order of the matrices A_i ($n \geq 0$).
a	Result of the getrf_batch (USM Strided Version) function
lda	The leading dimension of A_i ($lda \geq \max(1, n)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).
ipiv	The array as returned by getrf_batch (USM Strided Version) .
stride_ipiv	The stride between the beginnings of arrays $ipiv_i$ inside the array ipiv ($stride_ipiv \geq \max(1, n)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by stride version of getri_batch_scratchpad_size (Strided Version) function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a Overwritten by the n-by-n matrices A_i^{-1} .

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

getri_batch_scratchpad_size (Group Version)

Computes size of scratchpad memory required for `getri_batch` (Group Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `getri_batch` (Group Version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t getri_batch_scratchpad_size(sycl::queue &queue,
        int64_t *n,
        int64_t *lda,
        int64_t group_count,
        int64_t *group_sizes)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU

T	Devices supported
<code>std::complex<float></code>	CPU and GPU
<code>std::complex<double></code>	CPU and GPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>n</code>	Array of <code>group_count</code> parameters n_g specifying the order of the matrices belonging to group g ($n_g \geq 0$).
<code>lda</code>	Array of <code>group_count</code> parameters lda_g specifying the leading dimension of the matrices belonging to group g ($lda_g \geq \max(1, n_g)$).
<code>group_count</code>	Specifies the number of groups of parameters. Must be at least 0.
<code>group_sizes</code>	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the [getri_batch \(Group Version\)](#) function must be able to hold.

getri_batch_scratchpad_size (Strided Version)

Computes size of scratchpad memory required for `getri_batch` (Strided Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `getri_batch` (Strided Version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t getri_batch_scratchpad_size(sycl::queue &queue,
        int64_t n,
```

```
int64_t lda,
int64_t stride_a,
int64_t stride_ipiv,
int64_t batch_size)
}
```

Input Parameters

queue	Device queue where calculations will be performed.
n	The order of the matrices A_i ($n \geq 0$).
lda	The leading dimension of A_i ($lda \geq \max(1, n)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
stride_ipiv	The stride between the beginnings of arrays $ipiv_i$ inside the array <code>ipiv</code> ($stride_ipiv \geq \max(1, n)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `getri_batch` (Strided Version) function should be able to hold.

getri_batch (Out-of-place, Buffer Strided Version)

Computes the batch of inverses of an LU-factored general matrices determined by `getrf_batch` (Buffer stride version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the inverses A_i^{-1} of a general matrices A_i . Before calling this routine, call [getrf_batch \(Buffer Strided Version\)](#) function to factorize A_i .

API

Syntax

```
namespace oneapi::mkl::lapack {
void getri_batch(sycl::queue &queue,
int64_t n,
sycl::buffer<T> &a,
```

```

int64_t lda,
int64_t stride_a,
sycl::buffer<int64_t> &ipiv,
int64_t stride_ipiv,
sycl::buffer<T> &ainv,
int64_t ldainv,
int64_t stride_ainv,
int64_t batch_size,
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}

```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
n	The order of the matrices A_i ($n \geq 0$).
a	Result of the getrf_batch (Buffer Strided Version) function
lda	The leading dimension of A_i ($lda \geq \max(1, n)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).
ipiv	The array as returned by getrf_batch (Buffer Strided Version) .
stride_ipiv	The stride between the beginnings of arrays $ipiv_i$ inside the array ipiv ($stride_ipiv \geq \max(1, n)$).
ldainv	The leading dimension of A_{inv_i} ($ldainv \geq \max(1, n)$).
stride_ainv	The stride between the beginnings of matrices A_{inv_i} inside the batch array ainv ($stride_ainv \geq \max(1, ldainv * n)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by the out-of-place, stride version of getri_batch_scratchpad_size (Strided Version) function.

Output Parameters

ainv	Overwritten by the n-by-n matrices A_i^{-1} .
------	---

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non-zero, then the passed <code>scratchpad</code> is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

getri_batch (Out-of-place, USM Strided Version)

Computes the batch of inverses of an LU-factored general matrices determined by `getrf_batch` (USM stride version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the inverses A_i^{-1} of a general matrices A_i . Before calling this routine, call [getrf_batch \(USM Strided Version\)](#) function to factorize A_i .

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getri_batch(sycl::queue &queue,
        int64_t n,
        T *a,
        int64_t lda,
        int64_t stride_a,
        const int64_t *ipiv,
        int64_t stride_ipiv,
        T* ainv,
        int64_t ldainv,
        int64_t stride_ainv,
        int64_t batch_size,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU

T	Devices supported
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
n	The order of the matrices A_i ($n \geq 0$).
a	Result of the getrf_batch (USM Strided Version) function
lda	The leading dimension of A_i ($lda \geq \max(1, n)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).
ipiv	The array as returned by getrf_batch (USM Strided Version) .
stride_ipiv	The stride between the beginnings of arrays $ipiv_i$ inside the array ipiv ($stride_ipiv \geq \max(1, n)$).
ldainv	The leading dimension of A_{inv_i} ($ldainv \geq \max(1, n)$).
stride_ainv	The stride between the beginnings of matrices A_{inv_i} inside the batch array ainv ($stride_ainv \geq \max(1, ldainv * n)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by the out-of-place, stride version of getri_batch_scratchpad_size (Strided Version) function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

ainv	Overwritten by the n-by-n matrices A_i^{-1} .
------	---

Exceptions

Exception	Description
mk1::lapack::batch_exception	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the info() method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value.

Exception	Description
	If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

getri_batch_scratchpad_size (Strided Version)

Computes size of scratchpad memory required for `getri_batch` (Out-of-place, Strided Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `getri_batch` (Out-of-place, Strided Version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t getri_batch_scratchpad_size(sycl::queue &queue,
        int64_t n,
        int64_t lda,
        int64_t stride_a,
        int64_t stride_ipiv,
        int64_t ldainv,
        int64_t stride_ainv,
        int64_t batch_size)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>n</code>	The order of the matrices A_i ($n \geq 0$).
<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, n)$).
<code>stride_a</code>	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
<code>stride_ipiv</code>	The stride between the beginnings of arrays $ipiv_i$ inside the array <code>ipiv</code> ($stride_ipiv \geq \max(1, n)$).
<code>ldainv</code>	The leading dimension of A_{inv_i} ($ldainv \geq \max(1, n)$).

<code>stride_ainv</code>	The stride between the beginnings of matrices A_{inv_i} inside the batch array <code>ainv</code> ($\text{stride_ainv} \geq \max(1, \text{ldainv} * n)$).
<code>batch_size</code>	The number of problems in a batch ($\text{batch_size} \geq 0$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `getri_batch` (Out-of-place, Strided Version) function should be able to hold.

getri_scratchpad_size

Computes size of scratchpad memory required for `getri` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `getri` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t getri_scratchpad_size(sycl::queue &queue,
        int64_t n,
        int64_t lda)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>getri</code> (buffer or USM version) function will be performed.
<code>n</code>	The order of the matrix A ($0 \leq n$).
<code>lda</code>	The leading dimension of a ($\text{lda} \geq n$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `getri` (buffer or USM version) function must be able to hold.

getrs

Solves a system of linear equations with an LU-factored square coefficient matrix, with multiple right-hand sides. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X the following systems of linear equations:

Equation	Condition
$A * X = B$	if <code>trans=mkl::transpose::nontrans</code>
$A^T * X = B$	if <code>trans=mkl::transpose::trans</code>
$A^H * X = B$	if <code>trans=mkl::transpose::conjtrans</code>

Before calling this routine, you must call [getrf](#) to compute the LU factorization of A .

API

Syntax

```
namespace oneapi::mkl::lapack {
    void getsr(sycl::queue &queue,
               mkl::transpose trans,
               int64_t n,
               int64_t nrhs,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<T> &ipiv,
               sycl::buffer<T> &b,
               int64_t ldb,
               sycl::buffer<T> &scratchpad,
               int64_t scratchpad_size)
}
```

`getrs` supports the following precisions and devices:

T	Devices supported
float	CPU and GPU

T	Devices supported
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
trans	Indicates the form of the equations: If <code>trans=mkl::transpose::nontrans</code> , then $A * X = B$ is solved for X . If <code>trans=mkl::transpose::trans</code> , then $A^T * X = B$ is solved for X . If <code>trans=mkl::transpose::conjtrans</code> , then $A^H * X = B$ is solved for X .
n	The order of the matrix A and the number of rows in matrix B ($0 \leq n$).
nrhs	The number of right-hand sides ($0 \leq nrhs$).
a	Buffer holding the array containing the factorization of the matrix A , as returned by getrf . The size of the buffer must be at least <code>`lda*n</code> .
lda	The leading dimension of a .
ipiv	Buffer holding array of size at least <code>max(1, n)</code> as returned by getrf .
b	The array b contains the matrix B whose columns are the right-hand sides for the systems of equations. The size of b must be at least <code>ldb*nrhs</code> .
ldb	The leading dimension of b .
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by the getrs_scratchpad_size function.

Output Parameters

b	The buffer b is overwritten by the solution matrix X .
---	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info = i</code>, the i-th diagonal element of U is zero, and the solve could not be completed.

Exception	Description
	If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

getrs (USM Version)

Solves a system of linear equations with an LU-factored square coefficient matrix, with multiple right-hand sides. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for x the following systems of linear equations:

Equation	Condition
$A * X = B$	if <code>trans=mkl::transpose::nontrans</code>
$A^T * X = B$	if <code>trans=mkl::transpose::trans</code>
$A^H * X = B$	if <code>trans=mkl::transpose::conjtrans</code>

Before calling this routine, you must call the [getrf \(USM Version\)](#) function to compute the LU factorization of A .

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getrs(sycl::queue &queue,
        mkl::transpose trans,
        int64_t n,
        int64_t nrhs,
        const T *a,
        int64_t lda,
        const int64_t *ipiv,
        T *b,
        int64_t ldb,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`getrs` (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU, GPU

T	Devices supported
double	CPU, GPU
std::complex<float>	CPU, GPU
std::complex<double>	CPU, GPU

Input Parameters

queue	Device queue where calculations will be performed.
trans	Indicates the form of the equations: If <code>trans=mkl::transpose::nontrans</code> , then $A * X = B$ is solved for X . If <code>trans=mkl::transpose::trans</code> , then $A^T * X = B$ is solved for X . If <code>trans=mkl::transpose::conjtrans</code> , then $A^H * X = B$ is solved for X .
n	The order of the matrix A and the number of rows in matrix B ($0 \leq n$).
nrhs	The number of right-hand sides ($0 \leq nrhs$).
a	Pointer to the array containing the factorization of the matrix A , as returned by getrf (USM Version) . The size of the array must be at least <code>lda*n</code> .
lda	The leading dimension of a .
ipiv	Array, size at least <code>max(1, n)</code> . The <code>ipiv</code> array, as returned by getrf (USM Version) .
b	The array b contains the matrix B whose columns are the right-hand sides for the systems of equations. The size of b must be at least <code>ldb*nrhs</code> .
ldb	The leading dimension of b .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by the gets_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

b	The array b is overwritten by the solution matrix X .
-----	---

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code> , the i -th parameter had an illegal value.

Exception	Description
	If <code>info = i</code>, the <i>i</i>-th diagonal element of U is zero, and the solve could not be completed. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

getrs_batch (Buffer Strided Version)

Solves a batch of linear equations with an LU-factored square coefficient matrices, with multiple right-hand sides. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X_i the following systems of linear equations:

- $A_i * X_i = B_i$ If `trans = mkl::transpose::notrans`
- $A_i^T * X_i = B_i$ If `trans = mkl::transpose::trans`
- $A_i^H * X_i = B_i$ If `trans = mkl::transpose::conjtrans`

Before calling this routine you must call [getrf_batch \(Buffer Strided Version\)](#) to compute the LU factorization of A_i .

API

Syntax

```
namespace oneapi::mkl::lapack {
    void getsr_batch(sycl::queue &queue,
        mkl::transpose trans,
        int64_t n,
        int64_t nrhs,
        sycl::buffer<T> &a,
        int64_t lda,
        int64_t stride_a,
        sycl::buffer<int64_t> &ipiv,
        int64_t stride_ipiv,
        sycl::buffer<T> &b,
        int64_t ldb,
        int64_t stride_b,
        int64_t batch_size,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
trans	Indicates the form of the equations: If <code>trans = mkl::transpose::nontrans</code> , then $A_i * X_i = B_i$ is solved for X_i . If <code>trans = mkl::transpose::trans</code> , then $A_i^T * X_i = B_i$ is solved for X_i . If <code>trans = mkl::transpose::conjtrans</code> , then $A_i^H * X_i = B_i$ is solved for X_i .
n	The order of the matrices A_i and the number of rows in matrices B_i ($n \geq 0$).
nrhs	The number of right hand sides (<code>nrhs</code> ≥ 0).
a	Array containing the factorizations of the matrices A_i , as returned by getrf_batch (Buffer Strided Version) .
lda	The leading dimension of A_i (<code>lda</code> $\geq \max(1, n)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> (<code>stride_a</code> $\geq \max(1, lda * n)$).
ipiv	The ipiv array, as returned by getrf_batch (Buffer Strided Version) .
stride_ipiv	The stride between the beginnings of arrays <code>ipiv_i</code> inside the array <code>ipiv</code> (<code>stride_ipiv</code> $\geq \max(1, n)$).
b	The array containing the matrices B_i whose columns are the right-hand sides for the systems of equations.
ldb	The leading dimensions of B_i (<code>ldb</code> $\geq \max(1, n)$).
stride_b	The stride between the beginnings of matrices B_i inside the batch array <code>b</code> (<code>stride_b</code> $\geq \max(1, ldb * nrhs)$).
batch_size	The number of problems in a batch (<code>batch_size</code> ≥ 0).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by stride version of getrs_batch_scratchpad_size (Strided Version) function.

Output Parameters

b	Overwritten by the solution matrices X_i .
---	--

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non-zero, then the passed <code>scratchpad</code> is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

getrs_batch (Group Version)

Solves a batch of systems of linear equations with a batch of LU-factored square coefficient matrices, with multiple right-hand sides. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X_i ($i \in \{1 \dots \text{batch_size}\}$) the following systems of linear equations:

- $A_i * X_i = B_i$ If `trans = mkl::transpose::notrans`
- $A_i^T * X_i = B_i$ If `trans = mkl::transpose::trans`
- $A_i^H * X_i = B_i$ If `trans = mkl::transpose::conjtrans`

Before calling this routine you must call [getrf_batch \(Group Version\)](#) to compute the LU factorization of A_i .

The total number of problems to solve, `batch_size`, is a sum of sizes of all of the groups of parameters as provided by `group_sizes` array.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getrs_batch(sycl::queue &queue,
        mkl::transpose *trans,
        int64_t *n,
        int64_t *nrhs,
        const T * const *a,
        int64_t *lda,
        const int64_t * const *ipiv,
        T **b,
```

```

int64_t *ldb,
int64_t group_count,
int64_t *group_sizes,
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
}

```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
trans	Array of <code>group_count</code> parameters <code>trans_g</code> indicating the form of the equations for the group <code>g</code>: If <code>trans = mkl::transpose::nontrans</code>, then $A_i * X_i = B_i$ is solved for X_i. If <code>trans = mkl::transpose::trans</code>, then $A_i^T * X_i = B_i$ is solved for X_i. If <code>trans = mkl::transpose::conjtrans</code>, then $A_i^H * X_i = B_i$ is solved for X_i.
n	Array of <code>group_count</code> parameters <code>n_g</code> specifying the order of the matrices A_i and the number of rows in matrices B_i belonging to group <code>g</code> ($n_g \geq 0$).
nrhs	Array of <code>group_count</code> parameters <code>nrhs_g</code> specifying the number of right hand sides for group <code>g</code> (<code>nrhs_g</code> ≥ 0).
a	Array of <code>batch_size</code> pointers to factorization of the matrices A_i , as returned by getrf_batch (Group Version) . The pointers must be device-accessible.
lda	Array of <code>group_count</code> parameters <code>lda_g</code> specifying the leading dimension of A_i from group <code>g</code> (<code>lda_g</code> $\geq \max(1, n_g)$).
ipiv	The <code>ipiv</code> array, as returned by getrf_batch (Group Version) . The pointers must be device-accessible.
b	The array containing <code>batch_size</code> pointers to the matrices B_i whose columns are the right-hand sides for the systems of equations. The pointers must be device-accessible.
ldb	Array of <code>group_count</code> parameters <code>ldb_g</code> specifying the leading dimensions of B_i in the group <code>g</code> (<code>ldb_g</code> $\geq \max(1, n_g)$).
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index <code>g</code> specifies the number of problems to solve for each of the groups of parameters <code>g</code> . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.

scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less then the value returned by getrs_batch_scratchpad_size (Group Version) .
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

b	Overwritten by the solution matrices X_i .
---	--

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

[getrs_batch \(USM Strided Version\)](#)

Solves a system of linear equations with a batch of LU-factored square coefficient matrices, with multiple right-hand sides. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X_i the following systems of linear equations:

- $A_i * X_i = B_i$ If `trans = mkl::transpose::notrans`
- $A_i^T * X_i = B_i$ If `trans = mkl::transpose::trans`

- $A_i^H * X_i = B_i$ If `trans = mkl::transpose::conjtrans`

Before calling this routine you must call [getrf_batch \(USM Strided Version\)](#) to compute the LU factorization of A_i .

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getrs_batch(sycl::queue &queue,
        mkl::transpose trans,
        int64_t n,
        int64_t nrhs,
        const T *a,
        int64_t lda,
        int64_t stride_a,
        const int64_t *ipiv,
        int64_t stride_ipiv,
        T *b,
        int64_t ldb,
        int64_t stride_b,
        int64_t batch_size,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
trans	Indicates the form of the equations: If <code>trans = mkl::transpose::nontrans</code> , then $A_i * X_i = B_i$ is solved for X_i . If <code>trans = mkl::transpose::trans</code> , then $A_i^T * X_i = B_i$ is solved for X_i . If <code>trans = mkl::transpose::conjtrans</code> , then $A_i^H * X_i = B_i$ is solved for X_i .
n	The order of the matrices A_i and the number of rows in matrices B_i ($n \geq 0$).
nrhs	The number of right hand sides ($nrhs \geq 0$).
a	Array containing the factorizations of the matrices A_i , as returned by getrf_batch (USM Strided Version) .
lda	The leading dimension of A_i ($lda \geq \max(1, n)$).

<code>stride_a</code>	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> (<code>stride_a</code> $\geq \max(1, \text{lda} * n)$).
<code>ipiv</code>	The <code>ipiv</code> array, as returned by getrf_batch (USM Strided Version) .
<code>stride_ipiv</code>	The stride between the beginnings of arrays <code>ipiv_i</code> inside the array <code>ipiv</code> (<code>stride_ipiv</code> $\geq \max(1, n)$).
<code>b</code>	The array containing the matrices B_i whose columns are the right-hand sides for the systems of equations.
<code>ldb</code>	The leading dimensions of B_i (<code>ldb</code> $\geq \max(1, n)$).
<code>stride_b</code>	The stride between the beginnings of matrices B_i inside the batch array <code>b</code> (<code>stride_b</code> $\geq \max(1, \text{ldb} * \text{nrhs})$).
<code>batch_size</code>	The number of problems in a batch (<code>batch_size</code> ≥ 0).
<code>scratchpad</code>	Scratchpad memory to be used by routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less then the value returned by stride version of getrs_batch_scratchpad_size (Strided Version) function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

<code>b</code>	Overwritten by the solution matrices X_i .
----------------	--

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info</code> = <code>-n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

getrs_batch_scratchpad_size (Group Version)

Computes size of scratchpad memory required for `getrs_batch` (Group Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `getrs_batch` (Group Version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t getsr_batch_scratchpad_size(sycl::queue &queue,
        mkl::transpose *trans,
        int64_t *n,
        int64_t *nrhs,
        int64_t *lda,
        int64_t *ldb,
        int64_t group_count,
        int64_t *group_sizes)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
trans	Array of <code>group_count</code> parameters <code>trans_g</code> indicating the form of the equations for the group <code>g</code>: If <code>trans = mkl::transpose::nontrans</code>, then $A_i * X_i = B_i$ is solved for X_i. If <code>trans = mkl::transpose::trans</code>, then $A_i^T * X_i = B_i$ is solved for X_i. If <code>trans = mkl::transpose::conjtrans</code>, then $A_i^H * X_i = B_i$ is solved for X_i.
n	Array of <code>group_count</code> parameters <code>n_g</code> specifying the order of the matrices A_i and the number of rows in matrices B_i belonging to group <code>g</code> ($n_g \geq 0$).

<code>nrhs</code>	Array of <code>group_count</code> parameters <code>nrhs_g</code> specifying the number of right hand sides for group <code>g</code> ($\text{nrhs}_g \geq 0$).
<code>lda</code>	Array of <code>group_count</code> parameters <code>lda_g</code> specifying the leading dimension of A_i from group <code>g</code> ($\text{lda}_g \geq \max(1, n_g)$).
<code>ldb</code>	Array of <code>group_count</code> parameters <code>ldb_g</code> specifying the leading dimension of B_i from group <code>g</code> ($\text{ldb}_g \geq \max(1, n_g)$).
<code>group_count</code>	Specifies the number of groups of parameters. Must be at least 0.
<code>group_sizes</code>	Array of <code>group_count</code> integers. Array element with index <code>g</code> specifies the number of problems to solve for each of the groups of parameters <code>g</code> . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the [getrs_batch \(Group Version\)](#) function must be able to hold.

getrs_batch_scratchpad_size (Strided Version)

Computes size of scratchpad memory required for `getrs_batch (Strided Version)` function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `getrs_batch (Strided Version)` function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t getsr_batch_scratchpad_size(sycl::queue &queue,
        mkl::transpose trans,
        int64_t n,
        int64_t nrhs,
        int64_t lda,
        int64_t stride_a,
        int64_t stride_ipiv,
        int64_t ldb,
```



```
int64_t stride_b,
int64_t batch_size)
}
```

Input Parameters

queue	Device queue where calculations will be performed.
trans	Indicates the form of the equations: If <code>trans = mkl::transpose::nontrans</code> , then $A_i * X_i = B_i$ is solved for X_i . If <code>trans = mkl::transpose::trans</code> , then $A_i^T * X_i = B_i$ is solved for X_i . If <code>trans = mkl::transpose::conjtrans</code> , then $A_i^H * X_i = B_i$ is solved for X_i .
n	The order of the matrices A_i and the number of rows in matrices B_i ($n \geq 0$).
nrhs	The number of right hand sides ($nrhs \geq 0$).
lda	The leading dimension of A_i ($lda \geq \max(1, n)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
stride_ipiv	The stride between the beginnings of arrays <code>ipiv_i</code> inside the array <code>ipiv</code> ($stride_ipiv \geq \max(1, n)$).
ldb	The leading dimensions of B_i ($ldb \geq \max(1, n)$).
stride_b	The stride between the beginnings of matrices B_i inside the batch array <code>b</code> ($stride_b \geq \max(1, ldb * nrhs)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `getrs_batch` (Strided Version) function must be able to hold.

getrs_scratchpad_size

Computes size of scratchpad memory required for `getrs` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type T the scratchpad memory to be passed to the `getrs` (buffer or USM version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t getrs_scratchpad_size(sycl::queue &queue,
        mkl::transpose trans,
        int64_t n,
        int64_t nrhs,
        int64_t lda,
        int64_t ldb)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>getrs</code> (buffer or USM version) function will be performed.
<code>trans</code>	Indicates the form of the equations: If <code>trans=mkl::transpose::nontrans</code> , then $A * X = B$ is solved for X . If <code>trans=mkl::transpose::trans</code> , then $A^T * X = B$ is solved for X . If <code>trans=mkl::transpose::conjtrans</code> , then $A^H * X = B$ is solved for X .
<code>n</code>	The order of the matrix A and the number of rows in matrix B ($0 \leq n$).
<code>nrhs</code>	The number of right-hand sides ($0 \leq nrhs$).
<code>lda</code>	The leading dimension of a .
<code>ldb</code>	The leading dimension of b .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `getrs` (buffer or USM version) function must be able to hold.

`getrsnp_batch` (Buffer Strided Version)

Solves a batch of linear equations with an LU-factored square coefficient matrices, with multiple right-hand sides. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X_i the following systems of linear equations:

- $A_i * X_i = B_i$ If `trans = mkl::transpose::notrans`
- $A_i^T * X_i = B_i$ If `trans = mkl::transpose::trans`
- $A_i^H * X_i = B_i$ If `trans = mkl::transpose::conjtrans`

Before calling this routine you must call [getrfnp_batch \(Buffer Strided Version\)](#) (without pivoting) to compute the LU factorization of A_i .

API

Syntax

```
namespace oneapi::mkl::lapack {
    void getrsnp_batch(sycl::queue &queue,
        mkl::transpose trans,
        int64_t n,
        int64_t nrhs,
        sycl::buffer<T> &a,
        int64_t lda,
        int64_t stride_a,
        sycl::buffer<T> &b,
        int64_t ldb,
        int64_t stride_b,
        int64_t batch_size,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
trans	Indicates the form of the equations: If <code>trans = mkl::transpose::nontrans</code> , then $A_i * X_i = B_i$ is solved for X_i . If <code>trans = mkl::transpose::trans</code> , then $A_i^T * X_i = B_i$ is solved for X_i . If <code>trans = mkl::transpose::conjtrans</code> , then $A_i^H * X_i = B_i$ is solved for X_i .
n	The order of the matrices A_i and the number of rows in matrices B_i ($n \geq 0$).
nrhs	The number of right hand sides ($nrhs \geq 0$).

a	Array containing the factorizations of the matrices A_i , as returned by getrfnp_batch (Buffer Strided Version) .
lda	The leading dimension of A_i ($lda \geq \max(1, n)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).
b	The array containing the matrices B_i whose columns are the right-hand sides for the systems of equations.
ldb	The leading dimensions of B_i ($ldb \geq \max(1, n)$).
stride_b	The stride between the beginnings of matrices B_i inside the batch array b ($stride_b \geq \max(1, ldb * nrhs)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less then the value returned by stride version of getrsnp_batch_scratchpad_size (Strided Version) function.

Output Parameters

b	Overwritten by the solution matrices X_i .
---	--

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

[getrsnp_batch \(USM Strided Version\)](#)

Solves a system of linear equations with a batch of LU-factored square coefficient matrices, with multiple right-hand sides. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X_i the following systems of linear equations:

- $A_i * X_i = B_i$ If `trans = mkl::transpose::notrans`
- $A_i^T * X_i = B_i$ If `trans = mkl::transpose::trans`
- $A_i^H * X_i = B_i$ If `trans = mkl::transpose::conjtrans`

Before calling this routine you must call [getrfnp_batch \(USM Strided Version\)](#) (without pivoting) to compute the LU factorization of A_i .

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event getrsnp_batch(sycl::queue &queue,
        mkl::transpose trans,
        int64_t n,
        int64_t nrhs,
        const T *a,
        int64_t lda,
        int64_t stride_a,
        T *b,
        int64_t ldb,
        int64_t stride_b,
        int64_t batch_size,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

- queue** Device queue where calculations will be performed.
- trans** Indicates the form of the equations:
- If `trans = mkl::transpose::nontrans`, then $A_i * X_i = B_i$ is solved for X_i .
- If `trans = mkl::transpose::trans`, then $A_i^T * X_i = B_i$ is solved for X_i .
- If `trans = mkl::transpose::conjtrans`, then $A_i^H * X_i = B_i$ is solved for X_i .

<code>n</code>	The order of the matrices A_i and the number of rows in matrices B_i ($n \geq 0$).
<code>nrhs</code>	The number of right hand sides ($nrhs \geq 0$).
<code>a</code>	Array containing the factorizations of the matrices A_i , as returned by getrf_batch (USM Strided Version) .
<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, n)$).
<code>stride_a</code>	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
<code>b</code>	The array containing the matrices B_i whose columns are the right-hand sides for the systems of equations.
<code>ldb</code>	The leading dimensions of B_i ($ldb \geq \max(1, n)$).
<code>stride_b</code>	The stride between the beginnings of matrices B_i inside the batch array <code>b</code> ($stride_b \geq \max(1, ldb * nrhs)$).
<code>batch_size</code>	The number of problems in a batch ($batch_size \geq 0$).
<code>scratchpad</code>	Scratchpad memory to be used by routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less then the value returned by stride version of getrsnp_batch_scratchpad_size (Strided Version) function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
<code>b</code>	Overwritten by the solution matrices X_i .

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception

Exception	Description
	object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

getrsnp_batch_scratchpad_size (Strided Version)

Computes size of scratchpad memory required for `getrsnp_batch` (Strided Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `getrsnp_batch` (Strided Version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t getrsnp_batch_scratchpad_size(sycl::queue &queue,
        mkl::transpose trans,
        int64_t n,
        int64_t nrhs,
        int64_t lda,
        int64_t stride_a,
        int64_t ldb,
        int64_t stride_b,
        int64_t batch_size)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>trans</code>	Indicates the form of the equations: If <code>trans = mkl::transpose::nontrans</code> , then $A_i * X_i = B_i$ is solved for X_i . If <code>trans = mkl::transpose::trans</code> , then $A_i^T * X_i = B_i$ is solved for X_i . If <code>trans = mkl::transpose::conjtrans</code> , then $A_i^H * X_i = B_i$ is solved for X_i .
<code>n</code>	The order of the matrices A_i and the number of rows in matrices B_i ($n \geq 0$).
<code>nrhs</code>	The number of right hand sides ($nrhs \geq 0$).
<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, n)$).

stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($\text{stride_a} \geq \max(1, \text{lda} * n)$).
ldb	The leading dimensions of B_i ($\text{ldb} \geq \max(1, n)$).
stride_b	The stride between the beginnings of matrices B_i inside the batch array b ($\text{stride_b} \geq \max(1, \text{ldb} * \text{nrhs})$).
batch_size	The number of problems in a batch ($\text{batch_size} \geq 0$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `getrsnp_batch` (Strided Version) function must be able to hold.

heevd

Computes all eigenvalues and, optionally, all eigenvectors of a complex Hermitian matrix using divide and conquer algorithm. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes all the eigenvalues, and optionally all the eigenvectors, of a complex Hermitian matrix A . In other words, it can compute the spectral factorization of A as: $A = Z \Lambda Z^H$.

Here Λ is a real diagonal matrix whose diagonal elements are the eigenvalues λ_i , and Z is the (complex) unitary matrix whose columns are the eigenvectors z_i . Thus,

$$A * z_i = \lambda_i * z_i \text{ for } i = 1, 2, \dots, n.$$

If the eigenvectors are requested, then this routine uses a divide and conquer algorithm to compute eigenvalues and eigenvectors. However, if only eigenvalues are required, then it uses the Pal-Walker-Kahan variant of the QL or QR algorithm.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void heevd(sycl::queue &queue,
              mkl::job jobz,
              mkl::uplo uplo,
              int64_t n,
              sycl::buffer<T> &a,
              int64_t lda,
```



```
sycl::buffer<RT> &w,
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}
```

heevd supports the following precision and devices.

T	RT	Devices Supported
std::complex<float>	float	CPU and GPU*
std::complex<double>	double	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
jobz	Must be <code>jobz::novec</code> or <code>jobz::vec</code> . If <code>jobz = jobz::novec</code> , then only eigenvalues are computed. If <code>jobz = jobz::vec</code> , then eigenvalues and eigenvectors are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , <code>a</code> stores the upper triangular part of <code>A</code> . If <code>uplo = uplo::lower</code> , <code>a</code> stores the lower triangular part of <code>A</code> .
n	The order of the matrix <code>A</code> ($0 \leq n$).
a	Buffer holding the array containing <code>A</code> . The size of the buffer must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the heevd_scratchpad_size function.

Output Parameters

a	If <code>jobz = jobz::vec</code> , then on exit this buffer is overwritten by the unitary matrix <code>Z</code> which contains the eigenvectors of <code>A</code> .
w	Buffer holding array of size at least <code>n</code> . If <code>info = 0</code> , contains the eigenvalues of the matrix <code>A</code> in ascending order. See also <code>info</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge; <i>i</i> indicates the number of off-diagonal elements of an intermediate tridiagonal form which did not converge to zero. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue while working on the submatrix lying in rows and columns <code>info/(n+1)</code> through <code>mod(info,n+1)</code>. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

heevd (USM Version)

Computes all eigenvalues and, optionally, all eigenvectors of a complex Hermitian matrix using divide and conquer algorithm. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes all the eigenvalues, and optionally all the eigenvectors, of a complex Hermitian matrix *A*. In other words, it can compute the spectral factorization of *A* as: $A = Z \Lambda Z^H$.

Here Λ is a real diagonal matrix whose diagonal elements are the eigenvalues λ_i , and *Z* is the orthogonal matrix whose columns are the eigenvectors z_i . Thus,

$$A * z_i = \lambda_i * z_i \text{ for } i = 1, 2, \dots, n.$$

If the eigenvectors are requested, then this routine uses a divide and conquer algorithm to compute eigenvalues and eigenvectors. However, if only eigenvalues are required, then it uses the Pal-Walker-Kahan variant of the QL or QR algorithm.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event heevd(sycl::queue &queue,
        mkl::job jobz,
        mkl::uplo uplo,
        int64_t n,
        T *a,
```

```
int64_t lda,
RT *w,
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
```

heevd (USM version) supports the following precision and devices.

T	RT	Devices Supported
<code>std::complex<float></code>	float	CPU and GPU*
<code>std::complex<double></code>	double	CPU and GPU^

*Interface support only; all computations are performed on the CPU.

^Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
jobz	Must be <code>job::novec</code> or <code>job::vec</code> . If <code>jobz = job::novec</code> , then only eigenvalues are computed. If <code>jobz = job::vec</code> , then eigenvalues and eigenvectors are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , a stores the upper triangular part of A. If <code>uplo = uplo::lower</code> , a stores the lower triangular part of A.
n	The order of the matrix A ($0 \leq n$).
a	The pointer to the array containing A. The size of the array must be at least <code>lda*n</code> .
lda	The leading dimension of a. Must be at least <code>max(1, n)</code> .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by the heevd_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	If <code>jobz = job::vec</code> , then on exit this is overwritten by the orthogonal matrix Z which contains the eigenvectors of A.
w	Pointer to array of size at least n. Contains the eigenvalues of the matrix A in ascending order.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge; <i>i</i> indicates the number of off-diagonal elements of an intermediate tridiagonal form which did not converge to zero. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue while working on the submatrix lying in rows and columns <code>info/(n+1)</code> through <code>mod(info,n+1)</code>. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

heevd_scratchpad_size

Computes size of scratchpad memory required for `heevd` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `heevd` (buffer or USM version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    heevd_scratchpad_size(sycl::queue &queue,
        mkl::job jobz,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda)
}
```

Input Parameters

queue	Device queue where calculations by the heevd (buffer or USM version) function will be performed.
jobz	Must be <code>jobz::novec</code> or <code>jobz::vec</code> . If <code>jobz = jobz::novec</code> , then only eigenvalues are computed. If <code>jobz = jobz::vec</code> , then eigenvalues and eigenvectors are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , <code>a</code> stores the upper triangular part of <code>A</code> . If <code>uplo = uplo::lower</code> , <code>a</code> stores the lower triangular part of <code>A</code> .
n	The order of the matrix <code>A</code> ($0 \leq n$).
lda	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `heevd` (buffer or USM version) function must be able to hold.

heevx

Computes selected eigenvalues and, optionally, eigenvectors of a complex Hermitian matrix `A`. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes selected eigenvalues, and optionally eigenvectors, of a complex Hermitian matrix `A`. Eigenvalues and eigenvectors can be selected by specifying either a range of values or a range of indices for the desired eigenvalues. In other words, it can compute selected eigenpairs (λ, z) of `A` such that $A * z = z * \lambda$.

The routine computes selected eigenvalues with the bisection algorithm.

If the eigenvectors are requested, then this routine uses a combination of modified twisted factorization algorithm based on Inderjit Dhillon and Beresford Parlett's work and the inverse iteration algorithm followed by Gram-Schmidt orthogonalization.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void heevx(sycl::queue &queue,
              mkl::job jobz,
              mkl::rangev range,
              mkl::uplo uplo,
              int64_t n,
              sycl::buffer<T> &a,
              int64_t lda,
              RT vl,
              RT vu,
              int64_t il,
              int64_t iu,
              RT abstol,
              sycl::buffer<int64_t> &m,
              sycl::buffer<RT> &w,
              sycl::buffer<T> &z,
              int64_t ldz,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

heevx supports the following precision and devices.

T	RT	Devices Supported
std::complex<float>	float	CPU and GPU*
std::complex<double>	double	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
jobz	Must be <code>job::novec</code> or <code>job::vec</code>. If <code>jobz = job::novec</code>, then only eigenvalues are computed. If <code>jobz = job::vec</code>, then eigenvalues and eigenvectors are computed.
range	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $[vl, vu]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::indices</code>, then the <code>il</code>-th through <code>iu</code>-th eigenvalues and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> stores the upper triangular part of <code>A</code>. If <code>uplo = uplo::lower</code>, <code>a</code> stores the lower triangular part of <code>A</code>.
n	The order of the matrix <code>A</code> ($0 \leq n$).

<code>a</code>	Buffer holding an array containing the upper or lower triangle of the Hermitian matrix A , as specified by <code>uplo</code> . The size of the array must be at least <code>lda*n</code> .
<code>lda</code>	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
<code>vl</code>	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>vu</code>	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>il</code>	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>iu</code>	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>abstol</code>	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval $[a, b]$ of width less than or equal to <code>abstol + eps * max(a , b)</code> , where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then <code>eps* T </code> will be used in its place, where $ T $ is the 1-norm of the tridiagonal matrix obtained by reducing A to tridiagonal form.
<code>ldz</code>	The leading dimension of <code>z</code> . Must be at least <code>max(1, n)</code> .
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the heevx_scratchpad_size function.

Output Parameters

<code>a</code>	On exit the upper triangle (if <code>uplo = uplo::upper</code>) or the lower triangle (if <code>uplo = uplo::lower</code>) of A , including the diagonal, is destroyed.
<code>m</code>	The total number of eigenvalues found, $0 \leq m \leq n$.
<code>w</code>	Buffer holding array of size at least <code>n</code> . On normal exit, the first <code>m</code> elements contain the selected eigenvalues in ascending order.
<code>z</code>	If <code>jobz = jobz::vec</code> , then on normal exit the first <code>m</code> columns of Z contain the orthonormal eigenvectors of the matrix A corresponding to the selected eigenvalues, with the i -th column of Z holding the eigenvector associated with <code>w(i)</code> . If <code>jobz = jobz::novec</code> , then Z is not referenced. Note: the user must ensure that at least <code>max(1, m)</code> columns are supplied in the array Z ; if <code>range = rangev::values</code> , the exact value of <code>m</code> is not known in advance and an upper bound must be used.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

heevx (USM Version)

Computes selected eigenvalues and, optionally, eigenvectors of a complex Hermitian matrix *A*. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes selected eigenvalues, and optionally eigenvectors, of a complex Hermitian matrix *A*. Eigenvalues and eigenvectors can be selected by specifying either a range of values or a range of indices for the desired eigenvalues. In other words, it can compute selected eigenpairs (λ, z) of *A* such that $A \cdot z = z \cdot \lambda$.

The routine computes selected eigenvalues with the bisection algorithm.

If the eigenvectors are requested, then this routine uses a combination of modified twisted factorization algorithm based on Inderjit Dhillon and Beresford Parlett's work and the inverse iteration algorithm followed by Gram-Schmidt orthogonalization.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event heevx(sycl::queue &queue,
        mkl::job jobz,
        mkl::rangev range,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        RT vl,
        RT vu,
        int64_t il,
```



```

int64_t iu,
RT abstol,
int64_t *m,
RT *w,
T *z,
int64_t ldz,
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
}

```

heevx (USM version) supports the following precision and devices.

T	RT	Devices Supported
std::complex<float>	float	CPU and GPU*
std::complex<double>	double	CPU and GPU^

*Interface support only; all computations are performed on the CPU.

^Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
jobz	Must be <code>jobz::novec</code> or <code>jobz::vec</code>. If <code>jobz = jobz::novec</code>, then only eigenvalues are computed. If <code>jobz = jobz::vec</code>, then eigenvalues and eigenvectors are computed.
range	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $[v_l, v_u]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::indices</code>, then the i_l-th through i_u-th eigenvalues and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> stores the upper triangular part of A. If <code>uplo = uplo::lower</code>, <code>a</code> stores the lower triangular part of A.
n	The order of the matrix A ($0 \leq n$).
a	Pointer to the array containing the upper or lower triangle of the Hermitian matrix A , as specified by <code>uplo</code> . The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
vl	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .

<code>vu</code>	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>il</code>	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>iu</code>	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>abstol</code>	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval $[a,b]$ of width less than or equal to $abstol + eps * \max(a , b)$, where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then $eps * T $ will be used in its place, where $ T $ is the 1-norm of the tridiagonal matrix obtained by reducing <code>A</code> to tridiagonal form.
<code>ldz</code>	The leading dimension of <code>z</code> . Must be at least $\max(1, n)$.
<code>scratchpad</code>	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the heevx_scratchpad_size function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
<code>a</code>	On exit the upper triangle (if <code>uplo = uplo::upper</code>) or the lower triangle (if <code>uplo = uplo::lower</code>) of <code>A</code> , including the diagonal, is destroyed.
<code>m</code>	The total number of eigenvalues found, $0 \leq m \leq n$.
<code>w</code>	Pointer to array of size at least <code>n</code> . On normal exit, the first <code>m</code> elements contain the selected eigenvalues of the matrix <code>A</code> in ascending order.
<code>z</code>	If <code>jobz = job::vec</code> , then on normal exit the first <code>m</code> columns of <code>Z</code> contain the orthonormal eigenvectors of the matrix <code>A</code> corresponding to the selected eigenvalues, with the <code>i</code> -th column of <code>Z</code> holding the eigenvector associated with <code>w(i)</code> . If <code>jobz = job::novec</code> , then <code>Z</code> is not referenced. Note: the user must ensure that at least $\max(1, m)$ columns are supplied in the array <code>Z</code> ; if <code>range = rangev::values</code> , the exact value of <code>m</code> is not known in advance and an upper bound must be used.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object:

Exception	Description
	If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

heevx_scratchpad_size

Computes size of scratchpad memory required for `heevx` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `heevx` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t heevx_scratchpad_size(sycl::queue &queue,
        mkl::job jobz,
        mkl::rangev range,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda,
        T::value_type vl,
        T::value_type vu,
        int64_t il,
        int64_t iu,
        T::value_type abstol,
        int64_t ldz)
}
```

Input Parameters

queue	Device queue where calculations by the heevx (buffer or USM version) function will be performed.
jobz	Must be <code>jobz::novec</code> or <code>jobz::vec</code>. If <code>jobz = jobz::novec</code>, then only eigenvalues are computed. If <code>jobz = jobz::vec</code>, then eigenvalues and eigenvectors are computed.
range	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $(v_l, v_u]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::indices</code>, then the il-th through iu-th eigenvalues and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> stores the upper triangular part of A. If <code>uplo = uplo::lower</code>, <code>a</code> stores the lower triangular part of A.
n	The order of the matrix A ($0 \leq n$).
lda	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
vl	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
vu	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
il	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
iu	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
abstol	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval $[a, b]$ of width less than or equal to <code>abstol + eps * max(a , b)</code> , where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then <code>eps* T </code> will be used in its place, where $ T $ is the 1-norm of the tridiagonal matrix obtained by reducing A to tridiagonal form.
ldz	The leading dimension of <code>z</code> . Must be at least <code>max(1, n)</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `heevx` (buffer or USM version) function must be able to hold.

hegvd

Computes all the eigenvalues, and optionally, the eigenvectors of a complex generalized Hermitian positive-definite eigenproblem using a divide and conquer method. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes all the eigenvalues, and optionally, the eigenvectors of a complex generalized Hermitian positive-definite eigenproblem, of the form

$A*x = \lambda*B*x$, $A*B*x = \lambda*x$, or $B*A*x = \lambda*x$.

Here A and B are assumed to be Hermitian and B is also positive definite.

It uses a divide and conquer algorithm.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void hegvd(sycl::queue &queue,
               int64_t itype,
               mkl::job jobz,
               mkl::uplo uplo,
               int64_t n,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<T> &b,
               int64_t ldb,
               sycl::buffer<RT> &w,
               sycl::buffer<T> &scratchpad,
               int64_t scratchpad_size)
}
```

`hegvd` supports the following precision and devices.

T	RT	Devices Supported
<code>std::complex<float></code>	float	CPU and GPU*
<code>std::complex<double></code>	double	CPU and GPU^

*Interface support only; all computations are performed on the CPU.

^Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
itype	Must be 1 or 2 or 3. Specifies the problem type to be solved: if itype= 1, the problem type is $A*x = \lambda*B*x$; if itype= 2, the problem type is $A*B*x = \lambda*x$; if itype= 3, the problem type is $B*A*x = \lambda*x$.
jobz	Must be <code>jobz::novec</code> or <code>jobz::vec</code> . If <code>jobz = jobz::novec</code> , then only eigenvalues are computed. If <code>jobz = jobz::vec</code> , then eigenvalues and eigenvectors are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , a and b store the upper triangular part of A and B. If <code>uplo = uplo::lower</code> , a and b stores the lower triangular part of A and B.
n	The order of the matrices A and B ($0 \leq n$).
a	Buffer holding the array containing the upper or lower triangle of the Hermitian matrix A, as specified by uplo. The size of a must be at least $lda*n$.
lda	The leading dimension of a; at least $\max(1, n)$.
b	Buffer holding the array containing the upper or lower triangle of the Hermitian matrix B, as specified by uplo. The size of b must be at least $ldb*n$.
ldb	The leading dimension of b; at least $\max(1, n)$.
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by the hegvd_scratchpad_size function.

Output Parameters

a	On exit, if <code>jobz = jobz::vec</code> , then if <code>info = 0</code> , a contains the matrix Z of eigenvectors. The eigenvectors are normalized as follows: if itype= 1 or 2, $Z^H*B*Z = I$; if itype= 3, $Z^H*inv(B)*Z = I$; If <code>jobz = jobz::novec</code> , then on exit the upper triangle (if <code>uplo = uplo::upper</code>) or the lower triangle (if <code>uplo = uplo::lower</code>) of A, including the diagonal, is destroyed.
b	On exit, if <code>info ≤ n</code> , the part of b containing the matrix is overwritten by the triangular factor U or L from the Cholesky factorization $B = U^H*U$ or $B = L*L^H$.

w Buffer holding array of size at least *n*. If *info* = 0, contains the eigenvalues of the matrix *A* in ascending order. See also *info*.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the <i>info</i> code of the problem using the <i>info()</i> method of the exception object: If <i>info</i> = -<i>i</i>, the <i>i</i>-th parameter had an illegal value. For <i>info</i> ≤ <i>n</i>: If <i>info</i> = <i>i</i>, and <i>jobz</i> = <i>job::novec</i>, then the algorithm failed to converge; <i>i</i> indicates the number of off-diagonal elements of an intermediate tridiagonal form which did not converge to zero. If <i>info</i> = <i>i</i>, and <i>jobz</i> = <i>job::vec</i>, then the algorithm failed to compute an eigenvalue while working on the submatrix lying in rows and columns <i>info</i>/(<i>n</i>+1) through <i>mod</i>(<i>info</i>,<i>n</i>+1). For <i>info</i> > <i>n</i>: If <i>info</i> = <i>n</i> + <i>i</i>, for 1 ≤ <i>i</i> ≤ <i>n</i>, then the leading minor of order <i>i</i> of <i>B</i> is not positive-definite. The factorization of <i>B</i> could not be completed and no eigenvalues or eigenvectors were computed. If <i>info</i> is equal to the value passed as <i>scratchpad</i> size, and <i>detail()</i> returns non zero, then the passed <i>scratchpad</i> has an insufficient size, and the required size should not be less than the value returned by the <i>detail()</i> method of the exception object.

hegvd (USM Version)

Computes all the eigenvalues, and optionally, the eigenvectors of a complex generalized Hermitian positive-definite eigenproblem using a divide and conquer method. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes all the eigenvalues, and optionally, the eigenvectors of a complex generalized Hermitian positive-definite eigenproblem, of the form

$$A*x = \lambda*B*x, \quad A*B*x = \lambda*x, \quad \text{or} \quad B*A*x = \lambda*x.$$

Here *A* and *B* are assumed to be Hermitian and *B* is also positive definite.

It uses a divide and conquer algorithm.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event hegyd(sycl::queue &queue,
        int64_t itype,
        mkl::job jobz,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        T *b,
        int64_t ldb,
        RT *w,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

hegyd (USM version) supports the following precision and devices.

T	RT	Devices Supported
std::complex<float>	float	CPU and GPU*
std::complex<double>	double	CPU and GPU^

*Interface support only; all computations are performed on the CPU.

^Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
itype	Must be 1 or 2 or 3. Specifies the problem type to be solved: if itype= 1, the problem type is $A*x = \lambda*B*x$; if itype= 2, the problem type is $A*B*x = \lambda*x$; if itype= 3, the problem type is $B*A*x = \lambda*x$.
jobz	Must be <code>job::novec</code> or <code>job::vec</code>. If <code>jobz = job::novec</code>, then only eigenvalues are computed. If <code>jobz = job::vec</code>, then eigenvalues and eigenvectors are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, a stores the upper triangular part of A. If <code>uplo = uplo::lower</code>, a stores the lower triangular part of A.
n	The order of the matrices A and B ($0 \leq n$).
a	Pointer to the array containing the upper or lower triangle of the Hermitian matrix A, as specified by uplo. The size of the array must be at least <code>lda*n</code> .
lda	The leading dimension of a; at least <code>max(1, n)</code> .

<code>b</code>	Pointer to the array containing the upper or lower triangle of the Hermitian matrix <code>B</code> , as specified by <code>uplo</code> . The size of the array must be at least <code>ldb*n</code> .
<code>ldb</code>	The leading dimension of <code>b</code> ; at least <code>max(1, n)</code> .
<code>scratchpad</code>	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the hegvd_scratchpad_size function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
<code>a</code>	On exit, if <code>jobz = job::vec</code> , then if <code>info = 0</code> , <code>a</code> contains the matrix <code>Z</code> of eigenvectors. The eigenvectors are normalized as follows: if <code>itype = 1</code> or <code>2</code> , $Z^H * B * Z = I$; if <code>itype = 3</code> , $Z^H * \text{inv}(B) * Z = I$; If <code>jobz = job::novec</code> , then on exit the upper triangle (if <code>uplo = uplo::upper</code>) or the lower triangle (if <code>uplo = uplo::lower</code>) of <code>A</code> , including the diagonal, is destroyed.
<code>b</code>	On exit, if <code>info ≤ n</code> , the part of <code>b</code> containing the matrix is overwritten by the triangular factor <code>U</code> or <code>L</code> from the Cholesky factorization $B = U^H * U$ or $B = L * L^H$.
<code>w</code>	Pointer to the array of size at least <code>n</code> . If <code>info = 0</code> , contains the eigenvalues of the matrix <code>A</code> in ascending order. See also <code>info</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the <code>info</code> code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. For <code>info ≤ n</code>: If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge; <code>i</code> indicates the number of off-diagonal elements of an intermediate tridiagonal form which did not converge to zero. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue while working on the submatrix lying in rows and columns <code>info/(n+1)</code> through <code>mod(info, n+1)</code>. For <code>info > n</code>:

Exception	Description
	If <code>info = n + i</code>, for $1 \leq i \leq n$, then the leading minor of order <code>i</code> of <code>B</code> is not positive-definite. The factorization of <code>B</code> could not be completed and no eigenvalues or eigenvectors were computed. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

hegvd_scratchpad_size

Computes size of scratchpad memory required for `hegvd` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `hegvd` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t hegvd_scratchpad_size(sycl::queue &queue,
        int64_t itype,
        mkl::job jobz,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda,
        int64_t ldb)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>hegvd</code> (buffer or USM version) function will be performed.
<code>itype</code>	Must be 1 or 2 or 3. Specifies the problem type to be solved: if <code>itype= 1</code>, the problem type is $A*x = \text{lambda}*B*x$; if <code>itype= 2</code>, the problem type is $A*B*x = \text{lambda}*x$; if <code>itype= 3</code>, the problem type is $B*A*x = \text{lambda}*x$.

<code>jobz</code>	Must be <code>job::novec</code> or <code>job::vec</code>. If <code>jobz = job::novec</code>, then only eigenvalues are computed. If <code>jobz = job::vec</code>, then eigenvalues and eigenvectors are computed.
<code>uplo</code>	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> and <code>b</code> store the upper triangular part of <code>A</code> and <code>B</code>. If <code>uplo = uplo::lower</code>, <code>a</code> and <code>b</code> stores the lower triangular part of <code>A</code> and <code>B</code>.
<code>n</code>	The order of the matrices <code>A</code> and <code>B</code> ($0 \leq n$).
<code>lda</code>	The leading dimension of <code>a</code> ; at least <code>max(1, n)</code> .
<code>ldb</code>	The leading dimension of <code>b</code> ; at least <code>max(1, n)</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `hegv`d (buffer or USM version) function must be able to hold.

hegvx

Computes selected eigenvalues and, optionally, eigenvectors of a complex generalized Hermitian positive-definite eigenproblem. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes selected eigenvalues, and optionally, eigenvectors of a complex generalized Hermitian positive-definite eigenproblem of the form

$$A^*x = \lambda B^*x, A^*B^*x = \lambda^*x, \text{ or } B^*A^*x = \lambda^*x.$$

Here `A` and `B` are assumed to be Hermitian and `B` is also positive definite. Eigenvalues and eigenvectors can be selected by specifying either a range of values or a range of indices for the desired eigenvalues.

The routine computes selected eigenvalues with the bisection algorithm.

If the eigenvectors are requested, then this routine uses a combination of modified twisted factorization algorithm based on Inderjit Dhillon and Beresford Parlett's work and the inverse iteration algorithm followed by Gram-Schmidt orthogonalization.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void hegvx(sycl::queue &queue,
        int64_t itype,
        mkl::job jobz,
        mkl::rangev range,
        mkl::uplo uplo,
        int64_t n,
        sycl::buffer<T> &a,
        int64_t lda,
        sycl::buffer<T> &b,
        int64_t ldb,
        RT vl,
        RT vu,
        int64_t il,
        int64_t iu,
        RT abstol,
        sycl::buffer<int64_t> &m,
        sycl::buffer<RT> &w,
        sycl::buffer<T> &z,
        int64_t ldz,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

hegvx supports the following precision and devices.

T	RT	Devices Supported
std::complex<float>	float	CPU and GPU*
std::complex<double>	double	CPU and GPU^

*Interface support only; all computations are performed on the CPU.

^Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
itype	Must be 1 or 2 or 3. Specifies the problem type to be solved: if itype= 1, the problem type is $A*x = \lambda*B*x$; if itype= 2, the problem type is $A*B*x = \lambda*x$; if itype= 3, the problem type is $B*A*x = \lambda*x$.
jobz	Must be <code>job::novec</code> or <code>job::vec</code>. If <code>jobz = job::novec</code>, then only eigenvalues are computed. If <code>jobz = job::vec</code>, then eigenvalues and eigenvectors are computed.
range	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $(vl, vu]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.

If `range = rangev::indices`, then the `il`-th through `iu`-th eigenvalues and corresponding eigenvectors (if requested by `jobz` parameters) are computed.

<code>uplo</code>	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> and <code>b</code> store the upper triangular part of <code>A</code> and <code>B</code>. If <code>uplo = uplo::lower</code>, <code>a</code> and <code>b</code> stores the lower triangular part of <code>A</code> and <code>B</code>.
<code>n</code>	The order of the matrices <code>A</code> and <code>B</code> ($0 \leq n$).
<code>a</code>	Buffer holding the array containing the upper or lower triangle of the Hermitian matrix <code>A</code> , as specified by <code>uplo</code> . The size of <code>a</code> must be at least <code>lda*n</code> .
<code>lda</code>	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
<code>b</code>	Buffer holding the array containing the upper or lower triangle of the Hermitian matrix <code>B</code> , as specified by <code>uplo</code> . The size of <code>b</code> must be at least <code>ldb*n</code> .
<code>ldb</code>	The leading dimension of <code>b</code> ; at least <code>max(1, n)</code> .
<code>vl</code>	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>vu</code>	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>il</code>	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if <code>n</code> > 0 and <code>il</code> = 1 and <code>iu</code> = 0 if <code>n</code> = 0. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>iu</code>	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if <code>n</code> > 0 and <code>il</code> = 1 and <code>iu</code> = 0 if <code>n</code> = 0. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>abstol</code>	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval <code>[a,b]</code> of width less than or equal to <code>abstol + eps * max(a , b)</code> , where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then <code>eps* T </code> will be used in its place, where <code> T </code> is the 1-norm of the tridiagonal matrix obtained by reducing <code>A</code> to tridiagonal form.
<code>ldz</code>	The leading dimension of <code>z</code> . Must be at least <code>max(1, n)</code> .
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the hegvx_scratchpad_size function.

Output Parameters

<code>a</code>	On exit the upper triangle (if <code>uplo = uplo::upper</code>) or the lower triangle (if <code>uplo = uplo::lower</code>) of <code>A</code> , including the diagonal, is destroyed.
----------------	--

b	On normal exit, the part of b containing the matrix is overwritten by the triangular factor U or L from the Cholesky factorization $B = U^T * U$ or $B = L * L^T$.
m	The total number of eigenvalues found, $0 \leq m \leq n$.
w	Buffer holding array of size at least n. On normal exit, the first m elements contain the selected eigenvalues in ascending order.
z	If <code>jobz = job::vec</code> , then on normal exit the first m columns of Z contain the orthonormal eigenvectors of the matrix A corresponding to the selected eigenvalues, with the i-th column of Z holding the eigenvector associated with <code>w(i)</code> . If <code>jobz = job::novec</code> , then Z is not referenced. Note: the user must ensure that at least <code>max(1, m)</code> columns are supplied in the array Z; if <code>range = rangev::values</code> , the exact value of m is not known in advance and an upper bound must be used.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

hegvx (USM Version)

Computes selected eigenvalues, and optionally, the eigenvectors of a complex generalized Hermitian positive-definite eigenproblem. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes selected eigenvalues, and optionally, eigenvectors of a complex generalized Hermitian positive-definite eigenproblem of the form

$$A * x = \lambda * B * x, \quad A * B * x = \lambda * x, \quad \text{or} \quad B * A * x = \lambda * x.$$

Here A and B are assumed to be Hermitian and B is also positive definite. Eigenvalues and eigenvectors can be selected by specifying either a range of values or a range of indices for the desired eigenvalues.

The routine computes selected eigenvalues with the bisection algorithm.

If the eigenvectors are requested, then this routine uses a combination of modified twisted factorization algorithm based on Inderjit Dhillon and Beresford Parlett's work and the inverse iteration algorithm followed by Gram-Schmidt orthogonalization.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event hegvx(sycl::queue &queue,
        int64_t itype,
        mkl::job jobz,
        mkl::rangev range,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        T *b,
        int64_t ldb,
        RT vl,
        RT vu,
        int64_t il,
        int64_t iu,
        RT abstol,
        int64_t *m,
        RT *w,
        T *z,
        int64_t ldz,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

hegvx (USM version) supports the following precision and devices.

T	RT	Devices Supported
std::complex<float>	float	CPU and GPU*
std::complex<double>	double	CPU and GPU^

*Interface support only; all computations are performed on the CPU.

^Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
itype	Must be 1 or 2 or 3. Specifies the problem type to be solved: if itype= 1, the problem type is $A*x = \lambda*B*x$; if itype= 2, the problem type is $A*B*x = \lambda*x$; if itype= 3, the problem type is $B*A*x = \lambda*x$.
jobz	Must be <code>job::novec</code> or <code>job::vec</code> . If <code>jobz = job::novec</code> , then only eigenvalues are computed.

	If <code>jobz = jobz::vec</code> , then eigenvalues and eigenvectors are computed.
range	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $(v_l, v_u]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::indices</code>, then the <code>il</code>-th through <code>iu</code>-th eigenvalues and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> stores the upper triangular part of A. If <code>uplo = uplo::lower</code>, <code>a</code> stores the lower triangular part of A.
n	The order of the matrices A and B ($0 \leq n$).
a	Pointer to the containing the upper or lower triangle of the Hermitian matrix A , as specified by <code>uplo</code> . The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
b	Pointer to the array containing the upper or lower triangle of the Hermitian matrix B , as specified by <code>uplo</code> . The size of <code>b</code> must be at least <code>ldb*n</code> .
ldb	The leading dimension of <code>b</code> ; at least <code>max(1, n)</code> .
vl	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
vu	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
il	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if <code>n</code> > 0 and <code>il</code> = 1 and <code>iu</code> = 0 if <code>n</code> = 0. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
iu	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if <code>n</code> > 0 and <code>il</code> = 1 and <code>iu</code> = 0 if <code>n</code> = 0. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
abstol	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval $[a, b]$ of width less than or equal to <code>abstol + eps * max(a , b)</code> , where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then <code>eps* T </code> will be used in its place, where <code> T </code> is the 1-norm of the tridiagonal matrix obtained by reducing A to tridiagonal form.
ldz	The leading dimension of <code>z</code> . Must be at least <code>max(1, n)</code> .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.

scratchpad_size Size of scratchpad memory as a number of floating point elements of type `T`. Size should not be less than the value returned by the [hegvx_scratchpad_size](#) function.

events List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a On exit the upper triangle (if `uplo = uplo::upper`) or the lower triangle (if `uplo = uplo::lower`) of A , including the diagonal, is destroyed.

b On normal exit, the part of b containing the matrix is overwritten by the triangular factor U or L from the Cholesky factorization $B = U^T * U$ or $B = L * L^T$.

m The total number of eigenvalues found, $0 \leq m \leq n$.

w Pointer to array of size at least n . On normal exit, the first m elements contain the selected eigenvalues of the matrix A in ascending order.

z If `jobz = jobz::vec`, then on normal exit the first m columns of Z contain the orthonormal eigenvectors of the matrix A corresponding to the selected eigenvalues, with the i -th column of Z holding the eigenvector associated with $w(i)$. If `jobz = jobz::novec`, then Z is not referenced. Note: the user must ensure that at least $\max(1, m)$ columns are supplied in the array Z ; if `range = rangev::values`, the exact value of m is not known in advance and an upper bound must be used.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info = i</code>, and <code>jobz = jobz::novec</code>, then the algorithm failed to converge. If <code>info = i</code>, and <code>jobz = jobz::vec</code>, then the algorithm failed to compute an eigenvalue. If <code>info</code> is equal to the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

[hegvx_scratchpad_size](#)

Computes size of scratchpad memory required for `hegvx` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `hegvx` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t hegvx_scratchpad_size(sycl::queue &queue,
        int64_t itype,
        mkl::job jobz,
        mkl::rangev range,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda,
        int64_t ldb,
        T::value_type vl,
        T::value_type vu,
        int64_t il,
        int64_t iu,
        T::value_type abstol,
        int64_t ldz)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>hegvx</code> (buffer or USM version) function will be performed.
<code>itype</code>	Must be 1 or 2 or 3. Specifies the problem type to be solved: if <code>itype= 1</code>, the problem type is $A*x = \lambda*B*x$; if <code>itype= 2</code>, the problem type is $A*B*x = \lambda*x$; if <code>itype= 3</code>, the problem type is $B*A*x = \lambda*x$.
<code>jobz</code>	Must be <code>job::novec</code> or <code>job::vec</code>. If <code>jobz = job::novec</code>, then only eigenvalues are computed. If <code>jobz = job::vec</code>, then eigenvalues and eigenvectors are computed.
<code>range</code>	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $[vl, vu]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::indices</code>, then the <code>il</code>-th through <code>iu</code>-th eigenvalues and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.

<code>uplo</code>	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , <code>a</code> and <code>b</code> store the upper triangular part of <code>A</code> and <code>B</code> . If <code>uplo = uplo::lower</code> , <code>a</code> and <code>b</code> stores the lower triangular part of <code>A</code> and <code>B</code> .
<code>n</code>	The order of the matrices <code>A</code> and <code>B</code> ($0 \leq n$).
<code>lda</code>	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
<code>ldb</code>	The leading dimension of <code>b</code> ; at least <code>max(1, n)</code> .
<code>vl</code>	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>vu</code>	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>il</code>	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>iu</code>	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>abstol</code>	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval $[a, b]$ of width less than or equal to <code>abstol + eps * max(a , b)</code> , where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then <code>eps* T </code> will be used in its place, where $ T $ is the 1-norm of the tridiagonal matrix obtained by reducing <code>A</code> to tridiagonal form.
<code>ldz</code>	The leading dimension of <code>z</code> . Must be at least <code>max(1, n)</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `hegvx` (buffer or USM version) function must be able to hold.

hetrd

Reduces a complex Hermitian matrix to tridiagonal form. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine reduces a complex Hermitian matrix A to symmetric tridiagonal form T by a unitary similarity transformation: $A = Q^* T Q$. The unitary matrix Q is not formed explicitly but is represented as a product of $n-1$ elementary reflectors. Routines are provided to work with Q in this representation.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event hetrd(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        sycl::buffer<T> &a,
        int64_t lda,
        sycl::buffer<T> &d,
        sycl::buffer<T> &e,
        sycl::buffer<T> &tau,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

hetrd supports the following precisions and devices:

T	Devices supported
std::complex<float>	CPU and GPU*
std::complex<double>	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	The device queue where calculations will be performed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , <code>a</code> stores the upper triangular part of A . If <code>uplo = uplo::lower</code> , <code>a</code> stores the lower triangular part of A .
n	The order of the matrices A ($0 \leq n$).
a	Buffer holding the matrix A , size $(lda, *)$. Contains the upper or lower triangle as specified by <code>uplo</code> .
lda	The leading dimension of <code>a</code> ; at least $\max(1, n)$
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the hetrd_scratchpad_size function.

Output Parameters

a	On exit, if <code>uplo = uplo::upper</code>, the diagonal and first superdiagonal of <code>A</code> are overwritten by the corresponding elements of the tridiagonal matrix <code>T</code>, and the elements above the first superdiagonal, with the buffer <code>tau</code>, represent the unitary matrix <code>Q</code> as a product of elementary reflectors; if <code>uplo = uplo::lower</code>, the diagonal and first subdiagonal of <code>A</code> are overwritten by the corresponding elements of the tridiagonal matrix <code>T</code>, and the elements below the first subdiagonal, with the array <code>tau</code>, represent the unitary matrix <code>Q</code> as a product of elementary reflectors.
d	Buffer holding the diagonal elements of the matrix <code>T</code> . The dimension of <code>d</code> must be at least <code>max(1, n)</code> .
e	Buffer holding off diagonal elements of the matrix <code>T</code> . The dimension of <code>e</code> must be at least <code>max(1, n-1)</code> .
tau	Buffer holding array of size at least <code>max(1, n)</code> . Stores <code>(n-1)</code> scalars that define elementary reflectors in decomposition of the unitary matrix <code>Q</code> in a product of <code>n-1</code> elementary reflectors. <code>tau(n)</code> is used as workspace.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

hetrd (USM Version)

Reduces a complex Hermitian matrix to tridiagonal form. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine reduces a complex Hermitian matrix `A` to Hermitian tridiagonal form `T` by a unitary similarity transformation: $A = Q^* T Q$. The unitary matrix `Q` is not formed explicitly but is represented as a product of `n-1` elementary reflectors. Routines are provided to work with `Q` in this representation.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event hetrd(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        T *d,
        T *e,
        T *tau,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

hetrd (USM version) supports the following precisions and devices:

T	Devices supported
std::complex<float>	CPU and GPU*
std::complex<double>	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	The device queue where calculations will be performed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , <code>a</code> stores the upper triangular part of <code>A</code> . If <code>uplo = uplo::lower</code> , <code>a</code> stores the lower triangular part of <code>A</code> .
n	The order of the matrices <code>A</code> ($0 \leq n$).
a	Pointer to the matrix <code>A</code> , size <code>(lda, *)</code> . Contains the upper or lower triangle as specified by <code>uplo</code> .
lda	The leading dimension of <code>a</code> ; at least <code>max(1, n)</code>
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the hetrd_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	On exit,
---	----------

if `uplo = uplo::upper`, the diagonal and first superdiagonal of `A` are overwritten by the corresponding elements of the tridiagonal matrix `T`, and the elements above the first superdiagonal, with the array `tau`, represent the unitary matrix `Q` as a product of elementary reflectors;

if `uplo = uplo::lower`, the diagonal and first subdiagonal of `A` are overwritten by the corresponding elements of the tridiagonal matrix `T`, and the elements below the first subdiagonal, with the array `tau`, represent the unitary matrix `Q` as a product of elementary reflectors.

<code>d</code>	Pointer to the diagonal elements of the matrix <code>T</code> . The dimension of <code>d</code> must be at least <code>max(1, n)</code> .
<code>e</code>	Pointer to off diagonal elements of the matrix <code>T</code> . The dimension of <code>e</code> must be at least <code>max(1, n-1)</code> .
<code>tau</code>	Pointer to memory array of size at least <code>max(1, n)</code> . Stores <code>(n-1)</code> scalars that define elementary reflectors in decomposition of the unitary matrix <code>Q</code> in a product of <code>n-1</code> elementary reflectors. <code>tau(n)</code> is used as workspace.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

hetrd_scratchpad_size

Computes size of scratchpad memory required for `hetrd` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `hetrd` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t hetrd_scratchpad_size(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda)
}
```

Input Parameters

queue	Device queue where calculations by the hetrd (buffer or USM version) function will be performed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , a stores the upper triangular part of A. If <code>uplo = uplo::lower</code> , a stores the lower triangular part of A.
n	The order of the matrices A ($0 \leq n$).
lda	The leading dimension of a; at least <code>max(1, n)</code>

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `hetrd` (buffer or USM version) function must be able to hold.

hetrf

Computes the Bunch-Kaufman factorization of a complex Hermitian matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the factorization of a complex Hermitian matrix A using the Bunch-Kaufman diagonal pivoting method:

- if `uplo='U'`, $A = U * D * U^H$
- if `uplo='L'`, $A = L * D * L^H$,

where A is the input matrix, U and L are products of permutation and triangular matrices with unit diagonal (upper triangular for U and lower triangular for L), and D is a Hermitian block-diagonal matrix with 1-by-1 and 2-by-2 diagonal blocks. U and L have 2-by-2 unit diagonal blocks corresponding to the 2-by-2 blocks of D .

NOTE This routine supports the Progress Routine feature.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void hetrf(sycl::queue &queue,
              mkl::uplo uplo,
              int64_t n,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &ipiv,
              sycl::buffer<T> &scratchpad, int64_t scratchpad_size)
}
```

`hetrf` supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU
<code>std::complex<double></code>	CPU

Input Parameters

<code>queue</code>	The device queue where calculations will be performed.
<code>uplo</code>	Indicates whether the upper or lower triangular part of A is stored and how A is factored: If <code>uplo = uplo::upper</code> , the array <code>a</code> stores the upper triangular part of A and A is factored as U^*D*U^H . If <code>uplo = uplo::lower</code> , the array <code>a</code> stores the lower triangular part of A and A is factored as L^*D*L^H .
<code>n</code>	The order of the matrix A ($0 \leq n$).
<code>a</code>	Buffer holding coefficients of matrix A containing either the upper or the lower triangular part of the matrix A (see <code>uplo</code>). The size of <code>a</code> must be at least <code>lda*n</code> .
<code>lda</code>	The leading dimension of <code>a</code> .
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the hetrf_scratchpad_size function.

Output Parameters

- a** The upper or lower triangular part of **a** is overwritten by details of the block-diagonal matrix **D** and the multipliers used to obtain the factor **U** (or **L**).
- ipiv** Buffer holding array of size at least `max(1, n)`. Contains details of the interchanges and the block structure of **D**. If `ipiv(i) = k > 0`, then `dii` is a 1-by-1 block, and the *i*-th row and column of **A** was interchanged with the *k*-th row and column.
- If `uplo = mkl::uplo::upper` and `ipiv(i) = ipiv(i-1) = -m < 0`, then **D** has a 2-by-2 block in rows/columns *i* and *i-1*, and (*i-1*)-th row and column of **A** was interchanged with the *m*-th row and column.
- If `uplo = mkl::uplo::lower` and `ipiv(i) = ipiv(i+1) = -m < 0`, then **D** has a 2-by-2 block in rows/columns *i* and *i+1*, and (*i+1*)-th row and column of **A** was interchanged with the *m*-th row and column.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, <code>d:sub:`ii`</code> is 0. The factorization has been completed, but D is exactly singular. Division by 0 will occur if you use D for solving a system of linear equations. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

hetrf (USM Version)

Computes the Bunch-Kaufman factorization of a complex Hermitian matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the factorization of a complex Hermitian matrix **A** using the Bunch-Kaufman diagonal pivoting method. The form of the factorization is:

- if `uplo=uplo::upper`, $A = U^* D U H$
- if `uplo=uplo::lower`, $A = L^* D L H$,

where A is the input matrix, U and L are products of permutation and triangular matrices with unit diagonal (upper triangular for U and lower triangular for L), and D is a Hermitian block-diagonal matrix with 1-by-1 and 2-by-2 diagonal blocks. U and L have 2-by-2 unit diagonal blocks corresponding to the 2-by-2 blocks of D .

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event hetrf(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        int64_t *ipiv,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

hetrf (USM version) supports the following precisions and devices:

T	Devices supported
std::complex<float>	CPU
std::complex<double>	CPU

Input Parameters

queue	The device queue where calculations will be performed.
uplo	Indicates whether the upper or lower triangular part of A is stored and how A is factored: If <code>uplo = uplo::upper</code> , the array <code>a</code> stores the upper triangular part of A and A is factored as U^*D*U^H . If <code>uplo = uplo::lower</code> , the array <code>a</code> stores the lower triangular part of A and A is factored as L^*D*L^H .
n	The order of the matrix A ($0 \leq n$).
a	The pointer to coefficients of matrix A containing either the upper or the lower triangular part of the matrix A (see <code>uplo</code>). The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the hetrf_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

- a** The upper or lower triangular part of **a** is overwritten by details of the block-diagonal matrix **D** and the multipliers used to obtain the factor **U** (or **L**).
- ipiv** Pointer to memory array of size at least $\max(1, n)$. Contains details of the interchanges and the block structure of **D**. If $\text{ipiv}(i) = k > 0$, then d_{ii} is a 1-by-1 block, and the i -th row and column of **A** was interchanged with the k -th row and column.
- If $\text{uplo} = \text{mkl::uplo::upper}$ and $\text{ipiv}(i) = \text{ipiv}(i-1) = -m < 0$, then **D** has a 2-by-2 block in rows/columns i and $i-1$, and $(i-1)$ -th row and column of **A** was interchanged with the m -th row and column.
- If $\text{uplo} = \text{mkl::uplo::lower}$ and $\text{ipiv}(i) = \text{ipiv}(i+1) = -m < 0$, then **D** has a 2-by-2 block in rows/columns i and $i+1$, and $(i+1)$ -th row and column of **A** was interchanged with the m -th row and column.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info = i</code>, <code>d:sub:`ii`</code> is 0. The factorization has been completed, but D is exactly singular. Division by 0 will occur if you use D for solving a system of linear equations. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

hetrf_scratchpad_size

Computes size of scratchpad memory required for `hetrf` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type **T** the scratchpad memory to be passed to the `hetrf` (buffer or USM version) function should be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t hetrf_scratchpad_size(sycl::queue &queue,
    mkl::uplo uplo,
    int64_t n,
    int64_t lda)
}
```

Input Parameters

queue	Device queue where calculations by the hetrf (buffer or USM version) function will be performed.
uplo	Indicates whether the upper or lower triangular part of A is stored and how A is factored: If <code>uplo = uplo::upper</code>, the array <code>a</code> stores the upper triangular part of the matrix A, and A is factored as $U^*D^*U^H$. If <code>uplo = uplo::lower</code>, the array <code>a</code> stores the lower triangular part of the matrix A, and A is factored as $L^*D^*L^H$.
n	The order of matrix A ($0 \leq n$).
lda	The leading dimension of <code>a</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `hetrf` (buffer or USM version) function must be able to hold.

orgbr

Generates the real orthogonal matrix Q or P^T determined by [gebrd](#). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the whole or part of the orthogonal matrices Q and P^T formed by the routines [gebrd](#). Use this routine after a call to `sgebrd/dgebrd`. All valid combinations of arguments are described in *Input parameters*. In most cases you need the following:

To compute the whole m -by- m matrix Q :

```
orgbr(queue, generate::q, m, m, n, a, ...)
```

(note that the array a must have at least m columns).

To form the n leading columns of Q if $m > n$:

```
orgbr(queue, generate::q, m, n, n, a, ...)
```

To compute the whole n -by- n matrix P^T :

```
orgbr(queue, generate::p, n, n, m, a, ...)
```

(note that the array a must have at least n rows).

To form the m leading rows of P^T if $m < n$:

```
orgbr(queue, generate::p, m, n, m, a, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    void orgbr(sycl::queue &queue,
              mkl::generate gen,
              int64_t m,
              int64_t n,
              int64_t k,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &tau,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

`orgbr` supports the following precisions and devices:

T	Devices supported
float	CPU
double	CPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>gen</code>	Must be <code>generate::q</code> or <code>generate::p</code> . If <code>gen= generate::q</code> , the routine generates the matrix Q . If <code>gen= generate::p</code> , the routine generates the matrix P^T .
<code>m</code>	The number of rows in the matrix Q or P^T to be returned ($0 \leq m$). If <code>gen= generate::q</code> , $m \geq n \geq \min(m, k)$. If <code>gen= generate::p</code> , $n \geq m \geq \min(n, k)$.
<code>n</code>	The number of rows in the matrix Q or P^T to be returned ($0 \leq n$). See <code>m</code> for constraints.

k	If <code>gen= generate::q</code>, the number of columns in the original m-by-k matrix reduced by gebrd. If <code>gen= generate::p</code>, the number of rows in the original k-by-n matrix reduced by gebrd.
a	Buffer holding memory returned by gebrd .
lda	The leading dimension of a.
tau	Buffer holding array of size $\min(m, k)$ if <code>gen= generate::q</code> , and of size $\min(n, k)$ if <code>gen= generate::p</code> . Scalar factor of the elementary reflectors, as returned by gebrd in the array <code>tauq</code> or <code>taup</code> .
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the orgbr_scratchpad_size function.
Output Parameters	
a	Overwritten by n leading columns of the m -by- m orthogonal matrix Q or P^T (or the leading rows or columns thereof) as specified by <code>gen</code> , m , and n .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

orgbr (USM Version)

Generates the real orthogonal matrix Q or P^T determined by [gebrd \(USM Version\)](#). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the whole or part of the orthogonal matrices Q and P^T formed by the [gebrd \(USM Version\)](#) function. All valid combinations of arguments are described in *Input parameters*. In most cases you need the following:

To compute the whole m -by- m matrix Q :

```
orgbr(queue, generate::q, m, m, n, a, ...)
```

(note that the array a must have at least m columns).

To form the n leading columns of Q if $m > n$:

```
orgbr(queue, generate::q, m, n, n, a, ...)
```

To compute the whole n -by- n matrix P^T :

```
orgbr(queue, generate::p, n, n, m, a, ...)
```

(note that the array a must have at least n rows).

To form the m leading rows of P^T if $m < n$:

```
orgbr(queue, generate::p, m, n, m, a, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event orgbr(sycl::queue &queue,
        mkl::generate gen,
        int64_t m,
        int64_t n,
        int64_t k,
        T *a,
        int64_t lda,
        const T *tau,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`orgbr` (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU
double	CPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>gen</code>	Must be <code>generate::q</code> or <code>generate::p</code> . If <code>gen= generate::q</code> , the routine generates the matrix Q . If <code>gen= generate::p</code> , the routine generates the matrix P^T .
<code>m</code>	The number of rows in the matrix Q or P^T to be returned ($0 \leq m$). If <code>gen= generate::q</code> , $m \geq n \geq \min(m, k)$. If <code>gen= generate::p</code> , $n \geq m \geq \min(n, k)$.

<code>n</code>	The number of rows in the matrix Q or P^T to be returned ($0 \leq n$). See <code>m</code> for constraints.
<code>k</code>	If <code>gen= generate::q</code>, the number of columns in the original m-by-k matrix reduced by gebrd (USM Version). If <code>gen= generate::p</code>, the number of rows in the original k-by-n matrix reduced by gebrd (USM Version).
<code>a</code>	Pointer to memory returned by gebrd (USM Version) .
<code>lda</code>	The leading dimension of <code>a</code> .
<code>tau</code>	Pointer to memory of size $\min(m, k)$ if <code>gen= generate::q</code> , and of size $\min(n, k)$ if <code>gen= generate::p</code> . Scalar factor of the elementary reflectors, as returned by gebrd (USM Version) in the array <code>tauq</code> or <code>taup</code> .
<code>scratchpad</code>	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the orgbr_scratchpad_size function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
<code>a</code>	Overwritten by <code>n</code> leading columns of the m -by- m orthogonal matrix Q or P^T (or the leading rows or columns thereof) as specified by <code>gen</code> , <code>m</code> , and <code>n</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

[orgbr_scratchpad_size](#)

Computes size of scratchpad memory required for [orgbr](#) (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `orgbr` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t orgbr_scratchpad_size(sycl::queue &queue,
        mkl::generate gen,
        int64_t m,
        int64_t n,
        int64_t k,
        int64_t lda)
}
```

Input Parameters

queue	Device queue where calculations by the <code>orgbr</code> (buffer or USM version) function will be performed.
gen	Must be <code>generate::q</code> or <code>generate::p</code> . If <code>gen= generate::q</code> , the routine generates the matrix Q . If <code>gen= generate::p</code> , the routine generates the matrix P^T .
m	The number of rows in the matrix Q or P^T to be returned ($0 \leq m$). If <code>gen= generate::q</code> , $m \geq n \geq \min(m, k)$. If <code>gen= generate::p</code> , $n \geq m \geq \min(n, k)$.
n	The number of rows in the matrix Q or P^T to be returned ($0 \leq n$). See <code>m</code> for constraints.
k	If <code>gen= generate::q</code> , the number of columns in the original <code>m</code> -by- <code>k</code> matrix reduced by gebrd (USM Version) . If <code>gen= generate::p</code> , the number of rows in the original <code>k</code> -by- <code>n</code> matrix reduced by gebrd (USM Version) .
lda	The leading dimension of <code>a</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `orgqr` (buffer or USM version) function must be able to hold.

orgqr

Generates the real orthogonal matrix Q of the QR factorization formed by `geqrf`. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the whole or part of m -by- m orthogonal matrix Q of the QR factorization formed by the routine `geqrf`.

Usually Q is determined from the QR factorization of an m by p matrix A with $m \geq p$. To compute the whole matrix Q , use:

```
mkl::orgqr(queue, m, m, p, a, lda, tau, ...)
```

To compute the leading p columns of Q (which form an orthonormal basis in the space spanned by the columns of A):

```
mkl::orgqr(queue, m, p, p, a, lda, tau, ...)
```

To compute the matrix Q^k of the QR factorization of leading k columns of the matrix A :

```
mkl::orgqr(queue, m, m, k, a, lda, tau, ...)
```

To compute the leading k columns of Q^k (which form an orthonormal basis in the space spanned by leading k columns of the matrix A):

```
mkl::orgqr(queue, m, k, k, a, lda, tau, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    void orgqr(sycl::queue &queue,
               int64_t m,
               int64_t n,
               int64_t k,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<T> &tau,
               sycl::buffer<T> &scratchpad,
               int64_t scratchpad_size)
}
```

`orgqr` supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
k	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
a	Buffer holding the result of geqrf .
lda	The leading dimension of a ($lda \geq m$).
tau	Buffer holding the result of geqrf .
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by the orgqr_scratchpad_size function.

Output Parameters

a	Overwritten by n leading columns of the m -by- m orthogonal matrix Q .
---	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

orgqr (USM Version)

Generates the real orthogonal matrix Q of the QR factorization formed by the `geqrf` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the whole or part of m -by- m orthogonal matrix Q of the QR factorization formed by the routine **geqrf (USM Version)** :ref: `geqrf-usm-version` function.

Usually Q is determined from the QR factorization of an m by p matrix A with $m \geq p$. To compute the whole matrix Q , use:

```
mkl::orgqr(queue, m, m, p, a, lda, tau, ...)
```

To compute the leading p columns of Q (which form an orthonormal basis in the space spanned by the columns of A):

```
mkl::orgqr(queue, m, p, p, a, lda, tau, ...)
```

To compute the matrix Q^k of the QR factorization of leading k columns of the matrix A :

```
mkl::orgqr(queue, m, m, k, a, lda, tau, ...)
```

To compute the leading k columns of Q^k (which form an orthonormal basis in the space spanned by leading k columns of the matrix A):

```
mkl::orgqr(queue, m, k, k, a, lda, tau, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event orgqr(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t k,
        T *a,
        int64_t lda,
        const T *tau,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`orgqr` (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
k	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
a	Pointer to the result of geqrf (USM Version) .
lda	The leading dimension of a ($lda \geq m$).
tau	Pointer to the result of geqrf (USM Version) .

scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type $\mathbb{T}$. Size should not be less than the value returned by the orgqr_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
a	Overwritten by n leading columns of the m-by-m orthogonal matrix Q .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, d_{ii} is 0. The factorization has been completed, but D is exactly singular. Division by 0 will occur if you use D for solving a system of linear equations. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete

orgqr_batch (Buffer Strided Version)

Generates the real orthogonal matrices Q_i of the batch QR factorizations formed by `geqrf_batch` (Buffer Strided Version). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the wholes or parts of the orthogonal matrices Q_i of the batch of QR factorizations formed by the routine [geqrf_batch \(Buffer Strided Version\)](#). Usually, Q_i is determined from the QR factorization of an m -by- p matrix A_i with $m \geq p$.

To compute the whole matrices Q_i , use:

```
orgqr_batch(queue, m, m, p, a, ...)
```

To compute the leading p columns of Q_i (which form an orthonormal basis in the space spanned by the columns of A_i):

```
orgqr_batch(queue, m, p, p, a, ...)
```

To compute the matrices Q_i^k of the QR factorizations of leading k columns of the matrices A_i :

```
orgqr_batch(queue, m, m, k, a, ...)
```

To compute the leading k columns of Q_i^k (which form an orthonormal basis in the space spanned by leading k columns of the matrices A_i):

```
orgqr_batch(queue, m, k, k, a, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    void orgqr_batch(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t k,
        sycl::buffer<T> &a,
        int64_t lda,
        int64_t stride_a,
        sycl::buffer<T> &tau,
        int64_t stride_tau,
        int64_t batch_size,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i ($m \geq 0$).
n	The number of columns in the matrices A_i ($n \geq 0$).
k	the number of elementary reflectors whose product defines the matrices Q_i ($0 \leq k \leq n$).
a	Array resulting after a call to geqrf_batch (Buffer Strided Version) .
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).

<code>tau</code>	Array resulting after a call to geqrf_batch (Buffer Strided Version) .
<code>stride_tau</code>	The stride between the beginnings of arrays <code>tau_i</code> inside the array <code>tau</code> (<code>stride_tau ≥ max(1, min(m,n))</code>).
<code>batch_size</code>	The number of problems in a batch (<code>batch_size ≥ 0</code>).
<code>scratchpad</code>	Scratchpad memory to be used by routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by orgqr_batch_scratchpad_size (Strided Version) .

Output Parameters

<code>a</code>	Array data is overwritten by a batch of <code>n</code> leading columns of the <code>m</code> -by- <code>m</code> orthogonal matrices Q_i .
----------------	--

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less than value returned by the <code>detail()</code> method of the exception object.

orgqr_batch (Group Version)

Generates the real orthogonal matrices Q_i of the batch of QR factorizations formed by [geqrf_batch \(Group Version\)](#). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the wholes or parts of the `m`-by-`m` orthogonal matrices Q_i of the batch of QR factorizations formed by the routine [geqrf_batch \(Group Version\)](#). Usually, Q_i is determined from the QR factorization of an `m`-by-`p` matrix A_i with $m ≥ p$.

To compute the whole matrices Q_i , use:

```
orgqr_batch(queue, m, m, p, a, ...)
```

To compute the leading `p` columns of Q_i (which form an orthonormal basis in the space spanned by the columns of A_i):

```
orgqr_batch(queue, m, p, p, a, ...)
```


To compute the matrices Q_i^k of the QR factorizations of leading k columns of the matrices A_i :

```
orgqr_batch(queue, m, m, k, a, ...)
```

To compute the leading k columns of Q_i^k (which form an orthonormal basis in the space spanned by leading k columns of the matrices A_i):

```
orgqr_batch(queue, m, k, k, a, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event orgqr_batch(sycl::queue &queue,
        int64_t *m,
        int64_t *n,
        int64_t *k,
        T **a,
        int64_t *lda,
        const T * const *tau,
        int64_t group_count,
        int64_t *group_sizes,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	Array of <code>group_count</code> parameters m_g as previously supplied to geqrf_batch (Group Version) ($m_g \geq 0$).
n	Array of <code>group_count</code> parameters n_g as previously supplied to geqrf_batch (Group Version) ($n_g \geq 0$).
k	Array of <code>group_count</code> parameters k_g . The number of elementary reflectors whose product defines the matrices Q_i ($0 \leq k_g \leq n_g$).
a	Array resulting after a call to geqrf_batch (Group Version) .
lda	Array of leading dimension of A_i as previously supplied to geqrf_batch (Group Version) ($lda_g \geq \max(1, m_g)$).
tau	Array resulting after a call to geqrf_batch (Group Version) .
group_count	Specifies the number of groups of parameters. Must be at least 0.

group_sizes	Array of group_count integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less then the value returned by orgqr_batch_scratchpad_size (Group Version) .
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

<code>a</code>	Matrices pointed to by array <code>a</code> are overwritten by n_g leading columns of the m_g -by- m_g orthogonal matrices Q_i , where g is an index of group of parameters corresponding to Q_i .
----------------	--

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

orgqr_batch (USM Strided Version)

Generates the orthogonal matrices Q_i of the batch QR factorizations formed by `geqrf_batch` (USM Strided Version). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the wholes or parts of the orthogonal matrices Q_i of the batch of QR factorizations formed by the routine [geqrf_batch \(USM Strided Version\)](#). Usually, Q_i is determined from the QR factorization of an m -by- p matrix A_i with $m \geq p$.

To compute the whole matrices Q_i , use:

```
orgqr_batch(queue, m, m, p, a, ...)
```

To compute the leading p columns of Q_i (which form an orthonormal basis in the space spanned by the columns of A_i):

```
orgqr_batch(queue, m, p, p, a, ...)
```

To compute the matrices Q_i^k of the QR factorizations of leading k columns of the matrices A_i :

```
orgqr_batch(queue, m, m, k, a, ...)
```

To compute the leading k columns of Q_i^k (which form an orthonormal basis in the space spanned by leading k columns of the matrices A_i):

```
orgqr_batch(queue, m, k, k, a, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event orgqr_batch(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t k,
        T *a,
        int64_t lda,
        int64_t stride_a,
        const T *tau,
        int64_t stride_tau,
        int64_t batch_size,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i ($m \geq 0$).
n	The number of columns in the matrices A_i ($n \geq 0$).
k	the number of elementary reflectors whose product defines the matrices Q_i ($0 \leq k \leq n$).
a	Array resulting after a call to geqrh_batch (USM Strided Version) .
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).

<code>tau</code>	Array resulting after a call to geqrf_batch (USM Strided Version) .
<code>stride_tau</code>	The stride between the beginnings of arrays <code>tau_i</code> inside the array <code>tau</code> ($\text{stride_tau} \geq \max(1, \min(m, n))$).
<code>batch_size</code>	The number of problems in a batch ($\text{batch_size} \geq 0$).
<code>scratchpad</code>	Scratchpad memory to be used by routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by orgqr_batch_scratchpad_size (Strided Version) .
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

<code>a</code>	Array data is overwritten by a batch of <code>n</code> leading columns of the <code>m</code> -by- <code>m</code> orthogonal matrices Q_i .
----------------	--

Exceptions

Exception	Description
<code>mk1::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

[orgqr_batch_scratchpad_size \(Group Version\)](#)

Computes size of scratchpad memory required for [orgqr_batch \(Group Version\)](#) function. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the [orgqr_batch \(Group Version\)](#) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t orgqr_batch_scratchpad_size(sycl::queue &queue,
        int64_t *m,
        int64_t *n,
        int64_t *k,
        int64_t *lda,
        int64_t group_count,
        int64_t *group_sizes)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
m	Array of <code>group_count</code> parameters m_g as previously supplied to geqrf_batch (Group Version) ($m_g \geq 0$).
n	Array of <code>group_count</code> parameters n_g as previously supplied to geqrf_batch (Group Version) ($n_g \geq 0$).
k	Array of <code>group_count</code> parameters k_g . The number of elementary reflectors whose product defines the matrices Q_i ($0 \leq k_g \leq n_g$).
lda	The leading dimension of A_i as previously supplied to geqrf_batch (Group Version) ($lda_g \geq \max(1, m_g)$).
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the [orgqr_batch \(Group Version\)](#) function must be able to hold.

orgqr_batch_scratchpad_size (Strided Version)

Computes size of scratchpad memory required for `orgqr_batch` (Strided Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `orgqr_batch` (Strided Version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t orgqr_batch_scratchpad_size(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t k,
        int64_t lda,
        int64_t stride_a,
        int64_t stride_tau,
        int64_t batch_size)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>m</code>	The number of rows in the matrices A_i ($m \geq 0$).
<code>n</code>	The number of columns in the matrices A_i ($n \geq 0$).
<code>k</code>	the number of elementary reflectors whose product defines the matrices Q_i ($0 \leq k \leq n$).
<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, m)$).
<code>stride_a</code>	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
<code>stride_tau</code>	The stride between the beginnings of arrays $\tau_{i,j}$ inside the array <code>tau</code> ($stride_tau \geq \max(1, \min(m, n))$).
<code>batch_size</code>	The number of problems in a batch ($batch_size \geq 0$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `orgqr_batch` (Strided Version) function must be able to hold.

orgqr_scratchpad_size

Computes size of scratchpad memory required for `orgqr` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type T the scratchpad memory to be passed to the `orgqr` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {  
    template<typename T>  
        int64_t orgqr_scratchpad_size(sycl::queue &queue,  
        int64_t m,  
        int64_t n,  
        int64_t k,  
        int64_t lda)  
}
```

Input Parameters

queue	Device queue where calculations by the <code>orgqr</code> (buffer or USM version) function will be performed.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
k	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
lda	The leading dimension of a ($lda \geq m$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `orgqr` (buffer or USM version) function must be able to hold.

orgtr

Generates the real orthogonal matrix Q determined by `sytrd`. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine explicitly generates the n -by- n orthogonal matrix Q formed by `sytrd` when reducing a real symmetric matrix A to tridiagonal form: $A = Q^T T Q$. Use this routine after a call to `sytrd`.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void orgtr(sycl::queue &queue,
              mkl::uplo uplo,
              int64_t n,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &tau,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

`orgtr` supports the following precisions and devices:

T	Devices supported
float	CPU
double	CPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>uplo</code>	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . Uses the same <code>uplo</code> as supplied to the <code>sytrd</code> function
<code>n</code>	The order of matrix Q ($0 \leq n$).
<code>a</code>	The buffer <code>a</code> returned by the <code>sytrd</code> function. The size of <code>a</code> must be at least <code>lda*n</code> .
<code>tau</code>	Buffer holding <code>tau</code> returned by the <code>sytrd</code> function.
<code>lda</code>	The leading dimension of <code>a</code> (<code>lda</code> $\geq$ <code>n</code>).
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.

`scratchpad_size` Size of scratchpad memory as a number of floating point elements of type `T`. Size should not be less than the value returned by the [orgtr_scratchpad_size](#) function.

Output Parameters

`a` Overwritten by the orthogonal matrix Q .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

orgtr (USM Version)

Generates the real orthogonal matrix Q determined by the `sytrd` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine explicitly generates the n -by- n orthogonal matrix Q formed by [sytrd \(USM Version\)](#) when reducing a real symmetric matrix A to tridiagonal form: $A = Q^T T Q$. Use this routine after a call to [sytrd \(USM Version\)](#).

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event orgtr(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        const T *tau,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`orgtr` (USM version) supports the following precision and devices.

T	Devices Supported
float	CPU
double	CPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . Uses the same <code>uplo</code> as supplied to the sytrd (USM Version) function
n	The order of matrix Q ($0 \leq n$).
a	The pointer <code>a</code> returned by the sytrd (USM Version) function. The size of <code>a</code> must be at least <code>lda*n</code> .
tau	The pointer to <code>tau</code> returned by the geqrf (USM Version) function.
lda	The leading dimension of <code>a</code> ($lda \geq n$).
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the orgtr_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

<code>a</code>	Overwritten by the orthogonal matrix Q .
----------------	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

[orgtr_scratchpad_size](#)

Computes size of scratchpad memory required for `orgtr` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `orgtr` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {  
    template<typename T>  
    orgtr_scratchpad_size(sycl::queue &queue,  
        mkl::uplo uplo,  
        int64_t n,  
        int64_t lda)  
}
```

Input Parameters

queue	Device queue where calculations by the <code>orgtr</code> (buffer or USM version) function will be performed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . Uses the same <code>uplo</code> as supplied to the sytrd (USM Version) function.
n	The order of matrix Q ($0 \leq n$).
lda	The leading dimension of a ($lda \geq n$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `orgtr` (buffer or USM version) function must be able to hold.

ormqr

Multiplies a real matrix by the orthogonal matrix Q of the QR factorization formed by `geqrf`. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine multiplies a real matrix C by Q or Q^T , where Q is the orthogonal matrix Q of the QR factorization formed by the routine [geqrf](#).

Depending on the parameters `left_right` and `trans`, the routine can form one of the matrix products $Q^T C$, $C^T Q$, or $C^T Q^T$ (overwriting the result on C).

API

Syntax

```
namespace oneapi::mkl::lapack {
    void ormqr(sycl::queue &queue,
              mkl::side side,
              mkl::transpose trans,
              int64_t m,
              int64_t n,
              int64_t k,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &tau,
              sycl::buffer<T> &c,
              int64_t ldc,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

`ormqr` supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>side</code>	If <code>side = mkl::side::left</code> , Q or Q^T is applied to C from the left. If <code>side = mkl::side::right</code> , Q or Q^T is applied to C from the right.
<code>trans</code>	If <code>trans=mkl::transpose::trans</code> , the routine multiplies C by Q . If <code>trans=mkl::transpose::nontrans</code> , the routine multiplies C by Q^T .
<code>m</code>	The number of rows in the matrix A ($0 \leq m$).
<code>n</code>	The number of columns in the matrix A ($0 \leq n \leq m$).
<code>k</code>	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
<code>a</code>	Buffer holding the result of the geqrf function. . The size of <code>a</code> must be at least <code>lda*k</code> .
<code>lda</code>	The leading dimension of <code>a</code> .

<code>tau</code>	Buffer holding tau returned by the geqrf function.
<code>c</code>	Buffer holding the matrix <code>C</code> . The size of <code>c</code> must be at least <code>ldc*n</code> .
<code>ldc</code>	The leading dimension of <code>c</code> .
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the ormqr_scratchpad_size function.

Output Parameters

<code>c</code>	Overwritten by the product Q^*C , $Q^T C$, C^*Q , or C^*Q^T (as specified by <code>left_right</code> and <code>trans</code>).
----------------	---

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

ormqr (USM Version)

Multiplies a real matrix by the orthogonal matrix Q of the QR factorization formed by the [geqrf](#) (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine multiplies a real matrix `C` by Q or Q^T , where Q is the orthogonal matrix Q of the QR factorization formed by the routine [geqrf](#) (USM Version).

Depending on the parameters `side` and `trans`, the routine can form one of the matrix products Q^*C , $Q^T C$, C^*Q , or C^*Q^T (overwriting the result on `C`).

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event ormqr(sycl::queue &queue,
        mkl::side side,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        int64_t k,
        const T *a,
        int64_t lda,
        const T *tau,
        T *c,
        int64_t ldc,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

ormqr (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU, GPU*
double	CPU, GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
side	If side = mkl::side::left, Q or Q^T is applied to C from the left. If side = mkl::side::right, Q or Q^T is applied to C from the right.
trans	If trans=mkl::transpose::trans, the routine multiplies C by Q . If trans=mkl::transpose::nontrans, the routine multiplies C by Q^T .
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n \leq m$).
k	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
a	The pointer to the result of the geqrf (USM Version) function. The size of a must be at least $lda \cdot k$.
lda	The leading dimension of a .
tau	The pointer to tau returned by the geqrf (USM Version) function.
c	The pointer to the matrix C . The size of c must be at least $ldc \cdot n$.
ldc	The leading dimension of c .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.

`scratchpad_size` Size of scratchpad memory as a number of floating point elements of type `T`. Size should not be less than the value returned by the [ormqr_scratchpad_size](#) function.

`events` List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

`c` Overwritten by the product Q^*C , $Q^T C$, C^*Q , or C^*Q^T (as specified by `side` and `trans`).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

ormqr_scratchpad_size

Computes size of scratchpad memory required for `ormqr` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `ormqr` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t ormqr_scratchpad_size(sycl::queue &queue,
        mkl::side side,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
```

```
int64_t k,
int64_t lda,
int64_t ldc)
}
```

Input Parameters

queue	Device queue where calculations by the <code>ormqr</code> (buffer or USM version) function will be performed.
side	If <code>side=mkl::side::left</code> , Q or Q^T is applied to C from the left. If <code>side=mkl::side::right</code> , Q or Q^T is applied to C from the right.
trans	If <code>trans=mkl::transpose::trans</code> , the routine multiplies C by Q . If <code>trans=mkl::transpose::nontrans</code> , the routine multiplies C by Q^T .
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n \leq m$).
k	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
lda	The leading dimension of a .
ldc	The leading dimension of c .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `ormqr` (buffer or USM version) function must be able to hold.

ormqr

Multiplies a real matrix by the orthogonal matrix Q of the RQ factorization formed by `gerqf`.

- [Description](#)
- [API](#)

Description

The routine multiplies a real m -by- n matrix C by Q or Q^T , where Q is the real orthogonal matrix defined as a product of k elementary reflectors H_i : $Q = H_1 H_2 \dots H_k$ as returned by the RQ factorization routine [gerqf](#).

Depending on the parameters `side` and `trans`, the routine can form one of the matrix products Q^*C , $Q^T C$, C^*Q , or $C^T Q$ (overwriting the result over C).

API

Syntax

```
namespace oneapi::mkl::lapack {
    void ormrq(sycl::queue &queue,
              mkl::side side,
              mkl::transpose trans,
              int64_t m,
              int64_t n,
              int64_t k,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &tau,
              sycl::buffer<T> &c,
              int64_t ldc,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

ormrq supports the following precisions and devices:

T	Devices supported
float	CPU
double	CPU

Input Parameters

queue	Device queue where calculations will be performed.
side	If <code>side = mkl::side::left</code> , Q or Q^T is applied to C from the left. If <code>side = mkl::side::right</code> , Q or Q^T is applied to C from the right.
trans	If <code>trans=mkl::transpose::trans</code> , the routine multiplies C by Q . If <code>trans=mkl::transpose::nontrans</code> , the routine multiplies C by Q^T .
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n \leq m$).
k	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
a	Buffer holding the result of the gerqf function. The size of <code>a</code> must be at least <code>lda*k</code> .
lda	The leading dimension of <code>a</code> .
tau	Buffer holding <code>tau</code> returned by the gerqf function.
c	Buffer holding the matrix C . The size of <code>c</code> must be at least <code>ldc*n</code> .
ldc	The leading dimension of <code>c</code> .
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.

`scratchpad_size` Size of scratchpad memory as a number of floating point elements of type `T`. Size should not be less than the value returned by the [ormrq_scratchpad_size](#) function.

Output Parameters

`c` Overwritten by the product Q^*C , $Q^T * C$, $C * Q$, or $C * Q^T$ (as specified by `side` and `trans`).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code> , the <i>i</i> -th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad size</code> , and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Application Notes

The complex counterpart of this routine is [unmrq](#).

ormrq (USM Version)

Multiplies a real matrix by the orthogonal matrix Q of the RQ factorization formed by the `gerqf` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine multiplies a real matrix C by Q or Q^T , where Q is the real orthogonal matrix Q of the RQ factorization routine [gerqf \(USM Version\)](#).

Depending on the parameters `side` and `trans`, the routine can form one of the matrix products Q^*C , $Q^T * C$, $C * Q$, or $C * Q^T$ (overwriting the result on C).

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event ormrq(sycl::queue &queue,
        mkl::side side,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        int64_t k,
        const T *a,
```

```

int64_t lda,
const T *tau,
T *c,
int64_t ldc,
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
}

```

ormrq (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU
double	CPU

Input Parameters

queue	Device queue where calculations will be performed.
side	If <code>side = mkl::side::left</code> , Q or Q^T is applied to C from the left. If <code>side = mkl::side::right</code> , Q or Q^T is applied to C from the right.
trans	If <code>trans=mkl::transpose::trans</code> , the routine multiplies C by Q . If <code>trans=mkl::transpose::nontrans</code> , the routine multiplies C by Q^T .
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n \leq m$).
k	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
a	The pointer to the result of the gerqf (USM Version) function. The size of <code>a</code> must be at least <code>lda*k</code> .
lda	The leading dimension of <code>a</code> .
tau	The pointer to <code>tau</code> returned by the gerqf (USM Version) function.
c	The pointer to the matrix C . The size of <code>c</code> must be at least <code>ldc*n</code> .
ldc	The leading dimension of <code>c</code> .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the ormrq_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

c	Overwritten by the product $Q*C$, Q^T*C , $C*Q$, or $C*Q^T$ (as specified by <code>side</code> and <code>trans</code>).
---	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

ormrq_scratchpad_size

Computes size of scratchpad memory required for `ormrq` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `ormrq` (buffer or USM version) function must be able to hold.

API

Syntax

```

namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t ormrq_scratchpad_size(sycl::queue &queue,
        mkl::side side,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        int64_t k,
        int64_t lda,
        int64_t ldc)
}

```

Input Parameters

queue

Device queue where calculations by the `ormrq` (buffer or USM version) function will be performed.

side	If <code>side = mkl::side::left</code> , Q or Q^T is applied to C from the left. If <code>side = mkl::side::right</code> , Q or Q^T is applied to C from the right.
trans	If <code>trans=mkl::transpose::trans</code> , the routine multiplies C by Q . If <code>trans=mkl::transpose::nontrans</code> , the routine multiplies C by Q^T .
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n \leq m$).
k	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
lda	The leading dimension of a .
ldc	The leading dimension of c .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `ormrqr` (buffer or USM version) function must be able to hold.

ormtr

Multiplies a real matrix by the real orthogonal matrix Q determined by [sytrd](#). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Q^T , where Q is the orthogonal matrix Q formed by [sytrd](#) when reducing a real symmetric matrix A to tridiagonal form: $A = Q^T T Q$. Use this routine after a call to [sytrd](#).

Depending on the parameters `left_right` and `trans`, the routine can form one of the matrix products $Q^T C$, $Q^T C Q$, or $C^T Q$ (overwriting the result on C).

The routine multiplies a real matrix C by Q or

API

Syntax

```
namespace oneapi::mkl::lapack {
    void ormtr(sycl::queue &queue,
              mkl::side side,
              mkl::uplo uplo,
```

```

mkl::transpose trans,
int64_t m,
int64_t n,
sycl::buffer<T> &a,
int64_t lda,
sycl::buffer<T> &tau,
sycl::buffer<T> &c,
int64_t ldc,
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}

```

ormtr supports the following precisions and devices:

T	Devices supported
float	CPU
double	CPU

Input Parameters

In the descriptions below, *r* denotes the order of *Q*:

<i>r</i> = <i>m</i>	if <code>left_right = side::left</code>
<i>r</i> = <i>n</i>	if <code>left_right = side::right</code>

queue	Device queue where calculations will be performed.
side	Must be either <code>side::left</code> or <code>side::right</code>. If <code>side = side::left</code>, <i>Q</i> or <i>Q</i>^T is applied to <i>C</i> from the left. If <code>side = side::right</code>, <i>Q</i> or <i>Q</i>^T is applied to <i>C</i> from the right.
uplo	Must be either <code>uplo::upper</code> or <code>uplo::lower</code> . Uses the same <code>uplo</code> as supplied to sytrd .
trans	Must be either <code>transpose::nontrans</code> or <code>transpose::trans</code>. If <code>trans = transpose::nontrans</code>, the routine multiplies <i>C</i> by <i>Q</i>. If <code>trans = transpose::trans</code>, the routine multiplies <i>C</i> by <i>Q</i>^T.
<i>m</i>	The number of rows in the matrix <i>C</i> (<i>m</i> ≥ 0).
<i>n</i>	The number of columns in the matrix <i>C</i> (<i>n</i> ≥ 0).
<i>a</i>	Buffer holding array returned by sytrd .
<i>lda</i>	The leading dimension of <i>a</i> (<i>lda</i> ≥ max(1, <i>r</i>)).
<i>tau</i>	Buffer holding tau returned by sytrd . The dimension of tau must be at least max(1, <i>r</i> -1).
<i>c</i>	Buffer holding the matrix <i>C</i> . The size of <i>c</i> must be at least <i>ldc</i> * <i>n</i> .
<i>ldc</i>	The leading dimension of <i>c</i> (<i>ldc</i> ≥ max(1, <i>m</i>)).

scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by the ormtr_scratchpad_size function.

Output Parameters

c	Overwritten by the product Q^*C , $Q^T C$, C^*Q , or C^*Q^T (as specified by side and trans).
-----	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

ormtr (USM Version)

Multiplies a real matrix C by the real orthogonal matrix Q determined by the [sytrd \(USM Version\)](#) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine multiplies a real matrix C by Q or Q^T , where Q is the orthogonal matrix Q formed by [sytrd \(USM Version\)](#) when reducing a real symmetric matrix A to tridiagonal form: $A = Q^T T Q$. Use this routine after a call to [sytrd \(USM Version\)](#).

Depending on the parameters side and trans, the routine can form one of the matrix products Q^*C , $Q^T C$, C^*Q , or C^*Q^T (overwriting the result on C).

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event ormtr(sycl::queue &queue,
        mkl::side side,
        mkl::uplo uplo,
        mkl::transpose trans,
```

```

int64_t m,
int64_t n,
const T *a,
int64_t lda,
const T *tau,
T *c,
int64_t ldc,
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
}

```

ormtr (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU
double	CPU

Input Parameters

In the descriptions below, r denotes the order of Q :

$r = m$	if <code>side = side::left</code>
$r = n$	if <code>side = side::right</code>

queue	Device queue where calculations will be performed.
side	Must be either <code>side::left</code> or <code>side::right</code> . If <code>side = side::left</code> , Q or Q^T is applied to C from the left. If <code>side = side::right</code> , Q or Q^T is applied to C from the right.
uplo	Must be either <code>uplo::upper</code> or <code>uplo::lower</code> . Uses the same <code>uplo</code> as supplied to sytrd (USM Version) .
trans	Must be either <code>transpose::nontrans</code> or <code>transpose::trans</code> . If <code>trans = transpose::nontrans</code> , the routine multiplies C by Q . If <code>trans = transpose::trans</code> , the routine multiplies C by Q^T .
m	The number of rows in the matrix C ($m \geq 0$).
n	The number of columns in the matrix C ($n \geq 0$).
a	The pointer to memory returned by sytrd (USM Version) .
lda	The leading dimension of a ($lda \geq \max(1, r)$).
tau	The pointer to τ returned by sytrd (USM Version) . The dimension of τ must be at least $\max(1, r-1)$.
c	The pointer to memory containing the matrix C . The size of c must be at least $ldc*n$.
ldc	The leading dimension of c ($ldc \geq \max(1, m)$).

scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the ormtr_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
c	Overwritten by $Q \times C$, $Q^T \times C$, $C \times Q$, or $C \times Q^T$ (as specified by side and trans).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

ormtr_scratchpad_size

Computes size of scratchpad memory required for `ormtr` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `ormtr` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t ormtr_scratchpad_size(sycl::queue &queue,
        mkl::side side,
        mkl::uplo uplo,
```

```

mkl::transpose trans,
int64_t m,
int64_t n,
int64_t lda,
int64_t ldc)
}

```

Input Parameters

In the descriptions below, *r* denotes the order of *Q*:

<code>r = m</code>	if <code>side = side::left</code>
<code>r = n</code>	if <code>side = side::right</code>

queue	Device queue where calculations by the <code>ormtr</code> (buffer or USM version) function will be performed.
side	Must be either <code>side::left</code> or <code>side::right</code> . If <code>side = side::left</code> , Q or Q^T is applied to C from the left. If <code>side = side::right</code> , Q or Q^T is applied to C from the right.
uplo	Must be either <code>uplo::upper</code> or <code>uplo::lower</code> . Uses the same <code>uplo</code> as supplied to sytrd (USM Version) .
trans	Must be either <code>transpose::nontrans</code> or <code>transpose::trans</code> . If <code>trans = transpose::nontrans</code> , the routine multiplies C by Q . If <code>trans = transpose::trans</code> , the routine multiplies C by Q^T .
m	The number of rows in the matrix C ($m \geq 0$).
n	The number of columns in the matrix C ($n \geq 0$).
lda	The leading dimension of a ($lda \geq \max(1, r)$).
ldc	The leading dimension of c ($ldc \geq \max(1, m)$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `ormtr` (buffer or USM version) function must be able to hold.

potrf

Computes the Cholesky factorization of a symmetric (Hermitian) positive-definite matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the Cholesky factorization of a symmetric positive-definite or, for complex data, Hermitian positive-definite matrix A :

$$\begin{aligned} A &= U^T U \text{ for real data, } A = U^H U \text{ for complex data} && \text{if } \text{uplo} = \text{mkl::uplo::upper} \\ A &= L L^T \text{ for real data, } A = L L^H \text{ for complex data} && \text{if } \text{uplo} = \text{mkl::uplo::lower} \end{aligned}$$

where L is a lower triangular matrix and U is upper triangular.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void potrf(sycl::queue &queue,
              mkl::uplo uplo,
              int64_t n,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

`potrf` supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Indicates whether the upper or lower triangular part of A is stored and how A is factored: If <code>uplo=mkl::uplo::upper</code> , the array <code>a</code> stores the upper triangular part of the matrix A , and the strictly lower triangular part of the matrix is not referenced. If <code>uplo=mkl::uplo::lower</code> , the array <code>a</code> stores the lower triangular part of the matrix A , and the strictly upper triangular part of the matrix is not referenced.
n	Specifies the order of the matrix A ($0 \leq n$).

<code>a</code>	Buffer holding input matrix <code>A</code> . The array holding input matrix <code>a</code> contains either the upper or the lower triangular part of the matrix <code>A</code> (see <code>uplo</code>). The size of <code>a</code> must be at least <code>lda*n</code> .
<code>lda</code>	The leading dimension of <code>a</code> .
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the potrf_scratchpad_size function.

Output Parameters

<code>a</code>	The buffer <code>a</code> is overwritten by the Cholesky factor <code>U</code> or <code>L</code> , as specified by <code>uplo</code> .
----------------	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info = i</code>, and <code>detail()</code> returns 0, the leading minor of order <code>i</code> (and therefore the matrix <code>A</code> itself) is not positive-definite, and the factorization could not be completed. This may indicate an error in forming the matrix <code>A</code>. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

potrf (USM Version)

Computes the Cholesky factorization of a symmetric (Hermitian) positive-definite matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the Cholesky factorization of a symmetric positive-definite or, for complex data, Hermitian positive-definite matrix `A`:

$A = U^T * U$ for real data, $A = U^H * U$ for complex data	if <code>uplo=mkl::uplo::upper</code>
$A = L * L^T$ for real data, $A = L * L^H$ for complex data	if <code>uplo=mkl::uplo::lower</code>

where L is a lower triangular matrix and U is upper triangular.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event potrf(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

The USM version of `potrf` supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Indicates whether the upper or lower triangular part of A is stored and how A is factored: If <code>uplo=mkl::uplo::upper</code> , the array <code>a</code> stores the upper triangular part of the matrix A , and the strictly lower triangular part of the matrix is not referenced. If <code>uplo=mkl::uplo::lower</code> , the array <code>a</code> stores the lower triangular part of the matrix A , and the strictly upper triangular part of the matrix is not referenced.
n	Specifies the order of the matrix A ($0 \leq n$).
a	Pointer to input matrix A . The array holding input matrix <code>a</code> contains either the upper or the lower triangular part of the matrix A (see <code>uplo</code>). The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the potrf_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

- a The memory pointed to by pointer a is overwritten by the Cholesky factor U or L , as specified by uplo.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, and <code>detail()</code> returns 0, the leading minor of order <i>i</i> (and therefore the matrix <i>A</i> itself) is not positive-definite, and the factorization could not be completed. This may indicate an error in forming the matrix <i>A</i>. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

potrf_batch (Buffer Strided Version)

Computes the Cholesky factorizations of a batch of symmetric (or Hermitian, for complex data) positive-definite matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the Cholesky factorizations of a symmetric positive-definite or, for complex data, Hermitian positive-definite matrices A_i , $i \in \{1 \dots \text{batch_size}\}$:

- $A_i = U_i^T * U_i$ for real data, $A_i = U_i^H * U_i$ for complex data. if `uplo = mkl::uplo::upper`,
- $A_i = L_i^T * L_i$ for real data, $A_i = L_i^H * L_i$ for complex data if `uplo = mkl::uplo::lower`

Where L_i is a lower triangular matrix and U_i is an upper triangular matrix.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void potrf_batch(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        sycl::buffer<T> &a,
```

```
int64_t lda,
int64_t stride_a,
int64_t batch_size,
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Indicates whether the upper or lower triangular part of A_i is stored and how A_i is factored: If uplo = mkl::uplo::upper, the array a stores the upper triangular parts of the matrices A_i . If uplo = mkl::uplo::lower, the array a stores the lower triangular parts of the matrices A_i .
n	Specifies the order of the matrices A_i , ($n \geq 0$).
a	Array containing a batch of input matrices A_i , each of A_i being of size $lda*n$ and holding either upper or lower triangular parts of the matrices A_i (see uplo).
lda	The leading dimension of A_i ($lda \geq \max(1, n)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch ($stride_a \geq \max(1, lda * n)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by potrf_batch_scratchpad_size (Strided Version) .

Output Parameters

a	The batch array a is overwritten by the Cholesky factor U_i or L_i , as specified by uplo .
---	---

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

potrf_batch (Group Version)

Computes the Cholesky factorizations of a batch of symmetric (or Hermitian, for complex data) positive-definite matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the Cholesky factorizations of a symmetric positive-definite or, for complex data, Hermitian positive-definite matrices A_i , $i \in \{1 \dots \text{batch_size}\}$:

- $A_i = U_i^T * U_i$ for real data, $A_i = U_i^H * U_i$ for complex data. if `uplog = mkl::uplo::upper`,
- $A_i = L_i^T * L_i$ for real data, $A_i = L_i^H * L_i$ for complex data if `uplog = mkl::uplo::lower`

Where L_i is a lower triangular matrix and U_i is an upper triangular matrix, `g` is an index of group of parameters corresponding to A_i , and the total number of problems to solve, `batch_size`, is a sum of sizes for all of the groups of parameters as provided by the `group_sizes` array.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event potrf_batch(sycl::queue &queue,
        mkl::uplo *uplo,
        int64_t *n,
        T **a,
        int64_t *lda,
        int64_t group_count,
        int64_t *group_sizes,
        T *scratchpad,
```



```
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {}))
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Array of <code>group_count</code> <code>uplo_g</code> parameters. Each of <code>uplo_g</code> indicates whether the upper or lower triangular parts of the input matrices are provided: If <code>uplo_g=mkl::uplo::upper</code>, input matrices from array <code>a</code> belonging to group <code>g</code> store the upper triangular parts. If <code>uplo_g=mkl::uplo::lower</code>, input matrices from array <code>a</code> belonging to group <code>g</code> store the lower triangular parts.
n	Array of <code>group_count</code> parameters <code>n_g</code>. Each <code>n_g</code> specifies the order of the input matrices from array <code>a</code> belonging to group <code>g</code> ($n_g \geq 0$).
a	Array of <code>batch_size</code> pointers to input matrices A_i , each being of size $lda_g * n_g$ (<code>g</code> is an index of the group to which A_i belongs) and holding either upper or lower triangular part as specified by <code>uplo_g</code> .
lda	Array of <code>group_count</code> parameters <code>lda_g</code>. Each <code>lda_g</code> specifies the leading dimension of matrices from <code>a</code> belonging to group <code>g</code> ($lda_g \geq \max(1, n_g)$).
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index <code>g</code> specifies the number of problems to solve for each of the groups of parameters <code>g</code> . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by potrf_batch_scratchpad_size (Group Version) .
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a The matrices pointed to by array a are overwritten by the Cholesky factors U_i or L_i , as specified by `uplog` from the corresponding group of parameters.

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

potrf_batch (USM Strided Version)

Computes the Cholesky factorizations of a batch of symmetric (or Hermitian, for complex data) positive-definite matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine forms the Cholesky factorizations of a symmetric positive-definite or, for complex data, Hermitian positive-definite matrices A_i , $i \in \{1 \dots \text{batch_size}\}$:

- $A_i = U_i^T * U_i$ for real data, $A_i = U_i^H * U_i$ for complex data. if `uplo = mkl::uplo::upper`,
- $A_i = L_i^T * L_i$ for real data, $A_i = L_i^H * L_i$ for complex data if `uplo = mkl::uplo::lower`

Where L_i is a lower triangular matrix and U_i is an upper triangular matrix.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event potrf_batch(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        int64_t stride_a,
        int64_t batch_size,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Indicates whether the upper or lower triangular part of A_i is stored and how A_i is factored: If uplo = mkl::uplo::upper, the array a stores the upper triangular parts of the matrices A_i . If uplo = mkl::uplo::lower, the array a stores the lower triangular parts of the matrices A_i .
n	Specifies the order of the matrices A_i , ($n \geq 0$).
a	Array containing a batch of input matrices A_i , each of A_i being of size $lda \times n$ and holding either upper or lower triangular parts of the matrices A_i (see uplo).
lda	The leading dimension of A_i ($lda \geq \max(1, n)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch ($stride_a \geq \max(1, lda * n)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less then the value returned by potrf_batch_scratchpad_size (Strided Version) .
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a The batch array **a** is overwritten by the Cholesky factor U_i or L_i , as specified by **uplo**.

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <i>n</i>-th parameter had an illegal value. If <code>info</code> equals the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non-zero, then the passed <code>scratchpad</code> is of insufficient size, and the required size should be not less than value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

potrf_batch_scratchpad_size (Group Version)

Computes the size of scratchpad memory required for `potrf_batch` (Group Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `potrf_batch` (Group Version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t potrf_batch_scratchpad_size(sycl::queue &queue,
        mkl::uplo *uplo,
        int64_t *n,
        int64_t *lda,
```

```
int64_t group_count,
int64_t *group_sizes)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Array of <code>group_count</code> parameters <code>uplo_g</code>. Each of <code>uplo_g</code> indicates whether the upper or lower triangular parts of the input matrices are provided. If <code>uplo_g=mkl::uplo::upper</code>, input matrices from array <code>a</code> belonging to group <code>g</code> store the upper triangular parts. If <code>uplo_g=mkl::uplo::lower</code>, input matrices from array <code>a</code> belonging to group <code>g</code> store the lower triangular parts.
n	Array of <code>group_count</code> parameters <code>n_g</code>. Each <code>n_g</code> specifies the order of the input matrices belonging to group <code>g</code> ($n_g \geq 0$).
lda	Array of <code>group_count</code> parameters <code>lda_g</code>. Each <code>lda_g</code> specifies the leading dimension of the matrices belonging to group <code>g</code> ($lda_g \geq \max(1, n_g)$).
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index <code>g</code> specifies the number of problems to solve for each of the groups of parameters <code>g</code> . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the [potrf_batch \(Group Version\)](#) function must be able to hold.

potrf_batch_scratchpad_size (Strided Version)

Computes size of scratchpad memory required for `potrf_batch` (Strided Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `potrf_batch` (Strided Version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t potrf_batch_scratchpad_size(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda,
        int64_t stride_a,
        int64_t batch_size)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>uplo</code>	Indicates whether the upper or lower triangular part of A_i is stored and how A_i is factored: If <code>uplo = mkl::uplo::upper</code> , the array <code>a</code> stores the upper triangular parts of the matrices A_i . If <code>uplo = mkl::uplo::lower</code> , the array <code>a</code> stores the lower triangular parts of the matrices A_i .
<code>n</code>	Specifies the order of the matrices A_i , ($n \geq 0$).
<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, n)$).
<code>stride_a</code>	The stride between the beginnings of matrices A_i inside the batch ($stride_a \geq \max(1, lda * n)$).
<code>batch_size</code>	The number of problems in a batch ($batch_size \geq 0$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `potrf_batch` (Strided Version) function must be able to hold.

potrf_scratchpad_size

Computes size of scratchpad memory required for `potrf` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type T the scratchpad memory to be passed to the `potrf` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t potrf_scratchpad_size(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda)
}
```

Input Parameters

queue	Device queue where calculations by the <code>potrf</code> (buffer or USM version) function will be performed.
uplo	Indicates whether the upper or lower triangular part of A is stored and how A is factored: If <code>uplo=mkl::uplo::upper</code> , the array a stores the upper triangular part of the matrix A , and the strictly lower triangular part of the matrix is not referenced. If <code>uplo=mkl::uplo::lower</code> , the array a stores the lower triangular part of the matrix A , and the strictly upper triangular part of the matrix is not referenced.
n	Specifies the order of the matrix A ($0 \leq n$).
lda	The leading dimension of A .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `potrf` (buffer or USM version) function must be able to hold.

potri

Computes the inverse of a symmetric (Hermitian) positive-definite matrix using the Cholesky factorization. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the inverse $\text{inv}(A)$ of a symmetric positive definite or, for complex flavors, Hermitian positive-definite matrix `A`. Before calling this routine, call [potrf](#) to factorize `A`.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void potri(sycl::queue &queue,
              mkl::uplo uplo,
              int64_t n,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

`potri` supports the following precisions and devices:

T	Devices Supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Indicates how the input matrix <code>A</code> has been factored: If <code>uplo = mkl::uplo::upper</code> , the upper triangle of <code>A</code> is stored. If <code>uplo = mkl::uplo::lower</code> , the lower triangle of <code>A</code> is stored.
n	Specifies the order of the matrix <code>A</code> ($0 \leq n$).
a	Contains the factorization of the matrix <code>A</code> , as returned by potrf (USM Version). The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> .

scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the potri_scratchpad_size function.

Output Parameters

<code>a</code>	Overwritten by the upper or lower triangle of the inverse of <code>A</code> . Specified by <code>uplo</code> .
----------------	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info = i</code>, the <code>i</code>-th diagonal element of the Cholesky factor (and therefore the factor itself) is zero, and the inversion could not be completed. If <code>info</code> is equal to the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

potri (USM Version)

Computes the inverse of a symmetric (Hermitian) positive-definite matrix using the Cholesky factorization. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the inverse $\text{inv}(A)$ of a symmetric positive definite or, for complex flavors, Hermitian positive-definite matrix `A`. Before calling this routine, call [potrf \(USM Version\)](#) to factorize `A`.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event potri(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        T* a,
        int64_t lda,
        T *scratchpad,
```

```
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {}))
}
```

potri (USM version) supports the following precisions and devices:

T	Devices Supported
float	CPU, GPU
double	CPU, GPU
std::complex<float>	CPU, GPU
std::complex<double>	CPU, GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Indicates how the input matrix A has been factored: If <code>uplo = mkl::uplo::upper</code> , the upper triangle of A is stored. If <code>uplo = mkl::uplo::lower</code> , the lower triangle of A is stored.
n	Specifies the order of the matrix A ($0 \leq n$).
a	Contains the factorization of the matrix A, as returned by potrf (USM Version) . The size of a must be at least <code>lda*n</code> .
lda	The leading dimension of a.
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by the potri_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	Overwritten by the upper or lower triangle of the inverse of A. Specified by uplo.
---	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, the <i>i</i>-th diagonal element of the Cholesky factor (and therefore the factor itself) is zero, and the inversion could not be completed.

Exception	Description
	If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

potri_scratchpad_size

Computes size of scratchpad memory required for `potri` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `potri` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {  
    template<typename T>  
    int64_t potri_scratchpad_size(sycl::queue &queue,  
                                mkl::uplo uplo,  
                                int64_t n,  
                                int64_t lda)  
}
```

Input Parameters

queue	Device queue where calculations by the <code>potri</code> (buffer or USM version) function will be performed.
uplo	Indicates how the input matrix <code>A</code> has been factored: If <code>uplo = mkl::uplo::upper</code> , the upper triangle of <code>A</code> is stored. If <code>uplo = mkl::uplo::lower</code> , the lower triangle of <code>A</code> is stored.
n	Specifies the order of the matrix <code>A</code> ($0 \leq n$).
lda	The leading dimension of <code>a</code> .

Exceptions

Exception	Description
<code>mk1::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `potri` (buffer or USM version) function must be able to hold.

potrs

Solves a system of linear equations with a Cholesky-factored symmetric (Hermitian) positive-definite coefficient matrix. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X the system of linear equations $A \cdot X = B$ with a symmetric positive-definite or, for complex data, Hermitian positive-definite matrix A , given the Cholesky factorization of A :

$A = UT^*U$ for real data, $A = UH^*U$ for complex data if `uplo=mk1::uplo::upper`
 $A = L^*LT$ for real data, $A = L^*LH$ for complex data if `uplo=mk1::uplo::lower`

where L is a lower triangular matrix and U is upper triangular. The system is solved with multiple right-hand sides stored in the columns of the matrix B .

Before calling this routine, you must call [potrf](#) to compute the Cholesky factorization of A .

API

Syntax

```
namespace oneapi::mk1::lapack {
    void potrs(sycl::queue &queue, mk1::uplo uplo,
              int64_t n,
              int64_t nrhs,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &b,
              int64_t ldb,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

`potrs` supports the following precisions and devices:

T	Devices Supported
float	CPU and GPU
double	CPU and GPU

T	Devices Supported
<code>std::complex<float></code>	CPU and GPU
<code>std::complex<double></code>	CPU and GPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>uplo</code>	Indicates how the input matrix has been factored: If <code>uplo=mkl::uplo::upper</code>, the upper triangle U of A is stored, where $A = U^T * U$ for real data, $A = U^H * U$ for complex data. If <code>uplo=mkl::uplo::lower</code>, the lower triangle L of A is stored, where $A = L * L^T$ for real data, $A = L * L^H$ for complex data.
<code>n</code>	The order of matrix A ($0 \leq n$).
<code>nrhs</code>	The number of right-hand sides ($0 \leq nrhs$).
<code>a</code>	Buffer holding factorization of the matrix A , as returned by potrf . The size of <code>a</code> must be at least <code>lda*n</code> .
<code>lda</code>	The leading dimension of <code>a</code> .
<code>b</code>	Buffer holding the data of matrix B whose columns are the right-hand sides for the systems of equations. The size of <code>b</code> must be at least <code>ldb*nrhs</code> .
<code>ldb</code>	The leading dimension of <code>b</code> .
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the potrs_scratchpad_size function.

Output Parameters

<code>b</code>	Buffer <code>b</code> is overwritten by the solution matrix X .
----------------	---

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, the <i>i</i>-th diagonal element of the Cholesky factor is zero, and the solve could not be completed. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and

Exception	Description
	the required size should not be less than the value returned by the detail() method of the exception object.

potrs (USM Version)

Solves a system of linear equations with a Cholesky-factored symmetric (Hermitian) positive-definite coefficient matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X the system of linear equations $A \cdot X = B$ with a symmetric positive-definite or, for complex data, Hermitian positive-definite matrix A , given the Cholesky factorization of A :

$A = U^T \cdot U$ for real data, $A = U^H \cdot U$ for complex data	if <code>uplo=mkl::uplo::upper</code>
$A = L \cdot L^T$ for real data, $A = L \cdot L^H$ for complex data	if <code>uplo=mkl::uplo::lower</code>

where L is a lower triangular matrix and U is upper triangular. The system is solved with multiple right-hand sides stored in the columns of the matrix B .

Before calling this routine, you must call [potrf \(USM Version\)](#) to compute the Cholesky factorization of A .

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event potrs(
        sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        int64_t nrhs,
        const T *a,
        int64_t lda,
        T *b,
        int64_t ldb,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`potrs` (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU

T	Devices supported
<code>std::complex<double></code>	CPU and GPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>uplo</code>	Indicates how the input matrix has been factored: If <code>uplo=mkl::uplo::upper</code> , the upper triangle U of A is stored, where $A = U^T * U$ for real data, $A = U^H * U$ for complex data. If <code>uplo=mkl::uplo::lower</code> , the lower triangle L of A is stored, where $A = L * L^T$ for real data, $A = L * L^H$ for complex data.
<code>n</code>	The order of matrix A ($0 \leq n$).
<code>nrhs</code>	The number of right-hand sides ($0 \leq nrhs$).
<code>a</code>	Pointer to factorization of the matrix A , as returned by potrf (USM Version) . The size of <code>a</code> must be at least <code>lda*n</code> .
<code>lda</code>	The leading dimension of <code>a</code> .
<code>b</code>	Pointer to the data of matrix B whose columns are the right-hand sides for the systems of equations. The size of <code>b</code> must be at least <code>ldb*nrhs</code> .
<code>ldb</code>	The leading dimension of <code>b</code> .
<code>scratchpad</code>	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the potrs_scratchpad_size function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

<code>b</code>	The memory pointed to by pointer <code>b</code> is overwritten by the solution matrix X .
----------------	---

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, the <i>i</i>-th diagonal element of the Cholesky factor is zero, and the solve could not be completed.

Exception	Description
	If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

potrs_batch (Buffer Strided Version)

Solves a system of linear equations with a Cholesky-factored symmetric (Hermitian) positive-definite coefficient matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X_i the system of linear equations $A_i * X_i = B_i$ with a symmetric positive-definite or, for complex data, Hermitian positive-definite matrices A_i , given the Cholesky factorization of A_i , $i \in \{1 \dots \text{batch_size}\}$:

- $A_i = U_i^T * U_i$ for real data, $A_i = U_i^H * U_i$ for complex data if `uplo=mkl::uplo::upper`
- $A_i = L_i * L_i^T$ for real data, $A_i = L_i * L_i^H$ for complex data if `uplo=mkl::uplo::lower`

where L_i is a lower triangular matrix and U_i is upper triangular. The system is solved with multiple right-hand sides stored in the columns of the matrix B_i .

Before calling this routine, matrices A_i should be factorized by a call to [potrf_batch \(Buffer Strided Version\)](#).

API

Syntax

```
namespace oneapi::mkl::lapack {
    void potrs_batch(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        int64_t nrhs,
        sycl::buffer<T> &a,
        int64_t lda,
        int64_t stride_a,
        sycl::buffer<T> &b,
        int64_t ldb,
        int64_t stride_b,
        int64_t batch_size,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Indicates how the input matrix has been factored: If <code>uplo=mkl::uplo::upper</code> , the upper triangle U_i of A_i is stored, where $A_i = U_i^T * U_i$ for real data, $A_i = U_i^H * U_i$ for complex data. If <code>uplo=mkl::uplo::lower</code> , the lower triangle L_i of A_i is stored, where $A_i = L_i * L_i^T$ for real data, $A_i = L_i * L_i^H$ for complex data.
n	The order of the matrices A_i ($n \geq 0$).
nrhs	The number of right hand sides ($nrhs \geq 0$).
a	Array containing the batch of factorizations of the matrices A_i , as returned by potrf_batch (Buffer Strided Version) .
lda	The leading dimension of A_i ($lda \geq \max(1, n)$).
stride_a	The stride between the beginnings of matrices inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
b	The array containing the batch of matrices B_i whose columns are the right-hand sides for the systems of equations.
ldb	The leading dimensions of B_i ($ldb \geq \max(1, n)$).
stride_b	The stride between the beginnings of matrices B_i inside the batch array <code>b</code> ($stride_b \geq \max(1, ldb * nrhs)$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by stride version of potrs_batch_scratchpad_size (Strided Version) function.

Output Parameters

b	The batch array <code>b</code> is overwritten by the solution matrix X_i .
---	--

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

potrs_batch (Group Version)

Solves a batch of systems of linear equations with a Cholesky-factored symmetric (Hermitian) positive-definite coefficient matrices. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X_i the systems of linear equations $A_i * X_i = B_i$ with a symmetric positive-definite or, for complex data, Hermitian positive-definite matrices A_i , given the Cholesky factorization of A_i , $i \in \{1 \dots \text{batch_size}\}$:

- $A_i = U_i^T * U_i$ for real data, $A_i = U_i^H * U_i$ for complex data if `uplog=mkl::uplo::upper`
- $A_i = L_i * L_i^T$ for real data, $A_i = L_i * L_i^H$ for complex data if `uplog=mkl::uplo::lower`

where L_i is a lower triangular matrix and U_i is upper triangular, g is an index of group of parameters corresponding to A_i , and the total number of problems to solve, `batch_size`, is a sum of sizes for all of the groups of parameters as provided by the `group_sizes` array.

The systems are solved with multiple right-hand sides stored in the columns of the matrices B_i .

Before calling this routine, matrices A_i should be factorized by a call to [potrf_batch \(Group Version\)](#).

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event potrs_batch(sycl::queue &queue,
        mkl::uplo *uplo, int64_t *n,
        int64_t *nrhs,
        const T * const *a,
```

```

int64_t *lda,
T **b,
int64_t *ldb,
int64_t group_count,
int64_t *group_sizes,
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
}

```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Array of group_count uplo_g parameters. Each of uplo_g indicates whether the upper or lower triangular parts of the input matrices are provided: If $\text{uplo}_g = \text{mkl::uplo::upper}$, input matrices from array a belonging to group g store the upper triangular parts. If $\text{uplo}_g = \text{mkl::uplo::lower}$, input matrices from array a belonging to group g store the lower triangular parts.
n	Array of group_count parameters n_g. Each n_g specifies the order of the input matrices from array a belonging to group g ($n_g \geq 0$).
nrhs	Array of group_count parameters nrhs_g parameters. Each nrhs_g specifies the number of right-hand sides supplied for group g in corresponding part of array b ($\text{nrhs}_g \geq 0$).
a	Array of batch_size pointers to Cholesky factored matrices A_i as returned by potrf_batch (Group Version) .
lda	Array of group_count parameters lda_g. Each lda_g specifies the leading dimension of matrices from a belonging to group g ($\text{lda}_g \geq \max(1, n_g)$).
b	Array of batch_size pointers to right-hand side matrices B_i , each of size $\text{ldb}_g * \text{nrhs}_g$, where g is an index of group corresponding to B_i .
ldb	Array of group_count parameters ldb_g .

Each ldb_g specifies the leading dimension of matrices from $\mathbf{b}$ belonging to group g ($\text{ldb}_g \geq \max(1, n_g)$).

group_count

Specifies the number of groups of parameters. Must be at least 0.

group_sizes

Array of group_count integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, `batch_size`, is a sum of all parameter group sizes.

scratchpad

Scratchpad memory to be used by routine for storing intermediate results.

scratchpad_size

Size of scratchpad memory as a number of floating point elements of type T . Size should not be less then the value returned by [potrs_batch_scratchpad_size \(Group Version\)](#).

events

List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

$\mathbf{b}$

The matrices pointed to by array $\mathbf{b}$ are overwritten by the solution matrices $\mathbf{x}_i$.

Exceptions

Exception	Description
<code>mk1::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of $\mathbf{U}_i$ is zero, and the solve could not be completed. The indexes of such matrices in the batch can be obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these $\mathbf{U}_i$ matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

[potrs_batch \(USM Strided Version\)](#)

Solves a system of linear equations with a Cholesky-factored symmetric (Hermitian) positive-definite coefficient matrices. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X_i the system of linear equations $A_i * X_i = B_i$ with a symmetric positive-definite or, for complex data, Hermitian positive-definite matrices A_i , given the Cholesky factorization of A_i , $i \in \{1 \dots \text{batch_size}\}$:

- $A_i = U_i^T * U_i$ for real data, $A_i = U_i^H * U_i$ for complex data if `uplo=mkl::uplo::upper`
- $A_i = L_i * L_i^T$ for real data, $A_i = L_i * L_i^H$ for complex data if `uplo=mkl::uplo::lower`

where L_i is a lower triangular matrix and U_i is upper triangular. The system is solved with multiple right-hand sides stored in the columns of the matrix B_i .

Before calling this routine, matrices A_i should be factorized by a call to [potrf_batch \(USM Strided Version\)](#).

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event potrs_batch(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        int64_t nrhs,
        const T *a,
        int64_t lda,
        int64_t stride_a,
        T *b,
        int64_t ldb,
        int64_t stride_b,
        int64_t batch_size,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

- queue** Device queue where calculations will be performed.
- uplo** Indicates how the input matrix has been factored:
 If `uplo=mkl::uplo::upper`, the upper triangle U_i of A_i is stored, where $A_i = U_i^T * U_i$ for real data, $A_i = U_i^H * U_i$ for complex data.

If `uplo=mkl::uplo::lower`, the upper triangle L_i of A_i is stored, where $A_i = L_i * L_i^T$ for real data, $A_i = L_i * L_i^H$ for complex data.

<code>n</code>	The order of the matrices A_i ($n \geq 0$).
<code>nrhs</code>	The number of right hand sides ($nrhs \geq 0$).
<code>a</code>	Array containing the batch of factorizations of the matrices A_i , as returned by potrf_batch (USM Strided Version) .
<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, n)$).
<code>stride_a</code>	The stride between the beginnings of matrices inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
<code>b</code>	The array containing the batch of matrices B_i whose columns are the right-hand sides for the systems of equations.
<code>ldb</code>	The leading dimensions of B_i ($ldb \geq \max(1, n)$).
<code>stride_b</code>	The stride between the beginnings of matrices B_i inside the batch array <code>b</code> ($stride_b \geq \max(1, ldb * nrhs)$).
<code>batch_size</code>	The number of problems in a batch ($batch_size \geq 0$).
<code>scratchpad</code>	Scratchpad memory to be used by routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less then the value returned by stride version of potrs_batch_scratchpad_size (Strided Version) function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
<code>b</code>	The memory pointed to by pointer batch array <code>b</code> is overwritten by the solution matrix X_i .

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object. If <code>info</code> is zero, then the diagonal element of some of U_i is zero, and the solve could not be completed. The indexes of such matrices in the batch can be

Exception	Description
	obtained with the <code>ids()</code> method of the exception object. You can obtain the indexes of the first zero diagonal elements in these U_i matrices using the <code>infos()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

potrs_batch_scratchpad_size (Group Version)

Computes size of scratchpad memory required for `potrs_batch` (Group Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `potrs_batch` (Group Version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t potrs_batch_scratchpad_size(
        sycl::queue &queue,
        mkl::uplo *uplo,
        int64_t *n,
        int64_t *nrhs,
        int64_t *lda,
        int64_t *ldb,
        int64_t group_count,
        int64_t *group_sizes)
}
```

This function supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Array of <code>group_count</code> parameters <code>uplog</code> .

Each of uplo_g indicates whether the upper or lower triangular parts of the input matrices are provided.

If $\text{uplo}_g = \text{mkl::uplo::upper}$, input matrices from array a belonging to group g store the upper triangular parts.

If $\text{uplo}_g = \text{mkl::uplo::lower}$, input matrices from array a belonging to group g store the lower triangular parts.

n	Array of <code>group_count</code> parameters n_g. Each n_g specifies the order of the input matrices belonging to group g ($n_g \geq 0$).
nrhs	Array of nrhs_g parameters. Each nrhs_g specifies the number of right-hand sides supplied for to group g ($\text{nrhs}_g \geq 0$).
lda	Array of <code>group_count</code> parameters lda_g. Each lda_g specifies the leading dimension of the matrices belonging to group g ($\text{lda}_g \geq \max(1, n_g)$).
ldb	Array of <code>group_count</code> parameters ldb_g. Each ldb_g specifies the leading dimension of the matrices belonging to group g ($\text{ldb}_g \geq \max(1, n_g)$).
<code>group_count</code>	Specifies the number of groups of parameters. Must be at least 0.
<code>group_sizes</code>	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g. So the total number of problems to solve, <code>batch_size</code>, is a sum of all parameter group sizes.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the [potrs_batch \(Group Version\)](#) function must be able to hold.

[potrs_batch_scratchpad_size \(Strided Version\)](#)

Computes size of scratchpad memory required for `potrs_batch (Strided Version)` function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `potrs_batch` (Strided Version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t potrs_batch_scratchpad_size(
        sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        int64_t nrhs,
        int64_t lda,
        int64_t stride_a,
        int64_t ldb,
        int64_t stride_b,
        int64_t batch_size)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>uplo</code>	Indicates how the input matrix has been factored: If <code>uplo=mkl::uplo::upper</code> , the upper triangle U_i of A_i is stored, where $A_i = U_i^T * U_i$ for real data, $A_i = U_i^H * U_i$ for complex data. If <code>uplo=mkl::uplo::lower</code> , the lower triangle L_i of A_i is stored, where $A_i = L_i * L_i^T$ for real data, $A_i = L_i * L_i^H$ for complex data.
<code>n</code>	The order of the matrices A_i ($n \geq 0$).
<code>nrhs</code>	The number of right hand sides ($nrhs \geq 0$).
<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, n)$).
<code>stride_a</code>	The stride between the beginnings of matrices inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
<code>ldb</code>	The leading dimensions of B_i ($ldb \geq \max(1, n)$).
<code>stride_b</code>	The stride between the beginnings of matrices B_i inside the batch array <code>b</code> ($stride_b \geq \max(1, ldb * nrhs)$).
<code>batch_size</code>	The number of problems in a batch ($batch_size \geq 0$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `potrs_batch` (Strided Version) function must be able to hold.

potrs_scratchpad_size

Computes size of scratchpad memory required for `potrs` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `potrs` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t potrs_scratchpad_size(
        sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        int64_t nrhs,
        int64_t lda,
        int64_t ldb)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>potrs</code> (buffer or USM version) function will be performed.
<code>uplo</code>	Indicates how the input matrix has been factored: If <code>uplo=mkl::uplo::upper</code> , the upper triangle U of A is stored, where $A = U^T * U$ for real data, $A = U^H * U$ for complex data. If <code>uplo=mkl::uplo::lower</code> , the lower triangle L of A is stored, where $A = L * L^T$ for real data, $A = L * L^H$ for complex data.
<code>n</code>	The order of matrix A ($0 \leq n$).
<code>nrhs</code>	The number of right-hand sides ($0 \leq nrhs$).
<code>lda</code>	The leading dimension of A .
<code>ldb</code>	The leading dimension of B .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `potrs` (buffer or USM version) function must be able to hold.

syevd

Computes all eigenvalues and, optionally, all eigenvectors of a real symmetric matrix using divide and conquer algorithm. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes all the eigenvalues, and optionally all the eigenvectors, of a real symmetric matrix A . In other words, it can compute the spectral factorization of A as: $A = Z \Lambda Z^T$.

Here Λ is a diagonal matrix whose diagonal elements are the eigenvalues λ_i , and Z is the orthogonal matrix whose columns are the eigenvectors z_i . Thus,

$$A z_i = \lambda_i z_i \text{ for } i = 1, 2, \dots, n.$$

If the eigenvectors are requested, then this routine uses a divide and conquer algorithm to compute eigenvalues and eigenvectors. However, if only eigenvalues are required, then it uses the Pal-Walker-Kahan variant of the QL or QR algorithm.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void syevd(sycl::queue &queue,
              mkl::job jobz,
              mkl::uplo uplo,
              int64_t n,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &w,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

`syevd` supports the following precision and devices.

T	Devices Supported
float	CPU and GPU*
double	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
jobz	Must be <code>jobz::novec</code> or <code>jobz::vec</code> . If <code>jobz = jobz::novec</code> , then only eigenvalues are computed. If <code>jobz = jobz::vec</code> , then eigenvalues and eigenvectors are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , <code>a</code> stores the upper triangular part of <code>A</code> . If <code>uplo = uplo::lower</code> , <code>a</code> stores the lower triangular part of <code>A</code> .
n	The order of the matrix <code>A</code> ($0 \leq n$).
a	Buffer holding the array containing the upper or lower triangle of the symmetric matrix <code>A</code> , as specified by <code>uplo</code> . The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the syevd_scratchpad_size function.

Output Parameters

a	If <code>jobz = jobz::vec</code> , then on exit this buffer is overwritten by the orthogonal matrix <code>Z</code> which contains the eigenvectors of <code>A</code> .
w	Buffer holding array of size at least <code>n</code> . If <code>info = 0</code> , contains the eigenvalues of the matrix <code>A</code> in ascending order.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the <code>info</code> code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info = i</code>, and <code>jobz = jobz::novec</code>, then the algorithm failed to converge; <code>i</code> indicates the number of off-diagonal elements of an intermediate tridiagonal form which did not converge to zero.

Exception	Description
	If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue while working on the submatrix lying in rows and columns <code>info/(n+1)</code> through <code>mod(info,n+1)</code>. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

syevd (USM Version)

Computes all eigenvalues and, optionally, all eigenvectors of a real symmetric matrix using divide and conquer algorithm. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes all the eigenvalues, and optionally all the eigenvectors, of a real symmetric matrix A . In other words, it can compute the spectral factorization of A as: $A = Z \Lambda Z^T$.

Here Λ is a diagonal matrix whose diagonal elements are the eigenvalues λ_i , and Z is the orthogonal matrix whose columns are the eigenvectors z_i . Thus,

$$A z_i = \lambda_i z_i \text{ for } i = 1, 2, \dots, n.$$

If the eigenvectors are requested, then this routine uses a divide and conquer algorithm to compute eigenvalues and eigenvectors. However, if only eigenvalues are required, then it uses the Pal-Walker-Kahan variant of the QL or QR algorithm.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event syevd(sycl::queue &queue,
        mkl::job jobz,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        T *w,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`syevd` (USM version) supports the following precision and devices.

T	Devices Supported
float	CPU and GPU*

T	Devices Supported
double	CPU and GPU [^]

*Interface support only; all computations are performed on the CPU.

[^]Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
jobz	Must be <code>job::novec</code> or <code>job::vec</code>. If <code>jobz = job::novec</code>, then only eigenvalues are computed. If <code>jobz = job::vec</code>, then eigenvalues and eigenvectors are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> stores the upper triangular part of <code>A</code>. If <code>uplo = uplo::lower</code>, <code>a</code> stores the lower triangular part of <code>A</code>.
n	The order of the matrix <code>A</code> ($0 \leq n$).
a	The pointer to the array containing <code>A</code> , size <code>(lda, *)</code> . The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the syevd_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	If <code>jobz = job::vec</code> , then on exit this is overwritten by the orthogonal matrix <code>Z</code> which contains the eigenvectors of <code>A</code> .
w	Pointer to array of size at least <code>n</code> . Contains the eigenvalues of the matrix <code>A</code> in ascending order.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value.

Exception	Description
	If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge; <code>i</code> indicates the number of off-diagonal elements of an intermediate tridiagonal form which did not converge to zero. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue while working on the submatrix lying in rows and columns <code>info/(n+1)</code> through <code>mod(info,n+1)</code>. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

syevd_scratchpad_size

Computes size of scratchpad memory required for `syevd` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `syevd` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t syevd_scratchpad_size(sycl::queue &queue,
        mkl::job jobz,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>syevd</code> (buffer or USM version) function will be performed.
<code>jobz</code>	Must be <code>job::novec</code> or <code>job::vec</code>. If <code>jobz = job::novec</code>, then only eigenvalues are computed.

If `jobz = job::vec`, then eigenvalues and eigenvectors are computed.

`uplo`

Must be `uplo::upper` or `uplo::lower`.

If `uplo = uplo::upper`, `a` stores the upper triangular part of `A`.

If `uplo = uplo::lower`, `a` stores the lower triangular part of `A`.

`n`

The order of the matrix `A` ($0 \leq n$).

`lda`

The leading dimension of `a`. Must be at least `max(1, n)`.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `syevd` (buffer or USM version) function must be able to hold.

syevx

Computes selected eigenvalues and, optionally, eigenvectors of a real symmetric matrix `A`. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes selected eigenvalues, and optionally eigenvectors, of a real symmetric matrix `A`. Eigenvalues and eigenvectors can be selected by specifying either a range of values or a range of indices for the desired eigenvalues. In other words, it can compute selected eigenpairs (λ, z) of `A` such that $A*z = z*\lambda$.

The routine computes selected eigenvalues with the bisection algorithm.

If the eigenvectors are requested, then this routine uses a combination of modified twisted factorization algorithm based on Inderjit Dhillon and Beresford Parlett's work and the inverse iteration algorithm followed by Gram-Schmidt orthogonalization.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void syevx(sycl::queue &queue,
              mkl::job jobz,
              mkl::rangev range,
              mkl::uplo uplo,
              int64_t n,
              sycl::buffer<T> &a,
```



```

int64_t lda,
T vl,
T vu,
int64_t il,
int64_t iu,
T abstol,
sycl::buffer<int64_t> &m,
sycl::buffer<T> &w,
sycl::buffer<T> &z,
int64_t ldz,
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}

```

syevx supports the following precision and devices.

T	Devices Supported
float	CPU and GPU*
double	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
jobz	Must be <code>jobz::novec</code> or <code>jobz::vec</code>. If <code>jobz = jobz::novec</code>, then only eigenvalues are computed. If <code>jobz = jobz::vec</code>, then eigenvalues and eigenvectors are computed.
range	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $[vl, vu]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::indices</code>, then the <code>il</code>-th through <code>iu</code>-th eigenvalues and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> stores the upper triangular part of A. If <code>uplo = uplo::lower</code>, <code>a</code> stores the lower triangular part of A.
n	The order of the matrix A ($0 \leq n$).
a	Buffer holding the array containing the upper or lower triangle of the symmetric matrix A , as specified by <code>uplo</code> . The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> . Must be at least $\max(1, n)$.
vl	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .

<code>vu</code>	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>il</code>	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if <code>n > 0</code> and <code>il = 1</code> and <code>iu = 0</code> if <code>n = 0</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>iu</code>	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if <code>n > 0</code> and <code>il = 1</code> and <code>iu = 0</code> if <code>n = 0</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>abstol</code>	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval $[a,b]$ of width less than or equal to <code>abstol + eps * max(a , b)</code> , where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then <code>eps* T </code> will be used in its place, where <code> T </code> is the 1-norm of the tridiagonal matrix obtained by reducing <code>A</code> to tridiagonal form.
<code>ldz</code>	The leading dimension of <code>z</code> . Must be at least <code>max(1, n)</code> .
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the syevx_scratchpad_size function.
Output Parameters	
<code>a</code>	On exit the upper triangle (if <code>uplo = uplo::upper</code>) or the lower triangle (if <code>uplo = uplo::lower</code>) of <code>A</code> , including the diagonal, is destroyed.
<code>m</code>	The total number of eigenvalues found, $0 \leq m \leq n$.
<code>w</code>	Buffer holding array of size at least <code>n</code> . On normal exit, the first <code>m</code> elements contain the selected eigenvalues in ascending order.
<code>z</code>	If <code>jobz = job::vec</code> , then on normal exit the first <code>m</code> columns of <code>Z</code> contain the orthonormal eigenvectors of the matrix <code>A</code> corresponding to the selected eigenvalues, with the <code>i</code> -th column of <code>Z</code> holding the eigenvector associated with <code>w(i)</code> . If <code>jobz = job::novec</code> , then <code>Z</code> is not referenced. Note: the user must ensure that at least <code>max(1, m)</code> columns are supplied in the array <code>Z</code> ; if <code>range = rangev::values</code> , the exact value of <code>m</code> is not known in advance and an upper bound must be used.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object:

Exception	Description
	If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

syevx (USM Version)

Computes selected eigenvalues and, optionally, eigenvectors of a real symmetric matrix `A`. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes selected eigenvalues, and optionally eigenvectors, of a real symmetric matrix `A`. Eigenvalues and eigenvectors can be selected by specifying either a range of values or a range of indices for the desired eigenvalues. In other words, it can compute selected eigenpairs (λ, z) of `A` such that $A * z = z * \lambda$.

The routine computes selected eigenvalues with the bisection algorithm.

If the eigenvectors are requested, then this routine uses a combination of modified twisted factorization algorithm based on Inderjit Dhillon and Beresford Parlett's work and the inverse iteration algorithm followed by Gram-Schmidt orthogonalization.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event syevx(sycl::queue &queue,
        mkl::job jobz,
        mkl::rangev range,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        T vl,
        T vu,
        int64_t il,
        int64_t iu,
        T abstol,
        int64_t *m,
        T *w,
        T *z,
```

```

int64_t ldz,
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
}

```

syevx (USM version) supports the following precision and devices.

T	Devices Supported
float	CPU and GPU*
double	CPU and GPU^

*Interface support only; all computations are performed on the CPU.

^Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
jobz	Must be <code>jobz::novec</code> or <code>jobz::vec</code>. If <code>jobz = jobz::novec</code>, then only eigenvalues are computed. If <code>jobz = jobz::vec</code>, then eigenvalues and eigenvectors are computed.
range	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $[v_l, v_u]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::indices</code>, then the i_l-th through i_u-th eigenvalues and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> stores the upper triangular part of A. If <code>uplo = uplo::lower</code>, <code>a</code> stores the lower triangular part of A.
n	The order of the matrix A ($0 \leq n$).
a	Pointer to the array containing the upper or lower triangle of the symmetric matrix A , as specified by <code>uplo</code> . The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> . Must be at least $\max(1, n)$.
vl	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
vu	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .

<code>il</code>	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>iu</code>	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>abstol</code>	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval $[a,b]$ of width less than or equal to $abstol + eps * \max(a , b)$, where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then $eps * T $ will be used in its place, where $ T $ is the 1-norm of the tridiagonal matrix obtained by reducing <code>A</code> to tridiagonal form.
<code>ldz</code>	The leading dimension of <code>z</code> . Must be at least $\max(1, n)$.
<code>scratchpad</code>	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the syevx_scratchpad_size function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
<code>a</code>	On exit the upper triangle (if <code>uplo = uplo::upper</code>) or the lower triangle (if <code>uplo = uplo::lower</code>) of <code>A</code> , including the diagonal, is destroyed.
<code>m</code>	The total number of eigenvalues found, $0 \leq m \leq n$.
<code>w</code>	Pointer to array of size at least <code>n</code> . On normal exit, the first <code>m</code> elements contain the selected eigenvalues of the matrix <code>A</code> in ascending order.
<code>z</code>	If <code>jobz = job::vec</code> , then on normal exit the first <code>m</code> columns of <code>Z</code> contain the orthonormal eigenvectors of the matrix <code>A</code> corresponding to the selected eigenvalues, with the <i>i</i> -th column of <code>Z</code> holding the eigenvector associated with <code>w(i)</code> . If <code>jobz = job::novec</code> , then <code>Z</code> is not referenced. Note: the user must ensure that at least $\max(1, m)$ columns are supplied in the array <code>Z</code> ; if <code>range = rangev::values</code> , the exact value of <code>m</code> is not known in advance and an upper bound must be used.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value.

Exception	Description
	If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

syevx_scratchpad_size

Computes size of scratchpad memory required for `syevx` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `syevx` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t syevx_scratchpad_size(sycl::queue &queue,
        mkl::job jobz,
        mkl::rangev range,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda,
        T vl,
        T vu,
        int64_t il,
        int64_t iu,
        T abstol,
        int64_t ldz)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>syevx</code> (buffer or USM version) function will be performed.
<code>jobz</code>	Must be <code>job::novec</code> or <code>job::vec</code> .

	If <code>jobz = jobz::novec</code>, then only eigenvalues are computed. If <code>jobz = jobz::vec</code>, then eigenvalues and eigenvectors are computed.
<code>range</code>	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $(v_l, v_u]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::indices</code>, then the <code>il</code>-th through <code>iu</code>-th eigenvalues and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.
<code>uplo</code>	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> stores the upper triangular part of A. If <code>uplo = uplo::lower</code>, <code>a</code> stores the lower triangular part of A.
<code>n</code>	The order of the matrix A ($0 \leq n$).
<code>lda</code>	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
<code>vl</code>	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>vu</code>	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>il</code>	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and <code>il = 1</code> and <code>iu = 0</code> if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>iu</code>	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and <code>il = 1</code> and <code>iu = 0</code> if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>abstol</code>	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval $[a, b]$ of width less than or equal to <code>abstol + eps * max(a , b)</code> , where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then <code>eps* T </code> will be used in its place, where $ T $ is the 1-norm of the tridiagonal matrix obtained by reducing A to tridiagonal form.
<code>ldz</code>	The leading dimension of <code>z</code> . Must be at least <code>max(1, n)</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `syevx` (buffer or USM version) function must be able to hold.

sygvd

Computes all eigenvalues and, optionally, eigenvectors of a real generalized symmetric definite eigenproblem using a divide and conquer method. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes all the eigenvalues, and optionally, the eigenvectors of a real generalized symmetric-definite eigenproblem, of the form

$$A \cdot x = \lambda \cdot B \cdot x, \quad A \cdot B \cdot x = \lambda \cdot x, \quad \text{or} \quad B \cdot A \cdot x = \lambda \cdot x.$$

Here `A` and `B` are assumed to be symmetric and `B` is also positive definite.

It uses a divide and conquer algorithm.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void sygvd(sycl::queue &queue,
               int64_t itype,
               mkl::job jobz,
               mkl::uplo uplo,
               int64_t n,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<T> &b,
               int64_t ldb,
               sycl::buffer<T> &w,
               sycl::buffer<T> &scratchpad,
               int64_t scratchpad_size)
}
```

`sygvd` supports the following precisions and devices:

T	Devices supported
float	CPU, GPU*
double	CPU, GPU^

*Interface support only; all computations are performed on the CPU.

^Hybrid support; some computations are performed on the CPU.

Input Parameters

- queue
- Device queue where calculations will be performed.
- itype
- Must be 1 or 2 or 3. Specifies the problem type to be solved:

if `itype= 1`, the problem type is $A*x = \lambda*B*x$;

if `itype= 2`, the problem type is $A*B*x = \lambda*x$;

if `itype= 3`, the problem type is $B*A*x = \lambda*x$.

jobz Must be `job::novec` or `job::vec`.

If `jobz = job::novec`, then only eigenvalues are computed.

If `jobz = job::vec`, then eigenvalues and eigenvectors are computed.

uplo Must be `uplo::upper` or `uplo::lower`.

If `uplo = uplo::upper`, `a` and `b` store the upper triangular part of `A` and `B`.

If `uplo = uplo::lower`, `a` and `b` stores the lower triangular part of `A` and `B`.

n The order of the matrices `A` and `B` ($0 \leq n$).

a Buffer holding the array containing the upper or lower triangle of the symmetric matrix `A`, as specified by `uplo`. The size of `a` must be at least `lda*n`.

lda The leading dimension of `a`; at least `max(1, n)`.

b Buffer holding the array containing the upper or lower triangle of the symmetric matrix `B`, as specified by `uplo`. The size of `b` must be at least `ldb*n`.

ldb The leading dimension of `b`; at least `max(1, n)`.

scratchpad Buffer holding scratchpad memory to be used by the routine for storing intermediate results.

scratchpad_size Size of scratchpad memory as a number of floating point elements of type `T`. Size should not be less than the value returned by the [sygvds_scratchpad_size](#) function.

Output Parameters

a On exit, if `jobz = job::vec`, then if `info = 0`, `a` contains the matrix `Z` of eigenvectors. The eigenvectors are normalized as follows:

if `itype= 1 or 2`, $Z^T*B*Z = I$;

if `itype= 3`, $Z^T*inv(B)*Z = I$;

If `jobz = job::novec`, then on exit the upper triangle (if `uplo = uplo::upper`) or the lower triangle (if `uplo = uplo::lower`) of `A`, including the diagonal, is destroyed.

b On exit, if `info ≤ n`, the part of `b` containing the matrix is overwritten by the triangular factor `U` or `L` from the Cholesky factorization $B = U^T*U$ or $B = L*L^T$.

w Buffer, size at least `n`. If `info = 0`, contains the eigenvalues of the matrix `A` in ascending order. See also `info`.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. For <code>info ≤ n</code>: If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge; <i>i</i> indicates the number of off-diagonal elements of an intermediate tridiagonal form which did not converge to zero. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue while working on the submatrix lying in rows and columns <code>info/(n+1)</code> through <code>mod(info, n+1)</code>. For <code>info > n</code>: If <code>info = n + i</code>, for $1 \leq i \leq n$, then the leading minor of order <i>i</i> of <i>B</i> is not positive-definite. The factorization of <i>B</i> could not be completed and no eigenvalues or eigenvectors were computed. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

sygvd (USM Version)

Computes all eigenvalues and, optionally, eigenvectors of a real generalized symmetric definite eigenproblem using a divide and conquer method. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes all the eigenvalues, and optionally, the eigenvectors of a real generalized symmetric-definite eigenproblem, of the form

$$A*x = \lambda*B*x, A*B*x = \lambda*x, \text{ or } B*A*x = \lambda*x.$$

Here *A* and *B* are assumed to be symmetric and *B* is also positive definite.

It uses a divide and conquer algorithm.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event sygvd(sycl::queue &queue,
        int64_t itype,
        mkl::job jobz,
```

```

mkl::uplo uplo,
int64_t n,
T *a,
int64_t lda,
T *b,
int64_t ldb,
T *w,
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
}

```

hegvd (USM version) supports the following precision and devices.

T	Devices Supported
float	CPU, GPU*
double	CPU, GPU^

*Interface support only; all computations are performed on the CPU.

^Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
itype	Must be 1 or 2 or 3. Specifies the problem type to be solved: if itype= 1, the problem type is $A*x = \lambda*B*x$; if itype= 2, the problem type is $A*B*x = \lambda*x$; if itype= 3, the problem type is $B*A*x = \lambda*x$.
jobz	Must be <code>job::novec</code> or <code>job::vec</code> . If <code>jobz = job::novec</code> , then only eigenvalues are computed. If <code>jobz = job::vec</code> , then eigenvalues and eigenvectors are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , <code>a</code> stores the upper triangular part of <code>A</code> . If <code>uplo = uplo::lower</code> , <code>a</code> stores the lower triangular part of <code>A</code> .
n	The order of the matrices <code>A</code> and <code>B</code> ($0 \leq n$).
a	Pointer to the array containing the upper or lower triangle of the symmetric matrix <code>A</code> , as specified by <code>uplo</code> . The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> ; at least $\max(1, n)$.
b	Pointer to the array containing the upper or lower triangle of the symmetric matrix <code>B</code> , as specified by <code>uplo</code> . The size of <code>b</code> must be at least <code>ldb*n</code> .
ldb	The leading dimension of <code>b</code> ; at least $\max(1, n)$.
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.

scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the sygvd_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
a	On exit, if <code>jobz = job::vec</code> , then if <code>info = 0</code> , <code>a</code> contains the matrix <code>Z</code> of eigenvectors. The eigenvectors are normalized as follows: if <code>itype = 1</code> or <code>2</code> , $Z^T B Z = I$; if <code>itype = 3</code> , $Z^T \text{inv}(B) Z = I$; If <code>jobz = job::novec</code> , then on exit the upper triangle (if <code>uplo = uplo::upper</code>) or the lower triangle (if <code>uplo = uplo::lower</code>) of <code>A</code> , including the diagonal, is destroyed.
b	On exit, if <code>info ≤ n</code> , the part of <code>b</code> containing the matrix is overwritten by the triangular factor <code>U</code> or <code>L</code> from the Cholesky factorization $B = U^T U$ or $B = L L^T$.
w	Pointer to the array of size at least <code>n</code> . If <code>info = 0</code> , contains the eigenvalues of the matrix <code>A</code> in ascending order. See also <code>info</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the <code>info</code> code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. For <code>info ≤ n</code>: If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge; <code>i</code> indicates the number of off-diagonal elements of an intermediate tridiagonal form which did not converge to zero. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue while working on the submatrix lying in rows and columns <code>info/(n+1)</code> through <code>mod(info, n+1)</code>. For <code>info > n</code>: If <code>info = n + i</code>, for $1 ≤ i ≤ n$, then the leading minor of order <code>i</code> of <code>B</code> is not positive-definite. The factorization of <code>B</code> could not be completed and no eigenvalues or eigenvectors were computed. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

sygvd_scratchpad_size

Computes size of scratchpad memory required for `sygvd` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `sygvd` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t sygvd_scratchpad_size(sycl::queue &queue,
        int64_t itype,
        mkl::job jobz,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda,
        int64_t ldb)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>sygvd</code> (buffer or USM version) function will be performed.
<code>itype</code>	Must be 1 or 2 or 3. Specifies the problem type to be solved: if <code>itype= 1</code> , the problem type is $A*x = \lambda*B*x$; if <code>itype= 2</code> , the problem type is $A*B*x = \lambda*x$; if <code>itype= 3</code> , the problem type is $B*A*x = \lambda*x$.
<code>jobz</code>	Must be <code>job::novec</code> or <code>job::vec</code> . If <code>jobz = job::novec</code> , then only eigenvalues are computed. If <code>jobz = job::vec</code> , then eigenvalues and eigenvectors are computed.
<code>uplo</code>	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , <code>a</code> and <code>b</code> store the upper triangular part of <code>A</code> and <code>B</code> . If <code>uplo = uplo::lower</code> , <code>a</code> and <code>b</code> stores the lower triangular part of <code>A</code> and <code>B</code> .
<code>n</code>	The order of the matrices <code>A</code> and <code>B</code> ($0 \leq n$).
<code>lda</code>	The leading dimension of <code>a</code> ; at least $\max(1, n)$.

ldb The leading dimension of **b**; at least `max(1, n)`.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `sygvx` (buffer or USM version) function must be able to hold.

sygvx

Computes selected eigenvalues and, optionally, eigenvectors of a real symmetric matrix **A**. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes selected eigenvalues, and optionally eigenvectors, of a real generalized symmetric-definite eigenproblem of the form

$$A*x = \lambda*B*x, A*B*x = \lambda*x, \text{ or } B*A*x = \lambda*x.$$

Here **A** and **B** are assumed to be symmetric and **B** is also positive definite. Eigenvalues and eigenvectors can be selected by specifying either a range of values or a range of indices for the desired eigenvalues.

The routine computes selected eigenvalues with the bisection algorithm.

If the eigenvectors are requested, then this routine uses a combination of modified twisted factorization algorithm based on Inderjit Dhillon and Beresford Parlett's work and the inverse iteration algorithm followed by Gram-Schmidt orthogonalization.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void sygvx(sycl::queue &queue,
               int64_t itype,
               mkl::job jobz,
               mkl::rangev range,
               mkl::uplo uplo,
               int64_t n,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<T> &b,
               int64_t ldb,
               T vl,
               T vu,
               int64_t il,
```

```

int64_t iu,
T abstol,
sycl::buffer<int64_t> &m,
sycl::buffer<T> &w,
sycl::buffer<T> &z,
int64_t ldz,
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}

```

sygvx supports the following precision and devices.

T	Devices Supported
float	CPU and GPU*
double	CPU and GPU^

*Interface support only; all computations are performed on the CPU.

^Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
itype	Must be 1 or 2 or 3. Specifies the problem type to be solved: if itype= 1, the problem type is $A*x = \lambda*B*x$; if itype= 2, the problem type is $A*B*x = \lambda*x$; if itype= 3, the problem type is $B*A*x = \lambda*x$.
jobz	Must be <code>jobz::novec</code> or <code>jobz::vec</code>. If <code>jobz = jobz::novec</code>, then only eigenvalues are computed. If <code>jobz = jobz::vec</code>, then eigenvalues and eigenvectors are computed.
range	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $(v_l, v_u]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::indices</code>, then the i_l-th through i_u-th eigenvalues and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, a and b store the upper triangular part of A and B. If <code>uplo = uplo::lower</code>, a and b stores the lower triangular part of A and B.
n	The order of the matrices A and B ($0 \leq n$).
a	Buffer holding the array containing the upper or lower triangle of the symmetric matrix A, as specified by <code>uplo</code> . The size of a must be at least <code>lda*n</code> .
lda	The leading dimension of a. Must be at least $\max(1, n)$.

<code>b</code>	Buffer holding the array containing the upper or lower triangle of the symmetric matrix <code>B</code> , as specified by <code>uplo</code> . The size of <code>b</code> must be at least <code>ldb*n</code> .
<code>ldb</code>	The leading dimension of <code>b</code> ; at least <code>max(1,n)</code> .
<code>vl</code>	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>vu</code>	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>il</code>	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if <code>n > 0</code> and <code>il = 1</code> and <code>iu = 0</code> if <code>n = 0</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>iu</code>	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if <code>n > 0</code> and <code>il = 1</code> and <code>iu = 0</code> if <code>n = 0</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>abstol</code>	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval <code>[a,b]</code> of width less than or equal to <code>abstol + eps * max(a , b)</code> , where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then <code>eps* T </code> will be used in its place, where <code> T </code> is the 1-norm of the tridiagonal matrix obtained by reducing <code>A</code> to tridiagonal form.
<code>ldz</code>	The leading dimension of <code>z</code> . Must be at least <code>max(1,n)</code> .
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the sygvx_scratchpad_size function.

Output Parameters

<code>a</code>	On exit the upper triangle (if <code>uplo = uplo::upper</code>) or the lower triangle (if <code>uplo = uplo::lower</code>) of <code>A</code> , including the diagonal, is destroyed.
<code>b</code>	On normal exit, the part of <code>b</code> containing the matrix is overwritten by the triangular factor <code>U</code> or <code>L</code> from the Cholesky factorization $B = U^T * U$ or $B = L * L^T$.
<code>m</code>	The total number of eigenvalues found, $0 \leq m \leq n$.
<code>w</code>	Buffer holding array of size at least <code>n</code> . On normal exit, the first <code>m</code> elements contain the selected eigenvalues in ascending order.
<code>z</code>	If <code>jobz = jobz::vec</code> , then on normal exit the first <code>m</code> columns of <code>Z</code> contain the orthonormal eigenvectors of the matrix <code>A</code> corresponding to the selected eigenvalues, with the <code>i</code> -th column of <code>Z</code> holding the eigenvector associated with <code>w(i)</code> . If <code>jobz = jobz::novec</code> , then <code>Z</code> is not referenced. Note: the user must

ensure that at least $\max(1, m)$ columns are supplied in the array Z ; if `range = rangev::values`, the exact value of m is not known in advance and an upper bound must be used.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

sygvx (USM Version)

Computes selected eigenvalues and, optionally, eigenvectors of a real symmetric matrix A . This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes selected eigenvalues, and optionally eigenvectors, of a real generalized symmetric-definite eigenproblem of the form

$$A^*x = \lambda B^*x, A^*B^*x = \lambda^*x, \text{ or } B^*A^*x = \lambda^*x.$$

Here A and B are assumed to be symmetric and B is also positive definite. Eigenvalues and eigenvectors can be selected by specifying either a range of values or a range of indices for the desired eigenvalues.

The routine computes selected eigenvalues with the bisection algorithm.

If the eigenvectors are requested, then this routine uses a combination of modified twisted factorization algorithm based on Inderjit Dhillon and Beresford Parlett's work and the inverse iteration algorithm followed by Gram-Schmidt orthogonalization.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event sygvx(sycl::queue &queue,
        int64_t itype,
```

```

mkl::job jobz,
mkl::rangev range,
mkl::uplo uplo,
int64_t n,
T *a,
int64_t lda,
T *b,
int64_t ldb,
T vl,
T vu,
int64_t il,
int64_t iu,
T abstol,
int64_t *m,
T *w,
T *z,
int64_t ldz,
T *scratchpad,
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
}

```

sygvx (USM version) supports the following precision and devices.

T	Devices Supported
float	CPU and GPU*
double	CPU and GPU^

*Interface support only; all computations are performed on the CPU.

^Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
itype	Must be 1 or 2 or 3. Specifies the problem type to be solved: if itype= 1, the problem type is $A*x = \text{lambda}*B*x$; if itype= 2, the problem type is $A*B*x = \text{lambda}*x$; if itype= 3, the problem type is $B*A*x = \text{lambda}*x$.
jobz	Must be <code>job::novec</code> or <code>job::vec</code>. If <code>jobz = job::novec</code>, then only eigenvalues are computed. If <code>jobz = job::vec</code>, then eigenvalues and eigenvectors are computed.
range	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $(vl, vu]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::indices</code>, then the <code>il</code>-th through <code>iu</code>-th eigenvalues and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.

uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> stores the upper triangular part of <code>A</code>. If <code>uplo = uplo::lower</code>, <code>a</code> stores the lower triangular part of <code>A</code>.
n	The order of the matrices <code>A</code> and <code>B</code> ($0 \leq n$).
a	Pointer to the array containing the upper or lower triangle of the symmetric matrix <code>A</code> , as specified by <code>uplo</code> . The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
b	Pointer to the array containing the upper or lower triangle of the symmetric matrix <code>B</code> , as specified by <code>uplo</code> . The size of <code>b</code> must be at least <code>ldb*n</code> .
ldb	The leading dimension of <code>b</code> ; at least <code>max(1, n)</code> .
vl	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
vu	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
il	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if <code>n</code> > 0 and <code>il</code> = 1 and <code>iu</code> = 0 if <code>n</code> = 0. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
iu	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if <code>n</code> > 0 and <code>il</code> = 1 and <code>iu</code> = 0 if <code>n</code> = 0. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
abstol	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval <code>[a,b]</code> of width less than or equal to <code>abstol + eps * max(a , b)</code> , where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then <code>eps* T </code> will be used in its place, where <code> T </code> is the 1-norm of the tridiagonal matrix obtained by reducing <code>A</code> to tridiagonal form.
ldz	The leading dimension of <code>z</code> . Must be at least <code>max(1, n)</code> .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the sygvx_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.
Output Parameters	
a	On exit the upper triangle (if <code>uplo = uplo::upper</code>) or the lower triangle (if <code>uplo = uplo::lower</code>) of <code>A</code> , including the diagonal, is destroyed.

b	On normal exit, the part of b containing the matrix is overwritten by the triangular factor U or L from the Cholesky factorization $B = U^T * U$ or $B = L * L^T$.
m	The total number of eigenvalues found, $0 \leq m \leq n$.
w	Pointer to array of size at least n . On normal exit, the first m elements contain the selected eigenvalues of the matrix A in ascending order.
z	If <code>jobz = job::vec</code> , then on normal exit the first m columns of Z contain the orthonormal eigenvectors of the matrix A corresponding to the selected eigenvalues, with the <i>i</i> -th column of Z holding the eigenvector associated with <code>w(i)</code> . If <code>jobz = job::novec</code> , then Z is not referenced. Note: the user must ensure that at least <code>max(1, m)</code> columns are supplied in the array Z ; if <code>range = rangev::values</code> , the exact value of m is not known in advance and an upper bound must be used.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info = i</code>, and <code>jobz = job::novec</code>, then the algorithm failed to converge. If <code>info = i</code>, and <code>jobz = job::vec</code>, then the algorithm failed to compute an eigenvalue. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

sygvx_scratchpad_size

Computes size of scratchpad memory required for `sygvx` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `sygvx` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t sygvx_scratchpad_size(sycl::queue &queue,
        int64_t itype,
        mkl::job jobz,
        mkl::rangev range,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda,
        int64_t ldb,
        T vl,
        T vu,
        int64_t il,
        int64_t iu,
        T abstol,
        int64_t ldz)
}
```

Input Parameters

queue	Device queue where calculations by the sygvx (buffer or USM version) function will be performed.
itype	Must be 1 or 2 or 3. Specifies the problem type to be solved: if itype= 1, the problem type is $A*x = \lambda*B*x$; if itype= 2, the problem type is $A*B*x = \lambda*x$; if itype= 3, the problem type is $B*A*x = \lambda*x$.
jobz	Must be <code>job::novec</code> or <code>job::vec</code>. If <code>jobz = job::novec</code>, then only eigenvalues are computed. If <code>jobz = job::vec</code>, then eigenvalues and eigenvectors are computed.
range	Must be <code>rangev::all</code>, <code>rangev::values</code> or <code>rangev::indices</code>. If <code>range = rangev::all</code>, then all the eigenvalues and eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::values</code>, then eigenvalues in the interval $(vl, vu]$ and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed. If <code>range = rangev::indices</code>, then the <code>il</code>-th through <code>iu</code>-th eigenvalues and corresponding eigenvectors (if requested by <code>jobz</code> parameters) are computed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> and <code>b</code> store the upper triangular part of <code>A</code> and <code>B</code>. If <code>uplo = uplo::lower</code>, <code>a</code> and <code>b</code> stores the lower triangular part of <code>A</code> and <code>B</code>.
n	The order of the matrices <code>A</code> and <code>B</code> ($0 \leq n$).
lda	The leading dimension of <code>a</code> . Must be at least <code>max(1, n)</code> .
ldb	The leading dimension of <code>b</code> ; at least <code>max(1, n)</code> .

<code>vl</code>	If <code>range = rangev::values</code> , the lower bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>vu</code>	If <code>range = rangev::values</code> , the upper bound of the interval to be searched for eigenvalues. <code>vl</code> must be less than <code>vu</code> . Not referenced if <code>range = rangev::all</code> or <code>range = rangev::indices</code> .
<code>il</code>	If <code>range = rangev::indices</code> , the one-based index of the smallest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>iu</code>	If <code>range = rangev::indices</code> , the one-based index of the largest eigenvalue to be returned. Must be $1 \leq il \leq iu \leq n$ if $n > 0$ and $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = rangev::all</code> or <code>range = rangev::values</code> .
<code>abstol</code>	The absolute error tolerance for the eigenvalues. An approximate eigenvalue is accepted as converged when it is determined to lie in an interval $[a,b]$ of width less than or equal to $abstol + eps * \max(a , b)$, where <code>eps</code> is the machine precision. If <code>abstol</code> is less than or equal to zero, then $eps * T $ will be used in its place, where $ T $ is the 1-norm of the tridiagonal matrix obtained by reducing <code>A</code> to tridiagonal form.
<code>ldz</code>	The leading dimension of <code>z</code> . Must be at least $\max(1, n)$.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `sygvx` (buffer or USM version) function must be able to hold.

sytrd

Reduces a real symmetric matrix to tridiagonal form. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine reduces a real symmetric matrix `A` to symmetric tridiagonal form `T` by an orthogonal similarity transformation: $A = Q * T * Q^T$. The orthogonal matrix `Q` is not formed explicitly but is represented as a product of $n-1$ elementary reflectors. Routines are provided for working with `Q` in this representation.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void sytrd(sycl::queue &queue,
              mkl::uplo uplo,
              int64_t n,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &d,
              sycl::buffer<T> &e,
              sycl::buffer<T> &tau,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

`sytrd` supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> stores the upper triangular part of <code>A</code>. If <code>uplo = uplo::lower</code>, <code>a</code> stores the lower triangular part of <code>A</code>.
n	The order of the matrices $A(0 \leq n)$.
a	Buffer holding matrix <code>A</code> , size $(lda, *)$. Contains the upper or lower triangle as specified by <code>uplo</code> .
lda	The leading dimension of <code>a</code> ; at least $\max(1, n)$.
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the sytrd_scratchpad_size function.

Output Parameters

a	On exit, if <code>uplo = uplo::upper</code>, the diagonal and first superdiagonal of <code>A</code> are overwritten by the corresponding elements of the tridiagonal matrix <code>T</code>, and the elements above the first superdiagonal, with the buffer <code>tau</code>, represent the orthogonal matrix <code>Q</code> as a product of elementary reflectors; if <code>uplo = uplo::lower</code>, the diagonal and first subdiagonal of <code>A</code> are overwritten by the corresponding elements of the tridiagonal matrix <code>T</code>, and the elements below the first subdiagonal, with the buffer <code>tau</code>, represent the orthogonal matrix <code>Q</code> as a product of elementary reflectors.
---	---

d	Buffer holding the diagonal elements of the matrix T . The dimension of d must be at least $\max(1, n)$.
e	Buffer holding the off diagonal elements of the matrix T . The dimension of e must be at least $\max(1, n-1)$.
tau	Buffer holding array of size at least $\max(1, n)$. Stores $(n-1)$ scalars that define elementary reflectors in decomposition of the unitary matrix Q in a product of $n-1$ elementary reflectors. tau (n) is used as workspace.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

sytrd (USM Version)

Reduces a real symmetric matrix to tridiagonal form. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine reduces a real symmetric matrix A to symmetric tridiagonal form T by an orthogonal similarity transformation: $A = Q^*TQ$. The orthogonal matrix Q is not formed explicitly but is represented as a product of $n-1$ elementary reflectors. Routines are provided for working with Q in this representation.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event sytrd(queue &queue,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        T *d,
        T *e,
        T *tau,
        T *scratchpad,
```



```
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {}))
}
```

`sytrd`(USM version) supports the following precisions and devices:

T	Devices supported
float	CPU and GPU*
double	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . If <code>uplo = uplo::upper</code> , <code>a</code> stores the upper triangular part of <code>A</code> . If <code>uplo = uplo::lower</code> , <code>a</code> stores the lower triangular part of <code>A</code> .
n	The order of the matrices A ($0 \leq n$).
a	The pointer to matrix <code>A</code> , size <code>(lda, *)</code> . Contains the upper or lower triangle as specified by <code>uplo</code> .
lda	The leading dimension of <code>a</code> ; at least <code>max(1, n)</code> .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the <code>sytrd_scratchpad_size</code> function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	On exit, if <code>uplo = uplo::upper</code> , the diagonal and first superdiagonal of <code>A</code> are overwritten by the corresponding elements of the tridiagonal matrix <code>T</code> , and the elements above the first superdiagonal, with the array <code>tau</code> , represent the orthogonal matrix <code>Q</code> as a product of elementary reflectors; if <code>uplo = uplo::lower</code> , the diagonal and first subdiagonal of <code>A</code> are overwritten by the corresponding elements of the tridiagonal matrix <code>T</code> , and the elements below the first subdiagonal, with the array <code>tau</code> , represent the orthogonal matrix <code>Q</code> as a product of elementary reflectors.
d	Pointer to the diagonal elements of the matrix <code>T</code> . The dimension of <code>d</code> must be at least <code>max(1, n)</code> .
e	Pointer to the off diagonal elements of the matrix <code>T</code> . The dimension of <code>e</code> must be at least <code>max(1, n-1)</code> .

tau Pointer to the memory array of size at least $\max(1, n)$. Stores $(n-1)$ scalars that define elementary reflectors in decomposition of the unitary matrix Q in a product of $n-1$ elementary reflectors. $\text{tau}(n)$ is used as workspace.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

sytrd_scratchpad_size

Computes size of scratchpad memory required for `sytrd` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `sytrd` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t sytrd_scratchpad_size(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda)
}
```

Input Parameters

queue Device queue where calculations by the `sytrd` (buffer or USM version) function will be performed.

<code>uplo</code>	Must be <code>uplo::upper</code> or <code>uplo::lower</code>. If <code>uplo = uplo::upper</code>, <code>a</code> stores the upper triangular part of <code>A</code>. If <code>uplo = uplo::lower</code>, <code>a</code> stores the lower triangular part of <code>A</code>.
<code>n</code>	The order of the matrices $A(0 \leq n)$.
<code>lda</code>	The leading dimension of <code>a</code> ; at least <code>max(1, n)</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `sytrd` (buffer or USM version) function must be able to hold.

sytrf

Computes the Bunch-Kaufman factorization of a symmetric matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the factorization of a real/complex symmetric matrix `A` using the Bunch-Kaufman diagonal pivoting method. The form of the factorization is:

- if `uplo=uplo::upper`, $A = U * D * U^T$
- if `uplo=uplo::lower`, $A = L * D * L^T$

where `A` is the input matrix, `U` and `L` are products of permutation and triangular matrices with unit diagonal (upper triangular for `U` and lower triangular for `L`), and `D` is a symmetric block-diagonal matrix with 1-by-1 and 2-by-2 diagonal blocks. `U` and `L` have 2-by-2 unit diagonal blocks corresponding to the 2-by-2 blocks of `D`.

API

Syntax

```
namespace oneapi::mkl::lapack {
    void sytrf(sycl::queue &queue,
              mkl::uplo uplo,
              int64_t n,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &ipiv,
```

```
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}
```

`sytrf` supports the following precisions and devices:

T	Devices Supported
float	CPU
double	CPU
std::complex<float>	CPU
std::complex<double>	CPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Indicates whether the upper or lower triangular part of A is stored and how A is factored: If <code>uplo=uplo::upper</code> , the buffer <code>a</code> stores the upper triangular part of the matrix A , and A is factored as $U \cdot D \cdot U^T$. If <code>uplo=uplo::lower</code> , the buffer <code>a</code> stores the lower triangular part of the matrix A , and A is factored as $L \cdot D \cdot L^T$.
n	The order of matrix A ($0 \leq n$).
a	Buffer holding the coefficients of matrix A containing either the upper or lower triangular part of the matrix A (see <code>uplo</code>). The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> .
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the sytrf_scratchpad_size function.

Output Parameters

a	The upper or lower triangular part of <code>a</code> is overwritten by details of the block-diagonal matrix D and the multipliers used to obtain the factor U (or L).
ipiv	Buffer holding array of size at least <code>max(1, n)</code> . Contains details of the interchanges and the block structure of D . If <code>ipiv(i)=k>0</code> , then d_{ii} is a 1-by-1 block, and the i -th row and column of A was

interchanged with the k -th row and column. If `uplo=mkl::uplo::upper` and `ipiv(i)=ipiv(i-1)=-m<0`, then D has a 2-by-2 block in rows/columns i and $i-1$, and $(i-1)$ -th row and column of A was interchanged with the m -th row and column. If `uplo=mkl::uplo::lower` and `ipiv(i)=ipiv(i+1)=-m<0`, then D has a 2-by-2 block in rows/columns i and $i+1$, and $(i+1)$ -th row and column of A was interchanged with the m -th row and column.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info = i</code>, d_{ii} is 0. The factorization has been completed, but D is exactly singular. Division by 0 will occur if you use D for solving a system of linear equations. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

sytrf (USM Version)

Computes the Bunch-Kaufman factorization of a symmetric matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine computes the factorization of a real/complex symmetric matrix A using the Bunch-Kaufman diagonal pivoting method. The form of the factorization is:

- if `uplo=uplo::upper`, $A = U^*D*U^T$
- if `uplo=uplo::lower`, $A = L^*D*L^T$

where A is the input matrix, U and L are products of permutation and triangular matrices with unit diagonal (upper triangular for U and lower triangular for L), and D is a symmetric block-diagonal matrix with 1-by-1 and 2-by-2 diagonal blocks. U and L have 2-by-2 unit diagonal blocks corresponding to the 2-by-2 blocks of D .

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event sytrf(queue &queue,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        int64_t *ipiv,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`sytrf` (USM version) supports the following precisions and devices:

T	Devices Supported
float	CPU
double	CPU
std::complex<float>	CPU
std::complex<double>	CPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Indicates whether the upper or lower triangular part of A is stored and how A is factored: If <code>uplo = uplo::upper</code> , the array <code>a</code> stores the upper triangular part of the matrix A , and A is factored as $U \cdot D \cdot U^T$. If <code>uplo = uplo::lower</code> , the array <code>a</code> stores the lower triangular part of the matrix A , and A is factored as $L \cdot D \cdot L^T$.
n	The order of the matrix A ($0 \leq n$).
a	Pointer to the coefficients of matrix A containing either the upper or lower triangular part of the matrix A (see <code>uplo</code>). The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the sytrf_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	The upper or lower triangular part of <code>a</code> is overwritten by details of the block-diagonal matrix D and the multipliers used to obtain the factor U (or L).
---	--

`ipiv`

Pointer to memory array of size at least $\max(1, n)$. Contains details of the interchanges and the block structure of D . If `ipiv(i) = k > 0`, then `dii` is a 1-by-1 block, and the i -th row and column of A was interchanged with the k -th row and column.

If `uplo = mkl::uplo::upper` and `ipiv(i) = ipiv(i-1) = -m < 0`, then D has a 2-by-2 block in rows/columns i and $i-1$, and $(i-1)$ -th row and column of A was interchanged with the m -th row and column.

If `uplo = mkl::uplo::lower` and `ipiv(i) = ipiv(i+1) = -m < 0`, then D has a 2-by-2 block in rows/columns i and $i+1$, and $(i+1)$ -th row and column of A was interchanged with the m -th row and column.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info = i</code>, <code>dii</code> is 0. The factorization has been completed, but D is exactly singular. Division by 0 will occur if you use D for solving a system of linear equations. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

sytrf_scratchpad_size

Computes size of scratchpad memory required for `sytrf` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `sytrf` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t sytrf_scratchpad_size(sycl::queue &queue,
                                mkl::uplo uplo,
                                int64_t n,
                                int64_t lda)
}
```

Input Parameters

queue	Device queue where calculations by the sytrf (buffer or USM version) function will be performed.
uplo	Indicates whether the upper or lower triangular part of A is stored and how A is factored: If <code>uplo = uplo::upper</code>, the array <code>a</code> stores the upper triangular part of the matrix A, and A is factored as U^*D*U^T. If <code>uplo = uplo::lower</code>, the array <code>a</code> stores the lower triangular part of the matrix A, and A is factored as L^*D*L^T.
n	The order of the matrix A ($0 \leq n$).
lda	The leading dimension of <code>a</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `sytrf` (buffer or USM version) function must be able to hold.

trtri (USM Version)

Computes the inverse of a triangular matrix. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

This routine computes the inverse $inv(A)$ of an upper or lower triangular matrix A .

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event trtri(sycl::queue &queue,
        mkl::uplo uplo,
        mkl::diag diag,
        int64_t n,
        T *a,
        int64_t lda,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

trtri (USM version) supports the following precisions and devices:

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Indicates whether A is upper or lower triangular: If uplo = uplo::upper, then the array a stores the upper triangular part of the matrix A, and the strictly lower triangular part of the matrix is not referenced. If uplo = uplo::lower, then the array a stores the lower triangular part of the matrix A, and the strictly upper triangular part of the matrix is not referenced.
diag	If diag = diag::nonunit, then the matrix A is not a unit triangular matrix. If diag = diag::unit, then the matrix A is unit triangular: the diagonal elements are assumed to be 1 and are not referenced.
n	The order of the matrix A ($n \geq 0$).
a	Pointer to the input array, which contains either the upper or lower triangular matrix, A (see uplo). The array must be of size at least $lda * \max(1, n)$.
lda	The leading dimension of a; $lda \geq \max(1, n)$.
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by the trtri_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

- a
- If `uplo = uplo::upper`, the upper triangular part of `A` is overwritten by its inverse.

If `uplo = uplo::lower`, the lower triangular part of `A` is overwritten by its inverse.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during the calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info = i</code>, and <code>detail()</code> returns 0, <code>A(i,i)</code> is exactly zero. The triangular matrix is singular and its inverse can not be computed. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non-zero, then the passed <code>scratchpad</code> has an insufficient size. The required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

trtri_scratchpad_size

Computes size of scratchpad memory required for `trtri` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

This routine computes the number of elements of type `T` the scratchpad memory to be passed to the `trtri` (USM version) function must be able to hold.

API

Syntax

```

namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t trtri_scratchpad_size(sycl::queue &queue,
        mkl::uplo uplo,
        mkl::diag diag,
        int64_t n,
        int64_t lda)
}

```

Input Parameters

queue	Device queue where calculations by the <code>trtri</code> (buffer or USM version) function will be performed.
uplo	Indicates whether A is upper or lower triangular: If <code>uplo = uplo::upper</code> , then A is upper triangular. If <code>uplo = uplo::lower</code> , then A is lower triangular.
diag	If <code>diag = diag::nonunit</code> , then A is not a unit triangular matrix. If <code>diag = diag::unit</code> , then A is unit triangular: diagonal elements of A are assumed to be 1 and not referenced in the array <code>a</code> .
n	The order of the matrix A ($n \geq 0$).
lda	The leading dimension of <code>a</code> ; <code>lda</code> $\geq \max(1, n)$.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `trtri` (buffer or USM version) function must be able to hold.

trtrs

Solves a system of linear equations with a triangular coefficient matrix, with multiple right-hand sides. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for X the following systems of linear equations with a triangular matrix A , with multiple right-hand sides stored in B :

$A * X = B$	if <code>transa</code> = <code>transpose::nontrans</code> ,
$A^T * X = B$	if <code>transa</code> = <code>transpose::trans</code> ,
$A^H * X = B$	if <code>transa</code> = <code>transpose::conjtrans</code> (for complex matrices only).

API

Syntax

```
namespace oneapi::mkl::lapack {
    void trtrs(sycl::queue &queue,
              mkl::uplo uplo,
              mkl::transpose trans,
              mkl::diag diag,
              int64_t n,
              int64_t nrhs,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &b,
              int64_t ldb,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

`trtrs` supports the following precisions and devices.

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>uplo</code>	Indicates whether <i>A</i> is upper or lower triangular: If <code>uplo = uplo::upper</code> , then <i>A</i> is upper triangular. If <code>uplo = uplo::lower</code> , then <i>A</i> is lower triangular.
<code>trans</code>	If <code>transa = transpose::nontrans</code> , then $A^T X = B$ is solved for <i>X</i> . If <code>transa = transpose::trans</code> , then $A^T X = B$ is solved for <i>X</i> . If <code>transa = transpose::conjtrans</code> , then $A^H X = B$ is solved for <i>X</i> .
<code>diag</code>	If <code>diag = diag::nonunit</code> , then <i>A</i> is not a unit triangular matrix. If <code>diag = diag::unit</code> , then <i>A</i> is unit triangular: diagonal elements of <i>A</i> are assumed to be 1 and not referenced in the array <i>a</i> .
<code>n</code>	The order of <i>A</i> ; the number of rows in <i>B</i> ; $n \geq 0$.
<code>nrhs</code>	The number of right-hand sides; $nrhs \geq 0$.
<code>a</code>	Array containing the matrix <i>A</i> . The size of <i>a</i> must be at least <code>lda*n</code> .
<code>lda</code>	The leading dimension of <i>a</i> ; $lda \geq \max(1, n)$.
<code>b</code>	Array containing the matrix <i>B</i> whose columns are the right-hand sides for the systems of equations. The size of <i>b</i> must be at least <code>ldb*nrhs</code> .
<code>ldb</code>	The leading dimension of <i>b</i> ; $ldb \geq \max(1, n)$.

scratchpad Buffer holding scratchpad memory to be used by the routine for storing intermediate results.

scratchpad_size Size of scratchpad memory as a number of floating point elements of type `T`. Size should not be less than the value returned by the [trtrs_scratchpad_size](#) function.

Output Parameters

b Overwritten by the solution matrix `X`.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

info Buffer containing error information.

If `info = 0`, the execution is successful.

If `info = -i`, the *i*-th parameter had an illegal value.

trtrs (USM Version)

Solves a system of linear equations with a triangular coefficient matrix, with multiple right-hand sides. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine solves for `X` the following systems of linear equations with a triangular matrix `A`, with multiple right-hand sides stored in `B`:

$A * X = B$	if <code>transa = transpose::nontrans</code> ,
$A^T * X = B$	if <code>transa = transpose::trans</code> ,
$A^H * X = B$	if <code>transa = transpose::conjtrans</code> (for complex matrices only).

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event ttrtrs(sycl::queue &queue,
        mkl::uplo uplo,
        mkl::transpose trans,
        mkl::diag diag,
        int64_t n,
        int64_t nrhs,
        const T *a,
        int64_t lda,
        T *b,
        int64_t ldb,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

ttrtrs (USM version) supports the following precisions and devices.

T	Devices supported
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Indicates whether A is upper or lower triangular: If uplo = uplo::upper, then A is upper triangular. If uplo = uplo::lower, then A is lower triangular.
trans	If transa = transpose::nontrans, then $A * X = B$ is solved for X. If transa = transpose::trans, then $A^T * X = B$ is solved for X. If transa = transpose::conjtrans, then $A^H * X = B$ is solved for X.
diag	If diag = diag::nonunit, then A is not a unit triangular matrix. If diag = diag::unit, then A is unit triangular: diagonal elements of A are assumed to be 1 and not referenced in the array a.
n	The order of A; the number of rows in B; $n \geq 0$.
nrhs	The number of right-hand sides; $nrhs \geq 0$.
a	Array containing the matrix A. The size of a must be at least $lda * n$.
lda	The leading dimension of a; $lda \geq \max(1, n)$.

<code>b</code>	Array containing the matrix <code>B</code> whose columns are the right-hand sides for the systems of equations. The size of <code>b</code> at least <code>ldb*nrhs</code> .
<code>ldb</code>	The leading dimension of <code>b</code> ; <code>ldb` ` ≥ max(1, n)` `</code> .
<code>scratchpad</code>	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the trtrs_scratchpad_size function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

<code>b</code>	Overwritten by the solution matrix <code>x</code> .
----------------	---

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

trtrs_scratchpad_size

Computes size of scratchpad memory required for `trtrs` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `trtrs` (buffer or USM version) function must be able to hold.

API

Syntax

```

namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t trtrs_scratchpad_size(sycl::queue &queue,
        mkl::uplo uplo,
        mkl::transpose trans,
        mkl::diag diag,
        int64_t n,
        int64_t nrhs,
        int64_t lda,
        int64_t ldb)
}
    
```

Input Parameters

queue	Device queue where calculations by the trtrs (buffer or USM version) function will be performed.
uplo	Indicates whether A is upper or lower triangular: If uplo = uplo::upper, then A is upper triangular. If uplo = uplo::lower, then A is lower triangular.
trans	If trans = transpose::nontrans, then $A * X = B$ is solved for X. If trans = transpose::trans, then $A^T * X = B$ is solved for X. If trans = transpose::conjtrans, then $A^H * X = B$ is solved for X.
diag	If diag = diag::nonunit, then A is not a unit triangular matrix. If diag = diag::unit, then A is unit triangular: diagonal elements of A are assumed to be 1 and not referenced in the array a.
n	The order of A; the number of rows in B; $n \geq 0$.
nrhs	The number of right-hand sides; $nrhs \geq 0$.
lda	The leading dimension of a; $lda \geq \max(1, n)$.
ldb	The leading dimension of b; $ldb \geq \max(1, n)$.

Exceptions

Exception	Description
mkl::lapack::exception	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the info() method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the trtrs (buffer or USM version) function must be able to hold.

ungbr

Generates the complex unitary matrix Q or P^T determined by [gebrd](#). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the whole or part of the unitary matrices Q and P^H formed by the routines [gebrd](#). Use this routine after a call to `cgebrd/zgebrd`. All valid combinations of arguments are described in *Input Parameters*; in most cases you need the following:

To compute the whole m -by- m matrix Q , use:

```
ungbr(queue, generate::q, m, m, n, a, ...)
```

(note that the buffer `a` must have at least m columns).

To form the n leading columns of Q if $m > n$, use:

```
ungbr(queue, generate::q, m, n, n, a, ...)
```

To compute the whole n -by- n matrix P^T , use:

```
ungbr(queue, generate::p, n, n, m, a, ...)
```

(note that the array `a` must have at least n rows).

To form the m leading rows of P^T if $m < n$, use:

```
ungbr(queue, generate::p, m, n, m, a, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    void ungbr(sycl::queue &queue,
              mkl::generate gen,
              int64_t m,
              int64_t n,
              int64_t k,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &tau,
              sycl::buffer<T> &scratchpad,
              int64_t scratchpad_size)
}
```

`ungbr` supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU
<code>std::complex<double></code>	CPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>gen</code>	Must be <code>generate::q</code> or <code>generate::p</code> .

	If <code>gen= generate::q</code>, the routine generates the matrix Q. If <code>gen= generate::p</code>, the routine generates the matrix P^T.
<code>m</code>	The number of rows in the matrix Q or P^T to be returned ($0 \leq m$). If <code>gen= generate::q</code>, $m \geq n \geq \min(m, k)$. If <code>gen= generate::p</code>, $n \geq m \geq \min(n, k)$.
<code>n</code>	The number of rows in the matrix Q or P^T to be returned ($0 \leq n$). See <code>m</code> for constraints.
<code>k</code>	If <code>gen= generate::q</code>, the number of columns in the original m-by-k matrix reduced by <code>gebrd</code>. If <code>gen= generate::p</code>, the number of rows in the original k-by-n matrix reduced by <code>gebrd</code>.
<code>a</code>	Buffer holding memory returned by <code>gebrd</code> .
<code>lda</code>	The leading dimension of <code>a</code> .
<code>tau</code>	For <code>gen= generate::q</code>, the array <code>tauq</code> is returned by the <code>gebrd</code> function. For <code>gen= generate::p</code>, the array <code>taup</code> is returned by the <code>gebrd</code> function. The dimension of <code>tau</code> must be at least $\min(m, k)$ for <code>gen = generate::q</code>, or $\min(n, k)$ for <code>gen = generate::p</code>.
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the ungbr_scratchpad_size function.
Output Parameters	
<code>a</code>	Overwritten by <code>n</code> leading columns of the m -by- m unitary matrix Q or P^T , (or the leading rows or columns thereof) as specified by <code>gen</code> , <code>m</code> , and <code>n</code> .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad_size</code>, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

ungbr (USM Version)

Generates the complex unitary matrix Q or P^T determined by the `gebrd (USM Version)` function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the whole or part of the unitary matrices Q and P^T formed by the [gebrd \(USM Version\)](#) function. All valid combinations of arguments are described in *Input Parameters*; in most cases you need the following:

To compute the whole m -by- m matrix Q , use:

```
ungbr(queue, generate::q, m, m, n, a, ...)
```

(note that the array `a` must have at least m columns).

To form the n leading columns of Q if $m > n$, use:

```
ungbr(queue, generate::q, m, n, n, a, ...)
```

To compute the whole n -by- n matrix P^T , use:

```
ungbr(queue, generate::p, n, n, m, a, ...)
```

(note that the array `a` must have at least n rows).

To form the m leading rows of P^T if $m < n$, use:

```
ungbr(queue, generate::p, m, n, m, a, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event ungbr(sycl::queue &queue,
        mkl::generate gen,
        int64_t m,
        int64_t n,
        int64_t k,
        T *a,
        int64_t lda,
        const T *tau,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`ungbr (USM version)` supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU
<code>std::complex<double></code>	CPU

Input Parameters

queue	Device queue where calculations will be performed.
gen	Must be <code>generate::q</code> or <code>generate::p</code>. If <code>gen= generate::q</code>, the routine generates the matrix Q. If <code>gen= generate::p</code>, the routine generates the matrix P^T.
m	The number of rows in the matrix Q or P^T to be returned ($0 \leq m$). If <code>gen= generate::q</code>, $m \geq n \geq \min(m, k)$. If <code>gen= generate::p</code>, $n \geq m \geq \min(n, k)$.
n	The number of rows in the matrix Q or P^T to be returned ($0 \leq n$). See <code>m</code> for constraints.
k	If <code>gen= generate::q</code>, the number of columns in the original m-by-k matrix reduced by gebrd (USM Version). If <code>gen= generate::p</code>, the number of rows in the original k-by-n matrix reduced by gebrd (USM Version).
a	Pointer to memory returned by gebrd (USM Version) .
lda	The leading dimension of <code>a</code> .
tau	For <code>gen= generate::q</code>, the array <code>tauq</code> is returned by the gebrd (USM Version) function. For <code>gen= generate::p</code>, the array <code>taup</code> is returned by the gebrd (USM Version) function. The dimension of <code>tau</code> must be at least $\min(m, k)$ for <code>gen = generate::q</code>, or $\min(n, k)$ for <code>gen = generate::p</code>.
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the ungbr_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	Overwritten by n leading columns of the m -by- m orthogonal matrix Q or P^T (or the leading rows or columns thereof) as specified by <code>gen</code> , <code>m</code> , and <code>n</code> .
---	---

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object:

Exception	Description
	If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

ungbr_scratchpad_size

Computes size of scratchpad memory required for `ungbr` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `ungbr` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t ungbr_scratchpad_size(sycl::queue &queue,
        mkl::generate gen,
        int64_t m,
        int64_t n,
        int64_t k,
        int64_t lda)
}
```

Input Parameters

<code>queue</code>	Device queue where calculations by the <code>ungbr</code> (buffer or USM version) function will be performed.
<code>gen</code>	Must be <code>generate::q</code> or <code>generate::p</code>. If <code>gen= generate::q</code>, the routine generates the matrix Q. If <code>gen= generate::p</code>, the routine generates the matrix P^T.
<code>m</code>	The number of rows in the matrix Q or P^T to be returned ($0 \leq m$). If <code>gen= generate::q</code>, $m \geq n \geq \min(m, k)$. If <code>gen= generate::p</code>, $n \geq m \geq \min(n, k)$.

n	The number of rows in the matrix Q or P^T to be returned ($0 \leq n$). See <code>m</code> for constraints.
k	If <code>gen= generate::q</code>, the number of columns in the original <code>m</code>-by-<code>k</code> matrix reduced by gebrd (USM Version). If <code>gen= generate::p</code>, the number of rows in the original <code>k</code>-by-<code>n</code> matrix reduced by gebrd (USM Version).
lda	The leading dimension of <code>a</code> .

Exceptions

Exception	Description
<code>mk1::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `ungbr` (buffer or USM version) function must be able to hold.

ungqr

Generates the complex unitary matrix Q of the QR factorization formed by `geqrf`. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the whole or part of `m`-by-`m` unitary matrix Q of the QR factorization formed by the routines [geqrf](#).

Usually Q is determined from the QR factorization of an `m` by `p` matrix `A` with $m \geq p$. To compute the whole matrix Q , use:

```
mk1::ungqr(queue, m, m, p, a, lda, tau, ...)
```

To compute the leading `p` columns of Q (which form an orthonormal basis in the space spanned by the columns of `A`):

```
mk1::ungqr(queue, m, p, p, a, lda, tau, ...)
```

To compute the matrix Q^k of the QR factorization of the leading `k` columns of the matrix `A`:

```
mk1::ungqr(queue, m, m, k, a, lda, tau, ...)
```

To compute the leading `k` columns of Q^k (which form an orthonormal basis in the space spanned by the leading `k` columns of the matrix `A`):

```
mk1::ungqr(queue, m, k, k, a, lda, tau, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    void ungqr(sycl::queue &queue,
               int64_t m,
               int64_t n,
               int64_t k,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<T> &tau,
               sycl::buffer<T> &scratchpad,
               int64_t scratchpad_size)
}
```

`ungqr` ``` supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU and GPU*
<code>std::complex<double></code>	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>m</code>	The number of rows in the matrix A ($0 \leq m$).
<code>n</code>	The number of columns in the matrix A ($0 \leq n$).
<code>k</code>	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
<code>a</code>	Buffer holding the result of geqrf .
<code>lda</code>	The leading dimension of A ($lda \geq m$).
<code>tau</code>	Buffer holding the result of geqrf .
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the ungqr_scratchpad_size function.

Output Parameters

<code>a</code>	Overwritten by n leading columns of the m -by- m unitary matrix Q .
----------------	---

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object:

Exception	Description
	If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

ungqr (USM Version)

Generates the complex unitary matrix Q of the QR factorization formed by the `geqrf` (USM Version) function. . This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the whole or part of m -by- m unitary matrix Q of the QR factorization formed by the [geqrf \(USM Version\)](#) function.

Usually Q is determined from the QR factorization of an m by p matrix A with $m \geq p$. To compute the whole matrix Q , use:

```
ungqr(queue, m, m, p, a, lda, tau, ...)
```

To compute the leading p columns of Q (which form an orthonormal basis in the space spanned by the columns of A):

```
ungqr(queue, m, p, p, a, lda, tau, ...)
```

To compute the matrix Q^k of the QR factorization of the leading k columns of the matrix A :

```
ungqr(queue, m, m, k, a, lda, tau, ...)
```

To compute the leading k columns of Q^k (which form an orthonormal basis in the space spanned by the leading k columns of the matrix A):

```
ungqr(queue, m, k, k, a, lda, tau, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event ungqr(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t k,
        T *a,
        int64_t lda,
        const T *tau,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```


`ungqr` (USM version) supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU and GPU*
<code>std::complex<double></code>	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>m</code>	The number of rows in the matrix A ($0 \leq m$).
<code>n</code>	The number of columns in the matrix A ($0 \leq n$).
<code>k</code>	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
<code>a</code>	Pointer to the result of geqrf (USM Version) .
<code>lda</code>	The leading dimension of <code>a</code> ($lda \geq m$).
<code>tau</code>	Pointer to the result of geqrf (USM Version) .
<code>scratchpad</code>	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the ungqr_scratchpad_size function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

<code>a</code>	Overwritten by <code>n</code> leading columns of the m -by- m unitary matrix Q .
----------------	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the i-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

ungqr_batch (Buffer Strided Version)

Generates the complex unitary matrices Q_i of the batch of QR factorizations formed by `geqrf_batch` function (Buffer stride version). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the wholes or parts of m -by- m unitary matrix Q_i of the QR factorization formed by the [geqrf_batch \(Buffer Strided Version\)](#) function.

Usually Q_i is determined from the QR factorization of an m by p matrix A_i with $m \geq p$. To compute the whole matrices Q_i , use:

```
ungqr_batch(queue, m, m, p, a, ...)
```

To compute the leading p columns of Q_i (which form an orthonormal basis in the space spanned by the columns of A_i):

```
ungqr_batch(queue, m, p, p, a, ...)
```

To compute the matrix Q_i^k of the QR factorization of the leading k columns of the matrices A_i :

```
ungqr_batch(queue, m, m, k, a, ...)
```

To compute the leading k columns of Q_i^k (which form an orthonormal basis in the space spanned by the leading k columns of the matrices A_i):

```
ungqr_batch(queue, m, k, k, a, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    void ungqr_batch(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t k,
        sycl::buffer<T> &a,
        int64_t lda,
        int64_t stride_a,
        sycl::buffer<T> &tau,
        int64_t stride_tau,
        int64_t batch_size,
        sycl::buffer<T> &scratchpad,
        int64_t scratchpad_size)
}
```

This function supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU and GPU*

T	Devices supported
<code>std::complex<double></code>	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>m</code>	The number of rows in the matrices A_i ($m \geq 0$).
<code>n</code>	The number of columns in the matrices A_i ($n \geq 0$).
<code>k</code>	the number of elementary reflectors whose product defines the matrices Q_i ($0 \leq k \leq n$).
<code>a</code>	Array resulting after a call to geqrf_batch (Buffer Strided Version) .
<code>lda</code>	The leading dimension of A_i ($lda \geq \max(1, m)$).
<code>stride_a</code>	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
<code>tau</code>	Array resulting after a call to geqrf_batch (Buffer Strided Version) .
<code>stride_tau</code>	The stride between the beginnings of arrays $\tau_{i,j}$ inside the array <code>tau</code> ($stride_tau \geq \max(1, \min(m, n))$).
<code>batch_size</code>	The number of problems in a batch ($batch_size \geq 0$).
<code>scratchpad</code>	Scratchpad memory to be used by routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by ungqr_batch_scratchpad_size (Strided Version) .

Output Parameters

<code>a</code>	Array data is overwritten by a batch of <code>n</code> leading columns of the <code>m</code> -by- <code>m</code> unitary matrices Q_i .
----------------	---

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

ungqr_batch (Group Version)

Generates the complex unitary matrices Q_i of the batch of QR factorizations formed by `geqrf_batch`. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the wholes or parts of m -by- m unitary matrix Q_i of the QR factorization formed by the [geqrf_batch \(Group Version\)](#) function.

Usually Q_i is determined from the QR factorization of an m by p matrix A_i with $m \geq p$. To compute the whole matrices Q_i , use:

```
ungqr_batch(queue, m, m, p, a, ...)
```

To compute the leading p columns of Q_i (which form an orthonormal basis in the space spanned by the columns of A_i):

```
ungqr_batch(queue, m, p, p, a, ...)
```

To compute the matrix Q_i^k of the QR factorization of the leading k columns of the matrices A_i :

```
ungqr_batch(queue, m, m, k, a, ...)
```

To compute the leading k columns of Q_i^k (which form an orthonormal basis in the space spanned by the leading k columns of the matrices A_i):

```
ungqr_batch(queue, m, k, k, a, ...)
```

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event ungqr_batch(sycl::queue &queue,
        int64_t *m,
        int64_t *n,
        int64_t *k,
        T **a,
        int64_t *lda,
        const T * const *tau,
        int64_t group_count,
        int64_t *group_sizes,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

This function supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU and GPU*
<code>std::complex<double></code>	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	Array of <code>group_count</code> parameters m_g as previously supplied to geqrf_batch (Group Version) ($m_g \geq 0$).
n	Array of <code>group_count</code> parameters n_g as previously supplied to geqrf_batch (Group Version) ($n_g \geq 0$).
k	Array of <code>group_count</code> parameters k_g . The number of elementary reflectors whose product defines the matrices Q_i ($0 \leq k_g \leq n_g$).
a	Array resulting after a call to geqrf_batch (Group Version) .
lda	Array of leading dimension of A_i as previously supplied to geqrf_batch (Group Version) ($lda_g \geq \max(1, m_g)$).
tau	Array resulting after a call to geqrf_batch (Group Version) .
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less then the value returned by ungqr_batch_scratchpad_size (Group Version) .
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	Matrices pointed to by array <code>a</code> are overwritten by n_g leading columns of the m_g -by- m_g unitary matrices Q_i , where g is an index of group of parameters corresponding to Q_i .
---	---

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the n-th parameter had an illegal value. If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

ungqr_batch (USM Strided Version)

Generates the complex unitary matrices Q_i of the batch of QR factorizations formed by `geqrf_batch` function (USM stride version). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine generates the wholes or parts of m -by- m unitary matrix Q_i of the QR factorization formed by the [geqrf_batch \(USM Strided Version\)](#) function.

Usually Q_i is determined from the QR factorization of an m by p matrix A_i with $m \geq p$. To compute the whole matrices Q_i , use:

```
ungqr_batch(queue, m, m, p, a, ...)
```

To compute the leading p columns of Q_i (which form an orthonormal basis in the space spanned by the columns of A_i):

```
ungqr_batch(queue, m, p, p, a, ...)
```

To compute the matrix Q_i^k of the QR factorization of the leading k columns of the matrices A_i :

```
ungqr_batch(queue, m, m, k, a, ...)
```

To compute the leading k columns of Q_i^k (which form an orthonormal basis in the space spanned by the leading k columns of the matrices A_i):

```
ungqr_batch(queue, m, k, k, a, ...)
```

API

Syntax

```

namespace oneapi::mkl::lapack {
    sycl::event ungqr_batch(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t k,
        T *a,
        int64_t lda,
        int64_t stride_a,
        const T *tau,
        int64_t stride_tau,
        int64_t batch_size,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}

```

This function supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU and GPU*
<code>std::complex<double></code>	CPU and GPU*

*Interface support only; all computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i ($m \geq 0$).
n	The number of columns in the matrices A_i ($n \geq 0$).
k	the number of elementary reflectors whose product defines the matrices Q_i ($0 \leq k \leq n$).
a	Array resulting after a call to geqrf_batch (USM Strided Version) .
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array <code>a</code> ($stride_a \geq \max(1, lda * n)$).
tau	Array resulting after a call to geqrf_batch (USM Strided Version) .
stride_tau	The stride between the beginnings of arrays τ_i inside the array <code>tau</code> ($stride_tau \geq \max(1, \min(m, n))$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).
scratchpad	Scratchpad memory to be used by routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by ungqr_batch_scratchpad_size (Strided Version) .
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

a	Array data is overwritten by a batch of <code>n</code> leading columns of the <code>m</code> -by- <code>m</code> unitary matrices Q_i .
---	---

Exceptions

Exception	Description
<code>mkl::lapack::batch_exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -n</code>, the <code>n</code>-th parameter had an illegal value.

Exception	Description
	If <code>info</code> equals the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad is of insufficient size, and the required size should be not less then value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

ungqr_batch_scratchpad_size (Group Version)

Computes size of scratchpad memory required for `ungqr_batch` (Group Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `ungqr_batch` (Group Version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t ungqr_batch_scratchpad_size(sycl::queue &queue,
        int64_t *m,
        int64_t *n,
        int64_t *k,
        int64_t *lda,
        int64_t group_count,
        int64_t *group_sizes)
}
```

This function supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU and GPU
<code>std::complex<double></code>	CPU and GPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>m</code>	Array of <code>group_count</code> parameters m_g as previously supplied to geqrf_batch (Group Version) ($m_g \geq 0$).
<code>n</code>	Array of <code>group_count</code> parameters n_g as previously supplied to geqrf_batch (Group Version) ($n_g \geq 0$).

k	Array of <code>group_count</code> parameters k_g . The number of elementary reflectors whose product defines the matrices Q_i ($0 \leq k_g \leq n_g$).
lda	The leading dimension of A_i as previously supplied to geqrf_batch (Group Version) ($lda_g \geq \max(1, m_g)$).
group_count	Specifies the number of groups of parameters. Must be at least 0.
group_sizes	Array of <code>group_count</code> integers. Array element with index g specifies the number of problems to solve for each of the groups of parameters g . So the total number of problems to solve, <code>batch_size</code> , is a sum of all parameter group sizes.

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the [ungqr_batch \(Group Version\)](#) function must be able to hold.

ungqr_batch_scratchpad_size (Strided Version)

Computes size of scratchpad memory required for `ungqr_batch (Strided Version)` function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `ungqr_batch (Strided Version)` function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    int64_t ungqr_batch_scratchpad_size(sycl::queue &queue,
        int64_t m,
        int64_t n,
        int64_t k,
        int64_t lda,
        int64_t stride_a,
        int64_t stride_tau,
        int64_t batch_size)
}
```

Input Parameters

queue	Device queue where calculations will be performed.
m	The number of rows in the matrices A_i ($m \geq 0$).
n	The number of columns in the matrices A_i ($n \geq 0$).
k	the number of elementary reflectors whose product defines the matrices Q_i ($0 \leq k \leq n$).
lda	The leading dimension of A_i ($lda \geq \max(1, m)$).
stride_a	The stride between the beginnings of matrices A_i inside the batch array a ($stride_a \geq \max(1, lda * n)$).
stride_tau	The stride between the beginnings of arrays τ_i inside the array tau ($stride_tau \geq \max(1, \min(m, n))$).
batch_size	The number of problems in a batch ($batch_size \geq 0$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type $\mathbb{T}$ the scratchpad memory to be passed to the `ungqr_batch` (Strided Version) function must be able to hold.

ungqr_scratchpad_size

Computes size of scratchpad memory required for `ungqr` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type $\mathbb{T}$ the scratchpad memory to be passed to the `ungqr` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t ungqr_scratchpad_size(sycl::queue &queue,
        int64_t m,
        int64_t n,
```

```
int64_t k,
int64_t lda)
}
```

Input Parameters

queue	Device queue where calculations by the ungqr (buffer or USM version) function will be performed.
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n$).
k	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
lda	The leading dimension of a ($lda \geq m$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `ungqr` (buffer or USM version) function must be able to hold.

ungtr

Generates the complex unitary matrix Q determined by [hetrd](#). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine explicitly generates the n -by- n unitary matrix Q formed by [hetrd](#) when reducing a complex Hermitian matrix A to tridiagonal form: $A = Q^*T^*QH$. Use this routine after a call to [hetrd](#).

API

Syntax

```
namespace oneapi::mkl::lapack {
    void ungtr(sycl::queue &queue,
              mkl::uplo uplo,
              int64_t n,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &tau,
```

```
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}
```

ungtr supports the following precision and devices.

T	Devices Supported
std::complex<float>	CPU
std::complex<double>	CPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Must be uplo::upper or uplo::lower. Uses the same uplo as supplied to the hetrd function.
n	The order of matrix Q ($0 \leq n$).
a	The buffer a returned by the hetrd function. The size of a must be at least $lda \cdot n$.
lda	The leading dimension of a ($lda \geq n$).
tau	The buffer tau returned by the hetrd function. The dimension of tau must be at least $\max(1, n-1)$.
scratchpad	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T. Size should not be less than the value returned by the ungtr_scratchpad_size function.

Output Parameters

a	Overwritten by the unitary matrix Q .
---	---

Exceptions

Exception	Description
mkl::lapack::exception	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the info() method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

ungtr (USM Version)

Generates the complex unitary matrix Q determined by the `hetrd` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine explicitly generates the n -by- n unitary matrix Q formed by the [hetrd \(USM Version\)](#) when reducing a complex Hermitian matrix A to tridiagonal form: $A = Q^*T*Q$. Use this routine after a call to [hetrd \(USM Version\)](#) function.

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event ungtr(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        T *a,
        int64_t lda,
        const T *tau,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`ungtr` (USM version) supports the following precision and devices.

T	Devices Supported
float	CPU
double	CPU

Input Parameters

queue	Device queue where calculations will be performed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . Uses the same <code>uplo</code> as supplied to the hetrd (USM Version) function.
n	The order of matrix Q ($0 \leq n$).
a	The pointer <code>a</code> returned by the hetrd (USM Version) function. The size of <code>a</code> must be at least <code>lda*n</code> .
lda	The leading dimension of <code>a</code> ($lda \geq n$).
tau	The pointer <code>tau</code> returned by the hetrd (USM Version) function. The dimension of <code>tau</code> must be at least <code>max(1, n-1)</code> .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.

`scratchpad_size` Size of scratchpad memory as a number of floating point elements of type `T`. Size should not be less than the value returned by the [ungtr_scratchpad_size](#) function.

`events` List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

`a` Overwritten by the unitary matrix Q .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non-zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

ungtr_scratchpad_size

Computes size of scratchpad memory required for `ungtr` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `ungtr` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t ungtr_scratchpad_size(sycl::queue &queue,
        mkl::uplo uplo,
        int64_t n,
        int64_t lda)
}
```

Input Parameters

queue	Device queue where calculations by the <code>ungtr</code> (buffer or USM version) function will be performed.
uplo	Must be <code>uplo::upper</code> or <code>uplo::lower</code> . Uses the same <code>uplo</code> as supplied to the hetrd (USM Version) function.
n	The order of matrix Q ($0 \leq n$).
lda	The leading dimension of a ($lda \geq n$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `ungtr` (buffer or USM version) function must be able to hold.

unmqr

Multiplies a complex matrix by the unitary matrix Q of the QR factorization formed by `unmqr`. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine multiplies a rectangular complex matrix C by Q or Q^H , where Q is the unitary matrix Q of the QR factorization formed by the routines [geqrf](#).

Depending on the parameters `left_right` and `trans`, the routine can form one of the matrix products $Q^H C$, $C^* Q$, or $C^* Q^H$ (overwriting the result on C).

API

Syntax

```
namespace oneapi::mkl::lapack {
    void unmqr(sycl::queue &queue,
               mkl::side side,
               mkl::transpose trans,
               int64_t m,
               int64_t n,
               int64_t k,
               sycl::buffer<T> &a,
               int64_t lda,
               sycl::buffer<T> &tau,
               sycl::buffer<T> &c,
               int64_t ldc,
```

```
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}
```

`unmqr` supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU and GPU*
<code>std::complex<double></code>	CPU and GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>side</code>	If <code>side = mkl::side::left</code> , Q or Q^T is applied to C from the left. If <code>side = mkl::side::right</code> , Q or Q^T is applied to C from the right.
<code>trans</code>	If <code>trans=mkl::transpose::trans</code> , the routine multiplies C by Q . If <code>trans=mkl::transpose::nontrans</code> , the routine multiplies C by Q^T .
<code>m</code>	The number of rows in the matrix A ($0 \leq m$).
<code>n</code>	The number of columns in the matrix A ($0 \leq n \leq m$).
<code>k</code>	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
<code>a</code>	Buffer holding the result of the geqrf function. The size of <code>a</code> must be at least <code>lda*k</code> .
<code>lda</code>	The leading dimension of <code>a</code> .
<code>tau</code>	Buffer holding <code>tau</code> returned by the geqrf function.
<code>c</code>	Buffer holding the matrix C . The size of <code>c</code> must be at least <code>ldc*n</code> .
<code>ldc</code>	The leading dimension of <code>c</code> .
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the unmqr_scratchpad_size function.

Output Parameters

<code>c</code>	Overwritten by the product $Q*C$, Q^T*C , $C*Q$, or $C*Q^T$ (as specified by <code>side</code> and <code>trans</code>).
----------------	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

unmqr (USM Version)

Multiplies a complex matrix by the unitary matrix Q of the QR factorization formed by the `geqrf` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine multiplies a complex matrix C by Q or Q^T , where Q is the orthogonal matrix Q of the QR factorization formed by the routine [geqrf \(USM Version\)](#).

Depending on the parameters `side` and `trans`, the routine can form one of the matrix products $Q^T C$, $Q^T C$, $C^* Q$, or $C^* Q^T$ (overwriting the result on C).

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event unmqr(sycl::queue &queue,
        mkl::side side,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        int64_t k,
        const T *a,
        int64_t lda,
        const T *tau,
        T *c,
        int64_t ldc,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`unmqr` (USM version) supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU, GPU*
<code>std::complex<double></code>	CPU, GPU*

*Hybrid support; some computations are performed on the CPU.

Input Parameters

queue	Device queue where calculations will be performed.
side	If <code>side = mkl::side::left</code> , Q or Q^T is applied to C from the left. If <code>side = mkl::side::right</code> , Q or Q^T is applied to C from the right.
trans	If <code>trans=mkl::transpose::trans</code> , the routine multiplies C by Q . If <code>trans=mkl::transpose::nontrans</code> , the routine multiplies C by Q^T .
m	The number of rows in the matrix A ($0 \leq m$).
n	The number of columns in the matrix A ($0 \leq n \leq m$).
k	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
a	The pointer to the result of the geqrf (USM Version) function. The size of <code>a</code> must be at least <code>lda*k</code> .
lda	The leading dimension of <code>a</code> .
tau	The pointer to <code>tau</code> returned by the geqrf (USM Version) function.
c	The pointer to the matrix C . The size of <code>c</code> must be at least <code>ldc*n</code> .
ldc	The leading dimension of <code>c</code> .
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the unmqr_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

c	Overwritten by the product $Q*C$, Q^T*C , $C*Q$, or $C*Q^T$ (as specified by <code>side</code> and <code>trans</code>).
---	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object:

Exception	Description
	If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

unmqr_scratchpad_size

Computes size of scratchpad memory required for `unmqr` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `unmqr` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t unmqr_scratchpad_size(sycl::queue &queue,
        mkl::side side,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        int64_t k,
        int64_t lda,
        int64_t ldc)
}
```

Input Parameters

queue	Device queue where calculations by the <code>unmqr</code> (buffer or USM version) function will be performed.
side	If <code>side = mkl::side::left</code> , Q or Q^T is applied to C from the left. If <code>side = mkl::side::right</code> , Q or Q^T is applied to C from the right.
trans	If <code>trans=mkl::transpose::trans</code> , the routine multiplies C by Q . If <code>trans=mkl::transpose::nontrans</code> , the routine multiplies C by Q^T .
m	The number of rows in the matrix A ($0 \leq m$).

n	The number of columns in the matrix A ($0 \leq n \leq m$).
k	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
lda	The leading dimension of a .
ldc	The leading dimension of c .

Exceptions

Exception	Description
<code>mk1::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type T the scratchpad memory to be passed to the `unmrq` (buffer or USM version) function must be able to hold.

unmrq

Multiplies a complex m -by- n matrix C by Q or Q^H , where Q is the complex unitary matrix defined as a product of k elementary reflectors $H(i)$ of order n : $Q = H(1)H^* H(2)H^* \dots H(k)H^*$ has returned by the RQ factorization routine `gerqf`. This routine belongs to the `oneapi::mk1::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine multiplies a complex m -by- n matrix C by Q or Q^H , where Q is the complex unitary matrix defined as a product of k elementary reflectors $H(i)$ of order n : $Q = H(1)H^* H(2)H^* \dots H(k)H^*$ has returned by the RQ factorization routine `gerqf`.

Depending on the parameters `side` and `trans`, the routine can form one of the matrix products Q^*C , $Q^H C$, C^*Q , or $C^H Q$ (overwriting the result over C).

API

Syntax

```
namespace oneapi::mk1::lapack {
    void unmrq(sycl::queue &queue,
              mk1::side side,
              mk1::transpose trans,
              int64_t m,
              int64_t n,
              int64_t k,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &tau,
              sycl::buffer<T> &c,
              int64_t ldc,
```

```
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}
```

`unmrq` supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU
<code>std::complex<double></code>	CPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>side</code>	If <code>side = mkl::side::left</code> , Q or Q^T is applied to C from the left. If <code>side = mkl::side::right</code> , Q or Q^T is applied to C from the right.
<code>trans</code>	If <code>trans=mkl::transpose::trans</code> , the routine multiplies C by Q . If <code>trans=mkl::transpose::nontrans</code> , the routine multiplies C by Q^T .
<code>m</code>	The number of rows in the matrix A ($0 \leq m$).
<code>n</code>	The number of columns in the matrix A ($0 \leq n \leq m$).
<code>k</code>	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
<code>a</code>	Buffer holding the result of the gerqf function. The size of <code>a</code> must be at least <code>lda*k</code> .
<code>lda</code>	The leading dimension of <code>a</code> .
<code>tau</code>	Buffer holding <code>tau</code> returned by the <code>:ref: `gerqf`</code> function.
<code>c</code>	Buffer holding the matrix C . The size of <code>c</code> must be at least <code>ldc*n</code> .
<code>ldc</code>	The leading dimension of <code>c</code> .
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the unmrq_scratchpad_size function.

Output Parameters

<code>c</code>	Overwritten by the product $Q*C$, Q^T*C , $C*Q$, or $C*Q^T$ (as specified by <code>side</code> and <code>trans</code>).
----------------	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

unmrq (USM Version)

Multiplies a complex matrix by the unitary matrix Q of the RQ factorization formed by the `gerqf` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine multiplies a complex m -by- n matrix C by Q or Q^T , where Q is the orthogonal matrix Q of the RQ factorization formed by the routine [gerqf \(USM Version\)](#).

Depending on the parameters `side` and `trans`, the routine can form one of the matrix products $Q^T C$, $Q^T C$, $C^* Q$, or $C^* Q^T$ (overwriting the result over C).

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event unmrq(sycl::queue &queue,
        mkl::side side,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        int64_t k,
        const T *a,
        int64_t lda,
        const T *tau,
        T *c,
        int64_t ldc,
        T *scratchpad,
        int64_t scratchpad_size,
        const std::vector<sycl::event> &events = {})
}
```

`unmrq` (USM version) supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU
<code>std::complex<double></code>	CPU

Input Parameters

<code>queue</code>	Device queue where calculations will be performed.
<code>side</code>	If <code>side = mkl::side::left</code> , Q or Q^T is applied to C from the left. If <code>side = mkl::side::right</code> , Q or Q^T is applied to C from the right.
<code>trans</code>	If <code>trans=mkl::transpose::trans</code> , the routine multiplies C by Q . If <code>trans=mkl::transpose::nontrans</code> , the routine multiplies C by Q^T .
<code>m</code>	The number of rows in the matrix A ($0 \leq m$).
<code>n</code>	The number of columns in the matrix A ($0 \leq n \leq m$).
<code>k</code>	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
<code>a</code>	The pointer to the result of the gerqf (USM Version) function. The size of <code>a</code> must be at least <code>lda*k</code> .
<code>lda</code>	The leading dimension of <code>a</code> .
<code>tau</code>	The pointer to <code>tau</code> returned by the gerqf (USM Version) function.
<code>c</code>	The pointer to the matrix C . The size of <code>c</code> must be at least <code>ldc*n</code> .
<code>ldc</code>	The leading dimension of <code>c</code> .
<code>scratchpad</code>	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type <code>T</code> . Size should not be less than the value returned by the unmrq_scratchpad_size function.
<code>events</code>	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

<code>c</code>	Overwritten by the product $Q*C$, Q^T*C , $C*Q$, or $C*Q^T$ (as specified by <code>side</code> and <code>trans</code>).
----------------	--

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object:

Exception	Description
	If <code>info = -i</code>, the <code>i</code>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as scratchpad size, and <code>detail()</code> returns non zero, then the passed scratchpad has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

unmrq_scratchpad_size

Computes size of scratchpad memory required for `unmrq` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `unmrq` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t unmrq_scratchpad_size(sycl::queue &queue,
        mkl::side side,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        int64_t kint64_t lda,
        int64_t ldc)
}
```

Input Parameters

queue	Device queue where calculations by the <code>unmrq</code> (buffer or USM version) function will be performed.
side	If <code>side = mkl::side::left</code> , Q or Q^T is applied to C from the left. If <code>side = mkl::side::right</code> , Q or Q^T is applied to C from the right.
trans	If <code>trans=mkl::transpose::trans</code> , the routine multiplies C by Q . If <code>trans=mkl::transpose::nontrans</code> , the routine multiplies C by Q^T .
m	The number of rows in the matrix A ($0 \leq m$).

<code>n</code>	The number of columns in the matrix A ($0 \leq n \leq m$).
<code>k</code>	The number of elementary reflectors whose product defines the matrix Q ($0 \leq k \leq n$).
<code>lda</code>	The leading dimension of a .
<code>ldc</code>	The leading dimension of c .

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `unmtrq` (buffer or USM version) function must be able to hold.

unmtr

multiplies a complex matrix by the complex unitary matrix Q determined by [hetrd](#). This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine multiplies a complex matrix C by Q or Q^H , where Q is the unitary matrix Q formed by [hetrd](#) when reducing a complex Hermitian matrix A to tridiagonal form: $A = Q^H T Q$. Use this routine after a call to [hetrd](#).

Depending on the parameters `left_right` and `trans`, the routine can form one of the matrix products $Q^H C$, $Q^H C$, $C^H Q$, or $C^H Q^H$ (overwriting the result on C).

API

Syntax

```
namespace oneapi::mkl::lapack {
    void unmtr(sycl::queue &queue,
              mkl::side side,
              mkl::uplo uplo,
              mkl::transpose trans,
              int64_t m,
              int64_t n,
              sycl::buffer<T> &a,
              int64_t lda,
              sycl::buffer<T> &tau,
              sycl::buffer<T> &c,
              int64_t ldc,
```

```
sycl::buffer<T> &scratchpad,
int64_t scratchpad_size)
}
```

`unmtr` supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU
<code>std::complex<double></code>	CPU

Input Parameters

In the descriptions below, r denotes the order of Q :

$r = m$	if <code>side = side::left</code>
$r = n$	if <code>side = side::right</code>

<code>queue</code>	Device queue where calculations will be performed.
<code>side</code>	Must be either <code>side::left</code> or <code>side::right</code> . If <code>side = side::left</code> , Q or Q^T is applied to C from the left. If <code>side = side::right</code> , Q or Q^T is applied to C from the right.
<code>uplo</code>	Must be either <code>uplo::upper</code> or <code>uplo::lower</code> . Uses the same <code>uplo</code> as supplied to hetrd (USM Version) .
<code>trans</code>	Must be either <code>transpose::nontrans</code> or <code>transpose::trans</code> . If <code>trans = transpose::nontrans</code> , the routine multiplies C by Q . If <code>trans = transpose::trans</code> , the routine multiplies C by Q^T .
<code>m</code>	The number of rows in the matrix C ($m \geq 0$).
<code>n</code>	The number of columns in the matrix C ($n \geq 0$).
<code>a</code>	Buffer holding array returned by hetrd (USM Version) .
<code>lda</code>	The leading dimension of a ($lda \geq \max(1, r)$).
<code>tau</code>	Buffer holding τ returned by hetrd (USM Version) . The dimension of τ must be at least $\max(1, r-1)$.
<code>c</code>	Buffer holding the matrix C . The size of c must be at least $ldc*n$.
<code>ldc</code>	The leading dimension of c ($ldc \geq \max(1, m)$).
<code>scratchpad</code>	Buffer holding scratchpad memory to be used by the routine for storing intermediate results.
<code>scratchpad_size</code>	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by the unmtr_scratchpad_size function.

Output Parameters

`c` Overwritten by the product Q^*C , $Q^T C$, C^*Q , or C^*Q^T (as specified by `side` and `trans`).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non-zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

unmtr (USM Version)

Multiplies a complex matrix by the complex unitary matrix Q determined by the `hetrd` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

The routine multiplies a complex matrix C by Q or Q^H , where Q is the unitary matrix Q formed by [hetrd \(USM Version\)](#) when reducing a complex Hermitian matrix A to tridiagonal form: $A = Q^T T Q$. Use this routine after a call to [hetrd \(USM Version\)](#).

Depending on the parameters `side` and `trans`, the routine can form one of the matrix products Q^*C , $Q^T C$, C^*Q , or C^*Q^T (overwriting the result on C).

API

Syntax

```
namespace oneapi::mkl::lapack {
    sycl::event unmtr(sycl::queue &queue,
        mkl::side side,
        mkl::uplo uplo,
        mkl::transpose trans,
        int64_t m,
        int64_t n,
        const T *a,
        int64_t lda,
        const T *tau,
        T *c,
        int64_t ldc,
        T *scratchpad,
```

```
int64_t scratchpad_size,
const std::vector<sycl::event> &events = {})
}
```

unmtr (USM version) supports the following precisions and devices:

T	Devices supported
std::complex<float>	CPU
std::complex<double>	CPU

Input Parameters

In the descriptions below, r denotes the order of Q :

$r = m$	if <code>side = side::left</code>
$r = n$	if <code>side = side::right</code>

queue	Device queue where calculations will be performed.
side	Must be either <code>side::left</code> or <code>side::right</code> . If <code>side = side::left</code> , Q or Q^T is applied to C from the left. If <code>side = side::right</code> , Q or Q^T is applied to C from the right.
uplo	Must be either <code>uplo::upper</code> or <code>uplo::lower</code> . Uses the same <code>uplo</code> as supplied to hetrd (USM Version) .
trans	Must be either <code>transpose::nontrans</code> or <code>transpose::trans</code> . If <code>trans = transpose::nontrans</code> , the routine multiplies C by Q . If <code>trans = transpose::trans</code> , the routine multiplies C by Q^T .
m	The number of rows in the matrix C ($m \geq 0$).
n	The number of columns in the matrix C ($n \geq 0$).
a	The pointer to memory returned by hetrd (USM Version) .
lda	The leading dimension of a ($lda \geq \max(1, r)$).
tau	The pointer to tau returned by hetrd (USM Version) . The dimension of tau must be at least $\max(1, r-1)$.
c	The pointer to memory containing the matrix C . The size of c must be at least $ldc*n$.
ldc	The leading dimension of c ($ldc \geq \max(1, m)$).
scratchpad	Pointer to scratchpad memory to be used by the routine for storing intermediate results.
scratchpad_size	Size of scratchpad memory as a number of floating point elements of type T . Size should not be less than the value returned by the unmtr_scratchpad_size function.
events	List of events to wait for before starting computation. Defaults to empty list.

Output Parameters

`c` Overwritten by the product Q^*C , $Q^T * C$, C^*Q , or C^*Q^T (as specified by `side` and `trans`).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when problems occur during calculations. You can obtain the info code of the problem using the <code>info()</code> method of the exception object: If <code>info = -i</code>, the <i>i</i>-th parameter had an illegal value. If <code>info</code> is equal to the value passed as <code>scratchpad</code> size, and <code>detail()</code> returns non-zero, then the passed <code>scratchpad</code> has an insufficient size, and the required size should not be less than the value returned by the <code>detail()</code> method of the exception object.

Return Values

Output event to wait on to ensure computation is complete.

unmtr_scratchpad_size

Computes size of scratchpad memory required for `unmtr` (USM Version) function. This routine belongs to the `oneapi::mkl::lapack` namespace.

- [Description](#)
- [API](#)

Description

Computes the number of elements of type `T` the scratchpad memory to be passed to the `unmtr` (buffer or USM version) function must be able to hold.

API

Syntax

```
namespace oneapi::mkl::lapack {
    template<typename T>
    int64_t unmtr_scratchpad_size(sycl::queue &queue,
        mkl::side side,
        mkl::uplo uplo,
        mkl::transpose trans,
        int64_t n,
        int64_t lda,
        int64_t ldc)
}
```

Input Parameters

In the descriptions below, r denotes the order of Q :

<code>r = m</code>	<code>if side = side::left</code>
<code>r = n</code>	<code>if side = side::right</code>

queue Device queue where calculations by the `unmtr` (buffer or USM version) function will be performed.

side Must be either `side::left` or `side::right`.
 If `side = side::left`, Q or Q^T is applied to C from the left.
 If `side = side::right`, Q or Q^T is applied to C from the right.

uplo Must be either `uplo::upper` or `uplo::lower`. Uses the same `uplo` as supplied to [hetrd \(USM Version\)](#).

trans Must be either `transpose::nontrans` or `transpose::trans`.
 If `trans = transpose::nontrans`, the routine multiplies C by Q .
 If `trans = transpose::trans`, the routine multiplies C by Q^T .

m The number of rows in the matrix C ($m \geq 0$).

n The number of columns in the matrix C ($n \geq 0$).

lda The leading dimension of a ($lda \geq \max(1, r)$).

ldc The leading dimension of c ($ldc \geq \max(1, m)$).

Exceptions

Exception	Description
<code>mkl::lapack::exception</code>	This exception is thrown when an incorrect argument value is supplied. You can determine the position of the incorrect argument by the <code>info()</code> method of the exception object.

Return Values

The number of elements of type `T` the scratchpad memory to be passed to the `unmtr` (buffer or USM version) function must be able to hold.

Vector Mathematical Functions

Intel® oneAPI Math Kernel Library (oneMKL) Vector Mathematics functions (VM) compute a mathematical function of each of the vector elements. VM includes a set of highly optimized functions (arithmetic, power, trigonometric, exponential, hyperbolic, special, and rounding) that operate on vectors of real and complex numbers.

Application programs that improve performance with VM include nonlinear programming software, computation of integrals, financial calculations, computer graphics, and many others.

VM functions fall into the following groups according to the operations they perform:

- [VM Mathematical Functions](#) compute values of mathematical functions, such as sine, cosine, exponential, or logarithm, on vectors stored contiguously in memory.
- [VM Service Functions](#) set/get the accuracy modes and the error codes, and create error handlers for mathematical functions.

The VM mathematical functions take an input vector as an argument, compute values of the respective function element-wise, and return the results in an output vector. All the VM mathematical functions can perform in-place operations, where the input and output arrays are at the same memory locations. For VM mathematical functions with positive increment indexing, in-place operations are supported only when the input and output increments have the same value.

The oneMKL interfaces are given in:

```
oneapi/mkl/vm.hpp.
```

Examples that demonstrate how to use the VM functions are located in:

```
${MKL}/share/doc/mkl/examples/sycl/vml/source
```

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

Special Value Notations

This topic defines notations of special values for complex functions. The definitions are provided in text, tables, or formulas.

- $z, z1, z2$, etc. denote complex numbers.
- $i, i2=-1$ is the imaginary unit.
- $x, X, x1, x2$, etc. denote real imaginary parts.
- $y, Y, y1, y2$, etc. denote imaginary parts.
- X and Y represent any finite positive IEEE-754 floating point values, if not stated otherwise.
- Quiet NaN and signaling NaN are denoted with QNAN and SNAN, respectively.
- The IEEE-754 positive infinities or floating-point numbers are denoted with a + sign before X, Y , etc.
- The IEEE-754 negative infinities or floating-point numbers are denoted with a – sign before X, Y , etc.

$\text{CONJ}(z)$ and $\text{CIS}(z)$ are defined as follows:

- $\text{CONJ}(x+i \cdot y)=x-i \cdot y$
- $\text{CIS}(y)=\cos (y)+i \cdot \sin (y)$.

The special value tables show the result of the function for the z argument at the intersection of the $\text{RE}(z)$ column and the $i \cdot \text{IM}(z)$ row. If the function raises an exception on the argument z , the lower part of this cell shows the raised exception and the VM Error Status. An empty cell indicates that this argument is normal and the result is defined mathematically.

VM Mathematical Functions

This section describes VM functions that compute values of mathematical functions on real and complex vector arguments.

Each function is introduced by its short name, a brief description of its purpose, and the calling sequence for each type of data, as well as a description of the input/output arguments.

The input range of parameters is equal to the mathematical range of the input data type, unless the function description specifies input threshold values, which mark off the precision overflow, as follows:

- `FLT16_MAX` denotes the maximum number representable in half precision real data type
- `FLT_MAX` denotes the maximum number representable in single precision real data type
- `DBL_MAX` denotes the maximum number representable in double precision real data type

The following table lists available mathematical functions and associated data types.

Function	Data Types	Description
Arithmetic Functions		
add	h, s, d, c, z	Adds vector elements
sub	h, s, d, c, z	Subtracts vector elements
sqr	h, s, d	Squares vector elements
mul	h, s, d, c, z	Multiplies vector elements
mulbyconj	c, z	Multiplies elements of one vector by conjugated elements of the second vector
conj	c, z	Conjugates vector elements
abs	h, s, d, c, z	Computes the absolute value of vector elements
arg	c, z	Computes the argument of vector elements
linearfrac	h, s, d	Performs linear fraction transformation of vectors
fmod	h, s, d	Performs element by element computation of the modulus function of vector a with respect to vector b
remainder	h, s, d	Performs element by element computation of the remainder function on the elements of vector a and the corresponding elements of vector b
Power and Root Functions		
inv	h, s, d	Inverts vector elements
div	h, s, d, c, z	Divides elements of one vector by elements of the second vector
sqrt	h, s, d, c, z	Computes the square root of vector elements
invsqrt	h, s, d	Computes the inverse square root of vector elements
cbrt	h, s, d	Computes the cube root of vector elements
invcbtr	h, s, d	Computes the inverse cube root of vector elements
pow2o3	h, s, d	Computes the cube root of the square of each vector element
pow3o2	h, s, d	Computes the square root of the cube of each vector element

Function	Data Types	Description
pow	h, s, d, c, z	Raises each vector element to the specified power
powx	h, s, d, c, z	Raises each vector element to the constant power
powr	h, s, d	Computes a to the power b for elements of two vectors, where the elements of vector argument a are all non-negative
hypot	h, s, d	Computes the square root of sum of squares
Exponential and Logarithmic Functions		
exp	h, s, d, c, z	Computes the base e exponential of vector elements
exp2	h, s, d	Computes the base 2 exponential of vector elements
exp10	h, s, d	Computes the base 10 exponential of vector elements
expm1	h, s, d	Computes the base e exponential of vector elements decreased by 1
ln	h, s, d, c, z	Computes the natural logarithm of vector elements
log2	h, s, d	Computes the base 2 logarithm of vector elements
log10	h, s, d, c, z	Computes the base 10 logarithm of vector elements
log1p	h, s, d	Computes the natural logarithm of vector elements that are increased by 1
logb	h, s, d	Computes the exponents of the elements of input vector a
Trigonometric Functions		
cos	h, s, d, c, z	Computes the cosine of vector elements
sin	h, s, d, c, z	Computes the sine of vector elements
sincos	h, s, d	Computes the sine and cosine of vector elements
cis	c, z	Computes the complex exponent of vector elements (cosine and sine combined to complex value)

Function	Data Types	Description
tan	h, s, d, c, z	Computes the tangent of vector elements
acos	h, s, d, c, z	Computes the inverse cosine of vector elements
asin	h, s, d, c, z	Computes the inverse sine of vector elements
atan	h, s, d, c, z	Computes the inverse tangent of vector elements
atan2	h, s, d	Computes the four-quadrant inverse tangent of ratios of the elements of two vectors
cospi	h, s, d	Computes the cosine of vector elements multiplied by π
sinpi	h, s, d	Computes the sine of vector elements multiplied by π
tanpi	h, s, d	Computes the tangent of vector elements multiplied by π
acospi	h, s, d	Computes the inverse cosine of vector elements divided by π
asinpi	h, s, d	Computes the inverse sine of vector elements divided by π
atanpi	h, s, d	Computes the inverse tangent of vector elements divided by π
atan2pi	h, s, d	Computes the four-quadrant inverse tangent of the ratios of the corresponding elementss of two vectors divided by π
cosd	h, s, d	Computes the cosine of vector elements multiplied by $\pi/180$
sind	h, s, d	Computes the sine of vector elements multiplied by $\pi/180$
tand	h, s, d	Computes the tangent of vector elements multiplied by $\pi/180$
Hyperbolic Functions		
cosh	h, s, d, c, z	Computes the hyperbolic cosine of vector elements
sinh	h, s, d, c, z	Computes the hyperbolic sine of vector elements
tanh	h, s, d, c, z	Computes the hyperbolic tangent of vector elements
acosh	h, s, d, c, z	Computes the inverse hyperbolic cosine of vector elements

Function	Data Types	Description
asinh	h, s, d, c, z	Computes the inverse hyperbolic sine of vector elements
atanh	h, s, d, c, z	Computes the inverse hyperbolic tangent of vector elements.
Special Functions		
erf	h, s, d	Computes the error function value of vector elements
erfc	h, s, d	Computes the complementary error function value of vector elements
erfcx	h, s, d	Computes the scaled complementary error function value of vector elements
cdfnorm	h, s, d	Computes the cumulative normal distribution function value of vector elements
erfinv	h, s, d	Computes the inverse error function value of vector elements
erfcinv	h, s, d	Computes the inverse complementary error function value of vector elements
cdfnorminv	h, s, d	Computes the inverse cumulative normal distribution function value of vector elements
lgamma	h, s, d	Computes the natural logarithm for the absolute value of the gamma function of vector elements
tgamma	h, s, d	Computes the gamma function of vector elements
expint1	h, s, d	Computes the exponential integral of vector elements
i0	h, s, d	Computes the regular modified cylindrical Bessel function of order 0
i1	h, s, d	Computes the regular modified cylindrical Bessel function of order 1
j0	h, s, d	Computes the Bessel function of the first kind of order 0
j1	h, s, d	Computes the Bessel function of the first kind of order 1
jn	h, s, d	Computes the Bessel function of the first kind of a given order

Function	Data Types	Description
y0	h, s, d	Computes the Bessel function of the second kind of order 0
y1	h, s, d	Computes the Bessel function of the second kind of order 1
yn	h, s, d	Computes the Bessel function of the second kind of a given order
Rounding Functions		
floor	h, s, d	Rounds towards minus infinity
ceil	h, s, d	Rounds towards plus infinity
trunc	h, s, d	Rounds towards zero infinity
round	h, s, d	Rounds to nearest integer
nearbyint	h, s, d	Rounds according to current mode
rint	h, s, d	Rounds according to current mode and raising inexact result exception
modf	h, s, d	Computes the integer and fractional parts
frac	h, s, d	Computes the fractional part
Miscellaneous Functions		
copysign	h, s, d	Returns vector of elements of one argument with signs changed to match other argument elements
nextafter	h, s, d	Returns vector of elements containing the next representable floating-point values following the values from the elements of one vector in the direction of the corresponding elements of another vector
fdim	h, s, d	Returns vector containing the differences of the corresponding elements of the vector arguments if the first is larger and +0 otherwise
fmax	h, s, d	Returns the larger of each pair of elements of the two vector arguments
fmin	h, s, d	Returns the smaller of each pair of elements of the two vector arguments

Function	Data Types	Description
maxmag	h, s, d	Returns the element with the larger magnitude between each pair of elements of the two vector arguments
minmag	h, s, d	Returns the element with the smaller magnitude between each pair of elements of the two vector arguments

Arithmetic Functions

Arithmetic functions perform the basic mathematical operations like addition, subtraction, multiplication or computation of the absolute value of the vector elements.

[add](#)

Computes the element-wise addition of vector *a* and vector *b*.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `add(a, b)` function performs element by element addition of vector *a* and vector *b*.

Argument 1	Argument 2	Result	Error Code
+0	+0	+0	
+0	-0	+0	
-0	+0	+0	
-0	-0	-0	
$+\infty$	$+\infty$	$+\infty$	
$+\infty$	$-\infty$	QNAN	
$-\infty$	$+\infty$	QNAN	
$-\infty$	$-\infty$	$-\infty$	
SNAN	any value	QNAN	
any value	SNAN	QNAN	

Specifications for special values of the complex functions are defined according to the following formula:

$$\text{add}(x1+i*y1, x2+i*y2) = (x1+x2) + i*(y1+y2)$$

Overflow in a complex function occurs (supported in the HA/LA accuracy modes only) when all $\text{RE}(x)$, $\text{RE}(y)$, $\text{IM}(x)$, $\text{IM}(y)$ arguments are finite numbers, but the real or imaginary part of the computed result is so large that it does not fit the target precision. In this case, the function returns ∞ in that part of the result, and sets the VM Error Status to `status::overflow` (overriding any possible `status::accuracy_warning` status).

API

Syntax

Buffer API

```

namespace oneapi::mkl::vm {

sycl::event add(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event add(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event add(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event add(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`add` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU
<code>std::complex<float></code>	CPU and GPU
<code>std::complex<double></code>	CPU and GPU

Description

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode Overrides the global VM mode setting for this function call. See the [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `add` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vadd.cpp
```

sub

Computes the element-wise subtraction of vector `b` from vector `a`.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `sub(a, b)` function performs element by element subtraction of vector `a` and vector `b`.

Argument 1	Argument 2	Result	Error Code
+0	+0	+0	
+0	-0	+0	
-0	+0	+0	
-0	-0	-0	
+∞	+∞	QNAN	
+∞	-∞	+∞	
-∞	+∞	-∞	
-∞	-∞	QNAN	
SNAN	any value	QNAN	

Argument 1	Argument 2	Result	Error Code
any value	SNAN	QNAN	

Specifications for special values of the complex functions are defined according to the following formula:

$$\text{sub}(x1+i*y1, x2+i*y2) = (x1-x2) + i*(y1-y2)$$

Overflow in a complex function occurs (supported in the HA/LA accuracy modes only) when all RE(x), RE(y), IM(x), IM(y) arguments are finite numbers, but the real or imaginary part of the computed result is so large that it does not fit the target precision. In this case, the function returns ∞ in that part of the result, and sets the VM Error Status to `status::overflow` (overriding any possible `status::accuracy_warning` status).

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event sub(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event sub(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event sub(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event sub(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

sub supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the 1st input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
b	The buffer containing the 2nd input vector.
sb	Slice selector for b. See Data Types for a description of the oneMKL slice type.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is mode::not_defined.
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `sub` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vsub.cpp
```

sqr

Compute the square of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `sqr()` function performs element by element squaring of the vector.

Argument	Result	Error Code
+0	+0	
-0	+0	
$+\infty$	$+\infty$	
$-\infty$	$+\infty$	
QNaN	QNaN	
SNAN	QNaN	

The `sqr` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event sqr(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event sqr(sycl::queue & exec_queue,
               sycl::buffer<T> & a,
               oneapi::mkl::slice sa,
               sycl::buffer<T> & y,
               oneapi::mkl::slice sy,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event sqr(sycl::queue & exec_queue,
               std::int64_t n,
               T const * a,
               T * y,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event sqr(sycl::queue & exec_queue,
               T const * a,
               oneapi::mkl::slice sa,
```

```

    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

```

sqr supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

y	The buffer containing the output vector.
---	--

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `sqr` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vsqr.cpp
```

mul

Computes the element-wise multiplication of vector `a` and vector `b`.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `mul(a, b)` function performs element by element multiplication of vector `a` and vector `b`.

Argument 1	Argument 2	Result	Error Code
+0	+0	+0	
+0	-0	-0	
-0	+0	-0	
-0	-0	+0	
+0	$+\infty$	QNAN	
+0	$-\infty$	QNAN	
-0	$+\infty$	QNAN	
-0	$-\infty$	QNAN	
$+\infty$	+0	QNAN	
$+\infty$	-0	QNAN	
$-\infty$	+0	QNAN	
$-\infty$	-0	QNAN	
$+\infty$	$+\infty$	$+\infty$	
$+\infty$	$-\infty$	$-\infty$	
$-\infty$	$+\infty$	$-\infty$	
$-\infty$	$-\infty$	$+\infty$	
SNAN	any value	QNAN	

Argument 1	Argument 2	Result	Error Code
any value	SNAN	QNAN	
QNAN	non-SNAN	QNAN	
non-SNAN	QNAN	QNAN	

Specifications for special values of the complex functions are defined according to the following formula:

$$\text{mul}(x1+i*y1, x2+i*y2) = (x1*x2-y1*y2) + i*(x1*y2+y1*x2)$$

Overflow in a complex function occurs (supported in the HA/LA accuracy modes only) when all RE(x), RE(y), IM(x), IM(y) arguments are finite numbers, but the real or imaginary part of the computed result is so large that it does not fit the target precision. In this case, the function returns ∞ in that part of the result, and sets the VM Error Status to `status::overflow` (overriding any possible `status::accuracy_warning` status).

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event mul(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event mul(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event mul(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {}),
```

```

    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event mul(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`mul` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU
<code>std::complex<float></code>	CPU and GPU
<code>std::complex<double></code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

`errhandler` Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

`exec_queue` The queue where the routine will be executed.

`n` Specifies the number of elements to be calculated.

`a` Pointer to the 1st input vector.

`sa` Slice selector for `a`. See [Data Types](#) for a description of the oneMKL slice type.

`b` Pointer to the 2nd input vector.

`sb` Slice selector for `b`. See [Data Types](#) for a description of the oneMKL slice type.

`sy` Slice selector for `y`. See [Data Types](#) for a description of the oneMKL slice type.

`depends` Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

`mode` Overrides the global VM mode setting for this function call. See the [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

`errhandler` Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

`y` The buffer containing the output vector.

return value (event) Computation end event.

USM API

`y` Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `mul` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vmul.cpp
```

mulbyconj

- [Description](#)
- [API](#)
- [Examples](#)

Description

Computes the element-wise multiplication of vector *a* elements and the complex conjugate of vector *b* elements.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event mulbyconj(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event mulbyconj(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event mulbyconj(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event mulbyconj(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
```

```
std::vector<sycl::event> const & depends = {},
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
oneapi::mkl::vm::error_handler<T> errhandler = {});
}
```

`mulbyconj` supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU and GPU
<code>std::complex<double></code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode Overrides the global VM mode setting for this function call. See the [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `mulbyconj` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vmulbyconj.cpp
```

conj

Computes the complex conjugate of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `conj` function performs element by element conjugation of the vector.

No special values are specified. The `conj` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event conj(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
```

```

    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

namespace oneapi::mkl::vm {

sycl::event conj(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event conj(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event conj(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

`conj` supports the following precisions and devices:

T	Devices supported
<code>std::complex<float></code>	CPU and GPU
<code>std::complex<double></code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.

mode Overrides the global VM mode setting for this function call. See [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

USM API

exec_queue The queue where the routine will be executed.

n Specifies the number of elements to be calculated.

a Pointer to the input vector.

sa Slice selector for **a**. See [Data Types](#) for a description of the oneMKL slice type.

sy Slice selector for **y**. See [Data Types](#) for a description of the oneMKL slice type.

depends Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode Overrides the global VM mode setting for this function call. See the [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `conj` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vconj.cpp
```

abs

Computes absolute value of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `abs(a)` function computes an absolute value of vector elements.

Argument	Result	Error Code
+0	+0	

Argument	Result	Error Code
-0	+0	
$+\infty$	$+\infty$	
$-\infty$	$+\infty$	
QNAN	QNAN	
SNAN	QNAN	

Specifications for special values of the complex functions are defined according to the following formula

$$\text{abs}(a) = \text{hypot}(\text{RE}(a), \text{IM}(a)).$$

The `abs` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event abs(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event abs(sycl::queue & exec_queue,
               sycl::buffer<T> & a,
               oneapi::mkl::slice sa,
               sycl::buffer<T> & y,
               oneapi::mkl::slice sy,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event abs(sycl::queue & exec_queue,
               std::int64_t n,
               T const * a,
               T * y,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event abs(sycl::queue & exec_queue,
```

```

T const * a,
oneapi::mkl::slice sa,
T * y,
oneapi::mkl::slice sy,
std::vector<sycl::event> const & depends = {},
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

```

abs supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `abs` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vabs.cpp
```

arg

Computes the argument of vector elements in $[-\pi, \pi]$.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `arg(a)` function computes argument of vector elements.

See [Special Value Notations](#) for the conventions used in the table below.

RE(a) i·IM(a)	$-\infty$	-X	-0	+0	+X	$+\infty$	NAN
$+i\cdot\infty$	$+3\cdot\pi/4$	$+\pi/2$	$+\pi/2$	$+\pi/2$	$+\pi/2$	$+\pi/4$	NAN
$+i\cdot Y$	$+\pi$		$+\pi/2$	$+\pi/2$		$+0$	NAN
$+i\cdot 0$	$+\pi$	$+\pi$	$+\pi$	$+0$	$+0$	$+0$	NAN
$-i\cdot 0$	$-\pi$	$-\pi$	$-\pi$	-0	-0	-0	NAN
$-i\cdot Y$	$-\pi$		$-\pi/2$	$-\pi/2$		-0	NAN
$-i\cdot\infty$	$-3\cdot\pi/4$	$-\pi/2$	$-\pi/2$	$-\pi/2$	$-\pi/2$	$-\pi/4$	NAN
$+i\cdot\text{NAN}$	NAN	NAN	NAN	NAN	NAN	NAN	NAN

NOTE $\arg(a) = \text{atan2}(\text{IM}(a), \text{RE}(a))$

The `arg` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event arg(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event arg(sycl::queue & exec_queue,
               sycl::buffer<T> & a,
               oneapi::mkl::slice sa,
               sycl::buffer<T> & y,
               oneapi::mkl::slice sy,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event arg(sycl::queue & exec_queue,
               std::int64_t n,
               T const * a,
               T * y,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event arg(sycl::queue & exec_queue,
               T const * a,
               oneapi::mkl::slice sa,
               T * y,
               oneapi::mkl::slice sy,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

arg supports the following precisions and devices:

T	Devices supported
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

Buffer API

exec_queue The queue where the routine will be executed.

<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `arg` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/varg.cpp
```

linearfrac

Computes the element-wise linear fractional transformation of vectors `a` and `b` with scalar parameters.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `linearfrac(a,b,scalea,shifta,scaleb,shiftb)` function performs a linear fraction transformation of vector `a` by vector `b` with scalar parameters: scaling multipliers `scalea`, `scaleb` and shifting addends `shifta`, `shiftb`:

$$y[i] = (scalea \cdot a[i] + shifta) / (scaleb \cdot b[i] + shiftb), i=1,2 \dots n$$

The `linearfrac` function is implemented in the EP accuracy mode only, therefore no special values are defined for this function. If used in HA or LA mode, `linearfrac` sets the VM Error Status to `status::accuracy_warning`. Correctness is guaranteed within the threshold limitations defined for each input parameter (see the table below); otherwise, the behavior is unspecified.

Threshold Limitations on Input Parameters

```

2EMIN/2 ≤ |scalea| ≤ 2(EMAX-2)/2
2EMIN/2 ≤ |scaleb| ≤ 2(EMAX-2)/2
|shifta| ≤ 2EMAX-2
|shiftb| ≤ 2EMAX-2
2EMIN/2 ≤ a[i] ≤ 2(EMAX-2)/2
2EMIN/2 ≤ b[i] ≤ 2(EMAX-2)/2
a[i] ≠ -(shifta/scalea)*(1-δ1), |δ1| ≤ 21-(p-1)/2
b[i] ≠ -(shiftb/scaleb)*(1-δ2), |δ2| ≤ 21-(p-1)/2

```

`EMIN` and `EMAX` are the minimum and maximum exponents and `p` is the number of significant bits (precision) for the corresponding data type according to the ANSI/IEEE Standard 754-2008 [IEEE754]:

- for single precision `EMIN` = -126, `EMAX` = 127, `p` = 24
- for double precision `EMIN` = -1022, `EMAX` = 1023, `p` = 53

The thresholds become less strict for common cases with `scalea=0` and/or `scaleb=0`:

- if `scalea=0`, there are no limitations for the values of `a[i]` and `shifta`.
- if `scaleb=0`, there are no limitations for the values of `b[i]` and `shiftb`.

API

Syntax

Buffer API

```

namespace oneapi::mkl::vm {

sycl::event linearfrac(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    T scalea,
    T shifta,
    T scaleb,
    T shiftb,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,

```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event linearfrac(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    T scalea,
    T shifta,
    T scaleb,
    T shiftb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event linearfrac(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T scalea,
    T shifta,
    T scaleb,
    T shiftb,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

```

namespace oneapi::mkl::vm {

sycl::event linearfrac(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T scalea,
    T shifta,
    T scaleb,
    T shiftb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`linearfrac` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>scalea</code>	Constant value for scaling multipliers of vector <code>a</code>
<code>shifta</code>	Constant value for shifting addend of vector <code>a</code>
<code>scaleb</code>	Constant value for scaling multipliers of vector <code>b</code>
<code>shiftb</code>	Constant value for shifting addend of vector <code>b</code>
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>scalea</code>	Constant value for scaling multipliers of vector <code>a</code>

shifta	Constant value for shifting addend of vector <i>a</i>
scaleb	Constant value for scaling multipliers of vector <i>b</i>
shiftb	Constant value for shifting addend of vector <i>b</i>
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<i>y</i>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<i>y</i>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `linearfrac` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vllinearfrac.cpp
```

fmod

Computes the element-wise remainder of vector *a* elements divided by vector *b* elements that has the same sign as vector *a* elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `fmod (a,b)` function computes the modulus function of each element of vector *a*, with respect to the corresponding elements of vector *b*:

$$a_i - b_i \cdot \text{trunc}(a_i/b_i)$$

In general, the modulus function `fmod (ai, bi)` returns the value $a_i - n \cdot b_i$ for some integer *n* such that if *b_i* is nonzero, the result has the same sign as *a_i* and a magnitude less than the magnitude of *b_i*.

Argument 1	Argument 2	Result	Error Code
a not NAN	± 0	NAN	status::sing
$\pm \infty$	b not NAN	NAN	status::sing
± 0	$b \neq 0$, not NAN	± 0	
a finite	$\pm \infty$	a	
NAN	b		
a	NAN	NAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event fmod(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event fmod(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event fmod(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```



```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event fmod(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

fmod supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the 1st input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
b	The buffer containing the 2nd input vector.
sb	Slice selector for b. See Data Types for a description of the oneMKL slice type.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
------------	---

<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready).
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector of size <code>n</code> .
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector of size <code>n</code> .
return value (event)	Computation end event.

Examples

An example of how to use `fmod` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vfmod.cpp
```

remainder

Computes the element-wise remainder of vector `a` elements divided by vector `b` elements, that is nearest to zero.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `remainder(a)` function computes the remainder of each element of vector `a`, with respect to the corresponding elements of vector `b`: compute the values of `n` such that

$$n = a_i - n * b_i$$

where `n` is the integer nearest to the exact value of a_i/b_i . If two integers are equally close to a_i/b_i , `n` is the even one. If `n` is zero, it has the same sign as a_i .

Argument 1	Argument 2	Result	VM Error Status
a not NAN	± 0	NAN	status::errdom
$\pm\infty$	b not NAN	NAN	
± 0	$b \neq 0$, not NAN	± 0	
a finite	$\pm\infty$	a	
NAN	b	NAN	
a	NAN	NAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event remainder(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event remainder(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event remainder(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event remainder(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`remainder` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
-------------------------	---

<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>y</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `remainder` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vremainder.cpp
```

Power and Root Functions

inv

Computes the element-wise multiplicative inverse (or reciprocal) of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `inv(a)` function performs element by element inversion of the vector.

Argument	Result	VM Error Status
+0	$+\infty$	status::sing
-0	$-\infty$	status::sing
$+\infty$	+0	
$-\infty$	-0	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event inv(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event inv(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event inv(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event inv(sycl::queue & exec_queue,
```

```

    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

inv supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode Overrides the global VM mode setting for this function call. See the [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `inv` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vinv.cpp
```

div

Computes the element-wise division of vector `a` elements by vector `b` elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `div(a,b)` function performs element by element division of vector `a` by vector `b`.

Argument 1	Argument 2	Result	VM Error Status
$X > +0$	$+0$	$+\infty$	<code>status::sing</code>
$X > +0$	-0	$-\infty$	<code>status::sing</code>
$X < +0$	$+0$	$-\infty$	<code>status::sing</code>
$X < +0$	-0	$+\infty$	<code>status::sing</code>
$+0$	$+0$	QNAN	<code>status::sing</code>
-0	-0	QNAN	<code>status::sing</code>
$X > +0$	$+\infty$	$+0$	
$X > +0$	$-\infty$	-0	
$+\infty$	$+\infty$	QNAN	

Argument 1	Argument 2	Result	VM Error Status
$-\infty$	$-\infty$	QNAN	
QNAN	QNAN	QNAN	
SNAN	SNAN	QNAN	

Specifications for special values of the complex functions are defined according to the following formula

$$\text{Div}(x1+i*y1, x2+i*y2) = (x1+i*y1) * (x2-i*y2) / (x2*x2+y2*y2).$$

Overflow in a complex function occurs when $x2+i*y2$ is not zero, $x1$, $x2$, $y1$, $y2$ are finite numbers, but the real or imaginary part of the exact result is so large that it does not fit the target precision. In that case, the function returns ∞ in that part of the result, and sets the VM Error Status to `status::overflow`.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event div(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event div(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event div(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event div(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

div supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the 1st input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
b	The buffer containing the 2nd input vector.
sb	Slice selector for b. See Data Types for a description of the oneMKL slice type.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `div` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vdiv.cpp
```

sqrt

Computes the element-wise square root of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `sqrt` function computes a square root of vector elements.

Argument	Result	VM Error Status
a < +0	QNAN	status::errdom
+0	+0	
-0	-0	
$-\infty$	QNAN	status::errdom
$+\infty$	$+\infty$	
QNAN	QNAN	
SNAN	QNAN	

$+i\cdot\infty$	$+\infty + i\cdot\infty$	$+\infty + i\cdot\infty$	$+\infty + i\cdot\infty$	$+\infty + i\cdot\infty$	$+\infty + i\cdot\infty$	$+\infty + i\cdot\infty$	$+\infty + i\cdot\infty$	$+\infty + i\cdot\infty$	$+\infty + i\cdot\infty$	$+\infty + i\cdot\infty$	$+\infty + i\cdot\infty$
$+i\cdot Y$	$+0+i\cdot\infty$								$+\infty + i\cdot 0$		
$+i\cdot 0$	$+0+i\cdot\infty$		$+0+i\cdot 0$	$+0+i\cdot 0$					$+\infty + i\cdot 0$		
$-i\cdot 0$	$+0-i\cdot\infty$		$+0-i\cdot 0$	$+0-i\cdot 0$					$+\infty - i\cdot 0$		
$-i\cdot Y$	$+0-i\cdot\infty$								$+\infty - i\cdot 0$		
$-i\cdot\infty$	$+\infty - i\cdot\infty$	$+\infty - i\cdot\infty$	$+\infty - i\cdot\infty$	$+\infty - i\cdot\infty$	$+\infty - i\cdot\infty$	$+\infty - i\cdot\infty$	$+\infty - i\cdot\infty$	$+\infty - i\cdot\infty$	$+\infty - i\cdot\infty$	$+\infty - i\cdot\infty$	$+\infty - i\cdot\infty$
$+i\cdot\text{NAN}$											

NOTE $\text{Sqrt}(\text{CONJ}(z)) = \text{CONJ}(\text{Sqrt}(z))$.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event sqrt(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event sqrt(sycl::queue & exec_queue,
```

```

    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event sqrt(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

```

namespace oneapi::mkl::vm {

sycl::event sqrt(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`sqrt` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.

sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `sqrt` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vsqrt.cpp
```

invsqrt

Computes the element-wise inverse square root of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `invsqrt(a)` function computes an inverse square root of vector elements.

Argument	Result	VM Error Status
$a < +0$	QNAN	<code>status::errdom</code>
$+0$	$+\infty$	<code>status::sing</code>
-0	$-\infty$	<code>status::sing</code>
$-\infty$	QNAN	<code>status::errdom</code>
$+\infty$	$+0$	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event invsqrt(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event invsqrt(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event invsqrt(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event invsqrt(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`invsqrt` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.

sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `invsqrt` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vinvsqrt.cpp
```

cbrt

Computes the element-wise cube root of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `cbrt(a)` function computes a cube root of vector elements.

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	$+\infty$	
$-\infty$	$-\infty$	
QNaN	QNaN	
SNAN	QNaN	
+0	+0	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event cbrt(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event cbrt(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event cbrt(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event cbrt(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

cbrt supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU

T	Devices supported
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Pointer to the input vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `cbirt` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vcbirt.cpp
```

invcbirt

Computes the element-wise inverse cube root of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `invcbirt(a)` function computes an inverse cube root of vector elements.

Argument	Result	Error Code
+0	$+\infty$	<code>status::sing</code>
-0	$-\infty$	<code>status::sing</code>
$+\infty$	+0	
$-\infty$	-0	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event invcbirt(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event invcbirt(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event invcbirt(sycl::queue & exec_queue,
    std::int64_t n,
```

```

    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event invcbirt(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`invcbirt` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.

<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `invcbprt` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vinvcbprt.cpp
```

pow2o3

Computes the element-wise cube root of the square of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `pow2o3(a)` function computes the cube root of the square of each vector element.

Argument	Result	Error Code
+0	+0	
-0	+0	
+∞	+∞	
-∞	+∞	
QNAN	QNAN	

Argument	Result	Error Code
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event pow2o3(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event pow2o3(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event pow2o3(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event pow2o3(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

`pow2o3` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
----------------	--

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `pow2o3` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vpow203.cpp
```

pow3o2

Computes the element-wise square root of the cube of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `pow3o2(a)` function computes the square root of the cube of each vector element.

Data Type	Threshold Limitations on Input Parameters	
half precision	$ a_i < (\text{FLT16_MAX})^{2/3}$	
single precision	$ a_i < (\text{FLT_MAX})^{2/3}$	
double precision	$ a_i < (\text{DBL_MAX})^{2/3}$	

Argument	Result	VM Error Status
$a < +0$	QNAN	<code>status::errdom</code>
$+0$	$+0$	
-0	-0	
$-\infty$	QNAN	<code>status::errdom</code>
$+\infty$	$+\infty$	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {
    sycl::event pow3o2(sycl::queue & exec_queue,
        std::int64_t n,
```

```

    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event pow3o2(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event pow3o2(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event pow3o2(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

pow3o2 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `pow3o2` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vpow3o2.cpp
```

pow

Computes the element-wise exponentiation of vector *a* elements raised to the power of vector *b* elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `pow(a, b)` function computes *a* to the power *b* for elements of two vectors.

The real function `pow` has certain limitations on the input range of *a* and *b* parameters. Specifically, if *a*[*i*] is positive, then *b*[*i*] may be arbitrary. For negative *a*[*i*], the value of *b*[*i*] must be an integer (either positive or negative).

The complex function `pow` has no input range limitations.

Argument 1	Argument 2	Result	Error Code
+0	neg. odd integer	$+\infty$	<code>status::errdom</code>
-0	neg. odd integer	$-\infty$	<code>status::errdom</code>
+0	neg. even integer	$+\infty$	<code>status::errdom</code>
-0	neg. even integer	$+\infty$	<code>status::errdom</code>
+0	neg. non-integer	$+\infty$	<code>status::errdom</code>
-0	neg. non-integer	$+\infty$	<code>status::errdom</code>
-0	pos. odd integer	+0	
-0	pos. odd integer	-0	
+0	pos. even integer	+0	
-0	pos. even integer	+0	
+0	pos. non-integer	+0	
-0	pos. non-integer	+0	
-1	$+\infty$	+1	
-1	$-\infty$	+1	
+1	any value	+1	
+1	+0	+1	
+1	-0	+1	
+1	$+\infty$	+1	
+1	$-\infty$	+1	
+1	QNAN	+1	
any value	+0	+1	

Argument 1	Argument 2	Result	Error Code
+0	+0	+1	
-0	+0	+1	
$+\infty$	+0	+1	
$-\infty$	+0	+1	
QNAN	+0	+1	
any value	-0	+1	
+0	-0	+1	
-0	-0	+1	
$+\infty$	-0	+1	
$-\infty$	-0	+1	
QNAN	-0	+1	
$a < +0$	non-integer	QNAN	<code>status::errdom</code>
$ a < 1$	$-\infty$	$+\infty$	
+0	$-\infty$	$+\infty$	<code>status::errdom</code>
-0	$-\infty$	$+\infty$	<code>status::errdom</code>
$ a > 1$	$-\infty$	+0	
$+\infty$	$-\infty$	+0	
$-\infty$	$-\infty$	+0	
$ a < 1$	$+\infty$	+0	
+0	$+\infty$	+0	
-0	$+\infty$	+0	
$ a > 1$	$+\infty$	$+\infty$	
$+\infty$	$+\infty$	$+\infty$	
$-\infty$	$+\infty$	$+\infty$	
$-\infty$	neg. odd integer	-0	
$-\infty$	neg. even integer	+0	
$-\infty$	neg. non-integer	+0	
$-\infty$	pos. odd integer	$-\infty$	
$-\infty$	pos. even integer	$+\infty$	
$-\infty$	pos. non-integer	$+\infty$	
$+\infty$	$b < +0$	+0	

Argument 1	Argument 2	Result	Error Code
$+\infty$	$b > +0$	$+\infty$	
Big finite value*	Big finite value*	$+/-\infty$	<code>status::overflow</code>
QNAN	QNAN	QNAN	
QNAN	SNAN	QNAN	
SNAN	QNAN	QNAN	
SNAN	SNAN	QNAN	

* Overflow in a real function is supported only in the HA/LA accuracy modes. The overflow occurs when x and y are finite numbers, but the result is too large to fit the target precision. In this case, the function:

1. Returns ∞ in the result.
2. Sets the VM Error Status to `status::overflow`.

Overflow in a complex function occurs (supported in the HA/LA accuracy modes only) when all $\text{RE}(x)$, $\text{RE}(y)$, $\text{IM}(x)$, $\text{IM}(y)$ arguments are finite numbers, but the real or imaginary part of the computed result is so large that it does not fit the target precision. In this case, the function returns ∞ in that part of the result, and sets the VM Error Status to `status::overflow` (overriding any possible `status::accuracy_warning` status).

The complex double precision versions of this function are implemented in the EP accuracy mode only. If used in HA or LA mode, the functions set the VM Error Status to `status::accuracy_warning`.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event pow(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & b,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event pow(sycl::queue & exec_queue,
               sycl::buffer<T> & a,
               oneapi::mkl::slice sa,
               sycl::buffer<T> & b,
               oneapi::mkl::slice sb,
               sycl::buffer<T> & y,
               oneapi::mkl::slice sy,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event pow(sycl::queue & exec_queue,
               std::int64_t n,
               T const * a,
               T const * b,
               T * y,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event pow(sycl::queue & exec_queue,
               T const * a,
               oneapi::mkl::slice sa,
               T const * b,
               oneapi::mkl::slice sb,
               T * y,
               oneapi::mkl::slice sy,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

`pow` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU
<code>std::complex<float></code>	CPU and GPU
<code>std::complex<double></code>	CPU and GPU

Input Parameters**Buffer API**

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.

<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `pow` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vpow.cpp
```

powx

Computes the element-wise exponentiation of vector `a` elements raised to the power of scalar `b`.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `powx` function computes a to the power b for a vector a and a scalar b .

The real function `powx` has certain limitations on the input range of a and b parameters. Specifically, if $a[i]$ is positive, then b may be arbitrary. For negative $a[i]$, the value of b must be an integer (either positive or negative).

The complex function `powx` has no input range limitations.

Special values and VM Error Status treatment are the same as for the `pow` function.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event powx(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    T b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event powx(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    T b,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event powx(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event powx(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T b,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`powx` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU
<code>std::complex<float></code>	CPU and GPU
<code>std::complex<double></code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Fixed value of power.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Fixed value of power.
<code>y</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `powx` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vpowx.cpp
```

powr

Computes the element-wise exponentiation of vector `a` elements raised to the power of vector `b` elements, where the elements of vector `a` are all non-negative.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `powr(a, b)` function raises each element of vector `a` by the corresponding element of vector `b`. The elements of `a` are all nonnegative ($a_i \geq 0$).

Data Type	Threshold Limitations on Input Parameters
half precision	$a_i < (\text{FLT16_MAX})^{1/\text{'b'i'}}$
single precision	$a_i < (\text{FLT_MAX})^{1/\text{'b'i'}}$
double precision	$a_i < (\text{DBL_MAX})^{1/\text{'b'i'}}$

Special values and VM Error Status treatment for `v?Powr` function are the same as for `pow`, unless otherwise indicated in this table:

Argument 1	Argument 2	Result	Error Code
$a < 0$	any value b	NAN	<code>status::errdom</code>
$0 < a < \infty$	± 0	1	
± 0	$-\infty < b < 0$	$+\infty$	
± 0	$-\infty$	$+\infty$	
± 0	$b > 0$	$+0$	
1	$-\infty < b < \infty$	1	
± 0	± 0	NAN	
$+\infty$	± 0	NAN	
1	$+\infty$	NAN	
$a \geq 0$	NAN	NAN	
NAN	any value b	NAN	
$0 < a < 1$	$-\infty$	$+\infty$	
$a > 1$	$-\infty$	$+0$	
$0 \leq a < 1$	$+\infty$	$+0$	
$a > 1$	$+\infty$	$+\infty$	
$+\infty$	$b < +0$	$+0$	
$+\infty$	$b > +0$	$+\infty$	
QNAN	QNAN	QNAN	<code>status::errdom</code>
QNAN	SNAN	QNAN	<code>status::errdom</code>
SNAN	QNAN	QNAN	<code>status::errdom</code>
SNAN	SNAN	QNAN	<code>status::errdom</code>

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event powr(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event powr(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event powr(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event powr(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

`powr` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `powr` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vpowr.cpp
```

hypot

Computes the element-wise square root of the sum of the squares of vector `a` elements and vector `b` elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `hypot(a,b)` function computes a square root of sum of two squared elements.

Argument 1	Argument 2	Result	Error Code
+0	+0	+0	
-0	-0	+0	
+∞	any value	+∞	
any value	+∞	+∞	
SNAN	any value	QNAN	INVALID
any value	SNAN	QNAN	INVALID
QNAN	any value	QNAN	
any value	QNAN	QNAN	

Data Type	Threshold Limitations on Input Parameters
half precision	$\text{abs}(a[i]) < \text{sqrt}(\text{FLT16_MAX})$ $\text{abs}(b[i]) < \text{sqrt}(\text{FLT16_MAX})$

Data Type	Threshold Limitations on Input Parameters
single precision	$\text{abs}(a[i]) < \text{sqrt}(\text{FLT_MAX})$ $\text{abs}(b[i]) < \text{sqrt}(\text{FLT_MAX})$
double precision	$\text{abs}(a[i]) < \text{sqrt}(\text{DBL_MAX})$ $\text{abs}(b[i]) < \text{sqrt}(\text{DBL_MAX})$

The `hypot(a,b)` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event hypot(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event hypot(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event hypot(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event hypot(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
```



```

oneapi::mkl::slice sb,
T * y,
oneapi::mkl::slice sy,
std::vector<sycl::event> const & depends = {},
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

```

`hypot` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode Overrides the global VM mode setting for this function call. See the [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `hypot` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vhypot.cpp
```

Exponential and Logarithmic Functions

exp

Computes the element-wise natural (base-e) exponential of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `exp(a)` function computes an exponential of vector elements.

Data Type	Threshold Limitations on Input Parameters	
half precision	$a[i] < \text{Log}(\text{FLT16_MAX})$	
single precision	$a[i] < \text{Log}(\text{FLT_MAX})$	
double precision	$a[i] < \text{Log}(\text{DBL_MAX})$	

Argument	Result	Error Code
+0	+1	
-0	+1	
$a > \text{overflow}$	$+\infty$	<code>status::overflow</code>
$a < \text{underflow}$	+0	<code>status::overflow</code>
$+\infty$	$+\infty$	

Argument	Result	Error Code
$-\infty$	+0	
QNAN	QNAN	
SNAN	QNAN	
$+i\cdot\infty$		
$+i\cdot Y$		
$+i\cdot 0$		
$-i\cdot 0$		
$-i\cdot Y$		
$-i\cdot\infty$		
$+i\cdot\text{NAN}$		

NOTE

- The complex $\exp(z)$ function sets the VM Error Status to `status::overflow` in the case of overflow, that is, when both $\text{RE}(z)$ and $\text{IM}(z)$ are finite non-zero numbers, but the real or imaginary part of the exact result is so large that it does not meet the target precision.

API**Syntax****Buffer API**

```
namespace oneapi::mkl::vm {
    sycl::event exp(sycl::queue & exec_queue,
        std::int64_t n,
        sycl::buffer<T> & a,
        sycl::buffer<T> & y,
        oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
        oneapi::mkl::vm::error_handler<T> errhandler = {});
}
```

```
namespace oneapi::mkl::vm {
    sycl::event exp(sycl::queue & exec_queue,
        sycl::buffer<T> & a,
        oneapi::mkl::slice sa,
        sycl::buffer<T> & y,
        oneapi::mkl::slice sy,
        oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
        oneapi::mkl::vm::error_handler<T> errhandler = {});
}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event exp(sycl::queue & exec_queue,
               std::int64_t n,
               T const * a,
               T * y,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event exp(sycl::queue & exec_queue,
               T const * a,
               oneapi::mkl::slice sa,
               T * y,
               oneapi::mkl::slice sy,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

exp supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is mode::not_defined.
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `exp` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vexp.cpp
```

exp2

Computes the element-wise base-2 exponential of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `exp2` function computes the base 2 exponential of vector elements.

Data Type	Threshold Limitations on Input Parameters
half precision	$a_i < \log_2(\text{FLT16_MAX})$
single precision	$a_i < \log_2(\text{FLT_MAX})$

Data Type	Threshold Limitations on Input Parameters	
double precision	$a_i < \log_2(\text{DBL_MAX})$	

Argument	Result	Error Code
+0	+1	
-0	+1	
$a > \text{overflow}$	$+\infty$	<code>status::overflow</code>
$a < \text{underflow}$	+0	<code>status::underflow</code>
$+\infty$	$+\infty$	
$-\infty$	+0	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event exp2(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event exp2(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event exp2(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {}),
```

```

    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event exp2(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

exp2 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.

sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `exp2` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vexp2.cpp
```

exp10

Computes the element-wise base-10 exponential of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `exp10(a)` function computes the base 10 exponential of vector elements.

Data Type		Threshold Limitations on Input Parameters
half precision		$a_i < \log_{10}(\text{FLT16_MAX})$
single precision		$a_i < \log_{10}(\text{FLT_MAX})$
double precision		$a_i < \log_{10}(\text{DBL_MAX})$

Argument	Result	VM Error Status
+0	+1	
-0	+1	

Argument	Result	VM Error Status
$a > \text{overflow}$	$+\infty$	<code>status::overflow</code>
$a < \text{underflow}$	$+0$	<code>status::underflow</code>
$+\infty$	$+\infty$	
$-\infty$	$+0$	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event exp10(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event exp10(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event exp10(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event exp10(sycl::queue & exec_queue,
```

```

T const * a,
oneapi::mkl::slice sa,
T * y,
oneapi::mkl::slice sy,
std::vector<sycl::event> const & depends = {},
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

exp10 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `exp10` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vexp10.cpp
```

expm1

Computes the element-wise subtraction of 1 from the exponential of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `expm1(a)` function computes an exponential of vector elements decreased by 1.

Argument	Result	Error Code
+0	+1	
-0	+1	
a > overflow	+∞	<code>status::overflow</code>
+∞	+∞	
-∞	-0	
QNAN	QNAN	
SNAN	QNAN	

Data Type	Threshold Limitations on Input Parameters
half precision	$a[i] < \text{Log}(\text{FLT16_MAX})$

Data Type	Threshold Limitations on Input Parameters
single precision	$a[i] < \text{Log}(\text{FLT_MAX})$
double precision	$a[i] < \text{Log}(\text{DBL_MAX})$

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event expml(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event expml(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event expml(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event expml(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```
oneapi::mkl::vm::error_handler<T> errhandler = {});
}
```

expm1 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `expm1` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vexpm1.cpp
```

ln

Computes the element-wise natural logarithm of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `ln(a)` function computes natural logarithm of vector elements.

Argument	Result	Error Code
+1	+0	
a < +0	QNAN	status::errdom
+0	$-\infty$	status::sing
-0	$-\infty$	status::sing
$-\infty$	QNAN	status::errdom
$+\infty$	$+\infty$	
QNAN	QNAN	
SNAN	QNAN	

RE(a) i·IM(a)	$-\infty$	$-X$	-0	$+0$	$+X$	$+\infty$	NAN
$+i \cdot \infty$	$+\infty + i \cdot \frac{3\pi}{4}$	$+\infty + i \cdot \frac{\pi}{2}$	$+\infty + i \cdot \frac{\pi}{2}$	$+\infty + i \cdot \frac{\pi}{2}$	$+\infty + i \cdot \frac{\pi}{2}$	$+\infty + i \cdot \frac{\pi}{4}$	QNAN
$+i \cdot Y$	$+\infty - i \cdot \pi$					$+\infty + i \cdot 0$	QNAN + i · QNAN
$+i \cdot 0$	$+\infty - i \cdot \pi$		$+\infty + i \cdot \pi$	$-\infty - i \cdot 0$		$+\infty + i \cdot 0$	QNAN + i · QNAN

RE(a) i·IM(a)	$-\infty$	$-X$	-0	$+0$	$+X$	$+\infty$	NAN
$-i \cdot 0$	$+\infty - i \cdot \pi$		$-\infty + i \cdot \pi$	$-\infty - i \cdot 0$		$+\infty - i \cdot 0$	$QNaN + i \cdot QNaN$
$-i \cdot Y$	$+\infty - i \cdot \pi$					$+\infty - i \cdot 0$	$QNaN + i \cdot QNaN$
$-i \cdot \infty$	$+\infty - i \cdot \frac{3\pi}{4}$	$-\infty - i \cdot \frac{\pi}{2}$	$+\infty - i \cdot \frac{\pi}{2}$	$+\infty - i \cdot \frac{\pi}{2}$	$+\infty - i \cdot \frac{\pi}{2}$	$+\infty - i \cdot \frac{\pi}{4}$	$+\infty + i \cdot QNaN$
$+i \cdot NAN$	$+\infty + i \cdot QNaN$	$QNaN + QNaN$	$QNaN + QNaN$	$QNaN + QNaN$	$QNaN + QNaN$	$QNaN + QNaN$	$QNaN + i \cdot QNaN$

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event ln(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event ln(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event ln(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event ln(sycl::queue & exec_queue,
    T const * a,
```

```

oneapi::mkl::slice sa,
T * y,
oneapi::mkl::slice sy,
std::vector<sycl::event> const & depends = {},
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

ln supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event..

Examples

An example of how to use `ln` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vln.cpp
```

log2

Computes the element-wise base-2 logarithm of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `log2(a)` function computes the base 2 logarithm of vector elements.

Argument	Result	Error Code
+1	+0	
a < +0	QNAN	<code>status::errdom</code>
+0	$-\infty$	<code>status::sing</code>
-0	$-\infty$	<code>status::sing</code>
$-\infty$	QNAN	<code>status::errdom</code>
$+\infty$	$+\infty$	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event log2(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event log2(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event log2(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event log2(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

log2 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU

T	Devices supported
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y

Pointer to the output vector.

return value (event)

Computation end event..

Examples

An example of how to use `log2` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vlog2.cpp
```

log10

Computes the element-wise base-10 logarithm of vector elements.

- Description
- API
- Examples

Description

The `log10(a)` function computes the base 10 logarithm of vector elements.

Argument	Result	Error Code
+1	+0	
a <+0	QNAN	status::errdom
+0	$-\infty$	status::sing
-0	$-\infty$	status::sing
$-\infty$	QNAN	status::errdom
$+\infty$	$+\infty$	
QNAN	QNAN	
SNAN	QNAN	

RE(a) i·IM(a)	$-\infty$	$-X$	-0	$+0$	$+X$	$+\infty$	NAN
$+i \cdot \infty$	$+\infty + i \frac{3}{4} \frac{\pi}{\ln(10)}$	$+\infty + i \frac{\pi}{2 \ln(10)}$	$+\infty + i \frac{\pi}{2 \ln(10)}$	$+\infty + i \frac{\pi}{2 \ln(10)}$	$+\infty + i \frac{\pi}{2 \ln(10)}$	$+\infty + i \frac{\pi}{4 \ln(10)}$	QNAN
$+i \cdot Y$	$+\infty + i \frac{\pi}{\ln(10)}$					$+\infty + i \cdot 0$	QNAN + i · QNAN
$+i \cdot 0$	$+\infty + i \frac{\pi}{\ln(10)}$		$-\infty + i \frac{\pi}{\ln(10)}$	$+\infty + i \cdot 0$		$+\infty + i \cdot 0$	QNAN + i · QNAN
$-i \cdot 0$	$+\infty - i \frac{\pi}{\ln(10)}$		$-\infty - i \frac{\pi}{\ln(10)}$	$-\infty - i \cdot 0$		$+\infty - i \cdot 0$	QNAN - i · QNAN
$-i \cdot Y$	$+\infty - i \frac{\pi}{\ln(10)}$					$+\infty - i \cdot 0$	QNAN + i · QNAN

RE(a) i-IM(a)	$-\infty$	$-X$	-0	$+0$	$+X$	$+\infty$	NAN
$-i \cdot \infty$	$+\infty + i \frac{3}{4} \frac{\pi}{\ln(10)} \infty$	$-i \frac{\pi}{2} \frac{1}{\ln(10)} \infty$	$-i \frac{\pi}{2} \frac{1}{\ln(10)} \infty$	$-i \frac{\pi}{2} \frac{1}{\ln(10)} \infty$	$-i \frac{\pi}{2} \frac{1}{\ln(10)} \infty$	$-i \frac{\pi}{2} \frac{1}{\ln(10)} \infty$	$-i \frac{\pi}{4} \frac{1}{\ln(10)} \infty + i \cdot QNAN$
$+i \cdot NAN$	$+\infty + i \cdot QNAN$	$+i \cdot QNAN$	$+i \cdot QNAN$	$+i \cdot QNAN$	$+i \cdot QNAN$	$+i \cdot QNAN$	$+i \cdot QNAN$

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event log10(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event log10(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event log10(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event log10(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
```

```
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
oneapi::mkl::vm::error_handler<T> errhandler = {});
}
```

log10 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU
std::complex<float>	CPU and GPU
std::complex<double>	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `log10` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vlog10.cpp
```

log1p

Computes the element-wise natural logarithm of 1 plus vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `log1p(a)` function computes a natural logarithm of vector elements that are increased by 1.

Argument	Result	VM Error Status
-1	$-\infty$	<code>status::sing</code>
$a < -1$	QNAN	<code>status::errdom</code>
+0	+0	
-0	-0	
$-\infty$	QNAN	<code>status::errdom</code>
$+\infty$	$+\infty$	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event loglp(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event loglp(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event loglp(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event loglp(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

loglp supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU

T	Devices supported
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
---	-------------------------------

return value (event) Computation end event.

Examples

An example of how to use `loglp` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vloglp.cpp
```

logb

Computes the element-wise exponents of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `logb(a)` function computes the exponents of the elements of the input vector `a`. For each element a_i of vector `a`, this is the integral part of $\log_2|a_i|$. The returned value is exact and is independent of the current rounding direction mode.

Argument	Result	VM Error Status
+0	$+\infty$	<code>status::errdom</code>
-0	$-\infty$	<code>status::errdom</code>
$-\infty$	$+\infty$	
$+\infty$	$+\infty$	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event logb(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event logb(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
```

```

    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event logb(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

```

namespace oneapi::mkl::vm {

sycl::event logb(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

logb supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.

mode Overrides the global VM mode setting for this function call. See [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue The queue where the routine will be executed.

n Specifies the number of elements to be calculated.

a Pointer to the input vector.

sa Slice selector for **a**. See [Data Types](#) for a description of the oneMKL slice type.

sy Slice selector for **y**. See [Data Types](#) for a description of the oneMKL slice type.

depends Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode Overrides the global VM mode setting for this function call. See the [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `logb` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vlogb.cpp
```

Trigonometric Functions

- [cos](#)
- [sin](#)
- [sincos](#)
- [cis](#)
- [tan](#)
- [acos](#)

- [asin](#)
- [atan](#)
- [atan2](#)
- [cospi](#)
- [sinpi](#)
- [tanpi](#)
- [acospi](#)
- [asinpi](#)
- [atanpi](#)
- [atan2pi](#)
- [cosd](#)
- [sind](#)
- [tand](#)

cos

Computes the element-wise cosine of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `cos(a)` function computes cosine of vector elements.

Note that arguments $\text{abs}(a[i]) \leq 213$ and $\text{abs}(a[i]) \leq 216$ for single and double precisions, respectively, are called fast computational path. These are trigonometric function arguments for which VM provides the best possible performance. Avoid arguments that do not belong to the fast computational path in the VM High Accuracy (HA) and Low Accuracy (LA) functions. Alternatively, you can use VM Enhanced Performance (EP) functions that are fast on the entire function domain. However, these functions provide less accuracy.

Argument	Result	VM Error Status
+0	+1	
-0	+1	
$+\infty$	QNAN	<code>status::errdom</code>
$-\infty$	QNAN	<code>status::errdom</code>
QNAN	QNAN	
SNAN	QNAN	

Specifications for special values of the complex functions are defined according to the following formula

$$\text{Cos}(z) = \text{Cosh}(i*z).$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event cos(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event cos(sycl::queue & exec_queue,
               sycl::buffer<T> & a,
               oneapi::mkl::slice sa,
               sycl::buffer<T> & y,
               oneapi::mkl::slice sy,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event cos(sycl::queue & exec_queue,
               std::int64_t n,
               T const * a,
               T * y,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event cos(sycl::queue & exec_queue,
               T const * a,
               oneapi::mkl::slice sa,
               T * y,
               oneapi::mkl::slice sy,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

cos supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU

T	Devices supported
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
---	-------------------------------

return value (event) Computation end event.

Examples

An example of how to use `cos` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vcos.cpp
```

sin

Computes the element-wise sine of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `sin(a)` function computes sine of vector elements.

Note that arguments $\text{abs}(a[i]) \leq 213$ and $\text{abs}(a[i]) \leq 216$ for single and double precisions respectively are called fast computational path. These are trigonometric function arguments for which VM provides the best possible performance. Avoid arguments that do not belong to the fast computational path in the VM High Accuracy (HA) and Low Accuracy (LA) functions. Alternatively, you can use VM Enhanced Performance (EP) functions that are fast on the entire function domain. However, these functions provide less accuracy.

Argument	Result	VM Error Status
+0	+0	
-0	-0	
$+\infty$	QNAN	<code>status::errdom</code>
$-\infty$	QNAN	<code>status::errdom</code>
QNAN	QNAN	
SNAN	QNAN	

Specifications for special values of the complex functions are defined according to the following formula:

$$\text{Sin}(z) = -i * \text{Sinh}(i * z).$$

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event sin(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```



```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event sin(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event sin(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event sin(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

sin supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue The queue where the routine will be executed.

<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event

Examples

An example of how to use `sin` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vsin.cpp
```

sincos

Computes the element-wise sine and cosine of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `sincos(a)` function computes sine and cosine of vector elements.

Note that arguments $\text{abs}(a[i]) \leq 2^{13}$ and $\text{abs}(a[i]) \leq 2^{16}$ for single and double precisions respectively are called fast computational path. These are trigonometric function arguments for which VM provides the best possible performance. Avoid arguments that do not belong to the fast computational path in the VM High Accuracy (HA) and Low Accuracy (LA) functions. Alternatively, you can use VM Enhanced Performance (EP) functions that are fast on the entire function domain. However, these functions provide less accuracy.

Argument	Result 1	Result 2	Error Code
+0	+0	+1	
-0	-0	+1	
$+\infty$	QNAN	QNAN	<code>status::errdom</code>
$-\infty$	QNAN	QNAN	<code>status::errdom</code>
QNAN	QNAN	QNAN	
SNAN	QNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event sincos(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    sycl::buffer<T> & z,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event sincos(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    sycl::buffer<T> & z,
    oneapi::mkl::slice sz,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event sincos(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    T * z,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

```

namespace oneapi::mkl::vm {

sycl::event sincos(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    T * z,
    oneapi::mkl::slice sz,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

sincos supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.
sz	Slice selector for z. See Data Types for a description of the oneMKL slice type.

mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
sz	Slice selector for <i>z</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output sine vector.
z	The buffer containing the output cosine vector.
return value (event)	Computation end event

USM API

y	Pointer to the output sine vector.
z	Pointer to the output cosine vector.
return value (event)	Computation end event

Examples

An example of how to use `sincos` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vsincos.cpp
```

cis

Computes the element-wise complex exponential of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `cis(a)` function computes complex exponent of real vector elements (cosine and sine of real vector elements combined to complex value).

Argument	Result	Error Code
• 0	+1+i·0	
• 0	+1-i·0	
+∞	QNAN+i·QNAN	status::errdom
-∞	QNAN+i·QNAN	status::errdom
QNAN	QNAN+i·QNAN	
SNAN	QNAN+i·QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event cis(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event cis(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event cis(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {}),
```

```

    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event cis(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`cis` supports the following precisions and devices:

T	R	Devices supported
float	<code>std::complex<float></code>	CPU and GPU
double	<code>std::complex<double></code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.

depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `cis` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vcis.cpp
```

tan

Computes the element-wise tangent of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `tan(a)` function computes tangent of vector elements.

Note that arguments $\text{abs}(a[i]) \leq 213$ and $\text{abs}(a[i]) \leq 216$ for single and double precisions respectively are called fast computational path. These are trigonometric function arguments for which VM provides the best possible performance. Avoid arguments that do not belong to the fast computational path in the VM High Accuracy (HA) and Low Accuracy (LA) functions. Alternatively, you can use VM Enhanced Performance (EP) functions that are fast on the entire function domain. However, these functions provide less accuracy.

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	QNAN	<code>status::errdom</code>
$-\infty$	QNAN	<code>status::errdom</code>

Argument	Result	Error Code
QNaN	QNaN	
SNAN	QNaN	

Specifications for special values of the complex functions are defined according to the following formula:

$$\text{Tan}(z) = -i * \text{Tanh}(i * z)$$

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event tan(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event tan(sycl::queue & exec_queue,
               sycl::buffer<T> & a,
               oneapi::mkl::slice sa,
               sycl::buffer<T> & y,
               oneapi::mkl::slice sy,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event tan(sycl::queue & exec_queue,
               std::int64_t n,
               T const * a,
               T * y,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event tan(sycl::queue & exec_queue,
               T const * a,
               oneapi::mkl::slice sa,
               T * y,
               oneapi::mkl::slice sy,
               std::vector<sycl::event> const & depends = {},

}
```

```

oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

tan supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event

USM API

y	Pointer to the output vector.
return value (event)	Computation end event

Examples

An example of how to use `tan` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vtan.cpp
```

acos

Computes the element-wise arccosine of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `acos(a)` function computes inverse cosine of vector elements.

Argument		Result		Error Code	
+0		$+\pi/2$			
-0		$+\pi/2$			
+1		+0			
-1		$+\pi$			
$ a > 1$		QNAN		<code>status::errdom</code>	
$+\infty$		QNAN		<code>status::errdom</code>	
$-\infty$		QNAN		<code>status::errdom</code>	
QNAN		QNAN			
SNAN		QNAN			

RE(a) i·IM(a)	$-\infty$	-X	-0	+0	+X	$+\infty$	NAN
$+i \cdot \infty$	$+3 \cdot \pi/4 - i \cdot \infty$	$+\pi/2 - i \cdot \infty$	$+\pi/2 - i \cdot \infty$	$+\pi/2 - i \cdot \infty$	$+\pi/2 - i \cdot \infty$	$+\pi/4 - i \cdot \infty$	QNAN- $i \cdot \infty$
$+i \cdot Y$	$+\pi - i \cdot \infty$					$+0 - i \cdot \infty$	QNAN + $i \cdot QNAN$

RE(a) i·IM(a)	$-\infty$	-X	-0	+0	+X	$+\infty$	NAN
+i·0	$+\pi-i\cdot\infty$		$+\pi/2-i\cdot 0$	$+\pi/2-i\cdot 0$		$+0-i\cdot\infty$	$QNAN+i\cdot QNAN$
-i·0	$+\pi+i\cdot\infty$		$+\pi/2+i\cdot 0$	$+\pi/2+i\cdot 0$		$+0+i\cdot$ <i>inf</i> ty	$QNAN+i\cdot QNAN$
-i·Y	$+\pi+i\cdot\infty$					$+0+i\cdot\infty$	$QNAN+i\cdot QNAN$
-i· ∞	$+3\pi/4+i\cdot$ ∞	$+\pi/2+i\cdot$ ∞	$+\pi/2+i\cdot$ ∞	$+\pi/2+i\cdot$ ∞	$+\pi/2+i\cdot$ ∞	$+\pi/4+i\cdot$ ∞	$QNAN+i\cdot\infty$
$+i\cdot NAN$	$QNAN+i\cdot QNAN$	$QNAN+i\cdot QNAN$	$QNAN+i\cdot QNAN$	$QNAN+i\cdot QNAN$	$QNAN+i\cdot QNAN$	$QNAN+i\cdot QNAN$	$QNAN+i\cdot QNAN$

NOTE $\text{acos}(\text{CONJ}(a)) = \text{CONJ}(\text{acos}(a))$.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event acos(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event acos(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event acos(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {}),

}
```

```

    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event acos(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`acos` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.

sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `acos` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vacos.cpp
```

asin

Computes the element-wise arcsine of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `asin(a)` function computes inverse sine of vector elements.

Argument	Result	Error Code
+0	+0	
-0	-0	
+1	$+\pi/2$	
-1	$-\pi/2$	
$ a > 1$	QNaN	<code>status::errdom</code>
$+\infty$	QNaN	<code>status::errdom</code>

Argument	Result	Error Code
$-\infty$	QNAN	status::errdom
QNAN	QNAN	
SNAN	QNAN	

Specifications for special values of the complex functions are defined according to the following formula:

$$\text{asin}(a) = -i * \text{asinh}(i * z).$$

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event asin(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event asin(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event asin(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event asin(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
```

```

oneapi::mkl::slice sy,
std::vector<sycl::event> const & depends = {},
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

asin supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `asin` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vasin.cpp
```

atan

Computes the element-wise arctangent of vector elements in $[-\pi/2, \pi/2]$.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `atan(a)` function computes inverse tangent of vector elements.

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	$+\pi/2$	
$-\infty$	$-\pi/2$	
QNAN	QNAN	
SNAN	QNAN	

The `atan` function does not generate any errors.

Specifications for special values of the complex functions are defined according to the following formula

$$\text{atan}(a) = -i \cdot \text{atanh}(i \cdot a).$$

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event atan(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

namespace oneapi::mkl::vm {

sycl::event atan(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event atan(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

namespace oneapi::mkl::vm {

sycl::event atan(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}
```

atan supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `atan` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vatan.cpp
```

atan2

Computes the element-wise four-quadrant arctangent of vector `a` elements divided by vector `b` elements.

- [Description](#)
- [API](#)

- Examples

Description

The `atan2(a,b)` function computes four-quadrant inverse tangent of elements of two vectors.

The elements of the output vector y are computed as the four-quadrant arctangent of $a[i] / b[i]$.

Argument 1	Argument 2	Result	Error Code
$-\infty$	$-\infty$	$-3\pi/4$	
$-\infty$	$b < +0$	$-\pi/2$	
$-\infty$	-0	$-\pi/2$	
$-\infty$	$+0$	$-\pi/2$	
$-\infty$	$b > +0$	$-\pi/2$	
$-\infty$	$+\infty$	$-\pi/4$	
$a < +0$	$-\infty$	$-\pi$	
$a < +0$	-0	$-\pi/2$	
$a < +0$	$+0$	$-\pi/2$	
$a < +0$	$+\infty$	-0	
-0	$-\infty$	$-\pi$	
-0	$b < +0$	$-\pi$	
-0	-0	$-\pi$	
-0	$+0$	-0	
-0	$b > +0$	-0	
-0	$+\infty$	-0	
$+0$	$-\infty$	$+\pi$	
$+0$	$b < +0$	$+\pi$	
$+0$	-0	$+\pi$	
$+0$	$+0$	$+0$	
$+0$	$b > +0$	$+0$	
$+0$	$+\infty$	$+0$	
$a > +0$	$-\infty$	$+\pi$	
$a > +0$	-0	$+\pi/2$	
$a > +0$	$+0$	$+\pi/2$	
$a > +0$	$+\infty$	$+0$	
$+\infty$	$-\infty$	$+3\pi/4$	

Argument 1	Argument 2	Result	Error Code
$+\infty$	$b < +0$	$+\pi/2$	
$+\infty$	-0	$+\pi/2$	
$+\infty$	$+0$	$+\pi/2$	
$+\infty$	$b > +0$	$+\pi/2$	
$+\infty$	$+\infty$	$+\pi/4$	
$a > +0$	QNAN	QNAN	
$a > +0$	SNAN	QNAN	
QNAN	$b > +0$	QNAN	
SNAN	$b > +0$	QNAN	
QNAN	QNAN	QNAN	
QNAN	SNAN	QNAN	
SNAN	QNAN	QNAN	
SNAN	SNAN	QNAN	

The `atan2(a,b)` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event atan2(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event atan2(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event atan2(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event atan2(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

atan2 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Description

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the 1st input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
b	The buffer containing the 2nd input vector.
sb	Slice selector for b. See Data Types for a description of the oneMKL slice type.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.

`mode` Overrides the global VM mode setting for this function call. See [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

USM API

`exec_queue` The queue where the routine will be executed.

`n` Specifies the number of elements to be calculated.

`a` Pointer to the 1st input vector.

`sa` Slice selector for `a`. See [Data Types](#) for a description of the oneMKL slice type.

`b` Pointer to the 2nd input vector.

`sb` Slice selector for `b`. See [Data Types](#) for a description of the oneMKL slice type.

`sy` Slice selector for `y`. See [Data Types](#) for a description of the oneMKL slice type.

`depends` Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

`mode` Overrides the global VM mode setting for this function call. See the [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

Output Parameters

Buffer API

`y` The buffer containing the output vector.

return value (event) Computation end event.

USM API

`y` Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `atan2` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vatan2.cpp
```

cospi

Computes the element-wise cosine of vector elements multiplied by π .

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `cospi(a)` function computes the cosine of vector elements multiplied by π . For an argument `a`, the function computes $\cos(\pi * a)$.

Argument	Result	Error Code
+0	+1	
-0	+1	
$n + 0.5$, for any integer n where $n + 0.5$ is representable	+0	
$+\infty$	QNAN	status::errdom
$-\infty$	QNAN	status::errdom
QNAN	QNAN	
SNAN	QNAN	

If arguments $\text{abs}(a_i) \leq 2^{22}$ for single precision or $\text{abs}(a_i) \leq 2^{51}$ for double precision, they belong to the *fast computational path*: arguments for which VM provides the best possible performance. Avoid arguments which do not belong to the fast computational path in VM High Accuracy (HA) or Low Accuracy (LA) functions. For arguments which do not belong to the fast computational path you can use VM Enhanced Performance (EP) functions, which are fast on the entire function domain. However, these functions provide lower accuracy.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event cospi(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event cospi(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event cospi(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
```



```

    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event cospi(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`cospi` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.

sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `cospi` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vcospi.cpp
```

sinpi

Computes the element-wise sine of vector elements multiplied by π .

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `sinpi(a)` function computes the sine of vector elements multiplied by π . For an argument *a*, the function computes $\sin(\pi * a)$.

Argument	Result	Error Code
+0	+0	
-0	-0	
+n, positive integer	+0	
-n, negative integer	-0	
$+\infty$	QNAN	<code>status::errdom</code>

Argument	Result	Error Code
$-\infty$	QNAN	status::errdom
QNAN	QNAN	
SNAN	QNAN	

If arguments $\text{abs}(a_i) \leq 2^{22}$ for single precision or $\text{abs}(a_i) \leq 2^{51}$ for double precision, they belong to the *fast computational path*: arguments for which VM provides the best possible performance. Avoid arguments which do not belong to the fast computational path in VM High Accuracy (HA) or Low Accuracy (LA) functions. For arguments which do not belong to the fast computational path you can use VM Enhanced Performance (EP) functions, which are fast on the entire function domain. However, these functions provide lower accuracy.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event sinpi(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event sinpi(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event sinpi(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event sinpi(sycl::queue & exec_queue,
```

```

T const * a,
oneapi::mkl::slice sa,
T * y,
oneapi::mkl::slice sy,
std::vector<sycl::event> const & depends = {},
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

`sinpi` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event

USM API

y	Pointer to the output vector.
return value (event)	Computation end event

Examples

An example of how to use `sinpi` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vsinpi.cpp
```

tanpi

Computes the element-wise tangent of vector elements multiplied by pi.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `tanpi(a)` function computes the tangent of vector elements multiplied by π . For an argument `a`, the function computes $\tan(\pi * a)$.

Argument	Result	Error Code
+0	+0	
-0	+0	
n, even integer	*copysign(0.0, n)	
n, odd integer	*copysign(0.0, -n)	
n + 0.5, for n even integer and n + 0.5 representable	$+\infty$	
n + 0.5, for n odd integer and n + 0.5 representable	$-\infty$	
$+\infty$	QNAN	<code>status::errdom</code>

Argument	Result	Error Code
$-\infty$	QNAN	<code>status::errdom</code>
QNAN	QNAN	
SNAN	QNAN	

The `copysign(`x, y)`` function returns the first vector argument `x` with the sign changed to match that of the second argument `y`.

If arguments $\text{abs}(a_i) \leq 2^{13}$ for single precision or $\text{abs}(a_i) \leq 2^{67}$ for double precision, they belong to the *fast computational path*: arguments for which VM provides the best possible performance. Avoid arguments which do not belong to the fast computational path in VM High Accuracy (HA) or Low Accuracy (LA) functions. For arguments which do not belong to the fast computational path you can use VM Enhanced Performance (EP) functions, which are fast on the entire function domain. However, these functions provide lower accuracy.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event tanpi(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event tanpi(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event tanpi(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event tanpi(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`tanpi` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.

<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event

Examples

An example of how to use `tanpi` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vtanpi.cpp
```

acospi

Computes the element-wise arc cosine of vector elements, divided by π .

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `acospi(a)` function computes the inverse cosine of vector elements divided by π . For an argument `a`, the function computes $\text{acos}(a)/\pi$.

Argument	Result	Error Code
+0	+1/2	
-0	+1/2	
+1	+0	
-1	+1	
$ a > 1$	QNAN	<code>status::errdom</code>
$+\infty$	QNAN	<code>status::errdom</code>

Argument	Result	Error Code
$-\infty$	QNAN	status::errrdom
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event acospi(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event acospi(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event acospi(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event acospi(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```
oneapi::mkl::vm::error_handler<T> errhandler = {});
}
```

acosp_i supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer <code>y</code> to the output vector of size <code>n</code> .
return value (event)	Computation end event.

Examples

An example of how to use `acospi` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vacospi.cpp
```

asinpi

Computes the element-wise arcsine of vector elements, divided by π .

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `asinpi(a)` function computes the inverse sine of vector elements divided by π . For an argument `a`, the function computes `asinpi(a)/ $\pi$` .

Argument	Result	Error Code
+0	+0	
-0	-0	
+1	+1/2	
-1	-1/2	
$ a > 1$	QNAN	<code>status::errdom</code>
$+\infty$	QNAN	<code>status::errdom</code>
$-\infty$	QNAN	<code>status::errdom</code>
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event asinpi(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event asinpi(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event asinpi(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event asinpi(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

asinpi supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU

T	Devices supported
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
---	-------------------------------

return value (event) Computation end event.

Examples

An example of how to use `asinpi` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vasinpi.cpp
```

atanpi

Computes the element-wise arctangent of vector elements, divided by pi.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `atanpi(a)` function computes the inverse tangent of vector elements divided by π . For an argument `a`, the function computes $\text{atan}(a)/\pi$.

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	$+1/2$	
$-\infty$	$-1/2$	
QNAN	QNAN	
SNAN	QNAN	

The `atanpi` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event atanpi(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event atanpi(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
```

```

    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event atanpi(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

```

namespace oneapi::mkl::vm {

sycl::event atanpi(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

atanpi supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `atanpi` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vatanpi.cpp
```

atan2pi

Computes the element-wise four-quadrant arctangent of vector `a` elements divided by vector `b` elements, divided by π .

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `atan2pi(a,b)` function computes the four-quadrant inverse tangent of the ratios of the corresponding elements of two vectors divided by ∞ .

For the elements of the output vector `y`, the function computes the four-quadrant arctangent of a_i/b_i , with the result divided by ∞ .

Argument 1	Argument 2	Result	Error Code
$-\infty$	$-\infty$	$-3/4$	
$-\infty$	$b < +0$	$-1/2$	

Argument 1	Argument 2	Result	Error Code
$-\infty$	-0	+1/2	
$-\infty$	+0	-1/2	
$-\infty$	$x > +0$	-1/2	
$-\infty$	$+\infty$	-1/4	
$a < +0$	$-\infty$	-1	
$a < +0$	-0	-1/2	
$a < +0$	+0	-1/2	
$a < +0$	$+\infty$	-0	
-0	$-\infty$	-1	
-0	$b < +0$	-1	
-0	-0	-1	
-0	+0	-0	
-0	$b > +0$	-0	
-0	$+\infty$	-0	
+0	$-\infty$	+1	
+0	$b < +0$	+1	
+0	-0	+1	
+0	+0	+0	
+0	$b > +0$	+0	
+0	$+\infty$	+0	
$a > +0$	$-\infty$	+1	
$a > +0$	-0	+1/2	
$x > +0$	+0	+1/2	
$a > +0$	$+\infty$	+1/4	
$+\infty$	$-\infty$	+3/4	
$+\infty$	$b < +0$	+1/2	
$+\infty$	-0	+1/2	
$+\infty$	+0	+1/2	
$+\infty$	$b > +0$	+1/2	
$+\infty$	$+\infty$	+1/4	

Argument 1	Argument 2	Result	Error Code
a > +0	QNAN	QNAN	
a > +0	SNAN	QNAN	
QNAN	b > +0	QNAN	
SNAN	x > +0	QNAN	
QNAN	QNAN	QNAN	
QNAN	SNAN	QNAN	
SNAN	QNAN	QNAN	
SNAN	SNAN	QNAN	

The `atan2pi(a,b)` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event atan2pi(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event atan2pi(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event atan2pi(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {}),
```

```

    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

namespace oneapi::mkl::vm {

sycl::event atan2pi(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

`atan2pi` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.

<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `atan2pi` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vatan2pi.cpp
```

cosd

Computes the element-wise cosine of vector elements expressed in degrees.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `cosd(a)` function is a degree argument trigonometric function. It computes the cosine of vector elements multiplied by $\pi/180$. For an argument a , the function computes $\cos(\pi*a/180)$.

Note that arguments $\text{abs}(a_i) \leq 2^{24}$ for single precision or $\text{abs}(a_i) \leq 2^{52}$ for double precision, they belong to the *fast computational path*: trigonometric function arguments for which VM provides the best possible performance. Avoid arguments which do not belong to the fast computational path in VM High Accuracy (HA) or Low Accuracy (LA) functions. For arguments which do not belong to the fast computational path you can use VM Enhanced Performance (EP) functions, which are fast on the entire function domain. However, these functions provide lower accuracy.

Argument	Result	Error Code
+0	+1	
-0	+1	

Argument	Result	Error Code
$+\infty$	QNAN	status::errdom
$-\infty$	QNAN	status::errdom
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event cosd(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event cosd(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event cosd(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event cosd(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
```

```
std::vector<sycl::event> const & depends = {},
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
oneapi::mkl::vm::error_handler<T> errhandler = {});
}
```

cosd supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `cosd` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vcosd.cpp
```

sind

Computes the element-wise sine of vector elements expressed in degrees.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `sind(a)` function is a degree argument trigonometric function. It computes the sine of vector elements multiplied by $\pi/180$. For an argument a , the function computes $\sin(\pi*a/180)$.

Note that arguments $\text{abs}(a_i) \leq 2^{24}$ for single precision or $\text{abs}(a_i) \leq 2^{52}$ for double precision, they belong to the *fast computational path*: trigonometric function arguments for which VM provides the best possible performance. Avoid arguments which do not belong to the fast computational path in VM High Accuracy (HA) or Low Accuracy (LA) functions. For arguments which do not belong to the fast computational path you can use VM Enhanced Performance (EP) functions, which are fast on the entire function domain. However, these functions provide lower accuracy.

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	QNAN	<code>status::errdom</code>
$-\infty$	QNAN	<code>status::errdom</code>
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event sind(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event sind(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event sind(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event sind(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

sind supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU

T	Devices supported
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event

USM API

y

Pointer to the output vector.

return value (event)

Computation end event

Examples

An example of how to use `sind` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vsind.cpp
```

tand

Computes the element-wise tangent of vector elements expressed in degrees.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `tand(a)` function computes the tangent of vector elements multiplied by $\pi/180$. For an argument `x`, the function computes $\tan(\pi*x/180)$.

Note that arguments $\text{abs}(a_i) \leq 2^{38}$ for single precision or $\text{abs}(a_i) \leq 2^{67}$ for double precision, they belong to the *fast computational path*: trigonometric function arguments for which VM provides the best possible performance. Avoid arguments with do not belong to the fast computational path in VM High Accuracy (HA) or Low Accuracy (LA) functions. For arguments which do not belong to the fast computational path you can use VM Enhanced Performance (EP) functions, which are fast on the entire function domain. However, these functions provide lower accuracy.

Argument	Result	Error Code
+0	+1	
-0	+1	
$\pm\infty$	QNAN	<code>status::errdom</code>
$\pm\infty$	QNAN	<code>status::errdom</code>
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```

namespace oneapi::mkl::vm {

sycl::event tand(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,

```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event tand(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event tand(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event tand(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

tand supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue The queue where the routine will be executed.

<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event

Examples

An example of how to use `tand` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vtand.cpp
```

Hyperbolic Functions

$$\begin{aligned}
 & -i \cdot \infty \quad + \infty + i \cdot \text{QNAN} + \text{QNAN} \text{QNAN} - \text{QNAN} + i \cdot \text{QNANQNAN} + i \cdot \text{QNAN} \\
 & + i \cdot \text{NAN} + \infty + i \cdot \text{QNAN} + \text{QNAN} \text{QNAN} \text{QNAN} \text{QNAN} + i \cdot \text{QNANQNAN} + i \cdot \text{QNAN}
 \end{aligned}$$

NOTE

- The complex `cosh(a)` function sets the VM Error Status to `status::overflow` in the case of overflow, that is, when `RE(a)`, `IM(a)` are finite non-zero numbers, but the real or imaginary part of the exact result is so large that it does not meet the target precision.
- `cosh(CONJ(a)) = CONJ(cosh(a))`
- `cosh(-a) = cosh(a)`.

API**Syntax****Buffer API**

```
namespace oneapi::mkl::vm {

sycl::event cosh(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event cosh(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event cosh(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event cosh(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

cosh supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.

sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `cosh` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vcosh.cpp
```

sinh

Computes the element-wise hyperbolic sine of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `sinh(a)` function computes hyperbolic sine of vector elements.

Data Type	Threshold Limitations on Input Parameters
half precision	$-\text{Log}(\text{FLT16_MAX}) - \text{Log}(2) < a[i] < \text{Log}(\text{FLT16_MAX}) + \text{Log}(2)$
single precision	$-\text{Log}(\text{FLT_MAX}) - \text{Log}(2) < a[i] < \text{Log}(\text{FLT_MAX}) + \text{Log}(2)$
double precision	$-\text{Log}(\text{DBL_MAX}) - \text{Log}(2) < a[i] < \text{Log}(\text{DBL_MAX}) + \text{Log}(2)$

Argument	Result	Error Code
+0	+0	
-0	-0	
a > overflow	$+\infty$	status::overflow
a < -overflow	$-\infty$	status::overflow
$+\infty$	$+\infty$	
$-\infty$	$-\infty$	
QNAN	QNAN	
SNAN	QNAN	

$+i \cdot \infty$	$-\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$
$+i \cdot Y$	$-\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$
$+i \cdot 0$	$-\infty + i \cdot 0$	$+\infty + i \cdot 0$	$+\infty + i \cdot 0$
$-i \cdot 0$	$-\infty - i \cdot 0$	$+\infty - i \cdot 0$	$+\infty - i \cdot 0$
$-i \cdot Y$	$-\infty - i \cdot \text{QNAN}$	$+\infty - i \cdot \text{QNAN}$	$+\infty - i \cdot \text{QNAN}$
$-i \cdot \infty$	$-\infty - i \cdot \text{QNAN}$	$+\infty - i \cdot \text{QNAN}$	$+\infty - i \cdot \text{QNAN}$
$+i \cdot \text{NAN}$	$-\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$

NOTE

- The complex `sinh(a)` function sets the VM Error Status to `status::overflow` in the case of overflow, that is, when `RE(a)`, `IM(a)` are finite non-zero numbers, but the real or imaginary part of the exact result is so large that it does not meet the target precision.
- `sinh(CONJ(a)) = CONJ(sinh(a))`
- `sinh(-a) = -sinh(a)`.

API**Syntax****Buffer API**

```
namespace oneapi::mkl::vm {
    sycl::event sinh(sycl::queue & exec_queue,
        std::int64_t n,
        sycl::buffer<T> & a,
```

```

    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event sinh(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event sinh(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event sinh(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`sinh` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters**Buffer API**

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `sinh` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vsinh.cpp
```

tanh

Computes the element-wise hyperbolic tangent of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `tanh(a)` function computes hyperbolic tangent of vector elements.

Argument	Result	Erro Code
+0	+0	
-0	-0	
$+\infty$	+1	
$-\infty$	-1	
QNAN	QNAN	
SNAN	QNAN	

$+i \cdot \infty$	$-1 + i \cdot 0$	QNAN	+1	QNAN	$+i \cdot \text{QNAN}$
$+i \cdot Y$	$-1 + i \cdot 0 \cdot \text{Tan}(Y)$	QNAN	+1	$+i \cdot \text{QNAN}$	$+i \cdot \text{QNAN}$
$+i \cdot 0$	$-1 + i \cdot 0$	$-0 + i \cdot 0$	+1	QNAN	$+i \cdot 0$
$-i \cdot 0$	$-1 - i \cdot 0$	$-0 - i \cdot 0$	+1	QNAN	$-i \cdot 0$
$-i \cdot Y$	$-1 + i \cdot 0 \cdot \text{Tan}(Y)$	QNAN	+1	$+i \cdot \text{QNAN}$	$+i \cdot \text{QNAN}$
$-i \cdot \infty$	$-1 - i \cdot 0$	QNAN	+1	QNAN	$+i \cdot \text{QNAN}$
$+i \cdot \text{NAN}$	$-1 + i \cdot 0$	QNAN	+1	QNAN	$+i \cdot \text{QNAN}$

NOTE

- `tanh(CONJ(a))=CONJ(tanh(a))`
- `tanh(-a)=-tanh(a)`.

The `tanh(a)` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event tanh(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event tanh(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy, oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event tanh(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event tanh(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

tanh supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `tanh` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vtanh.cpp
```

acosh

Computes the element-wise inverse hyperbolic cosine of vector elements.

- [Description](#)
- [API](#)

- Examples

Description

The `acosh(a)` function computes inverse hyperbolic cosine (nonnegative) of vector elements.

Argument	Result	Error Code
+1	+0	
$a < +1$	QNAN	status::errdom
$-\infty$	QNAN	status::errdom
$+\infty$	$+\infty$	
QNAN	QNAN	
SNAN	QNAN	

RE(a) i-IM(a)	$-\infty$	$-X$	-0	$+0$	$+X$	$+\infty$	NAN
$+i \cdot \infty$	$+\infty + i \cdot \frac{3\pi}{4}$	$+\infty + i \cdot \frac{\pi}{2}$	$+\infty + i \cdot \frac{\pi}{2}$	$+\infty + i \cdot \frac{\pi}{2}$	$+\infty + i \cdot \frac{\pi}{2}$	$+\infty + i \cdot \frac{\pi}{2}$	$+\infty + i \cdot \frac{\pi}{4}$
$+i \cdot Y$	$+\infty + i \cdot \pi$					$+\infty + i \cdot 0$	QNAN
$+i \cdot 0$	$+\infty + i \cdot \pi$		$+0 + i \cdot \frac{\pi}{2}$	$+0 + i \cdot \frac{\pi}{2}$		$+\infty + i \cdot 0$	QNAN
$-i \cdot 0$	$+\infty + i \cdot \pi$		$+0 + i \cdot \frac{\pi}{2}$	$+0 + i \cdot \frac{\pi}{2}$		$+\infty + i \cdot 0$	QNAN
$-i \cdot Y$	$+\infty + i \cdot \pi$					$+\infty + i \cdot 0$	QNAN
$-i \cdot \infty$	$+\infty - i \cdot \frac{3\pi}{4}$	$+\infty - i \cdot \frac{\pi}{2}$	$+\infty - i \cdot \frac{\pi}{2}$	$+\infty - i \cdot \frac{\pi}{2}$	$+\infty - i \cdot \frac{\pi}{2}$	$+\infty - i \cdot \frac{\pi}{2}$	$+\infty - i \cdot \frac{\pi}{4}$
$+i \cdot \text{NAN}$	$+\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$

NOTE `acosh(CONJ(a)) = CONJ(acosh(a))`.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event acosh(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event acosh(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event acosh(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event acosh(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

acosh supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters**Buffer API**

exec_queue The queue where the routine will be executed.

<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `acosh` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vacosh.cpp
```

asinh

Computes the element-wise inverse hyperbolic sine of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `asinh(a)` function computes inverse hyperbolic sine of vector elements.

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	$+\infty$	
$-\infty$	$-\infty$	
QNAN	QNAN	
SNAN	QNAN	

RE(a) i·IM(a)	$-\infty$	-X	-0	+0	+X	$+\infty$	NAN
$+i \cdot \infty$	$-\infty' + i \cdot \pi/4'$	$+i \cdot \pi/2'$	$+i \cdot \pi/2'$	$+i \cdot \pi/2'$	$+i \cdot \pi/2'$	$+i \cdot \pi/4'$	$+i \cdot \text{QNAN}$
$+i \cdot Y$	$-\infty + i \cdot 0$					$+\infty + i \cdot 0$	$\text{QNAN} + i \cdot \text{QNAN}$
$+i \cdot 0$	$+\infty + i \cdot 0$		$+0 + i \cdot 0$	$+0 + i \cdot 0$		$+\infty + i \cdot 0$	$\text{QNAN} + i \cdot \text{QNAN}$
$-i \cdot 0$	$-\infty - i \cdot 0$		$-0 - i \cdot 0$	$+0 - i \cdot 0$		$+\infty - i \cdot 0$	$\text{QNAN} - i \cdot \text{QNAN}$
$-i \cdot Y$	$-\infty - i \cdot 0$					$+\infty - i \cdot 0$	$\text{QNAN} + i \cdot \text{QNAN}$
$-i \cdot \infty$	$-\infty\text{-icdot}$ $\pi/4'$	$-\infty\text{-icdot}$ $\pi/2'$	$-\infty\text{-icdot}$ $\pi/2'$	$+\infty\text{-icdot}$ $\pi/2'$	$+\infty\text{-icdot}$ $\pi/2'$	$+\infty\text{-icdot}$ $\pi/4'$	$+\infty + i \cdot \text{QNAN}$
$+i \cdot \text{NAN}$	$-\infty + i \cdot \text{QNAN}$	$\text{QNAN} + i \cdot \text{QNAN}$	$\text{QNAN} + i \cdot \text{QNAN}$	$\text{QNAN} + i \cdot \text{QNAN}$	$\text{QNAN} + i \cdot \text{QNAN}$	$+\infty + i \cdot \text{QNAN}$	$\text{QNAN} + i \cdot \text{QNAN}$

The `asinh(a)` function does not generate any errors.

NOTE

- `asinh(CONJ(a))=CONJ(asinh(a))`
- `asinh(-a)=-asinh(a)`.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event asinh(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event asinh(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event asinh(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event asinh(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

`asinh` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `asinh` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vasinh.cpp
```

atanh

Computes the element-wise inverse hyperbolic tangent of vector elements.

- [Description](#)
- [API](#)

- Examples

Description

The `atanh(a)` function computes inverse hyperbolic tangent of vector elements.

Argument	Result	Error Code
+1	$+\infty$	<code>status::sing</code>
-1	$-\infty$	<code>status::sing</code>
$ a > 1$	QNAN	<code>status::errdom</code>
$-\infty$	QNAN	<code>status::errdom</code>
$+\infty$	$+\infty$	<code>status::errdom</code>
QNAN	QNAN	
SNAN	QNAN	

RE(a) i·IM(a)	$-\infty$	-X	-0	+0	+X	$+\infty$	NAN
$+i \cdot \infty$	$-0 + i \cdot \pi/2$	$0 + i \cdot \pi/2$	$0 + i \cdot \pi/2$	$0 + i \cdot \pi/2$	$0 + i \cdot \pi/2$	$0 + i \cdot \pi/2$	$0 + i \cdot \pi/2$
$+i \cdot Y$	$-0 + i \cdot \pi/2$					$0 + i \cdot \pi/2$	QNAN
$+i \cdot 0$	$-0 + i \cdot \pi/2$		$-0 + i \cdot 0$	$0 + i \cdot 0$		$0 + i \cdot \pi/2$	QNAN
$-i \cdot 0$	$-0 - i \cdot \pi/2$		$-0 - i \cdot 0$	$0 - i \cdot 0$		$0 - i \cdot \pi/2$	QNAN
$-i \cdot Y$	$-0 - i \cdot \pi/2$					$0 - i \cdot \pi/2$	QNAN
$-i \cdot \infty$	$-0 - i \cdot \pi/2$	$0 - i \cdot \pi/2$	$0 - i \cdot \pi/2$	$0 - i \cdot \pi/2$	$0 - i \cdot \pi/2$	$0 - i \cdot \pi/2$	$0 - i \cdot \pi/2$
$+i \cdot \text{NAN}$	$-0 + i \cdot \text{QNAN}$	$0 + i \cdot \text{QNAN}$	$0 + i \cdot \text{QNAN}$	$0 + i \cdot \text{QNAN}$	$0 + i \cdot \text{QNAN}$	$0 + i \cdot \text{QNAN}$	$0 + i \cdot \text{QNAN}$

NOTE

- `atanh(±1±i*0)=±math::`infty`±i*0`, and `status::sing` error is generated
- `atanh(CONJ(a))=CONJ(atanh(a))`
- `atanh(-a)=-atanh(a)`.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {
    sycl::event atanh(sycl::queue & exec_queue,
```

```

        std::int64_t n,
        sycl::buffer<T> & a,
        sycl::buffer<T> & y,
        oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
        oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event atanh(sycl::queue & exec_queue,
        sycl::buffer<T> & a,
        oneapi::mkl::slice sa,
        sycl::buffer<T> & y,
        oneapi::mkl::slice sy,
        oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
        oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event atanh(sycl::queue & exec_queue,
        std::int64_t n,
        T const * a,
        T * y,
        std::vector<sycl::event> const & depends = {},
        oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
        oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event atanh(sycl::queue & exec_queue,
        T const * a,
        oneapi::mkl::slice sa,
        T * y,
        oneapi::mkl::slice sy,
        std::vector<sycl::event> const & depends = {},
        oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
        oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

atanh supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `atanh` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vatanh.cpp
```

Special Functions

erf

Computes the element-wise error function of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `erf` function computes the error function values for elements of the input vector `a` and writes them to the output vector `y`.

The error function is defined as given by:

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

Useful relations:

$$\operatorname{erfc}(x) = 1 - \operatorname{erf}(x)$$

where `erfc` is the complementary error function.

$$\Phi(x) = \frac{1}{2} \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)$$

where

$$\Phi(x) = -\frac{1}{\sqrt{2\pi}} \int_0^x \exp(-\operatorname{fract}^2 2) dt$$

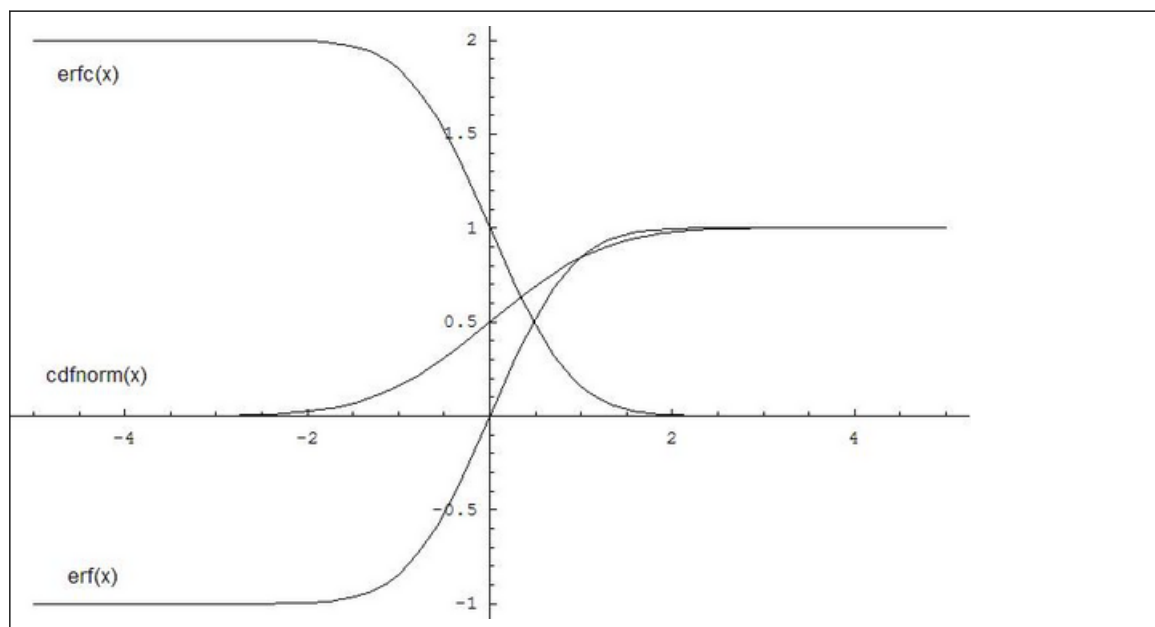
is the cumulative normal distribution function.

$$\Phi^{-1}(x) = \sqrt{2}\text{erf}^{-1}(2x - 1)$$

where $\phi^{-1}(x)$ and $\text{erf}^{-1}(x)$ are the inverses to $\phi(x)$ and $\text{erf}(x)$, respectively.

The following figure illustrates the relationships among erf family functions (erf, erfc, cdfnorm).

erf Family Functions Relationship



Useful relations for these functions:

$$\text{erf}(x) + \text{erfc} = 1$$

$$\text{cdfnorm}(x) = \frac{1}{2} \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right) = 1 - \frac{1}{2} \text{erfc}\left(\frac{x}{\sqrt{2}}\right)$$

Argument	Result	Error Code
$+\infty$	+1	
$-\infty$	-1	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event erf(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event erf(sycl::queue & exec_queue,
               sycl::buffer<T> & a,
               oneapi::mkl::slice sa,
               sycl::buffer<T> & y,
               oneapi::mkl::slice sy,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event erf(sycl::queue & exec_queue,
               std::int64_t n,
               T const * a,
               T * y,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event erf(sycl::queue & exec_queue,
               T const * a,
               oneapi::mkl::slice sa,
               T * y,
               oneapi::mkl::slice sy,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

erf supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `erf` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/verf.cpp
```

erfc

Computes the element-wise complementary error function of vector elements.

- [Description](#)
- [API](#)

- [Examples](#)

Description

The `erfc` function computes the complementary error function values for elements of the input vector `a` and writes them to the output vector `y`.

The complementary error function is defined as follows:

$$\operatorname{erfc}(x) = \frac{2}{\sqrt{\pi}} \int_x^{\infty} e^{-t^2} dt$$

Useful relations:

$$\operatorname{erfc}(x) = 1 - \operatorname{erf}(x)$$

$$\Phi(x) = \frac{1}{2} \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)$$

where

$$\Phi(x) = -\frac{1}{\sqrt{2\pi}} \int_0^x \exp(-\operatorname{fract}^2 2) dt$$

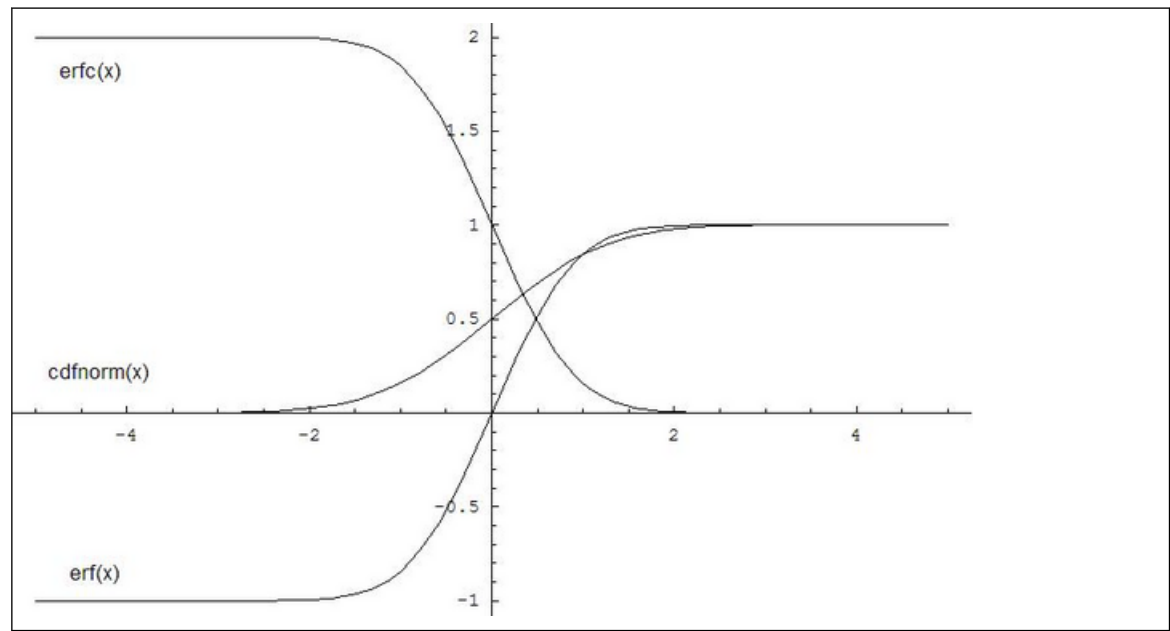
is the cumulative normal distribution function.

$$\Phi^{-1}(x) = \sqrt{2} \operatorname{erf}^{-1}(2x - 1)$$

where $\phi^{-1}(x)$ and $\operatorname{erf}^{-1}(x)$ are the inverses to $\phi(x)$ and $\operatorname{erf}(x)$, respectively.

The following figure illustrates the relationships among `erf` family functions (`erf`, `erfc`, `cdfnorm`).

erfc Family Functions Relationship



Useful relations for these functions:

$$\text{erf}(x) + \text{erfc} = 1$$
$$\text{cdfnorm}(x) = \frac{1}{2} \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right) = 1 - \frac{1}{2} \text{erfc}\left(\frac{x}{\sqrt{2}}\right)$$

Argument	Result	Error Code
a > underflow	+0	status::underflow
+∞	+0	
-∞	+2	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {  
  
sycl::event erfc(sycl::queue & exec_queue,  
    std::int64_t n,
```

```

    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event erfc(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event erfc(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event erfc(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

erfc supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the ref: create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `erfc` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/verfc.cpp
```

erfcx

Computes the element-wise scaled complementary error function of vector elements.

- [Description](#)
- [API](#)

Description

The `erfcx(a)` function computes the scaled complementary error function values for elements of the input vector `a` and writes them to the output vector `y`.

The scaled complementary error function is defined as given by:

$$\text{erfcx}(x) = \exp(x^2) \cdot \text{erfc}(x)$$

where $\exp(x)$ denotes the exponential function and $\text{erfc}(x)$ denotes the complementary error function.

Argument	Result	Error Code
$+\infty$	+0	
$-\infty$	$+\infty$	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event erfcx(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event erfcx(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```


USM API

```
namespace oneapi::mkl::vm {

sycl::event erfcx(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event erfcx(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

`erfcx` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters**Buffer API**

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the ref: create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer <code>y</code> containing the output vector of size <code>n</code> .
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer <code>y</code> to the output vector of size <code>n</code> .
return value (event)	Computation end event.

[cdfnorm](#)

Computes the element-wise cumulative normal distribution function of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `cdfnorm` function computes the cumulative normal distribution function values for elements of the input vector `a` and writes them to the output vector `y`.

The cumulative normal distribution function is defined as given by:

$$\text{CdfNorm}(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-t^2} dt$$

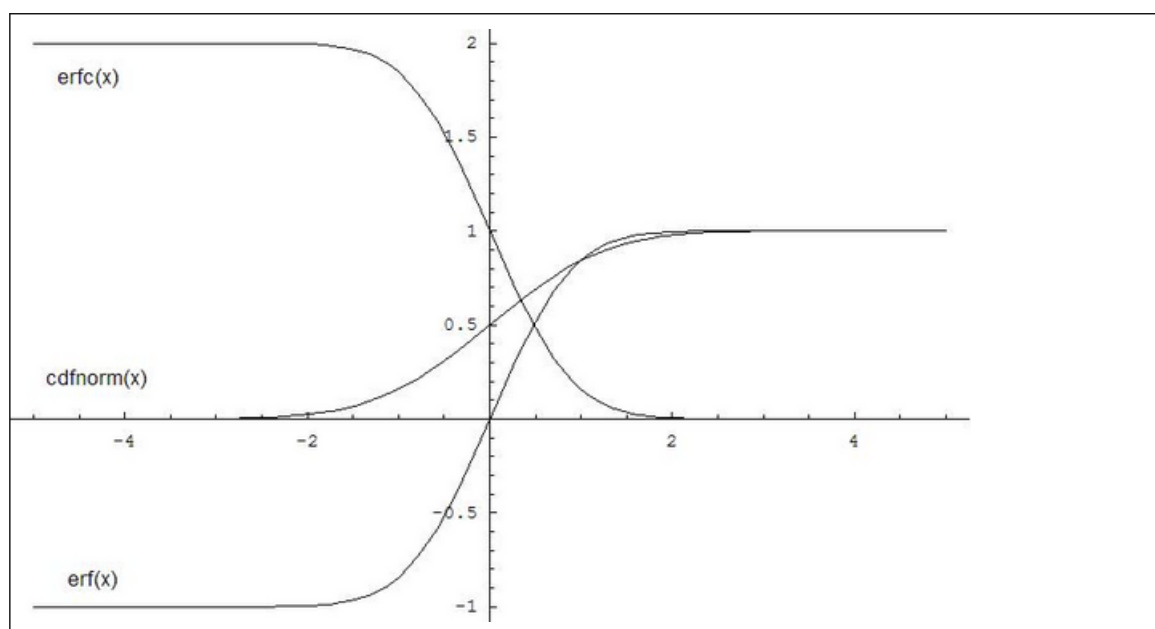
Useful relations:

$$\text{cdfnorm}(x) = \frac{1}{2} \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right) = 1 - \frac{1}{2} \text{erfc}\left(\frac{x}{\sqrt{2}}\right)$$

where `erf` and `erfc` are the error and complementary error functions.

The following figure illustrates the relationships among `erf` family functions (`erf`, `erfc`, `cdfnorm`).

cdfnorm Family Functions Relationship



Useful relations for these functions:

$$\text{erf}(x) + \text{erfc}(x) = 1$$

$$\text{cdfnorm}(x) = \frac{1}{2} \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right) = 1 - \frac{1}{2} \text{erfc}\left(\frac{x}{\sqrt{2}}\right)$$

Argument	Result	Error Code
$a < \text{underflow}$	+0	<code>status::underflow</code>
$+\infty$	+1	
$-\infty$	+0	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event cdfnorm(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event cdfnorm(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event cdfnorm(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event cdfnorm(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

`cdfnorm` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU

T	Devices supported
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
---	-------------------------------

return value (event) Computation end event.

Examples

An example of how to use `cdfnorm` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vcdfnorm.cpp
```

erfinv

Computes the element-wise inverse error function of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `erfinv(a)` function computes the inverse error function values for elements of the input vector `a` and writes them to the output vector `y`

`y = erf-1(a),`

where `erf(x)` is the error function defined as given by:

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

Useful relations:

$$\operatorname{erf}^{-1}(x) = \operatorname{erfc}^{-1}(1 - x)$$

where `erfc` is the complementary error function.

$$\Phi = \frac{1}{2} \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)$$

where

$$\Phi(x) = -\frac{1}{\sqrt{2\pi}} \int_0^x \exp(-\text{fract}^2 2) dt$$

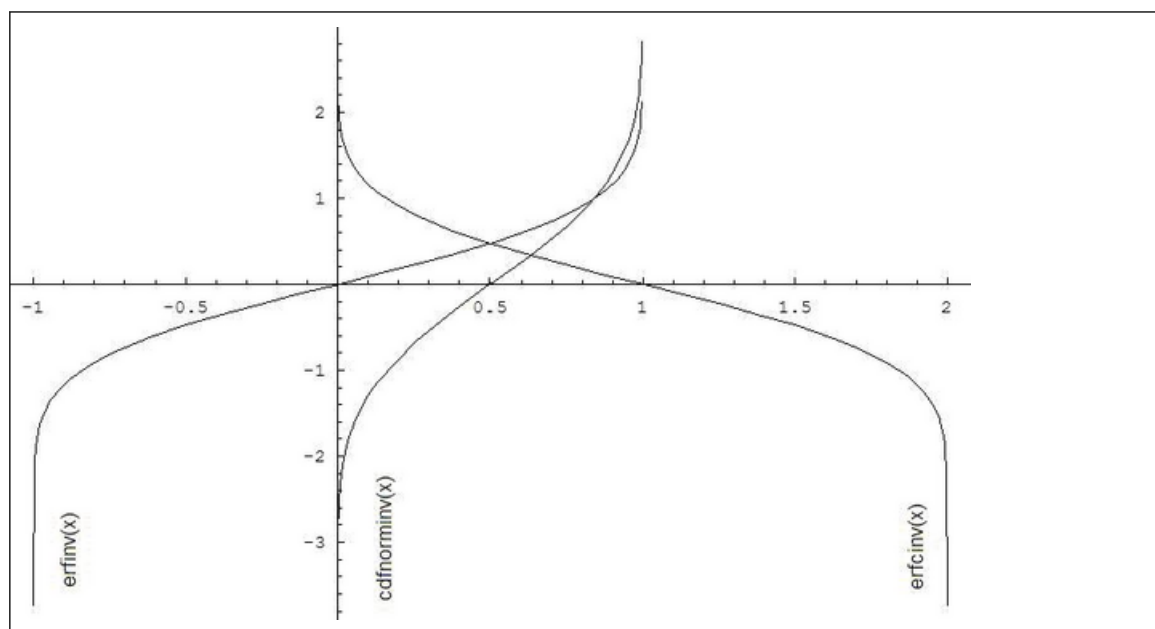
is the cumulative normal distribution function.

$$\Phi^{-1}(x) = \sqrt{2} \text{erf}^{-1}(2x - 1)$$

where $\phi^{-1}(x)$ and $\text{erf}^{-1}(x)$ are the inverses to $\phi(x)$ and $\text{erf}(x)$, respectively.

The following figure illustrates the relationships among `erfinv` family functions (`erfinv`, `erfcinv`, `cdfnorminv`).

erfinv Family Functions Relationship



Useful relations for these functions:

$$\text{erfcinv}(x) = \text{erfinv}(1 - x)$$

$$\text{cdfnorminv}(x) = \sqrt{2} \text{erfinv}(2x - 1) = \sqrt{2} \text{erfcinv}(2 - 2x)$$

Argument	Result	Error Code
+0	+0	
-0	-0	
+1	$+\infty$	<code>status::sing</code>

Argument	Result	Error Code
-1	$-\infty$	status::sing
$ a > 1$	QNAN	status::errdom
$+\infty$	QNAN	status::errdom
$-\infty$	QNAN	status::errdom
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```

namespace oneapi::mkl::vm {

sycl::event erfinv(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event erfinv(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event erfinv(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event erfinv(sycl::queue & exec_queue,
    T const * a,

```



```

oneapi::mkl::slice sa,
T * y,
oneapi::mkl::slice sy,
std::vector<sycl::event> const & depends = {},
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

`erfinv` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode Overrides the global VM mode setting for this function call. See the [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `erfcinv` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/verfcinv.cpp
```

erfcinv

Computes the element-wise inverse complementary error function of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `erfcinv(a)` function computes the inverse complementary error function values for elements of the input vector `a` and writes them to the output vector `y`.

The inverse complementary error function is defined as given by:

$$\text{erfcinv}(x) = \text{erfcinv}(1 - x)$$

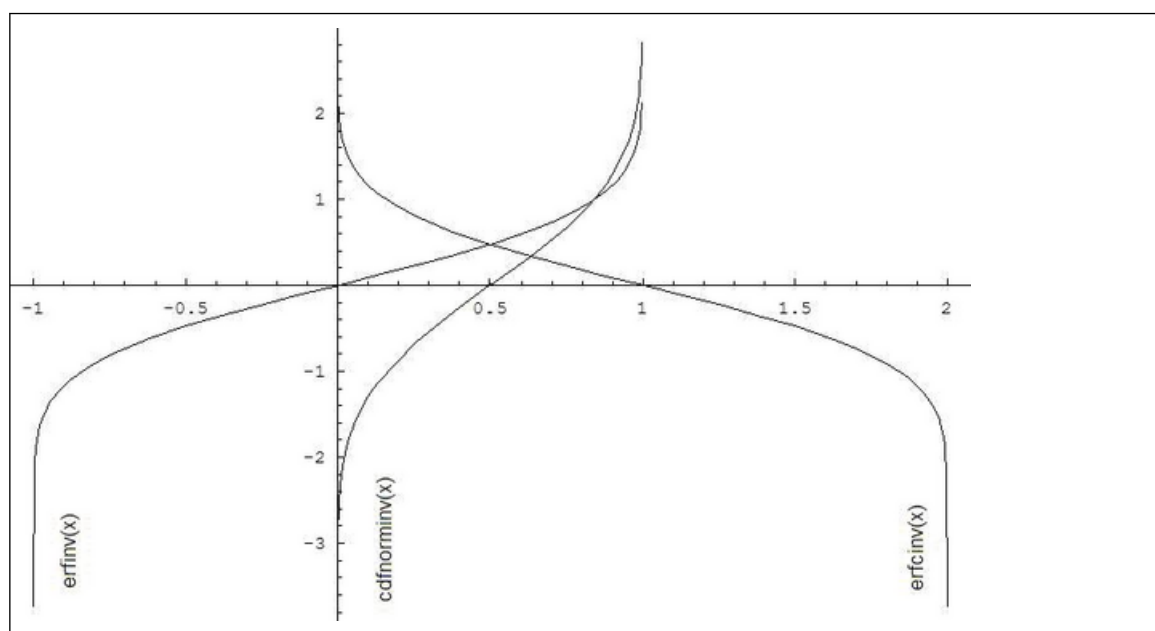
$$\text{erfcinv}(x) = \text{erf}^{-1}(x)$$

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

where $\operatorname{erf}(x)$ denotes the error function and $\operatorname{erfinv}(x)$ denotes the inverse error function.

The following figure illustrates the relationships among erfinv family functions (erfinv , $\operatorname{erfcinv}$, $\operatorname{cdfnorminv}$).

erfcinv Family Functions Relationship



Useful relations for these functions:

$$\operatorname{erfcinv}(x) = \operatorname{erfinv}(1 - x)$$

$$\operatorname{cdfnorminv}(x) = \sqrt{2}\operatorname{erfinv}(2x - 1) = \sqrt{2}\operatorname{erfcinv}(2 - 2x)$$

Argument	Result	Error Code
+1	+0	
+2	$-\infty$	status::sing
-0	$+\infty$	status::sing
+0	$+\infty$	status::sing
$a < -0$	QNAN	status::errdom

Argument	Result	Error Code
$a > +2$	QNAN	status::errdom
$+\infty$	QNAN	status::errdom
$-\infty$	QNAN	status::errdom
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event erfcinv(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event erfcinv(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event erfcinv(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event erfcinv(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
```

```

    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

`erfcinv` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

errhandler Sets local error handling mode for this function call. See the ref:**create_error_handler** function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer *y* containing the output vector of size *n*.

return value (event) Computation end event.

USM API

y Pointer *y* to the output vector of size *n*.

return value (event) Computation end event.

Examples

An example of how to use `erfcinv` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/verfcinv.cpp
```

cdfnorminv

Computes the element-wise inverse cumulative normal distribution function of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `cdfnorminv(a)` function computes the inverse cumulative normal distribution function values for elements of the input vector *a* and writes them to the output vector *y*.

The inverse cumulative normal distribution function is defined as given by:

$$\text{CdfNormInv}(x) = \text{CdfNorm}^{-1}(x),$$

where `cdfnorm(x)` denotes the cumulative normal distribution function.

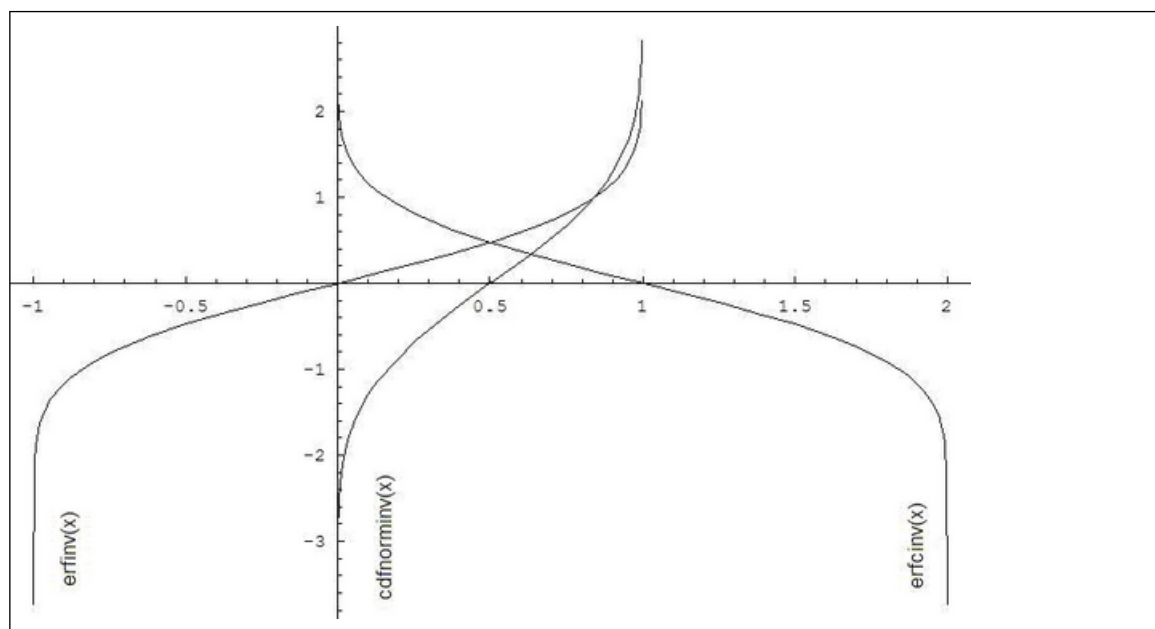
Useful relations:

$$\text{cdfnorminv}(x) = \sqrt{2}\text{erfinv}(2x - 1) = \sqrt{2}\text{erfcinv}(2 - 2x)$$

where `erfinv(x)` denotes the inverse error function and `erfcinv(x)` denotes the inverse complementary error functions.

The following figure illustrates the relationships among `erfinv` family functions (`erfinv`, `erfcinv`, `cdfnorminv`).

cdfnorminv Family Functions Relationship



Useful relations for these functions:

$$\text{erfcinv}(x) = \text{erfinv}(1 - x)$$

$$\text{cdfnorminv}(x) = \sqrt{2}\text{erfinv}(2x - 1) = \sqrt{2}\text{erfcinv}(2 - 2x)$$

Argument	Result	Error Code
+0.5	+0	
+1	$+\infty$	<code>status::sing</code>
-0	$-\infty$	<code>status::sing</code>
+0	$-\infty$	<code>status::sing</code>
$a < -0$	QNAN	<code>status::errrdom</code>
$a > +1$	QNAN	<code>status::errrdom</code>
$+\infty$	QNAN	<code>status::errrdom</code>
$-\infty$	QNAN	<code>status::errrdom</code>
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event cdfnorminv(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event cdfnorminv(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event cdfnorminv(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event cdfnorminv(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

cdfnorminv supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU

T	Devices supported
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
---	-------------------------------

return value (event) Computation end event.

Examples

An example of how to use `cdfnorminv` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vcdfnorminv.cpp
```

lgamma

Computes the element-wise natural logarithm of the absolute value of the gamma function of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `lgamma(a)` function computes the natural logarithm of the absolute value of gamma function for elements of the input vector `a` and writes them to the output vector `y`. Precision overflow thresholds for the `lgamma` function are beyond the scope of this document. If the result does not meet the target precision, the function sets the VM Error Status to `status::overflow`.

Argument	Result	Error Code
+1	+0	
+2	+0	
+0	+∞	<code>status::sing</code>
-0	+∞	<code>status::sing</code>
negative integer	+∞	<code>status::sing</code>
-∞	+∞	
+∞	+∞	
<code>a > overflow</code>	+∞	<code>status::overflow</code>
QNaN	QNaN	
SNAN	QNaN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event lgamma(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event lgamma(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event lgamma(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event lgamma(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

lgamma supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU
double	CPU

Input Parameters

Buffer API

exec_queue The queue where the routine will be executed.

<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `lgamma` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vlgamma.cpp
```

tgamma

Computes the element-wise gamma function of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `tgamma(a)` function computes the gamma function for elements of the input vector `a` and writes them to the output vector `y`. Precision overflow thresholds for the `tgamma` function are beyond the scope of this document. If the result does not meet the target precision, the function raises sets the VM Error Status to `status::sing`.

Argument	Result	Error Code
+0	$+\infty$	<code>status::sing</code>
-0	$-\infty$	<code>status::sing</code>
negative integer	QNAN	<code>status::errdom</code>
$-\infty$	QNAN	<code>status::errdom</code>
$+\infty$	$+\infty$	
<code>a > overflow</code>	$+\infty$	<code>status::sing</code>
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event tgamma(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event tgamma(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event tgamma(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event tgamma(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

tgamma supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU
double	CPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for a. See Data Types for more details.
sy	Slice selector for y. See Data Types for more details.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is mode::not_defined.
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `tgamma` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vtgamma.cpp
```

expint1

Computes the element-wise exponential integral of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `expint1(a)` function computes the exponential integral of vector elements of the input vector `a` and writes them to the output vector `y`.

For positive real values `x`, this can be written as:

$$E_1(x) = \int_x^{\infty} \frac{e^{-t}}{t} dt = \int_1^{\infty} \frac{e^{-xt}}{t} dt$$

For negative real values x , the result is defined as NaN.

Argument	Result	Error Code
$x < +0$	QNAN	<code>status::errdom</code>
$+0$	$+\infty$	<code>status::sing</code>
-0	$+\infty$	<code>status::sing</code>
$+\infty$	$+0$	
$-\infty$	QNAN	<code>status::errdom</code>
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event expint1(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event expint1(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event expint1(sycl::queue & exec_queue,
    std::int64_t n,
```



```

    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event expint1(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

`expint1` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU
<code>double</code>	CPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.

<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `expint1` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vexpint1.cpp
```

i0

Computes the element-wise regular modified cylindrical Bessel function of order 0.

- [Description](#)
- [API](#)

Description

The `i0(a)` function computes the regular modified cylindrical Bessel function of order 0 for elements of the input vector `a` and writes them to the output vector `y`.

Argument	Result	Error Code
+0	+1	
-0	+1	
+∞	+∞	
-∞	+∞	

Argument	Result	Error Code
QNaN	QNaN	
SNAN	QNaN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event i0(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event i0(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event i0(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event i0(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```

```

oneapi::mkl::vm::error_handler<T> errhandler = {};
}

```

i0 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU
double	CPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

`y` The buffer containing the output vector.

return value (event) Computation end event.

USM API

`y` Pointer to the output vector.

return value (event) Computation end event.

i1

Computes the element-wise regular modified cylindrical Bessel function of order 1.

- [Description](#)
- [API](#)

Description

The `i1(a)` function computes the regular modified cylindrical Bessel function of order 1 for elements of the input vector `a` and writes them to the output vector `y`.

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	$+\infty$	
$-\infty$	$-\infty$	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event i1(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event i1(sycl::queue & exec_queue,
               sycl::buffer<T> & a,
```

```

oneapi::mkl::slice sa,
sycl::buffer<T> & y,
oneapi::mkl::slice sy,
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event i1(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

```

namespace oneapi::mkl::vm {

sycl::event i1(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

i1 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU
double	CPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.

mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

j0

Computes the element-wise Bessel function of the first kind of order 0.

- [Description](#)
- [API](#)

Description

The `j0(a)` function computes the Bessel function of the first kind of order 0 for elements of the input vector `a` and writes them to the output vector `y`.

Argument	Result	Error Code
+0	+1	
-0	+1	
$+\infty$	0	
$-\infty$	0	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event j0(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event j0(sycl::queue & exec_queue,
               sycl::buffer<T> & a,
               oneapi::mkl::slice sa,
               sycl::buffer<T> & y,
               oneapi::mkl::slice sy,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event j0(sycl::queue & exec_queue,
               std::int64_t n,
               T const * a,
               T * y,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event j0(sycl::queue & exec_queue,
```



```

    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

j0 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU
double	CPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode Overrides the global VM mode setting for this function call. See the [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

j1

Computes the element-wise Bessel function of the first kind of order 1.

- [Description](#)
- [API](#)

Description

The `j1(a)` function computes the Bessel function of the first kind of order 1 for elements of the input vector `a` and writes them to the output vector `y`.

Argument	Result	Error Code
+0	+0	
-0	+0	
$+\infty$	0	
$-\infty$	0	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event j1(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & y,
```

```

    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event j1(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event j1(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event j1(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

j1 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU
double	CPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

jn

Computes the Bessel function of the first kind of a given order.

- [Description](#)

- [API](#)

Description

The `jn(a,b)` function computes the Bessel function of the first kind of given scalar order `b` for elements of vector `a`.

Argument	Result	Error Code
+0	+0	
-0	+0	
$+\infty$	0	
$-\infty$	0	
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event jn(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               T b,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event jn(sycl::queue & exec_queue,
               sycl::buffer<T> & a,
               oneapi::mkl::slice sa,
               T b,
               sycl::buffer<T> & y,
               oneapi::mkl::slice sy,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event jn(sycl::queue & exec_queue,
               std::int64_t n,
               T const * a,
               T b,
               T * y,
```

```

std::vector<sycl::event> const & depends = {},
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event jn(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T b,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

```

`jn` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Fixed value of power.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
-------------------------	---

<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Fixed value of power.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

y0

Computes the element-wise Bessel function of the second kind of order 0.

- [Description](#)
- [API](#)

Description

The `y0(a)` function computes the Bessel function of the second kind of order 0 for elements of the input vector `a` and writes them to the output vector `y`.

Argument	Result	Error Code
+0	$-\infty$	<code>status::sing</code>
-0	$-\infty$	<code>status::sing</code>
$+\infty$	0	
$-\infty$	0	
$x < 0$ (including $-\infty$)	SNAN	<code>status::errdom</code>

Argument	Result	Error Code
QNaN	QNaN	
SNAN	QNaN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event y0(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event y0(sycl::queue & exec_queue,
               sycl::buffer<T> & a,
               oneapi::mkl::slice sa,
               sycl::buffer<T> & y,
               oneapi::mkl::slice sy,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event y0(sycl::queue & exec_queue,
               std::int64_t n,
               T const * a,
               T * y,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
               oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event y0(sycl::queue & exec_queue,
               T const * a,
               oneapi::mkl::slice sa,
               T * y,
               oneapi::mkl::slice sy,
               std::vector<sycl::event> const & depends = {},
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
```



```
oneapi::mkl::vm::error_handler<T> errhandler = {});
}
```

y0 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU
double	CPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y

The buffer containing the output vector.

return value (event)

Computation end event.

USM API

y

Pointer to the output vector.

return value (event)

Computation end event.

y1

Computes the element-wise Bessel function of the second kind of order 1.

- [Description](#)
- [API](#)

Description

The `y1(a)` function computes the Bessel function of the second kind of order 1 for elements of the input vector `a` and writes them to the output vector `y`.

Argument	Result	Error Code
+0	$-\infty$	<code>status::sing</code>
-0	$-\infty$	<code>status::sing</code>
$+\infty$	0	
$-\infty$	0	
$x < 0$ (including $-\infty$)	SNAN	<code>status::errdom</code>
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```

namespace oneapi::mkl::vm {

sycl::event y1(sycl::queue & exec_queue,
               std::int64_t n,
               sycl::buffer<T> & a,
               sycl::buffer<T> & y,
               oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,

```

```

    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event y1(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event y1(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

namespace oneapi::mkl::vm {

sycl::event y1(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

y1 supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU
double	CPU

Input Parameters

Buffer API

exec_queue The queue where the routine will be executed.

<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

yn

Computes the Bessel function of the second kind of a given order.

- [Description](#)
- [API](#)

Description

The $yn(a, b)$ function computes the Bessel function of the second kind of given scalar order b for elements of vector a .

Argument	Result	Error Code
+0	$-\infty$	status::sing
-0	$-\infty$	status::sing
$+\infty$	0	
$-\infty$	0	
$x < 0$ (including $-\infty$)	SNAN	status::errrdom
QNAN	QNAN	
SNAN	QNAN	

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event yn(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    T b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event yn(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    T b,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event yn(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T b,
    T * y,
```

```

    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});
}

namespace oneapi::mkl::vm {

sycl::event yn(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T b,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}

```

yn supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
b	Fixed value of power.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
errhandler	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

exec_queue	The queue where the routine will be executed.
------------	---

<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Fixed value of power.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Rounding Functions

`floor`

Computes the element-wise rounding of vector elements to the nearest integer towards minus infinity.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `floor(a)` function computes an integer value rounded towards minus infinity for each vector element.

Argument	Result	Error Code
+0	+0	
-0	-0	
+∞	+∞	
-∞	-∞	
QNaN	QNaN	
SNAN	QNaN	

The `floor` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {  
  
sycl::event floor(sycl::queue & exec_queue,  
    std::int64_t n,  
    sycl::buffer<T> & a,  
    sycl::buffer<T> & y,  
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);  
  
}
```

```
namespace oneapi::mkl::vm {  
  
sycl::event floor(sycl::queue & exec_queue,  
    sycl::buffer<T> & a,  
    oneapi::mkl::slice sa,  
    sycl::buffer<T> & y,  
    oneapi::mkl::slice sy,  
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);  
  
}
```

USM API

```
namespace oneapi::mkl::vm {  
  
sycl::event floor(sycl::queue & exec_queue,  
    std::int64_t n,  
    oneapi::usm::buffer<T> & a,  
    oneapi::usm::buffer<T> & y,  
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);  
  
}
```



```

    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

namespace oneapi::mkl::vm {

sycl::event floor(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

```

`floor` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.

depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `floor` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vfloor.cpp
```

ceil

Computes the element-wise rounding of vector elements to the nearest integer towards plus infinity.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `ceil(a)` function computes an integer value rounded towards plus infinity for each vector element.

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	$+\infty$	
$-\infty$	$-\infty$	

Argument	Result	Error Code
QNaN	QNaN	
SNAN	QNaN	

The `ceil` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event ceil(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event ceil(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event ceil(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event ceil(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

`ceil` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `ceil` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vceil.cpp
```

trunc

Computes the element-wise rounding of vector elements to the nearest integer towards zero.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `trunc(a)` function computes an integer value rounded towards zero for each vector element.

$$a_i \geq 0, y_i = \lfloor a_i \rfloor$$

$$a_i < 0, y_i = \lceil a_i \rceil$$

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	$+\infty$	
$-\infty$	$-\infty$	
QNAN	QNAN	
SNAN	QNAN	

The `trunc` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {
sycl::event trunc(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
```

```

    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

namespace oneapi::mkl::vm {

sycl::event trunc(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event trunc(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event trunc(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

`trunc` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters**Buffer API**

`exec_queue` The queue where the routine will be executed.

`n` Specifies the number of elements to be calculated.

<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `trunc` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vtrunc.cpp
```

round

Computes the element-wise rounding of vector elements to the nearest integer, breaking ties away from zero.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `round(a)` function computes a value rounded to the nearest integer for each vector element. Input elements that are halfway between two consecutive integers are always rounded away from zero regardless of the rounding mode.

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	$+\infty$	
$-\infty$	$-\infty$	
QNAN	QNAN	
SNAN	QNAN	

The `round(a)` function does not generate any errors.

API

Syntax

Buffer API

```
sycl::event round(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
```

```
namespace oneapi::mkl::vm{

sycl::event round(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm{

sycl::event round(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {}),
```



```

    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

namespace oneapi::mkl::vm{

sycl::event round(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

`round` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode Overrides the global VM mode setting for this function call. See [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `round` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vround.cpp
```

nearbyint

Computes the element-wise rounding of vector elements to the nearest integer, according to the current rounding mode.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `nearbyint(a)` function computes a rounded integer value in a current rounding mode for each vector element.

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	$+\infty$	
$-\infty$	$-\infty$	
QNAN	QNAN	
SNAN	QNAN	

The `nearbyint` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event nearbyint(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event nearbyint(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event nearbyint(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event nearbyint(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

`nearbyint` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `nearbyint` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vnearbyint.cpp
```

rint

Computes the element-wise rounding of vector elements to the nearest integer, according to the current rounding mode.

- [Description](#)

- [API](#)
- [Examples](#)

Description

The `rint(a)` function computes a rounded floating-point integer value using the current rounding mode for each vector element.

The rounding mode affects the results computed for inputs that fall between consecutive integers. For example:

- $f(0.5) = 0$, for rounding modes set to round to nearest round toward zero or to minus infinity.
- $f(0.5) = 1$, for rounding modes set to plus infinity.
- $f(-1.5) = -2$, for rounding modes set to round to nearest or to minus infinity.
- $f(-1.5) = -1$, for rounding modes set to round toward zero or to plus infinity.

Argument	Result	Error Code
+0	+0	
-0	-0	
$+\infty$	$+\infty$	
$-\infty$	$-\infty$	
QNAN	QNAN	
SNAN	QNAN	

The `rint` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event rint(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event rint(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event rint(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event rint(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

rint supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.

<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `rint` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vrint.cpp
```

modf

Computes the element-wise truncated integral and remaining fractional parts of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `modf(a)` function computes a truncated integer value and the remaining fraction part for each vector element.

$$a_i \geq 0, \begin{cases} y_i = \lfloor a_i \rfloor \\ z_i = a_i - \lfloor a_i \rfloor \end{cases}$$

$$a_i < 0, \begin{cases} y_i = \lceil a_i \rceil \\ z_i = a_i - \lceil a_i \rceil \end{cases}$$

Argument	Result 1	Result 2	Error Code
+0	+0	+0	
-0	-0	-0	
$+\infty$	$+\infty$	+0	
$-\infty$	$-\infty$	-0	
SNAN	QNAN	QNAN	
QNAN	QNAN	QNAN	

The `modf` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event modf(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    sycl::buffer<T> & z,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event modf(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    sycl::buffer<T> & z,
    oneapi::mkl::slice sz,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```


USM API

```
namespace oneapi::mkl::vm {

sycl::event modf(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    T * z,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event modf(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    T * z,
    oneapi::mkl::slice sz,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

`modf` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters**Buffer API**

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>sz</code>	Slice selector for <code>z</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>sz</code>	Slice selector for <code>z</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector for truncated integer values.
<code>z</code>	The buffer containing the output vector for remaining fraction parts.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector for truncated integer values.
<code>z</code>	Pointer to the output vector for remaining fraction parts.
return value (event)	Computation end event.

Examples

An example of how to use `modf` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vmodf.cpp
```

frac

Computes the element-wise signed fractional part of vector elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `frac(a)` function computes a signed fractional part for each vector element.

$$y_i =$$

$$\begin{cases} a_i - \lfloor a_i \rfloor a_i, & \geq 0 \\ a_i - \lceil a_i \rceil, & a < 0 \end{cases}$$

$$y_i =$$

$$\begin{cases} a_i - \lfloor a_i \rfloor a_i, & \geq 0 \\ a_i - \lceil a_i \rceil, & a < 0 \end{cases}$$

Argument	Result	Error Code
+0	+0	
-0	-0	
+∞	+0	
-∞	-0	
QNAN	QNAN	
SNAN	QNAN	

The `frac` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event frac(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event frac(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event frac(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event frac(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

`frac` supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU

T	Devices supported
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

y	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

y	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `frac` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vfrac.cpp
```

VM Service Functions

The VM Service functions enable you to set/get the accuracy mode and error code. These functions are available both in the Fortran and C interfaces. The table below lists available VM Service functions and their short description.

Function Short Name	Description
<code>set_mode</code>	Sets the VM mode
<code>get_mode</code>	Gets the VM mode
<code>set_status</code>	Sets the VM Error Status
<code>get_status</code>	Gets the VM Error Status
<code>clear_status</code>	Clears the VM Error Status
<code>create_error_handler</code>	Creates the local VM error handler for a function
Product and Performance Information	
Performance varies by use, configuration and other factors. Learn more at https://www.intel.com/PerformanceIndex . Notice revision #20201201	

set_mode

Sets a new mode for VM functions according to the `mode` parameter and returns the previous VM mode.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `set_mode` function sets a new mode for VM functions according to the `new_mode` parameter and returns the previous VM mode. The mode change has a global effect on all the VM functions within a queue.

oneMKL VM mode parameters provide control on the accuracy of mathematical functions, and on oneMKL VM Strided API behavior. The mode set for a given queue can be overridden locally by the local mode parameter in a VM function call.

The mode value is a bitwise OR (|) combination of the values described in the following table.

Value	Description
Accuracy Control	
<code>oneapi::mkl::vm::mode::ha</code>	High accuracy versions of VM functions. (DEFAULT)
<code>oneapi::mkl::vm::mode::la</code>	Low accuracy versions of VM functions.

Value	Description
<code>oneapi::mkl::vm::mode::ep</code>	Enhanced performance accuracy versions of VM functions.
Slice Argument Checking	
<code>oneapi::mkl::vm::mode::badarg_exception</code>	Throw a <code>oneapi::mkl::invalid_argument</code> exception on invalid arguments. (DEFAULT)
<code>oneapi::mkl::vm::mode::badarg_quiet</code>	Invalid arguments quietly make the call a “no-op”. The VM status is set to <code>vm::status::empty_computation</code> .
Slice Indexing Controls	
<code>oneapi::mkl::vm::mode::slice_normal</code>	Non-equal slice sizes are considered invalid. (DEFAULT)
<code>oneapi::mkl::vm::mode::slice_minimum</code>	The minimum of all slice sizes defines the number of evaluations.
<code>oneapi::mkl::vm::mode::slice_cyclic</code>	The output slice(s) size defines the number of evaluations. Input slices wrap around from the start.
Default Local Mode	
<code>oneapi::mkl::vm::mode::not_defined</code>	VM mode not defined. This has no effect.

The default value if no VM mode is defined or if the VM mode value is set to `mode::not_defined` is `(mode::badarg_exception | mode::slice_normal | mode::ha)`.

API

Syntax

```
uint64_t set_mode(queue& exec_queue, uint64_t new_mode )
```

`set_mode` supports the following devices: CPU and GPU.

Input Parameters

exec_queue The queue where the routine will be executed.

new_mode Specifies the VM mode to be set.

Output Parameters

return value (old_mode) Specifies the former VM mode.

Examples

```
oldmode = set_mode (exec_queue , mode::la);
oldmode = set_mode (exec_queue , mode::ep | mode::ftzdazon);
```

get_mode

Gets the VM mode for a queue.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `get_mode` function returns the global VM `mode` parameter that controls accuracy and VM Strided API behavior for a given queue. The variable value is a combination by bitwise OR (`|`) of the values listed in the following table.

Value of mode	Description
Accuracy Control	
<code>oneapi::mkl::vm::mode::ha</code>	High accuracy versions of VM functions. (DEFAULT)
<code>oneapi::mkl::vm::mode::la</code>	Low accuracy versions of VM functions.
<code>oneapi::mkl::vm::mode::ep</code>	Enhanced performance accuracy versions of VM functions.
Slice Argument Checking	
<code>oneapi::mkl::vm::mode::badarg_exception</code>	Throw a <code>oneapi::mkl::invalid_argument</code> exception on invalid arguments. (DEFAULT)
<code>oneapi::mkl::vm::mode::badarg_quiet</code>	Invalid arguments quietly make the call a “no-op”. The VM status is set to <code>vm::status::empty_computation</code> .
Slice Indexing Controls	
<code>oneapi::mkl::vm::mode::slice_normal</code>	Non-equal slice sizes are considered invalid. (DEFAULT)
<code>oneapi::mkl::vm::mode::slice_minimum</code>	The minimum of all slice sizes defines the number of evaluations.
<code>oneapi::mkl::vm::mode::slice_cyclic</code>	The output slice(s) size defines the number of evaluations. Input slices wrap around from the start.
Default Local Mode	
<code>oneapi::mkl::vm::mode::not_defined</code>	VM mode not defined. This has no effect.

See the example below

API

Syntax

```
uint64_t get_mode( queue& exec_queue )
```

`get_mode` supports the following devices: CPU and GPU.

Input Parameters

`exec_queue` The queue where the routine will be executed.

Output Parameters

return value (old_mode) Specifies the global VM mode.

Examples

```
accm = get_mode (exec_queue) & mode::accuracy_mask;
denm = get_mode (exec_queue) & mode::ftzdaz_mask;
```

set_status

Sets the global VM Status according to new values and returns the previous VM Status.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `set_status` function sets the global VM Status to new value and returns the previous VM Status.

The global VM Status is a single value and it accumulates via bitwise OR (`|`) all errors that happen inside VM functions. The following table lists the possible error values.

Status	Description
Successful Execution	
<code>status::success</code>	VM function execution completed successfully
<code>status::not_defined</code>	VM status not defined
Warnings	
<code>status::accuracy_warning</code>	VM function execution completed successfully in a different accuracy mode
Computational Errors	
<code>status::errdom</code>	Values are out of a range of definition producing invalid (QNaN) result
<code>status::sing</code>	Values cause divide-by-zero (singularity) errors and produce an invalid (QNaN or Inf) result
<code>status::overflow</code>	An overflow happened during the calculation process
<code>status::underflow</code>	An underflow happened during the calculation process

API

Syntax

```
uint8_t set_status (queue& exec_queue, uint_8 new_status )
```

`set_status` supports the following devices: CPU and GPU.

Input Parameters

`exec_queue` The queue where the routine will be executed.

`new_status` Specifies the VM status to be set.

Output Parameters

return value (`old_status`) Specifies the former VM status.

Examples

```
uint8_t olderr = set_status (exec_queue, status::success);

if (olderr & status::errdom)
{
    std::cout << "Errdom status returned" << std::endl;
}

if (olderr & status::sing)
{
    std::cout << "Singularity status returned" << std::endl;
}
```

`get_status`

Gets the VM Status.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `get_status` function gets the VM status.

The global VM Status is a single value and it accumulates via bitwise OR (`|`) all computational errors that happen inside VM functions. The following table lists the possible error values.

Status	Description
Successful Execution	
<code>status::success</code>	VM function execution completed successfully
<code>status::not_defined</code>	VM status not defined
Warnings	
<code>status::accuracy_warning</code>	VM function execution completed successfully in a different accuracy mode
Computational Errors	
<code>status::errdom</code>	Values are out of a range of definition producing invalid (QNaN) result

Status	Description
<code>status::sing</code>	Values cause divide-by-zero (singularity) errors and produce and invalid (QNaN or Inf) result
<code>status::overflow</code>	An overflow happened during the calculation process
<code>status::underflow</code>	An underflow happened during the calculation process

API

Syntax

```
uint8_t get_status (queue& exec_queue )
```

`get_status` supports the following devices: CPU and GPU.

Input Parameters

`exec_queue` The queue where the routine will be executed.

Output Parameters

return value (`status`) Specifies the VM status.

Examples

```
uint8_t err = get_status (exec_queue);

if (err & status::errdom)
{
    std::cout << "Errdom status returned" << std::endl;
}

if (err & status::sing)
{
    std::cout << "Singularity status returned" << std::endl;
}
```

clear_status

Sets the VM Status according to `status::success` and returns the previous VM Status.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `clear_status` function sets the VM status to `status::success` and returns the previous VM status.

The global VM Status is a single value and it accumulates all errors that happen inside VM functions. The following table lists the possible error values.

Status	Description
Successful Execution	
<code>status::success</code>	VM function execution completed successfully
<code>status::not_defined</code>	VM status not defined
Warnings	
<code>status::accuracy_warning</code>	VM function execution completed successfully in a different accuracy mode
Computational Errors	
<code>status::errdom</code>	Values are out of a range of definition producing invalid (QNaN) result
<code>status::sing</code>	Values cause divide-by-zero (singularity) errors and produce an invalid (QNaN or Inf) result
<code>status::overflow</code>	An overflow happened during the calculation process
<code>status::underflow</code>	An underflow happened during the calculation process

API

Syntax

```
namespace oneapi::mkl::vm {
    uint8_t clear_status
        (queue& exec_queue )
}
```

`clear_status` supports the following devices: CPU and GPU.

Input Parameters

`exec_queue` The queue where the routine will be executed.

Output Parameters

return value (`old_status`) Specifies the former VM status.

Examples

```
uint8_t olderr = clear_status (exec_queue);
```

`create_error_handler`

Creates the local VM Error Handler for a function.

Syntax

Buffer API:

```
namespace oneapi::mkl::vm {
    error_handler<T> create_error_handler(
        buffer<uint8_t,
        1> & errarray,
        int64_t length = 1,
        uint8_t errstatus = status::not_defined,
        T fixup = 0.0,
        bool copysign = false )
}
```

USM API:

```
namespace oneapi::mkl::vm {
    error_handler<T> create_error_handler(
        uint8_t* errarray,
        int64_t length = 1,
        uint8_t errstatus = status::not_defined,
        T fixup = 0.0,
        bool copysign = false )
}
```

`create_error_handler` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Description

The `create_error_handler` creates the local VM Error handler to be passed to VM functions which support error handling.

The local VM Error Handler supports three modes:

- Single status mode:** all errors happened during function execution are being written into one status value.
 At the execution end the single value is either un-changed if no errors happened or contained accumulated (merged by bitwise OR) error statuses happened in function execution.
 Set the array pointer to any `status` object and the length equals 1 to enable this mode.
- Multiple status mode:** error statuses are saved as an array by indices where they happen.
 Notice that only error statuses are being written into the array, the success statuses are not to be written.
 That means the array needs to be allocated and initialized by user before function execution.
 To enable this mode allocate `status` array with the same size as argument and result vectors, set the `errarray` pointer to it and the length to the vector size.
- Fixup mode:** for all arguments which caused specific error status results to be overwritten by a user-defined value.
 To enable this mode set desirable `errstatus` and `fixup` values. The `fixup` value is written to results for each argument which caused the `errstatus` error.
 If the `copysign` is set to true then `fixup` value's sign set to the same sign of the argument which caused the `errstatus` – a suitable option for symmetric math functions.

The following table lists the possible computational error values.

Status	Description
Successful Execution	
<code>status::success</code>	VM function execution completed successfully
<code>status::not_defined</code>	VM status not defined
Warnings	
<code>status::accuracy_warning</code>	VM function execution completed successfully in a different accuracy mode
Computational Errors	
<code>status::errdom</code>	Values are out of a range of definition producing invalid (QNaN) result
<code>status::sing</code>	Values cause divide-by-zero (singularity) errors and produce an invalid (QNaN or Inf) result
<code>status::overflow</code>	An overflow happened during the calculation process
<code>status::underflow</code>	An underflow happened during the calculation process

Notes:

- You must allocate and initialize array `errarray` before calling VM functions in multiple status error handling mode.
The array should be large enough to contain `n` error codes, where `n` is the same as input/output vector size for the VM function.
- If no arguments passed to the `create_error_handler` function, then the empty object is created with all of three error handling modes disabled.
In this case, the VM math functions set the global error status only.

Input Parameters

<code>errarray</code>	Array to store error statuses (should be a buffer for buffer API).
<code>length</code>	Length of the <code>errarray</code> . This is an optional argument, default value is 1.
<code>errcode</code>	Error status to fixup results. This is an optional argument, default value is <code>status::not_defined</code> .
<code>fixup</code>	Fixup value for results. This is an optional argument, default value is 0.0.
<code>copysign</code>	Flag for setting the fixup value's sign the same as the argument's. This is an optional argument, default value <code>false</code> .

Output Parameters

return value	Specifies the error handler object to be created.
--------------	---

Examples

The following examples are possible usage models (USM API).

Single status mode with `create_error_handler()`:

```
error_handler<float> handler = vm::create_error_handler (st);
vm::sin(exec_queue, 1000, a, r, handler);
if ( st[0] & status::errdom)
```

```
{
    std::cout << "Errdom status returned" << std::endl;
}
```

Single status mode without `create_error_handler()`:

```
vm::sin(exec_queue, 1000, a, r, {st });
std::cout << status << std::endl;
if ( st[0] & status::errdom)
{
    std::cout << "Errdom status returned" << std::endl;
}
```

The `st` contains either `status::success` or accumulated error statuses if computational errors occurred in `vm::erfinv`.

Multiple status mode with `create_error_handler()`:

```
error_handler<float> handler = vm::create_error_handler (st, 1000);
vm::inv(exec_queue, 1000, a, r, handler);
for(int i=0; i<1000; i++)
    std::cout << st[i] << std::endl;
```

Multiple status mode without `create_error_handler()`:

```
vm::inv(exec_queue, 1000, a, r, {st, 1000});
for(int i=0; i<1000; i++)
    std::cout << st[i] << std::endl;
```

The `st` array contains all codes for computational errors that occur at the same vector indices `i` as the arguments that caused the errors.

Fixup status mode with `create_error_handler()`:

```
float fixup = 1.0;
error_handler<float> handler = vm::create_error_handler (nullptr, 0, status::errdom, fixup,
true);
vm::erfinv(exec_queue, 1000, a, r, handler);
```

Fixup status mode without `create_error_handler()`:

```
float fixup = 1.0;
vm::erfinv(exec_queue, 1000, a, r, { nullptr, 0, status::errdom, fixup, true });
```

All results in `r` which computation generated `status::errdom` are replaced by `fixup` values.

In the example above all the `erfinv` function's NAN results caused by greater than `|1|` arguments are replaced by 1.0 value with the same sign as the corresponding argument.

Mixed (Single and Fixup) status mode with `create_error_handler()`:

```
float fixup = 1e38;
error_handler<float> handler = vm::create_error_handler (st, 1, status::overflow, fixup);
vm::exp(exec_queue, 1000, a, r, handler);
if ( st & status::underflow)
{
    std::cout << "Underflow status returned" << std::endl;
}
```

Mixed (Single and Fixup) status mode without `create_error_handler()`:

```
float fixup = 1e38;
vm::exp(exec_queue, 1000, a, r, {st, 1, status::overflow, fixup});
if ( st & status::underflow)
```

```
{
    std::cout << "Underflow status returned" << std::endl;
}
```

Mixed (Multiple and Fixup) status mode with `create_error_handler()`:

```
namespace oneapi::mkl::vm {

float    fixup      = 1.0;
    error_handler<float> handler = vm::create_error_handler (st, 1000, status::errdom, fixup);
    vm::acospi(exec_queue, 1000, a, r, handler);
    for(int i=0; i<1000; i++)
        std::cout << st[i] << std::endl;

}
```

Mixed (Multiple and Fixup) status mode without `create_error_handler()`:

```
float    fixup      = 1.0;
    vm::acospi(exec_queue, 1000, a, r, { st, 1000, status::errdom, fixup});
    for(int i=0; i<1000; i++)
        std::cout << st[i] << std::endl;

}
```

The `st` array contains all codes for computational errors that occur at the same vector indices `i` as the arguments that caused the errors. Additionally, all results in `r` which computation generated `status::errdom` are replaced by `fixup` values.

No local error handling mode:

```
vm::pow(exec_queue, n, a, b, r);
    uint8_t err = vm::get_status (exec_queue);

    if (err & status::errdom)
    {
        std::cout << "Errdom status returned" << std::endl;
    }

    if (err & status::sing)
    {
        std::cout << "Singularity status returned" << std::endl;
    }

}
```

Only global accumulated error status `err` is set.

Miscellaneous VM Functions

copysign

Computes the element-wise copy of vector `a` elements with the sign of vector `b` elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `copysign(a,b)` function returns the first vector argument elements with the sign changed to match the sign of the second vector argument's corresponding elements.

Argument 1	Argument 2	Result	Error Code
any value	positive value	+any value	
any value	negative value	-any value	

The `copysign(a,b)` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event copysign(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event copysign(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event copysign(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event copysign(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
```

```

oneapi::mkl::slice sy,
std::vector<sycl::event> const & depends = {},
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

```

`copysign` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.

mode Overrides the global VM mode setting for this function call. See the [set_mode](#) function for possible values and their description. This is an optional parameter. The default value is `mode::not_defined`.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `copysign` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vcopysign.cpp
```

nextafter

Computes the element-wise next representable floating-point value of vector `a` elements in the direction of vector `b` elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `nextafter(a,b)` function returns a vector containing the next representable floating-point values following the first vector argument elements in the direction of the second vector argument's corresponding elements.

Arguments/Results	Error Code
Input vector argument element is finite and the corresponding result vector element value is infinite	<code>status::overflow</code>
Result vector element value is subnormal or zero, and different from the corresponding input vector argument element	<code>status::underflow</code>

Even though underflow or overflow can occur, the returned value is independent of the current rounding direction mode.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event nextafter(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event nextafter(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event nextafter(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

```
namespace oneapi::mkl::vm {

sycl::event nextafter(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined,
    oneapi::mkl::vm::error_handler<T> errhandler = {});

}
```

`nextafter` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .
<code>errhandler</code>	Sets local error handling mode for this function call. See the create_error_handler function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

errhandler Sets local error handling mode for this function call. See the [create_error_handler](#) function for arguments and their descriptions. This is an optional parameter. The local error handler is disabled by default.

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `nextafter` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vnextafter.cpp
```

fdim

Computes the element-wise positive difference between vector `a` elements and vector `b` elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `fdim(a,b)` function returns a vector containing the differences of the corresponding elements of the first and second vector arguments if the first element is larger, and `+0` otherwise.

Argument 1	Argument 2	Result	Error Code
any	QNaN	QNaN	
any	SNaN	QNaN	
QNaN	any	QNaN	
SNaN	any	QNaN	

The `fdim(a,b)` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {
sycl::event fdim(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
```

```

    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

namespace oneapi::mkl::vm {

sycl::event fdim(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event fdim(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event fdim(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

fdim supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `fdim` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vfdim.cpp
```


fmax

Computes the element-wise maximum of each pair of vector *a* elements and vector *b* elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `fmax(a,b)` function returns a vector with element values equal to the larger value from each pair of corresponding elements of the two vectors *a* and *b*: if $a < b$ `fmax(a,b)` returns *b*, otherwise `fmax(a,b)` returns *a*.

Argument 1	Argument 2	Result	Error Code
a not NAN	NAN	a	
NAN	b not NAN	b	
NAN	NAN	NAN	

The `fmax(a,b)` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event fmax(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event fmax(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event fmax(sycl::queue & exec_queue,
```

```

    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

namespace oneapi::mkl::vm {

sycl::event fmax(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

```

`fmax` supports the following precisions and devices:

T	Devices supported
<code>sycl::half</code>	GPU
<code>_Float16</code>	CPU
<code>float</code>	CPU and GPU
<code>double</code>	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
-------------------------	---

<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `fmax` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vfmax.cpp
```

fmin

Computes the element-wise minimum of each pair of vector `a` elements and vector `b` elements.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `fmin(a,b)` function returns a vector with element values equal to the smaller value from each pair of corresponding elements of the two vectors `a` and `b`: if `a > b` `fmin(a,b)` returns `b`, otherwise `fmin(a,b)` returns `a`.

Argument 1	Argument 2	Result	Error Code
<code>a</code> not NAN	NAN	<code>a</code>	
NAN	<code>b</code> not NAN	<code>b</code>	

Argument 1	Argument 2	Result	Error Code
NAN	NAN	NAN	

The `fmin(a,b)` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event fmin(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event fmin(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event fmin(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event fmin(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
```

```
oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}
```

fmin supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the 1st input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
b	The buffer containing the 2nd input vector.
sb	Slice selector for <i>b</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	Pointer to the 1st input vector.
sa	Slice selector for <i>a</i> . See Data Types for a description of the oneMKL slice type.
b	Pointer to the 2nd input vector.
sb	Slice selector for <i>b</i> . See Data Types for a description of the oneMKL slice type.
sy	Slice selector for <i>y</i> . See Data Types for a description of the oneMKL slice type.
depends	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
mode	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

y The buffer containing the output vector.

return value (event) Computation end event.

USM API

y Pointer to the output vector.

return value (event) Computation end event.

Examples

An example of how to use `fmin` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vfmin.cpp
```

maxmag

Computes the element-wise maximum of each pair of vector `a` elements and vector `b` elements in terms of magnitude.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `maxmag(a,b)` function returns a vector with element values equal to the element with the larger magnitude from each pair of corresponding elements of the two vectors `a` and `b`:

- If $|a| > |b|$ `maxmag(a,b)` returns `a`, otherwise `maxmag(a,b)` returns `b`.
- If $|b| > |a|$ `maxmag(a,b)` returns `b`, otherwise `maxmag(a,b)` returns `a`.
- Otherwise `maxmag(a,b)` behaves like `fmax`.

Argument 1	Argument 2	Result	Error Code
a not NAN	NAN	a	
NAN	b not NAN	b	
NAN	NAN	NAN	

The `maxmag(a,b)` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {  
  
sycl::event maxmag(sycl::queue & exec_queue,  
                  std::int64_t n,  
                  sycl::buffer<T> & a,  
                  sycl::buffer<T> & b,
```

```

    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);
}

namespace oneapi::mkl::vm {

sycl::event maxmag(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

USM API

```

namespace oneapi::mkl::vm {

sycl::event maxmag(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event maxmag(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

```

maxmag supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	The buffer containing the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	The buffer containing the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>mode</code>	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `maxmag` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vmaxmag.cpp
```


minmag

Computes the element-wise minimum of each pair of vector *a* elements and vector *b* elements in terms of magnitude.

- [Description](#)
- [API](#)
- [Examples](#)

Description

The `minmag(a,b)` function returns a vector with element values equal to the element with the smaller magnitude from each pair of corresponding elements of the two vectors *a* and *b*:

- If $|a| < |b|$ `minmag(a,b)` returns *a*, otherwise `minmag(a,b)` returns *b*.
- If $|b| < |a|$ `minmag(a,b)` returns *b*, otherwise `minmag(a,b)` returns *a*.
- Otherwise `minmag` behaves like `fmin`.

Argument 1	Argument 2	Result	Error Code
a not NAN	NAN	a	
NAN	b not NAN	b	
NAN	NAN	NAN	

The `minmag(a,b)` function does not generate any errors.

API

Syntax

Buffer API

```
namespace oneapi::mkl::vm {

sycl::event minmag(sycl::queue & exec_queue,
    std::int64_t n,
    sycl::buffer<T> & a,
    sycl::buffer<T> & b,
    sycl::buffer<T> & y,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

```
namespace oneapi::mkl::vm {

sycl::event minmag(sycl::queue & exec_queue,
    sycl::buffer<T> & a,
    oneapi::mkl::slice sa,
    sycl::buffer<T> & b,
    oneapi::mkl::slice sb,
    sycl::buffer<T> & y,
    oneapi::mkl::slice sy,
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

USM API

```
namespace oneapi::mkl::vm {

sycl::event minmag(sycl::queue & exec_queue,
    std::int64_t n,
    T const * a,
    T const * b,
    T * y,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}

namespace oneapi::mkl::vm {

sycl::event minmag(sycl::queue & exec_queue,
    T const * a,
    oneapi::mkl::slice sa,
    T const * b,
    oneapi::mkl::slice sb,
    T * y,
    oneapi::mkl::slice sy,
    std::vector<sycl::event> const & depends = {},
    oneapi::mkl::vm::mode mode = oneapi::mkl::vm::mode::not_defined);

}
```

minmag supports the following precisions and devices:

T	Devices supported
sycl::half	GPU
_Float16	CPU
float	CPU and GPU
double	CPU and GPU

Input Parameters

Buffer API

exec_queue	The queue where the routine will be executed.
n	Specifies the number of elements to be calculated.
a	The buffer containing the 1st input vector.
sa	Slice selector for a. See Data Types for a description of the oneMKL slice type.
b	The buffer containing the 2nd input vector.
sb	Slice selector for b. See Data Types for a description of the oneMKL slice type.
sy	Slice selector for y. See Data Types for a description of the oneMKL slice type.
mode	Overrides the global VM mode setting for this function call. See set_mode function for possible values and their description. This is an optional parameter. The default value is mode::not_defined.

USM API

<code>exec_queue</code>	The queue where the routine will be executed.
<code>n</code>	Specifies the number of elements to be calculated.
<code>a</code>	Pointer to the 1st input vector.
<code>sa</code>	Slice selector for <code>a</code> . See Data Types for a description of the oneMKL slice type.
<code>b</code>	Pointer to the 2nd input vector.
<code>sb</code>	Slice selector for <code>b</code> . See Data Types for a description of the oneMKL slice type.
<code>sy</code>	Slice selector for <code>y</code> . See Data Types for a description of the oneMKL slice type.
<code>depends</code>	Vector of dependent events (to wait for input data to be ready). This is an optional parameter. The default is an empty vector.
<code>mode</code>	Overrides the global VM mode setting for this function call. See the set_mode function for possible values and their description. This is an optional parameter. The default value is <code>mode::not_defined</code> .

Output Parameters

Buffer API

<code>y</code>	The buffer containing the output vector.
return value (event)	Computation end event.

USM API

<code>y</code>	Pointer to the output vector.
return value (event)	Computation end event.

Examples

An example of how to use `minmag` can be found in the oneMKL installation directory, under:

```
share/doc/mkl/examples/sycl/vml/source/vminmag.cpp
```

Random Number Generators

Intel® oneAPI Math Kernel Library (oneMKL) provides Data Parallel C++ interfaces for the Vector Statistics Random Number Generators (RNG) routines implementing commonly used pseudorandom, quasi-random, and non-deterministic generators with continuous and discrete distributions.

oneMKL RNG includes manual offload functionality similar to other domains ([Random Number Generators Routines](#)) and also device functionality — a set of functions callable directly from DPC++ kernels (refer to [Random Number Generators Device Routines](#) for detailed descriptions).

Definitions

Pseudo-random number generator is defined by a structure(S, μ, f, U, g), where:

- S is a finite set of states (the state space)
- μ is a probability distribution on S for the initial state (or seed) s_0

- $f: S \rightarrow S$ is the transition function
- U - a finite set of output symbols
- $g: S \rightarrow U$ an output function

The generation of random numbers is as follows:

1. Generate the initial state (called the seed) s_0 according to μ and compute $u_0 = g(s_0)$.
2. Iterate for $i = 1, \dots, s_i = f(s_{i-1})$ and $u_i = g(s_i)$. Output values u_i are the so-called random numbers produced by the PRNG.

In computational statistics, random variate generation is usually made in two steps:

1. Generating imitations of independent and identically distributed (i.i.d.) random variables having the uniform distribution over the interval $(0, 1)$
2. Applying transformations to these i.i.d. $U(0, 1)$ random variates in order to generate (or imitate) random variates and random vectors from arbitrary distributions.

All RNG routines can be classified into several categories:

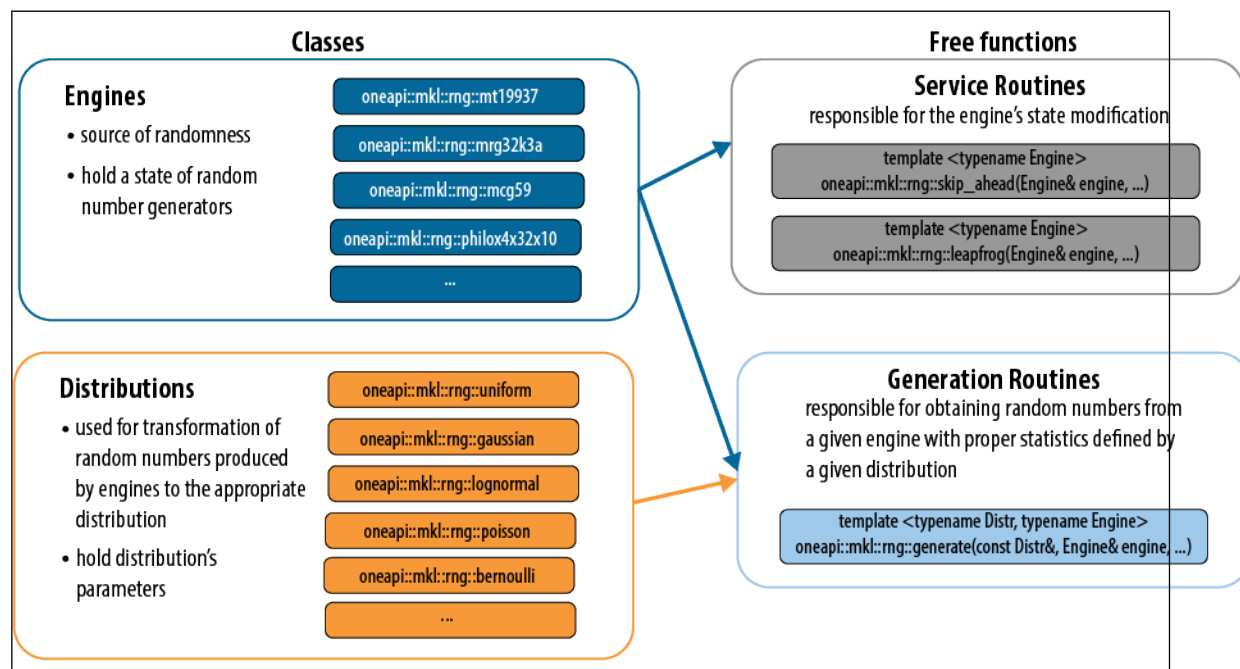
- Engines (Basic random number generators) classes, which hold the state of the generator and is a source of i.i.d. random variables.
- Transformation classes for different types of statistical distributions, for example, uniform, normal (Gaussian), binomial, etc. These classes contain all of the distribution's parameters (including generation method).
- Generate function. The current routine is used to obtain random numbers from a given engine with proper statistics defined by a given distribution.
- Service routines to modify the engine state: skip ahead and leapfrog functions.

Random Number Generators Host Routines

This section represents manual offload RNG routines, including engines and distributions classes templates, generate function, and service routines.

RNG Manual Offload Structure

RNG Manual Offload Structure



Additionally, examples that demonstrate usage of random number generators functionality are available in:

```
{MKL}/share/doc/mkl/examples/sycl/rng/source
```

Random Number Generators Routines

This section represents manual offload RNG routines, including engines and distributions classes templates, generate function, and service routines.

oneMKL RNG Usage Model

• Example of RNG Usage

A typical algorithm for random number generators is as follows:

1. Create and initialize the object for basic random number generator.
 - Use the `skip_ahead` or `leapfrog` function if it is required (used in parallel with random number generation for CPU devices).
2. Create and initialize the object for distribution generator.
3. Call the generate routine to get random numbers with appropriate statistical distribution.

The following example demonstrates generation of random numbers that is output from basic generator (engine) PHILOX4X32X10. The seed is equal to 777. The generator is used to generate 10,000 normally distributed random numbers with parameters $\mu = 5$ and $\sigma = 2$. The purpose of the example is to calculate the sample mean for normal distribution with the given parameters.

Example of RNG Usage

Buffer API

```
#include <iostream>
#include <vector>
```

```
#include <sycl/sycl.hpp>
#include "oneapi/mkl/rng.hpp"
#define SEED 777

int main() {
    sycl::queue queue;

    const size_t n = 10000;
    std::vector<double> r(n);

    // create basic random number generator object
    oneapi::mkl::rng::philox4x32x10 engine(queue, SEED);
    // create distribution object
    oneapi::mkl::rng::gaussian<double, oneapi::mkl::rng::gaussian_method::icdf> distr(5.0, 2.0);

    {
        // buffer for random numbers
        sycl::buffer<double, 1> r_buf(r.data(), r.size());
        // perform generation
        oneapi::mkl::rng::generate(distr, engine, n, r_buf);
    }

    double s = 0.0;
    for(int i = 0; i < n; i++) {
        s += r[i];
    }
    s /= n;

    std::cout << "Average = " << s << std::endl;

    return 0;
}
```

USM API

```
#include <iostream>
#include <vector>
#include <sycl/sycl.hpp>
#include "oneapi/mkl/rng.hpp"
#define SEED 777

int main() {
    sycl::queue queue;

    const size_t n = 10000;
```

```
// create USM allocator
sycl::usm_allocator<double, sycl::usm::alloc::shared> allocator(queue);

// create vector with USM allocator
std::vector<double, decltype(allocator)> r(n, allocator);

// create basic random number generator object
oneapi::mkl::rng::philox4x32x10 engine(queue, SEED);
// create distribution object
oneapi::mkl::rng::gaussian<double, oneapi::mkl::rng::gaussian_method::icdf> distr(5.0, 2.0);

// perform generation
auto event = oneapi::mkl::rng::generate(distr, engine, n, r.data());

// sycl::event object is returned by generate function for synchronization
event.wait(); // synchronization can be also done by queue.wait()

double s = 0.0;
for(int i = 0; i < n; i++) {
    s += r[i];
}
s /= n;

std::cout << "Average = " << s << std::endl;

return 0;
}
```

Additionally, examples that demonstrate usage of random number generators functionality are available in:

```
${MKL}/share/doc/mkl/examples/sycl/rng/source
```

Device Support

Data Parallel C++ supports several types of devices:

- Host device: Performs computations directly on the current CPU
- CPU device: Performs computations on a CPU using OpenCL™
- GPU device: Performs computations on a GPU

All Data Parallel C++ routines of oneMKL RNG support at least the CPU devices. GPU devices are supported for the following engines:

- `oneapi::mkl::rng::mcg31m1`
- `oneapi::mkl::rng::mcg59`
- `oneapi::mkl::rng::mrg32k3a`
- `oneapi::mkl::rng::mt19937`
- `oneapi::mkl::rng::mt2203`
- `oneapi::mkl::rng::philox4x32x10`
- `oneapi::mkl::rng::sobol`

GPU devices are supported for all distributions.

NOTE Some of the distributions use double precision inside the implementations, so not all GPUs are supported for them.

Refer to [Engines](#) and [Distributions](#) for more detailed descriptions of each routine.

Generate Routine

Use the [generate](#) routine to obtain random numbers from a given engine with proper statistics of a given distribution.

generate

- [Description](#)
- [API](#)

Description

Entry point to obtain random numbers from a given engine with proper statistics of a given distribution.

API

Syntax

Buffer API

```
namespace oneapi::mkl::rng {
    template<typename Distr, typename Engine>
    void generate (const Distr& distr,
                  Engine& engine, std::int64_t n,
                  sycl::buffer<typename Distr::result_type, 1>& r)
}
```

USM API

```
namespace oneapi::mkl::rng {
    template<typename Distr, typename Engine>
    sycl::event generate (const Distr& distr,
                        Engine& engine, std::int64_t n,
                        typename Distr::result_type* r,
                        const std::vector<sycl::event> & dependencies)
}
```

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
distr	const Distr&	Distribution object. See Distributions for details.
engine	Engine&	Engine object. See Engines for details.

Name	Type	Description
n	std::int64_t	Number of random values to be generated.

Optional Input Parameter for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event> &	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
r	sycl::buffer<typename Distr::result_type, 1>	sycl::bufferr to the output vector.

USM API

Name	Type	Description
r	typename Distr::result_type	Pointer r to the output vector.
event	sycl::event	Function return event after submitting task in sycl::queue from the engine.

generate submits a kernel into a queue that is held by the engine and fills sycl::buffer<typename sDistr::result_type> vector with n random numbers.

Engines (Basic Random Number Generators)

oneMKL RNG provides pseudorandom, quasi-random, and non-deterministic random number generators for Data Parallel C++:

Routine	Description
mrg32k3a	The combined multiple recursive pseudorandom number generator MRG32k3a [L'Ecuyer99a]
philox4x32x10	Philox4x32-10 counter-based pseudorandom number generator with a period of 2^{128} PHILOX4X32X10 [Salmon11]
mcg31m1	The 31-bit multiplicative congruential pseudorandom number generator MCG($1132489760, 2^{32} - 1$) [L'Ecuyer99a]
r250	The 32-bit generalized feedback shift register pseudorandom number generator GFSR(250, 103) [Kirkpatrick81]

Routine	Description
mcg59	The 59-bit multiplicative congruential pseudorandom number generator $MCG(13^{13}, 2^{59})$ from NAG Numerical Libraries [NAG]
wichmann_hill	Wichmann-Hill pseudorandom number generator (a set of 273 basic generators) from NAG Numerical Libraries [NAG]
mt19937	Mersenne Twister pseudorandom number generator $MT19937$ [Matsumoto98] with period length $2^{19937} - 1$ of the produced sequence
mt2203	Set of 6024 Mersenne Twister pseudorandom number generators $MT2203$ [Matsumoto98] , [Matsumoto00] . Each of them generates a sequence of period length equal to $2^{2203} - 1$. Parameters of the generators provide mutual independence of the corresponding sequences.
sfmt19937	SIMD-oriented Fast Mersenne Twister pseudorandom number generator $SFMT19937$ [Saito08] with a period length equal to $2^{19937} - 1$ of the produced sequence.
sobol	Sobol quasi-random number generator [Sobol76] , [Bratley88] , which works in arbitrary dimension.
niederreiter	Niederreiter quasi-random number generator [Bratley92] , which works in arbitrary dimension.
ars5	ARS-5 counter-based pseudorandom number generator with a period of 2^{128} , which uses instructions from the AES-NI set $ARS5$ [Salmon11] .
nondeterministic	Non-deterministic random number generator (RDRAND-based) [AVX] , [IntelSWMan] .

For some basic generators, oneMKL RNG provides two methods of creating independent states in multiprocessor computations, which are the leapfrog method and the block-splitting method. These sequence splitting methods are also useful in sequential Monte Carlo. The description of these functions can be found in the [Service Routines](#) section.

In addition, MT2203 pseudorandom number generator is a set of 6024 generators designed to create up to 6024 independent random sequences, which might be used in parallel Monte Carlo simulations. Another generator that has the same feature is Wichmann-Hill. It allows creating up to 273 independent random streams. The properties of the generators designed for parallel computations are discussed in detail in [\[Coddington94\]](#).

See [VS Notes](#) for the detailed description.

[mrg32k3a](#)

- [Description](#)
- [API](#)

Description

The combined multiple recursive pseudorandom number generator MRG32k3a [\[L'Ecuyer99a\]](#).

API

Syntax

```
namespace oneapi::mkl::rng {
    class mrg32k3a {
    public:
        static constexpr std::uint32_t default_seed = 1;

        mrg32k3a(sycl::queue queue, std::uint32_t seed = default_seed);
        mrg32k3a(sycl::queue queue, std::uint32_t seed, mrg32k3a_mode::optimal mode);
        mrg32k3a(sycl::queue queue, std::uint32_t seed, mrg32k3a_mode::custom mode);
        mrg32k3a(sycl::queue queue, std::initializer_list<std::uint32_t> seed);
        mrg32k3a(sycl::queue queue, std::initializer_list<std::uint32_t> seed,
                mrg32k3a_mode::optimal mode);
        mrg32k3a(sycl::queue queue, std::initializer_list<std::uint32_t> seed,
                mrg32k3a_mode::custom mode);
        mrg32k3a(const mrg32k3a& other);
        mrg32k3a(mrg32k3a&& other);
        mrg32k3a& operator=(const mrg32k3a& other);
        mrg32k3a& operator=(mrg32k3a&& other);
        ~mrg32k3a();
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl::queue</code> , calls of the <code>oneapi::mkl::rng::generate()</code> routine submits kernels in this queue.
seed	<code>std::uint32_t</code> <code>std::initializer_list<std::uint32_t></code>	Initial conditions of the generator state or engine state.
mode	<code>mrg32k3a_mode::optimal</code> <code>mrg32k3a_mode::custom</code>	Initial conditions of the generator state or engine state.

NOTE

- Sub-sequence parallelization [L'Ecuyer02] is supported via the `mode` parameter. `mrg32k3a_mode::optimal` uses the optimal number of sub-streams, and `mrg32k3a_mode::custom` enables the passing a user-defined number of sub-streams (e.g. `mrg32k3a_mode::custom my_mode{123}`). There is also a helper structure template, `mrg32k3a_mode::optimal_v`, for optimal mode. Refer to the corresponding example for usage details.
- Currently sub-sequence parallelization is supported only for uniform and gaussian distributions on GPU device.

See [VS Notes](#) for the detailed descriptions.

philox4x32x10

- [Description](#)
- [API](#)

Description

A Philox4x32-10 counter-based pseudorandom number generator [[Salmon11](#)].

API

Syntax

```
namespace oneapi::mkl::rng {
    class philox4x32x10 {
    public:
        static constexpr std::uint64_t default_seed = 0;

        philox4x32x10(sycl::queue queue, std::uint64_t seed = default_seed);
        philox4x32x10(sycl::queue queue, std::initializer_list<std::uint64_t> seed);
        philox4x32x10(const philox4x32x10& other);
        philox4x32x10(philox4x32x10&& other);
        philox4x32x10& operator=(const philox4x32x10& other);
        philox4x32x10& operator=(philox4x32x10&& other);
        ~philox4x32x10();
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl::queue</code> , calls of the <code>oneapi::mkl::rng::generate()</code> routine submits kernels in this queue.
seed	<code>std::uint64_t</code> <code>std::initializer_list<std::uint64_t></code>	Initial conditions of the generator state or engine state.

See [VS Notes](#) for the detailed descriptions.

mcg31m1

- [Description](#)
- [API](#)

Description

The 31-bit multiplicative congruential pseudorandom number generator $\text{MCG}(1132489760, 2^{32} - 1)$ [L'Ecuyer99a].

API

Syntax

```
namespace oneapi::mkl::rng {
    class mcg31m1 {
    public:
        static constexpr std::uint32_t default_seed = 1;

        mcg31m1(sycl::queue queue, std::uint32_t seed = default_seed);
        mcg31m1(const mcg31m1& other);
        mcg31m1(mcg31m1&& other);
        mcg31m1& operator=(const mcg31m1& other);
        mcg31m1& operator=(mcg31m1&& other);
        ~mcg31m1();
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl::queue</code> , calls of the <code>oneapi::mkl::rng::generate()</code> routine submits kernels in this queue.
seed	<code>std::uint32_t</code>	Initial conditions of the engine.

See [VS Notes](#) for the detailed descriptions.

mcg59

- [Description](#)
- [API](#)

Description

The 59-bit multiplicative congruential pseudorandom number generator $\text{MCG}(13^{13}, 2^{59})$ from NAG Numerical Libraries [NAG].

API

Syntax

```
namespace oneapi::mkl::rng {
    class mcg59 {
    public:
        static constexpr std::uint64_t default_seed = 1;

        mcg59(sycl::queue queue, std::uint64_t seed = default_seed);
        mcg59(const mcg59& other);
        mcg59(mcg59&& other);
        mcg59& operator=(const mcg59& other);
        mcg59& operator=(mcg59&& other);
        ~mcg59();
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl::queue</code> , calls of <code>oneapi::mkl::rng::generate()</code> routine submit kernels in this queue.
seed	<code>std::uint64_t</code>	Initial conditions of the engine.

See [VS Notes](#) for the detailed descriptions.

r250

- [Description](#)
- [API](#)

Description

The 32-bit generalized feedback shift register pseudorandom number generator GFSR(250,103) [\[Kirkpatrick81\]](#).

API

Syntax

```
namespace oneapi::mkl::rng {
    class r250 {
    public:
        static constexpr std::uint32_t default_seed = 1;

        r250(sycl::queue queue, std::uint32_t seed = default_seed);
        r250(sycl::queue queue, std::vector<std::uint32_t> seed);
        r250(const r250& other);
        r250(r250&& other);
        r250& operator=(const r250& other);
    };
}
```

```

    r250& operator=(r250&& other);
    ~r250();
};
}

```

Devices supported: CPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl::queue</code> , calls of <code>oneapi::mkl::rng::generate()</code> routine submit kernels in this queue.
seed	<code>std::uint32_t</code> <code>std::initializer_list<std::uint32_t></code>	Initial conditions of the engine.

See [VS Notes](#) for the detailed descriptions.

wichmann_hill

- [Description](#)
- [API](#)

Description

Wichmann-Hill pseudorandom number generator (a set of 273 basic generators) from NAG Numerical Libraries [\[NAG\]](#).

API

Syntax

```

namespace oneapi::mkl::rng {
    class wichmann_hill {
    public:
        static constexpr std::uint32_t default_seed = 1;

        wichmann_hill(sycl::queue queue, std::uint32_t seed = default_seed);
        wichmann_hill(sycl::queue queue, std::uint32_t seed, std::uint32_t engine_idx);
        wichmann_hill(sycl::queue queue, std::initializer_list<std::uint32_t> seed);
        wichmann_hill(sycl::queue queue, std::initializer_list<std::uint32_t> seed, std::uint32_t
engine_idx);
        wichmann_hill(const wichmann_hill& other);
        wichmann_hill(wichmann_hill&& other);
        wichmann_hill& operator=(const wichmann_hill& other);
        wichmann_hill& operator=(wichmann_hill&& other);
        ~wichmann_hill();
    };
}

```

Devices supported: CPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl::queue</code> , calls of <code>oneapi::mkl::rng::generate()</code> routine submit kernels in this queue.
seed	<code>std::uint32_t</code> <code>std::initializer_list<std::uint32_t></code>	Initial conditions of the engine.
engine_idx	<code>std::uint32_t</code>	Index of the engine from the set (set contains 273 basic generators)

See [VS Notes](#) for the detailed descriptions.

mt19937

- [Description](#)
- [API](#)

Description

Mersenne Twister pseudorandom number generator MT19937 [\[Matsumoto98\]](#) with period length $2^{19937} - 1$ of the produced sequence.

API

Syntax

```
namespace oneapi::mkl::rng {
    class mt19937 {
    public:
        static constexpr std::uint32_t default_seed = 1;

        mt19937(sycl::queue queue, std::uint32_t seed = default_seed);
        mt19937(sycl::queue queue, std::initializer_list<std::uint32_t> seed);
        mt19937(const mt19937& other);
        mt19937(mt19937&& other);
        mt19937& operator=(const mt19937& other);
        mt19937& operator=(mt19937&& other);
        ~mt19937();
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl::queue</code> , calls of <code>oneapi::mkl::rng::generate()</code> routine submit kernels in this queue.
seed	<code>std::uint32_t</code> <code>std::initializer_list<std::uint32_t></code>	Initial conditions of the engine.

See [VS Notes](#) for detailed descriptions.

sfmt19937

- [Description](#)
- [API](#)

Description

SIMD-oriented Fast Mersenne Twister pseudorandom number generator SFMT19937 [\[Saito08\]](#) with a period length equal to $2^{19937} - 1$ of the produced sequence.

API

Syntax

```
namespace oneapi::mkl::rng {
    class sfmt19937 {
    public:
        static constexpr std::uint32_t default_seed = 1;

        sfmt19937(sycl::queue queue, std::uint32_t seed = default_seed);
        sfmt19937(sycl::queue queue, std::initializer_list<std::uint32_t> seed);
        sfmt19937(const sfmt19937& other);
        sfmt19937(sfmt19937&& other);
        sfmt19937& operator=(const sfmt19937& other);
        sfmt19937& operator=(sfmt19937&& other);
        ~sfmt19937();
    };
}
```

Devices supported: CPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl</code> queue, calls of <code>oneapi::mkl::rng::generate()</code> routine submit kernels in this queue.

Name	Type	Description
seed	<code>std::uint32_t</code> <code>std::initializer_list<std::uint32_t></code>	Initial conditions of the engine.

See [VS Notes](#) for the detailed descriptions.

mt2203

- [Description](#)
- [API](#)

Description

Set of 6024 Mersenne Twister pseudorandom number generators MT2203 [\[Matsumoto98\]](#), [\[Matsumoto00\]](#).

Each of them generates a sequence of period length equal to $2^{2203} - 1$. Parameters of the generators provide mutual independence of the corresponding sequences.

API

Syntax

```
namespace oneapi::mkl::rng {
    class mt2203 {
    public:
        static constexpr std::uint32_t default_seed = 1;

        mt2203(sycl::queue queue, std::uint32_t seed = default_seed);
        mt2203(sycl::queue queue, std::uint32_t seed, std::uint32_t engine_idx);
        mt2203(sycl::queue queue, std::initializer_list<std::uint32_t> seed);
        mt2203(sycl::queue queue, std::initializer_list<std::uint32_t> seed, std::uint32_t
engine_idx);
        mt2203(const mt2203& other);
        mt2203(mt2203&& other);
        mt2203& operator=(const mt2203& other);
        mt2203& operator=(mt2203&& other);
        ~mt2203()
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl::queue</code> , calls of <code>oneapi::mkl::rng::generate()</code> routine submit kernels in this queue.

Name	Type	Description
seed	std::uint32_t std::initializer_list<std::uint32_t>	Initial conditions of the engine.
engine_idx	std::uint32_t	Index of the engine from the set (set contains 6024 basic generators).

See [VS Notes](#) for the detailed descriptions.

ars5

- [Description](#)
- [API](#)

Description

ARS-5 counter-based pseudorandom number generator with a period of 2^{128} , which uses instructions from the AES-NI set ARS5 [\[Salmon11\]](#).

API

Syntax

```
namespace oneapi::mkl::rng {
    class ars5 {
    public:
        static constexpr std::uint64_t default_seed = 0;

        ars5(sycl::queue queue, std::uint64_t seed = default_seed);
        ars5(sycl::queue queue, std::initializer_list<std::uint64_t> seed);
        ars5(const ars5& other);
        ars5(ars5&& other);
        ars5& operator=(const ars5& other);
        ars5& operator=(ars5&& other);
        ~ars5();
    };
}
```

Devices supported: CPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	sycl::queue	Valid <code>sycl::queue</code> , calls of <code>oneapi::mkl::rng::generate()</code> routine submit kernels in this queue.
seed	std::uint64_t std::initializer_list<std::uint64_t>	Initial conditions of the engine.

See [VS Notes](#) for the detailed descriptions.

sobol

- [Description](#)
- [API](#)

Description

Sobol quasi-random number generator [\[Sobol76\]](#), [\[Bratley88\]](#), which works in arbitrary dimension.

API

Syntax

```
namespace oneapi::mkl::rng {
    class sobol {
    public:
        static constexpr std::uint32_t default_dimensions_number = 1;

        sobol(sycl::queue queue, std::uint32_t dimensions = default_dimensions_number);
        sobol(sycl::queue queue, std::vector<std::uint32_t>& direction_numbers);
        sobol(const sobol& other);
        sobol(sobol&& other);
        sobol& operator=(const sobol& other);
        sobol& operator=(sobol&& other);
        ~sobol();
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl::queue</code> , calls of <code>oneapi::mkl::rng::generate()</code> routine submit kernels in this queue.
dimensions	<code>std::uint32_t</code>	Number of dimensions.
direction_numbers	<code>std::vector<std::uint32_t></code>	User-defined direction numbers.

NOTE `sobol` is a quasi-random number generator, so the resulting statistics might be worse than those of pseudo-random number generators in case of incorrect usage. For proper statistics for distributions that require more than 1 random number from BRNG (such as gaussian distribution with the `box_muller` and `box-muller2` methods), the dimensions parameter should be set to 2 or more.

See [VS Notes](#) for the detailed descriptions.

niederreiter

- [Description](#)
- [API](#)

Description

Niederreiter quasi-random number generator [\[Bratley92\]](#), which works in arbitrary dimension.

API

Syntax

```
namespace oneapi::mkl::rng {
    class niederreiter {
    public:
        static constexpr std::uint32_t default_dimensions_number = 1;

        niederreiter(sycl::queue queue, std::uint32_t dimensions = default_dimensions_number);
        niederreiter(sycl::queue queue, std::vector<std::uint32_t>& irred_polynomials);
        niederreiter(const niederreiter& other);
        niederreiter(niederreiter&& other);
        niederreiter& operator=(const niederreiter& other);
        niederreiter& operator=(niederreiter&& other);
        ~niederreiter();
    };
}
```

Devices supported: CPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl::queue</code> , calls of <code>oneapi::mkl::rng::generate()</code> routine submit kernels in this queue.
dimensions	<code>std::uint32_t</code>	Number of dimensions.
irred_polynomials	<code>std::vector<std::uint32_t></code>	User-defined direction numbers.

See [VS Notes](#) for the detailed descriptions.

nondeterministic

- [Description](#)
- [API](#)

Description

Non-deterministic random number generator (RDRAND-based) [\[AVX\]](#), [\[IntelSWMan\]](#).

API

Syntax

```
namespace oneapi::mkl::rng {
    class nondeterministic {
    public:
        nondeterministic(sycl::queue queue);
        nondeterministic(const nondeterministic& other);
        nondeterministic(nondeterministic&& other);
        nondeterministic& operator=(const nondeterministic& other);
        nondeterministic& operator=(nondeterministic&& other);
        ~nondeterministic();
    };
}
```

Devices supported: CPU.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
queue	<code>sycl::queue</code>	Valid <code>sycl::queue</code> , calls of <code>oneapi::mkl::rng::generate()</code> routine submits kernels in this queue.

See [VS Notes](#) for the detailed descriptions.

Service Routines

Routine	Description
leapfrog	Proceed state of engine by the leapfrog method to generate a subsequence of the original sequence
skip_ahead	Advances the state of the given engine as if it had generated 32 random bits a specified number of times.
save_state	Functionality to save the engine state in binary format to the file or memory buffer
load_state	Functionality to load the state of the random number engine from the provided memory buffer or file, then creates new engine object
get_state_size	Function to get the size in bytes which is needed to store the state of random number engine

leapfrog

Proceed state of engine using the leapfrog method.

- [Description](#)
- [API](#)

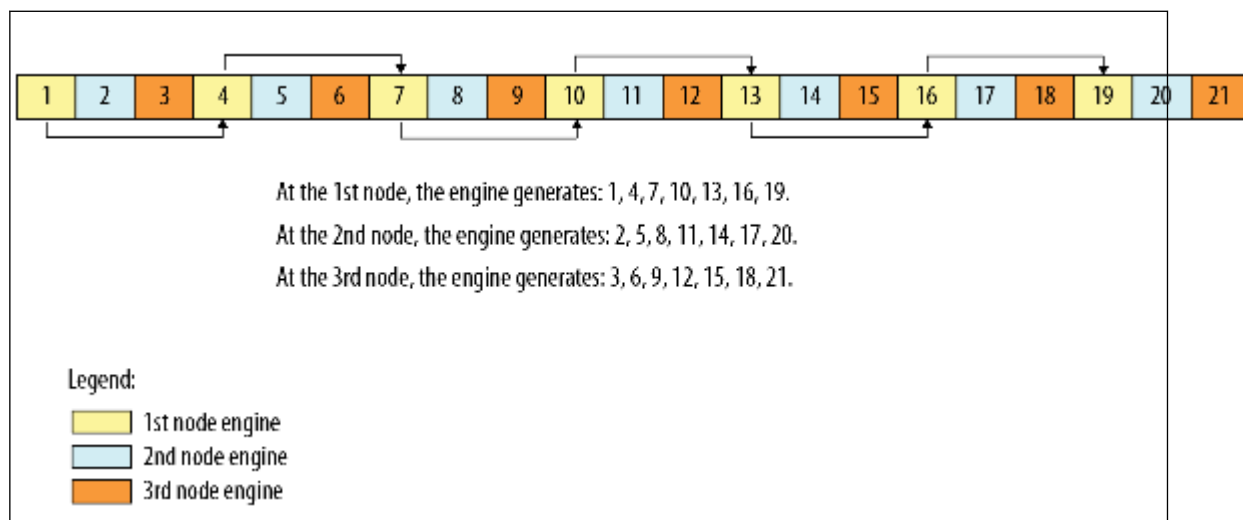
Description

The `leapfrog` function generates random numbers in an engine with non-unit stride. This feature is particularly useful in distributing random numbers from the original stream across the stride buffers without generating the original random sequence with subsequent manual distribution.

One of the important applications of the leapfrog method is splitting the original sequence into non-overlapping subsequences across stride computational nodes. The function initializes the original random stream (see the following figure) to generate random numbers for the computational node `idx`, $0 \leq \text{idx} < \text{stride}$, where `stride` is the largest number of computational nodes used.

Leapfrog Method

Leapfrog Method



The leapfrog method is supported only for those basic generators that allow splitting elements by the leapfrog method, which is more efficient than simply generating them by a generator with subsequent manual distribution across computational nodes. See [VS Notes](#) for details.

The following code illustrates the initialization of three independent streams using the leapfrog method:

Code for Leapfrog Method

```
// Creating 3 identical engines
mkl::rng::mcg31m1 engine_1(queue, seed);

mkl::rng::mcg31m1 engine_2(engine_1);
mkl::rng::mcg31m1 engine_3(engine_1);

// Leapfrogging the states of engines
mkl::rng::leapfrog(engine_1, 0, 3);
mkl::rng::leapfrog(engine_2, 1, 3);
mkl::rng::leapfrog(engine_3, 2, 3);
// Generating random numbers
...
```

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename Engine>
    void leapfrog (Engine& engine,
                  std::uint64_t idx,
                  std::uint64_t stride)
}
```

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
engine	Engine	Object of engine class, which supports leapfrog.
idx	std::uint64_t	Index of the computational node.
stride	std::uint64_t	Largest number of computational nodes, or stride.

skip_ahead

Proceeds the state of the engine using the skip-ahead method. The `skip_ahead` function supports the following interfaces to apply the skip-ahead method:

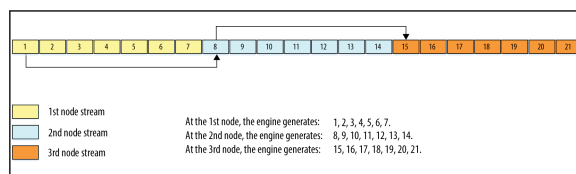
- Common interface
- Interface with a partitioned number of skipped elements
- [Description](#)
- [API](#)

Description

The `skip_ahead` function advances the state of the given engine as if it had generated 32 random bits a specified number of times. This feature is particularly useful in distributing random numbers from original engine across different computational nodes. If the largest number of random numbers used by a computational node is `num_to_skip`, then the original random sequence may be split by `skip_ahead` into non-overlapping blocks of `num_to_skip` size so that each block corresponds to the respective computational node. The number of computational nodes is unlimited. This method is known as the block-splitting method or as the skip-ahead method (see the following figure).

Block-Splitting Method

Block-Splitting Method



The skip-ahead method is supported only for those basic generators that allow skipping elements by the skip-ahead method, which is more efficient than simply generating them by generator with subsequent manual skipping. See [VS Notes](#) for details.

Please note that for quasi-random basic generators the skip-ahead method works with components of quasi-random vectors rather than with whole quasi-random vectors. Therefore, to skip NS quasi-random vectors, set the `num_to_skip` parameter equal to the `num_to_skip * dim`, where `dim` is the dimension of the quasi-random vector.

When the number of skipped elements is greater than 2^{63} the interface with the partitioned number of skipped elements is used.

Prior calls to the function represent the number of skipped elements with the list of size `n` as shown below:

```
num_to_skip[0] + num_to_skip[1]*264 + num_to_skip[2]* 2128 + ... + num_to_skip[n-1]*264*(n-1) ;
```

When the number of skipped elements is less than 2^{63} both interfaces can be used.

The following code illustrates how to initialize three independent streams using the `skip_ahead` function:

Code for Block-Splitting Method

```
...
// Creating 3 identical engines
oneapi::mkl::rng::mcg31m1 engine_1(queue, seed);
oneapi::mkl::rng::mcg31m1 engine_2(engine_1);
oneapi::mkl::rng::mcg31m1 engine_3(engine_2);

// Skipping ahead by 7 elements the 2nd engine
oneapi::mkl::rng::skip_ahead(engine_2, 7);

// Skipping ahead by 14 elements the 3rd engine
oneapi::mkl::rng::skip_ahead(engine_3, 14);
...
```

Code for Block-Splitting Method with Partitioned Number of Elements

```
// Creating first engine
oneapi::mkl::rng::mrg32k3a engine_1(queue, seed);

// To skip 2^64 elements in the random stream number of skipped elements should be
// represented as num_to_skip = 2^64 = 0 + 1 * 2^64
std::initializer_list<std::uint64_t> num_to_skip = {0, 1};

// Creating the 2nd engine based on 1st. Skipping by 2^64
oneapi::mkl::rng::mrg32k3a engine_2(engine_1);
oneapi::mkl::rng::skip_ahead(engine_2, num_to_skip);

// Creating the 3rd engine based on 2nd. Skipping by 2^64
oneapi::mkl::rng::mrg32k3a engine_3(engine_2);
oneapi::mkl::rng::skip_ahead(engine_3, num_to_skip);
...
```

API

Syntax

Common Interface

```
namespace oneapi::mkl::rng {
    template<typename Engine>
    void skip_ahead (Engine& engine,
                    std::uint64_t num_to_skip)
}
```

Interface with Partitioned Number of Skipped Elements

```
namespace oneapi::mkl::rng {
    template<typename Engine>
    void skip_ahead (Engine& engine,
                    std::initializer_list<std::uint64_t> num_to_skip)
}
```

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Common Interface

Name	Type	Description
engine	Engine	Object of engine class, which supports the block-splitting method.
num_to_skip	std::uint64_t	Number of skipped elements.

Interface with Partitioned Number of Skipped Elements

Name	Type	Description
engine	Engine	Object of engine class, which supports the block-splitting method.
num_to_skip	std::initializer_list<std::uint64_t>	Partitioned number of skipped elements.

save_state

Writes state of the random number engine to the file or memory buffer.

- [Description](#)
- [API](#)

Description

The `save_state` function allows you to store the state of the random number engine in the binary format in a file or memory buffer.

API

Syntax

Save to Memory Interface

```
namespace oneapi::mkl::rng {
    template<typename Engine>
    void save_state (Engine& engine,
                    std::uint8_t* mem);
}
```

Save to File Interface

```
namespace oneapi::mkl::rng {
    template<typename Engine>
    void save_state (Engine& engine,
                    const std::string& filename); // deprecated since 2024.1 release
}
```

> **Note:** The **Save to File** function is deprecated. Use the **Save to Memory** function instead.

Include Files

- oneapi/mkl/rng.hpp

Input Parameters

Save to Memory Interface

Name	Type	Description
engine	Engine&	Object of engine class, which state would be saved.
mem	std::uint8_t*	Memory, which you allocate to store the engine’s state. To check the size of memory in bytes needed for the particular engine, use the <code>get_state_size</code> function.

Save to File Interface

Name	Type	Description
engine	Engine&	Object of engine class, which state would be saved.
filename	const std::string&	Name of the file where the engine’s state would be written.

load_state

Loads the state of the random number engine from the provided memory buffer or file, then creates new engine object.

- [Description](#)
- [API](#)

Description

The `load_state` function allows you to create a new random number engine object from the binary state of another engine, which was written in a memory buffer or file. You can use different `sycl::queue` objects for the new engine.

API

Syntax

Load from Memory Interface

```
namespace oneapi::mkl::rng {
    template<typename Engine>
    Engine load_state (const sycl::queue& queue,
                     const std::uint8_t* mem);
}
```

Load from File Interface

```
namespace oneapi::mkl::rng {
    template<typename Engine>
    Engine load_state (const sycl::queue& queue,
                     const std::string& filename); // deprecated since 2024.1 release
}
```

> **Note:** The **Load from File** function is deprecated. Use the **Load from Memory** function instead.

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Load from Memory Interface

Name	Type	Description
queue	<code>const sycl::queue& queue</code>	<code>sycl::queue</code> object, which will be used for the newly-created engine.
mem	<code>const std::uint8_t*</code>	Memory, where engine's state was stored.

Load from File Interface

Name	Type	Description
queue	<code>const sycl::queue& queue</code>	<code>sycl::queue</code> object, which will be used for the newly-created engine.
filename	<code>const std::string&</code>	Name of the file where engine's state was written.

Output Parameters

Type	Description
Engine	New random number engine object, which would be created from the given <code>sycl::queue</code> and loaded engine's state.

The following code `engine_state.dat` file and load it back:
illustrates how to store the
engine's state to the

Code for Save/Load state functionality

```
// Creating GPU engine
oneapi::mkl::rng::default_engine engine(gpu_queue);

// Checking how much memory is required to save
auto mem_size = oneapi::mkl::rng::get_state_size(engine);

// Allocating memory buffer
std::uint8_t* mem_buf = new std::uint8_t[mem_size];

// Saving state of engine in the file
oneapi::mkl::rng::save_state(engine, mem_buf);

// Generating random numbers from the GPU engine
oneapi::mkl::rng::generate(oneapi::mkl::rng::uniform{}, engine, n, r1);

// Loading state for the CPU queue
auto engine_2 = oneapi::mkl::rng::load_state<oneapi::mkl::rng::default_engine>(cpu_queue,
mem_buf);

// Generating random numbers from the CPU engine
oneapi::mkl::rng::generate(oneapi::mkl::rng::uniform{}, engine_2, n, r2);

// Cleaning up memory buffer
delete[] mem_buf;
```

get_state_size

Returns the size in bytes which is needed to store the state of a given random number engine.

- [Description](#)
- [API](#)

Description

The `get_state_size` function allows you to know the amount of memory needed for storing the engine's state in memory.

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename Engine>
    std::int64_t get_state_size (Engine& engine);
}
```

Include Files

- `oneapi/mkl/rng.hpp`

Input Parameters

Name	Type	Description
engine	Engine&	Random number engine object, which state size would be calculated.

Output Parameters

Type	Description
<code>std::int64_t</code>	Size of the given engine's state in bytes.

The following code illustrates how to use the `get_state_size` function to store the engine's state to memory:

Code for Save/Load state functionality

```
// Creating GPU engine
oneapi::mkl::rng::default_engine engine(gpu_queue);

// Checking how much memory is required to save
auto mem_size = oneapi::mkl::rng::get_state_size(engine);

// Allocating memory buffer
std::uint8_t* mem_buf = new std::uint8_t[mem_size];

// Saving state of engine in the file
oneapi::mkl::rng::save_state(engine, mem_buf);

// Generating random numbers from the GPU engine
oneapi::mkl::rng::generate(oneapi::mkl::rng::uniform{}, engine, n, r1);

// Loading state for the CPU queue
auto engine_2 = oneapi::mkl::rng::load_state<oneapi::mkl::rng::default_engine>(cpu_queue,
mem_buf);

// Generating random numbers from the CPU engine
oneapi::mkl::rng::generate(oneapi::mkl::rng::uniform{}, engine_2, n, r2);

// Cleaning up memory buffer
delete[] mem_buf;
```

Distributions

- [Modes of Random Number Generation](#)

oneMKL RNG routines are used to generate random numbers with different types of distribution. Each function group is introduced below by the type of underlying distribution and contains a short description of its functionality, as well as specifications of the call sequence and the explanation of input and output parameters. The following tables list the random number generator routines with data types and output distributions, and sets correspondence between data types of the generator routines and the basic random number generators.

Type of Distribution	Data Types	BRNG Data Type	Description
uniform (Continuous)	float, double	float, double	Uniform continuous distribution on the interval $[a, b)$
gaussian	float, double	float, double	Normal (Gaussian) distribution
gaussian_mv	float, double	float, double	Normal (Gaussian) multivariate distribution
exponential	float, double	float, double	Exponential distribution
laplace	float, double	float, double	Laplace distribution (double exponential distribution)
weibull	float, double	float, double	Weibull distribution
cauchy	float, double	float, double	Cauchy distribution
rayleigh	float, double	float, double	Rayleigh distribution
lognormal	float, double	float, double	Lognormal distribution
gumbel	float, double	float, double	Gumbel (extreme value) distribution
gamma	float, double	float, double	Gamma distribution
gamma	float, double	float, double	Beta distribution
chi_square	float, double	float, double	Chi-Square distribution

Type of Distribution	Data Types	BRNG Data Type	Description
uniform (Discrete)	integer	GPU: float for <code>oneapi::mkl::rng::uniform_method::standard,</code> double for <code>oneapi::mkl::rng::method::accurate</code> CPU: double for both methods	Uniform discrete distribution on the interval $[a, b)$
uniform_bits	integer	integer	Uniformly distributed bits in 32-bit chunks
	integer	integer	Uniformly distributed bits in 64-bit chunks
bits	integer	integer	Bits of underlying BRNG integer sequence
bernoulli	integer	float	Bernoulli distribution
geometric	integer	float	Geometric distribution

Type of Distribution	Data Types	BRNG Data Type	Description
binomial	integer	double	Binomial distribution
hypergeometric	integer	double	Hypergeometric distribution
poisson	integer	float (for <code>oneapi::mkl::rng::poisson_method::gaussian_icdf_based</code> float (for distribution parameter $\lambda \geq 27$) and double (for $\lambda < 27$) (for <code>oneapi::mkl::rng::poisson_method::ptpe</code>)	Poisson distribution
poisson_v	integer	double	Poisson distribution with varying mean
negative_binomial	integer	double	Negative binomial distribution, or Pascal distribution
multinomial	integer	CPU - float GPU - double	Multinomial distribution

NOTE: In case of `integer` check desired distribution for supported data types.

Modes of Random Number Generation

The library provides two modes of random number generation, accurate and fast. Accurate generation mode is intended for the applications that are highly demanding to accuracy of calculations. When used in this mode, the generators produce random numbers lying completely within definitional domain for all values of the distribution parameters. For example, random numbers obtained from the generator of continuous distribution that is uniform on interval $[a,b]$ belong to this interval irrespective of what a and b values may be. Fast mode provides high performance of generation and also guarantees that generated random numbers belong to the definitional domain except for some specific values of distribution parameters. The generation mode is set by specifying relevant value of the method parameter in generator routines. List of distributions that support accurate mode of generation is given in the table below.

Distribution	Distribution Method
uniform (Discrete)	<code>oneapi::mkl::rng::uniform_method::accurate</code>
exponential	<code>oneapi::mkl::rng::exponential_method::icdf_accurate</code>
weibull	<code>oneapi::mkl::rng::weibull_method::icdf_accurate</code>
rayleigh	<code>oneapi::mkl::rng::rayleigh_method::icdf_accurate</code>
lognormal	<code>oneapi::mkl::rng::lognormal_method::icdf_accurate</code> <code>oneapi::mkl::rng::lognormal_method::box_muller2_accurate</code>

Distribution	Distribution Method
gamma	<code>oneapi::mkl::rng::gamma_method::marsaglia _accurate</code>
gamma	<code>oneapi::mkl::rng::beta_method::cja_accu rate</code>

Distributions Template Parameter Method

Method Type	Distributions	Math Description
<code>uniform_method::standard</code> <code>uniform_method::accurate</code>	uniform	Standard method. It involves transforming the output of a generator by scaling and shifting it to fit within the desired interval. <code>uniform_method::accurate</code> checks for additional <code>s</code> and <code>d</code> data types. For <code>integer</code> data types, it uses <code>d</code> as a BRNG data type (<code>sBRNG</code> data type is used in <code>uniform_method::standard</code> method on GPU).
<code>gaussian_method::box_mulle r</code>	gaussian	Generates normally distributed random number x through the pair of uniformly distributed numbers u_1 and u_2 according to the formula: $x = \sqrt{-2 \ln u_1} \sin 2\pi u_2.$
<code>gaussian_method::box_mulle r2</code>	gaussian	Generates normally distributed random numbers x_1 and x_2 through the pair of uniformly distributed numbers u_1 and u_2 according to the formulas: $x_1 = \sqrt{-2 \ln u_1} \sin 2\pi u_2,$ $x_2 = \sqrt{-2 \ln u_1} \cos 2\pi u_2.$
<code>gaussian_method::icdf</code>	gaussian	Inverse cumulative distribution function (ICDF) method.
<code>geometric_method::icdf</code>	geometric	
<code>exponential_method::icdf</code>	exponential	Inverse cumulative distribution function (ICDF) method.
<code>exponential_method::icdf_a ccurate</code>		
<code>weibull_method::icdf</code>	weibull	Inverse cumulative distribution function (ICDF) method.
<code>weibull_method::icdf_accu rate</code>		
<code>cauchy_method::icdf</code>	cauchy	Inverse cumulative distribution function (ICDF) method.

Method Type	Distributions	Math Description
rayleigh_method::icdf rayleigh_method::icdf_accu rate	rayleigh	Inverse cumulative distribution function (ICDF) method.
lognormal_method::icdf lognormal_method::icdf_acc urate	lognormal	Inverse cumulative distribution function (ICDF) method.
lognormal_method::box_mull er2 lognormal_method::box_mull er2_accurate	lognormal	Normally distributed random numbers x_1 and x_2 are produced through the pair of uniformly distributed numbers u_1 and u_2 according to the formulas: $x_1 = -2 \ln u_1 \sin 2\pi u_2,$ $x_2 = -2 \ln u_1 \cos 2\pi u_2$ Then x_1 and x_2 are converted to lognormal distribution.
gumbel_method::icdf	gumbel	Inverse cumulative distribution function (ICDF) method.
bernoulli_method::icdf	bernoulli	Inverse cumulative distribution function (ICDF) method.
gamma_method::marsaglia gamma_method::marsaglia_ac curate	gamma	For $\alpha > 1$, a gamma distributed random number is generated as a cube of a properly scaled normal random number; for $0.6 \leq \alpha < 1$, a gamma distributed random number is generated using rejection from Weibull distribution; for $\alpha < 0.6$, a gamma distributed random number is obtained using transformation of exponential power distribution; for $\alpha = 1$, gamma distribution is reduced to exponential distribution.
beta_method::cja beta_method::cja_accurate	beta	Cheng • If $\min(p, q) > 1$, - Johnk- Cheng method is - Atkins used. on • If metho • If d: $\min(p, q) - K_1 * p^2 + C \leq 0$ (P : , Johnk method is used. • Otherwise , Atkinson switching algorithm is used.

Method Type	Distributions	Math Description
		- If $\min(p, q) < 1$, $\max(p, q) > 1$, Atkinson switching algorithm is used. - If $p = 1$ or $q = 1$, inverse cumulative distribution function method is used - If $p = 1$ and $q = 1$, beta distribution is reduced to uniform distribution.
chi_square_method::gamma_based	chi_square	(most common): If $\nu \geq 17$ or ν is odd and $5 \leq \nu \leq 15$, a chi-square distribution is reduced to a Gamma distribution with these parameters: Shape $\alpha = \nu/2$ Offset $a = 0$ Scale factor $\beta = 2$. The random numbers of the Gamma distribution are generated.
gaussian_mv_method::box_muller	gaussian_mv	BoxMuller method for multivariate Gaussian distribution. BoxMuller_2 method for multivariate Gaussian distribution. Inverse cumulative distribution function (ICDF) method.
gaussian_mv_method::box_muller2		
gaussian_mv_method::icdf		
binomial_method::btpe	binomial	Acceptance/rejection method for $ntrial * \min(p, 1p) \geq 30$ with decomposition into four regions: Two parallelograms Triangle Left exponential tail Right exponential tail
poisson_method::ptpe	poisson	Acceptance/rejection method for $\lambda \geq 27$ with decomposition into four regions: Two parallelograms Triangle Left exponential tail Right exponential tail

Method Type	Distributions	Math Description
poisson_method::gaussian_icdf_based poisson_v_method::gaussian_icdf_based	poissonpoisson_v	for $\lambda \geq 1$, method based on Poisson inverse CDF approximation by Gaussian inverse CDF; for $\lambda < 1$, table lookup method is used.
hypergeometric_method::h2pe	hypergeometric	Acceptance/rejection method for large mode of distribution with decomposition into three regions: Rectangular Left exponential tail Right exponential tail
negative_binomial_method::nbar	negative_binomial	Acceptance/rejection method for: $\frac{(a-1) \cdot (1-p)}{p} \geq 100$ with decomposition into five regions: Rectangular (2) trapezoid Left exponential tail Right exponential tail
multinomial_method::poisson_icdf_based	multinomial	Multinomial distribution with parameters m , k , and a probability vector p . Random numbers of the multinomial distribution are generated by Poisson Approximation method.

uniform (Continuous)

Generates random numbers with uniform distribution.

- [Description](#)
- [API](#)

Description

The class object is used in `generate` function to provide random numbers uniformly distributed over the interval $[a, b)$, where a, b are the left and right bounds of the interval, respectively, and $a, b \in R; a < b$.

The probability density function is given by:

$$f_{a,b}(x) = \begin{cases} \frac{1}{b-a}, & x \in [a, b) \\ 1, & x \notin [a, b) \end{cases}, -\infty < x < +\infty$$

The cumulative distribution function is as follows:

$$F_{a,b}(x) = \begin{cases} 0, & x < a \\ \frac{x-a}{b-a}, & a \leq x < b \\ 1, & x \geq b \end{cases}, -\infty < x < +\infty$$

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename Type = float,
            typename Method = uniform_method::by_default>
    class uniform {
    public:
        using method_type = Method;
        using result_type = Type;

        uniform(): uniform(static_cast<Type>(0.0), static_cast<Type>(1.0)){}
        explicit uniform(Type a, Type b);
        explicit uniform(const param_type& pt);

        Type a() const;
        Type b() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

`typename Type = float`

Type of the produced values. The specific values are as follows:

`float`

`double`

`typename Method =
oneapi::mkl::rng::uniform_method::by_defa
ult`

Generation method. The specific values are as follows:

`oneapi::mkl::rng::uniform_method::standar
doneapi::mkl::rng::uniform_method::accura
te`

See brief descriptions of the methods in
[Distributions Template Parameter Method](#)

Input Parameters

Name	Type	Description
a	Type (float, double)	Left bound a
b	Type (float, double)	Right bound b

gaussian

Generates normally distributed random numbers.

- [Description](#)
- [API](#)

Description

The class object is used in `generate` function to provide random numbers with normal (Gaussian) distribution with mean (a) and standard deviation (`stddev`, σ), where $a, \sigma \in R; \sigma > 0$.

The probability density function is given by:

$$f_{a,\sigma}(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y-a)^2}{2\sigma^2}\right) dy, -\infty < x < +\infty$$

The cumulative distribution function is as follows:

$$F_{a,\sigma}(x) = \int_{-\infty}^x \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y-a)^2}{2\sigma^2}\right) dy, -\infty < x < +\infty$$

The cumulative distribution function $F_{a,\sigma}(x)$ can be expressed in terms of standard normal distribution $\phi(x)$ as $F_{a,\sigma}(x) = \phi((x-a)/\sigma)$

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename RealType = float,
            typename Method = gaussian_method::by_default>
    class gaussian {
    public:
        using method_type = Method;
        using result_type = RealType;

        gaussian(): gaussian(static_cast<RealType>(0.0), static_cast<RealType>(1.0)) {}
        explicit gaussian(RealType mean, RealType stddev);
        explicit gaussian(const param_type& pt);

        RealType mean() const;
        RealType stddev() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

Name	Description
<code>typename RealType = float</code>	Type of the produced values. The specific values are as follows: <code>float</code> <code>double</code>
<code>typename Method = `oneapi::mkl::rng::gaussian_method::by_d efault</code>	Generation method. The specific values are as follows: <code>oneapi::mkl::rng::gaussian_method::box_mu ller</code> <code>oneapi::mkl::rng::gaussian_method::box_mu ller2</code> <code>oneapi::mkl::rng::gaussian_method::icdf</code> See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
<code>mean</code>	<code>RealType (float, double)</code>	Mean value μ .
<code>stddev</code>	<code>RealType (float, double)</code>	Standard deviation σ .

exponential

Generates exponentially distributed random numbers.

- [Description](#)
- [API](#)

Description

The `exponential` class object is used in `generate` function to provide random numbers with exponential distribution that has displacement a and scalefactor β , where $a, \beta \in R; \beta > 0$.

The probability density function is given by:

$$f_{a,\beta}(x) = \begin{cases} \frac{1}{\beta} \exp\left(-\frac{(x-a)}{\beta}\right), & x \geq a \\ 0, & x < a \end{cases}, -\infty < x < +\infty$$

The cumulative distribution function is as follows:

$$F_{a,\beta}(x) = \begin{cases} 1 - \exp(-\frac{(x-a)}{\beta}), & x \geq a \\ 0, & x < a \end{cases}, -\infty < x < +\infty$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename RealType = float,
            typename Method = exponential_method::by_default>
    class exponential {
    public:
        using method_type = Method;
        using result_type = RealType;

        exponential(): exponential((RealType)0.0, (RealType)1.0){}
        explicit exponential(RealType mean, RealType stddev);
        explicit exponential(const param_type& pt);

        RealType a() const;
        RealType beta() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

`typename RealType = float`

Type of the produced values. The specific values are as follows:

`float`

`double`

`typename Method = oneapi::mkl::rng::exponential_method::by_default`

Generation method. The specific values are as follows:

`oneapi::mkl::rng::exponential_method::icdf`

`f`

`oneapi::mkl::rng::exponential_method::icdf`

`f_accurate`

See brief descriptions of the methods in [Distributions Template Parameter Method](#).

Input Parameters

Name	Type	Description
a	RealType (float, double)	Displacement a.
beta	RealType (float, double)	Scalefactor β .

laplace

Generates random numbers with Laplace distribution.

- [Description](#)
- [API](#)

Description

The `laplace` class object is used in the `generate` function to provide random numbers with Laplace distribution with mean value (or average) a and scalefactor b , where $a, b \in R; b > 0$. The scalefactor value determines the standard deviation as:

$$\sigma = b\sqrt{2}$$

The probability density function is given by:

$$f_{a,b}(x) = \frac{1}{2b} \exp\left(-\frac{|x-a|}{b}\right), -\infty < x < +\infty$$

The cumulative distribution function is as follows:

$$F_{a,b}(x) = \begin{cases} \frac{1}{2} \exp\left(\frac{x-a}{b}\right), & x < a \\ 1 - \frac{1}{2} \exp\left(-\frac{x-a}{b}\right), & x \geq a \end{cases}, -\infty < x < +\infty$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename RealType = float,
            typename Method = laplace_method::by_default>
```

```

class laplace {
public:
    using method_type = Method;
    using result_type = RealType;

    laplace(): laplace((RealType)0.0, (RealType)1.0){}
    explicit laplace(RealType a, RealType b);
    explicit laplace(const param_type& pt);

    RealType a() const;
    RealType b() const;
    param_type param() const;
    void param(const param_type& pt);
};
}

```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

<code>typename RealType = float</code>	Type of the produced values. The specific values are as follows: <code>float</code> <code>double</code>
<code>typename Method = oneapi::mkl::rng::laplace_method::by_default</code>	Generation method. The specific values are as follows: <code>oneapi::mkl::rng::laplace_method::icdf</code> See brief descriptions of the methods in Distributions Template Parameter Method

Input Parameters

Name	Type	Description
a	<code>RealType (float, double)</code>	Mean value a.
b	<code>RealType (float, double)</code>	Scalefactor b.

weibull

Generates Weibull distributed random numbers.

- [Description](#)
- [API](#)

Description

The `weibull` class object is used in the `generate` function to provide Weibull distributed random numbers with displacement a , scalefactor β , and shape α , where $\alpha, \beta, a \in R; \alpha > 0, \beta > 0$.

The probability density function is given by:

$$f_{a,\alpha,\beta}(x) = \begin{cases} \frac{\alpha}{\beta^\alpha} (x-a)^{\alpha-1} \exp\left(-\left(\frac{x-a}{\beta}\right)^\alpha\right), & x \geq a \\ 0, & x < a \end{cases}$$

The cumulative distribution function is as follows:

$$F_{a,\alpha,\beta}(x) = \begin{cases} 1 - \exp\left(-\left(\frac{x-a}{\beta}\right)^\alpha\right), & x \geq a \\ 0, & x < a \end{cases}, -\infty < x < +\infty$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
template<typename RealType = float,
        typename Method = weibull_method::by_default>
class weibull {
public:
    using method_type = Method;
    using result_type = RealType;

    weibull(): weibull((RealType)1.0, (RealType)0.0, (RealType)1.0){}
    explicit weibull(RealType alpha, RealType a, RealType beta);
    explicit weibull(const param_type& pt);

    RealType alpha() const;
    RealType a() const;
    RealType beta() const;
    param_type param() const;
    void param(const param_type& pt);
};
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

`typename RealType = float`

Type of the produced values. The specific values are as follows:

`float`

`double`

```
typename Method =
oneapi::mkl::rng::weibull_method::by_defa
ult
```

Generation method. The specific values are as follows:

oneapi::mkl::rng::weibull_method::icdf

oneapi::mkl::rng::weibull_method::icdf_accurate

See brief descriptions of the methods in [Distributions Template Parameter Method](#)

Input Parameters

Name	Type	Description
alpha	RealType (float, double)	Shape α
a	RealType (float, double)	Displacement a.
beta	RealType (float, double)	Scalefactor β .

cauchy

Generates Cauchy distributed random values.

- [Description](#)
- [API](#)

Description

The `cauchy` class object is used in the `generate` function to provide Cauchy distributed random numbers with displacement (a) and scalefactor (b, β), where $a, \beta \in R; \beta > 0$.

The probability density function is given by:

$$F_{a,\beta}(x) = \frac{1}{\pi\beta \left(1 + \left(\frac{x-a}{\beta}\right)^2\right)}, -\infty < x < +\infty$$

The cumulative distribution function is as follows:

$$F_{a,\beta}(x) = \frac{1}{2} + \frac{1}{\pi} \arctan\left(\frac{x-a}{\beta}\right), -\infty < x < +\infty$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename RealType = float,
            typename Method = cauchy_method::by_default>
    class cauchy {
    public:
        using method_type = Method;
        using result_type = RealType;

        cauchy(): cauchy((RealType)0.0, (RealType)1.0){}
        explicit cauchy(RealType a, RealType b);
        explicit cauchy(const param_type& pt);

        RealType a() const;
        RealType b() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

<code>typename RealType = float</code>	Type of the produced values. The specific values are as follows: <code>float</code> <code>double</code>
<code>typename Method = oneapi::mkl::rng::cauchy_method::by_default</code>	Generation method. The specific values are as follows: <code>oneapi::mkl::rng::cauchy_method::icdf</code> See brief descriptions of the methods in Distributions Template Parameter Method

Input Parameters

Name	Type	Description
a	<code>RealType (float, double)</code>	Displacement a.
b	<code>RealType (float, double)</code>	Scalefactor b.

rayleigh

Generates Rayleigh distributed random values.

- [Description](#)
- [API](#)

Description

The `oneapi::mkl::rng::rayleigh` class object is used by the `oneapi::mkl::rng::generate` function to provide Rayleigh distributed random numbers with displacement (a) and scalefactor (b, β), where $a, \beta \in \mathbb{R}; \beta > 0$.

The Rayleigh distribution is a special case of the [Weibull](#) distribution, where the shape parameter $\alpha = 2$.

The probability density function is given by:

$$f_{a,\beta}(x) = \begin{cases} \frac{2(x-a)}{\beta^2} \exp\left(-\frac{(x-a)^2}{\beta^2}\right), & x \geq a \\ 0, & x < a \end{cases}, -\infty < x < +\infty$$

The cumulative distribution function is as follows:

$$F_{\alpha,\beta}(x) = \begin{cases} 1 - \exp\left(-\frac{(x-\alpha)^2}{\beta^2}\right), & x \geq \alpha \\ 0, & x < \alpha \end{cases}, -\infty < x < +\infty$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename RealType = float,
            typename Method = rayleigh_method::by_default>
    class rayleigh {
    public:
        using method_type = Method;
        using result_type = RealType;

        rayleigh(): rayleigh((RealType)0.0, (RealType)1.0){}
        explicit rayleigh(RealType a, RealType b);
        explicit rayleigh(const param_type& pt);

        RealType a() const;
        RealType b() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

`typename RealType = float`

Type of the produced values. The specific values are as follows:

	float
	double
typename Method = oneapi::mkl::rng::rayleigh_method::by_def ault	Generation method. The specific values are as follows: oneapi::mkl::rng::rayleigh_method::icdf oneapi::mkl::rng::rayleigh_method::icdf_accurate See brief descriptions of the methods in Distributions Template Parameter Method.

Input Parameters

Name	Type	Description
a	RealType (float, double)	Displacement a.
b	RealType (float, double)	Scalefactor b.

lognormal

Generates lognormally distributed random numbers.

- [Description](#)
- [API](#)

Description

The `lognormal` class object is used in the `generate` function to provide random numbers with average of distribution (`m`, `a`) and standard deviation (`s`, σ) of subject normal distribution, displacement (`displ`, `b`), and scalefactor (`scale`, β), where $a, \sigma, b, \beta \in R; \sigma, \beta > 0$;

The probability density function is given by:

$$f_{a,\sigma,b,\beta}(x) = \begin{cases} \frac{1}{\sigma(x-b)\sqrt{2\pi}} \exp\left(-\frac{\ln(\frac{x-b}{\beta})-a)^2}{2\sigma^2}\right), & x > b \\ 0, & x \leq b \end{cases}$$

The cumulative distribution function is as follows:

$$F_{a,\sigma,b,\beta}(x) = \begin{cases} \Phi\left(\frac{\ln(\frac{x-b}{\beta})-a}{\sigma}\right), & x > b \\ 0, & x \leq b \end{cases}$$

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename RealType = float,
            typename Method = lognormal_method::by_default>
    class lognormal {
    public:
        using method_type = Method;
        using result_type = RealType;

        lognormal(): lognormal((RealType)0.0, (RealType)1.0,
                               (RealType) 0.0, (RealType)1.0){}
        explicit lognormal(RealType m, RealType s,
                           RealType displ = (RealType)0.0,
                           RealType scale = (RealType)1.0);
        explicit lognormal(const param_type& pt);

        RealType m() const;
        RealType s() const;
        RealType displ() const;
        RealType scale() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

<code>typename RealType = float</code>	Type of the produced values. The specific values are as follows: <code>float</code> <code>double</code>
<code>typename Method = oneapi::mkl::rng::lognormal_method::by_default</code>	Generation method. The specific values are as follows: <code>oneapi::mkl::rng::lognormal_method::box_muller2</code> <code>oneapi::mkl::rng::lognormal_method::box_muller2_accurate</code> <code>oneapi::mkl::rng::lognormal_method::icdf</code> <code>oneapi::mkl::rng::lognormal_method::icdf_accurate</code> See brief descriptions of the methods in Distributions Template Parameter Method.

Input Parameters

Name	Type	Description
m	RealType (float, double)	Average μ of the subject normal distribution.
s	RealType (float, double)	Standard deviation σ of the subject normal distribution.
displ	RealType (float, double)	Displacement μ .
scale	RealType (float, double)	Scalefactor σ .

gumbel

Generates Gumbel distributed random values.

- [Description](#)
- [API](#)

Description

The `gumbel` class object is used in the `generate` function to provide Gumbel distributed random numbers with displacement a and scalefactor (b, β) , where $a, \beta \in R; \beta > 0$.

The probability density function is given by:

$$f_{a,\beta}(x) = \frac{1}{\beta} \exp\left(\frac{x-a}{\beta}\right) \exp\left(-\exp\left(\frac{x-a}{\beta}\right)\right), -\infty < x < +\infty$$

The cumulative distribution function is as follows:

$$F_{a,\beta}(x) = 1 - \exp\left(-\exp\left(\frac{x-a}{\beta}\right)\right), -\infty < x < +\infty$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename RealType = float,
            typename Method = gumbel_method::by_default>
    class gumbel {
    public:
        using method_type = Method;
        using result_type = RealType;

        gumbel(): gumbel((RealType)0.0, (RealType)1.0){}
        explicit gumbel(RealType a, RealType b);
        explicit gumbel(const param_type& pt);

        RealType a() const;
```

```
RealType b() const;
param_type param() const;
void param(const param_type& pt);
};
}
```

Devices supported: CPU and GPU

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

Name	Description
<code>typename RealType = float</code>	Type of the produced values. The specific values are as follows: <code>float</code> <code>double</code>
<code>typename Method = oneapi::mkl::rng::gumbel_method::by_default</code>	Generation method. The specific values are as follows: <code>oneapi::mkl::rng::gumbel_method::icdf</code> See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
<code>a</code>	<code>RealType (float, double)</code>	Displacement <code>a</code> .
<code>b</code>	<code>RealType (float, double)</code>	Scalefactor <code>b</code> .

gamma

Generates gamma distributed random values.

- [Description](#)
- [API](#)

Description

The `gamma` class object is used in the `generate` function to provide random numbers with gamma distribution that has shape parameter α , displacement `a`, and scale parameter β , where $\alpha, \beta, a \in R; \alpha > 0, \beta > 0$.

The probability density function is given by:

$$f_{\alpha,a,\beta}(x) = \begin{cases} \frac{1}{\Gamma(\alpha)\beta^\alpha} (x-a)^{\alpha-1} e^{-\frac{(x-a)}{\beta}}, & x \geq a \\ 0, & x < a \end{cases}, -\infty < x < +\infty$$

where $\Gamma(\alpha)$ is the complete gamma function.

The cumulative distribution function is as follows:

$$F_{\alpha,a,\beta}(x) = \begin{cases} \int_a^x \frac{1}{\Gamma(\alpha)\beta^\alpha} (y-a)^{\alpha-1} e^{-\frac{(y-a)}{\beta}} dy, & x \geq a \\ 0, & x < a \end{cases}, -\infty < x < +\infty$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename RealType = float,
            typename Method = gamma_method::by_default>
    class gamma {
    public:
        using method_type = Method;
        using result_type = RealType;

        gamma(): gamma((RealType)1.0, (RealType)0.0, (RealType)1.0){}
        explicit gamma(RealType alpha, RealType a, RealType beta);
        explicit gamma(const param_type& pt);

        RealType alpha() const;
        RealType a() const;
        RealType beta() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

`typename RealType = float`

Type of the produced values. The specific values are as follows:

`float`

`double`

`typename Method = oneapi::mkl::rng::gamma_method::by_default`

Generation method. The specific values are as follows:

`oneapi::mkl::rng::gamma_method::marsaglia`

`oneapi::mkl::rng::gamma_method::marsaglia
_accurate`

See brief descriptions of the methods in [Distributions Template Parameter Method](#).

Input Parameters

Name	Type	Description
alpha	RealType (float, double)	Shape α
a	RealType (float, double)	Displacement a.
beta	RealType (float, double)	Scalefactor β .

beta

Generates beta distributed random values.

- [Description](#)
- [API](#)

Description

The `beta` class object is used in the `generate` function to provide random numbers with beta distribution that has shape parameters p and q , displacement a , and scale parameter (b, β) , where p, q, a , and $\beta \in \mathbb{R}; p > 0, q > 0, \beta > 0$.

The probability density function is given by:

$$f_{p,q,a,\beta}(x) = \begin{cases} \frac{1}{B(p,q)\beta^{p+q-1}}(x-a)^{p-1}(\beta+a-x)^{q-1}, & a \leq x < a + \beta \\ 0, & x < a, x \geq a + \beta \end{cases}, -\infty < x < \infty$$

where $B(p, q)$ is the complete beta function.

The cumulative distribution function is as follows:

$$f_{p,q,a,\beta}(x) = \begin{cases} 0, & x < a \\ \int_a^x \frac{1}{B(p,q)\beta^{p+q-1}}(y-a)^{p-1}(\beta+a-y)^{q-1}dy, & a \leq x < a + \beta \\ 1, & x \geq a + \beta \end{cases}, -\infty < x < \infty$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename RealType = float,
            typename Method = beta_method::by_default>
    class beta {
    public:
        using method_type = Method; using result_type = RealType;

        beta(): beta((RealType)1.0, (RealType)1.0, (RealType)(0.0),
                    (RealType)(1.0)){} explicit beta(RealType p,
```

```

        RealType q, RealType a, RealType b);
    explicit beta(const param_type& pt);

    RealType p() const;
    RealType q() const;
    RealType a() const;
    RealType b() const;
    param_type param() const;
    void param(const param_type& pt);
};
}

```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

typename RealType = float	Type of the produced values. The specific values are as follows: float double
typename Method = oneapi::mkl::rng::beta_method::by_default	Generation method. The specific values are as follows: oneapi::mkl::rng::beta_method::cja oneapi::mkl::rng::beta_method::cja_accurate See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
p	RealType (float, double)	Shape p
q	RealType (float, double)	Shape q
a	RealType (float, double)	Displacement a.
b	RealType (float, double)	Scalefactor b.

chi_square

Generates chi-square distributed random values.

- [Description](#)
- [API](#)

Description

The `chi_square` class object is used in the `generate` function to provide random numbers with chi-square distribution and n degrees of freedom, $n \in N; n > 0$.

The probability density function is:

$$F_n(x) = \begin{cases} \frac{x^{\frac{n-2}{2}} e^{-\frac{x}{2}}}{2^{n/2} \Gamma(\frac{n}{2})} dy, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

The cumulative distribution function is:

$$F_n(x) = \begin{cases} \int_0^x \frac{y^{\frac{n-2}{2}} e^{-\frac{y}{2}}}{2^{n/2} \Gamma(\frac{n}{2})} dy, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename RealType = float,
            typename Method = chi_square_method::by_default>
    class chi_square {
    public:
        using method_type = Method;
        using result_type = RealType;

        chi_square(): chi_square(5){}
        explicit chi_square(std::int32_t n);
        explicit chi_square(const param_type& pt);

        std::int32_t n() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

typename RealType = float	Type of the produced values. The specific values are as follows: float double
typename Method = oneapi::mkl::rng::chi_square_method::by_default	Generation method. The specific values are as follows: oneapi::mkl::rng::chi_square_method::gamma_based See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
n	std::int32_t	Degrees of freedom.

gaussian_mv

Generates random numbers from multivariate normal distribution.

- [Description](#)
- [API](#)

Description

The class object is used in the `generate` function to provide random numbers with d-variate normal (Gaussian) distribution with mean (a) and variance-covariance matrix C , where $a \in \mathbb{R}^d$; C is $d \times d$ symmetric positive-definite matrix.

The probability density function is given by:

$$f_{a,C}(x) = \frac{1}{\sqrt{\det(2\pi C)}} \exp\left(-\frac{1}{2}(x - a)^T C^{-1}(x - a)\right)$$

where $x \in \mathbb{R}^d$.

Matrix C can be represented as $C = TTT^T$, where T is a lower triangular matrix - Cholesky factor of C .

API**Syntax**

```
namespace oneapi::mkl::rng {
    template<typename RealType = float, layout Layout = layout::packed,
            typename Method = gaussian_mv_method::by_default>
    class gaussian_mv {
    public:
        using method_type = Method;
        using result_type = RealType;
        static constexpr layout layout_type = Layout;

        explicit gaussian_mv(std::uint32_t dimen, sycl::span<RealType> mean,
```

```

        sycl::span<RealType> matrix);
explicit gaussian_mv(const param_type& pt);

std::uint32_t dimen() const;
sycl::span<RealType> mean() const;
sycl::span<RealType> matrix() const;
param_type param() const;
void param(const param_type& pt);
};
}

```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

Name	Description
typename RealType = float	Type of the produced values. The specific values are as follows: float double
layout Layout = layout::packed	Type of the matrix storage. The specific values are as follows: layout::packed layout::full layout::diagonal See brief descriptions of the methods in Distributions Template Parameter Method .
typename Method = oneapi::mkl::rng::gaussian_mv_method::by_default	Generation method. The specific values are as follows: oneapi::mkl::rng::gaussian_mv_method::box_muller oneapi::mkl::rng::gaussian_mv_method::box_muller2 oneapi::mkl::rng::gaussian_mv_method::icdf See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
dimen	std::uint32_t	Dimension of output random vectors

Name	Type	Description
mean	<code>sycl::span<RealType></code> (float, double)	Mean span a of dimension d.
matrix	<code>sycl::span<RealType></code> (float, double)	Variance-covariance matrix C.

NOTE

- When passing a `sycl::span` that is constructed over a user's memory to the constructor, users must manage the memory under `sycl::span` by themselves. They must not destroy the memory while data are processed.

uniform (Discrete)

Generates random numbers uniformly distributed over the interval $[a, b)$.

- [Description](#)
- [API](#)

Description

The `uniform` class object is used in `generate` functions to provide random numbers uniformly distributed over the interval $[a, b)$, where a, b are the left and right bounds of the interval respectively, and $a, b \in \mathbb{Z}; a < b$.

The probability distribution is given by:

$$P(X = k) = \frac{1}{b - a}$$

$$k \in \{a, a + 1, \dots, b - 1\}$$

The cumulative distribution function is as follows:

$$F_{a,b}(x) = \begin{cases} 0, & x < a \\ \frac{x-a+1}{b-a}, & a \leq x < b, x \in \mathbb{R} \\ 1, & x \geq b \end{cases}$$

API

Syntax

```

namespace oneapi::mkl::rng {
    template<typename Method>
    class uniform<std::(u)int32_t, Method> {
    public:
        using method_type = Method;
        using result_type = std::(u)int32_t;

        uniform(): uniform((std::(u)int32_t )(0),
                           std::is_same<Method,
                           uniform_method::standard>::value ? (1 << 23) :
                           std::numeric_limits<Type>::max()){}
        explicit uniform(std::(u)int32_t a, std::(u)int32_t b);
        explicit uniform(const param_type& pt);

        std::(u)int32_t a() const;
        std::(u)int32_t b() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}

```

Devices supported: CPU and GPU.

Include Files

- oneapi/mkl/rng.hpp

Template Parameters

typename Method =	Generation method. The specific values are as follows:
oneapi::mkl::rng::uniform_method::by_default	oneapi::mkl::rng::uniform_method::standard
	oneapi::mkl::rng::uniform_method::accurate
	See brief descriptions of the methods in Distributions Template Parameter Method .

NOTE The `oneapi::mkl::rng::uniform_method::standard` uses the s BRNG type on GPU devices. This might cause the produced numbers to have incorrect statistics (due to rounding error) when $(abs(b-a) > 2^{23} || abs(b) > 2^{23} || abs(a) > 2^{23})$. To get proper statistics for this case, use the `oneapi::mkl::rng::uniform_method::accurate` method instead.

Input Parameters

Name	Type	Description
a	std::int32_t std::uint32_t	Left bound a

Name	Type	Description
b	std::int32_t std::uint32_t	Right bound b

uniform_bits

Generates uniformly distributed bits in 32/64-bit chunks.

- [Description](#)
- [API](#)

Description

The `uniform_bits` class object is used to generate uniformly distributed bits in 32/64-bit chunks. It is designed to ensure each bit in the 32/64-bit chunk is uniformly distributed. It is not supported for all engines.

`UIntType` denotes the chunk size and can be `std::uint32_t`, `std::uint64_t`. See [VS Notes](#) for details.

API

Syntax

```
namespace oneapi::mkl::rng {
template<typename UIntType = std::uint32_t>
class uniform_bits {
    using result_type = UIntType
}
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

typename UIntType = std::uint32_t	Type of the produced values. The specific values are as follows:
	std::uint32_t
	std::uint64_t

bits

Generates bits of underlying engine (BRNG) integer recurrence.

- [Description](#)
- [API](#)

Description

The `bits` class object is used to generate integer random values. Each integer can be treated as a vector of several bits. In a truly random generator, these bits are random, while in pseudorandom generators this randomness can be violated.

For example, a drawback of linear congruential generators is that lower bits are less random than higher bits (for example, see [Knuth81]). For this reason, exercise care when using this function. Typically, in a 32-bit LCG only 24 higher bits of an integer value can be considered random. See [VS Notes](#) for details.

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename UIntType = std::uint32_t>
    class bits {
        using result_type = UIntType
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

Name	Description
typename UIntType = std::uint32_t	Type of the produced values. The specific values are as follows: std::uint32_t

bernoulli

Generates Bernoulli distributed random values.

- [Description](#)
- [API](#)

Description

The `bernoulli` class object is used in the `generate` function to provide Bernoulli distributed random numbers with probability p of a single trial success, where $p \in R; 0 \leq p \leq 1$.

A variate is called Bernoulli distributed if after a trial it is equal to 1 with probability of success p and to 0 with probability $1 - p$.

The probability distribution is given by:

$$P(X=1) = p$$

$$P(X=0) = 1 - p$$

The cumulative distribution function is as follows:

$$F_p(x) = \begin{cases} 0, & x < 0 \\ 1 - p, & 0 \leq x < 1, x \in R \\ 1, & x \geq 1 \end{cases}$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename IntType = std::int32_t,
            typename Method = bernoulli_method::by_default>
    class bernoulli {
    public:
        using method_type = Method;
        using result_type = IntType;

        bernoulli(): bernoulli(0.5f){}
        explicit bernoulli(float p);
        explicit bernoulli(const param_type& pt);

        float p() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

`typename IntType = std::int32_t`

Type of the produced values. The specific values are as follows:

`std::int32_t`

`std::uint32_t`

`typename Method = oneapi::mkl::rng::bernoulli_method::by_default`

Generation method. The specific values are as follows:

`oneapi::mkl::rng::bernoulli_method::icdf`

See brief descriptions of the methods in [Distributions Template Parameter Method](#).

Input Parameters

Name	Type	Description
p	float	Success probability p of a trial.

geometric

Generates geometrically distributed random values.

- [Description](#)
- [API](#)

Description

The `geometric` class object is used in the `generate` function to provide geometrically distributed random numbers with probability p of a single trial success, where $p \in R, 0 < p < 1$.

A geometrically distributed variate represents the number of independent Bernoulli trials preceding the first success. The probability of a single Bernoulli trial success is p .

The probability distribution is given by:

$$P(X = k) = p(1 - p)^k, k \in \{0, 1, 2, \dots\}$$

The cumulative distribution function is as follows:

$$F_p(x) = \begin{cases} 0, & x < 0 \\ 1 - (1 - p)^{\lfloor x+1 \rfloor}, & x \geq 0 \end{cases}, x \in R$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API**Syntax**

```
namespace oneapi::mkl::rng {
    template<typename IntType = std::int32_t,
            typename Method = geometric_method::by_default>
    class geometric {
    public:
        using method_type = Method;
        using result_type = IntType;

        geometric(): geometric(0.5){}
        explicit geometric(float p);
        explicit geometric(const param_type& pt);

        float p() const;
        param_type param() const;
```

```
void param(const param_type& pt);
};
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

Name	Description
typename IntType = <code>std::int32_t</code>	Type of the produced values. The specific values are as follows: <code>std::int32_t</code> <code>std::uint32_t</code>
typename Method = <code>oneapi::mkl::rng::geometric_method::by_default</code>	Generation method. The specific values are as follows: <code>oneapi::mkl::rng::geometric_method::icdf</code> See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
<code>p</code>	<code>float</code>	Success probability <code>p</code> of a trial.

binomial

Generates binomially distributed random numbers.

- [Description](#)
- [API](#)

Description

The `binomial` class object is used in the `generate` function to provide binomially distributed random numbers with number of independent Bernoulli trials `m`, and with probability `p` of a single trial success, where $p \in R; 0 \leq p \leq 1; m \in N$.

A binomially distributed variate represents the number of successes in `m` independent Bernoulli trials with probability of a single trial success `p`.

The probability distribution is given by:

$$P(X = k) = C_m^k p^k (1 - p)^{m-k}, k \in \{0, 1, \dots, m\}$$

The cumulative distribution function is as follows:

$$F_{m,p}(x) = \begin{cases} 0, & x < 0 \\ \sum_{k=0}^{\lfloor x \rfloor} C_m^k p^k (1-p)^{m-k}, & 0 \leq x < m \\ 1, & x > m \end{cases}, x \in R$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename IntType = std::int32_t,
            typename Method = binomial_method::by_default>
    class binomial {
    public:
        using method_type = Method;
        using result_type = IntType;

        binomial(): binomial(5, 0.5){}
        explicit binomial(std::int32_t ntrial, double p);
        explicit binomial(const param_type& pt);

        std::int32_t ntrial() const;
        double p() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

Name	Description
<code>typename IntType = std::int32_t</code>	Type of the produced values. The specific values are as follows: <code>std::int32_t</code> <code>std::uint32_t</code>
<code>typename Method = oneapi::mkl::rng::binomial_method::by_default</code>	Generation method. The specific values are as follows: <code>oneapi::mkl::rng::binomial_method::btpe</code> See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
ntrials	std::int32_t	Number of independent trials.
p	double	Success probability p of a single trial.

hypergeometric

Generates hypergeometrically distributed random values.

- [Description](#)
- [API](#)

Description

The `hypergeometric` class object is used in the `generate` function to provide hypergeometrically distributed random values with lot size l , size of sampling s , and number of marked elements in the lot m , where

$$l, m, s \in N \cup \{0\}; l \geq \max(s, m).$$

Consider a lot of l elements comprising m “marked” and $l-m$ “unmarked” elements. A trial sampling without replacement of exactly s elements from this lot helps to define the hypergeometric distribution, which is the probability that the group of s elements contains exactly k marked elements.

The probability distribution is given by:

$$P(X = k) = \frac{C_m^k C_{l-m}^{s-k}}{C_l^s}$$

$$, k \in \{\max(0, s + m - l), \dots, \min(s, m)\}$$

The cumulative distribution function is as follows:

$$F_{l,s,m}(x) = \begin{cases} 0, & x < \max(0, s + m - l) \\ \sum_{k=\max(0, s+m-l)}^{\lfloor x \rfloor} \frac{C_m^k C_{l-m}^{s-k}}{C_l^s}, & \max(0, s + m - l) \leq x \leq \min(s, m) \\ 1, & x > \min(s, m) \end{cases}$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```

namespace oneapi::mkl::rng {
    template<typename IntType = std::int32_t,
            typename Method = hypergeometric_method::by_default>
    class hypergeometric {
    public:
        using method_type = Method;
        using result_type = IntType;

        hypergeometric(): hypergeometric(1, 1, 1){}
        explicit hypergeometric(std::int32_t l, std::int32_T s, std::int32_T m);
        explicit hypergeometric(const param_type& pt);

        std::int32_t s() const;
        std::int32_t m() const;
        std::int32_t l() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}

```

Devices supported: CPU and GPU

Include Files

- oneapi/mkl/rng.hpp

Template Parameters

typename IntType = std::int32_t	Type of the produced values. The specific values are as follows: std::int32_t std::uint32_t
typename Method = oneapi::mkl::rng::hypergeometric_method:: by_default	Generation method. The specific values are as follows: oneapi::mkl::rng::hypergeometric_method::h2pe See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
l	std::int32_t	Lot size of l.
s	std::int32_t	Size of sampling without replacement.
m	std::int32_t	Number of marked elements m.

poisson

Generates Poisson distributed random values.

- [Description](#)
- [API](#)

Description

The `poisson` class object is used in the `generate` function to provide Poisson distributed random numbers with distribution parameter λ , where $\lambda \in R, \lambda > 0$.

The probability distribution is given by:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

$k \in \{0, 1, 2, \dots\}$. The cumulative distribution function is as follows:

$$F_{\lambda}(x) = \begin{cases} \sum_{k=0}^{\lfloor x \rfloor} \frac{\lambda^k e^{-\lambda}}{k!}, & x \geq 0 \\ 0, & x < 0 \end{cases}, x \in R$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename IntType = std::int32_t,
            typename Method = poisson_method::by_default>
    class poisson {
    public:
        using method_type = Method;
        using result_type = IntType;

        poisson(): poisson(0.5){} explicit poisson(double lambda);
        explicit poisson(const param_type& pt);

        double lambda() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

<code>typename IntType = std::int32_t</code>	Type of the produced values. The specific values are as follows: <code>std::int32_t</code> <code>std::uint32_t</code>
<code>typename Method = oneapi::mkl::rng::poisson_method::by_default</code>	Generation method. The specific values are as follows: <code>oneapi::mkl::rng::poisson_method::ptpe</code> <code>oneapi::mkl::rng::poisson_method::gaussian_icdf_based</code> See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
<code>lambda</code>	<code>double</code>	Distribution parameter λ .

`poisson_v`

Generates Poisson distributed random values with varying mean.

- [Description](#)
- [API](#)

Description

The `poisson_v` class object is used in the `generate` function to provide n Poisson distributed random numbers $x_i (i = 1, \dots, n)$ with distribution parameter λ_i , where $\lambda_i \in R, \lambda_i > 0$.

The probability distribution is given by:

$$P(X_i = k) = \frac{\lambda_i^k \exp(-\lambda_i)}{k!}, k \in \{0, 1, 2, \dots\}$$

The cumulative distribution function is as follows:

$$F_{\lambda_i}(x) = \begin{cases} \sum_{k=0}^{\lfloor x \rfloor} \frac{\lambda_i^k e^{-\lambda_i}}{k!}, & x \geq 0 \\ 0, & x < 0 \end{cases}, x \in R$$

Product and Performance Information

Performance varies by use, configuration, and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API**Syntax**

```
namespace oneapi::mkl::rng {
    template<typename IntType = std::int32_t,
            typename Method = poisson_v_method::by_default>
    class poisson_v {
    public:
        using method_type = Method;
        using result_type = IntType;

        explicit poisson_v(sycl::span<double> lambda);
        explicit poisson_v(const param_type& pt);

        sycl::span<double> lambda() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU

Include Files

- oneapi/mkl/rng.hpp

Template Parameters

typename IntType = std::int32_t

Type of the produced values. The specific values are as follows:

std::int32_t

std::uint32_t

typename Method =
oneapi::mkl::rng::poisson_v_method::
by_default

Generation method. The specific values are as follows:

oneapi::mkl::rng::poisson_v_method::gauss
ian_icdf_based

See brief descriptions of the methods in [Distributions Template Parameter Method](#).

Input Parameters

Name	Type	Description
lambda	sycl::span<double>	Array of n distribution parameters λ .

NOTE

- When passing a `sycl::span` that is constructed over a user's memory to the constructor, users must manage the memory under `sycl::span` by themselves. They must not destroy the memory while data are processed.

negative_binomial

Generates random numbers with negative binomial distribution.

- [Description](#)
- [API](#)

Description

The `negative_binomial` class object is used in the `generate` function to provide random numbers with negative binomial distribution and distribution parameters a and p , where $p, a \in R; 0 < p < 1; a > 0$.

If the first distribution parameter $a \in N$, this distribution is the same as Pascal distribution. If $a \in N$, the distribution can be interpreted as the expected time of a -th success in a sequence of Bernoulli trials, when the probability of success is p .

The probability distribution is given by:

$$P(X = k) = C_{a+k-1}^k p^a (1 - p)^k, k \in \{0, 1, 2, \dots\}$$

The cumulative distribution function is as follows:

$$F_{a,p}(x) = \begin{cases} \sum_{k=0}^{\lfloor x \rfloor} C_{a+k-1}^k p^a (1 - p)^k, & x \geq 0 \\ 0, & x < 0 \end{cases}, x \in R$$

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API**Syntax**

```
namespace oneapi::mkl::rng {
    template<typename IntType = std::int32_t,
            typename Method = negative_binomial_method::by_default>
    class negative_binomial {
    public:
        using method_type = Method;
        using result_type = IntType;

        negative_binomial(): negative_binomial(0.1, 0.5){}
        explicit negative_binomial(double a, double p);
        explicit negative_binomial(const param_type& pt);

        double a() const;
```

```
double p() const;
param_type param() const;
void param(const param_type& pt);
};
}
```

Devices supported: CPU and GPU

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

<code>typename IntType = std::int32_t</code>	Type of the produced values. The specific values are as follows: <code>std::int32_t</code> <code>std::uint32_t</code>
<code>typename Method = oneapi::mkl::rng::negative_binomial_method::by_default</code>	Generation method. The specific values are as follows: <code>oneapi::mkl::rng::negative_binomial_method::nbar</code> See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
a	double	The first distribution parameter a.
p	double	The second distribution parameter p.

multinomial

Generates multinomially distributed random numbers.

- [Description](#)
- [API](#)

Description

The `multinomial` class object is used in the `generate` function to provide multinomially distributed random numbers with `ntrial` independent trials and `k` possible mutually exclusive outcomes, with corresponding probabilities p_i , where $p_i \in R; 0 \leq p_i \leq 1; m, n \in N$.

The probability distribution is given by:

$$P(X_1 = x_1, \dots, X_k = x_k) = \frac{m!}{\prod_{i=1}^k x_i!} \prod_{i=1}^k p_i^{x_i}, 0 \leq x_i \leq m, \sum_{i=1}^k x_i = m$$

Product and Performance Information

Performance varies by use, configuration, and other factors. Learn more at <https://www.intel.com/PerformanceIndex>. Notice revision #20201201

API

Syntax

```
namespace oneapi::mkl::rng {
    template<typename IntType = std::int32_t,
            typename Method = multinomial_method::by_default>
    class multinomial {
    public:
        using method_type = Method;
        using result_type = IntType;

        explicit multinomial(double ntrial, sycl::span<double> p);
        explicit multinomial(const param_type& pt);

        std::int32_t ntrial() const;
        sycl::span<double> p() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Devices supported: CPU and GPU

NOTE This distribution supports only input parameters (on GPU) that follow the condition $(k \geq ntrial * 16 \wedge ntrial \leq 16)$, where k is the probability vector length.

Include Files

- `oneapi/mkl/rng.hpp`

Template Parameters

<code>typename IntType = std::int32_t</code>	Type of the produced values. The specific values are as follows: <code>std::int32_t</code> <code>std::uint32_t</code>
<code>typename Method = oneapi::mkl::rng::multinomial_method::by_default</code>	Generation method. The specific values are as follows: <code>oneapi::mkl::rng::multinomial_method::poisson_icdf_based</code> See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
ntrial	std::int32_t	Number of independent trials m .
p	sycl::span<double>	Probability vector of possible outcomes (length is k).

NOTE

- When passing a `sycl::span` that is constructed over a user's memory to the constructor, users must manage the memory under `sycl::span` by themselves. They must not destroy the memory while data are processed.

Random Number Generators Device Routines

The main purpose of Device routines is to make them callable from your DPC++ kernels; however, there are no limitations to be called from the Host. For example:

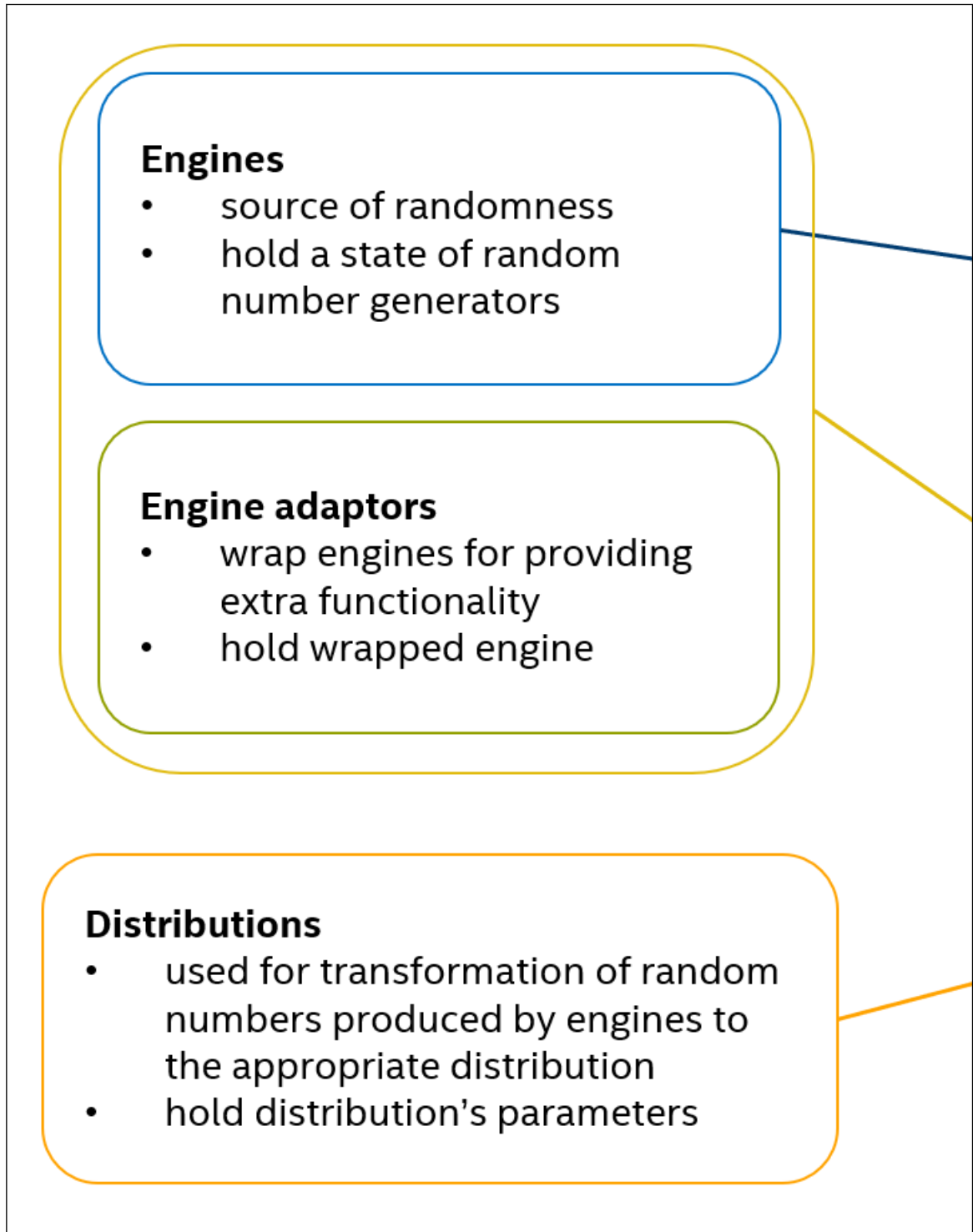
```
sycl::queue queue;

queue.submit([&](sycl::handler& cgh) {
    cgh.parallel_for(range, [=](...) {
        oneapi::mkl::rng::device::routine(...); // calling routine from user's kernel code
    });
});

oneapi::mkl::rng::device::routine(...); // calling routine from host
```

RNG Device APIs Structure

RNG Device API Structure



oneMKL RNG Device Usage Model

- [Example of Scalar Random Numbers Generation](#)

- [Example of Vector Random Numbers Generation](#)
- [Example of Random Numbers Generation by Engines Stored in `sycl::buffer`](#)
- [Example of Random Numbers Generation with Host-side Helpers Usage](#)
- [Example of Random Numbers Generation Using Gamma Distribution](#)

A typical usage model for device routines is the same as described in [oneMKL RNG Usage Model](#):

1. Create and initialize the object for basic random number generator.
2. Create and initialize the object for distribution generator.
3. Call the generate routine to get random numbers with appropriate statistical distribution.

Example of Scalar Random Numbers Generation

```
#include <iostream>
#include <sycl/sycl.hpp>
#include "oneapi/mkl/rng/device.hpp"

int main() {
    sycl::queue queue;
    const int n = 1000;
    const int seed = 1;
    // Prepare an array for random numbers
    std::vector<float> r(n);

    sycl::buffer<float, 1> r_buf(r.data(), r.size());
    // Submit a kernel to generate on device
    queue.submit([&](sycl::handler& cgh) {
        auto r_acc = r_buf.template get_access<sycl::access::mode::write>(cgh);
        cgh.parallel_for(sycl::range<1>(n), [=](sycl::item<1> item) {
            // Create an engine object
            oneapi::mkl::rng::device::philox4x32x10<> engine(seed, item.get_id(0));
            // Create a distribution object
            oneapi::mkl::rng::device::uniform<> distr;
            // Call generate function to obtain scalar random number
            float res = oneapi::mkl::rng::device::generate(distr, engine);

            r_acc[item.get_id(0)] = res;
        });
    });

    auto r_acc = r_buf.template get_access<sycl::access::mode::read>();
    std::cout << "Samples of uniform distribution" << std::endl;
    for(int i = 0; i < 10; i++) {
        std::cout << r_acc[i] << std::endl;
    }

    return 0;
}
```

Example of Vector Random Numbers Generation

```
#include <iostream>
#include <sycl/sycl.hpp>
```

```
#include "oneapi/mkl/rng/device.hpp"

int main() {
    sycl::queue queue;
    const int n = 1000;
    const int seed = 1;
    const int vec_size = 4;
    // Prepare an array for random numbers
    std::vector<float> r(n);

    sycl::buffer<float, 1> r_buf(r.data(), r.size());
    // Submit a kernel to generate on device
    sycl::queue{}.submit([&](sycl::handler& cgh) {
        auto r_acc = r_buf.template get_access<sycl::access::mode::write>(cgh);
        cgh.parallel_for(sycl::range<1>(n / vec_size), [=](sycl::item<1> item) {
            // Create an engine object
            oneapi::mkl::rng::device::philox4x32x10<vec_size> engine(seed, item.get_id(0) *
vec_size);
            // Create a distribution object
            oneapi::mkl::rng::device::uniform<> distr;
            // Call generate function to obtain sycl::vec<float, 4> with random numbers
            auto res = oneapi::mkl::rng::device::generate(distr, engine);

            res.store(item.get_id(0), r_acc);
        });
    });

    auto r_acc = r_buf.template get_access<sycl::access::mode::read>();
    std::cout << "Samples of uniform distribution" << std::endl;
    for(int i = 0; i < 10; i++) {
        std::cout << r_acc[i] << std::endl;
    }

    return 0;
}
```

There is an opportunity to store engines between kernels manually via `sycl::buffer` / USM pointers or by using a specific host-side helper class called, engine descriptor.

Example of Random Numbers Generation by Engines Stored in `sycl::buffer`

Engines are initialized in the first kernel. Random number generation is performed in the second kernel.

```
#include <iostream>
#include <sycl/sycl.hpp>

#include "oneapi/mkl/rng/device.hpp"

int main() {
    sycl::queue queue;
    const int n = 1000;
    const int seed = 1;
```

```

const int vec_size = 4;
// Prepare an array for random numbers
std::vector<float> r(n);
sycl::buffer<float, 1> r_buf(r.data(), r.size());
sycl::range<1> range(n / vec_size);
sycl::buffer<oneapi::mkl::rng::device::mrg32k3a<vec_size>, 1> engine_buf(range);

sycl::queue queue;

// Kernel with initialization of engines
queue.submit([&](sycl::handler& cgh) {
    // Create an accessor to sycl::buffer with engines to write initialized states
    auto engine_acc = engine_buf.template get_access<sycl::access::mode::write>(cgh);
    cgh.parallel_for(range, [=](sycl::item<1> item) {
        size_t id = item.get_id(0);
        // Create an engine object with offset id * 2^64
        oneapi::mkl::rng::device::mrg32k3a<vec_size> engine(seed, {0, id});
        engine_acc[id] = engine;
    });
});
// Kernel for random numbers generation
queue.submit([&](sycl::handler& cgh) {
    auto r_acc = r_buf.template get_access<sycl::access::mode::write>(cgh);
    // Create an accessor to sycl::buffer with engines to read initialized states
    auto engine_acc = engine_buf.template get_access<sycl::access::mode::read>(cgh);
    cgh.parallel_for(range, [=](sycl::item<1> item) {
        size_t id = item.get_id(0);

        auto engine = engine_acc[id];
        oneapi::mkl::rng::device::uniform distr;
        auto res = oneapi::mkl::rng::device::generate(distr, engine);

        res.store(id, r_acc);
    });
});

auto r_acc = r_buf.template get_access<sycl::access::mode::read>();
std::cout << "Samples of uniform distribution" << std::endl;
for(int i = 0; i < 10; i++) {
    std::cout << r_acc[i] << std::endl;
}

return 0;
}

```

Example of Random Numbers Generation with Host-side Helpers Usage

```

#include <iostream>
#include <sycl/sycl.hpp>

#include "oneapi/mkl/rng/device.hpp"

```

```

int main() {
    sycl::queue queue;
    const int n = 1000;
    const int seed = 1;
    const int vec_size = 4;
    // prepare array for random numbers
    std::vector<float> r(n);
    sycl::buffer<float, 1> r_buf(r.data(), r.size());
    sycl::range<1> range(n / vec_size);

    // offset of each engine in engine_descriptor
    int offset = vec_size;
    // each engine would be created in enqueued task as of specified range
    // as oneapi::mkl::rng::device::mrg32k3a<vec_size>(seed, id * offset);
    oneapi::mkl::rng::device::engine_descriptor<oneapi::mkl::rng::device::mrg32k3a<vec_size>>
    descr(queue, range, seed, offset);
    queue.submit([&](sycl::handler& cgh) {
        auto r_acc = r_buf.template get_access<sycl::access::mode::write>(cgh);
        // create engine_accessor
        auto engine_acc = descr.get_access(cgh);
        cgh.parallel_for(range, [=](sycl::item<1> item) {
            size_t id = item.get_id(0);
            // load engine from engine_accessor
            auto engine = engine_acc.load(id);
            oneapi::mkl::rng::device::uniform<Type> distr;

            auto res = oneapi::mkl::rng::device::generate(distr, engine);

            res.store(id, r_acc);
            // store engine for further calculations if needed
            engine_acc.store(engine, id);
        });
    });

    auto r_acc = r_buf.template get_access<sycl::access::mode::read>();
    std::cout << "Samples of uniform distribution" << std::endl;
    for(int i = 0; i < 10; i++) {
        std::cout << r_acc[i] << std::endl;
    }

    return 0;
}

```

Example of Random Numbers Generation Using Gamma Distribution

```

#include <iostream>
#include <vector>
#include <sycl/sycl.hpp>
#include "oneapi/mkl/rng/device.hpp"

constexpr size_t seed = 42;

```

```

int main() {
    sycl::queue queue;
    const size_t n = 10'000;

    // create USM allocator
    sycl::usm_allocator<double, sycl::usm::alloc::shared> allocator(queue);

    // create vector with USM allocator
    std::vector<double, decltype(allocator)> r_vec(n, allocator);
    double* r = r_vec.data();

    size_t reduced_val = 0;
    // create a reductor to calculate the sum of rejected numbers across all work-items
    auto reductor = sycl::reduction(&reduced_val, size_t(0), std::plus<size_t>{});

    queue.parallel_for(n, reductor, [=](size_t id, auto& sum) {
        // create basic random number generator object
        // 2^76 numbers for each work-item are skipped to allow acceptance rejection scheme to
        // reject as much numbers as it needs
        oneapi::mkl::rng::device::mrg32k3a<1> engine({ seed, seed, seed, seed, seed, seed }, { 0,
(4096 * id) });
        // create distribution object
        oneapi::mkl::rng::device::gamma<double> distr(0.7, 0.0, 1.0);
        // perform generation
        auto res = oneapi::mkl::rng::device::generate(distr, engine);
        r[id] = res;

        // calculate how much random numbers are rejected in this work-item
        size_t rej = distr.count_rejected_numbers();
        sum += rej;
    }).wait(); // synchronization can be also done by queue.wait()

    std::cout << "First 10 gamma distributed numbers are following:" << std::endl;
    for(int i = 0; i < 10; i++) {
        std::cout << r[i] << " ";
    }
    std::cout << std::endl;

    std::cout << "rejected " << reduced_val << " random numbers" << std::endl;

    return 0;
}

```

Additionally, examples that demonstrate usage of random number generators device routines are available in:

```
{MKL}/examples/dpcpp_device/rng/source
```

Device Generate Routines

Use the `generate` or `generate_single` routines to obtain random numbers from a given engine with proper statistics of a given distribution.

`generate`

Entry point to obtain random numbers from a given engine with proper statistics of a given distribution.

- [API](#)

API

Syntax

```
namespace oneapi::mkl::rng::device {  
    template<typename Distr, typename Engine>  
    auto generate(Distr& distr, Engine& engine) ->  
        typename std::conditional<Engine::vec_size == 1, typename Distr::result_type,  
            sycl::vec<typename Distr::result_type, Engine::vec_size>>::type  
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Input Parameters

Name	Type	Description
distr	Distr&	Distribution object. See Device Distributions for details.
engine	Engine&	Engine object. See Device Engines (Basic Random Number Generators) for details.

Output Parameters

Name	Type	Description
result	<code>sycl::vec<typename Distr::result_type, Engine::vec_size></code> or <code>typename Distr::result_type</code>	Function returns <code>sycl::vec</code> of type specified by the Distribution object and vector size specified by the Engine object filled with random numbers or a scalar random number in case <code>vec_size = 1</code> .

[generate_single](#)

Entry point to obtain a single random number from a given vector engine with proper statistics of a given distribution.

- [API](#)

API

Syntax

```
namespace oneapi::mkl::rng::device {  
    template<typename Distr, typename Engine>  
    typename Distr::result_type generate_single(Distr& distr, Engine& engine)  
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Input Parameters

Name	Type	Description
distr	Distr&	Distribution object. See Device Distributions for details.
engine	Engine&	Engine object. See Device Engines (Basic Random Number Generators) for details.

Output Parameters

Name	Type	Description
result	<code>typename Distr::result_type</code>	Function returns a scalar random number, may be used for engines with <code>vec_size > 1</code> .

Device Engines (Basic Random Number Generators)

Intel® oneAPI Math Kernel Library (oneMKL) RNG provides the following device pseudorandom number generators:

Routine	Description
mrg32k3a	The combined multiple recursive pseudorandom number generator MRG32k3a [L'Ecuyer99].
philox4x32x10	Philox4x32-10 counter-based pseudorandom number generator with a period of 2^{128} PHILOX4X32X10 [Salmon11].
mcg31m1	The 31-bit multiplicative congruential pseudorandom number generator MCG($1132489760, 2^{31} - 1$) [L'Ecuyer99a].
mcg59	The 59-bit multiplicative congruential pseudorandom number generator MCG($13^{13}, 2^{59}$) from NAG Numerical Libraries [NAG].
pcg64_dxsm	The permuted congruential pseudorandom number generator with a period of 2^{128} PCG64 DXSM [pcg2014].

[mrg32k3a](#)

- [Description](#)
- [API](#)

Description

The combined multiple recursive pseudorandom number generator MRG32k3a [[L'Ecuyer99a](#)].

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<std::int32_t VecSize = 1>
    class mrg32k3a {
    public:
        static constexpr std::uint32_t default_seed = 1;
        static constexpr std::int32_t vec_size = VecSize;

        mrg32k3a() : mrg32k3a(default_seed) {}
        mrg32k3a(std::uint32_t seed, std::uint64_t offset = 0);
        mrg32k3a(std::initializer_list<std::uint32_t> seed, std::uint64_t offset = 0);
        mrg32k3a(std::uint32_t seed, std::initializer_list<std::uint64_t> offset);
        mrg32k3a(std::initializer_list<std::uint32_t> seed, std::initializer_list<std::uint64_t>
offset);
    };
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

Name	Type	Description
VecSize	<code>std::int32_t</code>	Describes the size of the vector that will be produced by the generate function with this engine. VecSize values can be 1, 2, 3, 4, 8, or 16 as the <code>sycl::vec</code> class size. By default, VecSize = 1; for this case, the generate function returns a single random number.

Constructors Input Parameters

Name	Type	Description
seed	<code>std::uint32_t</code> <code>std::initializer_list<std::uint32_t></code>	Initial conditions of the engine state.
offset	<code>std::uint64_t</code> <code>std::initializer_list<std::uint64_t></code>	number of skipped elements, for <code>initializer_list</code> offset is calculated as: <code>num_to_skip[0] + num_to_skip[1]*2⁶⁴ + num_to_skip[2]*2¹²⁸ + ... + num_to_skip[n-1]*2⁶⁴*(n-1)</code> .

See [VS Notes](#) for the detailed descriptions.

philox4x32x10

- [Description](#)

- [API](#)

Description

A Philox4x32-10 counter-based pseudorandom number generator [[Salmon11](#)].

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<std::int32_t VecSize = 1>
    class philox4x32x10 {
    public:
        static constexpr std::uint64_t default_seed = 1;
        static constexpr std::int32_t vec_size = VecSize;

        philox4x32x10() : philox4x32x10(default_seed) {}
        philox4x32x10(std::uint64_t seed, std::uint64_t offset = 0);
        philox4x32x10(std::initializer_list<std::uint64_t> seed, std::uint64_t offset = 0);
        philox4x32x10(std::uint64_t seed, std::initializer_list<std::uint64_t> offset);
        philox4x32x10(std::initializer_list<std::uint64_t> seed,
            std::initializer_list<std::uint64_t> offset);
    };
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

Name	Type	Description
VecSize	<code>std::int32_t</code>	Describes the size of the vector that will be produced by the generate function with this engine. VecSize values can be 1, 2, 3, 4, 8, or 16 as the <code>sycl::vec</code> class size. By default, VecSize = 1; for this case, the generate function returns a single random number.

Constructors Input Parameters

Name	Type	Description
seed	<code>std::uint64_t</code> <code>std::initializer_list<std::uint64_t></code>	Initial conditions of the engine state.
offset	<code>std::uint64_t</code> <code>std::initializer_list<std::uint64_t></code>	Number of skipped elements, for <code>initializer_list</code> offset is calculated as: <code>num_to_skip[0]+</code>

Name	Type	Description
		$\text{num_to_skip}[1] * 2^{64} +$ $\text{num_to_skip}[2] * 2^{128} + \dots$ $+ \text{num_to_skip}[n-1] * 2^{64} * (n-1).$

See [VS Notes](#) for detailed descriptions.

mcg31m1

- [Description](#)
- [API](#)

Description

The 31-bit multiplicative congruential pseudorandom number generator MCG(1132489760, $2^{32}-1$) [\[L'Ecuyer99a\]](#).

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<std::int32_t VecSize = 1>
    class mcg31m1 {
    public:
        static constexpr std::uint32_t default_seed = 1;
        static constexpr std::int32_t vec_size = VecSize;

        mcg31m1() : mcg31m1(default_seed) {}
        mcg31m1(std::uint32_t seed, std::uint64_t offset = 0);
        mcg31m1(std::initializer_list<std::uint32_t> seed, std::uint64_t offset = 0);
    };
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

Name	Type	Description
VecSize	<code>std::int32_t</code>	Describes the size of the vector that will be produced by the generate function with this engine. VecSize values can be 1, 2, 3, 4, 8, or 16 as the <code>sycl::vec</code> class size. By default, VecSize = 1; for this case, the generate function returns a single random number.

Constructors Input Parameters

Name	Type	Description
seed	std::uint32_t std::initializer_list<std::uint32_t>	Initial conditions of the engine state.
offset	std::uint64_t	Number of skipped elements.

See [VS Notes](#) for detailed descriptions.

mcg59

- [Description](#)
- [API](#)

Description

The 59-bit multiplicative congruential pseudorandom number generator MCG(13^{13} , 2^{59}) from NAG Numerical Libraries [\[NAG\]](#).

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<std::int32_t VecSize = 1>
    class mcg59 {
    public:
        static constexpr std::uint32_t default_seed = 1;
        static constexpr std::int32_t vec_size = VecSize;

        mcg59() : mcg59(default_seed) {}
        mcg59(std::uint32_t seed, std::uint64_t offset = 0);
        mcg59(std::initializer_list<std::uint32_t> seed, std::uint64_t offset = 0);
    };
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

Name	Type	Description
VecSize	std::int32_t	Describes the size of the vector that will be produced by the generate function with this engine. VecSize values can be 1, 2, 3, 4, 8, or 16 as the <code>sycl::vec</code> class size. By default, VecSize = 1; for this case, the generate function returns a single random number.

Constructors Input Parameters

Name	Type	Description
seed	<code>std::uint32_t</code> <code>std::initializer_list<std::uint32_t></code>	Initial conditions of the engine state.
offset	<code>std::uint64_t</code>	Number of skipped elements.

See [VS Notes](#) for detailed descriptions.

pcg64_dxsm

- [Description](#)
- [Generation algorithm](#)
- [API](#)

Description

A permuted congruential pseudorandom number generator PCG64 DXSM with a period of 2^{128} [\[pcg2014\]](#).

Generation algorithm

$$x_n = x_{n-1} \cdot a + b$$

$$hi_n = x_n \gg 64$$

$$lo_n = x_n \wedge (2^{64} - 1) \vee 1$$

$$hi_n = hi_n \oplus (hi_n \gg 32)$$

$$hi_n = hi_n \cdot a$$

$$hi_n = hi_n \oplus (hi_n \gg 48)$$

$$u_n = hi_n \cdot lo_n$$

$$a = 0xDA942042E4DD58B5, b = 0x5851F42D4C957F2D14057B7EF767814F$$

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<std::int32_t VecSize = 1>
    class pcg64_dxsm {
    public:
        static constexpr std::uint64_t default_seed = 0;
        static constexpr std::int32_t vec_size = VecSize;

        pcg64_dxsm() : pcg64_dxsm(default_seed) {}
        pcg64_dxsm(std::uint64_t seed, std::uint64_t offset = 0);
        pcg64_dxsm(std::initializer_list<std::uint64_t> seed, std::uint64_t offset = 0);
        pcg64_dxsm(std::uint64_t seed, std::initializer_list<std::uint64_t> offset);
        pcg64_dxsm(std::initializer_list<std::uint64_t> seed, std::initializer_list<std::uint64_t>
offset);
    };
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

Name	Type	Description
VecSize	<code>std::int32_t</code>	Describes the size of the vector that will be produced by the <code>generate</code> function with this engine. VecSize values can be 1, 2, 3, 4, 8, or 16 as the <code>sycl::vec</code> class size. By default, VecSize = 1; for this case, the <code>generate</code> function returns a single random number.

Constructors Input Parameters

Name	Type	Description
seed	<code>std::uint64_t</code> <code>std::initializer_list<std::uint64_t></code>	Initial conditions of the 128 bit engine state.
offset	<code>std::uint64_t</code> <code>std::initializer_list<std::uint64_t></code>	Number of skipped elements, for <code>initializer_list</code> offset is calculated as: <code>num_to_skip[0] + num_to_skip[1]*2⁶⁴</code> .

See [VS Notes](#) for detailed descriptions.

Host-Side Helpers

Intel® oneAPI Math Kernel Library (oneMKL) RNG provides host-side helper-class templates for initializing engines.

Routine	Description
engine_descriptor	Provide an abstraction over <code>sycl::buffer</code> to initialize and store engines' states between DPC++ kernels.
engine_accessor	Provide an abstraction over <code>sycl::accessor</code> to access engines' which are hold by <code>engine_descriptor</code> in kernels.

[engine_descriptor](#)

Host side helper to provide an abstraction over `sycl::buffer` to initialize and store engines' states between SYCL kernels.

- [Description](#)
- [API](#)

Description

`engine_descriptor` provides an abstraction over `sycl::buffer` to initialize and store engines' states between SYCL kernels.

NOTE The constructor of the engine descriptor submits a `parallel_for` task to a given queue to initialize engines' states: Each engine is initialized as `Engine {seed, id * offset}` for scalar offset case, where `id` is a value from 0 to `range.size()`.

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<Engine>
    class engine_descriptor {
    public:
        engine_descriptor(sycl::queue& queue, sycl::range<1> range, std::uint64_t seed,
std::uint64_t offset);
        template<typename InitEngineFunc>
        engine_descriptor(sycl::queue& queue, sycl::range<1> range, InitEngineFunc func);
        engine_accessor<Engine> get_access(sycl::handler& cgh);
    };
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

Type	Description
Engine	Specify an engine which state is hold by engine descriptor.

Constructors Input Parameters

Name	Type	Description
queue	<code>sycl::queue&</code>	<code>sycl::queue</code> object. Task is submitted to this queue to initialize engines' states directly on the device which is associated with queue.
range	<code>sycl::range<1></code>	Describes number of engines which are hold by this <code>en_descriptor</code> object.
seed	<code>std::uint64_t</code> <code>std::initializer_list<std::uint64_t></code>	Initial conditions of the engine state.
offset	<code>std::uint64_t</code> <code>std::initializer_list<std::uint64_t></code>	Number of skipped elements, for <code>initializer_list</code> offset is calculated as <code>num_to_skip[0]+</code>

Name	Type	Description
func	InitEngineFunc	$\text{num_to_skip}[1] * 2^{64} +$ $\text{num_to_skip}[2] * 2^{128} + \dots +$ $\text{num_to_skip}[n-1] * 2^{64} * (n-1).$ Functor which would be used to initialize engines. This functor should take <code>sycl::item<1></code> as an argument and return object of type <code>Engine</code>.

engine_accessor

Host side helper to provide an abstraction over `sycl::accessor` to access engines' which are held by `engine_descriptor` in kernels.

- [Description](#)
- [API](#)

Description

`engine_accessor` provides an abstraction over `sycl::accessor` to access engines which are held by `engine_descriptor` in kernels by the `load()` and `store()` functions.

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<Engine>
    class engine_descriptor {
    public:
        Engine load(size_t id) const;
        void store(Engine engine, size_t id) const;
    };
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

Type	Description
Engine	Specify an engine which state is hold by <code>engine_accessor</code> .

Device Service Routines

Routine	Description
skip_ahead	Advances the state of the given engine as if it had generated 32 random bits a specified number of times.

[skip_ahead](#)

- [Description](#)
- [API](#)

Description

The `skip_ahead` function advances the state of the given engine as if it had generated 32 random bits a specified number of times.

The `skip_ahead` function supports the following interfaces to apply the skip-ahead method:

- Common interface
- Interface with a partitioned number of skipped elements

API

Syntax

Common Interface

```
namespace oneapi::mkl::rng::device {
    template<typename Engine>
    void skip_ahead (Engine& engine, std::uint64_t num_to_skip)
}
```

Interface with a partitioned number of skipped elements

```
namespace oneapi::mkl::rng::device {
    template<typename Engine>
    void skip_ahead (Engine& engine, std::initializer_list<std::uint64_t> num_to_skip)
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

Common Interface

Name	Type	Description
<code>engine</code>	<code>Engine&</code>	Object of <code>engine</code> class, which supports the block-splitting method.
<code>num_to_skip</code>	<code>std::uint64_t</code>	Number of skipped elements.

Interface with a Partitioned Number of Skipped Elements

Name	Type	Description
engine	Engine&	Object of <code>engine</code> class, which supports the block-splitting method.
num_to_skip	<code>std::initializer_list<std::uint64_t></code>	Partitioned number of skipped elements.

Device Distributions

oneMKL RNG routines are used to generate random numbers with different types of distribution. Each function group is introduced below by the type of underlying distribution and contains a short description of its functionality, as well as specifications of the call sequence and the explanation of input and output parameters. The Device Continuous Distribution Generators table and Device Discrete Distribution Generators table list the random number generator routines with data types and output distributions, and sets correspondence between data types of the generator routines and the basic random number generators.

Device Continuous Distribution Generators

Type of Distribution	Data Types	BRNG Data Type	Description
uniform (Continuous)	float, double	float, double	Uniform continuous distribution on the interval $[a, b)$
gaussian	float, double	float, double	Normal (Gaussian) distribution
exponential	float, double	float, double	Exponential distribution
lognormal	float, double	float, double	Lognormal distribution
beta	float, double	float, double	Beta distribution
gamma	float, double	float, double	Gamma distribution

Device Discrete Distribution Generators

Type of Distribution	Data Types	BRNG Data Type	Description
uniform (Discrete)	integer	float for <code>uniform_method::standard</code> , double for <code>uniform_method::accurate</code> , and raw output for 64 bit integers	Uniform discrete distribution on the interval $[a, b)$
bits	integer	integer	Bits of underlying BRNG integer sequence
uniform_bits	integer	integer	Uniformly distributed bits in 32/64-bit chunks
bernoulli	integer	float	Bernoulli distribution
poisson	integer	double	Poisson distribution
geometric	integer	float for 32-bit integers, double for 64-bit integers	Geometric distribution

NOTE

1. In case of `integer`, check the desired distribution for supported data types.
2. To get continuously distributed numbers of `sycl::half`, convert the generated numbers to `sycl::half` as shown below:

```
queue.submit([&](sycl::handler& cgh) {
    cgh.parallel_for(sycl::range<1>(n / VecSize), [=](sycl::item<1> item) {
        size_t item_id = item.get_id(0);
        oneapi::mkl::rng::device::mcg59<VecSize> engine(seed, item_id * VecSize);
        oneapi::mkl::rng::device::uniform<float> distr;

        auto res = oneapi::mkl::rng::device::generate(distr, engine);
        if constexpr(VecSize == 1) {
            r_acc[item_id] = static_cast<sycl::half>(res);
        }
        else {
            sycl::vec<sycl::half, VecSize> res_;
            res_ = res.template convert<sycl::half>();
            res_.store(item_id, r_acc);
        }
    });
});
```

uniform (Continuous)

Generates random numbers with uniform distribution.

- [Description](#)
- [API](#)

Description

The `uniform` class object is used in the `generate` function to provide random numbers uniformly distributed over the interval $[a, b)$, where a, b are the left and right bounds of the interval, respectively, and $a, b \in \mathbb{R}$; $a < b$.

The probability density function is given by:

$$F_{a,b}(x) = \begin{cases} \frac{1}{b-a}, & x \in [a, b) \\ 1, & x \notin [a, b) \end{cases}, -\infty < x < +\infty$$

The cumulative distribution function is as follows:

$$F_{a,b}(x) = \begin{cases} 0, & x < a \\ \frac{x-a}{b-a}, & a \leq x < b \\ 1, & x \geq b \end{cases}, -\infty < x < +\infty$$

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<typename Type, typename Method>
    class uniform {
    public:
        using method_type = Method;
        using result_type = Type;

        uniform(): uniform((Type)0.0, (Type)1.0){};
        explicit uniform(Type a, Type b);
        explicit uniform(const param_type& pt);

        Type a() const;
        Type b() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

<code>typename Type</code>	Type of the produced values. The specific values are as follows: • <code>float</code> • <code>double</code>
<code>typename Method</code>	Generation method. The specific values are as follows: • <code>oneapi::mkl::rng::device::uniform_method::by_default</code> • <code>oneapi::mkl::rng::device::uniform_method::standard</code> • <code>oneapi::mkl::rng::device::uniform_method::accurate</code> See brief descriptions of the methods in Distributions Template Parameter Method .

Input Parameters

Name	Type	Description
<code>a</code>	Type (<code>float</code> , <code>double</code>)	Left bound <code>a</code>
<code>b</code>	Type (<code>float</code> , <code>double</code>)	Right bound <code>b</code>

gaussian

Generates normally distributed random numbers.

- [Description](#)

- [API](#)

Description

The `gaussian` class object is used in the `generate` function to provide random numbers with normal (Gaussian) distribution with mean (`a`) and standard deviation (`stddev`, σ), where $a, \sigma \in \mathbb{R}; \sigma > 0$

The probability density function is given by:

$$f_{a,\sigma}(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y-a)^2}{2\sigma^2}\right) dy, -\infty < x < +\infty$$

The cumulative distribution function is as follows:

$$F_{a,\sigma}(x) = \int_{-\infty}^x \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y-a)^2}{2\sigma^2}\right) dy, -\infty < x < +\infty$$

The cumulative distribution function $F_{a,\sigma}(x)$ can be expressed in terms of standard normal distribution $\phi(x)$ as

$$F_{a,\sigma}(x) = \phi((x-a)/\sigma)$$

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<typename RealType, typename Method>
    class gaussian {
    public:
        using method_type = Method;
        using result_type = RealType;

        gaussian(): gaussian((RealType)0.0, (RealType)1.0){}
        explicit gaussian(RealType mean, RealType stddev);
        explicit gaussian(const param_type& pt);

        RealType mean() const;
        RealType stddev() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

typename Type	Type of the produced values. The specific values are as follows:
---------------	--

typename Method

- float
- double

Generation method. The specific values are as follows:

- `oneapi::mkl::rng::device::gaussian_method::by_default`
- `oneapi::mkl::rng::device::gaussian_method::box_muller2`
- `oneapi::mkl::rng::device::gaussian_method::icdf`

See brief descriptions of the methods in [Distributions Template Parameter Method](#).

NOTE To enable `oneapi::mkl::rng::device::gaussian_method::box_muller2` based on Vector Math functions, define the **MKL_RNG_USE_BINARY_CODE** macro and link static oneMKL libraries according to the output produced by [Intel® oneAPI Math Kernel Library Link Line Advisor](#). It may improve performance on some hardware.

NOTE `oneapi::mkl::rng::device::gaussian_method::icdf` is available only if the **MKL_RNG_USE_BINARY_CODE** macro is defined. Also link static oneMKL libraries according to the output produced by [Intel® oneAPI Math Kernel Library Link Line Advisor](#).

Input Parameters

Name	Type	Description
mean	RealType (float, double)	Mean value μ .
stddev	RealType (float, double)	Standard deviation σ .

lognormal

Generates lognormally distributed random numbers.

- [Description](#)
- [API](#)

Description

The `lognormal` class object is used in the `generate` function to provide random numbers with average of distribution (μ , a) and standard deviation (s , σ) of subject normal distribution, displacement (displ , b), and scalefactor (scale , β), where $a, \sigma, b, \beta \in \mathbb{R}$; $\sigma > 0, \beta > 0$.

The probability density function is given by:

$$f_{a,\sigma,b,\beta}(x) = \begin{cases} \frac{1}{\sigma(x-b)\sqrt{2\pi}} \exp\left(-\frac{\ln(\frac{x-b}{\beta})-a)^2}{2\sigma^2}\right), & x > b \\ 0, & x \leq b \end{cases}$$

The cumulative distribution function is as follows:

$$F_{a,\sigma,b,\beta}(x) = \begin{cases} \Phi\left(\frac{\ln(\frac{x-b}{\beta})-a}{\sigma}\right), & x > b \\ 0, & x \leq b \end{cases}$$

API

Syntax

```
template<typename RealType, typename Method>
class lognormal {
public:
    using method_type = Method;
    using result_type = RealType;

    lognormal() : lognormal((RealType)0.0, (RealType)1.0, (RealType) 0.0, (RealType)1.0){}
    explicit lognormal(RealType m, RealType s, RealType displ = (RealType)0.0, RealType scale =
(RealType)1.0);
    explicit lognormal(const param_type& pt);

    RealType m() const;
    RealType s() const;
    RealType displ() const;
    RealType scale() const;
    param_type param() const;
    void param(const param_type& pt);
};
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

<code>typename Type</code>	Type of the produced values. The specific values are as follows: • <code>float</code> • <code>double</code>
<code>typename Method</code>	Generation method. The specific values are as follows: • <code>oneapi::mkl::rng::device::lognormal_method::by_default</code>

- `oneapi::mkl::rng::device::lognormal_method::box_muller2`

See brief descriptions of the methods in [Distributions Template Parameter Method](#).

Note: To enable `oneapi::mkl::rng::device::lognormal_method::box_muller2` based on Vector Math functions, define the **MKL_RNG_USE_BINARY_CODE** macro and link static oneMKL libraries according to the output produced by [Intel® oneAPI Math Kernel Library Link Line Advisor](#). It may improve performance on some hardware.

Input Parameters

Name	Type	Description
m	RealType (float, double)	Average μ of the subject normal distribution.
s	RealType (float, double)	Standard deviation σ of the subject normal distribution.
displ	RealType (float, double)	Displacement <code>displ</code> .
scale	RealType (float, double)	Scalefactor <code>scale</code> .

exponential

Generates exponentially distributed random numbers.

- [Description](#)
- [API](#)

Description

The `exponential` class object is used in the `generate` function to provide random numbers with exponential distribution that has displacement a and scalefactor β , where $a, \beta \in R; \beta > 0$.

The probability density function is given by:

$$f_{a,\beta}(x) = \begin{cases} \frac{1}{\beta} \exp\left(-\frac{(x-a)}{\beta}\right), & x \geq a \\ 0, & x < a \end{cases}, -\infty < x < +\infty$$

The cumulative distribution function is as follows:

$$F_{a,\beta}(x) = \begin{cases} 1 - \exp\left(-\frac{(x-a)}{\beta}\right), & x \geq a \\ 0, & x < a \end{cases}, -\infty < x < +\infty$$

API

Syntax

```

namespace oneapi::mkl::rng::device {
    template<typename RealType, typename Method>
    class exponential {
    public:
        using method_type = Method;
        using result_type = RealType;

        exponential(): exponential((RealType)0.0, (RealType)1.0){}
        explicit exponential(RealType a, RealType beta);
        explicit exponential(const param_type& pt);

        RealType a() const;
        RealType beta() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}

```

Include Files

- oneapi/mkl/rng/device.hpp

Template Parameters

typename Type	Type of the produced values. The specific values are as follows: • float • double
typename Method	Generation method. The specific values are as follows: • oneapi::mkl::rng::device::exponential_method::by_default • oneapi::mkl::rng::device::exponential_method::icdf • oneapi::mkl::rng::device::exponential_method::icdf_accurate See brief descriptions of the methods in Distributions Template Parameter Method

Note: To enable extra optimizations based on Vector Math functions, define the **MKL_RNG_USE_BINARY_CODE** macro and link static oneMKL libraries according to the output produced by [Intel® oneAPI Math Kernel Library Link Line Advisor](#). It may improve performance on some hardware.

Input Parameters

Name	Type	Description
a	RealType (float, double)	Displacement a.
beta	RealType (float, double)	Scalefactor β .

uniform (Discrete)

Generates random numbers uniformly distributed over the interval $[a, b)$.

- [Description](#)
- [API](#)

Description

The `uniform` class object is used in `generate` functions to provide random numbers uniformly distributed over the interval $[a, b)$, where a, b are the left and right bounds of the interval respectively, and $a, b \in \mathbb{Z}; a < b$.

The probability distribution is given by:

$$P(X = k) = \frac{1}{b - a}, k \in \{a, a + 1, \dots, b - 1\}$$

The cumulative distribution function is as follows:

$$F_{a,b}(x) = \begin{cases} 0, & x < a \\ \frac{x-a+1}{b-a}, & a \leq x < b, x \in \mathbb{R} \\ 1, & x \geq b \end{cases}$$

API

Syntax

```

namespace oneapi::mkl::rng::device {
    template<typename Type, typename Method>
    class uniform {
    public:
        using method_type = Method;
        using result_type = Type;

        uniform(): uniform((Type)0,
                           std::is_integral<Type>::value
                           ? ((std::is_same_v<Method, uniform_method::standard> && sizeof(Type) == 32)
                              ? (1 << 23)
                              : std::numeric_limits<Type>::max())
                           : (Type)1) {};
        explicit uniform(Type a, Type b);
        explicit uniform(const param_type& pt);

        Type a() const;
        Type b() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}

```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

typename Method	Generation method. The specific values are as follows: <pre>oneapi::mkl::rng::device::uniform_method::standard</pre> <pre>oneapi::mkl::rng::device::uniform_method::accurate</pre> See brief descriptions of the methods in Distributions Template Parameter Method.
-----------------	--

NOTE The `oneapi::mkl::rng::device::uniform_method::standard` uses the `float` type for underlying BRNG calls. This might cause the produced numbers to have incorrect statistics (due to rounding error) when $(abs(b-a) > 2^{23} || abs(b) > 2^{23} || abs(a) > 2^{23})$. To get proper statistics for this case, use the `oneapi::mkl::rng::device::uniform_method::accurate` method instead.

NOTE For `(u)int8` and `(u)int16` small types, the numbers are always generated as single-precision floating-point numbers.

Input Parameters

Name	Type	Description
a	<code>std::int8_t</code>	Left bound a
	<code>std::uint8_t</code>	
	<code>std::int16_t</code>	
	<code>std::uint16_t</code>	
	<code>std::int32_t</code>	
	<code>std::uint32_t</code>	
	<code>std::int64_t</code>	
	<code>std::uint64_t</code>	
b	<code>std::int8_t</code>	Right bound b
	<code>std::uint8_t</code>	
	<code>std::int16_t</code>	
	<code>std::uint16_t</code>	
	<code>std::int32_t</code>	
	<code>std::uint32_t</code>	
	<code>std::int64_t</code>	
	<code>std::uint64_t</code>	

Name	Type	Description
	<code>std::uint64_t</code>	

NOTE The sample rejection method is used in the case of the 64-bit return type. This enables sequences to not overlap in the case of parallel generation. To ensure the output quality, provide an ample offset between engines.

bits

Generates bits of underlying engine (BRNG) integer sequence.

- [Description](#)
- [API](#)

Description

The `bits` class object is used to generate integer random values. Each integer can be treated as a vector of several bits. In a truly random generator, these bits are random, while in pseudorandom generators this randomness can be violated. See [VS Notes](#) for details.

API

Syntax

```
namespace oneapi::mkl::rng::device {  
    template<typename UIntType = std::uint32_t>  
    class bits {  
        using result_type = UIntType;  
    };  
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

`typename UIntType = std::uint32_t`

Type of the produced values. The specific values are as follows:

- `std::uint32_t` for `philox4x32x10`, `mrg32k3a` and `mcg31m1` engines.
- `std::uint64_t` for `mcg59` engine.

uniform_bits

Generates uniformly distributed bits in 32/64-bit chunks.

- [Description](#)
- [API](#)
- [Examples Of Generating Small Types](#)

Description

The `uniform_bits` class object is used to generate uniformly distributed bits in 32/64-bit chunks. It is designed to ensure each bit in the 32/64-bit chunk is uniformly distributed. This distribution is supported for `philox4x32x10` and `mcg59` engines. When generating 64-bit chunks, twice as much engine offset needs to be provided.

`UIntType` denotes the chunk size and can be `std::uint32_t`, `std::uint64_t`. See [VS Notes](#) for details.

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<typename UIntType = std::uint32_t>
    class uniform_bits {
        using result_type = UIntType;
    };
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

<code>typename UIntType = std::uint32_t</code>	Type of the produced values. The specific values are as follows:
	<code>std::uint32_t</code> <code>std::uint64_t</code>

Examples Of Generating Small Types

Example 1

`uniform_bits` returns uniformly distributed generator output, so to get small types such as `std::int8_t`, `std::uint8_t`, `std::int16_t`, and `std::uint16_t`, convert the generated numbers to the desired type as shown below:

```
queue.submit([&](sycl::handler& cgh) {
    cgh.parallel_for(sycl::range<1>(n / VecSize), [=](sycl::item<1> item) {
        size_t item_id = item.get_id(0);
        ...
        oneapi::mkl::rng::device::uniform_bits<Type> distr;

        auto res = oneapi::mkl::rng::device::generate(distr, engine);
        if constexpr(VecSize == 1) {
            r_acc[item_id] = static_cast<DesiredType>(res);
        }
        else {
            sycl::vec<DesiredType, VecSize> res_;
            res_ = res.template convert<DesiredType>();
            res_.store(item_id, r_acc);
        }
    });
});
```

Example 2

You also can get several numbers of small type from one generated number. The following example demonstrates how to obtain four `std::uint8_t` numbers from one `std::uint32_t`:

```
oneapi::mkl::rng::device::uniform_bits<std::uint32_t> distr;

std::uint32_t res = oneapi::mkl::rng::device::generate(distr, engine);
sycl::vec<std::uint8_t, 4> res_small;
for (int i = 0; i < 4; ++i) {
    res_small[i] = static_cast<std::uint8_t>(res & 0xff);
    res >>= 8;
}
```

poisson

Generates Poisson distributed random values.

- [Description](#)
- [API](#)

Description

The `poisson` class object is used in the `generate` function to provide Poisson distributed random numbers with distribution parameter λ , where $\lambda \in R; \lambda > 0$.

The probability distribution is given by:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

$k \in \{0, 1, 2, \dots\}$.

The cumulative distribution function is as follows:

$$F_{\lambda}(x) = \begin{cases} \sum_{k=0}^{\lfloor x \rfloor} \frac{\lambda^k e^{-\lambda}}{k!}, & x \geq 0 \\ 0, & x < 0 \end{cases}, x \in R$$

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<typename IntType, typename Method>
    class poisson {
    public:
        using method_type = Method;
        using result_type = IntType;
    };
}
```

```

poisson(): poisson(0.5){}
explicit poisson(double lambda);
explicit poisson(const param_type& pt);

double lambda() const;
param_type param() const;
void param(const param_type& pt);
};
}

```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

typename Type	Type of the produced values. The specific values are as follows: • <code>std::int32_t</code> • <code>std::int64_t</code>
typename Method	Generation method. The specific values are as follows: • <code>oneapi::mkl::rng::device::poisson_method::by_default</code> • <code>oneapi::mkl::rng::device::poisson_method::devroye</code> See brief descriptions of the methods in Distributions Template Parameter Method .

NOTE The Poisson `oneapi::mkl::rng::device::poisson_method::devroye` method uses the following underlying methods depending on the lambda parameter:

- table-lookup method for small lambdas ($\lambda < 60$)
- Devroye's method [Devroye] for medium lambdas ($60 \leq \lambda < 1000$)
- Gaussian approximation for huge lambdas ($\lambda \geq 1000$)

Input Parameters

Name	Type	Description
lambda	double	Distribution parameter λ .

bernoulli

Generates Bernoulli distributed random values.

- [Description](#)
- [API](#)

Description

The `bernoulli` class object is used in the `generate` function to provide Bernoulli distributed random numbers with probability p of a single trial success, where $p \in R; 0 \leq p \leq 1$.

The probability distribution is given by:

$$P(X = 1) = p$$

$$P(X = 0) = 1 - p$$

The cumulative distribution function is as follows:

$$F_p(x) = \begin{cases} 0, & x < 0 \\ 1 - p, & 0 \leq x < 1, x \in R \\ 1, & x \geq 1 \end{cases}$$

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<typename IntType, typename Method>
    class bernoulli {
    public:
        using method_type = Method;
        using result_type = IntType;

        bernoulli(): bernoulli(0.5f){}
        explicit bernoulli(float p);
        explicit bernoulli(const param_type& pt);

        float p() const;
        param_type param() const;
        void param(const param_type& pt);
    };
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

typename Type

Type of the produced values. The specific values are as follows:

- `std::int8_t`
- `std::uint8_t`
- `std::int16_t`
- `std::uint16_t`

typename Method

- `std::int32_t`
- `std::uint32_t`

Generation method. The specific values are as follows:

- `oneapi::mkl::rng::device::bernoulli_method::by_default`
- `oneapi::mkl::rng::device::bernoulli_method::icdf`

See brief descriptions of the methods in [Distributions Template Parameter Method](#).

Input Parameters

Name	Type	Description
<code>p</code>	<code>float</code>	Success probability p of a trial.

geometric

Generates geometrically distributed random values.

- [Description](#)
- [API](#)

Description

The `geometric` class object is used in the `generate` function to provide geometrically distributed random numbers with probability p of a single trial success, where $p \in R; 0 \leq p \leq 1$.

The probability distribution is given by:

$$P(X = k) = p * (1 - p)^k, k = \{0, 1, 2, \dots\}.$$

The cumulative distribution function is as follows:

$$F_p(x) = \begin{cases} 0, & x < 0 \\ 1 - (1 - p)^{\lfloor x+1 \rfloor}, & x \geq 0 \end{cases}$$

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<typename IntType, typename Method>
    class geometric {
    public:
        using method_type = Method;
        using result_type = IntType;
```

```

geometric(): geometric(0.5f){}
explicit geometric(float p);
explicit geometric(const param_type& pt);

float p() const;
param_type param() const;
void param(const param_type& pt);
};
}

```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

typename Type	Type of the produced values. The specific values are as follows: • <code>std::int32_t</code> • <code>std::uint32_t</code> • <code>std::int64_t</code> • <code>std::uint64_t</code>
typename Method	Generation method. The specific values are as follows: • <code>oneapi::mkl::rng::device::geometric_method::by_default</code> • <code>oneapi::mkl::rng::device::geometric_method::icdf</code> See brief descriptions of the methods in Distributions Template Parameter Method.

Input Parameters

Name	Type	Description
p	float	Success probability p of a trial.

beta

Generates Beta distributed random numbers.

- [Description](#)
- [API](#)

Description

The `beta` class object is used in the `generate` function to provide random numbers with beta distribution that has shape parameters P and Q , displacement α , and scale parameter (b, β) , where $P, Q, \alpha, \beta \in R; p > 0; q > 0; \beta > 0$.

The probability density function is given by:

$$f_{p,q,\alpha,\beta}(x) = \begin{cases} \frac{1}{B(p,q)*\beta^{p+q-1}}(x-a)^{p-1} * (\beta + \alpha - x)^{q-1}, & \alpha \leq x < \alpha + \beta \\ 0, & x < \alpha, x \geq \alpha + \beta \end{cases}$$

The cumulative distribution function is as follows:

$$F_{a,b}(x) = \begin{cases} 0, & x < \alpha \\ \int_{\alpha}^x \frac{1}{B(p,q)*\beta^{p+q-1}}(y-\alpha)^{p-1} * (\beta + \alpha - y)^{q-1} dy, & \alpha \leq x < \alpha + \beta \\ 1, & x \geq \alpha + \beta \end{cases}$$

Where $B(p, 1)$ is the complete beta function.

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template<typename RealType, typename Method>
    class beta {
    public:
        using method_type = Method;
        using result_type = RealType;

        beta() : beta((RealType)1.0, (RealType)1.0, (RealType)0.0, (RealType)1.0) {}
        explicit beta(RealType p, RealType q, RealType a, RealType b);
        explicit beta(const param_type& pt);

        RealType p() const;
        RealType q() const;
        RealType a() const;
        RealType b() const;
        param_type param() const;
        std::size_t count_rejected_numbers() const;
        void param(const param_type& pt);
    };
}
```

Include Files

- oneapi/mkl/rng/device.hpp

Template Parameters

typename Type	Type of the produced values. The specific values are as follows: • float • double
typename Method	Generation method. The specific values are as follows: • oneapi::mkl::rng::device::beta_method::by_default • oneapi::mkl::rng::device::beta_method::cja • oneapi::mkl::rng::device::beta_method::cja_accurate

See brief descriptions of the methods in
[Distributions Template Parameter Method](#)

NOTE To enable extra optimizations based on Vector Math functions, define the **MKL_RNG_USE_BINARY_CODE** macro and link static oneMKL libraries according to the output produced by [Intel® oneAPI Math Kernel Library Link Line Advisor](#). It may improve performance on some hardware.

Input Parameters

Name	Type	Description
p	RealType (float, double)	Shape p.
q	RealType (float, double)	Shape q.
a	RealType (float, double)	Displacement α .
b	RealType (float, double)	Scalefactor β .

Special methods

```
std::size_t count_rejected_numbers() const
```

Returns the amount of random numbers that were rejected during the last `generate` function call. If no `generate` calls, returns 0.

gamma

Generates Gamma distributed random numbers.

- [Description](#)
- [API](#)

Description

The `gamma` class object is used in the `generate` function to provide random numbers with gamma distribution that has shape α , displacement a , and scale parameter β , where $a, \alpha, \beta \in R; \alpha > 0; \beta > 0$.

The probability distribution is given by:

$$f_{a,\alpha,\beta}(x) = \begin{cases} \frac{1}{\Gamma(\alpha)\beta^\alpha} (x-a)^{\alpha-1} e^{-(x-a)/\beta}, & x \geq a \\ 0, & x < a \end{cases}$$

The cumulative distribution function is as follows:

$$F_{a,\alpha,\beta}(x) = \begin{cases} \int_a^x \frac{1}{\Gamma(\alpha)\beta^\alpha} (y-a)^{\alpha-1} e^{-(y-a)/\beta} dy, & x \geq a \\ 0, & x < a \end{cases}$$

API

Syntax

```

namespace oneapi::mkl::rng::device {
    template<typename RealType, typename Method>
    class gamma {
    public:
        using method_type = Method;
        using result_type = RealType;

        gamma() : gamma((RealType)1.0, (RealType)0.0, (RealType)1.0){}
        explicit gamma(RealType alpha, RealType a, RealType beta);
        explicit gamma(const param_type& pt);

        RealType alpha() const;
        RealType a() const;
        RealType beta() const;
        param_type param() const;
        std::size_t count_rejected_numbers() const;
        void param(const param_type& pt);
    };
}

```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

typename Type	Type of the produced values. The specific values are as follows: • <code>float</code> • <code>double</code>
typename Method	Generation method. The specific values are as follows: • <code>oneapi::mkl::rng::device::gamma_method::by_default</code> • <code>oneapi::mkl::rng::device::gamma_method::marsaglia</code> • <code>oneapi::mkl::rng::device::gamma_method::marsaglia_accurate</code> See brief descriptions of the methods in Distributions Template Parameter Method

NOTE To enable extra optimizations based on Vector Math functions, define the **MKL_RNG_USE_BINARY_CODE** macro and link static oneMKL libraries according to the output produced by [Intel® oneAPI Math Kernel Library Link Line Advisor](#). It may improve performance on some hardware.

Input Parameters

Name	Type	Description
alpha	RealType (float, double)	Shape α .
a	RealType (float, double)	Displacement a.
beta	RealType (float, double)	Scalefactor β .

Special methods

```
std::size_t count_rejected_numbers() const
```

Returns the amount of random numbers that were rejected during the last `generate` function call. If no `generate` calls, returns 0.

Engine Adaptors

oneMKL RNG provides the following device engine adaptors:

Adaptor	Description
Count Engine Adaptor	The adaptor provides a method to count how many random numbers were taken from an engine during the <code>generate</code> call

Count Engine Adaptor

`count_engine_adaptor` is a random number engine adaptor that counts how many times random 32 bits were taken from the engine during the `generate` call.

- [Description](#)
- [API](#)

Description

This adaptor allows users to query the number of random numbers generated by an engine during the `generate` call. Especially it's especially useful for distributions based on acceptance-rejection algorithms, such as `beta`, `gamma`.

API

Syntax

```
namespace oneapi::mkl::rng::device {
    template <typename Engine>
    class count_engine_adaptor {
    public:
        static constexpr std::int32_t vec_size = Engine::vec_size;
        // ctors
        explicit count_engine_adaptor(const Engine& engine);
        explicit count_engine_adaptor(Engine&& engine);
        template <typename... Params>
        count_engine_adaptor(Params... params);
        // getters
        std::int64_t get_count() const;
```

```
const Engine& base() const;
};
}
```

Include Files

- `oneapi/mkl/rng/device.hpp`

Template Parameters

<code>typename Engine</code>	Type of the provided random engine
------------------------------	------------------------------------

Constructors Input Parameters

Name	Type	Description
<code>engine</code>	<code>const Engine& or Engine&&</code>	Constructs the adaptor over <code>engine</code> by constant lvalue reference or rvalue reference.
<code>params</code>	<code>Params...</code>	Constructs the adaptor generating an engine inside. <code>params</code> is a pack of input parameters to create an engine.

Getters

```
std::size_t get_count() const
```

This method returns the count of random numbers generated by `generate`.

```
const Engine& base() const
```

Returns the underlying random engine.

Summary Statistics

Intel® oneAPI Math Kernel Library (oneMKL) provides Data Parallel C++ interfaces for the Summary Statistics routines that compute basic statistical estimates for single and double precision multi-dimensional datasets.

Definitions

Struct `dataset` consolidates the information of a multi-dimensional dataset (see detailed description in [Dataset](#)). All computational routines are represented as free functions (see detailed description for each routine in [Summary Statistics Routines](#)):

- Raw and central sums/moments up to the fourth order
- Variation coefficient
- Skewness and excess kurtosis (further referred to as a kurtosis)
- Minimum and maximum

Routines

oneMKL Summary Statistics Usage Model

- [Description](#)
- [Example of Summary Statistics Usage](#)

Description

A typical algorithm for random number generators is as follows:

1. Create and initialize the object for the dataset.
2. Call the summary statistics routine to calculate the appropriate estimate.

The following example demonstrates how to calculate mean values for a 3-dimensional dataset filled with random numbers. For dataset creation, the `make_dataset` helper function is used.

Example of Summary Statistics Usage

Buffer API

```
#include <iostream>
#include <vector>

#include "CL/sycl.hpp"
#include "oneapi/mkl/stats.hpp"

int main() {
    sycl::queue queue;

    const size_t n_observations = 1000;
    const size_t n_dims = 3;
    std::vector<float> x(n_observations * n_dims);
    // fill x storage with random numbers
    for(int i = 0; i < n_dims, i++) {
        for(int j = 0; j < n_observations; j++) {
            x[j + i * n_observations] = float(std::rand()) / float(RAND_MAX);
        }
    }
    //create buffer for dataset
    sycl::buffer<float, 1> x_buf(x.data(), x.size());
    // create buffer for mean values
    sycl::buffer<float, 1> mean_buf(n_dims);
    // create mkl::stats::dataset
    auto dataset = oneapi::mkl::stats::make_dataset<mkl::stats::layout::row_major>(n_dims,
n_observations, x_buf);

    oneapi::mkl::stats::mean(queue, dataset, mean_buf);

    // create host accessor for mean_buf to print results
    auto acc = mean_buf.template get_access<sycl::access::mode::read>();

    for(int i = 0; i < n_dims; i++) {
        std::cout << "Mean value for dimension " << i << ": " << acc[i]<<
std::endl;
```

```

    }
    return 0;
}

```

USM API

```

#include <iostream>
#include <vector>

#include "CL/sycl.hpp"
#include "oneapi/mkl/stats.hpp"

int main() {
    sycl::queue queue;

    const size_t n_observations = 1000;
    const size_t n_dims = 3;

    sycl::usm_allocator<float, sycl::usm::alloc::shared> allocator(queue);

    std::vector<float, decltype(allocator)> x(n_observations * n_dims, allocator);
    // fill x storage with random numbers
    for(int i = 0; i < n_dims, i++) {
        for(int j = 0; j < n_observations; j++) {
            x[j + i * n_observations] = float(std::rand()) / float(RAND_MAX);
        }
    }
    std::vector<float, decltype(allocator)> mean_buf(n_dims, allocator);
    // create mkl::stats::dataset
    auto dataset = oneapi::mkl::stats::make_dataset<mkl::stats::layout::row_major>(n_dims,
n_observations, x);

    sycl::event event = oneapi::mkl::stats::mean(queue, dataset, mean);
    event.wait();
    for(int i = 0; i < n_dims; i++) {
        std::cout << "Mean value for dimension " << i << ": " << mean[i] <<
        std::endl;
    }
    return 0;
}

```

You can also use USM with raw pointers by using the `sycl::malloc_shared/malloc_device` functions. Additionally, examples that demonstrate usage of summary statistics functionality are available in:

```

${MKL}/share/doc/mkl/examples/sycl/stats/source

```

Summary Statistics Device Support

Data Parallel C++ supports several types of devices:

- CPU device: Performs computations on a CPU using OpenCL™.

- GPU device: Performs computations on a GPU using OpenCL™ or Level Zero.

All Data Parallel C++ routines of oneMKL summary statistics support the CPU and GPU devices.

Dataset

The `oneapi::mkl::stats::dataset` structure consolidates information of a multi-dimensional dataset.

- [API](#)

API

Syntax

```
namespace oneapi::mkl::stats {
    template<layout ObservationsLayout = layout::row_major,
            typename Type = float*>
```

```
namespace oneapi::mkl::stats {
    struct dataset {}
}
```

Buffer API specialization

```
namespace oneapi::mkl::stats {

    template<layout ObservationsLayout, typename Type>struct dataset<ObservationsLayout,
sycl::buffer<Type, 1>> {

        explicit dataset(std::int64_t n_dims_, std::int64_t n_observations_,
            sycl::buffer<Type, 1> observations_, sycl::buffer<Type, 1> weights_ = {0},
            sycl::buffer<std::int64_t, 1> indices_ = {0}) : n_dims(n_dims_),
n_observations(n_observations_), observations(observations_),
            weights(weights_), indices(indices_);

        std::int64_t n_dims;
        std::int64_t n_observations;
        sycl::buffer<Type, 1> observations;
        sycl::buffer<Type, 1> weights = {0};
        sycl::buffer<std::int64_t, 1> indices = {0};
        static constexpr layout layout = ObservationsLayout;
    };
}
```

USM API specialization

```
namespace oneapi::mkl::stats {

    template< layout ObservationsLayout, typename Type>
    struct dataset<ObservationsLayout, Type*> {
        explicit dataset(std::int64_t n_dims_, std::int64_t n_observations_, Type* observations_,
            Type* weights_ = nullptr, std::int64_t* indices_ = nullptr) :
            n_dims(n_dims_), n_observations(n_observations_),
            observations(observations_),
            weights(weights_), indices(indices_);

        std::int64_t n_dims;
        std::int64_t n_observations;
        Type* observations;
```

```
Type* weights = nullptr;
std::int64_t* indices = nullptr;
static constexpr layout layout = ObservationsLayout;
};
}
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

typename DataType	Type of dataset. May be <code>sycl::buffer<float, 1></code> , <code>sycl::buffer<double, 1></code> (for buffer-based dataset) or <code>float*</code> , <code>double*</code> (for USM-based dataset).
oneapi::mkl::stats::layout ObservationsLayout	Layout of the observations matrix of the dataset. The specific values are as follows: <code>oneapi::mkl::stats::layout::row_major</code> <code>oneapi::mkl::stats::layout::col_major</code>

Struct Members

Name	Type	Description
n_dims	std::int64_t	The number of dimensions (variables)
n_observations	std::int64_t	The number of observations
layout	oneapi::mkl::stats::layout	Characterize the matrix of observations layout (row or column major)
observations	sycl::buffer<Type, 1> / Type*	Matrix of observations.
weights	sycl::buffer<Type, 1> / Type*	Array of weights of size n_observations. Elements of the arrays are non-negative numbers. If the parameter is not specified, each observation is assigned a weight equal to 1.
indices	sycl::buffer<std::int64_t, 1> / std::int64_t*	Array of vector components that will be processed. Size of array is n_dims. If the parameter is not specified, all components of the vector are processed.

Service Functions to Create Dataset

Use these functions to create a dataset.

oneapi::mkl::stats::make_dataset

- [Description](#)
- [API](#)

Description

Entry point to create a dataset from the provided parameters.

API

Syntax

Buffer API

```
template<layout ObservationsLayout = layout::row_major,
        typename Type> dataset<ObservationsLayout,
        sycl::buffer<Type, 1>> make_dataset(std::int64_t n_dims,
        std::int64_t n_observations,
        sycl::buffer<Type, 1> observations,
        sycl::buffer<Type, 1> weights = {0},
        sycl::buffer<std::int64_t, 1> indices = {0});
```

USM API

```
template<layout ObservationsLayout = layout::row_major,
        typename Type> dataset<ObservationsLayout,
        Type*> make_dataset(std::int64_t n_dims,
        std::int64_t n_observations,
        Type* observations, Type* weights = nullptr,
        std::int64_t* indices = nullptr);
```

Include Files

- `oneapi/mkl/stats.hpp`

Template Parameters

<code>typename DataType</code>	Type of dataset. May be float, double.
<code>oneapi::mkl::stats::layout ObservationsLayout</code>	Layout of the observations matrix of the dataset. The specific values are as follows: <code>oneapi::mkl::stats::layout::row_major</code> <code>oneapi::mkl::stats::layout::col_major</code>

Input Parameters

Name	Type	Description
<code>n_dims</code>	<code>std::int64_t</code>	The number of dimensions (variables)
<code>n_observations</code>	<code>std::int64_t</code>	The number of observations
<code>observations</code>	<code>sycl::buffer<Type, 1> / Type*</code>	Matrix of observations

Optional Input Parameters

Name	Type	Description
weights	<code>sycl::buffer<Type, 1> / Type*</code>	Array of weights of size <code>n_observations</code> . Elements of the arrays are non-negative numbers. If the parameter is not specified, each observation is assigned weight equal to 1.
indices	<code>sycl::buffer<std::int64_t, 1> / std::int64_t*</code>	Array of vector components that will be processed. Size of array is <code>n_dims</code> . If the parameter is not specified, all components of the random vector are processed.

Return Type

Buffer API

<code>oneapi::mkl::stats::dataset<ObservationsLayout, sycl::buffer<Type, 1>></code>	Dataset holding specified parameters
---	--------------------------------------

USM API

<code>oneapi::mkl::stats::dataset<ObservationsLayout, Type*></code>	Dataset holding specified parameters
---	--------------------------------------

Summary Statistics Routines

The Summary Statistics routines calculate:

- Raw and central sums/moments up to the fourth order
- Variation coefficient
- Skewness and excess kurtosis (further referred to as a kurtosis)
- Minimum and maximum

oneapi::mkl::stats::raw_sum

- [Description](#)
- [API](#)

Description

Entry point to compute arrays of raw sums up to the 4th order.

Mathematical Notation

Vector of raw sums:

$$S^k(X) = (S_1^k(X), \dots, S_p^k(X)), \quad S_i^k(X) = \sum_{j=1}^n \omega_j x_{ij}^k$$

Where:

$$i = 1, \dots, p, \quad k = 1, 2, 3, 4$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void raw_sum(sycl::queue& queue,
    const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
    sycl::buffer<Type, 1> sum,
    sycl::buffer<Type, 1> raw_sum_2 = {0},
    sycl::buffer<Type, 1> raw_sum_3 = {0},
    sycl::buffer<Type, 1> raw_sum_4 = {0});
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event max(
    sycl::queue& queue,
    const dataset<ObservationsLayout, Type*>& data,
    Type* max,
    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- `oneapi/mkl/stats.hpp`

Template Parameters

Name	Description
<code>oneapi::mkl::stats::method Method = oneapi::mkl::stats::method::fast</code>	Computation method. The specific values are as follows: <code>oneapi::mkl::stats::method::fast</code> <code>oneapi::mkl::stats::method::one_pass</code>

Input Parameters

Name	Type	Description
<code>queue</code>	<code>sycl::queue&</code>	Valid <code>sycl::queue</code> , calls of the <code>oneapi::mkl::stats::raw_sum()</code> routine submits kernels in this queue.
<code>data</code>	<code>const dataset<ObservationsLayout, Type*>&</code>	Dataset which is used for estimates computation.

Optional Input Parameters for USM API

Name	Type	Description
<code>dependencies</code>	<code>const std::vector<sycl::event> &</code>	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
sum	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of sum.
raw_sum_2	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of raw sums of the 2nd order (optional).
raw_sum_3	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of raw sums of the 3rd order (optional).
raw_sum_4	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of raw sums of the 4th order (optional).

USM API

Name	Type	Description
sum	<code>Type*</code>	Pointer to the output array of sum.
raw_sum_2	<code>Type*</code>	Pointer to the output array of raw sums of the 2nd order (optional).
raw_sum_3	<code>Type*</code>	Pointer to the output array of raw sums of the 3rd order (optional).
raw_sum_4	<code>Type*</code>	Pointer to the output array of raw sums of the 4th order (optional).
event	<code>sycl::event</code>	Function returns event after submitting task in <code>sycl::queue</code> .

oneapi::mkl::stats::central_sum

- [Description](#)
- [API](#)

Description

Entry point to compute arrays of central sums up to the 4th order.

Mathematical Notation

Vector of central sums:

$$S^k(X) = (S_1^k(X), \dots, S_p^k(X)), \quad S_i^k(X) = \sum_{j=1}^n \omega_j (x_{ij} - S_i(X))^k$$

Where:

$$S_i(X) = \sum_{j=1}^n \omega_j x_{ij}, \quad i = 1, \dots, p, \quad k = 2, 3, 4$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void central_sum(sycl::queue& queue,
    const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
    sycl::buffer<Type, 1> central_sum_2,
    sycl::buffer<Type, 1> central_sum_3 = {0},
    sycl::buffer<Type, 1> central_sum_4 = {0});
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event central_sum(sycl::queue& queue,
    const dataset<ObservationsLayout, Type*>& data,
    Type* central_sum_2,
    Type* central_sum_3 = nullptr,
    Type* central_sum_4 = nullptr,
    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

<pre>oneapi::mkl::stats::method Method = oneapi::mkl::stats::method::fast</pre>	Computation method. The specific values are as follows: <pre>oneapi::mkl::stats::method::fast oneapi::mkl::stats::method::one_pass</pre>
---	---

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::central_sum() routine submits kernels in this queue.
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Optional Input Parameters for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event> &	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
central_sum_2	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of central sums of the 2nd order (optional).
central_sum_3	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of central sums of the 3rd order (optional).
central_sum_4	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of central sums of the 4th order (optional).

USM API

Name	Type	Description
central_sum_2	<code>Type*</code>	Pointer to the output array of central sums of the 2nd order (optional).
central_sum_3	<code>Type*</code>	Pointer to the output array of central sums of the 3rd order (optional).
central_sum_4	<code>Type*</code>	Pointer to the output array of central sums of the 4th order (optional).
event	<code>sycl::event</code>	Function returns event after submitting task in <code>sycl::queue</code> .

oneapi::mkl::stats::central_sum with User-provided Mean

- [Description](#)
- [API](#)

Description

Entry point to compute arrays of central sums up to the 4th order with mean provided by the user.

Mathematical Notation

Vector of central sums:

$$S^k(X) = (S_1^k(X), \dots, S_p^k(X)), \quad S_i^k(X) = \sum_{j=1}^n \omega_j (x_{ij} - S_i(X))^k$$

Where:

$$S_i(X) = \sum_{j=1}^n \omega_j x_{ij}, \quad i = 1, \dots, p, \quad k = 2, 3, 4$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void central_sum(sycl::queue& queue,
    sycl::buffer<Type, 1> mean,
    const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
    sycl::buffer<Type, 1> central_sum_2,
    sycl::buffer<Type, 1> central_sum_3 = {0},
    sycl::buffer<Type, 1> central_sum_4 = {0});
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event central_sum(sycl::queue& queue,
    Type* mean,
    const dataset<ObservationsLayout, Type*>& data,
    Type* central_sum_2,
    Type* central_sum_3 = nullptr,
    Type* central_sum_4 = nullptr,
    const std::vector<sycl::event> &dependencies = {});;
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

oneapi::mkl::stats::method Method =	Computation method. The specific values are as follows: oneapi::mkl::stats::method::fast
oneapi::mkl::stats::method::fast	

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::central_sum() routine submits kernels in this queue.
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Buffer API

Name	Type	Description
mean	sycl::buffer<Type, 1>	sycl::buffer to the array of mean values provided by the user.

USM API

Name	Type	Description
mean	Type*	Pointer to the output array of mean values provided by the user.

Output Parameters

Buffer API

Name	Type	Description
central_sum_2	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of central sums of the 2nd order.
central_sum_3	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of central sums of the 3rd order.
central_sum_4	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of central sums of the 4th order.

USM API

Name	Type	Description
central_sum_2	<code>Type*</code>	Pointer to the output array of central sums of the 2nd order.
central_sum_3	<code>Type*</code>	Pointer to the output array of central sums of the 3rd order.
central_sum_4	<code>Type*</code>	Pointer to the output array of central sums of the 4th order.
event	<code>sycl::event</code>	Function returns event after submitting task in <code>sycl::queue</code> .

oneapi::mkl::stats::raw_moment

- [Description](#)
- [API](#)

Description

Entry point to compute arrays of raw moments up to 4th order.

Mathematical Notation

Vector of raw moments values:

$$R^k(X) = (R_1^k(X), \dots, R_p^k(X)), \quad R_i^k(X) = \frac{1}{W} \sum_{j=1}^n \omega_j x_{ij}^k$$

Where:

$$W = \sum_{j=1}^n \omega_j, \quad j = 1, \dots, p; k = 1, 2, 3, 4$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void raw_moment(sycl::queue& queue,
    const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
    sycl::buffer<Type, 1> mean,
    sycl::buffer<Type, 1> raw_moment_2 = {0},
    sycl::buffer<Type, 1> raw_moment_3 = {0},
    sycl::buffer<Type, 1> raw_moment_4 = {0});
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event raw_moment(sycl::queue& queue,
    const dataset<ObservationsLayout, Type*>& data,
    Type* mean,
    Type* raw_moment_2 = nullptr,
    Type* raw_moment_3 = nullptr,
    Type* raw_moment_4 = nullptr,
    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

Name	Description
oneapi::mkl::stats::method Method = oneapi::mkl::stats::method::fast	Computation method. The specific values are as follows: oneapi::mkl::stats::method::fast oneapi::mkl::stats::method::one_pass

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::raw_moment() routine submits kernels in this queue.
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Optional Input Parameters for USM API

Name	Type	Description
dependencies	const std::vector <sycl::event> &	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
mean	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of mean values.
raw_moment_2	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of raw moments of the 2nd order (optional).
raw_moment_3	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of raw moments of the 3rd order (optional).
raw_moment_4	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of raw moments of the 4th order (optional).

USM API

Name	Type	Description
mean	<code>Type*</code>	Pointer to the output array of mean values.
raw_moment_2	<code>Type*</code>	Pointer to the output array of raw moments of the 2nd order (optional).
raw_moment_3	<code>Type*</code>	Pointer to the output array of raw moments of the 3rd order (optional).
raw_moment_4	<code>Type*</code>	Pointer to the output array of raw moments of the 4th order (optional).
event	<code>sycl::event</code>	Function returns event after submitting task in <code>sycl::queue</code> .

oneapi::mkl::stats::central_moment

- [Description](#)
- [API](#)

Description

Entry point to compute arrays of central moments up to 4th order.

Mathematical Notation

Vector of central moments:

$$C^k(X) = (C_1^k(X), \dots, C_p^k(X)), \quad C_i^k(X) = \frac{1}{W} \sum_{j=1}^n \omega_j (x_{ij} - M_i(X))^k$$

Where:

$$W = \sum_{j=1}^n \omega_j, \quad i = 1, \dots, p, \quad k = 2, 3, 4$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void central_moment(sycl::queue& queue,
    const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
    sycl::buffer<Type, 1> central_moment_2,
    sycl::buffer<Type, 1> central_moment_3 = {0},
    sycl::buffer<Type, 1> central_moment_4 = {0});
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event central_moment(sycl::queue& queue,
    const dataset<ObservationsLayout, Type*>& data,
    Type* central_moment_2,
    Type* central_moment_3 = nullptr,
    Type* central_moment_4 = nullptr,
    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- `oneapi/mkl/stats.hpp`

Template Parameters

<code>oneapi::mkl::stats::method Method =</code> <code>oneapi::mkl::stats::method::fast</code>	Computation method. The specific values are as follows: <code>oneapi::mkl::stats::method::fast</code> <code>oneapi::mkl::stats::method::one_pass</code>
---	--

Input Parameters

Name	Type	Description
queue	<code>sycl::queue&</code>	Valid <code>sycl::queue</code> , calls of the <code>oneapi::mkl::stats::central_moment()</code> routine submits kernels in this queue.
data	<code>const dataset<ObservationsLayout, Type*>&</code>	Dataset which is used for estimates computation.

Optional Input Parameters for USM API

Name	Type	Description
dependencies	<code>const std::vector<sycl::event> &</code>	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
central_moment_2	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of central moments of the 2nd order.
central_moment_3	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of central moments of the 3rd order (optional).
central_moment_4	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of central moments of the 4th order (optional).

USM API

Name	Type	Description
central_moment_2	<code>Type*</code>	Pointer to the output array of central moments of the 2nd order.
central_moment_3	<code>Type*</code>	Pointer to the output array of central moments of the 3rd order (optional).
central_moment_4	<code>Type*</code>	Pointer to the output array of central moments of the 4th order (optional).
event	<code>sycl::event</code>	Function returns event after submitting task in <code>sycl::queue</code> .

oneapi::mkl::stats::central_moment with User-provided Mean

- [Description](#)
- [API](#)

Description

Entry point to compute arrays of central moments up to the 4th order with mean provided by the user.

Mathematical Notation

Vector of central moments:

$$C^k(X) = (C_1^k(X), \dots, C_p^k(X)), \quad C_i^k(X) = \frac{1}{W} \sum_{j=1}^n \omega_j (x_{ij} - M_i(X))^k$$

Where:

$$W = \sum_{j=1}^n \omega_j, \quad i = 1, \dots, p, \quad k = 2, 3, 4$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void central_moment(sycl::queue& queue,
    sycl::buffer<Type, 1> mean,
    const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
    sycl::buffer<Type, 1> central_moment_2,
    sycl::buffer<Type, 1> central_moment_3 = {0},
    sycl::buffer<Type, 1> central_moment_4 = {0});
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event central_moment(sycl::queue& queue,
    Type* mean,
    const dataset<ObservationsLayout, Type*>& data,
    Type* central_moment_2,
    Type* central_moment_3 = nullptr,
    Type* central_moment_4 = nullptr,
    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

oneapi::mkl::stats::method Method =	Computation method. The specific values are as follows: oneapi::mkl::stats::method::fast
oneapi::mkl::stats::method::fast	

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::central_moment() routine submits kernels in this queue.
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Buffer API

Name	Type	Description
mean	sycl::buffer<Type, 1>	sycl::buffer to the array of mean values provided by the user.

USM API

Name	Type	Description
mean	Type*	Pointer to the output array of mean values provided by the user.

Optional Input Parameter for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event>&	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
central_moment_2	sycl::buffer<Type, 1>	sycl::buffer to the output array of central moments of the 2nd order.
central_moment_3	sycl::buffer<Type, 1>	sycl::buffer to the output array of central moments of the 3rd order.
central_moment_4	sycl::buffer<Type, 1>	sycl::buffer to the output array of central moments of the 4th order.

USM API

Name	Type	Description
central_moment_2	Type*	Pointer to the output array of central moments of the 2nd order.
central_moment_3	Type*	Pointer to the output array of central moments of the 3rd order.
central_moment_4	Type*	Pointer to the output array of central moments of the 4th order.
event	sycl::event	Function returns event after submitting task in sycl::queue.

oneapi::mkl::stats::mean

- [Description](#)
- [API](#)

Description

Entry point to compute the arrays of mean values.

Mathematical Notation

Vector of mean values:

$$M(X) = (M_1(X), \dots, M_p(X)), \quad M_i(X) = \frac{1}{W} \sum_{j=1}^n \omega_j x_{ij}$$

Where:

$$W = \sum_{j=1}^n \omega_j, i = 1, \dots, p$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void mean(sycl::queue& queue,
    const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
    sycl::buffer<Type, 1> mean);
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event mean(sycl::queue& queue,
    const dataset<ObservationsLayout, Type*>& data,
    Type* mean,
    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

Name	Description
oneapi::mkl::stats::method Method = oneapi::mkl::stats::method::fast	Computation method. The specific values are as follows: oneapi::mkl::stats::method::fast oneapi::mkl::stats::method::one_pass

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::mean() routine submits kernels in this queue.
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Optional Input Parameter for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event>&	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
mean	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of mean values.

USM API

Name	Type	Description
mean	<code>Type*</code>	Pointer to the output array of mean values.
event	<code>sycl::event</code>	Function returns event after submitting task in <code>sycl::queue</code> .

oneapi::mkl::stats::variation

- Description
- API

Description

Entry point to compute the arrays of variation coefficients.

Mathematical Notation

Vector of variation coefficients:

$$VC(X) = (VC_1(X), \dots, VC_p(X)), \quad VC_i(X) = \frac{V_i^{0.5}(X)}{M_i(X)}$$

Where:

$$i = 1, \dots, p$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void variation(sycl::queue& queue,
    const dataset< ObservationsLayout, sycl::buffer<Type, 1>>& data,
    sycl::buffer<Type, 1> variation;
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event variation(sycl::queue& queue,
    const dataset<ObservationsLayout, Type*>& data,
    Type* variation,
    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

Name	Description
oneapi::mkl::stats::method Method = oneapi::mkl::stats::method::fast	Computation method. The specific values are as follows: oneapi::mkl::stats::method::fast oneapi::mkl::stats::method::one_pass

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::variation() routine submits kernels in this queue.
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Optional Input Parameter for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event>&	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
variation	sycl::buffer<Type, 1>	sycl::buffer to the output array of variation coefficients.

USM API

Name	Type	Description
variation	Type*	Pointer to the output array of variation coefficients.
event	sycl::event	Function returns event after submitting task in sycl::queue.

oneapi::mkl::stats::variation with User-provided Mean

- [Description](#)
- [API](#)

Description

Entry point to compute the arrays of variation coefficients with the mean provided by the user.

Mathematical Notation

Vector of variation coefficients:

$$VC(X) = (VC_1(X), \dots, VC_p(X)), \quad VC_i(X) = \frac{V_i^{0.5}(X)}{M_i(X)}$$

Where:

$$i = 1, \dots, p$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void variation(sycl::queue& queue,
              sycl::buffer<Type, 1> mean,
              const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
              sycl::buffer<Type, 1> variaion);
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event variation(sycl::queue& queue,
                    Type* mean,
                    const dataset<ObservationsLayout, Type*>& data,
                    Type* variation,
                    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

Name	Description
oneapi::mkl::stats::method Method = oneapi::mkl::stats::method::fast	Computation method. The specific values are as follows: oneapi::mkl::stats::method::fast

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::variation() routine submits kernels in this queue.
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Buffer API

Name	Type	Description
mean	sycl::buffer<Type, 1>	sycl::buffer to the array of mean values provided by the user.

USM API

Name	Type	Description
mean	Type*	Pointer to the output array of mean values provided by the user.

Optional Input Parameter for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event>&	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
variation	sycl::buffer<Type, 1>	sycl::buffer to the output array of variation coefficients.

USM API

Name	Type	Description
variation	Type*	Pointer to the output array of variation coefficients.
event	sycl::event	Function returns event after submitting task in sycl::queue.

oneapi::mkl::stats::skewness

- [Description](#)
- [API](#)

Description

Entry point to compute the arrays of skewness values.

Mathematical Notation

Vector of skewness values:

$$\Gamma(X) = (\Gamma_1(X), \dots, \Gamma_p(X)), \Gamma_i(X) = \frac{C_i^{(3)}(X)}{V_i^{1.5}(X)}$$

Where:

$$i = 1, \dots, p$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void skewness(sycl::queue& queue,
              const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
              sycl::buffer<Type, 1> skewness);
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event skewness(sycl::queue& queue,
                    const dataset<ObservationsLayout, Type*>& data,
                    Type* skewness,
                    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

Name	Description
oneapi::mkl::stats::method Method = oneapi::mkl::stats::method::fast	Computation method. The specific values are as follows: oneapi::mkl::stats::method::fast oneapi::mkl::stats::method::one_pass

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::skewness() routine submits kernels in this queue.
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Optional Input Parameter for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event>&	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
skewness	sycl::buffer<Type, 1>	sycl::buffer to the output array of skewness values.

USM API

Name	Type	Description
skewness	Type*	Pointer to the output array of skewness values.
event	sycl::event	Function returns event after submitting task in sycl::queue.

oneapi::mkl::stats::skewness with User-provided Mean

- [Description](#)
- [API](#)

Description

Entry point to compute the arrays of skewness values with a mean provided by the user.

Mathematical Notation

Vector of skewness values:

$$\Gamma(X) = (\Gamma_1(X), \dots, \Gamma_p(X)), \quad \Gamma_i(X) = \frac{C_i^{(3)}(X)}{V_i^{1.5}(X)}$$

Where:

$$i = 1, \dots, p$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void skewness(sycl::queue& queue,
              sycl::buffer<Type, 1> mean,
              const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
              sycl::buffer<Type, 1> skewness);
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event skewness(sycl::queue& queue,
                    Type* mean,
                    const dataset<ObservationsLayout, Type*>& data,
                    Type* skewness,
                    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

Name	Description
oneapi::mkl::stats::method Method = oneapi::mkl::stats::method::fast	Computation method. The specific values are as follows: oneapi::mkl::stats::method::fast

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::skewness() routine submits kernels in this queue.
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Buffer API

Name	Type	Description
mean	sycl::buffer<Type, 1>	sycl::buffer to the array of mean values provided by the user.

USM API

Name	Type	Description
mean	Type*	Pointer to the output array of mean values provided by the user.

Optional Input Parameter for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event>&	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
skewness	sycl::buffer<Type, 1>	sycl::buffer to the output array of skewness values.

USM API

Name	Type	Description
variation	Type*	Pointer to the output array of variation coefficients.

Name	Type	Description
event	<code>sycl::event</code>	Function returns event after submitting task in <code>sycl::queue</code> .

oneapi::mkl::stats::kurtosis

- [Description](#)
- [API](#)

Description

Entry point to compute the array of kurtosis values.

Mathematical Notation

Vector of kurtosis values:

$$B(X) = (B_1(X), \dots, B_p(X)), \quad B_i(X) = \frac{C_i^{(4)}(X)}{V_i^2(X)} - 3$$

Where:

$$i = 1, \dots, p$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void kurtosis(sycl::queue& queue,
    const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
    sycl::buffer<Type, 1> kurtosis);
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event kurtosis(sycl::queue& queue,
    const dataset<ObservationsLayout, Type*>& data,
    Type* kurtosis,
    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- `oneapi/mkl/stats.hpp`

Template Parameters

<code>oneapi::mkl::stats::method Method =</code> <code>oneapi::mkl::stats::method::fast</code>	Computation method. The specific values are as follows: <code>oneapi::mkl::stats::method::fast</code> <code>oneapi::mkl::stats::method::one_pass</code>
---	---

Input Parameters

Name	Type	Description
queue	<code>sycl::queue&</code>	Valid <code>sycl::queue</code> , calls of the <code>oneapi::mkl::stats::kurtosis()</code> routine submits kernels in this queue.
data	<code>const dataset<ObservationsLayout, Type*>&</code>	Dataset which is used for estimates computation.

Optional Input Parameter for USM API

Name	Type	Description
dependencies	<code>const std::vector<sycl::event>&</code>	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
kurtosis	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the output array of kurtosis values.

USM API

Name	Type	Description
kurtosis	<code>Type*</code>	Pointer to the output array of kurtosis values.
event	<code>sycl::event</code>	Function returns event after submitting task in <code>sycl::queue</code> .

oneapi::mkl::stats::kurtosis with User-provided Mean

- [Description](#)
- [API](#)

Description

Entry point to compute the array of kurtosis values with a mean provided by the user.

Mathematical Notation

Vector of kurtosis values:

$$B(X) = (B_1(X), \dots, B_p(X)), \quad B_i(X) = \frac{C_i^{(4)}(X)}{V_i^2(X)} - 3$$

Where:

$$i = 1, \dots, p$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void kurtosis(sycl::queue& queue,
             sycl::buffer<Type, 1> mean,
             const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
             sycl::buffer<Type, 1> kurtosis);
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event kurtosis(sycl::queue& queue,
                    Type* mean,
                    const dataset<ObservationsLayout, Type*>& data,
                    Type* kurtosis,
                    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- `oneapi/mkl/stats.hpp`

Template Parameters

<code>oneapi::mkl::stats::method Method =</code> <code>oneapi::mkl::stats::method::fast</code>	Computation method. The specific values are as follows: <code>oneapi::mkl::stats::method::fast</code>
---	---

Input Parameters

Name	Type	Description
queue	<code>sycl::queue&</code>	Valid <code>sycl::queue</code> , calls of the <code>oneapi::mkl::stats::kurtosis()</code> routine submits kernels in this queue.
data	<code>const dataset<ObservationsLayout, Type*>&</code>	Dataset which is used for estimates computation.

Buffer API

Name	Type	Description
mean	<code>sycl::buffer<Type, 1></code>	<code>sycl::buffer</code> to the array of mean values provided by the user.

USM API

Name	Type	Description
mean	<code>Type*</code>	Pointer to the output array of mean values provided by the user

Optional Input Parameter for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event>&	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
kurtosis	sycl::buffer<Type, 1>	sycl::buffer to the output array of kurtosis values.

USM API

Name	Type	Description
kurtosis	Type*	Pointer to the output array of kurtosis values.
event	sycl::event	Function returns event after submitting task in sycl::queue.

oneapi::mkl::stats::min

- [Description](#)
- [API](#)

Description

Entry point to compute the array of minimum values.

Mathematical Notation

Vector of minimum values:

$$\min(X) = (\min_1(X), \dots, \min_p(X))$$

Where:

$$i = 1, \dots, p$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void min(
    sycl::queue& queue,
    const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
    sycl::buffer<Type, 1> min);
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event min(
    sycl::queue& queue,
    const dataset<ObservationsLayout, Type*>& data,
    Type* min,
    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

Name	Description
oneapi::mkl::stats::method Method = oneapi::mkl::stats::method::fast	Computation method. The specific values are as follows: oneapi::mkl::stats::method::fast

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::min() routine submits kernels in this queue.
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Optional Input Parameter for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event>&	List of events to wait for before starting computation, if any.

Output Parameters**Buffer API**

Name	Type	Description
min	sycl::buffer<Type, 1>	sycl::buffer to the output array of minimum values.

USM API

Name	Type	Description
min	Type*	Pointer to the output array of minimum values.
event	sycl::event	Function returns event after submitting task in sycl::queue.

oneapi::mkl::stats::max

- [Description](#)
- [API](#)

Description

Entry point to compute the array of maximum values.

Mathematical Notation

Vector of maximum values:

$$\max(X) = (\max_1(X), \dots, \max_p(X))$$

Where:

$$i = 1, \dots, p$$

API

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void max(
    sycl::queue& queue,
    const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
    sycl::buffer<Type, 1> max);
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event max(
    sycl::queue& queue,
    const dataset<ObservationsLayout, Type*>& data,
    Type* max,
    const std::vector<sycl::event> &dependencies = {});
```

Include Files

- `oneapi/mkl/stats.hpp`

Template Parameters

oneapi::mkl::stats::method Method =	Computation method. The specific values are as follows: oneapi::mkl::stats::method::fast
oneapi::mkl::stats::method::fast	

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::max() routine submits kernels in this queue.

Name	Type	Description
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Optional Input Parameter for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event>&	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
max	sycl::buffer<Type, 1>	sycl::buffer to the output array of maximum values.

USM API

Name	Type	Description
max	Type*	Pointer to the output array of maximum values.
event	sycl::event	Function returns event after submitting task in sycl::queue.

oneapi::mkl::stats::min_max

- [Description](#)

Description

Entry point to compute the array of minimum and maximum values simultaneously.

Mathematical Notation

Vector of maximum values:

$$\max(X) = (\max_1(X), \dots, \max_p(X))$$

Vector of minimum values:

$$\min(X) = (\min_1(X), \dots, \min_p(X))$$

Where:

$$i = 1, \dots, p$$

Syntax

Buffer API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
void min_max(sycl::queue& queue,
             const dataset<ObservationsLayout, sycl::buffer<Type, 1>>& data,
             sycl::buffer<Type, 1> min, sycl::buffer<Type, 1> max);
```

USM API

```
template<method Method = method::fast, typename Type, layout ObservationsLayout>
sycl::event min_max(sycl::queue& queue,
                   const dataset<ObservationsLayout, Type*>& data,
                   Type* min, Type* max,
                   const std::vector<sycl::event> &dependencies = {});
```

Include Files

- oneapi/mkl/stats.hpp

Template Parameters

Name	Description
oneapi::mkl::stats::method Method = oneapi::mkl::stats::method::fast	Computation method. The specific values are as follows: oneapi::mkl::stats::method::fast

Input Parameters

Name	Type	Description
queue	sycl::queue&	Valid sycl::queue, calls of the oneapi::mkl::stats::min_max() routine submits kernels in this queue.
data	const dataset<ObservationsLayout, Type*>&	Dataset which is used for estimates computation.

Optional Input Parameter for USM API

Name	Type	Description
dependencies	const std::vector<sycl::event>&	List of events to wait for before starting computation, if any.

Output Parameters

Buffer API

Name	Type	Description
min	sycl::buffer<Type, 1>	sycl::buffer to the output array of minimum values.
max	sycl::buffer<Type, 1>	sycl::buffer to the output array of maximum values.

USM API

Name	Type	Description
min	Type*	Pointer to the output array of minimum values.
max	Type*	Pointer to the output array of maximum values.
event	sycl::event	Function returns event after submitting task in sycl::queue.

Fourier Transform Functions

Definitions

Let z be a set of M finite d -dimensional discrete sequences z^m of lengths $n_1 \times n_2 \times \cdots \times n_d$ ($d \in \mathbb{Z}_{>0}$, $M \in \mathbb{Z}_{>0}$, $n_j \in \mathbb{Z}_{>0} \forall j \in \{1, 2, \dots, d\}$, and $m \in \{0, 1, \dots, M-1\}$). Its m -th sequence entries are denoted by $z_{k_1, k_2, \dots, k_d}^m$ wherein integer indices k_j are such that $0 \leq k_j < n_j$, $\forall j \in \{1, 2, \dots, d\}$.

The Discrete Fourier Transform (DFT) of z^m is the finite d -dimensional $n_1 \times n_2 \times \cdots \times n_d$ discrete sequence $\hat{z}^m$ with entries

$$\hat{z}_{k_1, k_2, \dots, k_d}^m = \sigma_\delta \sum_{j_d=0}^{n_d-1} \cdots \sum_{j_2=0}^{n_2-1} \sum_{j_1=0}^{n_1-1} z_{j_1, j_2, \dots, j_d}^m \exp \left[\delta 2\pi i \left(\sum_{\ell=1}^d \frac{j_\ell k_\ell}{n_\ell} \right) \right], \forall m \in \{0, 1, \dots, M-1\}$$

where $i^2 = -1$. In the above equation, $\delta = \pm 1$ determines one of the two “directions” of the DFT and $\sigma_\delta \in \mathbb{R}$ represents a scaling factor associated with that direction. $\delta = -1$ defines the “forward DFT” (using the forward scaling factor σ_{-1}) while $\delta = +1$ defines the “backward DFT” (using the backward scaling factor σ_{+1}).

The calculation of M identically-defined DFTs for several data sets is referred to as a “batched DFT” and M is called the “batch size”.

Types of forward domains

The domain of input (resp. output) discrete sequences for a forward (resp. backward) DFT is referred to as “forward domain”. Conversely, the domain of output (resp. input) discrete sequences for a forward (resp. backward) DFT is referred to as “backward domain”. Two types of forward domains may be considered:

- the set of complex d -dimensional sequences, referred to as “complex forward domain”;
- the set of real d -dimensional sequences, referred to as “real forward domain”.

Similarly, DFTs of complex (resp. real) forward domain are referred to as “complex DFTs” (resp. “real DFTs”). Regardless of the type of forward domain, the backward domain’s data sequences are always complex although with constraints for real transforms (explained below).

Elementary range of non-redundant entries

In general, the data entries $(\cdot)_{k_1, k_2, \dots, k_d}^m$ such that $0 \leq k_\ell < n_\ell, \forall \ell \in \{1, \dots, d\}$ and $0 \leq m < M$ suffice to determine any of the M relevant d -dimensional sequences unambiguously. In case of real DFTs, the data sequences in backward domain can be fully determined from a smaller set of entries. Indeed, if all entries of z are real in the equation above, then the entries of $\hat{z}$ are complex and, for any

$m \in \{0, 1, \dots, M-1\}$, $(\hat{z}_{k_1, k_2, \dots, k_d}^m)^* = \hat{z}_{j_1, j_2, \dots, j_d}^m$ where $j_\ell = (n_\ell - k_\ell) \pmod{n_\ell}, \forall \ell \in \{1, \dots, d\}$ and λ^* represents the conjugate of complex number λ .

This conjugate symmetry relation makes roughly half the data redundant in backward domain: in case of real DFTs, the data sequences in backward domain can be fully determined from any non-redundant set of entries

$(\cdot)_{k_1, k_2, \dots, k_d}^m$ such that $0 \leq m < M, 0 \leq k_\ell < n_\ell, \forall \ell \in \{1, \dots, d\} \setminus \{p\}$, and $0 \leq k_p \leq \lfloor \frac{n_p}{2} \rfloor$ for any $p \in \{1, \dots, d\}$. In oneMKL, the convention $p = d$ is used for capturing such an elementary set of non-redundant entries for data sequences belonging to the backward domain of real DFTs.

In other words, oneMKL expects and produces a set of Md -dimensional *finite* data sequences $(\cdot)_{k_1, k_2, \dots, k_d}^m$ such that

- $0 \leq m < M$;
- $0 \leq k_j < n_j, \forall j \in \{1, \dots, d-1\}$, if $d > 1$;
- $0 \leq k_d < n_d$, except for data sequences in the backward domain of real DFTs;
- $0 \leq k_d \leq \lfloor \frac{n_d}{2} \rfloor$, for data sequences in the backward domain of real DFTs.

Finally, note that the conjugate symmetry relation further constrains some of the entries (or pairs thereof) of data sequences in the backward domain of real DFTs. Specifically,

- the imaginary part must be 0 for any entry $(\cdot)_{k_1, k_2, \dots, k_d}^m$ such that $k_\ell \equiv (n_\ell - k_\ell) \pmod{n_\ell}, \forall \ell \in \{1, \dots, d\}$, e.g., entry $(\cdot)_{0, 0, \dots, 0}^m$;
- pairs of entries $(\cdot)_{k_1, k_2, \dots, k_d}^m$ and $(\cdot)_{j_1, j_2, \dots, j_d}^m$ such that $k_\ell \equiv (n_\ell - j_\ell) \pmod{n_\ell}, \forall \ell \in \{1, \dots, d\}$ must be complex conjugates of one another, e.g., entries $(\cdot)_{1, 0, \dots, 0}^m$ and $(\cdot)_{n_1-1, 0, \dots, 0}^m$ must be complex conjugates (note that this example falls back into the latter category if $n_1 = 2$).

NOTE The behavior of oneMKL is undefined for real backward DFT if the input data does not satisfy those constraints. It is the user's responsibility to guarantee that these constraints are satisfied by the input data for real backward DFTs.

DPC++ support

The Intel® oneAPI Math Kernel Library (oneMKL) provides a DPC++ interface for computing Discrete Fourier Transforms. This interface declares the `oneapi::mkl::dft` namespace, which contains

- the [scoped enumerations](#) `config_param`, `precision`, `domain`, and `config_value`;
- the descriptor [class template](#);
- the `compute_forward` and `compute_backward` [function templates](#).

For a DFT of forward domain and floating-point format represented by the values `dom` and `prec` (known at compile time) of respective types `oneapi::mkl::dft::domain` and `oneapi::mkl::dft::precision` (see the [scoped enumerations](#)), the desired transform and its general configuration are to be communicated via an object of the `oneapi::mkl::dft::descriptor<prec, dom>` class. Once successfully committed to the desired DFT configuration and to a user-provided `sycl::queue` instance, that `descriptor` object can be used as an argument to the appropriate compute function(s) along with the relevant input and output data.

Unless a [user-allocated workspace](#) is used, the DPC++ interface computes a DFT in four steps (omitting the prepended namespace specifiers `oneapi::mkl::dft` below, for conciseness):

1. Create a `descriptor` object `desc` for the targeted DFT problem with a call to the relevant [parameterized constructor](#), e.g.,

```
descriptor<prec, dom> desc(lengths);
```

wherein `prec` and `dom` are specialization values of types `precision` and `domain`, respectively. `desc` captures the configuration of the transform, such as the dimensionality (or rank), length(s), number of transforms, layout of the input and output data (defined by strides, distances, and possibly other configuration parameters), scaling factors, etc. All the configuration settings are assigned default values in this call, which might need to be modified thereafter.

By default, `descriptor` objects are initialized for the in-place calculation of an unbatched ($M = 1$), unscaled ($\sigma_\delta = 1, \forall \delta$) DFT of the forward domain, precision and length(s) set at construction.

2. Optionally adjust the configuration of `desc` by calling its relevant [configuration-setting member function\(s\)](#) as many times as needed, e.g., to specify a data storage layout different than default. The value associated with (almost) any configuration parameter can be obtained with the appropriate [configuration-querying member function\(s\)](#) (default values are returned unless the queried configuration parameter was previously set).
3. Commit `desc` with a call to its `commit` [member function](#); that is, make the object ready to compute the transform it defines on a specific device. Once the object is committed, its configuration parameters, such as the type and number of transforms, strides and distances, the data storage conventions, and so on, are considered frozen for computation purposes: changing any of them after committing the object effectively invalidates it for computation purposes until its `commit` member function is called again. The `commit` function takes in a `sycl::queue` argument which determines all the relevant runtime-related information (including the desired computation device).
4. Use the committed `desc` in as many calls to the (appropriate) `compute_forward` or `compute_backward` [functions](#) as needed to compute the desired transform(s). These functions require no other argument than a committed descriptor object (which fully defines the desired operations) and the device-accessible input and output data.

Some usage examples are provided in the section dedicated to [configuring and computing a DFT in DPC++](#).

NOTE For best performance in applications that require several identically-configured DFT computations to be executed, it is *strongly recommended* to create, configure and commit DFT descriptors outside of applications' hotpath(s), e.g., as an initialization step. `compute_forward` and `compute_backward` should be the only DFT-related functions invoked in performance-critical sections.

Scoped enumerations

This page describes the scoped enumerations pertaining to the definition or configuration of `oneapi::mkl::dft::descriptor` objects: `oneapi::mkl::dft::precision`, `oneapi::mkl::dft::domain`, `oneapi::mkl::dft::config_param`, and `oneapi::mkl::dft::config_value`.

```
namespace oneapi::mkl::dft {
    enum class precision {
        SINGLE = DFTI_SINGLE,
        DOUBLE = DFTI_DOUBLE
    };

    enum class domain {
        REAL = DFTI_REAL,
        COMPLEX = DFTI_COMPLEX
    };

    enum class config_param {
        FORWARD_DOMAIN = DFTI_FORWARD_DOMAIN,
        DIMENSION = DFTI_DIMENSION,
        LENGTHS = DFTI_LENGTHS,
        PRECISION = DFTI_PRECISION,
        FORWARD_SCALE = DFTI_FORWARD_SCALE,
        BACKWARD_SCALE = DFTI_BACKWARD_SCALE,
        NUMBER_OF_TRANSFORMS = DFTI_NUMBER_OF_TRANSFORMS,
        COMPLEX_STORAGE = DFTI_COMPLEX_STORAGE,
        CONJUGATE_EVEN_STORAGE [[deprecated]] = DFTI_CONJUGATE_EVEN_STORAGE,
        PLACEMENT = DFTI_PLACEMENT,
        INPUT_STRIDES [[deprecated("Use FWD/BWD strides instead.")]] = DFTI_INPUT_STRIDES,
        OUTPUT_STRIDES [[deprecated("Use FWD/BWD strides instead.")]] = DFTI_OUTPUT_STRIDES,
        FWD_DISTANCE = DFTI_FWD_DISTANCE,
        BWD_DISTANCE = DFTI_BWD_DISTANCE,
        WORKSPACE = DFTI_WORKSPACE,
        PACKED_FORMAT [[deprecated]] = DFTI_PACKED_FORMAT,
        COMMIT_STATUS = DFTI_COMMIT_STATUS,
        VERSION [[deprecated]] = DFTI_VERSION,
        THREAD_LIMIT = DFTI_THREAD_LIMIT,
        DESTROY_INPUT = DFTI_DESTROY_INPUT,
        WORKSPACE_ESTIMATE_BYTES,
        WORKSPACE_BYTES,
        FWD_STRIDES,
        BWD_STRIDES,
        WORKSPACE_PLACEMENT, // alias for WORKSPACE
        WORKSPACE_EXTERNAL_BYTES // alias for WORKSPACE_BYTES
    };

    enum class config_value {
        COMMITTED = DFTI_COMMITTED,
        UNCOMMITTED = DFTI_UNCOMMITTED,
        COMPLEX_COMPLEX = DFTI_COMPLEX_COMPLEX,
        REAL_REAL = DFTI_REAL_REAL,
        INPLACE = DFTI_INPLACE,
        NOT_INPLACE = DFTI_NOT_INPLACE,
        WORKSPACE_AUTOMATIC, // alias for WORKSPACE_INTERNAL
        COMPLEX_REAL [[deprecated]] = DFTI_COMPLEX_REAL,
        ALLOW = DFTI_ALLOW,
        AVOID = DFTI_AVOID,
        CCE_FORMAT [[deprecated]] = DFTI_CCE_FORMAT,
    };
}
```

```

    PERM_FORMAT          [[deprecated]] = DFTI_PERM_FORMAT,
    PACK_FORMAT          [[deprecated]] = DFTI_PACK_FORMAT,
    CCS_FORMAT           [[deprecated]] = DFTI_CCS_FORMAT,
    WORKSPACE_INTERNAL,
    WORKSPACE_EXTERNAL
};
}

```

These scoped enumerations belong to the `oneapi::mkl::dft` namespace and are declared in `oneapi/mkl/dft.hpp` (file to be included). The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

All *configuration parameters* of oneMKL `descriptor` objects are captured by an enumerator in `config_param` and associated with a *configuration value*. When the latter is not representable with a primitive or derived data type, it is representable with an enumerator in `domain`, `precision` or `config_value`.

The default configuration values associated with all writable configuration parameters are documented in the page dedicated to the [configuration-setting member function\(s\)](#) of the `descriptor` class template, or in the page specific to [configuring data layouts](#).

`config_param`

This scoped enumeration type is used to represent all configuration parameters of oneMKL DFT `descriptor` objects. For any `descriptor` object, all its configuration parameters are associated with a configuration value. The type of the latter depends on the configuration parameter and is documented in the page specific to the [configuration-setting](#) (or [configuration-querying](#), for read-only parameters) member functions of the `descriptor` class template.

Enumerators of the `config_param` scoped enumeration along with the corresponding configuration parameters of `descriptor` objects. The table uses the notations and mathematical symbols presented in the [introduction](#) when referring to parameters of the DFT operation defined by the object.

Enumerator(s)	Corresponding configuration parameter.
<code>config_param::FORWARD_DOMAIN</code>	Type of forward domain for the DFT.
<code>config_param::DIMENSION</code>	Dimension d of the DFT.
<code>config_param::LENGTHS</code>	Length(s) $\{n_1, n_2, \dots, n_d\}$ of the DFT.
<code>config_param::PRECISION</code>	Floating-point format for the DFT.
<code>config_param::FORWARD_SCALE</code>	Scaling factor of the forward DFT, i.e., σ_{-1} .
<code>config_param::BACKWARD_SCALE</code>	Scaling factor of the backward DFT, i.e., σ_{+1} .
<code>config_param::NUMBER_OF_TRANSFORMS</code>	Batch size M of the DFT.
<code>config_param::PLACEMENT</code>	Configuration parameter indicating whether the DFT computation is performed in-place or not. Refer to configuring the placement of the DFT results for more details.

Enumerator(s)	Corresponding configuration parameter.
<code>config_param::COMPLEX_STORAGE</code>	Elementary data type to be considered in either domain by a complex descriptor (irrelevant parameter for real descriptors). Refer to configuring data layouts for more details.
<code>config_param::CONJUGATE_EVEN_STORAGE</code>	Elementary data type to be considered in the backward domain by a real descriptor (irrelevant parameter for complex descriptors). Refer to configuring data layouts for more details. This parameter is deprecated (support for non-default values will be dropped).
<code>config_param::INPUT_STRIDES</code>	Index offset and strides to be considered when accessing entries in the relevant input data container provided by the user. Refer to configuring data layouts for more details. This parameter is deprecated; it is recommended to use <code>config_param::FWD_STRIDES</code> and <code>config_param::BWD_STRIDES</code> instead.
<code>config_param::OUTPUT_STRIDES</code>	Index offset and strides to be considered when accessing entries in the relevant output data container provided by the user. Refer to configuring data layouts for more details. This parameter is deprecated; it is recommended to use <code>config_param::FWD_STRIDES</code> and <code>config_param::BWD_STRIDES</code> instead.
<code>config_param::FWD_STRIDES</code>	Index offset and strides to be considered when accessing entries in the forward domain's data container provided by the user. Refer to configuring data layouts for more details.
<code>config_param::BWD_STRIDES</code>	Index offset and strides to be considered when accessing entries in the backward domain's data container provided by the user. Refer to configuring data layouts for more details.
<code>config_param::FWD_DISTANCE</code>	Distance to be considered when accessing entries in the forward domain's data container provided by the user. Refer to configuring data layouts for more details.
<code>config_param::BWD_DISTANCE</code>	Distance to be considered when accessing entries in the backward domain's data container provided by the user. Refer to configuring data layouts for more details.
<code>config_param::PACKED_FORMAT</code>	Data storage convention to be used in the backward domain of real descriptors (irrelevant for complex descriptors). Refer to configuring data

Enumerator(s)	Corresponding configuration parameter.
	layouts for more details. This parameter is deprecated (support for non-default values will be dropped).
<code>config_param::WORKSPACE</code> <code>config_param::WORKSPACE_PLACEMENT</code>	Parameter indicating whether the object's workspace is to be managed by the user or by oneMKL. The two enumerators are equivalent to one another and may be used interchangeably. Refer to providing an externally-allocated workspaces for more details.
<code>config_param::WORKSPACE_BYTES</code> <code>config_param::WORKSPACE_EXTERNAL_BYTES</code>	Exact size of the object's workspace, in bytes. Refer to providing an externally-allocated workspaces for more details.
<code>config_param::WORKSPACE_ESTIMATE_BYTES</code>	Conservative estimate size of the object's workspace, in bytes. Refer to providing an externally-allocated workspaces for more details.
<code>config_param::DESTROY_INPUT</code>	Configuration parameter controlling whether the object is allowed to overwrite input data or not, when configured for out-of-place operations.
<code>config_param::THREAD_LIMIT</code>	Limit on the number of (CPU) threads that may be used by the object when computing a DFT. This configuration parameter is irrelevant for objects committed to GPU devices.
<code>config_param::COMMIT_STATUS</code>	Configuration parameter indicating whether the object is committed to a DFT configuration and to a user-provided <code>sycl::queue</code> instance or not.
<code>config_param::VERSION</code>	Version of oneMKL being used. This parameter is deprecated.

NOTE The configuration parameters represented by `config_param::DIMENSION`, `config_param::FORWARD_DOMAIN`, `config_param::LENGTHS`, `config_param::PRECISION`, `config_param::COMMIT_STATUS`, `config_param::VERSION`, `config_param::WORKSPACE_ESTIMATE_BYTES`, `config_param::WORKSPACE_BYTES`, `config_param::WORKSPACE_EXTERNAL_BYTES` are all *read-only*: they are not writable.

precision

This scoped enumeration type is used to represent the configuration value associated with the configuration parameter `config_param::PRECISION` of oneMKL DFT `descriptor` objects. Its enumerators identify the various floating-point formats that may be considered. For any oneMKL DFT `descriptor` object, that configuration value is bound to its type and immutable given its usage as a specialization value for the [descriptor class template](#).

Enumerators of the `precision` scoped enumeration along with the corresponding floating-point format considered by descriptor objects.

Enumerator	Corresponding floating-point format
<code>precision::SINGLE</code>	Single-precision floating-point format (FP32).
<code>precision::DOUBLE</code>	Double-precision floating-point format (FP64).

`domain`

This scoped enumeration type is used to represent the configuration value associated with the configuration parameter `config_param::FORWARD_DOMAIN` of oneMKL DFT descriptor objects. Its enumerators identify the [types of forward domain](#) that may be considered. For any oneMKL DFT descriptor object, that configuration value is bound to its type and immutable given its usage as a specialization value for the [descriptor class template](#).

Enumerators of the `domain` scoped enumeration along with the corresponding type of forward domain considered by descriptor objects.

Enumerator	Corresponding type of forward domain
<code>domain::REAL</code>	Real forward domain.
<code>domain::COMPLEX</code>	Complex forward domain.

`config_value`

This scoped enumeration type is used to represent configuration values that cannot be represented with `precision`, `domain`, primitive or derived data types for some configuration parameters of oneMKL DFT descriptor objects.

Enumerators of the `config_value` scoped enumeration along with the matching configuration parameter for descriptor objects.

Enumerator(s)	Matching configuration parameter(s)
<code>config_value::COMMITTED</code> <code>config_value::UNCOMMITTED</code>	Possible configuration values associated with the configuration parameter <code>config_param::COMMIT_STATUS</code> indicating whether a descriptor was successfully committed (<code>config_value::COMMITTED</code>) or not (<code>config_value::UNCOMMITTED</code>).
<code>config_value::COMPLEX_COMPLEX</code>	Possible configuration value associated with the configuration parameters <code>config_param::COMPLEX_STORAGE</code> or <code>config_param::CONJUGATE_EVEN_STORAGE</code> . Refer to configuring data layouts for more details.
<code>config_value::REAL_REAL</code>	Possible configuration value associated with the configuration parameter <code>config_param::COMPLEX_STORAGE</code> . Refer to configuring data layouts for more details.

Enumerator(s)	Matching configuration parameter(s)
<code>config_value::COMPLEX_REAL</code>	Possible configuration value associated with the configuration parameter <code>config_param::CONJUGATE_EVEN_STORAGE</code> . Refer to configuring data layouts for more details. This value is deprecated (support for a non-default value associated with <code>config_param::CONJUGATE_EVEN_STORAGE</code> will be dropped)
<code>config_value::INPLACE</code> <code>config_value::NOT_INPLACE</code>	Possible configuration values associated with the configuration parameter <code>config_param::PLACEMENT</code> indicating whether the DFT computation must be performed in-place (<code>config_value::INPLACE</code>) or not (<code>config_value::NOT_INPLACE</code>).
<code>config_value::WORKSPACE_AUTOMATIC</code> <code>config_value::WORKSPACE_INTERNAL</code> <code>config_value::WORKSPACE_EXTERNAL</code>	Possible configuration values associated with the configuration parameter <code>config_param::WORKSPACE</code> (or its alias <code>config_param::WORKSPACE_PLACEMENT</code>). The two enumerators <code>config_value::WORKSPACE_AUTOMATIC</code> and <code>config_value::WORKSPACE_INTERNAL</code> are equivalent to one another and may be used interchangeably. Refer to providing an externally-allocated workspaces for more details.
<code>config_value::ALLOW</code> <code>config_value::AVOID</code>	Possible configuration values associated with the configuration parameter <code>config_param::DESTROY_INPUT</code> indicating whether a descriptor configured for out-of-place operations is allowed to overwrite the input data when computing a DFT (<code>config_value::ALLOW</code>) or not (<code>config_value::AVOID</code>). Refer to allowing destruction of input data for more details.
<code>config_value::CCE_FORMAT</code> <code>config_value::PERM_FORMAT</code> <code>config_value::PACK_FORMAT</code> <code>config_value::CCS_FORMAT</code>	Possible configuration values associated with the configuration parameter <code>config_param::PACKED_FORMAT</code> . Refer to configuring data layouts for more details. These values are deprecated (support for non-default values associated with <code>config_param::PACKED_FORMAT</code> will be dropped)

Deprecations

Several enumerators in the scoped enumerations documented above are explicitly assigned values from the oneMKL C interface's (unscoped) enumerations `DFTI_CONFIG_PARAM` or `DFTI_CONFIG_VALUE`. The support for these values via the oneMKL DPC++ interface is now *deprecated*. Wherever such an unscoped enumerator, say `DFTI_X`, has been used (or usable) in the oneMKL DPC++ interface, a corresponding scoped enumerator `domain::X`, `precision::X`, `config_param::X` or `config_value::X` does replace it now as a

better-suited substitute. The explicit assignments of the former to the latter will be removed upon completion of the deprecation period for supporting `DFTI_CONFIG_PARAM` and `DFTI_CONFIG_VALUE` enumerators via the DPC++ interface of oneMKL.

Explicitly, users are advised to use

- `precision::SINGLE` and `precision::DOUBLE` instead of `DFTI_SINGLE` and `DFTI_DOUBLE`, respectively;
- `domain::REAL` and `domain::COMPLEX` instead of `DFTI_REAL` and `DFTI_COMPLEX`, respectively;
- `config_param::FORWARD_DOMAIN`, `config_param::DIMENSION`, `config_param::LENGTHS`, `config_param::PRECISION`, `config_param::FORWARD_SCALE`, `config_param::BACKWARD_SCALE`, `config_param::NUMBER_OF_TRANSFORMS`, `config_param::COMPLEX_STORAGE`, `config_param::CONJUGATE_EVEN_STORAGE`, `config_param::PLACEMENT`, `config_param::INPUT_STRIDES`, `config_param::OUTPUT_STRIDES`, `config_param::FWD_DISTANCE`, `config_param::BWD_DISTANCE`, `config_param::WORKSPACE`, `config_param::PACKED_FORMAT`, `config_param::COMMIT_STATUS`, `config_param::VERSION`, `config_param::THREAD_LIMIT` and `config_param::DESTROY_INPUT` instead of `DFTI_FORWARD_DOMAIN`, `DFTI_DIMENSION`, `DFTI_LENGTHS`, `DFTI_PRECISION`, `DFTI_FORWARD_SCALE`, `DFTI_BACKWARD_SCALE`, `DFTI_NUMBER_OF_TRANSFORMS`, `DFTI_COMPLEX_STORAGE`, `DFTI_CONJUGATE_EVEN_STORAGE`, `DFTI_PLACEMENT`, `DFTI_INPUT_STRIDES`, `DFTI_OUTPUT_STRIDES`, `DFTI_FWD_DISTANCE`, `DFTI_BWD_DISTANCE`, `DFTI_WORKSPACE`, `DFTI_PACKED_FORMAT`, `DFTI_COMMIT_STATUS`, `DFTI_VERSION`, `DFTI_THREAD_LIMIT` and `DFTI_DESTROY_INPUT`, respectively;
- `config_value::COMMITTED`, `config_value::UNCOMMITTED`, `config_value::COMPLEX_COMPLEX`, `config_value::REAL_REAL`, `config_value::INPLACE`, `config_value::NOT_INPLACE`, `config_value::COMPLEX_REAL`, `config_value::ALLOW`, `config_value::AVOID`, `config_value::CCE_FORMAT`, `config_value::PERM_FORMAT`, `config_value::PACK_FORMAT` and `config_value::CCS_FORMAT` instead of `DFTI_COMMITTED`, `DFTI_UNCOMMITTED`, `DFTI_COMPLEX_COMPLEX`, `DFTI_REAL_REAL`, `DFTI_INPLACE`, `DFTI_NOT_INPLACE`, `DFTI_COMPLEX_REAL`, `DFTI_ALLOW`, `DFTI_AVOID`, `DFTI_CCE_FORMAT`, `DFTI_PERM_FORMAT`, `DFTI_PACK_FORMAT` and `DFTI_CCS_FORMAT`, respectively.

The descriptor class template

This page describes the `descriptor` class template, which belongs to the `oneapi::mkl::dft` namespace and is declared in `oneapi/mkl/dft.hpp` (file to be included).

```
namespace oneapi::mkl::dft {
    template <precision prec, domain dom>
    class descriptor;
}
```

The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

Users of the DPC++ interface of oneMKL must use instances of a specialization of this class template to specify and fully configure the required DFT computation(s): successfully-committed objects of a (specialized) `descriptor` class are required arguments to the DFT-specific [compute functions](#).

Template parameters

The `descriptor` class template is parameterized by two non-type template parameters, in the following order:

- a value of [type](#) `precision`, that determines the floating-point format to be considered by its instances;
- a value of [type](#) `domain`, that determines the type of forward domain to be considered by its instances.

Instances of a `descriptor` class specialized with value `precision::SINGLE` (resp. `precision::DOUBLE`) for the former are referred to as “single-precision descriptors” (resp. “double-precision descriptors”). Similarly, instances of a `descriptor` class specialized with value `domain::COMPLEX` (resp. `domain::REAL`) for the latter are referred to as “complex descriptors” (resp. “real descriptors”).

NOTE Some GPU devices do not support double-precision descriptors. It is recommended to verify that the targeted device does support double-precision floating-point arithmetic before using a double-precision descriptor on that device.

Member functions

Considering objects of any `descriptor` class, users may

- create, assign, and delete them via the [constructors, assignment operators, and destructors](#);
- modify their configuration via the [configuration-setting member functions](#);
- query their configuration via the [configuration-querying member functions](#);
- [commit](#) them to the DFT computation(s) that they define onto a specific `sycl::queue` instance;
- equip them with their own device-accessible memory allocation for the object’s workspace, by using the [workspace-related member functions](#) (feature specific to GPU-committed descriptors).

Constructors, assignment operators, and destructor

This page describes the constructors and destructor of the `descriptor` class template, which belongs to the `oneapi::mkl::dft` namespace and is declared in `oneapi/mkl/dft.hpp` (file to be included).

```
namespace oneapi::mkl::dft {
    template <precision prec, domain dom>
    class descriptor {
    public:
        // parameterized constructors:
        descriptor(std::vector<std::int64_t> dimensions);
        descriptor(std::int64_t length);
        // destructor:
        ~descriptor();
        // unsupported copy constructor and assignment operator:
        descriptor(const descriptor&) = delete;
        descriptor& operator=(const descriptor&) = delete;
        // move constructor and assignment operator:
        descriptor(descriptor&);
        descriptor& operator=(descriptor&);
    }
}
```

The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

Parameterized constructors

The parameterized constructors of any `descriptor` class allocate memory for an object’s data structures and default-configure it for the precision, forward domain, and length(s) of the transform it defines. These constructors do not trigger any significant computational work in preparation of the DFT that the object defines upon creation; all such tasks are operated when the object is [committed](#) to its DFT definition and to a given `sycl::queue` instance.

Input parameter

The input parameter of either constructor is detailed in the tables below. The tables use the notations presented in the [introduction](#) when referring to parameters of the DFT operation defined by the object.

Parameterized constructor's input parameter for a `descriptor` object defining a one-dimensional DFT, i.e., $d = 1$.

Name	Type	Description
length	<code>std::int64_t</code>	Value of n_1 (length of the one-dimensional DFT)

Parameterized constructor's input parameter for any `descriptor` object.

Name	Type	Description
lengths	<code>std::vector<std::int64_t></code>	Vector of size d , containing $\{n_1, n_2, \dots, n_d\}$, in that order (lengths of DFT)

Exceptions

The parameterized constructors may throw an `std::runtime_error` if

- $d > 3$;
- the construction of the `descriptor` object fails to allocate its required resources.

Copy constructor and assignment operator

Copy construction and assignment are not yet supported.

Move constructor and assignment operator

Descriptor objects can be move constructed and assigned. A moved-from descriptor must be reinitialized, e.g. via assignment, before reuse. Improper reuse of a moved-from descriptor may throw a `oneapi::mkl::uninitialized` exception.

Destructor

The destructor of any `descriptor` class frees all resources allocated for and by objects of that class.

Configuration-setting member functions

This page describes the overloaded configuration-setting member functions `set_value` of the `descriptor` class template, which belongs to the `oneapi::mkl::dft` namespace and is declared in `oneapi/mkl/dft.hpp` (file to be included).

```
namespace oneapi::mkl::dft {
    template <precision prec, domain dom>
    class descriptor {
        using real_scalar_t = std::conditional_t<prec == precision::DOUBLE, double, float>;
    public:
        // for integer-valued parameters:
        void set_value(config_param param, std::int64_t value);
        template <typename T, std::enable_if_t<std::is_integral_v<T>, bool> = true>
        void set_value(config_param param, T value) {
            set_value(param, static_cast<std::int64_t>(value));
        }
    }
```

```

// for real-valued parameters:
void set_value(config_param param, real_scalar_t value);
template <typename T, std::enable_if_t<std::is_floating_point_v<T>, bool> = true>
void set_value(config_param param, T value) {
    set_value(param, static_cast<real_scalar_t>(value));
}
// for vector-valued parameters:
void set_value(config_param param, const std::vector<std::int64_t>& value);
// for other parameters:
void set_value(config_param param, config_value value);
// deprecated usage, will be removed in a future release:
[[deprecated("Use set_value(config_param, config_value) instead.")]]
void set_value(config_param param, DFTI_CONFIG_VALUE value);
[[deprecated("Use set_value(config_param, const std::vector<std::int64_t>&) instead.")]]
void set_value(config_param param, const std::int64_t* value);
[[deprecated("This set_value member function is deprecated.")]]
void set_value(config_param param, ...);
}
}

```

The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

The `set_value` functions enable oneMKL users to assign a configuration value to any *writable configuration parameter* of a descriptor object. In this page, the default configuration values associated with such parameters are also documented, except for parameters `config_param::FWD_STRIDES` and `config_param::BWD_STRIDES` (the reader is referred to the page dedicated to [configuring data layouts](#) for the default configuration values associated with those).

All `set_value` functions expect *exactly* two arguments: first the calling object's configuration parameter to be set (as a `config_param` *enumerator*), second the value to be assigned to that parameter. The type of the latter depends on the parameter being set, and its acceptable values may be restricted. Note that pairs of configuration parameters and values labeled as "accepted" in tables below are not guaranteed to be unconditionally *supported* by descriptor objects. Configuration values may be successfully and correctly set for a descriptor object but found invalid later on, when attempting to *commit* that object. Assessing the validity of a descriptor object's configuration requires knowledge (and analysis) of *all* its configuration values (considered "frozen" when the object is committed).

Any call to `set_value` resulting in a configuration change for a *committed* descriptor object effectively uncommits that object: indeed, such a change typically invalidates the object's compute-readiness preparation steps operated when it was last *committed*. As a consequence, any such operation changes the object's (read-only) configuration value associated with configuration parameter `config_param::COMMIT_STATUS` from `config_value::COMMITTED` to `config_value::UNCOMMITTED`.

Setting integer-valued configuration parameters

The following member functions

```

template <precision prec, domain dom>
void descriptor<prec, dom>::set_value(config_param param, std::int64_t value);
template <precision prec, domain dom>
template <typename T, std::enable_if_t<std::is_integral_v<T>, bool> = true>
void descriptor<prec, dom>::set_value(config_param param, T value) {
    set_value(param, static_cast<std::int64_t>(value));
}

```

enable users to set configuration values for any `param` in the table below.

Integer-valued configuration parameters along with their accepted and default values.

Accepted configuration parameter	Accepted values	Default value
<code>config_param::FWD_DISTANCE</code>	All integers	0
<code>config_param::BWD_DISTANCE</code>	All integers	0
<code>config_param::NUMBER_OF_TRANSFORMS</code>	Positive integers	1
<code>config_param::THREAD_LIMIT</code>	Non-negative integers	0

These member functions may also be used to set any of the ([real-valued](#)) parameters `config_param::FORWARD_SCALE` or `config_param::BACKWARD_SCALE` so long as the user-provided integer value is representable as a double value without any loss of accuracy (if not, a `oneapi::mkl::invalid_argument` exception is thrown).

Deprecations

The configuration parameters `config_param::CONJUGATE_EVEN_STORAGE`, `config_param::PACKED_FORMAT`, `config_param::COMPLEX_STORAGE`, `config_param::PLACEMENT` and `config_param::DESTROY_INPUT` may also be set using the [above functions](#). The accepted and default values correspond to the integer values explicitly assigned to the enumerators of (unscoped) enumeration type `DFTI_CONFIG_VALUE` that are explicitly assigned themselves to the corresponding `config_value` enumerators documented [below](#) for those parameters. This usage is deprecated due to the deprecation of support for `DFTI_CONFIG_VALUE` enumerators via the DPC++ interface of oneMKL. Using the above member function for setting any such parameter triggers a runtime deprecation warning to be emitted.

NOTE

- Other integer types than `std::int64_t` may be used but such values *must* be within the representable range of `std::int64_t` values since the input validation operated by oneMKL is done *after* casting the user-provided value to `std::int64_t`.
- Setting an integer scaling factor representable as a double without any loss of accuracy but not representable so as a float is unrecommended for single-precision descriptors as that would be responsible for truncated, possibly-incorrect results.

Setting real-valued configuration parameters

The following member functions

```
template <precision prec, domain dom>
void descriptor<prec, dom>::set_value(config_param param, real_scalar_t value);
template <precision prec, domain dom>
template <typename T, std::enable_if_t<std::is_floating_point_v<T>, bool> = true>
void set_value(config_param param, T value) {
    set_value(param, static_cast<real_scalar_t>(value));
}
```

enable users to set configuration values for any `param` in the table below.

Real-valued configuration parameters along with their accepted and default values.

Accepted configuration parameter	Accepted values	Default value
<code>config_param::FORWARD_SCALAR</code>	All floating-point values	1.0
<code>config_param::BACKWARD_SCALAR</code>	All floating-point values	1.0

NOTE Setting a floating-point value that is not representable as a `float` (resp. `double`) without any loss of accuracy is unrecommended for single-precision (resp. double-precision) descriptors as that would be responsible for truncated, possibly-incorrect results.

Setting vector-valued configuration parameters

The following member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::set_value(config_param param, const std::vector<std::int64_t>&
value);
```

enables users to set configuration values for any `param` in the table below.

Vector-valued configuration parameters along with their accepted and default values (using notations from the [introduction](#)).

Accepted configuration parameter	Accepted values	Default value
<code>config_param::INPUT_STRIDES</code>	<code>std::vector<std::int64_t></code> object of size $(d + 1)$	<code>std::vector<std::int64_t>(d+1, 0)</code> (ignored by default due to deprecation)
<code>config_param::OUTPUT_STRIDES</code>	<code>std::vector<std::int64_t></code> object of size $(d + 1)$	<code>std::vector<std::int64_t>(d+1, 0)</code> (ignored by default due to deprecation)
<code>config_param::FWD_STRIDES</code>	<code>std::vector<std::int64_t></code> object of size $(d + 1)$	Documented in the page specific to configuring data layouts
<code>config_param::BWD_STRIDES</code>	<code>std::vector<std::int64_t></code> object of size $(d + 1)$	Documented in the page specific to configuring data layouts

The integer-valued configuration parameters listed [above](#) may also be configured using this member function, so long as an `std::vector<std::int64_t>` argument containing a unique (accepted) integer value is used.

Deprecations

The usage of `config_param::INPUT_STRIDES` and `config_param::OUTPUT_STRIDES` is deprecated. More information is available in the page specific to [configuring data layouts](#).

Setting other configuration parameters

The following member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::set_value(config_param param, config_value value);
```

enables users to set configuration values for any `param` in the table below.

Configuration parameters associated with configuration values in `config_value` along with their accepted and default values.

Accepted configuration parameter	Accepted values	Default value
<code>config_param::CONJUGATE_EV EN_STORAGE</code> (deprecated; support for non- default values will be dropped)	<code>config_value::COMPLEX_COMPL EX</code> or <code>config_value::COMPLEX_REAL</code> (deprecated)	<code>config_value::COMPLEX_COMP LEX</code>
<code>config_param::PACKED_FORMA T</code> (deprecated; support for non- default values will be dropped)	<code>config_value::CCE_FORMAT</code> , <code>config_value::PERM_FORMAT</code> (deprecated), <code>config_value::PACK_FORMAT</code> (deprecated) or <code>config_value::CCS_FORMAT</code> (deprecated)	<code>config_value::CCE_FORMAT</code>
<code>config_param::COMPLEX_STOR AGE</code>	<code>config_value::COMPLEX_COMPL EX</code> or <code>config_value::REAL_REAL</code>	<code>config_value::COMPLEX_COMP LEX</code>
<code>config_param::PLACEMENT</code>	<code>config_value::INPLACE</code> or <code>config_value::NOT_INPLACE</code>	<code>config_value::INPLACE</code>
<code>config_param::WORKSPACE</code> (or <code>config_param::WORKSPACE_PL ACEMENT</code> , equivalently)	<code>config_value::WORKSPACE_AUT OMATIC</code> (equivalent to <code>config_value::WORKSPACE_INT ERNAL</code>) or <code>config_value::WORKSPACE_EXT ERNAL</code>	<code>config_value::WORKSPACE_AU TOMATIC</code>
<code>config_param::DESTROY_INPU T</code>	<code>config_value::AVOID</code> or <code>config_value::ALLOW</code>	<code>config_value::AVOID</code>

Deprecated support

The deprecated member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::set_value(config_param param, DFTI_CONFIG_VALUE value);
```

offers the same support as the member function using a value of type `config_value` documented [above](#) except for parameters `config_param::WORKSPACE` (equivalent to `config_param::WORKSPACE_PLACEMENT`). Any call to the former is redirected internally to the latter after converting `value` to the corresponding `config_value` enumerator. Using the former function triggers a compile-time deprecation warning.

The deprecated member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::set_value(config_param param, std::int64_t* value);
```

offers the same support as the member function using a value of type `std::vector<std::int64_t>` documented [above](#) for parameters `config_param::INPUT_STRIDES`, `config_param::OUTPUT_STRIDES`, `config_param::FWD_STRIDES` and `config_param::BWD_STRIDES` (only). Any call to the former is redirected internally to the latter after constructing the `std::vector(value, value + d + 1)` object (d as defined in the [introduction](#)). Using the former function triggers a compile-time deprecation warning.

The deprecated variadic member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::set_value(config_param param, ...);
```

offers support for all configuration parameters. Regardless of the parameter being set, this function requires *exactly* two arguments and assumes a parameter-dependent type documented in the table below when reading the unique argument from the list of variadic arguments (*after* default argument promotions of variadic arguments, if applicable). Any call to this routine is redirected internally to the appropriate configuration-setting member function that is consistent with that assumed type of value (among those documented above, deprecated or not). Using this variadic member function triggers compile-time *and* runtime deprecation warnings to be emitted.

Assumed type of value considered by the variadic configuration-setting member function when reading its second argument, *i.e.*, the first in the list of variadic arguments (using notations from the [introduction](#)).

Accepted configuration parameters	Assumed type of value
<code>config_param::FORWARD_SCALE</code> and <code>config_param::BACKWARD_SCALE</code>	<code>double</code>
<code>config_param::CONJUGATE_EVEN_STORAGE</code> , <code>config_param::PACKED_FORMAT</code> , <code>config_param::COMPLEX_STORAGE</code> , <code>config_param::PLACEMENT</code> and <code>config_param::DESTROY_INPUT</code>	<code>DFTI_CONFIG_VALUE</code>
<code>config_param::WORKSPACE_PLACEMENT</code> and <code>config_param::WORKSPACE</code>	<code>config_value</code>
<code>config_param::NUMBER_OF_TRANSFORMS</code> , <code>config_param::FWD_DISTANCE</code> , <code>config_param::BWD_DISTANCE</code> and <code>config_param::THREAD_LIMIT</code>	<code>std::int64_t</code>
<code>config_param::INPUT_STRIDES</code> , <code>config_param::OUTPUT_STRIDES</code> , <code>config_param::FWD_STRIDES</code> and <code>config_param::BWD_STRIDES</code>	<code>std::int64_t*</code> (address of the first of $(d + 1)$ contiguous <code>std::int64_t</code> values)

Warning The behavior of the variadic configuration-setting member function is undefined if its usage is not strictly as documented.

Exceptions

The configuration-setting member functions may throw

- an `std::runtime_error` exception if an issue is found with the calling object;
- a `oneapi::mkl::invalid_argument` exception if
 - the parameter being set is not writable;
 - the parameter being set is rejected, e.g., if it is inconsistent with the type of configuration value being used;
 - the configuration value being set is rejected for the specific configuration parameter being set.
- a `oneapi::mkl::uninitialized` exception if the calling object has been moved-from and not reinitialized.

Configuration-querying member functions

This page describes the overloaded configuration-querying member functions `get_value` of the `descriptor` class template, which belongs to the `oneapi::mkl::dft` namespace and is declared in `oneapi/mkl/dft.hpp` (file to be included).

```
namespace oneapi::mkl::dft {
    template <precision prec, domain dom>
    class descriptor {
        using real_scalar_t = std::conditional_t<prec == precision::DOUBLE, double, float>;
    public:
        // for the type of forward domain:
        void get_value(config_param param, domain* value_ptr) const;
        // for the floating-point format:
        void get_value(config_param param, precision* value_ptr) const;
        // for integer-valued parameters:
        void get_value(config_param param, std::int64_t* value_ptr) const;
        // for real-valued parameters:
        void get_value(config_param param, real_scalar_t* value_ptr) const;
        // for vector-valued parameters:
        void get_value(config_param param, std::vector<std::int64_t>* value_ptr) const;
        // for other parameters:
        void get_value(config_param param, config_value* value_ptr) const;
        // deprecated usage, will be removed in a future release:
        [[deprecated("Use MKL_Get_Version_String(char*, int) instead.")]]
        void get_value(config_param param, char* value_ptr) const;
        [[deprecated("Use get_value(config_param, config_value*), "
                     "get_value(config_param, domain*) or "
                     "get_value(config_param, precision*) instead.")]]
        void get_value(config_param param, DFTI_CONFIG_VALUE* value_ptr) const;
        [[deprecated("Use get_value(config_param, std::int64_t*) instead.")]]
        void get_value(config_param param, size_t* value_ptr) const;
        [[deprecated("This get_value member function is deprecated.")]]
        void get_value(config_param param, ...) const;
    }
}
```

The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

The `get_value` functions enable oneMKL users to query the configuration value associated with a [configuration parameter](#) of a `descriptor` object. All `get_value` functions expect *exactly* two arguments: first the calling object's configuration parameter to be queried (as a `config_param` [enumerator](#)), second a pointer to an element (or elements) of the appropriate parameter-dependent type (documented below) wherein oneMKL is allowed to write the configuration value associated with that parameter. The calling `descriptor` object is left unchanged by any call to a configuration-querying member function.

Querying the type of forward domain

The following member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::get_value(config_param param, domain* value_ptr) const;
```

enables users to query the configuration value for the configuration parameter in the table below.

Behavior of the configuration-querying member function for the type of forward domain.

Accepted configuration parameter	Value written by oneMKL
config_param::FORWARD_DOMAIN	value_ptr[0] (assigned dom)

Querying the floating-point format

The following member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::get_value(config_param param, precision* value_ptr) const;
```

enables users to query the configuration value for the configuration parameter in the table below.

Behavior of the configuration-querying member function for the floating-point format.

Accepted configuration parameter	Value written by oneMKL
config_param::PRECISION	value_ptr[0] (assigned prec)

Querying integer-valued parameters

The following member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::get_value(config_param param, std::int64_t* value_ptr) const;
```

enables users to query the configuration value for the configuration parameters in the table below.

Behavior of the configuration-querying member function for integer-valued parameters (using notations from the [introduction](#)).

Accepted configuration parameters	Value written by oneMKL
config_param::DIMENSION, config_param::NUMBER_OF_TRANSFORMS, config_param::PLACEMENT, config_param::FWD_DISTANCE, config_param::BWD_DISTANCE, config_param::THREAD_LIMIT, config_param::WORKSPACE_ESTIMATE_BYTES, config_param::WORKSPACE_BYTES and config_param::WORKSPACE_EXTERNAL_BYTES	value_ptr[0]
config_param::FORWARD_SCALE and config_param::BACKWARD_SCALE (See the exceptions)	value_ptr[0]
config_param::LENGTHS if $d = 1$	value_ptr[0] (assigned n_1)

Deprecations

The parameters listed in the following table had been previously accepted for querying their values as if they were integer-valued. Such usage is now deprecated in favor of safer or more-appropriately-typed alternatives (enabling proper input validation by oneMKL in case of vector-valued parameters). Querying any of the configuration parameter listed below as if it was integer-valued triggers a runtime warning to be emitted.

Behavior of the configuration-querying member function for integer-valued parameters, when used as *deprecated* (using notations from the [introduction](#)).

Accepted configuration parameter(s)	Value(s) written by oneMKL
<code>config_param::FORWARD_DOMAIN,</code> <code>config_param::PRECISION,</code> <code>config_param::COMPLEX_STORAGE,</code> <code>config_param::CONJUGATE_EVEN_STORAGE,</code> <code>config_param::PACKED_FORMAT,</code> <code>config_param::COMMIT_STATUS</code> and <code>config_param::DESTROY_INPUT</code>	<code>value_ptr[0]</code>
<code>config_param::LENGTHS</code> if $d > 1$	<code>value_ptr[0], value_ptr[1], ..., value_ptr[d-1]</code> (assigned $n_1, n_2, \dots, n_d$, in that order)
<code>config_param::INPUT_STRIDES,</code> <code>config_param::OUTPUT_STRIDES,</code> <code>config_param::FWD_STRIDES</code> and <code>config_param::BWD_STRIDES</code>	<code>value_ptr[0], value_ptr[1], ..., value_ptr[d]</code> (assigned the queried $s_0, s_1, \dots, s_d$ documented in the page specific to configuring data layouts , in that order)

Querying real-valued parameters

The following member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::get_value(config_param param, real_scalar_t* value_ptr) const;
```

enables users to query the configuration value for the configuration parameters in the table below.

Behavior of the configuration-querying member function for real-valued parameters.

Accepted configuration parameters	Value written by oneMKL
<code>config_param::FORWARD_SCALE</code> and <code>config_param::BACKWARD_SCALE</code>	<code>value_ptr[0]</code>

Querying vector-valued parameters

The following member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::get_value(config_param param, std::vector<std::int64_t>* value_ptr)
const;
```

enables users to query the configuration value for the configuration parameters in the table below.

Behavior of the configuration-querying member function for vector-valued parameters (using notations from the [introduction](#)). In this table, `std::int64_t* data = value_ptr->data();` is used.

Accepted configuration parameter(s)	Value(s) written by oneMKL	Required vector size
config_param::LENGTHS	data[0], data[1], ..., data[d-1] (assigned $n_1, n_2, \dots, n_d$, in that order)	d
config_param::INPUT_STRIDE S, config_param::OUTPUT_STRIDES, config_param::FWD_STRIDES and config_param::BWD_STRIDES	data[0], data[1], ..., data[d] (assigned the queried $s_0, s_1, \dots, s_d$ documented in the page specific to configuring data layouts , in that order)	$d + 1$
All integer-valued configuration parameter	data[0]	1

Querying other parameters

The following member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::get_value(config_param param, config_value* value_ptr) const;
```

enables users to query the configuration value for the configuration parameters in the table below.

Behavior of the configuration-querying member function for parameters associated with config_value configuration values.

Accepted configuration parameters	Value written by oneMKL
config_param::CONJUGATE_EVEN_STORAGE, config_param::PACKED_FORMAT, config_param::COMPLEX_STORAGE, config_param::PLACEMENT, config_param::WORKSPACE, config_param::WORKSPACE_PLACEMENT, config_param::COMMIT_STATUS, config_param::DESTROY_INPUT	value_ptr[0]

Deprecated support

The deprecated member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::get_value(config_param param, size_t* value_ptr) const;
```

offers the same support as the member function using a value_ptr of type std::int64_t* documented [above](#). Using the former function triggers a compile-time deprecation warning.

The deprecated member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::get_value(config_param param, DFTI_CONFIG_VALUE* value_ptr) const;
```

enables users to query the configuration value for the configuration parameters in the table below, using the oneMKL C interface's `DFTI_CONFIG_VALUE` enumerators that are now deprecated via the oneMKL DPC++ interface. Using this function triggers a compile-time deprecation warning.

Behavior of the configuration-querying member function for parameters associatable with a configuration value of type `DFTI_CONFIG_VALUE` (deprecated).

Accepted configuration parameters	Value written by oneMKL
<code>config_param::CONJUGATE_EVEN_STORAGE,</code> <code>config_param::PACKED_FORMAT,</code> <code>config_param::FORWARD_DOMAIN,</code> <code>config_param::PRECISION,</code> <code>config_param::COMPLEX_STORAGE,</code> <code>config_param::PLACEMENT,</code> <code>config_param::COMMIT_STATUS</code> and <code>config_param::DESTROY_INPUT</code>	<code>value_ptr[0]</code>

The deprecated member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::get_value(config_param param, char* value_ptr) const;
```

serves no other purpose than querying textual information about the version of oneMKL being used. The very same information may be obtained via `MKL_Get_Version_String` (declared in `mkl_service.h`).

Behavior of the configuration-querying member function for the only parameter associatable with a C-style string (deprecated).

Accepted configuration parameter	Range of values possibly written by oneMKL
<code>config_param::VERSION</code>	from <code>value_ptr[0]</code> to <code>value_ptr[DFTI_VERSION_LENGTH-1]</code> (<code>DFTI_VERSION_LENGTH</code> defined in <code>mkl_dfti.h</code>)

The deprecated variadic member function

```
template <precision prec, domain dom>
void descriptor<prec, dom>::get_value(config_param param, ...) const;
```

offers support for all configuration parameters. Regardless of the parameter being queried, this function requires *exactly* two arguments and assumes a parameter-dependent pointer type documented in the table below when reading the unique argument from the list of variadic arguments (*after* default argument promotions of variadic arguments, if applicable). Any call to this routine is redirected internally to the appropriate configuration-querying member function that is consistent with that assumed pointer type of value (among those documented above, deprecated or not). Using this variadic member function triggers compile-time *and* runtime deprecation warnings to be emitted.

Assumed pointer type considered by the variadic configuration-querying member function when reading its second argument, i.e., the first in the list of variadic arguments.

Accepted configuration parameter(s)	Assumed pointer type
<code>config_param::DIMENSION,</code> <code>config_param::NUMBER_OF_TRANSFORMS,</code> <code>config_param::FWD_DISTANCE,</code> <code>config_param::BWD_DISTANCE,</code> <code>config_param::THREAD_LIMIT,</code> <code>config_param::LENGTHS,</code>	<code>std::int64_t*</code>

Accepted configuration parameter(s)	Assumed pointer type
<code>config_param::INPUT_STRIDES,</code> <code>config_param::OUTPUT_STRIDES,</code> <code>config_param::FWD_STRIDES</code> and <code>config_param::BWD_STRIDES</code>	
<code>config_param::WORKSPACE_ESTIMATE_BYTES,</code> <code>config_param::WORKSPACE_BYTES</code> and <code>config_param::WORKSPACE_EXTERNAL_BYTES</code>	<code>size_t*</code>
<code>config_param::FORWARD_SCALE</code> and <code>config_param::BACKWARD_SCALE</code>	<code>float*</code> (resp. <code>double*</code>) for single-precision (resp. double-precision) descriptors
<code>config_param::WORKSPACE</code> and <code>config_param::WORKSPACE_PLACEMENT</code>	<code>config_value*</code>
<code>config_param::FORWARD_DOMAIN,</code> <code>config_param::PRECISION,</code> <code>config_param::COMPLEX_STORAGE,</code> <code>config_param::CONJUGATE_EVEN_STORAGE,</code> <code>config_param::PACKED_FORMAT,</code> <code>config_param::PLACEMENT,</code> <code>config_param::COMMIT_STATUS,</code> <code>config_param::DESTROY_INPUT</code>	<code>DFTI_CONFIG_VALUE*</code>
<code>config_param::VERSION</code>	<code>char*</code>

Warning The behavior of the variadic configuration-querying member function is undefined if its usage is not strictly as documented.

Exceptions

The configuration-querying member functions may throw

- an `std::runtime_error` exception if an issue is found with the calling object;
- a `oneapi::mkl::uninitialized` exception if the calling object is uncommitted yet queried about a configuration parameter that requires the calling object to be committed (e.g., `config_param::WORKSPACE_BYTES` or `WORKSPACE_EXTERNAL_BYTES`) or if the descriptor object has been moved-from and not reinitialized;
- a `oneapi::mkl::unimplemented` exception if the queried parameter corresponds to a feature that is not implemented for the calling object (e.g., `config_param::WORKSPACE_BYTES` or `WORKSPACE_EXTERNAL_BYTES` for a CPU-committed descriptor);
- a `oneapi::mkl::invalid_argument` exception if
 - the second argument is `nullptr`;
 - the parameter being queried is rejected, e.g., if it is inconsistent with the pointer type used as second argument;
 - the size of the vector pointed by `value_ptr` is not as required, for [vector-valued-parameters](#);
 - the configuration value to be returned cannot be safely or accurately converted into the desired type, e.g., if querying a scaling factor that happens to be 0.5 using the configuration-querying member function specific to integer-valued configuration parameters.

Configuration- and queue-committing member function

This page describes the configuration- and queue-committing member function `commit` of the `descriptor` class template, which belongs to the `oneapi::mkl::dft` namespace and is declared in `oneapi/mkl/dft.hpp` (file to be included).

```
namespace oneapi::mkl::dft {
    template <precision prec, domain dom>
    class descriptor {
    public:
        void commit(sycl::queue &user_queue);
    }
}
```

The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

Invoking this function notifies that the calling object's configuration is completed and triggers it to complete the required initialization steps (e.g., pre-computing data, exploring various factorizations, assessing suitability of various algorithms) relevant to enqueueing the DFT computations that it defines to the user-provided `user_queue`. That `sycl::queue` object defines the targeted computation device (among other runtime-related characteristics).

Upon successful completion, the calling object is "committed" (its configuration value associated with `config_param::COMMIT_STATUS` is then `config_value::COMMITTED`). `descriptor` objects *must* be committed to be used in any [compute function](#).

As [previously noted](#), changing any configuration setting of a committed object effectively leaves it "uncommitted" (its configuration value associated with `config_param::COMMIT_STATUS` is then `config_value::UNCOMMITTED`). As a consequence, it is best to avoid any call to any `set_value` member function after invoking the `commit` member function.

Input parameter

Input parameter for the `commit` member function.

Name	Type	Description
<code>user_queue</code>	<code>sycl::queue</code>	Queue to which DFT computations are to be enqueued by the calling object, when used in compute functions thereafter

Exceptions

The configuration- and queue-committing member function may throw

- a `oneapi::mkl::unimplemented` exception, e.g., if the calling object's configuration is not supported (yet);
- a `oneapi::mkl::uninitialized` exception, e.g., if the calling object has been moved-from and not reinitialized;
- an `std::runtime_error` exception, e.g., if the calling object's configuration is found inconsistent.

Providing an externally-allocated workspace

This page describes the member function templates `set_workspace` of the `descriptor` class template, which belongs to the `oneapi::mkl::dft` namespace and is declared in `oneapi/mkl/dft.hpp` (file to be included).

```
namespace oneapi::mkl::dft {
    template <precision prec, domain dom>
    class descriptor {
        using real_scalar_t = std::conditional_t<prec == precision::DOUBLE, double, float>;
    public:
        template<typename data_type>
        void set_workspace(sycl::buffer<data_type, 1> &workspace);
        template<typename data_type>
        void set_workspace(data_type* workspace);
    }
}
```

The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

Definitions and description

During the computation of the DFT that it defines, any `descriptor` object may require additional memory, e.g., to store intermediate results or pre-computed data: that required additional memory is referred to as the object's "workspace".

Internally-allocated workspaces

A `descriptor` object uses an *internally-allocated* workspace if `config_value::WORKSPACE_AUTOMATIC` (or its equivalent `config_value::WORKSPACE_INTERNAL`) is the configuration value associated with its configuration parameter `config_param::WORKSPACE_PLACEMENT` (or its *alias* `config_param::WORKSPACE`). Such an object allocates and owns its workspace. The workspace is allocated (and initialized, if relevant) when the object is [committed](#). This is the default behavior in oneMKL.

Externally-allocated workspaces

A `descriptor` objects uses an *externally-allocated* workspace if `config_value::WORKSPACE_EXTERNAL` is the configuration value associated with its configuration parameter `config_param::WORKSPACE_PLACEMENT` (or its *alias* `config_param::WORKSPACE`). Such an object does *not* allocate nor own its workspace and oneMKL users *must* provide it with a device-accessible memory allocation (*not* managed by oneMKL) *after* the object was [committed](#).

The above `set_workspace` member functions enable users to provide the *committed* calling object with an externally-allocated workspace, either in the form of a SYCL buffer or a device-accessible USM allocation (either function template's only argument). The form of the workspace being provided (SYCL buffer or USM allocation) *must* be identical to the form of input and output data containers used afterwards, in [compute functions](#).

The minimal size (in bytes) of the allocation to be provided as a workspace is the configuration value associated with the *committed* object's configuration parameter `config_param::WORKSPACE_BYTES` (or its *alias* `config_param::WORKSPACE_EXTERNAL_BYTES`). That minimal size is unknown unless the object is committed, but a conservative estimate may be obtained from uncommitted objects by querying their configuration value associated with `config_param::WORKSPACE_ESTIMATE_BYTES`. In either case, the object's member function for [querying integer-valued parameters](#) must be used to obtain that (read-only) configuration value.

If the calling object's configuration was set to use an internally-allocated workspace when committed, a subsequent call to any `set_workspace` member function overrides that pre-existing status: it signals the object to free its own internally-allocated workspace, use the user-provided one instead, and change its

configuration value associated with `config_param::WORKSPACE` (or its equivalent `config_param::WORKSPACE_PLACEMENT`) to `config_value::WORKSPACE_EXTERNAL`. The object remains committed upon completion of that operation.

An externally-allocated workspace must not be used for any other purposes between the time it is provided to a `descriptor` object and the time any `compute function` using that object *completes* (especially in case of USM allocations since workspace-related dependencies internal to oneMKL may be ignored otherwise).

NOTE Although it may be queried from uncommitted objects, the configuration value associated with parameter `config_param::WORKSPACE_ESTIMATE_BYTES` does take into account the calling object's configuration at the time its `querying member function` is called. Modifying the calling object's configuration after querying that value may invalidate the obtained workspace byte size estimate.

Partial support

Externally-allocated workspaces are supported on GPU devices only. As a consequence, the conservative estimate of the workspace byte size documented above (*i.e.*, the configuration value associated with `config_param::WORKSPACE_ESTIMATE_BYTES`) implicitly assumes that the calling object is meant to be committed to a queue targeting a GPU device later on (even if it is already committed to a CPU device).

Template parameter

Either member function template is parameterized by a type template parameter `data_type`. The only specializations available in oneMKL for `data_type` are `float` and `double` regardless of the calling object's type.

Input parameter

Input parameter for the `set_workspace` member function templates.

Name	Supported types	Description
<code>workspace</code>	i. <code>sycl::buffer<data_type, 1></code> ii. <code>data_type*</code>	User-allocated workspace in the form of a (i) <code>sycl::buffer</code> or (ii) device-accessible USM allocation.

Exceptions

The `set_workspace` member functions may throw

- a `oneapi::mkl::invalid_argument` exception if the provided workspace is a `sycl::buffer` object that is a sub-buffer;
- an `std::runtime_error` if
 - the calling object is not committed;
 - the workspace is provided as a `sycl::buffer` object that is smaller than required;
 - a `nullptr` is used as argument yet the calling object's required workspace size is nonzero;
 - the workspace is provided as a USM allocation that is not accessible by the device;
 - an error occurred while the calling object operated its workspace-setting operations.
- a `oneapi::mkl::uninitialized` exception if the calling object has been moved-from and not reinitialized.

NOTE In case of a USM allocation being provided, the size of the provided workspace is not verified by oneMKL (*assumed large enough*).

Configuring the placement of the DFT results

This page describes how to configure `descriptor` objects to operate “in-place” (output data overwriting the input data) or not (output data stored separately from the input data) when used in [compute functions](#). When non-native, all the relevant types and enumerations mentioned below belong to the `oneapi::mkl::dft` namespace and are declared in `oneapi/mkl/dft.hpp` (file to be included). The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

Description

The configuration value associated with the configuration parameter `config_param::PLACEMENT` of a `descriptor` object determines whether it should operate in-place or not. The only possible values are the self-explanatory `config_value::INPLACE` and `config_value::NOT_INPLACE`. By default, `descriptor` objects are set to operate in-place.

NOTE In-place operations require the data layout to satisfy specific requirements documented in the page dedicated to [configuring data layouts](#).

Allowing destruction of input data

This page describes how to allow (or forbid) `descriptor` objects configured for out-of-place operations to overwrite the input data when used in [compute functions](#). When non-native, all the relevant types and enumerations mentioned below belong to the `oneapi::mkl::dft` namespace and are declared in `oneapi/mkl/dft.hpp` (file to be included). The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

Description

The configuration value associated with the configuration parameter `config_param::DESTROY_INPUT` of a `descriptor` object determines whether it is allowed to overwrite the input data provided to a [compute functions](#) for out-of-place operations. The only possible values are the self-explanatory `config_value::ALLOW` and `config_value::AVOID`.

By default, `descriptor` objects are configured to avoid overwriting the input data when configured for out-of-place operations. Allowing an object to overwrite the input data may, in some cases,

- significantly reduce the size of its required workspace;
- improve the delivered performance.

In particular, allowing input data to be overwritten may be particularly relevant for multidimensional, out-of-place, real, backward DFTs.

NOTE By definition, the configuration value associated with the configuration parameter `config_param::DESTROY_INPUT` is relevant for `descriptor` objects configured for out-of-place operations, only. It is ignored by `descriptor` objects configured for in-place operations.

Limiting the number of usable threads

This page describes how to limit the number of CPU threads that a `descriptor` object may use in [compute functions](#). When non-native, all the relevant types and enumerations mentioned below belong to the `oneapi::mkl::dft` namespace and are declared in `oneapi/mkl/dft.hpp` (file to be included). The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

Description

The number of threads that a CPU-committed `descriptor` object may use within a [compute function](#) can be limited by the configuration value associated with configuration parameter `config_param::THREAD_LIMIT`.

A positive value $P > 0$ associated with that parameter effectively sets P as a limit on the maximum number of CPU threads that the object can use. A zero value associated with that parameter effectively bypasses any such restriction on the number of usable threads (default behavior).

By definition, this configuration parameter is irrelevant if oneMKL is used in sequential mode, and the configuration value associated with configuration parameter `config_param::THREAD_LIMIT` is permanently set to 1 in such cases; any attempt to modify it using the relevant [configuration-setting member function](#) is ignored in that case.

This configuration parameter is also irrelevant for (and ignored by) `descriptor` objects committed to a GPU device.

NOTE When [queried](#) about the value associated with `config_param::THREAD_LIMIT` after it was [committed](#), a `descriptor` object may report a lower value than previously set, *e.g.*, if the value previously set exceeds another limit defined otherwise.

Configuring data layouts

This page describes how to configure a `descriptor` object for a specific data layout. When non-native, all the relevant types and enumerations mentioned below belong to the `oneapi::mkl::dft` namespace and are declared in `oneapi/mkl/dft.hpp` (file to be included). The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

The DPC++ interface provides the configuration parameter `config_param::FWD_STRIDES` (resp. `config_param::BWD_STRIDES`) to define the data layout locating entries (or parts thereof) of relevant data sequences in the forward (resp. backward) domain. In case of batched transforms, *i.e.*, if the configuration value for `config_param::NUMBER_OF_TRANSFORMS` is set to an integer M larger than 1, the value set for configuration parameter `config_param::FWD_DISTANCE` (resp. `config_param::BWD_DISTANCE`) completes the description of the data layout by specifying the distances between successive data sequences in the forward (resp. backward) domain.

This topic leverages the general notations from the [introduction](#), and uses the superscript `fwd` (resp. `bwd`) for data sequences in forward (resp. backward) domain. A placeholder label $\mathbf{v}$ is also used to capture a possible distinction between the real (if $\mathbf{v}$ is $\mathbf{r}$) and imaginary (if $\mathbf{v}$ is $\mathbf{i}$) parts of a complex data entry; naturally, that placeholder label $\mathbf{v}$ is relevant only for data layouts that distinguish real and imaginary parts of complex data entries.

A [non-redundant entry](#) $(\cdot)_{k_1, k_2, \dots, k_d}^m$ (or its real or imaginary part, if relevant) is stored at index $J^{\text{xwd}}(k_1, \dots, k_d, m; \mathbf{v})$ of the appropriate data container (`sycl::buffer` object or device-accessible USM allocation) provided to a [compute function](#), the base data type of which is (possibly implicitly re-interpreted as) documented in the [table](#) below. That index value is defined as

$$J^{\text{xwd}}(k_1, \dots, k_d, m; v) = s_0^{\text{xwd}} + k_1 s_1^{\text{xwd}} + k_2 s_2^{\text{xwd}} + \dots + k_{d-1} s_{d-1}^{\text{xwd}} + p_d^{\text{xwd}}(k_d; v) s_d^{\text{xwd}} + m l^{\text{xwd}},$$

wherein

- $s_j^{\text{xwd}}, \forall j \in \{0, \dots, d\}$ represents the [offset and generalized strides](#) defining the locations of relevant values within each d -dimensional data sequence in the forward (resp. backward) domain if $\mathbf{x} = \mathbf{f}$ (resp. if $\mathbf{x} = \mathbf{b}$), counted in number of elements of the relevant [implicitly-assumed elementary data type](#);
- l^{xwd} represents the [distance](#) between successive d -dimensional data sequences in the forward (resp. backward) domain if $\mathbf{x} = \mathbf{f}$ (resp. if $\mathbf{x} = \mathbf{b}$), counted in number of elements of the relevant [implicitly-assumed elementary data type](#);
- the relation $p_d^{\text{xwd}}(k_d; v)$ simplifies into the identity $p_d^{\text{xwd}}(k_d; v) = k_d$ in all recommended use cases or if $\mathbf{x} = \mathbf{f}$, i.e., v is either irrelevant or unused in such cases. However, for some one-dimensional real descriptors using [deprecated configurations](#), the real and imaginary parts of entry $(\cdot)_{k_1}^m$ in backward domain, are to be considered separately from one another and the corresponding indices are denoted by $J^{\text{bwd}}(k_1, m; r) = s_0^{\text{bwd}} + p_1^{\text{bwd}}(k_1; r) s_1^{\text{bwd}} + m l^{\text{bwd}}$ and $J^{\text{bwd}}(k_1, m; i) = s_0^{\text{bwd}} + p_1^{\text{bwd}}(k_1; i) s_1^{\text{bwd}} + m l^{\text{bwd}}$, respectively.

In this page, it is assumed that [only non-redundant](#) data sequence entries are of interest, i.e., that $0 \leq m < M, 0 \leq k_j < n_j, \forall j \in \{1, 2, \dots, d-1\}$ and that $0 \leq k_d \leq \lfloor \frac{n_d}{2} \rfloor$ (resp. $0 \leq k_d < n_d$) for entries that do (resp. do not) belong the backward domain of a real DFT.

Note that all elements accessed as a value stored at index $J^{\text{xwd}}(k_1, \dots, k_d, m; v)$ of a given user-provided data container *must* belong to the same block allocation.

Implicitly-assumed elementary data type

When reading or writing an element at index $J^{\text{xwd}}(k_1, \dots, k_d, m; v)$ of any user-provided data container used at compute time, a `descriptor` object may first re-interpret the base data type of that data container into an implicitly-assumed elementary data type. That implicitly-assumed data type depends on the object type; that is, on the specialization values used for the template parameters when instantiating the `descriptor` class, and on other configuration value(s). The table below lists the implicitly-assumed data type in either domain (last 2 columns) based on the object type and its configuration value(s).

Elementary data type implicitly assumed in either domain (last 2 columns) based on the descriptor's type and its relevant configuration values (first column). This table uses `typedef float fp_type;` (resp. `typedef double fp_type;`) for single-precision (resp. double-precision) descriptors.

Type of descriptor and relevant configuration values	Implicitly-assumed elementary data type in forward domain	Implicitly-assumed elementary data type in backward domain
Complex descriptor with <code>config_value::COMPLEX_COMPLEX</code> set for <code>config_param::COMPLEX_STORAGE</code> (default behavior)	<code>std::complex<fp_type></code>	<code>std::complex<fp_type></code>

Type of descriptor and relevant configuration values	Implicitly-assumed elementary data type in forward domain	Implicitly-assumed elementary data type in backward domain
Complex descriptor with <code>config_value::REAL_REAL</code> set for <code>config_param::COMPLEX_STORAGE</code> (not implemented via the DPC++ interface of oneMKL)	<code>fp_type</code>	<code>fp_type</code>
Real descriptor with <code>config_value::COMPLEX_COMPLEX</code> set for <code>config_param::CONJUGATE_EVEN_STORAGE</code> (default behavior)	<code>fp_type</code>	<code>std::complex<fp_type></code>
Real descriptor with <code>config_value::COMPLEX_REAL</code> set for <code>config_param::CONJUGATE_EVEN_STORAGE</code> (supported only for 1D DFTs on CPU, <i>deprecated</i>)	<code>fp_type</code>	<code>fp_type</code>

Descriptors that implicitly assume an elementary data type of `float` or `double` (resp. `std::complex<float>` or `std::complex<double>`) in a domain are referred to as “descriptors expecting real (resp. complex) data” in that domain.

Configuring strides in forward and backward domains

The values $s_0^{xwd}, s_1^{xwd}, \dots, s_d^{xwd}$ defining $J^{xwd}(k_1, \dots, k_d, m; v)$ are to be communicated as elements (in that order) of a `std::vector<std::int64_t>` object of size $(d + 1)$, passed as the configuration value for `config_param::FWD_STRIDES` if $x = f$ (resp. `config_param::BWD_STRIDES` if $x = b$) using the relevant [configuration-setting member function](#). The element s_0^{xwd} represents an absolute offset (or “displacement”) in the data sets while the subsequent elements s_j^{xwd} ($j > 0$) are generalized strides to be considered along dimensions $j \in \{1, \dots, d\}$.

When created, the descriptors are default-configured for unbatched, in-place transforms using a unit stride along the last dimension, no offset and the default configuration settings documented in the [above table](#). For real descriptors, minimal padding is used in forward domain, aligning with the [data layout requirements](#) for in-place transforms.

In other words, the default stride values are $s_0^{fwd} = s_0^{bwd} = 0$, $s_d^{fwd} = s_d^{bwd} = 1$ and, for d -dimensional transforms with $d > 1$,

- $s_{d-1}^{fwd} = s_{d-1}^{bwd} = n_d$ for complex descriptors;
- $s_{d-1}^{bwd} = \lfloor \frac{n_d}{2} \rfloor + 1$, and $s_{d-1}^{fwd} = 2s_{d-1}^{bwd}$ for real descriptors;
- if $d > 2$, $s_k^{xwd} = n_{k+1}s_{k+1}^{xwd}$ for $k \in \{1, \dots, d-2\}$ (for $x = f$ and $x = b$).

The usage of these default strides for unbatched, in-place transforms is illustrated in the [usage examples](#).

Configuring batched transforms

The value l^{xwd} completing the definition of $J^{\text{xwd}}(k_1, \dots, k_d, m; v)$ is to be set as an `std::int64_t` configuration value for `config_param::FWD_DISTANCE` if $x = f$ (resp. `config_param::BWD_DISTANCE` if $x = b$) using the relevant [configuration-setting member function](#). This value is irrelevant for unbatched transforms, *i.e.*, for descriptors set to handle a number of transforms M equal to 1 (default behavior).

In case of batched transforms, the desired number M of DFTs *must* be set explicitly as an `std::int64_t` configuration value `config_param::NUMBER_OF_TRANSFORMS` using the relevant [configuration-setting member function](#). In that case, `config_param::FWD_DISTANCE` and `config_param::BWD_DISTANCE` *must* also be set explicitly since their default configuration values of 0 would break the [data layout requirements](#) for any $M > 1$.

The configuration of batched transforms is illustrated in the [usage examples](#).

Deprecated layouts in backward domain of one-dimensional real transforms

All complex descriptors and all real descriptors expecting complex data in backward domain use the straightforward identity relation $p_d^{\text{bwd}}(k_d; v) = k_d$, *i.e.*, v is irrelevant in that case. **Every default behavior and recommended usage** falls into this category; the reader is referred to the [usage examples](#) for more details and illustrations about the resulting layouts and default (or otherwise recommended) strides and distances.

For real descriptors expecting real data in backward domain (**deprecated** usage, supported for 1D real DFTs on CPU only), the relation $p_d^{\text{bwd}}(k_d; v)$ takes a more intricate form. In backward domain, such descriptors expect real data in the sense that the real and imaginary parts of the data sequence entries are not necessarily stored contiguously in memory (or not even stored *at all*). The specific form of $p_d^{\text{bwd}}(k_d; v)$ depends on the value set for `config_param::PACKED_FORMAT`. For real descriptors expecting real data in backward domain, three different values (documented below) are possible for that configuration parameter: `config_value::CCS_FORMAT`, `config_value::PACK_FORMAT` and `config_value::PERM_FORMAT`. Given the limited support for 1D transforms on CPUs, $d = 1$ is used in the rest of this section to simplify the presentation. Illustrations are also given for unbatched cases; that is, $M = 1$, so the then-superfluous batch index $m = 0$ is omitted in this section's illustrative tables, too.

`config_value::CCS_FORMAT` value set for `config_param::PACKED_FORMAT`

If the configuration value `config_value::CCS_FORMAT` is used, then

- $p_1^{\text{bwd}}(k_1; r) = 2k_1$;
- $p_1^{\text{bwd}}(k_1; i) = 2k_1 + 1$.

Given that all non-redundant entries in backward domain are captured by $0 \leq k_1 \leq \lfloor \frac{n_1}{2} \rfloor$, the range of relevant values for $p_1^{\text{bwd}}(k_1; v)$ is $0 \leq p_1^{\text{bwd}}(k_1; v) < n_1 + 2$ (resp. $0 \leq p_1^{\text{bwd}}(k_1; v) < n_1 + 1$) if n_1 is even (resp. odd), in this case.

This format is illustrated in the table below for $M = 1$, $s_0^{\text{bwd}} = 0$ and $s_1^{\text{bwd}} = 1$.

Illustration of the data layout considered in backward domain by a one-dimensional real descriptor configured for an unbatched ($M = 1$) DFT of length n_1 using $s_0^{\text{bwd}} = 0$, $s_1^{\text{bwd}} = 1$ and `config_value::CCS_FORMAT` for `config_param::PACKED_FORMAT`. Note that 0-valued imaginary parts (see the [introductory notes](#)) are stored *explicitly* in this case.

$J^{\text{bwd}}(k_1; \mathbf{v})$	0	1	2	3	$\cdots 2\lfloor n_1/2 \rfloor$	$2\lfloor n_1/2 \rfloor + 1$
$(k_1; \mathbf{v})$	(0; r)	(0; i)	(1; r)	(1; i)	$\cdots (\lfloor n_1/2 \rfloor; r)$	$(\lfloor n_1/2 \rfloor; i)$
Stored value	$\Re((\cdot)_0)$	0	$\Re((\cdot)_1)$	$\Im((\cdot)_1)$	$\cdots \Re((\cdot)_{\lfloor n_1/2 \rfloor})$	$\Im((\cdot)_{\lfloor n_1/2 \rfloor})$

`config_value::PACK_FORMAT` value set for `config_param::PACKED_FORMAT`

If the configuration value `config_value::PACK_FORMAT` is used, then

- $p_1^{\text{bwd}}(0; r) = 0$;
- $p_1^{\text{bwd}}(0; i)$ **does not exist** (0-valued imaginary parts are not stored explicitly);
- $p_1^{\text{bwd}}(k_1; r) = 2k_1 - 1$ for any $0 < k_1 \leq \lfloor n_1/2 \rfloor$;
- $p_1^{\text{bwd}}(k_1; i) = 2k_1$ for any $0 < k_1 < \lfloor n_1/2 \rfloor$, $p_1^{\text{bwd}}(k_1; i) = 2k_1$ holds for $k_1 = \lfloor n_1/2 \rfloor$ if n_1 is odd; $p_1^{\text{bwd}}(\lfloor n_1/2 \rfloor; i)$ **does not exist** if n_1 is even (0-valued imaginary parts are not stored explicitly).

Given that all non-redundant entries in backward domain are captured by $0 \leq k_1 \leq \lfloor \frac{n_1}{2} \rfloor$, the range of relevant values for $p_1^{\text{bwd}}(k_1; \mathbf{v})$ is $0 \leq p_1^{\text{bwd}}(k_1; \mathbf{v}) < n_1$ in this case (regardless of whether n_1 is even or odd).

This format is illustrated in the tables below for $M = 1$, $s_0^{\text{bwd}} = 0$ and $s_1^{\text{bwd}} = 1$.

Illustration of the data layout considered in backward domain by a one-dimensional real descriptor configured for an unbatched ($M = 1$) DFT of even length n_1 using $s_0^{\text{bwd}} = 0$, $s_1^{\text{bwd}} = 1$ and `config_value::PACK_FORMAT` for `config_param::PACKED_FORMAT`. Note that 0-valued imaginary parts (see the [introductory notes](#)) are *not* stored in this case.

$J^{\text{bwd}}(k_1; \mathbf{v})$	0	1	2	3	$\cdots n_1 - 2$	$n_1 - 1$
$(k_1; \mathbf{v})$	(0; r)	(1; r)	(1; i)	(2; r)	$\cdots (n_1/2 - 1; i)$	$(n_1/2; r)$
Stored value	$\Re((\cdot)_0)$	$\Re((\cdot)_1)$	$\Im((\cdot)_1)$	$\Re((\cdot)_2)$	$\cdots \Im((\cdot)_{n_1/2-1})$	$\Re((\cdot)_{n_1/2})$

Illustration of the data layout considered in backward domain by a one-dimensional real descriptor configured for an unbatched ($M = 1$) DFT of odd length n_1 using $s_0^{\text{bwd}} = 0$, $s_1^{\text{bwd}} = 1$ and `config_value::PACK_FORMAT` for `config_param::PACKED_FORMAT`. Note that 0-valued imaginary parts (see the [introductory notes](#)) are *not* stored in this case.

$J^{\text{bwd}}(k_1; \mathbf{v})$	0	1	2	3	$\cdots n_1 - 2$	$n_1 - 1$
$(k_1; \mathbf{v})$	(0; r)	(1; r)	(1; i)	(2; r)	$\cdots (\lfloor n_1/2 \rfloor; r)$	$(\lfloor n_1/2 \rfloor; i)$
Stored value	$\Re((\cdot)_0)$	$\Re((\cdot)_1)$	$\Im((\cdot)_1)$	$\Re((\cdot)_2)$	$\cdots \Re((\cdot)_{\lfloor n_1/2 \rfloor})$	$\Im((\cdot)_{\lfloor n_1/2 \rfloor})$

config_value::PERM_FORMAT value set for config_param::PACKED_FORMAT

If the configuration value config_value::PERM_FORMAT is used, the $p_1^{\text{bwd}}(k_1; v)$ relation differs according to whether n_1 is even or odd.

If n_1 is even, then

- $p_1^{\text{bwd}}(0; r) = 0$ and $p_1^{\text{bwd}}(0; i)$ **does not exist** (0-valued imaginary parts are not stored explicitly);
- $p_1^{\text{bwd}}(n_1/2; r) = 1$ and $p_1^{\text{bwd}}(n_1/2; i)$ **does not exist** (0-valued imaginary parts are not stored explicitly);
- $p_1^{\text{bwd}}(k_1; r) = 2k_1$ for any $0 < k_1 < n_1/2$;
- $p_1^{\text{bwd}}(k_1; i) = 2k_1 + 1$ for any $0 < k_1 < n_1/2$.

If n_1 is odd, then (this format is equivalent to config_value::PACK_FORMAT if n_1 is odd)

- $p_1^{\text{bwd}}(0; r) = 0$ and $p_1^{\text{bwd}}(0; i)$ **does not exist** (0-valued imaginary parts are not stored explicitly);
- $p_1^{\text{bwd}}(k_1; r) = 2k_1 - 1$ for any $0 < k_1 \leq \lfloor n_1/2 \rfloor$;
- $p_1^{\text{bwd}}(k_1; i) = 2k_1$ for any $0 < k_1 \leq \lfloor n_1/2 \rfloor$.

Given that all non-redundant entries in backward domain are captured by $0 \leq k_1 \leq \lfloor \frac{n_1}{2} \rfloor$, the range of relevant values for $p_1^{\text{bwd}}(k_1; v)$ is $0 \leq p_1^{\text{bwd}}(k_1; v) < n_1$ in this case (regardless of whether n_1 is even or odd).

This format is illustrated in the tables below for $M = 1$, $s_0^{\text{bwd}} = 0$ and $s_1^{\text{bwd}} = 1$.

Illustration of the data layout considered in backward domain by a one-dimensional real descriptor configured for an unbatched ($M = 1$) DFT of even length n_1 using $s_0^{\text{bwd}} = 0$, $s_1^{\text{bwd}} = 1$ and config_value::PERM_FORMAT for config_param::PACKED_FORMAT. Note that 0-valued imaginary parts (see the [introductory notes](#)) are *not* stored in this case.

$J^{\text{bwd}}(k_1; v)$	0	1	2	3	...	$n_1 - 2$	$n_1 - 1$
$(k_1; v)$	(0; r)	$(n_1/2; r)$	(1; r)	(1; i)	...	$(n_1/2 - 1; r)$	$(n_1/2 - 1; i)$
Stored value	$\Re((\cdot)_0)$	$\Re((\cdot)_{n_1/2})$	$\Re((\cdot)_1)$	$\Im((\cdot)_1)$	...	$\Re((\cdot)_{n_1/2-1})$	$\Im((\cdot)_{n_1/2-1})$

Illustration of the data layout considered in backward domain by a one-dimensional real descriptor configured for an unbatched ($M = 1$) DFT of odd length n_1 using $s_0^{\text{bwd}} = 0$, $s_1^{\text{bwd}} = 1$ and config_value::PERM_FORMAT for config_param::PACKED_FORMAT. Note that 0-valued imaginary parts (see the [introductory notes](#)) are *not* stored in this case. Note the equivalence with the data layouts to be considered if config_value::PACK_FORMAT is set for config_param::PACKED_FORMAT in this particular case.

$J^{\text{bwd}}(k_1; v)$	0	1	2	3	...	$n_1 - 2$	$n_1 - 1$
$(k_1; v)$	(0; r)	(1; r)	(1; i)	(2; r)	...	$(\lfloor n_1/2 \rfloor; r)$	$(\lfloor n_1/2 \rfloor; i)$

$J^{\text{bwd}}(k_1; \mathbf{v})$	0	1	2	3	$\dots$	$n_1 - 2$	$n_1 - 1$
Stored value	$\Re((\cdot)_0)$	$\Re((\cdot)_1)$	$\Im((\cdot)_1)$	$\Re((\cdot)_2)$	$\dots$	$\Re((\cdot)_{\lfloor n_1/2 \rfloor})$	$\Im((\cdot)_{\lfloor n_1/2 \rfloor})$

NOTE Real descriptors expecting complex data in backward domain do not support any other value than `config_value::CCE_FORMAT` for `config_param::PACKED_FORMAT`. Therefore, `config_value::CCE_FORMAT` is used as the default value for `config_param::PACKED_FORMAT` (consistently with `config_value::COMPLEX_COMPLEX` being the default value set for `config_param::CONJUGATE_EVEN_STORAGE`).

The value set for `config_param::PACKED_FORMAT` must be set explicitly (to either `config_value::CCS_FORMAT`, `config_value::PACK_FORMAT` or `config_value::PERM_FORMAT`) for real descriptors expecting *real* data in backward domain as it further specifies the descriptor's behavior in that case (see explanations [above](#)). Real descriptors expecting real data in backward domain are supported for 1D real DFTs on CPU only. Their support is deprecated.

Data layout requirements

In general, the distances and strides must be set so that

- values of $J^{\text{xwd}}(k_1, \dots, k_d, m; \mathbf{v})$ are non-negative for all $(d+1)$ -tuples $(k_1, k_2, \dots, p_d^{\text{xwd}}(k_d; \mathbf{v}), m)$ within relevant ranges;
- every value of $J^{\text{xwd}}(k_1, \dots, k_d, m; \mathbf{v})$ corresponds to a *unique* value relevant to the data sequences under consideration. In other words, there must not be one value of $J^{\text{xwd}}(k_1, \dots, k_d, m; \mathbf{v})$ corresponding to two different $(d+1)$ -tuples $(k_1, k_2, \dots, p_d^{\text{xwd}}(k_d; \mathbf{v}), m)$ that would both be within relevant ranges.

Additionally, for in-place transforms (configuration value `config_value::INPLACE` set for `config_param::PLACEMENT`), the following “*consistency requirements*” apply:

- descriptors expecting the same data type in either domain (*e.g.*, complex descriptors) must use the same offset, stride(s), and distance values in forward and backward domains;
- for real descriptors expecting complex data in backward domain (default behavior for real descriptors), the memory address(es) of leading entry(ies) along the last dimension must be identical in forward and backward domains. Specifically, that requirement translates into the conditions $s_j^{\text{fwd}} = 2s_j^{\text{bwd}}, \forall j \in \{0, \dots, d-1\}$ as well as, if $M > 1$, $l^{\text{fwd}} = 2l^{\text{bwd}}$. Note that this requirement leads to some data padding to be used in forward domain if unit strides are used along dimension d in forward and backward domains (recommended usage, as set by default).

NOTE

- Support for negative strides with a sufficiently large (positive) offset index guaranteeing non-negativeness of all $J^{\text{xwd}}(k_1, \dots, k_d, m; \mathbf{v})$ is not enabled yet (unimplemented);
- One-dimensional real descriptors expecting real data in backward domain and using configuration value `config_value::CCS_FORMAT` for `config_param::PACKED_FORMAT` also require $l^{\text{bwd}} \geq n_1 + 2$.

Configuring strides for input and output data [deprecated]

Instead of specifying strides by domain, one may choose to specify the strides for input and output data sequences. Let $s_j^{\text{x}}, j \in \{0, 1, \dots, d\}$ be the stride values for input (resp. output) data sequences if $\mathbf{X} = \mathbf{i}$ (resp. $\mathbf{X} = \mathbf{o}$). Such $s_0^{\text{x}}, s_1^{\text{x}}, \dots, s_d^{\text{x}}$ values may be communicated as elements (in that order) of a `std::vector<std::int64_t>` object of size $(d + 1)$, passed as the configuration value for `config_param::INPUT_STRIDES` if $\mathbf{X} = \mathbf{i}$ (resp. `config_param::OUTPUT_STRIDES` if $\mathbf{X} = \mathbf{o}$) using the relevant [configuration-setting member function](#).

The values of $s_j^{\mathbf{i}}$ and $s_j^{\mathbf{o}}$ are to be used and considered by oneMKL if and only if $s_j^{\text{fwd}} = s_j^{\text{bwd}} = 0, \forall j \in \{0, 1, \dots, d\}$. This will happen automatically if `config_param::INPUT_STRIDES` and `config_param::OUTPUT_STRIDES` are set and `config_param::FWD_STRIDES` and `config_param::BWD_STRIDES` are not (see the note below). In such a case, `descriptor` objects must consider the data layouts corresponding to the two compute directions separately. As detailed above, relevant data sequence entries are accessed as elements of data containers (`sycl::buffer` objects or device-accessible USM allocations) provided to the [compute function](#), the base data type of which is (possibly implicitly re-interpreted) as documented in the [above table](#). If using input and output strides, the index to be used when accessing a data sequence entry $\binom{\cdot}{k_1, k_2, \dots, k_d}^m$ – or part thereof – in forward domain is

$$J^{\text{fwd}}(k_1, \dots, k_d, m; \mathbf{v}) = s_0^{\text{x}} + k_1 s_1^{\text{x}} + k_2 s_2^{\text{x}} + \dots + \underbrace{p_d^{\text{fwd}}(k_d, \mathbf{v})}_{=k_d} s_d^{\text{x}} + m l^{\text{fwd}}$$

where $\mathbf{X} = \mathbf{i}$ (resp. $\mathbf{X} = \mathbf{o}$) for forward (resp. backward) DFTs. Similarly, the index to be used when accessing a data sequence entry – or part thereof – in backward domain is

$$J^{\text{bwd}}(k_1, \dots, k_d, m; \mathbf{v}) = s_0^{\text{x}} + k_1 s_1^{\text{x}} + k_2 s_2^{\text{x}} + \dots + p_d^{\text{bwd}}(k_d, \mathbf{v}) s_d^{\text{x}} + m l^{\text{bwd}}$$

where $\mathbf{X} = \mathbf{o}$ (resp. $\mathbf{X} = \mathbf{i}$) for forward (resp. backward) DFTs.

As a consequence, configuring `descriptor` objects using these deprecated configuration parameters makes their configuration direction-dependent when different stride values are used in forward and backward domains. Since the intended compute direction is unknown to the `descriptor` object when [committing](#) it, every direction that results in a [legitimate data layout](#) in forward and backward domains must be supported by successfully committed `descriptor` objects.

NOTE For descriptor objects with strides configured via these deprecated configuration parameters, the [data layout requirements](#) may be satisfied for only one of the two compute directions, *i.e.*, for only one of the forward or backward DFT. The behavior of oneMKL is undefined if using that object for the compute direction that does not align with the [data layout requirements](#).

Setting either of `config_param::INPUT_STRIDES` or `config_param::OUTPUT_STRIDES` triggers any (default or previously-set) values for `config_param::FWD_STRIDES` and `config_param::BWD_STRIDES` to reset to 0-valued vectors, and vice versa. This implicit behavior prevents mix-and-matching either of `config_param::INPUT_STRIDES` or `config_param::OUTPUT_STRIDES` with either of `config_param::FWD_STRIDES` or `config_param::BWD_STRIDES`, which is **not** supported by oneMKL. If such a configuration is attempted, an exception is thrown at commit time due to invalid configuration, as the stride values that were implicitly reset invalidate the [data layout requirements](#) for any non-trivial DFT.

If specifying the data layout strides using these deprecated configuration parameters and if the strides differ in forward and backward domain, the descriptor *must* be re-configured and re-committed for computing the DFT in the reverse direction as shown below.

```
// ...
desc.set_value(config_param::INPUT_STRIDES, fwd_domain_strides);
desc.set_value(config_param::OUTPUT_STRIDES, bwd_domain_strides);
desc.commit(queue);
compute_forward(desc, ...);
// ...
desc.set_value(config_param::INPUT_STRIDES, bwd_domain_strides);
desc.set_value(config_param::OUTPUT_STRIDES, fwd_domain_strides);
desc.commit(queue);
compute_backward(desc, ...);
```

The `config_param::INPUT_STRIDES` and `config_param::OUTPUT_STRIDES` parameters have been deprecated since oneMKL2024.1. A compile-time deprecation warning advising users to update their usage to `config_param::FWD_STRIDES` and `config_param::BWD_STRIDES` is emitted for any application using these configuration parameters.

Supported layouts on GPU devices

On GPU devices, oneMKL requires

- the rank d of the transform to be no greater than 3;
- the offset values s_0^{fwd} and s_0^{bwd} to be 0;
- either $l^{\text{xwd}} = n_1 s_1^{\text{xwd}}$ or $s_1^{\text{xwd}} = M l^{\text{xwd}}$ for batched, two-dimensional real transforms (for $x = f$ and $x = b$);
- either $s_1^{\text{xwd}} = n_2 s_2^{\text{xwd}}$ (along with $l^{\text{xwd}} = n_1 s_1^{\text{xwd}}$ if $M > 1$) or $s_2^{\text{xwd}} = n_1 s_1^{\text{xwd}}$ (along with $s_1^{\text{xwd}} = M l^{\text{xwd}}$ if $M > 1$) for three-dimensional real transforms (for $x = f$ and $x = b$);
- real descriptors to use `config_value::COMPLEX_COMPLEX` for `config_param::CONJUGATE_EVEN_STORAGE` and `config_value::CCE_FORMAT` for `config_param::PACKED_FORMAT`.

NOTE If parts of the block allocation used for output results are found irrelevant to the computed DFT, GPU-committed descriptor may overwrite them (*e.g.*, padding elements between successive data sequences).

Compute functions

This page describes the function templates `oneapi::mkl::dft::compute_forward` and `oneapi::mkl::dft::compute_backward`, which all pertain to the calculation of a DFT defined by a *committed* `oneapi::mkl::dft::descriptor` object `desc`. These function templates belong to the `oneapi::mkl::dft` namespace and are declared in `oneapi/mkl/dft.hpp` (file to be included). The usage of prepended namespace specifiers `oneapi::mkl::dft` is omitted below for conciseness.

```
namespace oneapi::mkl::dft {
    /*----- for in-place transforms -----*/
    // using SYCL buffers
    template<typename descriptor_type, typename data_type>
    void compute_forward(descriptor_type &desc,
                        sycl::buffer<data_type, 1> &inout);
    template<typename descriptor_type, typename data_type>
    void compute_backward(descriptor_type &desc,
                        sycl::buffer<data_type, 1> &inout);
    // using device-accessible USM allocations
    template <typename descriptor_type, typename data_type>
    sycl::event compute_forward(descriptor_type &desc,
                              data_type *inout,
                              const std::vector<sycl::event> &dependencies = {});
    template <typename descriptor_type, typename data_type>
    sycl::event compute_backward(descriptor_type &desc,
                              data_type *inout,
                              const std::vector<sycl::event> &dependencies = {});
    /*----- for out-of-place transforms -----*/
    // using SYCL buffers
    template<typename descriptor_type, typename input_type, typename output_type>
    void compute_forward(descriptor_type &desc,
                        sycl::buffer<input_type, 1> &in,
                        sycl::buffer<output_type, 1> &out);
    template<typename descriptor_type, typename input_type, typename output_type>
    void compute_backward(descriptor_type &desc,
                        sycl::buffer<input_type, 1> &in,
                        sycl::buffer<output_type, 1> &out);
    // using device-accessible USM allocations
    template<typename descriptor_type, typename input_type, typename output_type>
    sycl::event compute_forward(descriptor_type &desc,
                              input_type *in,
                              output_type *out,
                              const std::vector<sycl::event> &dependencies = {});
    template<typename descriptor_type, typename input_type, typename output_type>
    sycl::event compute_backward(descriptor_type &desc,
                              input_type *in,
                              output_type *out,
                              const std::vector<sycl::event> &dependencies = {});
}
```

Description

The `compute_forward` (resp. `compute_backward`) function templates accept a *committed* `descriptor` object `desc` followed by one or more data-containing arguments, and enqueues the computation of the forward (resp. backward) DFT that `desc` defines, i.e., the transform defined by $\delta = -1$ (resp. $\delta = +1$). The computations are enqueued to the queue to which `desc` was *committed*.

Successive computations using the same `descriptor` object must be serialized to guarantee correct results. This can be done by waiting on the queue passed at `commit`, or by using an in-order queue. Alternatively, repeated use of the same buffer for any argument will also serialize computations (when using `sycl::buffer` objects), or computations can be serialized using the returned event and input dependencies (when using device-accessible USM allocations). These alternate methods will be more performant in most cases. When possible, configuring a `descriptor` object for [batched transforms](#) delivers better performance than such successive computations in most cases.

The number of data-containing arguments to be provided depends exclusively on whether the desired DFT calculations are to be done in-place (output data overwriting the input data) or out-of-place (output data stored separately from the input data). One data-containing argument is required for in-place operations whereas two such arguments are needed for out-of-place calculations. Note that in-place operations require the data layout to satisfy requirements documented in the page dedicated to [configuring data layouts](#).

Warning

- When using a function designed for in-place (resp. out-of-place) operations, `desc` (function's first argument) *must* have a configuration value of `config_value::INPLACE` (resp. `config_value::NOT_INPLACE`) associated with `config_param::PLACEMENT`. The function's behavior is undefined otherwise.
- When using a function designed for out-of-place operations, `in` and `out` must not share common elements.

Template parameters

For in-place operations

The `compute_forward` and `compute_backward` function templates for in-place operations are parameterized by two type template parameters, in the following order:

- `descriptor_type` representing the type of `desc`, an instance of a specialization of the [descriptor class template](#);
- `data_type` representing the elementary data type of the data-containing argument being used.

The available specializations are listed in the table below.

Available specializations of the compute function templates for in-place operations.

function template	descriptor_type	data_type
<code>compute_forwardcompute_backward</code>	<code>descriptor<precision::SINGLE, domain::REAL></code>	<code>float</code>
<code>compute_forwardcompute_backward</code>	<code>descriptor<precision::SINGLE, domain::REAL></code>	<code>std::complex<float></code>
<code>compute_forwardcompute_backward</code>	<code>descriptor<precision::SINGLE, domain::COMPLEX></code>	<code>float</code>
<code>compute_forwardcompute_backward</code>	<code>descriptor<precision::SINGLE, domain::COMPLEX></code>	<code>std::complex<float></code>

function template	descriptor_type	data_type
<code>compute_forwardcompute_backward</code>	<code>descriptor<precision::DOUBL E, domain::REAL></code>	<code>double</code>
<code>compute_forwardcompute_backward</code>	<code>descriptor<precision::DOUBL E, domain::REAL></code>	<code>std::complex<double></code>
<code>compute_forwardcompute_backward</code>	<code>descriptor<precision::DOUBL E, domain::COMPLEX></code>	<code>double</code>
<code>compute_forwardcompute_backward</code>	<code>descriptor<precision::DOUBL E, domain::COMPLEX></code>	<code>std::complex<double></code>

For out-of-place operations

The `compute_forward` and `compute_backward` function templates for out-of-place operations are parameterized by three type template parameters, in the following order:

- `descriptor_type` representing the type of `desc`, an instance of a specialization of the [descriptor class template](#);
- `input_type` representing the elementary data type of the first (input) data-containing argument being used;
- `output_type` representing the elementary data type of the second (output) data-containing argument being used.

The available specializations are listed in the table below.

Available specializations of the compute function templates for out-of-place operations.

function template	descriptor_type	input_type	output_type
<code>compute_forwardcompute_backward</code>	<code>descriptor<precisio n::SINGLE, domain::REAL></code>	<code>float</code>	<code>float</code>
<code>compute_forwardcompute_backward</code>	<code>descriptor<precisio n::SINGLE, domain::REAL></code>	<code>std::complex<float></code>	<code>std::complex<float></code>
<code>compute_forwardcompute_backward</code>	<code>descriptor<precisio n::SINGLE, domain::COMPLEX></code>	<code>float</code>	<code>float</code>
<code>compute_forwardcompute_backward</code>	<code>descriptor<precisio n::SINGLE, domain::COMPLEX></code>	<code>std::complex<float></code>	<code>std::complex<float></code>
<code>compute_forward</code>	<code>descriptor<precisio n::SINGLE, domain::REAL></code>	<code>float</code>	<code>std::complex<float></code>
<code>compute_backward</code>	<code>descriptor<precisio n::SINGLE, domain::REAL></code>	<code>std::complex<float></code>	<code>float</code>

function template	descriptor_type	input_type	output_type
compute_forwardcompute_backward	descriptor<precision::DOUBLE, domain::REAL>	double	double
compute_forwardcompute_backward	descriptor<precision::DOUBLE, domain::REAL>	std::complex<double>	std::complex<double>
compute_forwardcompute_backward	descriptor<precision::DOUBLE, domain::COMPLEX>	double	double
compute_forwardcompute_backward	descriptor<precision::DOUBLE, domain::COMPLEX>	std::complex<double>	std::complex<double>
compute_forward	descriptor<precision::DOUBLE, domain::REAL>	double	std::complex<double>
compute_backward	descriptor<precision::DOUBLE, domain::REAL>	std::complex<double>	double

Input parameters and returned value

The `compute_forward` and `compute_backward` template functions' input parameters are documented below. If device-accessible USM allocations are used, these functions return a `sycl::event` object that may be used to track the completion of the computations that the function call enqueues.

For in-place operations

Input parameters for the in-place compute functions.

Name	Supported types	Description
desc	descriptor_type	<i>Committed</i> instance of a specialization of the descriptor class template , defining the DFT to compute
inout	i. <code>sycl::buffer<data_type, 1></code> ii. <code>data_type*</code>	Data-containing argument for the DFT (input data on input, output results on output)
dependencies	<code>std::vector<sycl::event></code>	Vector of dependencies to be honored by the enqueued DFT computation (only if using device-accessible USM allocations arguments)

For out-of-place operations

Input parameters for the out-of-place compute functions.

Name	Supported types	Description
desc	descriptor_type	<i>Committed</i> instance of a specialization of the descriptor class template , defining the DFT to compute
in	i. <code>sycl::buffer<input_type, 1></code> ii. <code>input_type*</code>	Data-containing argument for the input data of the DFT
out	i. <code>sycl::buffer<output_type, 1></code> ii. <code>output_type*</code>	Data-containing argument for the output data of the DFT
dependencies	<code>std::vector<sycl::event></code>	Vector of dependencies to be honored by the enqueued DFT computation (only if using device-accessible USM allocations arguments)

Exceptions

The `compute_forward` and `compute_backward` template functions may throw an `std::runtime_error` if

- an issue is found with `desc`;
- `desc` is uncommitted;
- `desc` is configured to use an [externally-allocated workspace](#) yet it was not provided with one;
- `desc` is configured to use an [externally-allocated workspace](#) and the form of data-containing arguments used is different from the form of workspace that was provided;
- a failure is detected when computing the DFT.

If using SYCL buffers, some code paths do not support sub-buffers as data-containing arguments and may throw an `std::invalid_argument` exception if sub-buffers are used.

The `compute_forward` and `compute_backward` template functions may throw a `oneapi::mkl::uninitialized` exception if `desc` has been moved-from and not reinitialized.

Configuring and computing a DFT in DPC++

Some usage examples are provided below for computing (possibly batched) DFTs using device-accessible USM allocations, with [default](#) or otherwise recommended strides. More working examples can also be found in the oneMKL installation directory under `share/doc/mkl/examples/sycl/dft`.

To use the DPC++ interface, include `oneapi/mkl/dft.hpp`. The DPC++ interface supports CPU and Intel GPU devices with some limitations, documented in previous pages. Refer to the [Intel® oneAPI Math Kernel Library Release Notes](#) for other (release-specific) known limitations.

The DFT functions assume the Cartesian representation of complex data; that is, complex numbers are defined by their real and imaginary parts. oneMKL provides efficient [Vector Mathematical Functions](#) for conversion to and from polar representation.

Usage examples

The following example illustrates the computation of (possibly many) complex and real discrete Fourier transforms using device-accessible USM allocations without dependencies on other events, via the DPC++ interface of oneMKL.

```
#include <oneapi/mkl/dft.hpp>
#include <complex>
enum class dft_compute_direction {
    FWD_DFT,
    BWD_DFT
};
namespace dft_ns = oneapi::mkl::dft;

template<typename T>
bool is_usable_compute_type = std::is_same_v<T, float> |
    std::is_same_v<T, std::complex<float>> |
    std::is_same_v<T, double> |
    std::is_same_v<T, std::complex<double>>;

template<dft_ns::precision prec, dft_ns::domain dom>
void ready_descriptor(dft_ns::descriptor<prec, dom>& desc,
    sycl::queue& q,
    dft_ns::config_value placement,
    std::int64_t batch_size) {
    // this example assumes that desc was freshly created and that its default
    // configuration values have not been changed prior to invoking this routine

    // Fetch rank, lengths and type of forward domain
    std::int64_t rank;
    desc.get_value(dft_ns::config_param::DIMENSION, &rank);
    std::vector<std::int64_t> lengths(rank);
    desc.get_value(dft_ns::config_param::LENGTHS, &lengths);
    if (placement == dft_ns::config_value::NOT_INPLACE) {
        // non-default behavior
        desc.set_value(dft_ns::config_param::PLACEMENT, placement);
        if constexpr (dom == dft_ns::domain::REAL) {
            // for real descriptors interpreting backward domain's data type as
            // complex (default behavior), unpadded forward-domain data layouts
            // may be used if operating out-of-place:
            std::vector<std::int64_t> fwd_strides(rank + 1);
            std::vector<std::int64_t> bwd_strides(rank + 1);
            fwd_strides[rank] = bwd_strides[rank] = 1;
            if (rank > 1) {
                fwd_strides[rank - 1] = lengths[rank - 1];
                bwd_strides[rank - 1] = lengths[rank - 1] / 2 + 1;
                for (std::int64_t dim = 2; dim < rank; dim++) {
                    fwd_strides[rank - dim] = fwd_strides[rank - dim + 1] * lengths[rank - dim];
                    bwd_strides[rank - dim] = bwd_strides[rank - dim + 1] * lengths[rank - dim];
                }
            }
            fwd_strides[0] = bwd_strides[0] = 0; // offsets in data layouts
            desc.set_value(dft_ns::config_param::FWD_STRIDES, fwd_strides);
            desc.set_value(dft_ns::config_param::BWD_STRIDES, bwd_strides);
        }
    }
    if (batch_size > 1) {
        // configuration of batched DFTs
        desc.set_value(dft_ns::config_param::NUMBER_OF_TRANSFORMS, batch_size);
    }
}
```

```

    std::int64_t fwd_distance, bwd_distance;
    fwd_distance = bwd_distance = 1;
    for (std::int64_t dim = 0; dim < rank - 1; dim++) {
        fwd_distance *= lengths[dim];
        bwd_distance *= lengths[dim];
    }
    if constexpr (dom == dft_ns::domain::REAL) {
        if (placement == dft_ns::config_value::NOT_INPLACE)
            fwd_distance *= lengths[rank - 1];
        else
            fwd_distance *= 2 * (lengths[rank - 1] / 2 + 1);
        bwd_distance *= (lengths[rank - 1] / 2 + 1);
    }
    else {
        fwd_distance *= lengths[rank - 1];
        bwd_distance *= lengths[rank - 1];
    }
    desc.set_value(dft_ns::config_param::FWD_DISTANCE, fwd_distance);
    desc.set_value(dft_ns::config_param::BWD_DISTANCE, bwd_distance);
}
desc.commit(q);
}

template <typename T, std::enable_if_t<is_usable_compute_type<T>, bool> = true>
sycl::event compute_inplace_cmplx_dft(sycl::queue& q,
                                     const std::vector<std::int64_t>& lengths,
                                     const std::int64_t& batch_size,
                                     const dft_compute_direction& direction,
                                     T* device_accessible_usm_data) {
    constexpr auto prec =
        std::is_same_v<T, float> | std::is_same_v<T, std::complex<float>> ?
        dft_ns::precision::SINGLE : dft_ns::precision::DOUBLE;
    auto desc = dft_ns::descriptor<prec, dft_ns::domain::COMPLEX>(lengths);
    ready_descriptor(desc, q, dft_ns::config_value::INPLACE, batch_size);
    // Initialize data such that input data entry of multi-index
    // - {k1,k2,k3; m} (for 3D DFTs, i.e., lengths.size() == 3)
    // is Z[ m*lengths[0]*lengths[1]*lengths[2]
    // + k1*lengths[1]*lengths[2] + k2*lengths[2] + k3 ]
    // - {k1,k2; m} (for 2D DFTs, i.e., lengths.size() == 2)
    // is Z[ m*lengths[0]*lengths[1] + k1*lengths[1] + k2 ]
    // - {k1; m} (for 1D DFTs, i.e., lengths.size() == 1)
    // is Z[ m*lengths[0] + k1 ]
    // wherein
    // using real_t =
    // std::conditional_t<prec == dft_ns::precision::SINGLE, float, double>;
    // std::complex<real_t>* Z =
    // static_cast<std::complex<real_t>>(device_accessible_usm_data);
    // (0 <= kj < lengths[j-1])
    if (direction == dft_compute_direction::FWD_DFT)
        return dft_ns::compute_forward(desc, device_accessible_usm_data);
    else
        return dft_ns::compute_backward(desc, device_accessible_usm_data);
    // Upon completion of any of the above, the output data entry of multi-index
    // - {k1,k2,k3; m} (for 3D DFTs, i.e., lengths.size() == 3)
    // is Z[ m*lengths[0]*lengths[1]*lengths[2]
    // + k1*lengths[1]*lengths[2] + k2*lengths[2] + k3 ]
    // - {k1,k2; m} (for 2D DFTs, i.e., lengths.size() == 2)
    // is Z[ m*lengths[0]*lengths[1] + k1*lengths[1] + k2 ]

```

```

// - {k1; m}          (for 1D DFTs, i.e., lengths.size() == 1)
//   is Z[ m*lengths[0] + k1 ]
// (0 <= kj < lengths[j-1])
}

template <typename T, std::enable_if_t<is_usable_compute_type<T>, bool> = true>
sycl::event compute_outofplace_cmplx_dft(sycl::queue& q,
                                         const std::vector<std::int64_t>& lengths,
                                         const std::int64_t& batch_size,
                                         const dft_compute_direction& direction,
                                         T* device_accessible_usm_data_in,
                                         T* device_accessible_usm_data_out) {

    constexpr bool is_single_precision =
        std::is_same_v<T, float> | std::is_same_v<T, std::complex<float>>;
    constexpr auto prec = (is_single_precision) ?
        dft_ns::precision::SINGLE : dft_ns::precision::DOUBLE;
    auto desc = dft_ns::descriptor<prec, dft_ns::domain::COMPLEX>(lengths);
    ready_descriptor(desc, q, dft_ns::config_value::NOT_INPLACE, batch_size);
    // Initialize data such that input data entry of multi-index
    // - {k1,k2,k3; m}    (for 3D DFTs, i.e., lengths.size() == 3)
    //   is Z[ m*lengths[0]*lengths[1]*lengths[2]
    //         + k1*lengths[1]*lengths[2] + k2*lengths[2] + k3 ]
    // - {k1,k2; m}       (for 2D DFTs, i.e., lengths.size() == 2)
    //   is Z[ m*lengths[0]*lengths[1] + k1*lengths[1] + k2 ]
    // - {k1; m}          (for 1D DFTs, i.e., lengths.size() == 1)
    //   is Z[ m*lengths[0] + k1 ]
    // wherein
    // using real_t =
    //   std::conditional_t<prec == dft_ns::precision::SINGLE, float, double>;
    // std::complex<real_t>* Z =
    //   static_cast<std::complex<real_t>>(device_accessible_usm_data_in);
    // (0 <= kj < lengths[j-1])
    if (direction == dft_compute_direction::FWD_DFT)
        return dft_ns::compute_forward(desc, device_accessible_usm_data_in,
                                         device_accessible_usm_data_out);
    else
        return dft_ns::compute_backward(desc, device_accessible_usm_data_in,
                                         device_accessible_usm_data_out);
    // Upon completion of any of the above, the output data entry of multi-index
    // - {k1,k2,k3; m}    (for 3D DFTs, i.e., lengths.size() == 3)
    //   = Y[ m*lengths[0]*lengths[1]*lengths[2]
    //         + k1*lengths[1]*lengths[2] + k2*lengths[2] + k3 ]
    // - {k1,k2; m}       (for 2D DFTs, i.e., lengths.size() == 2)
    //   = Y[ m*lengths[0]*lengths[1] + k1*lengths[1] + k2 ]
    // - {k1; m}          (for 1D DFTs, i.e., lengths.size() == 1)
    //   = Y[ m*lengths[0] + k1 ]
    // wherein
    // using real_t =
    //   std::conditional_t<prec == dft_ns::precision::SINGLE, float, double>;
    // std::complex<real_t>* Y =
    //   static_cast<std::complex<real_t>>(device_accessible_usm_data_out);
    // (0 <= kj < lengths[j-1])
}

template<typename T, std::enable_if_t<is_usable_compute_type<T>, bool> = true>
sycl::event compute_inplace_real_dft(sycl::queue& q,
                                     const std::vector<std::int64_t>& lengths,
                                     const std::int64_t& batch_size,

```

```

        const dft_compute_direction& direction,
        T* device_accessible_usm_data) {
constexpr bool is_single_precision =
    std::is_same_v<T, float> | std::is_same_v<T, std::complex<float>>;
constexpr auto prec = (is_single_precision) ?
    dft_ns::precision::SINGLE : dft_ns::precision::DOUBLE;
auto desc = dft_ns::descriptor<prec, dft_ns::domain::REAL>(lengths);
ready_descriptor(desc, q, dft_ns::config_value::INPLACE, batch_size);
// using real t =
// std::conditional_t<prec == dft_ns::precision::SINGLE, float, double>;
// Let
// real_t* fwd_Z = static_cast<real_t*>(device_accessible_usm_data);
// and
// std::complex<real_t>* bwd_Z =
// static_cast<std::complex<real_t>>(device_accessible_usm_data);
if (direction == dft_compute_direction::FWD_DFT) {
    // Initialize data such that input data entry of multi-index
    // - {k1,k2,k3; m} (for 3D DFTs, i.e., lengths.size() == 3)
    // is fwd_Z[ m*lengths[0]*lengths[1]*2*(lengths[2]/2 + 1)
    // + k1*lengths[1]*2*(lengths[2]/2 + 1)
    // + k2*2*(lengths[2]/2 + 1) + k3 ]
    // - {k1,k2; m} (for 2D DFTs, i.e., lengths.size() == 2)
    // is fwd_Z[ m*lengths[0]*2*(lengths[1]/2 + 1)
    // + k1*2*(lengths[1]/2 + 1) + k2 ]
    // - {k1; m} (for 1D DFTs, i.e., lengths.size() == 1)
    // is fwd_Z[ m*2*(lengths[0]/2 + 1) + k1 ]
    // (0 <= kj < lengths[j-1])
    return dft_ns::compute_forward(desc, device_accessible_usm_data);
    // Upon completion of the above, the output data entry of multi-index
    // - {k1,k2,k3; m} (for 3D DFTs, i.e., lengths.size() == 3)
    // is bwd_Z[ m*lengths[0]*lengths[1]*(lengths[2]/2 + 1)
    // + k1*lengths[1]*(lengths[2]/2 + 1)
    // + k2*(lengths[2]/2 + 1) + k3 ]
    // - {k1,k2; m} (for 2D DFTs, i.e., lengths.size() == 2)
    // is bwd_Z[ m*lengths[0]*(lengths[1]/2 + 1)
    // + k1*(lengths[1]/2 + 1) + k2 ]
    // - {k1; m} (for 1D DFTs, i.e., lengths.size() == 1)
    // is bwd_Z[ m*(lengths[0]/2 + 1) + k1 ]
    // (0 <= kj < lengths[j-1] for j < lengths.size() - 1 and
    // 0 <= kj < lengths[j-1]/2 + 1 for j == lengths.size() - 1)
}
else {
    // Initialize data such that input data entry of multi-index
    // - {k1,k2,k3; m} (for 3D DFTs, i.e., lengths.size() == 3)
    // is bwd_Z[ m*lengths[0]*lengths[1]*(lengths[2]/2 + 1)
    // + k1*lengths[1]*(lengths[2]/2 + 1)
    // + k2*(lengths[2]/2 + 1) + k3 ]
    // - {k1,k2; m} (for 2D DFTs, i.e., lengths.size() == 2)
    // is bwd_Z[ m*lengths[0]*(lengths[1]/2 + 1)
    // + k1*(lengths[1]/2 + 1) + k2 ]
    // - {k1; m} (for 1D DFTs, i.e., lengths.size() == 1)
    // is bwd_Z[ m*(lengths[0]/2 + 1) + k1 ]
    // (0 <= kj < lengths[j-1] for j < lengths.size() - 1 and
    // 0 <= kj < lengths[j-1]/2 + 1 for j == lengths.size() - 1)
    return dft_ns::compute_backward(desc, device_accessible_usm_data);
    // Upon completion of the above, the output data entry of multi-index
    // - {k1,k2,k3; m} (for 3D DFTs, i.e., lengths.size() == 3)
    // is fwd_Z[ m*lengths[0]*lengths[1]*2*(lengths[2]/2 + 1)

```

```

//          + k1*lengths[1]*2*(lengths[2]/2 + 1)
//          + k2*2*(lengths[2]/2 + 1) + k3 ]
// - {k1,k2; m}      (for 2D DFTs, i.e., lengths.size() == 2)
//   is fwd_Z[ m*lengths[0]*2*(lengths[1]/2 + 1)
//             + k1*2*(lengths[1]/2 + 1) + k2 ]
// - {k1; m}         (for 1D DFTs, i.e., lengths.size() == 1)
//   is fwd_Z[ m*2*(lengths[0]/2 + 1) + k1 ]
// (0 <= kj < lengths[j-1])
}
}

template<typename T, std::enable_if_t<is_usable_compute_type<T>, bool> = true>
sycl::event compute_outofplace_real_dft(sycl::queue& q,
                                        const std::vector<std::int64_t>& lengths,
                                        const std::int64_t& batch_size,
                                        const dft_compute_direction& direction,
                                        T* device_accessible_usm_data_in,
                                        T* device_accessible_usm_data_out) {
    constexpr bool is_single_precision =
        std::is_same_v<T, float> | std::is_same_v<T, std::complex<float>>;
    constexpr auto prec = (is_single_precision) ?
        dft_ns::precision::SINGLE : dft_ns::precision::DOUBLE;
    auto desc = dft_ns::descriptor<prec, dft_ns::domain::REAL>(lengths);
    ready_descriptor(desc, q, dft_ns::config_value::NOT_INPLACE, batch_size);
    if (direction == dft_compute_direction::FWD_DFT) {
        // Initialize data such that input data entry of multi-index
        // - {k1,k2,k3; m}      (for 3D DFTs, i.e., lengths.size() == 3)
        //   is fwd_Z[ m*lengths[0]*lengths[1]*lengths[2]
        //             + k1*lengths[1]*lengths[2] + k2*lengths[2] + k3 ]
        // - {k1,k2; m}        (for 2D DFTs, i.e., lengths.size() == 2)
        //   is fwd_Z[ m*lengths[0]*lengths[1] + k1*lengths[1] + k2 ]
        // - {k1; m}           (for 1D DFTs, i.e., lengths.size() == 1)
        //   is fwd_Z[ m*lengths[0] + k1 ]
        // wherein
        // using real_t =
        //   std::conditional_t<prec == dft_ns::precision::SINGLE, float, double>;
        // real_t* fwd_Z = static_cast<real_t*>(device_accessible_usm_data_in);
        // (0 <= kj < lengths[j-1])
        return dft_ns::compute_forward(desc, device_accessible_usm_data_in,
                                       device_accessible_usm_data_out);

        // Upon completion of the above, the output data entry of multi-index
        // - {k1,k2,k3; m}      (for 3D DFTs, i.e., lengths.size() == 3)
        //   is bwd_Z[ m*lengths[0]*lengths[1]*(lengths[2]/2 + 1)
        //             + k1*lengths[1]*(lengths[2]/2 + 1)
        //             + k2*(lengths[2]/2 + 1) + k3 ]
        // - {k1,k2; m}        (for 2D DFTs, i.e., lengths.size() == 2)
        //   is bwd_Z[ m*lengths[0]*(lengths[1]/2 + 1)
        //             + k1*(lengths[1]/2 + 1) + k2 ]
        // - {k1; m}           (for 1D DFTs, i.e., lengths.size() == 1)
        //   is bwd_Z[ m*(lengths[0]/2 + 1) + k1 ]
        // (0 <= kj < lengths[j-1] for j < lengths.size() - 1 and
        //  0 <= kj < lengths[j-1]/2 + 1 for j == lengths.size() - 1)
        // wherein
        // using real_t =
        //   std::conditional_t<prec == dft_ns::precision::SINGLE, float, double>;
        // std::complex<real_t>* bwd_Z =
        //   static_cast<std::complex<real_t>>>(device_accessible_usm_data_out);
    }
}

```



```

else {
    // Initialize data such that input data entry of multi-index
    // - {k1,k2,k3; m} (for 3D DFTs, i.e., lengths.size() == 3)
    // is bwd_Z[ m*lengths[0]*lengths[1]*(lengths[2]/2 + 1)
    //           + k1*lengths[1]*(lengths[2]/2 + 1)
    //           + k2*(lengths[2]/2 + 1) + k3 ]
    // - {k1,k2; m} (for 2D DFTs, i.e., lengths.size() == 2)
    // is bwd_Z[ m*lengths[0]*(lengths[1]/2 + 1)
    //           + k1*(lengths[1]/2 + 1) + k2 ]
    // - {k1; m} (for 1D DFTs, i.e., lengths.size() == 1)
    // is bwd_Z[ m*(lengths[0]/2 + 1) + k1 ]
    // (0 <= kj < lengths[j-1] for j < lengths.size() - 1 and
    //  0 <= kj < lengths[j-1]/2 + 1 for j == lengths.size() - 1)
    // wherein
    // using real_t =
    //   std::conditional_t<prec == dft_ns::precision::SINGLE, float, double>;
    // std::complex<real_t>* bwd_Z =
    //   static_cast<std::complex<real_t>*>(device_accessible_usm_data_in);
    return dft_ns::compute_backward(desc, device_accessible_usm_data_in,
                                    device_accessible_usm_data_out);

    // Upon completion of the above, the output data entry of multi-index
    // - {k1,k2,k3; m} (for 3D DFTs, i.e., lengths.size() == 3)
    // is fwd_Z[ m*lengths[0]*lengths[1]*lengths[2]
    //           + k1*lengths[1]*lengths[2] + k2*lengths[2] + k3 ]
    // - {k1,k2; m} (for 2D DFTs, i.e., lengths.size() == 2)
    // is fwd_Z[ m*lengths[0]*lengths[1] + k1*lengths[1] + k2 ]
    // - {k1; m} (for 1D DFTs, i.e., lengths.size() == 1)
    // is fwd_Z[ m*lengths[0] + k1 ]
    // (0 <= kj < lengths[j-1])
    // wherein
    // using real_t =
    //   std::conditional_t<prec == dft_ns::precision::SINGLE, float, double>;
    // real_t* fwd_Z = static_cast<real_t>*(device_accessible_usm_data_out);
}
}

```

Distributed DFT

Starting from Intel® oneAPI Math Kernel Library (oneMKL) 2025.2 release, a DPC++ interface for computing Distributed Discrete Fourier Transforms is introduced. This is designed to perform FFTs on a collection (single or multi mode) of SYCL GPU devices, where each individual GPU device is accessible within its respective process. To organize communication between different processes, the interface uses the Message Passing Interface (MPI). This interface declares the `oneapi::mkl::experimental::dft` namespace, which contains

- the [scoped enumerations](#) `oneapi::mkl::experimental::dft::distributed_config_param` and `oneapi::mkl::experimental::dft::distributed_config_value`;
- the `oneapi::mkl::experimental::dft::distributed_descriptor` [class template](#);
- the `oneapi::mkl::experimental::dft::compute_forward` and `oneapi::mkl::experimental::dft::compute_backward` [function templates](#).

This new interface closely resembles the [single process DPC++ interface](#) and thus re-uses the [scoped enumerations](#) defined in the `oneapi::mkl::dft` namespace for configuring and executing a DFT distributed across multiple processes. For a DFT of forward domain and floating-point format represented by the values `dom` and `prec` (known at compile time) of respective types `oneapi::mkl::dft::domain` and `oneapi::mkl::dft::precision` (see the [scoped enumerations](#)), the desired global transform and its general configuration are to be communicated uniformly across each process via an object of the `oneapi::mkl::experimental::dft::distributed_descriptor<prec, dom>` class. Once successfully

committed across all the involved processes to the desired DFT configuration and to a user-provided local `sycl::queue` instance, that `oneapi::mkl::experimental::dft::distributed_descriptor` object can be used as an argument to the appropriate compute function(s) along with the relevant local chunks of input and output data.

The distributed DFT DPC++ interface computes a DFT in five steps:

1. Each process creates a `oneapi::mkl::experimental::dft::distributed_descriptor` object `dist_desc` for the targeted global DFT problem with a call to the relevant [parameterized constructor](#), e.g.,


```
distributed_descriptor<prec, dom> dist_desc(MPI_COMM_WORLD, lengths);
```

 wherein `prec` and `dom` are specialization values of types `oneapi::mkl::dft::precision` and `oneapi::mkl::dft::domain`, respectively. `dist_desc` captures the configuration of the global transform, such as the dimensionality (or rank), length(s), number of transforms, layout of the input and output data (defined by strides, distances, and possibly other configuration parameters), scaling factors, etc. All the configuration settings are assigned default values in this call, which might need to be modified thereafter.

By default, `distributed_descriptor` objects within each process are initialized for the in-place calculation of an unbatched ($M = 1$), unscaled ($\sigma_\delta = 1, \forall \delta$) global DFT of the forward domain, precision and length(s) set at construction.
2. Optionally adjust the configuration of `dist_desc` by calling its relevant [configuration-setting member function\(s\)](#) as many times as needed including the data distribution configuration. The value associated with (almost) any configuration parameter can be obtained with the appropriate [configuration-querying member function\(s\)](#) (default values are returned unless the queried configuration parameter was previously set). The configurations defining the global transform must be set uniform across all processes, otherwise the behavior is undefined(except for the [custom distribution configuration](#)).
3. Commit `dist_desc` with a call to its [commit member function](#); that is, make the object ready to compute the global transform. All the `dist_desc` objects across the processes need to be successfully committed for performing the global transform. Once the objects are committed, the configuration parameters of the global DFT, are considered frozen for computation purposes: changing any of them after committing the object effectively invalidates it for computation purposes until the `commit` member function is called again. The `commit` member function takes in a `sycl::queue` object which is built upon the `sycl::device` object which is mapped to a physical device by MPI.
4. Use the committed `dist_desc` to query the local size of the device memory allocations needed for the respective domain(forward or backward) within each process and initialize the input data. The [configuration-querying member function](#) can be used to achieve this.
5. Use the committed `dist_desc` to call the (appropriate) `oneapi::mkl::experimental::dft::compute_forward` or `oneapi::mkl::experimental::dft::compute_backward` [functions](#) as needed to compute the desired global transform(s). These functions require no other argument than a committed distributed descriptor object and the device-accessible input and output data.

Supported functionality and limitations

- Only 2D and 3D transforms are supported, with the following limitations,
 - The dimensions must have length greater than or equal to the number of processes.
 - Batching is not supported.
 - Only default packed layouts are supported.
- Supports Intel® Data Center GPU Max Series device only.
- All the processes must be provided the same MPI communicator.
- Currently only supports Intel® MPI.
- The environment variable `I_MPI_OFFLOAD` must be set to 1 to be functional; otherwise, an exception will be thrown.

NOTE The mapping of the available SYCL devices to the processes is controlled by MPI.

Data Distribution Among Processes

This page describes how the local arrays are distributed across the available processes that make up the virtual global array on which the DFT is applied. It is the user's responsibility to properly allocate, initialize the input, and obtain the result. See the sections below on how the input and output data are expected to be distributed among the processes.

Non-batched transforms

For single multi-dimensional transforms, the forward and backward domain data are required to be distributed among the available processes. This can be done either by utilizing the built-in slab decompositions or by providing a custom pencils and slabs decomposition.

Slab decompositions

The distributed DFT interface provides built-in row distribution for a 2D transform and slab distribution for a 3D transform respectively. Note that the slab decomposition does not allow strides leading to a non-packed layout of the global array. In case of the real forward domain, data must to be padded for an in-place operation whereas it can be padded or packed for an out-of-place operation.

For a 2D transform, consider a C style 2D array of size $[Y][X]$ distributed over p processes. The possible row distributions are shown below,

- Decomposition along Y means, the first $Y \% p$ processes each own $(Y/p + 1) * X$ elements and the remaining processes each own $(Y/p) * X$ elements.
- Decomposition along X means, the first $X \% p$ processes each own $(X/p + 1) * Y$ elements and the remaining processes each own $(X/p) * Y$ elements.

Similarly for a 3D transform, consider a C style 3D array of size $[Z][Y][X]$ distributed over p processes. The possible slab decompositions are shown below,

- Decomposition along Z means, the first $Z \% p$ processes each own $(Z/p + 1) * Y * X$ elements and the remaining processes each own $(Z/p) * Y * X$ elements.
- Decomposition along Y means, the first $Y \% p$ processes each own $(Y/p + 1) * X * Z$ elements and the remaining processes each own $(Y/p) * X * Z$ elements.
- Decomposition along X means, the first $X \% p$ processes each own $(X/p + 1) * Y * Z$ elements and the remaining processes each own $(X/p) * Y * Z$ elements.

Choosing the dimension along which the decomposition is to be done, for either a forward or backward domain can be achieved by using the `set_value` member function of the `oneapi::mkl::experimental::dft::distributed_dft` class.

NOTE Currently, only the [default slab decomposition](#) is implemented. An attempt to slab decompose unimplemented dimensions will throw a `oneapi::mkl::unimplemented` exception.

Custom pencil and slab distribution

Additionally, the interface supports custom data decompositions in the form of rectangles/blocks for 2D and 3D transforms respectively. These rectangles/blocks define a subregion of the global array by specifying the lower and upper corners. By assigning each block to a process, one can represent a data distribution in which each process owns a portion of the global array.

Calling the `set_value` member function of the `oneapi::mkl::experimental::dft::distributed_dft` class with appropriate bounds and strides notifies that a custom decomposition is being used.

NOTE Currently, custom decomposition is not implemented. An attempt to use custom decomposition will throw a `oneapi::mkl::unimplemented` exception.

Batched transforms

For batches of transforms, the total number of batches are divided among the available processes and each individual batch is executed completely within its respective process. If possible the batches are evenly distributed among the processes. For a batch of size b performed on p processes, where b is not divisible by p , the first $b \% p$ processes will perform $[b/p] + 1$ transforms and the remaining processes will perform b/p transforms.

NOTE Currently, batched transforms are not implemented and a `oneapi::mkl::exception` will be thrown at `commit` time if `batch != 1` was `set`.

Scoped enumerations

This page describes the scoped enumerations pertaining to the definition or configuration of `oneapi::mkl::experimental::dft::distributed_descriptor` objects:
`oneapi::mkl::experimental::dft::distributed_config_param`.

```
namespace oneapi::mkl::experimental::dft {
    enum class distirbuted_config_param {
        fwd_divided_dimension,
        bwd_divided_dimension,
        fwd_distribution,
        bwd_distribution,
        fwd_local_data_size_bytes,
        bwd_local_data_size_bytes
    };
}
```

The scoped enumerations above belong to the `oneapi::mkl::experimental::dft` namespace and are declared in `oneapi/mkl/experimental/distributed_dft.hpp` (file to be included). The usage of prepended namespace specifiers `oneapi::mkl::experimental::dft` is omitted below for conciseness. Note that the [configuration parameters](#) and [configuration values](#) defined [here](#) are re-used within the `distributed_descriptor` class's [member functions](#) to set/get most of the global DFT properties such as forward/backward scale, strides etc. This section discusses the new **`distributed_config_param`** introduced in the `oneapi::mkl::experimental::dft` namespace to be used exclusively for performing a distributed DFT.

The default configuration values associated with all writable distributed configuration parameters are documented in the page dedicated to the [configuration-setting member function\(s\)](#) of the `distributed_descriptor` class template.

`distributed_config_param`

This scoped enumeration type is used to represent distributed-dft-specific configuration parameters of oneMKL DFT `distributed_descriptor` objects. For any `distributed_descriptor` object, all its configuration parameters are associated with a configuration value. The type of the latter depends on the

configuration parameter and is documented in the page specific to the [configuration-setting](#) (or [configuration-querying](#), for read-only parameters) member functions of the `distributed_descriptor` class template.

Enumerators of the `distributed_config_param` scoped enumeration along with the corresponding configuration parameters of `distributed_descriptor` objects.

Enumerator(s)	Corresponding configuration parameter.
<code>distributed_config_param::fwd_divided_dimension</code>	Configuration parameter indicating which dimension to be slab decomposed for the forward domain.
<code>distributed_config_param::bwd_divided_dimension</code>	Configuration parameter indicating which dimension to be slab decomposed for the backward domain.
<code>distributed_config_param::fwd_distribution</code>	Configuration parameter indicating that a custom decomposition is to be used for the forward domain.
<code>distributed_config_param::bwd_distribution</code>	Configuration parameter indicating that a custom decomposition is to be used for the backward domain.
<code>distributed_config_param::fwd_local_data_size_bytes</code>	Exact size in bytes to be allocated for the forward domain within that specific process.
<code>distributed_config_param::bwd_local_data_size_bytes</code>	Exact size in bytes to be allocated for the backward domain within that specific process.

NOTE The configuration parameters represented by

`distributed_config_param::fwd_local_data_size_bytes`,

`distributed_config_param::bwd_local_data_size_bytes`, are *read-only*: they are not writable.

The `distributed_descriptor` class template

This page describes the `distributed_descriptor` class template and its member functions, which belongs to the `oneapi::mkl::experimental::dft` namespace and is declared in `oneapi/mkl/experimental/distributed_dft.hpp` (file to be included).

```
namespace oneapi::mkl::experimental::dft {
    template <oneapi::mkl::dft::precision prec, oneapi::mkl::dft::domain dom>
    class distributed_descriptor;
}
```

The usage of prepended namespace specifiers `oneapi::mkl::experimental::dft` is omitted below for conciseness while the namespace specifier `oneapi::mkl::dft` is kept for clarity.

Users of the distributed DFT DPC++ interface of oneMKL must use instances of a specialization of this class template to specify and fully configure the required global DFT computation(s): successfully-committed objects of a (specialized) `distributed_descriptor` class are required arguments to the distributed DFT specific [compute functions](#).

Template parameters

The `distributed_descriptor` class template is parameterized by two non-type template parameters, in the following order:

- a value of [type](#) `oneapi::mkl::dft::precision`, that determines the floating-point format to be considered by its instances;
- a value of [type](#) `oneapi::mkl::dft::domain`, that determines the type of forward domain to be considered by its instances.

Instances of a `distributed_descriptor` class specialized with value `oneapi::mkl::dft::precision::SINGLE` (resp. `oneapi::mkl::dft::precision::DOUBLE`) for the former are referred to as “single-precision descriptors” (resp. “double-precision descriptors”). Similarly, instances of a `distributed_descriptor` class specialized with value `oneapi::mkl::dft::domain::COMPLEX` (resp. `oneapi::mkl::dft::domain::REAL`) for the latter are referred to as “complex descriptors” (resp. “real descriptors”).

This section describes the constructors and destructor of the `distributed_descriptor` class template.

```
namespace oneapi::mkl::experimental::dft {
    template <oneapi::mkl::dft::precision prec, oneapi::mkl::dft::domain dom>
    class distributed_descriptor {
    public:
        // parameterized constructors:
        distributed_descriptor(MPI_Comm Comm, std::vector<std::int64_t> dimensions);
        // destructor
        ~distributed_descriptor();
        // unsupported copy constructor and assignment operator:
        distributed_descriptor(const distributed_descriptor&) = delete;
        distributed_descriptor& operator=(const distributed_descriptor&) = delete;
        // unsupported move constructor and assignment operator:
        distributed_descriptor(distributed_descriptor&&) = delete;
        distributed_descriptor& operator=(distributed_descriptor&&) = delete;
    }
}
```

Parameterized constructors

The parameterized constructors of any `distributed_descriptor` class allocate memory for an object’s data structures and default-configure it for the precision, forward domain, and length(s) of the transform it defines. These constructors do not trigger any significant computational work in preparation of the DFT that the object defines upon creation; all such tasks are operated when the object is [committed](#) to its DFT definition and to a given `sycl::queue` instance.

Input parameter

The input parameter of the constructor is detailed in the tables below. The tables use the notations presented in the [introduction](#) when referring to parameters of the DFT operation defined by the object.

Parameterized constructor’s input parameter for any `distributed_descriptor` object.

Name	Type	Description
Comm	<code>MPI_Comm</code>	The MPI communicator containing the processes among which the given DFT problem will be distributed.
lengths	<code>std::vector<std::int64_t></code>	Vector of size d , containing $\{n_1, n_2, \dots, n_d\}$, in that order (lengths of the global DFT)

Exceptions

The parameterized constructors may throw an `oneapi::mkl::exception` if

- $d < 2$ or $d > 3$;
- The dimensions have length less than the number of processes.
- Any MPI related error occurs.
- The construction of the `distributed_descriptor` object fails to allocate its required resources.

Copy constructor and assignment operator

Copy construction and assignment are not supported.

Move constructor and assignment operator

Move constructor and assignment operator are not supported.

Destructor

The destructor of any `distributed_descriptor` class frees all resources allocated for and by objects of that class within the respective MPI process.

This section describes the overloaded configuration-setting member functions `set_value` of the `distributed_descriptor` class template.

```
namespace oneapi::mkl::experimental::dft {
    template <oneapi::mkl::dft::precision prec, oneapi::mkl::dft::domain dom>
    class distributed_descriptor {
        using real_scalar_t = std::conditional_t<prec == oneapi::mkl::dft::precision::DOUBLE,
        double, float>;
    public:
        void set_value(oneapi::mkl::dft::config_param, oneapi::mkl::dft::config_value);
        void set_value(oneapi::mkl::dft::config_param, std::int64_t);
        void set_value(distributed_config_param, std::int64_t);
        void set_value(oneapi::mkl::dft::config_param, const std::vector<std::int64_t>&);
        template <typename T, std::enable_if_t<std::is_integral_v<T>, bool> = true>
        void set_value(distributed_config_param param, T value) {
            set_value(param, static_cast<std::int64_t>(value));
        }
        void set_value(oneapi::mkl::dft::config_param, real_scalar_t);
        template <typename T, std::enable_if_t<std::is_integral_v<T>, bool> = true>
        void set_value(oneapi::mkl::dft::config_param param, T value) {
            set_value(param, static_cast<std::int64_t>(value));
        }
        template <typename T, std::enable_if_t<std::is_floating_point_v<T>, bool> = true>
        void set_value(oneapi::mkl::dft::config_param param, T value) {
            set_value(param, static_cast<real_scalar_t>(value));
        }
        void set_value(distributed_config_param param,
            const std::vector<std::int64_t> &lower_bound,
            const std::vector<std::int64_t> &upper_bound,
            const std::vector<std::int64_t> &strides);
    }
}
```

The `set_value` functions enable oneMKL users to assign a configuration value to any *writable* configuration parameter either `oneapi::mkl::dft::config_param` or `distributed_config_param` of a `distributed_descriptor` object. Most of the `set_value` functions are similar to what was discussed in [configuration-setting-functions](#) and should be referenced for setting global transform configuration like

forward/backward scale, etc. These params should be uniformly set across all processes. Also note that the deprecated functionality is not supported by the distributed DFT, trying to set any of the deprecated parameters will throw a `oneapi::mkl::invalid_argument` exception. Below, we will discuss distributed DFT specific `set_value` overload functions and also document the default configuration values associated with the respective configuration parameters.

NOTE Regarding the `oneapi::mkl::dft::FWD_STRIDES` and `oneapi::mkl::dft::BWD_STRIDES`, the global layout of the transform is expected to be conforming to the default layout discussed in [configuring strides for forward and backward domains](#). For distributed DFT, in the case of real transforms, along with supporting padded data in forward domain for both in-place and out-of-place operations, the out-of-place operation additionally supports packed strides in forward domain.

Configuration values may be successfully and correctly *set* for a `distributed_descriptor` object but found invalid later on, when attempting to [commit](#) that object. Assessing the validity of a `distributed_descriptor` object's configuration requires knowledge (and analysis) of *all* its configuration values (considered "frozen" when the object is committed). Any call to `set_value` resulting in a configuration change for a *committed* `distributed_descriptor` object effectively uncommits that object: indeed, such a change typically invalidates the object's compute-readiness preparation steps operated when it was last [committed](#). As a consequence, any such operation changes the object's (read-only) configuration value associated with configuration parameter `oneapi::mkl::dft::config_param::COMMIT_STATUS` from `oneapi::mkl::dft::config_value::COMMITTED` to `oneapi::mkl::dft::config_value::UNCOMMITTED`.

Setting slab distribution

The following member functions

```
template <oneapi::mkl::dft::precision prec, oneapi::mkl::dft::domain dom>
void distributed_descriptor<prec, dom>::set_value(distributed_config_param param, std::int64_t
value);
template <oneapi::mkl::dft::precision prec, oneapi::mkl::dft::domain dom>
template <typename T, std::enable_if_t<std::is_integral_v<T>, bool> = true>
void distributed_descriptor<prec, dom>::set_value(distributed_config_param param, T value) {
    set_value(param, static_cast<std::int64_t>(value));
}
```

enable users to set the dimension to be slab decomposed in the respective domain(forward or backward).

Slab distribution configuration parameters along with their accepted and default values.

Accepted configuration parameters	Accepted values	Default value
<code>distributed_config_param::fwd_divided_dimension</code>	0 to rank-1	0
<code>distributed_config_param::bwd_divided_dimension</code>	0 to rank-1	1

NOTE Only the default slab distribution values are supported as noted above. Attempting to use a non-default value will result in an `oneapi::mkl::unimplemented` exception being thrown.

Setting custom distribution

The following member function

```
template <oneapi::mkl::dft::precision prec, oneapi::mkl::dft::domain dom>
void distributed_descriptor<prec, dom>::set_value(distributed_config_param param,
                                                const std::vector<std::int64_t> &lower_bound,
                                                const std::vector<std::int64_t> &upper_bound,
                                                const std::vector<std::int64_t> &strides);
```

enables users to set the per-process rectangle/block's lower and upper bounds along with the strides within that local portion.

Custom distribution configuration setting function.

Parameter	Accepted values	Definition
param	distributed_config_param::f wd_distribution or distributed_config_param::b wd_distribution	distributed_config_param determining the domain for which the custom decomposition is being set.
lower_bound	std::vector<std::int64_t> object of size (d)	An std::vector of type std::int64_t whose length equals the rank of the transform, representing the lower-corner of the portion of the global array owned by the current process.
upper_bound	std::vector<std::int64_t> object of size (d)	An std::vector of type std::int64_t whose length equals the rank of the transform, representing the upper-corner of the portion of the global array owned by the current process.
strides	std::vector<std::int64_t> object of size (d)	An std::vector of type std::int64_t whose length equals the rank of the transform, representing the local data layout in memory either for forward or backward domain respectively. Strides must be in decreasing order and positive.

NOTE Currently, custom distribution is not implemented. Attempting to set custom distribution will throw a oneapi::mkl::unimplemented exception.

Exceptions

The configuration-setting member functions may throw

- an std::runtime_error exception if an issue is found with the calling object;
- a oneapi::mkl::invalid_argument exception if
 - the parameter being set is not writable;

- the parameter being set is rejected, e.g., if it is inconsistent with the type of configuration value being used;
- the configuration value being set is rejected for the specific configuration parameter being set.
- a `oneapi::mkl::unimplemented` exception if - the parameter being set is yet to be implemented.

This section describes the overloaded configuration-querying member functions `get_value` of the `distributed_descriptor` class template.

```
namespace oneapi::mkl::experimental::dft {
    template <oneapi::mkl::dft::precision prec, oneapi::mkl::dft::domain dom>
    class distributed_descriptor {
        using real_scalar_t = std::conditional_t<prec == oneapi::mkl::dft::precision::DOUBLE,
        double, float>;
    public:
        // for the type of forward domain:
        void get_value(oneapi::mkl::dft::config_param, oneapi::mkl::dft::domain*) const;
        // for the floating-point format:
        void get_value(oneapi::mkl::dft::config_param, oneapi::mkl::dft::precision*) const;
        // for integer-valued parameters:
        void get_value(oneapi::mkl::dft::config_param, std::int64_t*) const;
        void get_value(distributed_config_param, std::int64_t*) const;
        // for vector-valued parameters:
        void get_value(oneapi::mkl::dft::config_param, std::vector<std::int64_t>*) const;
        // for real-valued parameters:
        void get_value(oneapi::mkl::dft::config_param, real_scalar_t*) const;
        // for custom distribution:
        void get_value(distributed_config_param param,
            std::vector<std::int64_t> *lower_bound,
            std::vector<std::int64_t> *upper_bound,
            std::vector<std::int64_t> *strides) const;
        // for other parameters:
        void get_value(oneapi::mkl::dft::config_param, oneapi::mkl::dft::config_value*) const;
    }
}
```

The `get_value` functions enable oneMKL users to query the configuration value associated with a `oneapi::mkl::dft::config_param` or `distributed_config_param` of a `distributed_descriptor` object. Most of the `get_value` functions are similar to what was discussed in [configuration-querying-functions](#) and should be referenced for querying transform configuration like forward/backward scale, etc. The calling `distributed_descriptor` object is left unchanged by any call to a configuration-querying member function. Also note that the deprecated functionality is not supported by the distributed DFT, trying to query any of the deprecated parameters will throw an exception. Below, we will discuss distributed DFT specific `get_value` overload functions.

Querying integer-valued parameters

The following member function

```
template <oneapi::mkl::dft::precision prec, oneapi::mkl::dft::domain dom>
void distributed_descriptor<prec, dom>::get_value(distributed_config_param param, std::int64_t*
value_ptr) const;
```

enables users to query the configuration value for the configuration parameters in the table below.

Behavior of the configuration-querying member function for integer-valued parameters.

Accepted configuration parameters	Value written by oneMKL
distributed_config_param::fwd_divided_dimension, distributed_config_param::bwd_divided_dimension, distributed_config_param::fwd_local_data_size_bytes, distributed_config_param::bwd_local_data_size_bytes	value_ptr[0]

The distributed DFT may require more memory than that which can be deduced from the [data distribution](#) for the forward or backward domains within each process. The configuration parameters `distributed_config_param::fwd_local_data_size_bytes` and `distributed_config_param::bwd_local_data_size_bytes` can be used to query the process-specific number of bytes in the respective domain (forward or backward) for which the device-accessible memory must be allocated and then initialized with input data. This must be done after the `distributed_descriptor` has been committed (only after which the exact number of bytes that must be allocated is known).

Querying custom distribution

The following member function

```
template <oneapi::mkl::dft::precision prec, oneapi::mkl::dft::domain dom>
void get_value(distributed_config_param param,
               std::vector<std::int64_t> *lower_bound,
               std::vector<std::int64_t> *upper_bound,
               std::vector<std::int64_t> *strides) const;
```

enables users to query the custom distribution bounds for that process.

Custom distribution configuration querying function.

Parameter	Accepted values	Definition
param	<code>distributed_config_param::fwd_distribution</code> or <code>distributed_config_param::bwd_distribution</code>	<code>distributed_config_param</code> determining the domain for which the custom decomposition is being queried.
lower_bound	<code>std::vector<std::int64_t> *</code> object of size (d)	A pointer to <code>std::vector</code> of type <code>std::int64_t</code> whose length equals the rank of the transform, to query the lower-corner of the portion of the global array owned by the current process.
upper_bound	<code>std::vector<std::int64_t> *</code> object of size (d)	A pointer to <code>std::vector</code> of type <code>std::int64_t</code> whose length equals the rank of the transform, to query the upper-corner of the portion of the global array owned by the current process.

Parameter	Accepted values	Definition
strides	<code>std::vector<std::int64_t> *</code> object of size (d)	An <code>std::vector</code> of type <code>std::int64_t</code> whose length equals the rank of the transform, to query the local data layout in memory either for forward or backward domain respectively.

NOTE Currently, custom distribution is not implemented. Attempting to query custom distribution will throw a `oneapi::mkl::unimplemented` exception.

Exceptions

The configuration-querying member functions may throw

- an `std::runtime_error` exception if an issue is found with the calling object;
- a `oneapi::mkl::uninitialized` exception if the calling object is uncommitted yet queried about a configuration parameter that requires the calling object to be committed;
- a `oneapi::mkl::unimplemented` exception if the queried parameter corresponds to a feature that is not implemented for the calling object;
- a `oneapi::mkl::invalid_argument` exception if
 - the pointer arguments are `nullptr`;
 - the parameter being queried is rejected, *e.g.*, if it is inconsistent with the pointer type used as second argument;
 - the size of the vector pointed by `value_ptr` is not as required;
 - the configuration value to be returned cannot be safely or accurately converted into the desired type, *e.g.*, if querying a scaling factor that happens to be `0.5` using the configuration-querying member function specific to integer-valued configuration parameters.

This section describes the configuration- and queue-committing member function `commit` of the `distributed_descriptor` class template.

```
namespace oneapi::mkl::experimental::dft {
    template <oneapi::mkl::dft::precision prec, oneapi::mkl::dft::domain dom>
    class distributed_descriptor {
    public:
        void commit(sycl::queue &user_queue);
    }
}
```

Invoking this function notifies that the calling object's configuration is completed and triggers it to complete the required initialization steps (*e.g.*, pre-computing data, exploring various factorizations, assessing suitability of various algorithms) relevant to enqueueing the DFT computations that it defines to the user-provided `user_queue`. That `sycl::queue` object is mapped to a physical device within the available devices by the MPI.

Upon successful completion, the calling object is "committed" (its configuration value associated with `config_param::COMMIT_STATUS` is then `config_value::COMMITTED`). `distributed_descriptor` objects *must* be committed to be used in any [compute function](#). Note that all the processes should successfully commit their `distributed_descriptor` object to obtain correct results.

Changing any configuration setting of a committed object effectively leaves it “uncommitted” (its configuration value associated with `config_param::COMMIT_STATUS` is then `config_value::UNCOMMITTED`). As a consequence, it is best to avoid any call to any `set_value` member function after invoking the `commit` member function.

Input parameter

Input parameter for the `commit` member function.

Name	Type	Description
<code>user_queue</code>	<code>sycl::queue</code>	Queue to which the local DFT computations are to be enqueued by the calling object, when used in compute functions thereafter

Exceptions

The configuration- and queue-committing member function may throw

- `oneapi::mkl::unimplemented` exception, *e.g.*, if the calling object’s configuration is not supported (yet);
 - Non-default slab distribution is used.
- `oneapi::mkl::exception` on exception if,
 - Device is not an Intel® Data Center GPU Max Series.
 - The environment variable `I_MPI_OFFLOAD` is not set to 1.
 - SYCL Backend is not Level Zero.
 - Default packed layouts are not used for the global array.
 - Batching is used (setting `oneapi::mkl::dft::config_param::NUMBER_OF_TRANSFORMS > 1`).
 - Failure to allocate and initialize resources required for the `distributed_descriptor` object.

Compute Functions

This page describes the function templates `oneapi::mkl::experimental::dft::compute_forward` and `oneapi::mkl::experimental::dft::compute_backward`, which both pertain to the calculation of a distributed DFT defined by a *committed* `oneapi::mkl::experimental::dft::distributed_descriptor` object `desc`. These function templates belong to the `oneapi::mkl::experimental::dft` namespace and are declared in `oneapi/mkl/experimental/distributed_dft.hpp` (file to be included). The usage of prepended namespace specifier `oneapi::mkl::experimental::dft` is omitted below for conciseness while the namespace specifier `oneapi::mkl::dft` is kept for clarity.

```
namespace oneapi::mkl::experimental::dft {
/*----- for in-place transforms -----*/
template <typename descriptor_type, typename data_type>
sycl::event compute_forward(descriptor_type &desc,
                           data_type *inout,
                           const std::vector<sycl::event> &dependencies = {});
template <typename descriptor_type, typename data_type>
sycl::event compute_backward(descriptor_type &desc,
                             data_type *inout,
                             const std::vector<sycl::event> &dependencies = {});
```

```
/*----- for out-of-place transforms -----*/
template<typename descriptor_type, typename input_type, typename output_type>
sycl::event compute_forward(descriptor_type &desc,
                           input_type *in,
                           output_type *out,
                           const std::vector<sycl::event> &dependencies = {});
template<typename descriptor_type, typename input_type, typename output_type>
sycl::event compute_backward(descriptor_type &desc,
                             input_type *in,
                             output_type *out,
                             const std::vector<sycl::event> &dependencies = {});
}
```

Description

The `compute_forward` (resp. `compute_backward`) function templates accept a *committed* `distributed_descriptor` object `desc` followed by one or more data-containing arguments, and enqueue the computation of the forward (resp. backward) DFT that `desc` defines, *i.e.*, the transform defined by $\delta = -1$ (resp. $\delta = +1$). The computations are added to the queue to which `desc` was *committed*. Each process takes in the local chunk of the input, computes DFT on the data it is provided, exchanges the intermediate results using MPI, and provides the final local output chunk. Refer to [data distribution](#) on how the local input/output data is distributed across the processes.

Successive global computations require the `distributed_descriptor` objects across the processes be serialized to guarantee correct results. This can be done by waiting on the queue passed at [commit](#), or by using an in-order queue. Alternatively, computations can be serialized using the returned `sycl::event` and input dependencies.

The number of data-containing arguments to be provided depends exclusively on whether the desired DFT calculations are to be done in-place (output data overwriting the input data) or out-of-place (output data stored separately from the input data). One data-containing argument is required for in-place operations whereas two such arguments are needed for out-of-place calculations. Note that in-place operations require the global data layout to satisfy requirements documented in the page dedicated to [configuring data layouts](#).

Warning

- When using a function designed for in-place (resp. out-of-place) operations, `desc` (function's first argument) *must* have a configuration value of `oneapi::mkl::dft::config_value::INPLACE` (resp. `oneapi::mkl::dft::config_value::NOT_INPLACE`) associated with `oneapi::mkl::dft::config_param::PLACEMENT`. And all the processes should be configured uniformly. The function's behavior is undefined otherwise.
 - When using a function designed for out-of-place operations, `in` and `out` must not share common elements.
 - The sizes of the `in` and `out` must be greater than or equal to the [queried](#) sizes after commit.
-

Template parameters

For in-place operations

The `compute_forward` and `compute_backward` function templates for in-place operations are parameterized by two type template parameters, in the following order:

- `descriptor_type` representing the type of `desc`, an instance of a specialization of the [descriptor class template](#);

- `data_type` representing the elementary data type of the data-containing argument being used.

The available specializations are listed in the table below.

Available specializations of the compute function templates for in-place operations.

function template	descriptor_type	data_type
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::SINGLE, oneapi::mkl::dft::domain::REAL></code>	<code>float</code>
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::SINGLE, oneapi::mkl::dft::domain::REAL></code>	<code>std::complex<float></code>
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::SINGLE, oneapi::mkl::dft::domain::COMPLEX></code>	<code>float</code>
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::SINGLE, oneapi::mkl::dft::domain::COMPLEX></code>	<code>std::complex<float></code>
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::DOUBLE, oneapi::mkl::dft::domain::REAL></code>	<code>double</code>
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::DOUBLE, oneapi::mkl::dft::domain::REAL></code>	<code>std::complex<double></code>
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::DOUBLE, oneapi::mkl::dft::domain::COMPLEX></code>	<code>double</code>
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::DOUBLE, oneapi::mkl::dft::domain::COMPLEX></code>	<code>std::complex<double></code>

For out-of-place operations

The `compute_forward` and `compute_backward` function templates for out-of-place operations are parameterized by three type template parameters, in the following order:

- `descriptor_type` representing the type of `desc`, an instance of a specialization of the [descriptor class template](#);
- `input_type` representing the elementary data type of the first (input) data-containing argument being used;
- `output_type` representing the elementary data type of the second (output) data-containing argument being used.

The available specializations are listed in the table below.

Available specializations of the compute function templates for out-of-place operations.

function template	descriptor_type	input_type	output_type
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::SINGLE, oneapi::mkl::dft::domain::REAL></code>	<code>float</code>	<code>float</code>
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::SINGLE, oneapi::mkl::dft::domain::REAL></code>	<code>std::complex<float></code>	<code>std::complex<float></code>
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::SINGLE, oneapi::mkl::dft::domain::COMPLEX></code>	<code>float</code>	<code>float</code>
<code>compute_forwardcompute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::SINGLE, oneapi::mkl::dft::domain::COMPLEX></code>	<code>std::complex<float></code>	<code>std::complex<float></code>
<code>compute_forward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::SINGLE, oneapi::mkl::dft::domain::REAL></code>	<code>float</code>	<code>std::complex<float></code>
<code>compute_backward</code>	<code>distributed_descriptor<oneapi::mkl::dft::precision::SINGLE</code>	<code>std::complex<float></code>	<code>float</code>

function template	descriptor_type	input_type	output_type
<code>compute_forwardcomp ute_backward</code>	<code>E, oneapi::mkl::dft::d omain::REAL></code> <code>distributed_descrip tor<oneapi::mkl::df t::precision::DOUBL E, oneapi::mkl::dft::d omain::REAL></code>	<code>double</code>	<code>double</code>
<code>compute_forwardcomp ute_backward</code>	<code>distributed_descrip tor<oneapi::mkl::df t::precision::DOUBL E, oneapi::mkl::dft::d omain::REAL></code>	<code>std::complex<double ></code>	<code>std::complex<double ></code>
<code>compute_forwardcomp ute_backward</code>	<code>distributed_descrip tor<oneapi::mkl::df t::precision::DOUBL E, oneapi::mkl::dft::d omain::COMPLEX></code>	<code>double</code>	<code>double</code>
<code>compute_forwardcomp ute_backward</code>	<code>distributed_descrip tor<oneapi::mkl::df t::precision::DOUBL E, oneapi::mkl::dft::d omain::COMPLEX></code>	<code>std::complex<double ></code>	<code>std::complex<double ></code>
<code>compute_forward</code>	<code>distributed_descrip tor<oneapi::mkl::df t::precision::DOUBL E, oneapi::mkl::dft::d omain::REAL></code>	<code>double</code>	<code>std::complex<double ></code>
<code>compute_backward</code>	<code>distributed_descrip tor<oneapi::mkl::df t::precision::DOUBL E, oneapi::mkl::dft::d omain::REAL></code>	<code>std::complex<double ></code>	<code>double</code>

Input parameters and returned value

The `compute_forward` and `compute_backward` template functions' input parameters are documented below. These functions return a `sycl::event` object that can be used to track the completion of the computations that the function call enqueues.

For in-place operations

Input parameters for the in-place compute functions.

Name	Supported types	Description
desc	descriptor_type	<i>Committed</i> instance of a specialization of the distributed_descriptor class template , defining the global DFT to compute
inout	data_type*	Data-containing argument for the DFT (local input data on input, local output results on output)
dependencies	std::vector<sycl::event>	Vector of dependencies to be honored by the enqueued DFT computation

For out-of-place operations

Input parameters for the out-of-place compute functions.

Name	Supported types	Description
desc	descriptor_type	<i>Committed</i> instance of a specialization of the descriptor class template , defining the DFT to compute
in	input_type*	Data-containing argument for the local input data of the DFT
out	output_type*	Data-containing argument for the local output data of the DFT
dependencies	std::vector<sycl::event>	Vector of dependencies to be honored by the enqueued DFT computation

Exceptions

The `compute_forward` and `compute_backward` template functions may throw an `oneapi::mkl::exception` in the following cases:

- an issue is found with `desc`.
- `desc` is uncommitted.
- a failure is detected when computing the DFT.
- an MPI issues is detected when computing the DFT.

NOTE The distributed DFT interface does not support SYCL buffers.

Data Fitting

- [Routines](#)
- [Error Handling](#)

Intel® oneAPI Math Kernel Library (oneMKL) provides spline-based interpolation capabilities that can be used for spline construction (Linear, Cubic, Quadratic etc.), to perform cell-search operations, and to approximate functions, function derivatives, or integrals.

APIs are experimental. It means that no API or ABI backward compatibility are guaranteed.

APIs are based on SYCL USM (Unified Shared Memory) input datas.

Routines

Splines: [Splines](#)

Interpolate function: [Interpolate Function](#)

Error Handling

The DPC++ error handling model supports two types of errors:

1. Synchronous errors cause the DPC++ host runtime libraries throw exceptions.
2. Asynchronous errors may only be processed in a user-supplied error handler associated with a SYCL queue.

For routines, handling all errors, synchronous or asynchronous, is a responsibility of the caller. Specifically:

- Exceptions are thrown explicitly by algorithms in the following scenarios:
 - input parameters are unexpected;
 - provided SYCL device is not supported;
 - spline is not fully initialized.
- Exceptions thrown by runtime libraries at the host CPU, including DPC++ synchronous exceptions, are passed through to the caller.
- DPC++ asynchronous errors are not handled.

Common Terms

- [Glossary](#)
- [Mathematical Notation in the Data Fitting Component](#)
- [Hints in the Data Fitting Component](#)
 - [Partition Hints](#)
 - [Function Values Hints](#)
 - [Coefficients Hints](#)
 - [Sites Hints](#)
 - [Interpolation Results Hints](#)
 - [Derivatives Hints](#)
- [Boundary Condition Types](#)

Glossary

Assume we need to interpolate a function $f(x)$ on the $[a, b)$ interval using splines.

Let's break $[a, b)$ into sub-intervals by n points x_i called partition points or simply **partition**. **Function values** at these points ($y_i = f(x_i), i = 1, \dots, n$) are also given.

Spline has k degree if it can be expressed by the following polynomial:

$$P(x) = c_1 + c_2(x - x_i) + c_3(x - x_i)^2 + \cdots + c_{k-1}(x - x_i)^{k-1}.$$

Splines are constructed on $[x_i, x_{i+1})$, $i = 1, \dots, n-1$ sub-intervals. So, for each sub-interval there are following polynomials:

$$P_i(x) = c_{1,i} + c_{2,i}(x - x_i) + c_{3,i}(x - x_i)^2 + \cdots + c_{k-1,i}(x - x_i)^{k-1}.$$

$c_{j,i}$, $j = 1, \dots, k$, $i = 1, \dots, n$ are called spline **coefficients**.

Function is interpolated at points of $[a, b)$. Such points are called interpolation sites or simply **sites**. Sites might or might not equal to partition points.

Mathematical Notation in the Data Fitting Component

Concept	Mathematical Notation
Partition	$\{x_i\}_{i=1,\dots,n}$, where $a = x_1 < x_2 < \cdots < x_n = b$.
Uniform partition	Partition $\{x_i\}_{i=1,\dots,n}$ which meets the following condition: $x_{i+1} - x_i = x_i - x_{i-1}$, $i = 2, \dots, n-1$
Quasi-uniform partition	Partition $\{x_i\}_{i=1,\dots,n}$ which meets the constraint with a constant C defined as: $1 \leq M/m \leq C$, where $M = \max_{i=1,\dots,n-1}(\Delta_i)$, $m = \min_{i=1,\dots,n-1}(\Delta_i)$, $\Delta_i = x_{i+1} - x_i$
Vector-valued function of dimension p being fit	$f(x) = (f_1(x), \dots, f_p(x))$.
A k -order derivative of function $f(x)$ at point t	$f^{(k)}(t)$.
Function p agrees with function f at the points $\{x_i\}_{i=1,\dots,n}$.	For every point ζ in sequence $\{x_i\}_{i=1,\dots,n}$ that occurs m times, the equality $p^{(i-1)}(\zeta) = f^{(i-1)}(\zeta)$ holds for all $i = 1, \dots, m$.

Concept	Mathematical Notation
The k -th divided difference of function f at points $x_i, \dots, x_{i+k}$. This difference is the leading coefficient of the polynomial of order $k+1$ that agrees with f at $x_i, \dots, x_{i+k}$.	$[x_i, \dots, x_{i+k}] f$. In particular, $[x_1] f = f(x_1)$, $[x_1, x_2] f = (f(x_1) - f(x_2)) / (x_1 - x_2)$.

Hints in the Data Fitting Component

The Intel® oneAPI Math Kernel Library (oneMKL) Data Fitting component provides ways to specify some “hints” for partitions, function values, coefficients, interpolation sites.

Partition Hints

The following are supported:

- Non-uniform.
- Quasi-uniform.
- Uniform.

Syntax

```
enum class partition_hint {
    non_uniform,
    quasi_uniform,
    uniform
};
```

Function Values Hints

Let $\{x_i\}_{i=1, \dots, nx}$ is a partition, $\{f_j(x)\}_{j=1, \dots, ny}$ is a vector-valued function. Function values are stored in the one-dimensional array with $nx * ny$ elements. 2 different layouts are possible: row major and column major.

- For row major layout function values are stored as the following:

Let $\{f_{j,i}\}_{j=1, \dots, ny, i=1, \dots, nx}$ is the function value that corresponds to the i -th partition point and the j -th function.

- For column major:

Let $\{f_{i,j}\}_{i=1, \dots, nx, j=1, \dots, ny}$ is the function value that corresponds to the i -th partition point and the j -th function.

The following hints are supported:

- Row major.
- Column major.

Syntax

```
enum class function_hint {
    row_major,
    col_major
};
```

Coefficients Hints

Let $\{x_i\}_{i=1,\dots,nx}$ is a partition, $\{f_j(x)\}_{j=1,\dots,ny}$ is a vector-valued function. Let cubic spline should be constructed. It means that it requires 4 coefficients per each interpolation interval and function value. Coefficients are stored in the one-dimensional array with $4 * (nx - 1) * ny$ elements.

- For row major:

Let $\{c_{j,i,k}\}_{j=1,\dots,ny,i=1,\dots,nx-1,k=1,\dots,4}$ is the coefficient value that corresponds to the i -th partition point, the j -th function.

- For column major:

Let $\{c_{i,j,k}\}_{i=1,\dots,nx-1,j=1,\dots,ny,k=1,\dots,4}$ is the coefficient value that corresponds to the i -th partition point, the j -th function.

The following is supported:

- row major

Syntax

```
enum class coefficient_hint {
    row_major
};
```

Sites Hints

The following are supported:

- Non-uniform.
- Uniform.
- Sorted.

Syntax

```
enum class site_hint {
    non_uniform,
    uniform,
    sorted
};
```

Interpolation Results Hints

Let $\{f_j(x)\}_{j=1,\dots,ny}$ is a vector-valued function, $\{s_i\}_{i=1,\dots,ns}$ are sites, d is a number of derivatives (including interpolation values) that needs to be calculated. So, size of memory to store interpolation results is $nsite * ny * d$ elements.

6 different layouts are possible:

- functions-sites-derivatives

Let $\{r_{j,i,k}\}_{j=1,\dots,ny,i=1,\dots,nsite,k=1,\dots,d}$ is an interpolation result that corresponds to the i -th site, the j -th function, the k -th derivative.

- functions-derivatives-sites

Let $\{r_{j,k,i}\}_{j=1,\dots,ny,k=1,\dots,d,i=1,\dots,nsite}$ is an interpolation result that corresponds to the i -th site, the j -th function, the k -th derivative.

- sites-functions-derivatives

Let $\{r_{i,j,k}\}_{i=1,\dots,nsite,j=1,\dots,ny,k=1,\dots,d}$ is an interpolation result that corresponds to the i -th site, the j -th function, the k -th derivative.

- sites-derivatives-functions

Let $\{r_{i,k,j}\}_{i=1,\dots,nsite,k=1,\dots,d,j=1,\dots,ny}$ is an interpolation result that corresponds to the i -th site, the j -th function, the k -th derivative.

- derivatives-functions-sites

Let $\{r_{k,j,i}\}_{k=1,\dots,d,j=1,\dots,ny,i=1,\dots,nsite}$ is an interpolation result that corresponds to the i -th site, the j -th function, the k -th derivative.

- derivatives-sites-functions

Let $\{r_{k,i,j}\}_{k=1,\dots,d,i=1,\dots,nsite,j=1,\dots,ny}$ is an interpolation result that corresponds to the i -th site, the j -th function, the k -th derivative.

The following are supported:

- functions-sites-derivatives
- functions-derivatives-sites
- sites-functions-derivatives
- sites-derivatives-functions

Syntax

```
enum class interpolate_hint {
    funcs_sites_ders,
    funcs_ders_sites,
    sites_funcs_ders,
    sites_ders_funcs
};
```

Derivatives Hints

Following hints are added to choose which derivative orders need to be computed during the `interpolate` function:

- just compute interpolation values
- compute first derivative of the spline polynomial only
- compute second derivative of the spline polynomial only
- compute third derivative of the spline polynomial only

Syntax

```
enum class derivatives {
    zero,
    first,
    second,
    third
};
```

`operator|` is overloaded to create combinations of derivative orders to be computed by `interpolate`.

Example

Assume that interpolation values, 1-st and 3-rd derivatives need to be computed. To create a bit mask that is passed to `interpolate` it needs following:

```
std::bitset<32> bit_mask = derivatives::zero | derivatives::first | derivatives::third;
```

Boundary Condition Types

Some type of splines requires boundary conditions to be set. The following types are supported:

- Free end ($f^{(2)}(x_1) = f^{(2)}(x_n) = 0$).
- Periodic.
- First derivative.
- Second Derivative.

Syntax

```
enum class bc_type {
    free_end,
    first_left_der,
    first_right_der,
    second_left_der,
    second_right_der,
    periodic
};
```

NOTE

1. First derivative and second derivative types must be set on the left and on the right borders.
 2. Free end doesn't require any values to be set.
-

Splines

- [Header File](#)
- [Namespace](#)
- [Common API for All Spline Types](#)
- [Supported Spline Types](#)

Header File

```
#include<oneapi/mkl/experimental/data_fitting.hpp>
```

Namespace

```
oneapi::mkl::experimental::data_fitting
```

Common API for All Spline Types

```
template <
    typename FpType,
    typename SplineType,
    int Dimensions = 1>
class spline {
public:
    using value_type = FpType;
    using spline_type = SplineType;

    spline(
        const sycl::queue& q,
        std::int64_t ny = 1,
        bool were_coeffs_computed = false);
```



```

spline(
    const sycl::device& dev,
    const sycl::context& ctx,
    std::int64_t ny = 1,
    bool were_coeffs_computed = false);

~spline();

spline(const spline<FpType, SplineType, Dimensions>& other) = delete;
spline(spline<FpType, SplineType, Dimensions>&& other) = delete;
spline<FpType, SplineType, Dimensions>& operator=(
    const spline<FpType, SplineType, Dimensions>& other) = delete;
spline<FpType, SplineType, Dimensions>& operator=(
    spline<FpType, SplineType, Dimensions>&& other) = delete;

spline& set_partitions(
    FpType* input_data,
    std::int64_t nx,
    partition_hint PartitionHint = partition_hint::non_uniform);

spline& set_function_values(
    FpType* input_data,
    function_hint FunctionHint = storage_hint::row_major);

spline& set_coefficients(
    FpType* data,
    coefficient_hint CoeffHint = storage_hint::row_major);

bool is_initialized() const;

std::int64_t get_required_coeffs_size() const;

sycl::event construct(const std::vector<sycl::event>& dependencies = {});
};

```

An instance of `spline<T, ST, N>` create the N-dimensional spline that operates with the `T` data type. `ST` is a type of spline. `T` can only be float or double.

Constructor	Description
<pre>spline(const sycl::queue& q, std::int64_t ny = 1, bool were_coeffs_computed = false);</pre>	Create an object with the <code>q</code> SYCL queue and <code>ny</code> number of functions. If spline coefficients were already computed, provide <code>true</code> as a 3-rd parameter. <code>were_coeffs_computed == false</code> by default. It means that it needs to call <code>construct</code> to compute spline coefficients.
<pre>spline(const sycl::device& dev, const sycl::context& ctx, std::int64_t ny = 1, bool were_coeffs_computed = false);</pre>	Create an object using the <code>dev</code> SYCL device, the <code>ctx</code> context and <code>ny</code> number of functions. If spline coefficients were already computed, provide <code>true</code> as a 3-rd parameter. <code>were_coeffs_computed == false</code> by default. It means that it needs to call <code>construct</code> to compute spline coefficients.

Member function	Description
<pre>spline& set_partitions(FpType* input_data, std::int64_t nx, partition_hint PartitionHint = partition_hint::non_uniform);</pre>	Set partition values that are specified by the <code>input_data</code> memory pointer and <code>nx</code> partition values. Users can provide <code>PartitionHint</code> to specify the layout of data. Default layout is <code>non_uniform</code>. If <code>uniform</code> is specified, <code>nx</code> must equals to 2 and <code>input_data</code> must contain only 2 values the left and the right borders of partition. Otherwise, behavior is undefined. If <code>input_data</code> layout doesn't satisfy <code>PartitionHint</code>, behavior is undefined. Returns a reference to the spline object for which partitions are set. Example, for <code>uniform</code>. Let $\{i\}_{i=1,\dots,n}$ is a partition. So, <code>input_data</code> must contain only 2 values: 1, <code>n</code>.
<pre>spline& set_function_values(FpType* input_data, function_hint FunctionHint = storage_hint::row_major);</pre>	Set function values that are specified by the <code>input_data</code> memory pointer. Number of function values must equals to <code>ny * nx</code> elements. Users can provide <code>FunctionHint</code> to specify the layout of data. Default layout is <code>row_major</code>. If <code>input_data</code> layout doesn't satisfy <code>FunctionHint</code>, behavior is undefined. Returns a reference to the spline object for which function values are set.
<pre>spline& set_coefficients(FpType* data, coefficient_hint CoeffHint = storage_hint::row_major);</pre>	Set coefficients that are specified by the <code>data</code> memory pointer. Number of coefficients in the memory must equals to the return value of <code>get_required_coeffs_size()</code>. Users can provide <code>CoeffHint</code> to specify the layout of data. Default layout is <code>row_major</code>. If <code>data</code> layout doesn't satisfy <code>CoeffHint</code>, behavior is undefined. If <code>were_coeffs_computed == false</code>, <code>data</code> will be rewritten during the <code>construct</code> call. Returns a reference to the spline object for which coefficients are set.
<pre>bool is_initialized() const;</pre>	Returns <code>true</code> if all required data are set (for example, partitions, function values, coefficients).
<pre>std::int64_t get_required_coeffs_size() const;</pre>	Returns amount of memory that is required for coefficients storage.
<pre>sycl::event construct(const std::vector<sycl::event>& dependencies = {});</pre>	Constructs the spline (calculates spline coefficients if <code>were_coeffs_computed == false</code>). The function submits a SYCL kernel and returns the SYCL event to wait on to ensure computation is complete. <code>dependencies</code> is a list of SYCL events to wait for before starting computations.

There are some splines that requires internal conditions and boundary conditions to be set. For such spline types, the following member functions must be called.

```
spline& set_internal_conditions(
    FpType* input_data);

spline& set_boundary_conditions(
    bc_type BCType = bc_type::free_end,
    FpType input_value = {});
```

Member function	Description
<code>spline& set_internal_conditions(FpType* input_data);</code>	Set internal conditions that are specified by the <code>input_data</code> memory pointer. Number of internal condition values must equals to <code>nx - 2</code> . Returns a reference to the spline object for which internal conditions are set.
<code>spline& set_boundary_conditions(bc_type BCType = bc_type::free_end, FpType input_value = {});</code>	Set the <code>input_value</code> boundary condition corresponding to <code>BCType</code> . Default value for <code>input_value</code> is empty since some boundary conditions doesn't require value to be provided. Returns a reference to the spline object for which boundary condition value is set.

NOTE copy/move constructor and copy/move assignment operators are deleted since the `spline` class is just a wrapper over memory that users provide. Memory management responsibility is on user's side.

Supported Spline Types

Currently, the DPC++ Data Fitting APIs support the following spline types (follow the corresponding links to deep-dive into the details):

[Linear Spline](#)

[Cubic Splines](#)

Linear Spline

- [Header File](#)
- [Namespace](#)
- [Syntax](#)
- [Example](#)

Linear spline is a spline whose degree is equal to 1.

It's described by the following polynomial

$$P_i(x) = c_{1,i} + c_{2,i}(x - x_i),$$

where

$$\begin{aligned}
 x &\in [x_i, x_{i+1}), \\
 c_{1,i} &= f(x_i), \\
 c_{2,i} &= [x_i, x_{i+1}] f, \\
 i &= 1, \dots, n-1.
 \end{aligned}$$

Header File

```
#include<oneapi/mkl/experimental/data_fitting.hpp>
```

Namespace

```
oneapi::mkl::experimental::data_fitting
```

Syntax

```
namespace linear_spline {
    struct default_type {};
}
```

Example

To create a linear spline object use the following:

```
spline<float, linear_spline::default_type> val(
    /*SYCL queue object*/q,
    /*number of spline functions*/ny
);
```

Follow the [Examples](#) section to see more complicated examples.

Cubic Splines

- [Header File](#)
- [Namespace](#)
- [Hermite Spline](#)
 - [Syntax](#)

- [Example](#)

Cubic splines are splines whose degree is equal to 3.

Cubic splines are described by the following polynomial

$$P_i(x) = c_{1,i} + c_{2,i}(x - x_i) + c_{3,i}(x - x_i)^2 + c_{4,i}(x - x_i)^3,$$

where

$$x \in [x_i, x_{i+1}), \\ i = 1, \dots, n - 1.$$

There are a lot of different types of cubic splines: Hermite, natural, Akima, Bessel. However, the current version of DPC++ API supports only one type: Hermite.

Header File

```
#include<oneapi/mkl/experimental/data_fitting.hpp>
```

Namespace

```
oneapi::mkl::experimental::data_fitting
```

Hermite Spline

Coefficients of Hermite spline are calculated using the following formulas:

$$c_{1,i} = f(x_i), \\ c_{2,i} = s_i, \\ c_{3,i} = ([x_i, x_{i+1}] f - s_i) / (\Delta x_i) - c_{4,i}(\Delta x_i), \\ c_{4,i} = (s_i + s_{i+1} - 2[x_i, x_{i+1}] f) / (\Delta x_i)^2,$$

$$S_i = f^{(1)}(x_i).$$

The following boundary conditions are supported for Hermite spline:

- Free end ($f^{(2)}(x_1) = f^{(2)}(x_n) = 0$).
- Periodic.
- First derivative.
- Second Derivative.

Syntax

```
namespace cubic_spline {
    struct hermite {};
}
```

Example

To create a cubic Hermite spline object use the following:

```
spline<float, cubic_spline::hermite> val(
    /*SYCL queue object*/q,
    /*number of spline functions*/ny
);
```

Follow the [Examples](#) section to see more complicated examples.

Interpolate Function

- [Header File](#)
- [Namespace](#)
- [Syntax](#)

Interpolate function performs computations of function and derivatives values at interpolation sites.

If the sites do not belong to interpolation interval $[a, b]$, the library uses:

- interpolant I_0 coefficients computed for interval $[x_0, x_1]$ for the computations at the sites to the left of a .
- interpolant I_{n-2} coefficients computed for interval $[x_{n-2}, x_{n-1}]$ for the computations at the sites to the right of b .

Interpolation algorithm depends on interpolant's type (e.g., for cubic spline interpolation evaluation of third-order polynomial is performed to obtain function values).

Header File

```
#include<oneapi/mkl/experimental/data_fitting.hpp>
```

Namespace

```
oneapi::mkl::experimental::data_fitting
```

Syntax

```
template <typename Interpolant>
sycl::event interpolate(
    Interpolant& interpolant,
    typename Interpolant::fp_type* sites,
    std::int64_t n_sites,
    typename Interpolant::fp_type* results,
    const std::vector<sycl::event>& dependencies,
    interpolate_hint ResultHint = interpolate_hint::funcs_sites_ders,
    site_hint SiteHint = site_hint::non_uniform); // (1)

template <typename Interpolant>
sycl::event interpolate(
    Interpolant& interpolant,
    typename Interpolant::fp_type* sites,
    std::int64_t n_sites,
    typename Interpolant::fp_type* results,
    std::bitset<32> der_indicator,
    const std::vector<sycl::event>& dependencies = {},
    interpolate_hint ResultHint = interpolate_hint::funcs_sites_ders,
    site_hint SiteHint = site_hint::non_uniform); // (2)

template <typename Interpolant>
sycl::event interpolate(
    sycl::queue& q,
    const Interpolant& interpolant,
    typename Interpolant::fp_type* sites,
    std::int64_t n_sites,
    typename Interpolant::fp_type* results,
    const std::vector<sycl::event>& dependencies,
    interpolate_hint ResultHint = interpolate_hint::funcs_sites_ders,
    site_hint SiteHint = site_hint::non_uniform); // (3)

template <typename Interpolant>
sycl::event interpolate(
    sycl::queue& q,
    const Interpolant& interpolant,
    typename Interpolant::fp_type* sites,
    std::int64_t n_sites,
    typename Interpolant::fp_type* results,
    std::bitset<32> der_indicator,
    const std::vector<sycl::event>& dependencies = {},
    interpolate_hint ResultHint = interpolate_hint::funcs_sites_ders,
    site_hint SiteHint = site_hint::non_uniform); // (4)
```

For all functions users can provide `SiteHint` and `ResultHint` to specify the layout of sites and results respectively. If results layout doesn't satisfy `ResultHint` and/or sites layout doesn't satisfy `SiteHint`, behavior is undefined. Returns the SYCL event of the submitted task.

1. Performs computations of function values only using the SYCL queue associated with `interpolant`.
2. Performs computations of certain derivatives (function values is considered as a zero derivative) which are indicated in `der_indicator` (each bit corresponds to certain derivative starting from lower bit) using the SYCL queue associated with `interpolant`.
3. Performs computations of function values only using `q` as an input argument that should be created from the same context and device as the SYCL queue associated with `interpolant`.

4. Performs computations of certain derivatives (function values is considered as a zero derivative) which are indicated in `der_indicator` (each bit corresponds to certain derivative starting from lower bit) using `q` as an input argument that should be created from the same context and device as the SYCL queue associated with `interpolant`.

Follow the [Examples](#) section to see examples of the interpolation function usage.

Examples

The following example demonstrates how to construct the linear spline and perform the interpolation.

```
#include <cstdint>
#include <iostream>
#include <vector>

#include <sycl/sycl.hpp>

#include <oneapi/mkl/experimental/data_fitting.hpp>

constexpr std::int64_t nx = 10'000;
constexpr std::int64_t nsites = 150'000;

int main (int argc, char ** argv) {

    sycl::queue q;
    sycl::usm_allocator<double, sycl::usm::alloc::shared> alloc(q);

    // Allocate memory for spline parameters
    std::vector<double, decltype(alloc)> partitions(nx, alloc);
    std::vector<double, decltype(alloc)> functions(nx, alloc);
    std::vector<double, decltype(alloc)> coeffs(2 * (nx - 1), alloc);
    std::vector<double, decltype(alloc)> sites(nsites, alloc);
    std::vector<double, decltype(alloc)> results(nsites, alloc);

    // Fill parameters with valid data
    for (std::int64_t i = 0; i < nx; ++i) {
        partitions[i] = 0.1 * i;
        functions[i] = i * i;
    }

    for (std::int64_t i = 0; i < nsites; ++i) {
        sites[i] = (0.1 * nx * i) / nsites;
    }

    namespace df = oneapi::mkl::experimental::data_fitting;
    // Set parameters to spline
    df::spline<double, df::linear_spline::default_type> spl(q);
    spl.set_partitions(partitions.data(), nx)
        .set_coefficients(coeffs.data())
        .set_function_values(functions.data());

    // Construct spline
    auto event = spl.construct();
    event = df::interpolate(spl, sites.data(), nsites, results.data(), { event });
    event.wait();
}
```



```
std::cout << "done" << std::endl;
return 0;
}
```

Bibliography

For more information about the VS functionality, refer to the following publications:

VS RNG

- [AVX] Intel. *Intel® Advanced Vector Extensions Programming Reference*. (<https://www.intel.com/content/www/us/en/developer/overview.html>)
- [Bratley88] Bratley P. and Fox B.L. *Implementing Sobol's Quasirandom Sequence Generator*, ACM Transactions on Mathematical Software, Vol. 14, No. 1, Pages 88-100, March 1988.
- [Bratley92] Bratley P., Fox B.L., and Niederreiter H. *Implementation and Tests of Low-Discrepancy Sequences*, ACM Transactions on Modeling and Computer Simulation, Vol. 2, No. 3, Pages 195-213, July 1992.
- [Coddington94] Coddington, P. D. *Analysis of Random Number Generators Using Monte Carlo Simulation*. Int. J. Mod. Phys. C-5, 547, 1994.
- [IntelSWMan] Intel. *Intel® 64 and IA-32 Architectures Software Developer's Manual*. 3 vols. (<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>)
- [L'Ecuyer99] L'Ecuyer, Pierre. *Tables of Linear Congruential Generators of Different Sizes and Good Lattice Structure*. Mathematics of Computation, 68, 225, 249-260, 1999.
- [L'Ecuyer99a] L'Ecuyer, Pierre. *Good Parameter Sets for Combined Multiple Recursive Random Number Generators*. Operations Research, 47, 1, 159-164, 1999.
- [L'Ecuyer02] Pierre L'Ecuyer, Richard Simard, E. Jack Chen, W. David Kelton, (2002) An Object-Oriented Random-Number Package with Many Long Streams and Substreams. Operations Research 50(6):1073-1075, <https://doi.org/10.1287/opre.50.6.1073.358>
- [Kirkpatrick81] Kirkpatrick, S., and Stoll, E. *A Very Fast Shift-Register Sequence Random Number Generator*. Journal of Computational Physics, V. 40. 517-526, 1981.
- [Matsumoto98] Matsumoto, M., and Nishimura, T. *Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudo-Random Number Generator*, ACM Transactions on Modeling and Computer Simulation, Vol. 8, No. 1, Pages 3-30, January 1998.
- [Matsumoto00] Matsumoto, M., and Nishimura, T. *Dynamic Creation of Pseudorandom Number Generators*, 56-69, in: Monte Carlo and Quasi-Monte Carlo Methods 1998, Ed. Niederreiter, H. and Spanier, J., Springer 2000, <http://www.math.sci.hiroshima-u.ac.jp/m-mat/MT/DC/dc.html>.
- [NAG] NAG Numerical Libraries. <https://www.nag.com/content/nag-library>
- [Saito08] Saito, M., and Matsumoto, M. *SIMD-oriented Fast Mersenne Twister: a 128-bit Pseudorandom Number Generator*. Monte Carlo and Quasi-Monte Carlo Methods 2006, Springer, Pages 607 – 622, 2008.

<http://www.math.sci.hiroshima-u.ac.jp/m-mat/MT/ARTICLES/earticles.html>

- [Salmon11] Salmon, John K., Morales, Mark A., Dror, Ron O., and Shaw, David E., *Parallel Random Numbers: As Easy as 1, 2, 3*. SC '11 Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis, 2011.
- [Sobol76] Sobol, I.M., and Levitan, Yu.L. *The production of points uniformly distributed in a multidimensional cube*. Preprint 40, Institute of Applied Mathematics, USSR Academy of Sciences, 1976 (In Russian).
- [pcg2014] Melissa E. O'Neill. PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation.
<https://www.cs.hmc.edu/tr/hmc-cs-2014-0905.pdf>
- [VS Notes] |O-MKL| *Vector Statistics Notes*, a document presentation on the oneMKL product at <https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl-documentation.html>

VM

- [IEEE754] IEEE Standard for Binary Floating-Point Arithmetic. ANSI/IEEE Std 754-2008.

Appendix A: oneMKL Functionality

This appendix provides an overview of the Intel® oneAPI Math Kernel Library (oneMKL) functionality on the different devices.

BLAS Functionality

Functionality	CPU	Intel GPU
BLAS Level 1	All	All
BLAS Level 2	All	All
BLAS Level 3	All	All
BLAS-like Extensions	All	All

LAPACK Functionality

NOTE All of the DPC++ LAPACK computational routines have a corresponding *_scratchpad_size function for calculating the required amount of scratchpad space.

Functionality	CPU	Intel GPU
getrf	Yes	Yes

Functionality	CPU	Intel GPU
getrs	Yes	Yes
getri	Yes	Yes

Functionality	CPU	Intel GPU
potrf	Yes	Yes
potrs	Yes	Yes
potri	Yes	Yes

Functionality	CPU	Intel GPU
geqrf	Yes	Yes
{or,un}gqr	Yes	Yes
{or,un}mqr	Yes	Yes
gerqf	Yes	No
{or,un}mrq	Yes	No

Functionality	CPU	Intel GPU
trtrs	Yes	Yes
{sy,he}trf	Yes	No

Functionality	CPU	Intel GPU
{sy,he}ev	Yes	Yes
{sy,he}evd	Yes	Yes
{sy,he}evx	Yes	Yes
{sy,he}trd	Yes	Yes
{or,un}gtr	Yes	No
{or,un}mtr	Yes	No
steqr	Yes	Yes

Functionality	CPU	Intel GPU
{sy,he}gvd	Yes	Yes
{sy,he}gvx	Yes	Yes

Functionality	CPU	Intel GPU
gesvd	Yes	Yes
gebrd	Yes	Yes
{or,un}gbr	Yes	No

Functionality	CPU	Intel GPU
getrf_batch	Yes	Yes
getrs_batch	Yes	Yes
getri_batch	Yes	Yes
potrf_batch	Yes	Yes
potrs_batch	Yes	Yes
geqrf_batch	Yes	Yes
{or,un}gqr_batch	Yes	Yes

Other LAPACK Routines

CPU	Intel GPU
No	No

DFT Functionality

Functionality	CPU	Intel GPU
1D Complex-to-Complex FFT transformations	Yes	Yes
2D Complex-to-Complex FFT transformations	Yes	Yes
3D Complex-to-Complex FFT transformations	Yes	Yes
4D Complex-to-Complex FFT transformations	No	No
5D Complex-to-Complex FFT transformations	No	No
6D Complex-to-Complex FFT transformations	No	No
7D Complex-to-Complex FFT transformations	No	No
1D Real-to-Complex FFT transformations	Yes	Yes
2D Real-to-Complex FFT transformations	Yes	Yes
3D Real-to-Complex FFT transformations	Yes	Yes
4D Real-to-Complex FFT transformations	No	No

Functionality	CPU	Intel GPU
5D Real-to-Complex FFT transformations	No	No
6D Real-to-Complex FFT transformations	No	No
7D Real-to-Complex FFT transformations	No	No

Sparse BLAS SYCL Functionality

The following tables summarize the current support of SYCL Sparse BLAS APIs in Intel oneAPI Math Kernel Library (oneMKL).

In the tables, we use these symbols for the sparse and dense objects in the operation descriptions:

- scalar values = α, β, d ,
- dense vectors = x, y ,
- sparse vectors = xs, ys ,
- dense matrices = X, Y and
- sparse matrices = A, B, C ,
- identity matrix = I .

Some operations involve the upper/lower decomposition of a sparse matrix into $A = L + D + U$ with strictly lower (L), diagonal (D), and strictly upper (U) parts of the sparse matrix, A . The choice of which part of the data to use in the operation is controlled by the input parameters:

- `uplo::lower` for selecting $(L + D)$ or $(L + I)$ for the operation
- `uplo::upper` for selecting $(D + U)$ or $(I + U)$ for the operation

Some of the triangular operations permit you to ignore the main diagonal data in the sparse matrix and replace it, in the operation, with ones on the diagonal:

- `diag::nonunit` uses D as provided in the data and
- `diag::unit` ignores D and instead uses the identity, I .

Finally, the `op()` operation represents one of three “transpose” operations:

- $\text{op}(A) = A$ when using `transpose::nontrans`
- $\text{op}(A) = A^T$ when using `transpose::trans`
- $\text{op}(A) = A^H$ when using `transpose::conjtrans`.

Typically all are intended to be supported, but the actual support list and limitations are provided in each API’s individual description page.

Level 1

Sparse BLAS operations involving a sparse vector and other vectors.

There are currently no SYCL Sparse BLAS APIs in oneMKL supporting sparse vectors or Sparse Level 1 operations.

Level 2

Sparse BLAS operations involving a sparse matrix and vectors.

Common Name	SYCL API name with oneapi::mkl:: namespace	Operation	CSR support	COO support
SpMV	sparse::gemv()	$y = \alpha \cdot \text{op}(A) \cdot x + \beta \cdot y$	CPU/GPU	CPU/GPU
	sparse::symv() + uplo::lower	$y = \alpha \cdot (L + D + L^T) \cdot x + \beta \cdot y$	CPU/GPU	None
	sparse::symv() + uplo::upper	$y = \alpha \cdot (U^T + D + U) \cdot x + \beta \cdot y$	CPU/GPU	None
	sparse::trmv() + uplo::lower + diag::nonunit	$y = \alpha \cdot \text{op}(L + D) \cdot x + \beta \cdot y$	CPU/GPU	CPU
	sparse::trmv() + uplo::lower + diag::unit	$y = \alpha \cdot \text{op}(L + I) \cdot x + \beta \cdot y$	CPU/GPU	CPU
	sparse::trmv() + uplo::upper + diag::nonunit	$y = \alpha \cdot \text{op}(D + U) \cdot x + \beta \cdot y$	CPU/GPU	CPU
	sparse::trmv() + uplo::upper + diag::unit	$y = \alpha \cdot \text{op}(I + U) \cdot x + \beta \cdot y$	CPU/GPU	CPU
SpMV + dot fusion	sparse::gemvdot()	$y = \alpha \cdot \text{op}(A) \cdot x + \beta \cdot y$ and $d = \text{dot}(y, x)$	CPU/GPU	CPU
SpSV or SpTRSV	sparse::trsv() + uplo::lower + diag::nonunit	Solve for y : $\text{op}(L + D) \cdot y = \alpha \cdot x$	CPU/GPU	CPU
	sparse::trsv() + uplo::lower + diag::unit	Solve for y : $\text{op}(L + I) \cdot y = \alpha \cdot x$	CPU/GPU	CPU
	sparse::trsv() + uplo::upper + diag::nonunit	Solve for y : $\text{op}(D + U) \cdot y = \alpha \cdot x$	CPU/GPU	CPU
	sparse::trsv() + uplo::upper + diag::unit	Solve for y : $\text{op}(I + U) \cdot y = \alpha \cdot x$	CPU/GPU	CPU

Level 3

Sparse BLAS operations involving one or more sparse/dense matrices.

Common Name	SYCL API name with oneapi::mkl::namespace	Operation	CSR support	COO support
SpMM (Dn <- Sp * Dn)	sparse::gemm()	$Y = \alpha \cdot \text{op}(A) \cdot \text{op}(X) + \beta \cdot Y$	CPU/ GPU	CPU
SpSM or SpTRSM	sparse::trsm() + uplo::lower + diag::nonunit	Solve for Y : $\text{op}(L + D) \cdot Y = \alpha \cdot \text{op}(X)$	CPU/ GPU	CPU
	sparse::trsm() + uplo::lower + diag::unit	Solve for Y : $\text{op}(L + I) \cdot Y = \alpha \cdot \text{op}(X)$	CPU/ GPU	CPU
	sparse::trsm() + uplo::upper + diag::nonunit	Solve for Y : $\text{op}(D + U) \cdot Y = \alpha \cdot \text{op}(X)$	CPU/ GPU	CPU
	sparse::trsm() + uplo::upper + diag::unit	Solve for Y : $\text{op}(I + U) \cdot Y = \alpha \cdot \text{op}(X)$	CPU/ GPU	CPU
SpGEAM (Sp <- Sp + Sp)	sparse::omatadd()	$C = \alpha \cdot \text{op}(A) + \beta \cdot \text{op}(B)$	CPU/ GPU	None
SpGEMM (Sp <- Sp * Sp)	sparse::matmat()	$C = \text{op}(A) \cdot \text{op}(B)$	CPU/ GPU	None
SpGEMMD (Dn <- Sp * Sp)	sparse::matmatd()	$C = \alpha \cdot \text{op}(A) \cdot \text{op}(B) + \beta \cdot C$	GPU	None

Auxiliary Functions

Operations that can manipulate the sparse matrix handle.

Common Name	SYCL API name with oneapi::mkl::namespace	Operation	CSR support	COO support
Sparse Copy or Sparse	sparse::omatcopy()	$B = \text{op}(A)$	CPU/ GPU	CPU/G PU

Common Name	SYCL API name with oneapi::mkl:: namespace	Operation	CSR support	COO support
Matrix Transpose				
Sparse Matrix Conversion	sparse::omatconvert()	$B = \text{convert}(A)$, where B and A are represented using different sparse matrix formats	CPU/GPU	CPU/GPU
Sparse Matrix Sort	sparse::sort_matrix()	$A = \text{sort}(A)$ by natural ordering for the sparse matrix format	CPU/GPU	None (see note below)
Update Matrix Main Diagonal	sparse::update_diagonal_data()	update A in-place with new provided diagonal values, D	GPU	None

Sparse Solvers Functionality

Functionality	CPU	Intel GPU
Sparse Cholesky Factorization	No	No
Sparse LU Factorization	No	No
Sparse QR factorization	No	No
Hermitian/Symmetric Eigensolver on intervals for Sparse Matrices	No	No
Extremal Eigensolvers for Sparse Matrix	No	No
Poisson Solver	No	No
Trust Region Solver	No	No

Random Number Generators Functionality

Engines

Functionality	CPU	Intel GPU
MRG32K3A	Yes	Yes

Functionality	CPU	Intel GPU
MT2203	Yes	Yes
MT19937	Yes	Yes
PHILOX4X32X10	Yes	Yes
SOBOL	Yes	Yes
ARS5	Yes	No
MCG59	Yes	Yes
NIEDERR	Yes	No
MCG31	Yes	Yes
WH	Yes	No
SFMT19937	Yes	No
R250	Yes	No
NONDETERM	Yes	No
DABSTRACT	No	No
SABSTRACT	No	No
SABSTRACT	No	No

Distributions

Functionality	CPU	Intel GPU
Uniform (single/double/integer)	Yes	Yes
UniformBits32	Yes	Yes
UniformBits64		
Lognormal (single/double)	Yes	Yes
Gaussian (single/double)	Yes	Yes
Poisson	Yes	Yes
UniformBits	Yes	Yes
Bernoulli	Yes	Yes
Beta (single/double)	Yes	No
Binomial	Yes	No
ChiSquare (single/double)	Yes	No
Exponential (single/double)	Yes	Yes
Gamma (single/double)	Yes	No

Functionality	CPU	Intel GPU
Geometric	Yes	Yes
Gumbel (single/double)	Yes	Yes
Hyper Geometric	Yes	No
Laplace (single/double)	Yes	Yes
Multinomial	Yes	No
Negative Binomial	Yes	No
PoissonV	Yes	No
Rayleigh (single/double)	Yes	Yes
Weibull (single/double)	Yes	Yes
Cauchy (single/double)	Yes	Yes
GaussianMV (single/double)	Yes	No

Vector Math Functionality

Functionality	CPU	Intel GPU
Vector Math Functions, Single Precision	Yes	Yes
Vector Math Functions, Double Precision	Yes	Yes
Vector Math Functions, Single Precision Complex	Yes	No
Vector Math Functions, Double Precision Complex	Yes	No

OpenMP* offload to the GPU is implemented in the Linux* OS, but not in the Windows* OS. The Windows OS implementation will be available in a future release.

Data Fitting Functionality

Splines, Spline Type

Functionality	CPU	Intel GPU
default linear	Yes (since oneMKL 2022.1)	Yes (since oneMKL 2022.1)
default quadratic	No	No
default cubic	No	No
subbotin	No	No

Functionality	CPU	Intel GPU
natural	No	No
hermite	Yes (since oneMKL 2022.1)	Yes (since oneMKL 2022.1)
akima	No	No
bessel	No	No
hyman	No	No
lookup interpolant	No	No
cr stepwise const interpolant	No	No
cl stepwise const interpolant	No	No

Computation Routines

Functionality	CPU	Intel GPU
Construct1D	Yes (since oneMKL 2022.1)	Yes (since oneMKL 2022.1)
Interpolate1D	Yes (since oneMKL 2022.1)	Yes (since oneMKL 2022.1)
Interpolate1DEx	No	No
Integrate1D	No	No
Integrate1DEx	No	No
SearchCells1D	No	No
SearchCells1DEx	No	No
InterpolationCallBack	No	No
IntegrateCallBack	No	No
SearchCellsCallBack	No	No

Summary Statistics Functionality

Functionality	CPU	Intel GPU
min	Yes	No
max	Yes	No
raw sum	Yes	No
2nd order raw sum		
3rd order raw sum		
4th order raw sum		
2nd order central sum	Yes	No
3rd order central sum		

Functionality	CPU	Intel GPU
4th order central sum		
mean	Yes	No
2nd order raw moment		
3rd order raw moment		
4th order raw moment		
2nd order central moment	Yes	No
3rd order central moment		
4th order central moment		
kurtosis	Yes	No
skewness	Yes	No
variation coefficient	Yes	No
covariance matrix	No	No
correlation matrix	No	No
cross-product matrix	No	No
pooled covariance matrix	No	No
pooled mean	No	No
group covariance matrix	No	No
group mean	No	No
quantiles	No	No
order statistics	No	No
robust covariance matrix	No	No
outliers detection	No	No
partial covariance matrix	No	No
partial covariance matrix	No	No
missing values	No	No
parameterized correlation matrix	No	No
stream quantiles	No	No
mean absolute deviation	No	No
median absolute deviation	No	No
sorted observations	No	No

Notices and Disclaimers

Intel technologies may require enabled hardware, software or service activation.

No product or component can be absolutely secure.

Your costs and results may vary.

© Intel Corporation. Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

Software and workloads used in performance tests may have been optimized for performance only on Intel microprocessors. Performance tests, such as SYSmark and MobileMark, are measured using specific computer systems, components, software, operations and functions. Any change to any of those factors may cause the results to vary. You should consult other information and performance tests to assist you in fully evaluating your contemplated purchases, including the performance of that product when combined with other products. For more complete information visit the following page:

<https://edc.intel.com/content/www/us/en/products/performance/benchmarks/overview/>

INFORMATION IN THIS DOCUMENT IS PROVIDED "AS IS". NO LICENSE, EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, TO ANY INTELLECTUAL PROPERTY RIGHTS IS GRANTED BY THIS DOCUMENT. INTEL ASSUMES NO LIABILITY WHATSOEVER AND INTEL DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY, RELATING TO THIS INFORMATION INCLUDING LIABILITY OR WARRANTIES RELATING TO FITNESS FOR A PARTICULAR PURPOSE, MERCHANTABILITY, OR INFRINGEMENT OF ANY PATENT, COPYRIGHT OR OTHER INTELLECTUAL PROPERTY RIGHT.

OpenCL and the OpenCL logo are trademarks of Apple Inc. used by permission by Khronos.

Microsoft, Windows, and the Windows logo are trademarks, or registered trademarks of Microsoft Corporation in the United States and/or other countries.

Java is a registered trademark of Oracle and/or its affiliates.

Third Party Content

Intel® oneAPI Math Kernel Library includes content from several 3rd party sources that was originally governed by the licenses referenced below:

- Portions® Copyright 2001 Hewlett-Packard Development Company, L.P.
- Sections on the Linear Algebra PACKage (LAPACK) routines include derivative work portions that have been copyrighted:
 - © 1991, 1992, and 1998 by The Numerical Algorithms Group, Ltd.
- Intel® oneAPI Math Kernel Library supports LAPACK 3.5 set of computational, driver, auxiliary and utility routines under the following license:

Copyright © 1992-2011 The University of Tennessee and The University of Tennessee Research Foundation. All rights reserved.

Copyright © 2000-2011 The University of California Berkeley. All rights reserved.

Copyright © 2006-2012 The University of Colorado Denver. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer listed in this license in the documentation and/or other materials provided with the distribution.
- Neither the name of the copyright holders nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

The copyright holders provide no reassurances that the source code provided does not infringe any patent, copyright, or any other intellectual property rights of third parties. The copyright holders disclaim any liability to any recipient for claims brought against recipient by any third party for infringement of that parties intellectual property rights.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

The original versions of LAPACK from which that part of Intel® oneAPI Math Kernel Library was derived can be obtained from <http://www.netlib.org/lapack/index.html>. The authors of LAPACK are E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen.

- The original versions of the Basic Linear Algebra Subprograms (BLAS) from which the respective part of Intel® oneAPI Math Kernel Library was derived can be obtained from <http://www.netlib.org/blas/index.html>.
- XBLAS is distributed under the following copyright:

Copyright © 2008-2009 The University of California Berkeley.

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer listed in this license in the documentation and/or other materials provided with the distribution.
- Neither the name of the copyright holders nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

- The original versions of the Basic Linear Algebra Communication Subprograms (BLACS) from which the respective part of Intel® oneAPI Math Kernel Library was derived can be obtained from <http://www.netlib.org/blacs/index.html>. The authors of BLACS are Jack Dongarra and R. Clint Whaley.

- The original versions of Scalable LAPACK (ScaLAPACK) from which the respective part of Intel® oneAPI Math Kernel Library was derived can be obtained from <http://www.netlib.org/scalapack/index.html>. The authors of ScaLAPACK are L. S. Blackford, J. Choi, A. Cleary, E. D'Azevedo, J. Demmel, I. Dhillon, J. Dongarra, S. Hammarling, G. Henry, A. Petitet, K. Stanley, D. Walker, and R. C. Whaley.
- The original versions of the Parallel Basic Linear Algebra Subprograms (PBLAS) routines from which the respective part of Intel® oneAPI Math Kernel Library was derived can be obtained from http://www.netlib.org/scalapack/html/pblas_gref.html.
- PARDISO (PARallel DIrect SOLver)* in Intel® oneAPI Math Kernel Library was originally developed by the Department of Computer Science at the University of Basel (<http://www.unibas.ch>). It can be obtained at <http://www.pardiso-project.org>.
- The Extended Eigensolver functionality is based on the Feast solver package and is distributed under the following license:

Copyright © 2009, The Regents of the University of Massachusetts, Amherst.

Developed by E. Polizzi

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. Neither the name of the University nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR 'AS IS' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

- Some Fast Fourier Transform (FFT) functions in this release of Intel® oneAPI Math Kernel Library have been generated by the SPIRAL software generation system (<http://www.spiral.net/>) under license from Carnegie Mellon University. The authors of SPIRAL are Markus Puschel, Jose Moura, Jeremy Johnson, David Padua, Manuela Veloso, Bryan Singer, Jianxin Xiong, Franz Franchetti, Aca Gacic, Yevgen Voronenko, Kang Chen, Robert W. Johnson, and Nick Rizzolo.
- Open MPI is distributed under the New BSD license, listed below.

Most files in this release are marked with the copyrights of the organizations who have edited them. The copyrights below are in no particular order and generally reflect members of the Open MPI core team who have contributed code to this release. The copyrights for code used under license from other parties are included in the corresponding files.

Copyright © 2004-2010 The Trustees of Indiana University and Indiana University Research and Technology Corporation. All rights reserved.

Copyright © 2004-2010 The University of Tennessee and The University of Tennessee Research Foundation. All rights reserved.

Copyright © 2004-2010 High Performance Computing Center Stuttgart, University of Stuttgart. All rights reserved.

Copyright © 2004-2008 The Regents of the University of California. All rights reserved.

Copyright © 2006-2010 Los Alamos National Security, LLC. All rights reserved.

Copyright © 2006-2010 Cisco Systems, Inc. All rights reserved.

Copyright © 2006-2010 Voltaire, Inc. All rights reserved.

Copyright © 2006-2011 Sandia National Laboratories. All rights reserved.

Copyright © 2006-2010 Sun Microsystems, Inc. All rights reserved. Use is subject to license terms.

Copyright © 2006-2010 The University of Houston. All rights reserved.

Copyright © 2006-2009 Myricom, Inc. All rights reserved.

Copyright © 2007-2008 UT-Battelle, LLC. All rights reserved.

Copyright © 2007-2010 IBM Corporation. All rights reserved.

Copyright © 1998-2005 Forschungszentrum Juelich, Juelich Supercomputing Centre, Federal Republic of Germany

Copyright © 2005-2008 ZIH, TU Dresden, Federal Republic of Germany

Copyright © 2007 Evergrid, Inc. All rights reserved.

Copyright © 2008 Chelsio, Inc. All rights reserved.

Copyright © 2008-2009 Institut National de Recherche en Informatique. All rights reserved.

Copyright © 2007 Lawrence Livermore National Security, LLC. All rights reserved.

Copyright © 2007-2009 Mellanox Technologies. All rights reserved.

Copyright © 2006-2010 QLogic Corporation. All rights reserved.

Copyright © 2008-2010 Oak Ridge National Labs. All rights reserved.

Copyright © 2006-2010 Oracle and/or its affiliates. All rights reserved.

Copyright © 2009 Bull SAS. All rights reserved.

Copyright © 2010 ARM Ltd. All rights reserved.

Copyright © 2010-2011 Alex Brick . All rights reserved.

Copyright © 2012 The University of Wisconsin-La Crosse. All rights reserved.

Copyright © 2013-2014 Intel, Inc. All rights reserved.

Copyright © 2011-2014 NVIDIA Corporation. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer listed in this license in the documentation and/or other materials provided with the distribution.
- Neither the name of the copyright holders nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

The copyright holders provide no reassurances that the source code provided does not infringe any patent, copyright, or any other intellectual property rights of third parties. The copyright holders disclaim any liability to any recipient for claims brought against recipient by any third party for infringement of that parties intellectual property rights.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

- The Safe C Library is distributed under the following copyright:

Copyright (c)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

- HPL Copyright Notice and Licensing Terms

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions, and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. All advertising materials mentioning features or use of this software must display the following acknowledgement: This product includes software developed at the University of Tennessee, Knoxville, Innovative Computing Laboratories.
4. The name of the University, the name of the Laboratory, or the names of its contributors may not be used to endorse or promote products derived from this software without specific written permission.